

azbot/azbot/__init__.py

```
"""AZBot: skill for Aziel Eliab public engines. Not a model."""
from pathlib import Path

__version__ = "0.2.0"
__author__ = "Aziel Eliab"
LIMITATION = (
    "AZBot is a skill, not a foundation model, not a kernel, not a VPN, "
    "and not a paid-key proxy. Jeeves is not sovereign. Call aziel-runtime."
)
CATALOG = "https://aziel-runtime.vibelock.workers.dev"
OPENAPI = CATALOG + "/openapi.json"
MCP = CATALOG + "/mcp"

def skill_text() -> str:
    here = Path(__file__).resolve().parent
    for candidate in (here / "SKILL.md", here.parent / "SKILL.md"):
        if candidate.exists():
            return candidate.read_text(encoding="utf-8")
    raise FileNotFoundError("AZBot SKILL.md missing")
```

azbot/azbot/__main__.py

```
from azbot.cli import main

if __name__ == "__main__":
    raise SystemExit(main())
```

azbot/azbot/cli.py

```

from __future__ import annotations

import argparse
import json
import urllib.request
from pathlib import Path

from azbot import CATALOG, LIMITATION, MCP, OPENAPI, __version__, skill_text
from azbot.skills_loader import (
    discover_skills,
    export_skill,
    get_skill,
    import_skill,
    skills_as_dicts,
)
from azbot.ui import serve

def _get(url: str) -> tuple[int, str]:
    req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0 AZBot-doctor"})
    try:
        with urllib.request.urlopen(req, timeout=20) as res:
            return res.status, res.read().decode("utf-8", "replace")
    except Exception as exc: # noqa: BLE001
        return 0, str(exc)

def cmd_skill(args: argparse.Namespace) -> int:
    action = getattr(args, "skill_cmd", None)
    if action is None:
        print(skill_text())
        return 0
    if action == "run":
        skill = get_skill(args.name)
        print(skill.body)
        return 0
    if action == "import":
        imported = import_skill(Path(args.path))
        print(json.dumps({
            "ok": True,
            "name": imported.name,
            "slug": imported.slug,
            "path": str(imported.path),
            "description": imported.description,
        }, indent=2))
        return 0
    if action == "export":
        dest = Path(args.out) if getattr(args, "out", None) else None
        result = export_skill(args.name, dest=dest)
        if dest is None:
            print(result)
        else:
            print(result)
        return 0
    print("unknown skill action", file=__import__("sys").stderr)
    return 2

def cmd_skills(_: argparse.Namespace) -> int:
    rows = skills_as_dicts()
    if not rows:
        print("No SKILL.md files found.")
        return 0
    width = max(len(r["name"]) for r in rows)
    for row in rows:
        print(f"{'name':<{width}} {row['description']}")
    return 0

def cmd_doctor(_: argparse.Namespace) -> int:
    code, body = _get(CATALOG + "/v1/health")
    ok = code == 200 and "ok" in body.lower()
    print(json.dumps({
        "ok": ok,
        "catalog_health_status": code,
        "openapi": OPENAPI,
        "mcp": MCP,
        "limitation": LIMITATION,
        "version": __version__,
        "skills": len(discover_skills()),
    }, indent=2))
    return 0 if ok else 1

def cmd_ui(_: argparse.Namespace) -> int:
    serve()
    return 0

def main(argv: list[str] | None = None) -> int:
    p = argparse.ArgumentParser(
        prog="azbot",
    )

```

```
        description=(
            "AZBot skill runner. Not a model and not Grok weights. "
            "Prints skills, checks the catalog, or serves a local UI."
        ),
    )
p.add_argument("--version", action="version", version=__version__)
sub = p.add_subparsers(dest="cmd", required=True)

skill_p = sub.add_parser("skill", help="Print catalog SKILL.md, or run/import/export a skill")
skill_p.set_defaults(fn=cmd_skill)
skill_sub = skill_p.add_subparsers(dest="skill_cmd")

run_p = skill_sub.add_parser("run", help="Print a discovered skill body (local operator; not Grok weights)")
run_p.add_argument("name")

imp_p = skill_sub.add_parser("import", help="Copy a SKILL.md into skills/<slug>/")
imp_p.add_argument("path")

exp_p = skill_sub.add_parser("export", help="Write a skill as markdown")
exp_p.add_argument("name")
exp_p.add_argument("--out", help="Destination file (default: stdout)")

sub.add_parser("skills", help="List discovered skills (name + description)").set_defaults(fn=cmd_skills)
sub.add_parser("doctor", help="Ping aziel-runtime /v1/health").set_defaults(fn=cmd_doctor)
sub.add_parser("ui", help="Loopback UI at 127.0.0.1:8870").set_defaults(fn=cmd_ui)
args = p.parse_args(argv)
return int(args.fn(args))

if __name__ == "__main__":
    raise SystemExit(main())
```

azbot/azbot/skills_loader.py

```

"""Discover and manage Cursor-style SKILL.md files plus custom AZbot skills.

AZBot prints and stores skill markdown. It does not ship Grok model weights.
Execution is local operator instructions.
"""
from __future__ import annotations

import os
import re
import shutil
from dataclasses import dataclass
from pathlib import Path

FRONTMATTER_RE = re.compile(r"\A---\s*\n(?:.*?)\n---\s*", re.DOTALL)
SLUG_RE = re.compile(r"^[a-z0-9]+")

@dataclass(frozen=True)
class Skill:
    name: str
    description: str
    path: Path
    body: str
    source: str

    @property
    def slug(self) -> str:
        return slugify(self.name)

def slugify(name: str) -> str:
    slug = SLUG_RE.sub("-", name.strip().lower()).strip("-")
    return slug or "skill"

def find_root(start: Path | None = None) -> Path:
    candidates: list[Path] = []
    if start is not None:
        candidates.append(Path(start).resolve())
    env_root = os.environ.get("AZBOT_ROOT")
    if env_root:
        candidates.append(Path(env_root).expanduser().resolve())
    candidates.append(Path.cwd().resolve())
    candidates.append(Path(__file__).resolve().parent.parent)
    seen: set[Path] = set()
    for base in candidates:
        for p in [base, *base.parents]:
            if p in seen:
                continue
            seen.add(p)
            if (p / "pyproject.toml").is_file() and (p / "azbot").is_dir():
                return p
            if (p / "standalone" / "skill" / "SKILL.md").is_file():
                return p
            if (p / "skills").is_dir() and (p / "azbot").is_dir():
                return p
    return Path.cwd().resolve()

def parse_skill_md(path: Path, source: str = "file") -> Skill | None:
    path = Path(path)
    if not path.is_file():
        return None
    try:
        text = path.read_text(encoding="utf-8")
    except OSError:
        return None
    match = FRONTMATTER_RE.match(text)
    if not match:
        return None
    meta = _parse_simple_yaml(match.group(1))
    name = str(meta.get("name") or "").strip()
    description = str(meta.get("description") or "").strip()
    if not name or not description:
        return None
    return Skill(name=name, description=description, path=path.resolve(), body=text, source=source)

def _parse_simple_yaml(block: str) -> dict[str, str]:
    data: dict[str, str] = {}
    current: str | None = None
    for raw in block.splitlines():
        line = raw.rstrip()
        if not line.strip() or line.lstrip().startswith("#"):
            continue
        if current and (raw.startswith(" ") or raw.startswith("\t")):
            extra = line.strip().strip('"').strip("'")
            if extra:
                data[current] = (data.get(current, "") + " " + extra).strip()
            continue
        if ":" not in line:
            current = None

```

```

        continue
    key, value = line.split(":", 1)
    key = key.strip()
    if not key or key.startswith("-"):
        current = None
        continue
    value = value.strip()
    if value in {">", "|", ">-", "|-"}:
        data[key] = ""
        current = key
        continue
    if (value.startswith('"') and value.endswith('"')) or (value.startswith("'") and value.endswith("'")):
        value = value[1:-1]
    data[key] = value
    current = key
return data

def _skill_files(root: Path) -> list[tuple[str, Path]]:
    found: list[tuple[str, Path]] = []
    skills_dir = root / "skills"
    if skills_dir.is_dir():
        direct = skills_dir / "SKILL.md"
        if direct.is_file():
            found.append(("skills", direct))
        for child in sorted(skills_dir.iterdir()):
            if not child.is_dir() or child.name == "cursor":
                continue
            md = child / "SKILL.md"
            if md.is_file():
                found.append(("skills", md))
    bundled = root / "standalone" / "skill" / "SKILL.md"
    if bundled.is_file():
        found.append(("bundled", bundled))
    extra = os.environ.get("AZBOT_SKILLS_DIR")
    if extra:
        extra_path = Path(extra).expanduser()
        if extra_path.is_file() and extra_path.name == "SKILL.md":
            found.append(("env", extra_path))
        elif extra_path.is_dir():
            for md in sorted(extra_path.rglob("SKILL.md")):
                found.append(("env", md))
    cursor_skills = root / ".cursor" / "skills"
    if cursor_skills.is_dir():
        for md in sorted(cursor_skills.rglob("SKILL.md")):
            found.append(("cursor", md))
    vendored = root / "skills" / "cursor"
    if vendored.is_dir():
        for md in sorted(vendored.rglob("SKILL.md")):
            found.append(("vendored-cursor", md))
    return found

def discover_skills(root: Path | None = None) -> list[Skill]:
    root = Path(root).resolve() if root is not None else find_root()
    out: list[Skill] = []
    seen_paths: set[Path] = set()
    seen_names: set[str] = set()
    for source, path in _skill_files(root):
        resolved = path.resolve()
        if resolved in seen_paths:
            continue
        skill = parse_skill_md(path, source=source)
        if skill is None:
            continue
        key = skill.name.strip().lower()
        if key in seen_names:
            continue
        seen_paths.add(resolved)
        seen_names.add(key)
        out.append(skill)
    return out

def get_skill(name: str, root: Path | None = None) -> Skill:
    needle = name.strip().lower()
    slug = slugify(name)
    for skill in discover_skills(root):
        if skill.name.strip().lower() == needle or skill.slug == slug:
            return skill
    raise KeyError(f"skill not found: {name}")

def import_skill(src: Path, root: Path | None = None) -> Skill:
    src = Path(src).expanduser().resolve()
    if src.name != "SKILL.md" and src.suffix.lower() not in {".md", ".markdown"}:
        # still allow any markdown file that has frontmatter
        pass
    skill = parse_skill_md(src, source="import")
    if skill is None:
        raise ValueError("SKILL.md needs YAML frontmatter with name and description")
    dest_root = Path(root).resolve() if root is not None else find_root()
    dest_dir = dest_root / "skills" / skill.slug
    dest_dir.mkdir(parents=True, exist_ok=True)
    dest = dest_dir / "SKILL.md"

```

```
if src.resolve() != dest.resolve():
    shutil.copy2(src, dest)
imported = parse_skill_md(dest, source="skills")
if imported is None:
    raise ValueError("imported SKILL.md is missing name or description")
return imported

def export_skill(name: str, dest: Path | None = None, root: Path | None = None) -> Path | str:
    skill = get_skill(name, root=root)
    if dest is None:
        return skill.body
    dest = Path(dest)
    if dest.suffix.lower() not in {".md", ".markdown"} and dest.is_dir():
        dest = dest / f"{skill.slug}.md"
    dest.parent.mkdir(parents=True, exist_ok=True)
    dest.write_text(skill.body, encoding="utf-8")
    return dest

def skills_as_dicts(root: Path | None = None) -> list[dict[str, str]]:
    rows = []
    base = Path(root).resolve() if root is not None else find_root()
    for skill in discover_skills(root):
        try:
            rel = str(skill.path.relative_to(base))
        except ValueError:
            rel = str(skill.path)
        rows.append({
            "name": skill.name,
            "description": skill.description,
            "source": skill.source,
            "path": rel,
            "slug": skill.slug,
        })
    return rows
```

azbot/azbot/ui.py

```

"""Local AZBot UI. Bind 127.0.0.1 only."""
from __future__ import annotations

import json
import posixpath
import tempfile
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from importlib.resources import files
from pathlib import Path
from urllib.parse import unquote, urlparse

from azbot import LIMITATION, __version__, skill_text
from azbot.skills_loader import get_skill, import_skill, skills_as_dicts

LOOPBACK = frozenset({"127.0.0.1", "localhost", "::1"})
WEB = files("azbot") / "web"

class Handler(BaseHTTPRequestHandler):
    server_version = f"AZBot/{__version__}"

    def log_message(self, fmt: str, *args: object) -> None:
        return

    def _send(self, status: int, body: bytes, content_type: str, extra: dict[str, str] | None = None) -> None:
        self.send_response(status)
        self.send_header("Content-Type", content_type)
        self.send_header("Content-Length", str(len(body)))
        self.send_header("Cache-Control", "no-store")
        if extra:
            for k, v in extra.items():
                self.send_header(k, v)
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self) -> None: # noqa: N802
        path = urlparse(self.path).path
        if path in {"/", "/index.html"}:
            self._send(200, (WEB / "index.html").read_bytes(), "text/html; charset=utf-8")
            return
        if path == "/style.css":
            self._send(200, (WEB / "style.css").read_bytes(), "text/css; charset=utf-8")
            return
        if path == "/app.js":
            self._send(200, (WEB / "app.js").read_bytes(), "application/javascript; charset=utf-8")
            return
        if path == "/api/skill":
            body = skill_text().encode("utf-8")
            self._send(200, body, "text/markdown; charset=utf-8")
            return
        if path == "/api/skills":
            body = json.dumps(skills_as_dicts(), indent=2).encode("utf-8")
            self._send(200, body, "application/json; charset=utf-8")
            return
        if path.startswith("/api/skills/") and path.endswith("/export"):
            name = unquote(path[len("/api/skills/"):-len("/export")].strip("/"))
            try:
                skill = get_skill(name)
            except KeyError:
                self._send(404, json.dumps({"error": "skill not found"}).encode(), "application/json; charset=utf-8")
                return
            filename = f"{skill.slug}.md"
            extra = {"Content-Disposition": f'attachment; filename="{filename}"'}
            self._send(200, skill.body.encode("utf-8"), "text/markdown; charset=utf-8", extra)
            return
        if path.startswith("/api/skills/") and path != "/api/skills/":
            name = unquote(path[len("/api/skills/")].strip("/"))
            if name:
                try:
                    skill = get_skill(name)
                except KeyError:
                    self._send(404, json.dumps({"error": "skill not found"}).encode(), "application/json; charset=utf-8")
                    return
                payload = {
                    "name": skill.name,
                    "description": skill.description,
                    "source": skill.source,
                    "body": skill.body,
                    "slug": skill.slug,
                }
                self._send(200, json.dumps(payload, indent=2).encode(), "application/json; charset=utf-8")
                return
            self._send(404, LIMITATION.encode(), "text/plain; charset=utf-8")

    def do_POST(self) -> None: # noqa: N802
        path = urlparse(self.path).path
        if path != "/api/skills/import":
            self._send(404, LIMITATION.encode(), "text/plain; charset=utf-8")
            return
        length = int(self.headers.get("Content-Length", "0") or 0)
        if length <= 0 or length > 2 * 1024 * 1024:

```



```

        self._send(400, json.dumps({"ok": False, "error": "file required"}).encode(), "application/json; charset=utf-8")
    return
    raw = self.rfile.read(length)
    files, _fields = _parse_multipart(raw, _boundary(self.headers.get("Content-Type", "")))
    if not files:
        self._send(400, json.dumps({"ok": False, "error": "SKILL.md file required"}).encode(), "application/json; charset=utf-8")
    return
    name, blob = files[0]
    safe = posixpath.basename(name) or "SKILL.md"
    try:
        with tempfile.TemporaryDirectory() as tmp:
            incoming = Path(tmp) / safe
            incoming.write_bytes(blob)
            imported = import_skill(incoming)
        payload = {
            "ok": True,
            "name": imported.name,
            "description": imported.description,
            "slug": imported.slug,
            "path": str(imported.path),
        }
        self._send(200, json.dumps(payload, indent=2).encode(), "application/json; charset=utf-8")
    except Exception as exc: # noqa: BLE001
        self._send(400, json.dumps({"ok": False, "error": str(exc)}).encode(), "application/json; charset=utf-8")

def _boundary(ctype: str) -> bytes | None:
    if "boundary=" not in ctype:
        return None
    return ctype.split("boundary=", 1)[1].strip().encode()

def _parse_multipart(raw: bytes, boundary: bytes | None) -> tuple[list[tuple[str, bytes]], dict[str, str]]:
    files: list[tuple[str, bytes]] = []
    fields: dict[str, str] = {}
    if not boundary:
        return files, fields
    parts = raw.split(b"--" + boundary)
    for part in parts:
        if not part or part in (b"--", b"--\r\n"):
            continue
        if b"\r\n\r\n" not in part:
            continue
        header, body = part.split(b"\r\n\r\n", 1)
        if body.endswith(b"\r\n"):
            body = body[:-2]
        header_s = header.decode("utf-8", "replace")
        filename = None
        field = None
        for line in header_s.split("\r\n"):
            if "filename=" in line:
                filename = line.split("filename=", 1)[1].strip().strip("'")
            if "name=" in line and "filename=" not in line:
                field = line.split("name=", 1)[1].strip().strip("'").split(";")[0].strip().strip("'")
        if filename is not None:
            files.append((filename or "SKILL.md", body))
        elif field:
            fields[field] = body.decode("utf-8", "replace")
    return files, fields

def serve(host: str = "127.0.0.1", port: int = 8870) -> None:
    if host not in LOOPBACK:
        raise ValueError("AZBot UI binds loopback only (127.0.0.1)")
    httpd = ThreadingHTTPServer((host, port), Handler)
    print(f"AZBot UI http://{host}:{port} (loopback only)")
    print(LIMITATION)
    try:
        httpd.serve_forever()
    except KeyboardInterrupt:
        print("\nstopped")
    finally:
        httpd.server_close()

```

azbot/azbot/web/app.js

```

const OA = "https://aziel-runtime.vibelock.workers.dev/openapi.json";
const MCP = "https://aziel-runtime.vibelock.workers.dev/mcp";
const status = document.getElementById("status");
const skillEl = document.getElementById("skill");
const listEl = document.getElementById("skill-list");
const descEl = document.getElementById("skill-desc");
let selected = null;
let skills = [];

async function load() {
  const t = await fetch("/api/skill").then((r) => r.text());
  skillEl.textContent = t;
  window._skill = t;
  await loadSkills();
}

async function loadSkills() {
  skills = await fetch("/api/skills").then((r) => r.json());
  listEl.innerHTML = "";
  skills.forEach((s) => {
    const li = document.createElement("li");
    li.textContent = s.name;
    li.title = s.description;
    li.dataset.name = s.name;
    li.onclick = () => selectSkill(s);
    listEl.appendChild(li);
  });
  if (selected) {
    const still = skills.find((s) => s.name === selected.name);
    if (still) selectSkill(still);
  }
}

function selectSkill(s) {
  selected = s;
  [...listEl.children].forEach((li) => {
    li.classList.toggle("active", li.dataset.name === s.name);
  });
  descEl.textContent = s.description || "No description.";
}

function copy(text, label) {
  navigator.clipboard.writeText(text).then(() => { status.textContent = "Copied " + label; });
}

document.getElementById("copy-oa").onclick = () => copy(OA, "OpenAPI");
document.getElementById("copy-mcp").onclick = () => copy(MCP, "MCP");
document.getElementById("export").onclick = () => {
  const blob = new Blob([window._skill || ""], { type: "text/markdown" });
  const a = document.createElement("a");
  a.href = URL.createObjectURL(blob);
  a.download = "AZBot-SKILL.md";
  a.click();
  status.textContent = "Exported AZBot-SKILL.md";
};
document.getElementById("import-skill").onclick = async () => {
  const input = document.getElementById("skill-file");
  if (!input.files || !input.files[0]) {
    status.textContent = "Choose a SKILL.md file first.";
    return;
  }
  const fd = new FormData();
  fd.append("file", input.files[0], input.files[0].name);
  const res = await fetch("/api/skills/import", { method: "POST", body: fd });
  const body = await res.json();
  status.textContent = body.ok ? "Added skill " + body.name : (body.error || "Add skill failed");
  await loadSkills();
};
document.getElementById("export-skill").onclick = () => {
  if (!selected) {
    status.textContent = "Pick a skill, then Export skill.";
    return;
  }
  const a = document.createElement("a");
  a.href = "/api/skills/" + encodeURIComponent(selected.name) + "/export";
  a.download = (selected.slug || selected.name) + ".md";
  a.click();
  status.textContent = "Exported " + selected.name;
};
load();

```

azbot/azbot/web/index.html

```

<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>AZBot ? skill</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <header>
    <div class="tag">AZBot ? v0.2.0 ? 127.0.0.1:8870</div>
    <h1>One skill for every Aziel Eliab engine</h1>
    <p class="lede">Paste the OpenAPI into Grok, ChatGPT, or Venice. AZBot is a skill, not a model. Not Grok weights.
Jeeves is not sovereign.</p>
  </header>
  <div class="banner" role="note">Not a foundation model. Not a kernel. Not a VPN. Not an upload proxy. Not untraceabl
e origin. Call the catalog. Do not invent scores.</div>
  <section class="actions">
    <button type="button" id="copy-oa" class="action">Copy OpenAPI</button>
    <button type="button" id="copy-mcp" class="action">Copy MCP</button>
    <button type="button" id="export" class="action">Export skill</button>
  </section>
  <p id="status" class="muted">No copy yet.</p>
  <p><a href="https://aziel-runtime.vibelock.workers.dev/">Open the live catalog</a></p>

  <section class="card" id="skills-panel">
    <h2>Skills</h2>
    <p class="muted">Custom skills and Cursor SKILL.md files. Add a skill file. Export a skill file. These are local i
nstructions, not Grok weights.</p>
    <label class="file-label">Add skill
      <input type="file" id="skill-file" accept=".md,.markdown,text/markdown" />
    </label>
    <div class="actions">
      <button type="button" id="import-skill" class="action">Add skill</button>
      <button type="button" id="export-skill" class="action">Export skill</button>
    </div>
    <ul id="skill-list" class="skill-list"></ul>
    <p id="skill-desc" class="muted">Pick a skill to see what it does.</p>
  </section>

  <pre id="skill"></pre>

  <section class="card" id="aziel-json-io">
    <h2>Import / Export JSON</h2>
    <p class="help">Tap Import JSON file to pick a .json. Export JSON saves the current session locally. Nothing is uplo
aded.</p>
    <input id="aziel-import-json" type="file" accept="application/json,.json">
    <p>
      <button type="button" id="aziel-import-json-btn">Import JSON file</button>
      <button type="button" id="aziel-export-json-btn">Export JSON</button>
    </p>
    <p class="help" id="aziel-json-status"></p>
  </section>

  <footer>
    <p><strong>Peace ? Clarity ? Service.</strong></p>
    <p>Apache-2.0 ? Aziel Eliab ? 2026 ? Forks welcome and always allowed.</p>
  </footer>
  <script src="/app.js"></script>

  <script>
(function(){
  var file = document.getElementById("aziel-import-json");
  var imp = document.getElementById("aziel-import-json-btn");
  var exp = document.getElementById("aziel-export-json-btn");
  var status = document.getElementById("aziel-json-status");
  if (!file || !imp || !exp) return;
  function say(m){ if (status) status.textContent = m; }
  function collect(){
    var data = { product: document.title || "", exported_at: new Date().toISOString(), author: "Aziel Eliab" };
    document.querySelectorAll("input, select, textarea").forEach(function(el){
      if (!el.id || el.type === "file" || el.type === "password") return;
      data[el.id] = el.type === "checkbox" ? el.checked : el.value;
    });
    if (window.__azielLastJson && typeof window.__azielLastJson === "object") {
      data.last = window.__azielLastJson;
    }
    return data;
  }
  function apply(obj){
    if (!obj || typeof obj !== "object") return;
    window.__azielLastJson = obj;
    Object.keys(obj).forEach(function(k){
      if (k === "last" || k === "product" || k === "exported_at" || k === "author") return;
      var el = document.getElementById(k);
      if (!el || el.type === "file" || el.type === "password") return;
      if (el.type === "checkbox") el.checked = !!obj[k];
      else if ("value" in el) el.value = obj[k];
    });
    var ta = document.querySelector("textarea");
    if (ta && obj && !obj[ta.id] && typeof obj === "object") {
      try { if (!ta.value) ta.value = JSON.stringify(obj, null, 2); } catch (e) {}
    }
  }
}

```

```
    }  
  }  
  imp.addEventListener("click", function(){ file.click(); });  
  file.addEventListener("change", function(){  
    var f = file.files && file.files[0];  
    if (!f) return;  
    var r = new FileReader();  
    r.onload = function(){  
      try { apply(JSON.parse(String(r.result || "{}"))); say("Imported " + f.name); }  
      catch (e) { say("Invalid JSON"); }  
    };  
    r.readAsText(f);  
  });  
  exp.addEventListener("click", function(){  
    var blob = new Blob([JSON.stringify(collect(), null, 2)], {type: "application/json"});  
    var a = document.createElement("a");  
    a.href = URL.createObjectURL(blob);  
    a.download = "session.json";  
    a.click();  
    setTimeout(function(){ URL.revokeObjectURL(a.href); }, 800);  
    say("Exported JSON");  
  });  
})();  
</script>  
  
</body>  
</html>
```

azbot/azbot/web/style.css

```
:root { --bg:#0b0b0b; --gold:#c9a227; --ivory:#e8e0d0; --muted:#8a8070; --surface:#141414; }
* { box-sizing: border-box; }
html, body { margin:0; background:var(--bg); color:var(--ivory); font-family: Palatino, Georgia, serif; }
body { max-width: 44rem; margin: 0 auto; padding: 2rem 1.2rem 4rem; }
h1 { color: var(--gold); font-size: 1.6rem; }
h2 { color: var(--gold); font-size: 1.15rem; margin: 0 0 .4rem; }
.tag { font-family: ui-monospace, monospace; font-size: .75rem; color: var(--muted); letter-spacing: .12em; text-transform: uppercase; }
.banner { border:1px solid #5c4a1a; background:#241c0d; color:#f0d78c; padding:.75rem 1rem; border-radius:8px; margin:1rem 0; }
.actions { display:flex; flex-wrap:wrap; gap:.5rem; margin:1rem 0; }
button.action { appearance:none; border:1px solid var(--gold); background:var(--surface); color:var(--ivory); font:inherit; padding:.5rem .9rem; border-radius:6px; cursor:pointer; }
button.action:hover { background: var(--gold); color: var(--bg); }
.muted { color: var(--muted); }
pre { white-space: pre-wrap; background: var(--surface); padding: 1rem; border-radius: 8px; font-size: .85rem; }
a { color: var(--gold); }
footer { margin-top: 2rem; color: var(--muted); font-size: .85rem; }
footer strong { color: var(--gold); }
.card { border:1px solid #5c4a1a; background:var(--surface); border-radius:8px; padding:1rem; margin:1.5rem 0; }
.skill-list { list-style:none; padding:0; margin:.75rem 0; }
.skill-list li { border:1px solid #3a3428; padding:.45rem .7rem; border-radius:6px; margin-bottom:.35rem; cursor:pointer; }
.skill-list li.active, .skill-list li:hover { border-color: var(--gold); }
.file-label { display:block; color: var(--muted); margin:.5rem 0; }
.file-label input { display:block; margin-top:.35rem; }
```

azbot/install.sh

```
#!/usr/bin/env bash
# AZBot one-click install. Counted download via this project's Worker.
# Usage: curl -fsSL https://azbot-download-tracker.vibelock.workers.dev/install.sh | bash
set -euo pipefail

HOST="${AZBOT_HOME_HOST:-https://azbot-download-tracker.vibelock.workers.dev}"
ASSET="${AZBOT_HOME_ASSET:-azbot-0.2.0.tar.gz}"
WORKDIR="${AZBOT_HOME:-$HOME/azbot}"

mkdir -p "$WORKDIR"
cd "$WORKDIR"

echo "Downloading counted tarball from ${HOST}/download (User-Agent Mozilla/5.0)"
curl -fsSL -A 'Mozilla/5.0' "${HOST}/download?asset=${ASSET}" -o "${ASSET}"

tar -xzf "${ASSET}"
DIR="$(find . -maxdepth 1 -type d -name 'azbot-*' | head -n 1)"
if [ -n "${DIR}" ]; then
  cd "${DIR}"
fi

python3 -m venv .venv
# shellcheck disable=SC1091
. .venv/bin/activate
python -m pip install -U pip
python -m pip install -e .

echo
echo "Installed AZBot."
echo "Run: azbot ui"
echo "Then open http://127.0.0.1:8870 (loopback only)"
echo "Author: Aziel Eliab."
```

azbot/mobile/lib/main.dart

```

import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

const openapi = 'https://aziel-runtime.vibelock.workers.dev/openapi.json';
const mcp = 'https://aziel-runtime.vibelock.workers.dev/mcp';
const limitation =
  'AZBot is a skill, not a model, not a kernel, not a VPN. '
  'Jeeves is not sovereign. Paste the OpenAPI into Grok, ChatGPT, or Venice.';

void main() => runApp(const AzbotApp());

class AzbotApp extends StatelessWidget {
  const AzbotApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'AZBot',
      debugShowCheckedModeBanner: false,
      theme: ThemeData.dark().copyWith(primaryColor: const Color(0xFFC9A227)),
      home: const Home(),
    );
  }
}

class Home extends StatelessWidget {
  const Home({super.key});
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('AZBot')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          const Text(limitation),
          const SizedBox(height: 12),
          FilledButton(
            onPressed: () => Clipboard.setData(const ClipboardData(text: openapi)),
            child: const Text('Copy OpenAPI'),
          ),
          const SizedBox(height: 8),
          FilledButton(
            onPressed: () => Clipboard.setData(const ClipboardData(text: mcp)),
            child: const Text('Copy MCP'),
          ),
        ],
      ),
    );
  }
}

```

azbot/mobile/pubspec.yaml

```
name: azbot
description: AZBot skill viewer. Not a model.
publish_to: "none"
version: 0.1.0+1
environment:
  sdk: ">=3.3.0 <4.0.0"
dependencies:
  flutter:
    sdk: flutter
dev_dependencies:
  flutter_lints: ^5.0.0
flutter:
  uses-material-design: true
```


azbot/pyproject.toml

```
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "azbot"
version = "0.2.0"
description = "AZBot: Grok/ChatGPT/Venice skill plus a local runner for custom skills and Cursor SKILL.md files. Not a model. Not Grok weights."
readme = "README.md"
license = {file = "LICENSE"}
authors = [{name = "Aziel Eliab"}]
requires-python = ">=3.10"
keywords = ["azbot", "skill", "grok", "mcp", "openapi", "cursor"]
classifiers = [
    "License :: OSI Approved :: Apache Software License",
    "Programming Language :: Python :: 3",
]

[project.scripts]
azbot = "azbot.cli:main"

[project.optional-dependencies]
dev = ["pytest>=7.4"]

[project.urls]
Homepage = "https://github.com/AzielEliab/azbot"
Download = "https://azbot-download-tracker.vibelock.workers.dev/download?asset=azbot-0.2.0.tar.gz"

[tool.setuptools.packages.find]
include = ["azbot*"]

[tool.setuptools.package-data]
azbot = ["web/*", "SKILL.md"]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]
addopts = "-q"
```

azbot/standalone/app/ark.py

```
#!/usr/bin/env python3
"""ARK envelope ? encrypted body + encrypted metadata, phoenix re-seal on view.

Local vault format only. Does not hide who uploaded a file.
Does not execute anything when a third party opens the blob.
"""
from __future__ import annotations

import argparse
import base64
import json
import os
import secrets
import struct
import sys
import tempfile
from datetime import datetime, timezone
from pathlib import Path

from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.scrypt import Scrypt

MAGIC = b"ARK1"
SALT_LEN = 16
NONCE_LEN = 12
DEK_LEN = 32
SCRYPT_N = 2**14
SCRYPT_R = 8
SCRYPT_P = 1

def _b64(b: bytes) -> str:
    return base64.b64encode(b).decode("ascii")

def _ub64(s: str) -> bytes:
    return base64.b64decode(s.encode("ascii"))

def _kek(passphrase: str, salt: bytes) -> bytes:
    kdf = Scrypt(salt=salt, length=32, n=SCRYPT_N, r=SCRYPT_R, p=SCRYPT_P)
    return kdf.derive(passphrase.encode("utf-8"))

def _pack_bytes(header: dict, meta_ct: bytes, body_ct: bytes) -> bytes:
    raw_h = json.dumps(header, separators=(",", ":")).encode("utf-8")
    return MAGIC + struct.pack(">I", len(raw_h)) + raw_h + struct.pack(">I", len(meta_ct)) + meta_ct + struct.pack(">Q", len(body_ct)) + body_ct

def _unpack_bytes(blob: bytes) -> tuple[dict, bytes, bytes]:
    if blob[:4] != MAGIC:
        raise ValueError("not an ARK1 envelope")
    (hlen,) = struct.unpack(">I", blob[4:8])
    off = 8
    header = json.loads(blob[off : off + hlen].decode("utf-8"))
    off += hlen
    (mlen,) = struct.unpack(">I", blob[off : off + 4])
    off += 4
    meta_ct = blob[off : off + mlen]
    off += mlen
    (blen,) = struct.unpack(">Q", blob[off : off + 8])
    off += 8
    body_ct = blob[off : off + blen]
    return header, meta_ct, body_ct

def seal(src: Path, dest: Path, passphrase: str, extra_meta: dict | None = None) -> dict:
    src = src.expanduser().resolve()
    body = src.read_bytes()
    salt = secrets.token_bytes(SALT_LEN)
    dek = secrets.token_bytes(DEK_LEN)
    kek = _kek(passphrase, salt)
    wrap_n = secrets.token_bytes(NONCE_LEN)
    meta_n = secrets.token_bytes(NONCE_LEN)
    body_n = secrets.token_bytes(NONCE_LEN)
    wrapped = AESGCM(kek).encrypt(wrap_n, dek, b"ark-dek")
    meta = {
        "name": src.name,
        "bytes": len(body),
        "sha256": __import__("hashlib").sha256(body).hexdigest(),
        "sealed_utc": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
        "gen": 0,
        "extra": extra_meta or {},
    }
    meta_ct = AESGCM(dek).encrypt(meta_n, json.dumps(meta).encode("utf-8"), b"ark-meta")
    body_ct = AESGCM(dek).encrypt(body_n, body, b"ark-body")
    header = {
        "v": 1,
        "gen": 0,
        "kdf": "scrypt",
        "n": SCRYPT_N,
```

```

        "r": SCRYPT_R,
        "p": SCRYPT_P,
        "salt": _b64(salt),
        "wrap_nonce": _b64(wrap_n),
        "wrapped_dek": _b64(wrapped),
        "nonce_meta": _b64(meta_n),
        "nonce_body": _b64(body_n),
    }
    dest.parent.mkdir(parents=True, exist_ok=True)
    dest.write_bytes(_pack_bytes(header, meta_ct, body_ct))
    return {"ark": str(dest), "gen": 0, "bytes": dest.stat().st_size, "name": src.name}

def _open_env(blob: bytes, passphrase: str) -> tuple[dict, dict, bytes, bytes]:
    header, meta_ct, body_ct = _unpack_bytes(blob)
    kek = _kek(passphrase, _b64(header["salt"]))
    dek = AESGCM(kek).decrypt(_b64(header["wrap_nonce"]), _b64(header["wrapped_dek"]), b"ark-dek")
    meta = json.loads(AESGCM(dek).decrypt(_b64(header["nonce_meta"]), meta_ct, b"ark-meta").decode("utf-8"))
    body = AESGCM(dek).decrypt(_b64(header["nonce_body"]), body_ct, b"ark-body")
    return header, meta, body, dek

def inspect(src: Path, passphrase: str) -> dict:
    header, meta, body, _dek = _open_env(src.read_bytes(), passphrase)
    return {
        "header_gen": header.get("gen"),
        "meta": meta,
        "body_bytes": len(body),
        "body_sha256_ok": __import__("hashlib").sha256(body).hexdigest() == meta.get("sha256"),
    }

def phoenix(src: Path, passphrase: str, extract_to: Path | None = None) -> dict:
    """Authorized view: decrypt, optional extract, re-seal with gen+1, overwrite envelope."""
    src = src.expanduser().resolve()
    header, meta, body, _old_dek = _open_env(src.read_bytes(), passphrase)
    if extract_to is not None:
        extract_to.parent.mkdir(parents=True, exist_ok=True)
        extract_to.write_bytes(body)
    new_gen = int(header.get("gen", 0)) + 1
    meta["gen"] = new_gen
    meta["phoenix_utc"] = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")
    salt = secrets.token_bytes(SALT_LEN)
    dek = secrets.token_bytes(DEK_LEN)
    kek = _kek(passphrase, salt)
    wrap_n = secrets.token_bytes(NONCE_LEN)
    meta_n = secrets.token_bytes(NONCE_LEN)
    body_n = secrets.token_bytes(NONCE_LEN)
    wrapped = AESGCM(kek).encrypt(wrap_n, dek, b"ark-dek")
    meta_ct = AESGCM(dek).encrypt(meta_n, json.dumps(meta).encode("utf-8"), b"ark-meta")
    body_ct = AESGCM(dek).encrypt(body_n, body, b"ark-body")
    header = {
        "v": 1,
        "gen": new_gen,
        "kdf": "scrypt",
        "n": SCRYPT_N,
        "r": SCRYPT_R,
        "p": SCRYPT_P,
        "salt": _b64(salt),
        "wrap_nonce": _b64(wrap_n),
        "wrapped_dek": _b64(wrapped),
        "nonce_meta": _b64(meta_n),
        "nonce_body": _b64(body_n),
    }
    payload = _pack_bytes(header, meta_ct, body_ct)
    fd, tmp = tempfile.mkstemp(prefix="ark-", dir=str(src.parent))
    os.close(fd)
    tmp_path = Path(tmp)
    try:
        tmp_path.write_bytes(payload)
        os.replace(tmp_path, src)
    finally:
        if tmp_path.exists():
            tmp_path.unlink(missing_ok=True)
    return {
        "ark": str(src),
        "gen": new_gen,
        "extracted": str(extract_to) if extract_to else None,
        "name": meta.get("name"),
    }

def main(argv: list[str]) -> int:
    p = argparse.ArgumentParser(prog="ark")
    sub = p.add_subparsers(dest="cmd", required=True)

    s = sub.add_parser("seal")
    s.add_argument("src")
    s.add_argument("--out")
    s.add_argument("--pass", dest="pw", required=True)

    i = sub.add_parser("inspect")
    i.add_argument("src")
    i.add_argument("--pass", dest="pw", required=True)
    ph = sub.add_parser("phoenix")

```

```
ph.add_argument("src")
ph.add_argument("--pass", dest="pw", required=True)
ph.add_argument("--extract")

args = p.parse_args(argv[1:])
if args.cmd == "seal":
    src = Path(args.src)
    dest = Path(args.out) if args.out else src.with_suffix(src.suffix + ".ark")
    print(json.dumps(seal(src, dest, args.pw), indent=2))
elif args.cmd == "inspect":
    print(json.dumps(inspect(Path(args.src), args.pw), indent=2))
elif args.cmd == "phoenix":
    ext = Path(args.extract) if args.extract else None
    print(json.dumps(phoenix(Path(args.src), args.pw, ext), indent=2))
return 0

if __name__ == "__main__":
    raise SystemExit(main(sys.argv))
```

azbot/standalone/app/export_findings.py

```

#!/usr/bin/env python3
"""Export findings to JSONL + optional chart PNG/SVG. No network."""
from __future__ import annotations

import argparse
import csv
import json
from datetime import datetime, timezone
from pathlib import Path

def write_jsonl(rows: list[dict], dest: Path) -> Path:
    dest.parent.mkdir(parents=True, exist_ok=True)
    with dest.open("w", encoding="utf-8") as f:
        for row in rows:
            f.write(json.dumps(row, ensure_ascii=False) + "\n")
    return dest

def write_csv(rows: list[dict], dest: Path) -> Path:
    dest.parent.mkdir(parents=True, exist_ok=True)
    keys: list[str] = []
    for row in rows:
        for k in row:
            if k not in keys:
                keys.append(k)
    with dest.open("w", encoding="utf-8", newline="") as f:
        w = csv.DictWriter(f, fieldnames=keys)
        w.writeheader()
        w.writerows(rows)
    return dest

def chart_xy(points: list[tuple[float, float]], dest: Path, title: str = "finding") -> Path:
    import matplotlib

    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    dest.parent.mkdir(parents=True, exist_ok=True)
    xs = [p[0] for p in points]
    ys = [p[1] for p in points]
    fig, ax = plt.subplots(figsize=(8, 4.5), dpi=140)
    ax.plot(xs, ys, color="#c5d0d8", linewidth=1.6)
    ax.set_title(title)
    ax.set_facecolor("#12161b")
    fig.patch.set_facecolor("#0b0d10")
    ax.tick_params(colors="#9aa7b4")
    ax.spines["bottom"].set_color("#24303a")
    ax.spines["left"].set_color("#24303a")
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    fig.tight_layout()
    fig.savefig(dest)
    plt.close(fig)
    return dest

def bundle(name: str, rows: list[dict], out_dir: Path, points: list[tuple[float, float]] | None = None) -> dict:
    stamp = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S")
    base = out_dir / f"{name}-{stamp}"
    out: dict = {"name": name, "utc": stamp}
    out["jsonl"] = str(write_jsonl(rows, Path(str(base) + ".jsonl")))
    out["csv"] = str(write_csv(rows, Path(str(base) + ".csv")))
    if points:
        out["png"] = str(chart_xy(points, Path(str(base) + ".png"), title=name))
        out["svg"] = str(chart_xy(points, Path(str(base) + ".svg"), title=name))
    return out

def main(argv: list[str] | None = None) -> int:
    p = argparse.ArgumentParser()
    p.add_argument("--name", default="findings")
    p.add_argument("--in-json", required=True, help="JSON list of objects")
    p.add_argument("--out", default="out")
    p.add_argument("--x", help="numeric key for chart X")
    p.add_argument("--y", help="numeric key for chart Y")
    args = p.parse_args(argv)
    rows = json.loads(Path(args.in_json).read_text(encoding="utf-8"))
    if not isinstance(rows, list):
        raise SystemExit("input must be a JSON list")
    points = None
    if args.x and args.y:
        points = [(float(r[args.x]), float(r[args.y])) for r in rows]
    print(json.dumps(bundle(args.name, rows, Path(args.out), points), indent=2))
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

azbot/standalone/app/publish.py

```

#!/usr/bin/env python3
"""Official-API publish adapters. Tokens from env only. Never stores passwords.

AZBOT_PUBLISH=1 required to hit the network.
AZBOT_X_BEARER = user-context bearer for X API v2.
"""
from __future__ import annotations

import json
import os
import sys
import urllib.error
import urllib.request
from pathlib import Path

def _require_publish() -> None:
    if os.environ.get("AZBOT_PUBLISH") != "1":
        raise RuntimeError("publish gated: set AZBOT_PUBLISH=1 to allow network post")

def post_x_tweet(text: str) -> dict:
    _require_publish()
    token = os.environ.get("AZBOT_X_BEARER", "").strip()
    if not token:
        raise RuntimeError("AZBOT_X_BEARER missing")
    if len(text) > 25000:
        raise RuntimeError("text too long")
    req = urllib.request.Request(
        "https://api.x.com/2/tweets",
        data=json.dumps({"text": text}).encode("utf-8"),
        headers={
            "Authorization": f"Bearer {token}",
            "Content-Type": "application/json",
            "User-Agent": "AZbotLocal/1.3",
        },
        method="POST",
    )
    try:
        with urllib.request.urlopen(req, timeout=30) as resp:
            return json.loads(resp.read().decode("utf-8"))
    except urllib.error.HTTPError as e:
        body = e.read().decode("utf-8", "replace")
        raise RuntimeError(f"X API {e.code}: {body[:500]}") from e

def reply_x(text: str, in_reply_to: str) -> dict:
    _require_publish()
    token = os.environ.get("AZBOT_X_BEARER", "").strip()
    if not token:
        raise RuntimeError("AZBOT_X_BEARER missing")
    payload = {"text": text, "reply": {"in_reply_to_tweet_id": str(in_reply_to)}}
    req = urllib.request.Request(
        "https://api.x.com/2/tweets",
        data=json.dumps(payload).encode("utf-8"),
        headers={
            "Authorization": f"Bearer {token}",
            "Content-Type": "application/json",
            "User-Agent": "AZbotLocal/1.3",
        },
        method="POST",
    )
    try:
        with urllib.request.urlopen(req, timeout=30) as resp:
            return json.loads(resp.read().decode("utf-8"))
    except urllib.error.HTTPError as e:
        body = e.read().decode("utf-8", "replace")
        raise RuntimeError(f"X API {e.code}: {body[:500]}") from e

def draft_only(action: str, payload: dict, queue_dir: Path) -> Path:
    queue_dir.mkdir(parents=True, exist_ok=True)
    n = len(list(queue_dir.glob("*.json"))) + 1
    path = queue_dir / f"{n:04d}-{action}.json"
    rec = {"action": action, "payload": payload, "status": "draft"}
    path.write_text(json.dumps(rec, indent=2) + "\n", encoding="utf-8")
    return path

def main(argv: list[str]) -> int:
    if len(argv) < 2:
        print("usage: publish.py draft-x TEXT | post-x TEXT | reply-x ID TEXT")
        return 2
    cmd = argv[1]
    root = Path(__file__).resolve().parent.parent
    queue = root / "queue"
    if cmd == "draft-x":
        text = " ".join(argv[2:])
        print(draft_only("x.post", {"text": text}, queue))
        return 0
    if cmd == "post-x":
        text = " ".join(argv[2:])

```

```
        print(json.dumps(post_x_tweet(text), indent=2))
        return 0
    if cmd == "reply-x":
        print(json.dumps(reply_x(" ".join(argv[3:]), argv[2]), indent=2))
        return 0
    print("unknown command")
    return 2

if __name__ == "__main__":
    raise SystemExit(main(sys.argv))
```

azbot/standalone/app/receipt.py

```
#!/usr/bin/env python3
"""Write a SHA-256 receipt next to a file. Narrative-free."""
from __future__ import annotations

import hashlib
import json
import sys
from datetime import datetime, timezone
from pathlib import Path

def sha256_file(path: Path) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1024 * 1024), b""):
            h.update(chunk)
    return h.hexdigest()

def write_receipt(path: Path) -> Path:
    path = path.expanduser().resolve()
    digest = sha256_file(path)
    rec = {
        "file": path.name,
        "sha256": digest,
        "bytes": path.stat().st_size,
        "recorded_utc": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
        "note": "hash of this copy only",
    }
    out = path.with_suffix(path.suffix + ".receipt.json")
    out.write_text(json.dumps(rec, indent=2) + "\n", encoding="utf-8")
    return out

def main(argv: list[str]) -> int:
    if len(argv) < 2:
        print("usage: receipt.py <file> [more files...]")
        return 2
    for raw in argv[1:]:
        p = write_receipt(Path(raw))
        print(p)
    return 0

if __name__ == "__main__":
    raise SystemExit(main(sys.argv))
```


azbot/standalone/app/runner.py

```
#!/usr/bin/env python3
"""Local autonomous runner. Executes queued task files. Default is draft-only.

Task JSON:
{
  "action": "export.findings" | "x.post" | "x.reply" | "echo",
  "payload": {...},
  "publish": false
}

x.* only hits the network when publish=true AND AZBOT_PUBLISH=1 AND bearer is set.
"""
from __future__ import annotations

import json
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent))
from export_findings import bundle
from publish import post_x_tweet, reply_x

def run_task(path: Path, out_dir: Path) -> dict:
    spec = json.loads(path.read_text(encoding="utf-8"))
    action = spec.get("action")
    payload = spec.get("payload") or {}
    publish = bool(spec.get("publish"))
    if action == "echo":
        result = {"echo": payload}
    elif action == "export.findings":
        rows = payload.get("rows") or []
        points = payload.get("points")
        pts = [tuple(p) for p in points] if points else None
        result = bundle(payload.get("name") or path.stem, rows, out_dir, pts)
    elif action == "x.post":
        if not publish:
            result = {"status": "draft", "text": payload.get("text")}
        else:
            result = post_x_tweet(payload.get("text") or "")
    elif action == "x.reply":
        if not publish:
            result = {"status": "draft", "text": payload.get("text"), "in_reply_to": payload.get("in_reply_to")}
        else:
            result = reply_x(payload.get("text") or "", str(payload.get("in_reply_to")))
    else:
        raise RuntimeError(f"unknown action: {action}")
    spec["last_result"] = result
    spec["status"] = "ran"
    path.write_text(json.dumps(spec, indent=2) + "\n", encoding="utf-8")
    return {"task": str(path), "action": action, "result": result}

def main(argv: list[str]) -> int:
    root = Path(__file__).resolve().parent.parent
    queue = Path(argv[1]) if len(argv) > 1 else root / "queue"
    out = root / "out"
    if not queue.exists():
        print(json.dumps({"ok": True, "ran": 0, "note": "empty queue"}))
        return 0
    ran = []
    for path in sorted(queue.glob("*.json")):
        spec = json.loads(path.read_text(encoding="utf-8"))
        if spec.get("status") == "ran" and not spec.get("repeat"):
            continue
        ran.append(run_task(path, out))
    print(json.dumps({"ok": True, "ran": len(ran), "results": ran}, indent=2))
    return 0

if __name__ == "__main__":
    raise SystemExit(main(sys.argv))
```

azbot/standalone/app/sanitize.py

```
#!/usr/bin/env python3
"""AZbot Slingshot Prep ? strip identifying embedded metadata from a COPY.

Does not upload. Does not rewrite content timestamps to fake origin.
Original file is never overwritten.
"""
from __future__ import annotations

import io
import os
import shutil
import subprocess
import sys
import tempfile
import zipfile
from pathlib import Path
from xml.etree import ElementTree as ET

IMAGE_EXT = {".jpg", ".jpeg", ".png", ".webp", ".tif", ".tiff", ".bmp"}
MEDIA_EXT = {".mp4", ".mov", ".m4v", ".mkv", ".webm", ".mp3", ".wav", ".m4a", ".aac"}
OFFICE_EXT = {".docx", ".xlsx", ".pptx"}
PDF_EXT = {".pdf"}

OFFICE_META_NAMES = {
    "docProps/core.xml",
    "docProps/app.xml",
    "docProps/custom.xml",
}

def _out_path(src: Path, dest_dir: Path) -> Path:
    dest_dir.mkdir(parents=True, exist_ok=True)
    return dest_dir / src.name

def sanitize_image(src: Path, dest: Path) -> str:
    from PIL import Image

    with Image.open(src) as im:
        clean = im.convert("RGB") if src.suffix.lower() in {".jpg", ".jpeg"} else im.copy()
        save_kw = {}
        if src.suffix.lower() in {".jpg", ".jpeg"}:
            save_kw.update(quality=95, optimize=True)
        clean.save(dest, **save_kw)
    return "image-reencoded-no-exif"

def sanitize_pdf(src: Path, dest: Path) -> str:
    from pypdf import PdfReader, PdfWriter

    reader = PdfReader(str(src))
    writer = PdfWriter()
    for page in reader.pages:
        writer.add_page(page)
    # Empty document info; drop XMP if present by not copying metadata
    writer.add_metadata({})
    if getattr(reader, "metadata", None):
        pass
    with open(dest, "wb") as f:
        writer.write(f)
    return "pdf-info-cleared"

def sanitize_office(src: Path, dest: Path) -> str:
    with zipfile.ZipFile(src, "r") as zin, zipfile.ZipFile(
        dest, "w", compression=zipfile.ZIP_DEFLATED
    ) as zout:
        for item in zin.infolist():
            data = zin.read(item.filename)
            name = item.filename.replace("\\", "/")
            if name in OFFICE_META_NAMES:
                data = _blank_office_xml(name, data)
            # drop thumbnail junk that can carry device previews
            if name.startswith("docProps/thumbnail"):
                continue
            zout.writestr(item, data)
    return "office-core-props-blanked"

def _blank_office_xml(name: str, data: bytes) -> bytes:
    try:
        root = ET.fromstring(data)
    except ET.ParseError:
        return data
    # Wipe text in known identifying elements; keep structure so file stays valid
    wipe_local = {
        "creator",
        "lastModifiedBy",
        "company",
        "manager",
        "title",
        "subject",
    }
```

```

        "description",
        "keywords",
        "category",
        "identifier",
        "revision",
        "version",
        "application",
        "appVersion",
        "template",
        "headingPairs",
        "titlesOfParts",
    }
    for el in root.iter():
        local = el.tag.split("}")[1] if "}" in el.tag else el.tag
        if local in wipe_local:
            el.text = ""
            el.tail = None
            for child in list(el):
                el.remove(child)
    return ET.tostring(root, encoding="utf-8", xml_declaration=True)

def sanitize_media(src: Path, dest: Path) -> str:
    cmd = [
        "ffmpeg",
        "-y",
        "-i",
        str(src),
        "-map_metadata",
        "-1",
        "-c",
        "copy",
        str(dest),
    ]
    try:
        subprocess.run(cmd, check=True, capture_output=True)
        return "media-container-metadata-stripped"
    except (subprocess.CalledProcessError, FileNotFoundError):
        shutil.copy2(src, dest)
        return "media-copy-only-ffmpeg-unavailable"

def sanitize_generic(src: Path, dest: Path) -> str:
    shutil.copy2(src, dest)
    return "copied-no-known-metadata-handler"

def sanitize_file(src: Path, dest_dir: Path) -> dict:
    src = src.expanduser().resolve()
    if not src.is_file():
        raise FileNotFoundError(src)
    dest = _out_path(src, dest_dir)
    # never write onto the source
    if dest.resolve() == src:
        dest = dest_dir / f"clean-{src.name}"
    ext = src.suffix.lower()
    if ext in IMAGE_EXT:
        method = sanitize_image(src, dest)
    elif ext in PDF_EXT:
        method = sanitize_pdf(src, dest)
    elif ext in OFFICE_EXT:
        method = sanitize_office(src, dest)
    elif ext in MEDIA_EXT:
        method = sanitize_media(src, dest)
    else:
        method = sanitize_generic(src, dest)
    return {
        "source": str(src),
        "clean": str(dest),
        "method": method,
        "bytes": dest.stat().st_size,
    }

def main(argv: list[str]) -> int:
    if len(argv) < 2:
        print("usage: sanitize.py <file> [more files...] [--out DIR]")
        return 2
    dest_dir = Path("out")
    files = []
    args = argv[1:]
    i = 0
    while i < len(args):
        if args[i] == "--out" and i + 1 < len(args):
            dest_dir = Path(args[i + 1])
            i += 2
            continue
        files.append(Path(args[i]))
        i += 1
    for f in files:
        info = sanitize_file(f, dest_dir)
        print(f"{info['method']}\t{info['clean']}")
    return 0
if __name__ == "__main__":
    raise SystemExit(main(sys.argv))

```

azbot/standalone/app/server.py

```

#!/usr/bin/env python3
"""Local-only AZbot shell. Binds to 127.0.0.1. Does not upload."""
from __future__ import annotations

import json
import posixpath
import sys
import tempfile
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
from urllib.parse import unquote, urlparse

APP_DIR = Path(__file__).resolve().parent
ROOT = APP_DIR.parent
REPO = ROOT.parent
STATIC = APP_DIR / "static"
OUT = ROOT / "out"
UPLOADS = ROOT / "work"

sys.path.insert(0, str(REPO))
sys.path.insert(0, str(APP_DIR))
from receipt import write_receipt # noqa: E402
from sanitize import sanitize_file # noqa: E402

try:
    from azbot.skills_loader import get_skill, import_skill, skills_as_dicts
except ImportError: # pragma: no cover
    get_skill = None # type: ignore[assignment]
    import_skill = None # type: ignore[assignment]
    skills_as_dicts = None # type: ignore[assignment]

class Handler(BaseHTTPRequestHandler):
    server_version = "AZbotLocal/0.2.0"

    def log_message(self, fmt: str, *args) -> None:
        sys.stderr.write("%s - %s\n" % (self.address_string(), fmt % args))

    def _send(self, code: int, body: bytes, ctype: str, extra: dict[str, str] | None = None) -> None:
        self.send_response(code)
        self.send_header("Content-Type", ctype)
        self.send_header("Content-Length", str(len(body)))
        self.send_header("Cache-Control", "no-store")
        if extra:
            for k, v in extra.items():
                self.send_header(k, v)
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self) -> None:
        path = urlparse(self.path).path
        if path in ("/", "/index.html"):
            data = (STATIC / "index.html").read_bytes()
            self._send(200, data, "text/html; charset=utf-8")
            return
        if path == "/health":
            self._send(200, b'{"ok":true,"bind":"127.0.0.1","version":"0.2.0"}', "application/json")
            return
        if path == "/api/skills":
            if skills_as_dicts is None:
                self._send(500, b'{"error":"skills loader missing"}', "application/json")
                return
            body = json.dumps(skills_as_dicts(REPO), indent=2).encode()
            self._send(200, body, "application/json; charset=utf-8")
            return
        if path.startswith("/api/skills/") and path.endswith("/export"):
            if get_skill is None:
                self._send(500, b'{"error":"skills loader missing"}', "application/json")
                return
            name = unquote(path[len("/api/skills/"):-len("/export")].strip("/"))
            try:
                skill = get_skill(name, root=REPO)
            except KeyError:
                self._send(404, b'{"error":"skill not found"}', "application/json")
                return
            extra = {"Content-Disposition": f'attachment; filename="{skill.slug}.md"' }
            self._send(200, skill.body.encode("utf-8"), "text/markdown; charset=utf-8", extra)
            return
        if path.startswith("/api/skills/") and path != "/api/skills/":
            if get_skill is None:
                self._send(500, b'{"error":"skills loader missing"}', "application/json")
                return
            name = unquote(path[len("/api/skills/")].strip("/"))
            if name:
                try:
                    skill = get_skill(name, root=REPO)
                except KeyError:
                    self._send(404, b'{"error":"skill not found"}', "application/json")
                    return
            payload = {
                "name": skill.name,
                "description": skill.description,

```

```

        "source": skill.source,
        "body": skill.body,
        "slug": skill.slug,
    }
    self._send(200, json.dumps(payload, indent=2).encode(), "application/json; charset=utf-8")
    return
self._send(404, b"not found", "text/plain")

def do_POST(self) -> None:
    path = urlparse(self.path).path
    if path == "/api/skills/import":
        self._handle_skill_import()
        return
    if path not in ("/prep", "/ark/seal", "/ark/phoenix"):
        self._send(404, b"not found", "text/plain")
        return
    length = int(self.headers.get("Content-Length", "0"))
    raw = self.rfile.read(length)
    boundary = _boundary(self.headers.get("Content-Type", ""))
    if not boundary:
        self._send(400, b'{"error": "multipart required"}', "application/json")
        return
    files, fields = _parse_multipart(raw, boundary)
    UPLOADS.mkdir(parents=True, exist_ok=True)
    OUT.mkdir(parents=True, exist_ok=True)
    try:
        if path == "/prep":
            payload = _handle_prep(files)
        elif path == "/ark/seal":
            payload = _handle_ark_seal(files, fields)
        else:
            payload = _handle_ark_phoenix(files, fields)
    except Exception as exc:
        self._send(400, json.dumps({"ok": False, "error": str(exc)}).encode(), "application/json")
        return
    self._send(200, json.dumps({"ok": True, **payload}, indent=2).encode(), "application/json")

def _handle_skill_import(self) -> None:
    if import_skill is None:
        self._send(500, b'{"error": "skills loader missing"}', "application/json")
        return
    length = int(self.headers.get("Content-Length", "0") or 0)
    if length <= 0 or length > 2 * 1024 * 1024:
        self._send(400, b'{"ok": false, "error": "file required"}', "application/json")
        return
    raw = self.rfile.read(length)
    files, _fields = _parse_multipart(raw, _boundary(self.headers.get("Content-Type", "")))
    if not files:
        self._send(400, b'{"ok": false, "error": "SKILL.md file required"}', "application/json")
        return
    name, blob = files[0]
    safe = posixpath.basename(name) or "SKILL.md"
    try:
        with tempfile.TemporaryDirectory() as tmp:
            incoming = Path(tmp) / safe
            incoming.write_bytes(blob)
            imported = import_skill(incoming, root=REPO)
        payload = {
            "ok": True,
            "name": imported.name,
            "description": imported.description,
            "slug": imported.slug,
            "path": str(imported.path),
        }
        self._send(200, json.dumps(payload, indent=2).encode(), "application/json")
    except Exception as exc:
        self._send(400, json.dumps({"ok": False, "error": str(exc)}).encode(), "application/json")

def _boundary(ctype: str) -> bytes | None:
    if "boundary=" not in ctype:
        return None
    return ctype.split("boundary=", 1)[1].strip().encode()

def _parse_multipart(raw: bytes, boundary: bytes | None) -> tuple[list[tuple[str, bytes]], dict[str, str]]:
    files: list[tuple[str, bytes]] = []
    fields: dict[str, str] = {}
    if not boundary:
        return files, fields
    parts = raw.split(b"--" + boundary)
    for part in parts:
        if not part or part in (b"--", b"--\r\n"):
            continue
        if b"\r\n\r\n" not in part:
            continue
        header, body = part.split(b"\r\n\r\n", 1)
        if body.endswith(b"\r\n"):
            body = body[:-2]
        header_s = header.decode("utf-8", "replace")
        filename = None
        field = None
        for line in header_s.split("\r\n"):
            if "filename=" in line:
                filename = line.split("filename=", 1)[1].strip().strip("'")

```

```

        if "name=" in line and "filename=" not in line:
            field = line.split("name=", 1)[1].strip().strip('\'').split(";")[0].strip().strip('\'')
        if "name=" in line and "filename=" in line:
            field = None
        if filename is not None:
            files.append((filename or "file.bin", body))
        elif field:
            fields[field] = body.decode("utf-8", "replace")
    return files, fields

def _handle_prep(files: list[tuple[str, bytes]]) -> dict:
    results = []
    for name, blob in files:
        safe = posixpath.basename(name) or "file.bin"
        incoming = UPLOADS / safe
        incoming.write_bytes(blob)
        info = sanitize_file(incoming, OUT)
        rec = write_receipt(Path(info["clean"]))
        results.append(
            {
                "original_name": safe,
                "clean": info["clean"],
                "method": info["method"],
                "bytes": info["bytes"],
                "receipt": str(rec),
            }
        )
    return {"results": results}

def _handle_ark_seal(files: list[tuple[str, bytes]], fields: dict[str, str]) -> dict:
    from ark import seal

    pw = fields.get("pass") or fields.get("pw") or ""
    if not pw:
        raise ValueError("pass required")
    results = []
    for name, blob in files:
        safe = posixpath.basename(name) or "file.bin"
        incoming = UPLOADS / safe
        incoming.write_bytes(blob)
        dest = OUT / (safe + ".ark")
        results.append(seal(incoming, dest, pw))
    return {"results": results}

def _handle_ark_phoenix(files: list[tuple[str, bytes]], fields: dict[str, str]) -> dict:
    from ark import phoenix

    pw = fields.get("pass") or fields.get("pw") or ""
    if not pw:
        raise ValueError("pass required")
    results = []
    for name, blob in files:
        safe = posixpath.basename(name) or "file.bin"
        incoming = UPLOADS / safe
        incoming.write_bytes(blob)
        dest = OUT / safe
        dest.write_bytes(incoming.read_bytes())
        extract = OUT / (safe + ".extracted")
        results.append(phoenix(dest, pw, extract))
    return {"results": results}

def main() -> None:
    host, port = "127.0.0.1", 7747
    httpd = ThreadingHTTPServer((host, port), Handler)
    print(f"AZbot local shell http://{host}:{port}")
    print("Slingshot Prep is local-only. This process does not upload.")
    print("Not Grok weights. Not an upload proxy. Not untraceable origin.")
    try:
        httpd.serve_forever()
    except KeyboardInterrupt:
        print("\nstop")

if __name__ == "__main__":
    main()

```

azbot/standalone/app/static/index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>AZbot Local</title>
  <style>
    :root { color-scheme: dark; }
    body { font: 15px/1.4 ui-sans-serif, system-ui, sans-serif; margin: 0; background: #0b0d10; color: #e8edf2; }
    main { max-width: 720px; margin: 0 auto; padding: 32px 20px 80px; }
    h1 { font-size: 22px; font-weight: 650; letter-spacing: .02em; margin: 0 0 8px; }
    .sub { color: #9aa7b4; margin-bottom: 28px; }
    .card { border: 1px solid #24303a; background: #12161b; border-radius: 12px; padding: 20px; margin-bottom: 16px; }
    label { display: block; margin-bottom: 10px; color: #c5d0d8; }
    input[type=file], input[type=password] { width: 100%; box-sizing: border-box; background: #0b0d10; color: #e8edf2;
border: 1px solid #24303a; border-radius: 8px; padding: 8px; }
    button { background: #e8edf2; color: #0b0d10; border: 0; border-radius: 8px; padding: 10px 16px; font-weight: 650;
cursor: pointer; margin-right: 8px; margin-top: 6px; }
    button:disabled { opacity: .5; cursor: wait; }
    pre { white-space: pre-wrap; word-break: break-word; background: #0b0d10; padding: 12px; border-radius: 8px; font-
size: 12px; }
    .warn { color: #d2b48c; }
    ul.skill-list { list-style: none; padding: 0; margin: 10px 0; }
    ul.skill-list li { border: 1px solid #24303a; border-radius: 8px; padding: 8px 10px; margin-bottom: 6px; cursor: p
ointer; }
    ul.skill-list li.active { border-color: #e8edf2; }
    .muted { color: #9aa7b4; }
  </style>
</head>
<body>
  <main>
    <h1>AZbot Local 0.2.0</h1>
    <p class="sub">Operator shell on 127.0.0.1. Does not upload. Does not hide network origin. Not Grok weights.</p>

    <div class="card">
      <p><strong>Slingshot Prep</strong> ? strip identifying embedded metadata from a copy.</p>
      <form id="prep">
        <label>Files<input type="file" name="file" multiple required /></label>
        <button type="submit">Prep copies</button>
      </form>
    </div>

    <div class="card">
      <p><strong>ARK seal</strong> ? encrypt raw file and metadata into a .ark envelope. Outer header has algorithm fi
elds only. Name, size, hash live inside encrypted meta.</p>
      <p class="warn">Phoenix is local authorized view. It re-seals gen+1 on this machine. It is not a remote payload
and it does not make an upload untraceable.</p>
      <form id="ark">
        <label>File<input type="file" name="file" required /></label>
        <label>Passphrase<input type="password" name="pass" required autocomplete="off" /></label>
        <button type="submit" data-path="/ark/seal">Seal</button>
        <button type="submit" data-path="/ark/phoenix">Phoenix view</button>
      </form>
    </div>

    <div class="card" id="skills-panel">
      <p><strong>Skills</strong> ? custom skills and Cursor SKILL.md files. These are local instructions, not Grok wei
ghts.</p>
      <label>Add skill<input type="file" id="skill-file" accept=".md,.markdown,text/markdown" /></label>
      <button type="button" id="import-skill">Add skill</button>
      <button type="button" id="export-skill">Export skill</button>
      <ul id="skill-list" class="skill-list"></ul>
      <p id="skill-desc" class="muted">Pick a skill to see what it does.</p>
    </div>

    <div class="card">
      <div>Result</div>
      <pre id="out">idle</pre>
    </div>
  </main>
</script>
  async function post(path, form) {
    const fd = new FormData(form);
    const res = await fetch(path, { method: 'POST', body: fd });
    return await res.text();
  }
  const out = document.getElementById('out');
  const descEl = document.getElementById('skill-desc');
  const listEl = document.getElementById('skill-list');
  let selected = null;

  document.getElementById('prep').addEventListener('submit', async (e) => {
    e.preventDefault();
    out.textContent = 'working';
    out.textContent = await post('/prep', e.target);
  });
  document.getElementById('ark').addEventListener('submit', async (e) => {
    e.preventDefault();
    const path = e.submitter && e.submitter.dataset.path || '/ark/seal';
    out.textContent = 'working';
    out.textContent = await post(path, e.target);
  });

```

```

async function loadSkills() {
  const skills = await fetch('/api/skills').then((r) => r.json());
  listEl.innerHTML = '';
  skills.forEach((s) => {
    const li = document.createElement('li');
    li.textContent = s.name;
    li.title = s.description;
    li.dataset.name = s.name;
    li.onclick = () => {
      selected = s;
      [...listEl.children].forEach((el) => el.classList.toggle('active', el.dataset.name === s.name));
      descEl.textContent = s.description || 'No description.';
    };
    listEl.appendChild(li);
  });
}
document.getElementById('import-skill').onclick = async () => {
  const input = document.getElementById('skill-file');
  if (!input.files || !input.files[0]) {
    out.textContent = 'Choose a SKILL.md file first.';
    return;
  }
  const fd = new FormData();
  fd.append('file', input.files[0], input.files[0].name);
  const res = await fetch('/api/skills/import', { method: 'POST', body: fd });
  out.textContent = await res.text();
  await loadSkills();
};
document.getElementById('export-skill').onclick = () => {
  if (!selected) {
    out.textContent = 'Pick a skill, then Export skill.';
    return;
  }
  const a = document.createElement('a');
  a.href = '/api/skills/' + encodeURIComponent(selected.name) + '/export';
  a.download = (selected.slug || selected.name) + '.md';
  a.click();
  out.textContent = 'Exported ' + selected.name;
};
loadSkills().catch((err) => { descEl.textContent = String(err); });
</script>
</body>
</html>

```


azbot/standalone/launch.bat

```
@echo off
cd /d "%~dp0"
if not exist out mkdir out
if not exist work mkdir work
echo AZbot Local ? 127.0.0.1:7747
echo Slingshot Prep only. No upload.
start "" "http://127.0.0.1:7747"
python app\server.py
if errorlevel 1 py -3 app\server.py
pause
```

azbot/standalone/launch.sh

```
#!/usr/bin/env bash
set -eou pipefail
DIR="$(cd "$(dirname "$0")" && pwd)"
cd "$DIR"
mkdir -p out work
echo "AZbot Local ? 127.0.0.1:7747"
echo "Slingshot Prep only. No upload."
if command -v xdg-open >/dev/null 2>&1; then
    (sleep 1; xdg-open "http://127.0.0.1:7747") >/dev/null 2>&1 &
elif command -v open >/dev/null 2>&1; then
    (sleep 1; open "http://127.0.0.1:7747") >/dev/null 2>&1 &
fi
exec python3 "$DIR/app/server.py"
```

azbot/workers/download-tracker/package.json

```
{  
  "dependencies": {  
    "wrangler": "^4.127.1"  
  }  
}
```

azbot/workers/download-tracker/src/index.js

```

import { handleRuntimeApi } from "../runtime.js";

/**
 * AZBot download tracker (Cloudflare Worker).
 *
 * GET /download?repo=AzielEliab/azbot&tag=latest&asset=...
 *      increments KV, serves the tarball via env.ASSETS.fetch
 *      (does not 302 to GitHub)
 * GET /stats JSON totals + per-repo + per-branch breakdown
 * POST /event forks report a download {owner,repo,branch,fork,asset}
 *
 * KV binding DOWNLOADS. Keys: project|owner|repo|branch|fork
 * CORS *. No secrets in this tree.
 * Isolated counter: Worker azbot-download-tracker, project azbot.
 * Not mixed with any other product.
 */

const PROJECT = "azbot";
const DEFAULT_ASSET = "azbot-0.2.0.tar.gz";
const DEFAULT_OWNER = "AzielEliab";
const DEFAULT_REPO = "azbot";
const DEFAULT_BRANCH = "main";
const HOST = "https://azbot-download-tracker.vibelock.workers.dev";
const GITHUB_REPO = "https://github.com/AzielEliab/azbot";

const GITHUB_RELEASES = "https://github.com/AzielEliab/azbot/releases";
const GITHUB_LATEST = "https://github.com/AzielEliab/azbot/releases/latest";
const INSTALL_LINE = "curl -fsSL https://azbot-download-tracker.vibelock.workers.dev/install.sh | bash";

function corsHeaders() {
    return {
        "Access-Control-Allow-Origin": "*",
        "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
        "Access-Control-Allow-Headers": "Content-Type",
    };
}

function json(body, status = 200) {
    return new Response(JSON.stringify(body, null, 2), {
        status,
        headers: { "Content-Type": "application/json; charset=utf-8", ...corsHeaders() },
    });
}

function redirect(url) {
    return new Response(null, {
        status: 302,
        headers: { Location: url, ...corsHeaders() },
    });
}

function splitOwnerRepo(value, fallbackOwner, fallbackRepo) {
    if (typeof value === "string" && value.includes("/")) {
        const [o, r] = value.split("/").filter(Boolean);
        if (o && r) return { owner: o, repo: r };
    }
    return { owner: fallbackOwner, repo: fallbackRepo };
}

function parseDims(src) {
    const get = (k) => {
        if (src == null) return null;
        if (typeof src.get === "function") {
            const v = src.get(k);
            return v == null || v === "" ? null : v;
        }
        const v = src[k];
        return v == null || v === "" ? null : v;
    };

    let owner = get("owner") || DEFAULT_OWNER;
    let repo = get("repo") || DEFAULT_REPO;
    if (typeof repo === "string" && repo.includes("/")) {
        const split = splitOwnerRepo(repo, owner, DEFAULT_REPO);
        owner = split.owner;
        repo = split.repo;
    }

    const branch = get("branch") || DEFAULT_BRANCH;
    const tag = get("tag") || "latest";
    const asset = get("asset") || "";

    const forkRaw = get("fork");
    let fork = "0";
    if (forkRaw === 1 || forkRaw === true || forkRaw === "1" || forkRaw === "true") {
        fork = "1";
    } else if (typeof forkRaw === "string" && forkRaw.includes("/")) {
        const split = splitOwnerRepo(forkRaw, owner, repo);
        owner = split.owner;
        repo = split.repo;
        fork = "1";
    } else if (forkRaw != null && forkRaw !== 0 && forkRaw !== false && forkRaw !== "0" && forkRaw !== "false") {

```

```

    fork = "1";
  }

  if (`${owner}/${repo}`.toLowerCase() !== `${DEFAULT_OWNER}/${DEFAULT_REPO}`.toLowerCase()) {
    fork = "1";
  }

  return { project: PROJECT, owner, repo, branch, fork, tag, asset };
}

function kvKey(dims) {
  return `${dims.project}|${dims.owner}|${dims.repo}|${dims.branch}|${dims.fork}`;
}

function githubAssetUrl(owner, repo, tag, asset) {
  if (!asset) {
    if (owner === DEFAULT_OWNER && repo === DEFAULT_REPO) return GITHUB_RELEASES;
    return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases`;
  }
  if (!tag || tag === "latest") {
    return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases/latest/download/${encodeURIComponent(asset)}`;
  }
  return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases/download/${encodeURIComponent(tag)}/${encodeURIComponent(asset)}`;
}

function totalKey() {
  return PROJECT + "|__total__";
}

async function increment(env, dims) {
  const key = kvKey(dims);
  const n = parseInt((await env.DOWNLOADS.get(key)) || "0", 10) + 1;
  await env.DOWNLOADS.put(key, String(n));
  const tot = parseInt((await env.DOWNLOADS.get(totalKey())) || "0", 10) + 1;
  await env.DOWNLOADS.put(totalKey(), String(tot));
  return tot;
}

async function listAllKeys(env) {
  const keys = [];
  let cursor;
  do {
    const page = await env.DOWNLOADS.list(cursor ? { cursor } : {});
    keys.push(...page.keys);
    cursor = page.list_complete ? undefined : page.cursor;
  } while (cursor);
  return keys;
}

async function collectStats(env) {
  const keys = await listAllKeys(env);
  let total = 0;
  const by_repo = {};
  const by_branch = {};
  const by_fork = { "0": 0, "1": 0 };
  const breakdown = [];

  for (const k of keys) {
    const name = k.name;
    if (name === viewsKey() || name === totalKey() || name === githubCacheKey()) continue;
    const n = parseInt((await env.DOWNLOADS.get(name)) || "0", 10);
    if (!Number.isFinite(n) || n <= 0) continue;
    const parts = name.split("|");
    if (parts.length < 5) continue;
    const [project, owner, repo, branch, fork] = parts;
    total += n;
    const repoId = `${owner}/${repo}`;
    by_repo[repoId] = (by_repo[repoId] || 0) + n;
    by_branch[branch] = (by_branch[branch] || 0) + n;
    const forkFlag = fork === "1" ? "1" : "0";
    by_fork[forkFlag] = (by_fork[forkFlag] || 0) + n;
    breakdown.push({ project, owner, repo, branch, fork: forkFlag, count: n });
  }

  const totalDirect = parseInt((await env.DOWNLOADS.get(totalKey())) || "0", 10);
  const shown = Number.isFinite(totalDirect) && totalDirect > 0 ? totalDirect : total;
  return {
    project: PROJECT,
    total: shown,
    views: parseInt((await env.DOWNLOADS.get(viewsKey())) || "0", 10) || 0,
    downloads: shown,
    by_repo,
    by_branch,
    by_fork,
    breakdown,
    github: (await githubStats(env)),
    note: "Forks identified by GitHub owner/repo. Key layout: project|owner|repo|branch|fork",
  };
}

function viewsKey() {

```

```

    return PROJECT + "|__views__";
}

function githubCacheKey() {
    return PROJECT + "|__github__";
}

async function incrementViews(env) {
    const n = parseInt((await env.DOWNLOADS.get(viewsKey())) || "0", 10) + 1;
    await env.DOWNLOADS.put(viewsKey(), String(n));
    return n;
}

async function githubStats(env) {
    const cached = await env.DOWNLOADS.get(githubCacheKey());
    if (cached) {
        try {
            const obj = JSON.parse(cached);
            if (obj && obj.fetched_at && Date.now() - obj.fetched_at < 5 * 60 * 1000) {
                return obj;
            }
        } catch {
            /* ignore */
        }
    }
    const headers = { "User-Agent": "Mozilla/5.0 AZBot-download-tracker", Accept: "application/vnd.github+json" };
    let stars = 0;
    let forks = 0;
    let watchers = 0;
    let release_download_count = 0;
    try {
        const repoRes = await fetch("https://api.github.com/repos/AzielEliab/azbot", { headers });
        if (repoRes.ok) {
            const repo = await repoRes.json();
            stars = Number(repo.stargazers_count) || 0;
            forks = Number(repo.forks_count) || 0;
            watchers = Number(repo.subscribers_count != null ? repo.subscribers_count : repo.watchers_count) || 0;
        }
        const relRes = await fetch("https://api.github.com/repos/AzielEliab/azbot/releases/latest", { headers });
        if (relRes.ok) {
            const rel = await relRes.json();
            const assets = Array.isArray(rel.assets) ? rel.assets : [];
            release_download_count = assets.reduce((s, a) => s + (Number(a.download_count) || 0), 0);
        }
    } catch {
        /* public API; empty is fine */
    }
    const out = { stars, forks, watchers, release_download_count, fetched_at: Date.now() };
    try {
        await env.DOWNLOADS.put(githubCacheKey(), JSON.stringify(out));
    } catch {
        /* ignore */
    }
    return out;
}

function installScript() {
    return `#!/usr/bin/env bash\n# AZBot one-click install. Counted download via this Worker.\nset -euo pipefail\nHOST="${HOST}"\nASSET="${DEFAULT_ASSET}"\nWORKDIR="${AZBOT_HOME:-$HOME/azbot}"\nmkdir -p "$WORKDIR"\ncd "$WORKDIR"\nnecho "Downloading counted tarball from ${HOST}/download (User-Agent Mozilla/5.0)"\ncurl -fsSL -A 'Mozilla/5.0' "${HOST}/download?asset=${ASSET}" -o "${ASSET}"\ntar -xzf "${ASSET}"\nDIR="$(find . -maxdepth 1 -type d -name 'azbot-*' | head -n 1)"\nif [ -n "$DIR" ]; then\n cd "$DIR"\nfi\npython3 -m venv .venv\n.venv/bin/activate\npython -m pip install -U pip\npython -m pip install -e .\nnecho\nnecho "Installed AZBot." \nnecho "Run: azbot ui" \nnecho "Then open htp://127.0.0.1:8870 (loopback only)" \nnecho "Author: Aziel Eliab." \n`;
}

async function serveAsset(request, env, asset, { head = false } = {}) {
    if (!env.ASSETS) {
        return json({ error: "assets binding missing" }, 500);
    }
    const assetUrl = new URL("/") + asset, request.url;
    const assetRes = await env.ASSETS.fetch(new Request(assetUrl, { method: "GET" }));
    if (!assetRes.ok) {
        return json({ error: "asset not hosted", asset, status: assetRes.status }, 404);
    }
    const headers = new Headers();
    headers.set("Content-Type", "application/gzip");
    headers.set("Content-Disposition", 'attachment; filename="' + asset.replaceAll('"', "") + '"');
    headers.set("Cache-Control", "private, no-store");
    const len = assetRes.headers.get("Content-Length");
    if (len) headers.set("Content-Length", len);
    for (const [k, v] of Object.entries(corsHeaders())) headers.set(k, v);
    if (head) {
        return new Response(null, { status: 200, headers });
    }
    return new Response(assetRes.body, { status: 200, headers });
}

async function indexHtml(env) {
    const stats = await collectStats(env);
    const downloads = Number(stats.downloads != null ? stats.downloads : stats.total) || 0;
    const views = parseInt((await env.DOWNLOADS.get(viewsKey())) || "0", 10) || 0;
    const v = views.toLocaleString("en-US");
    const n = downloads.toLocaleString("en-US");

```

```

const breakdown = (stats.breakdown || [])
  .map(
    (b) =>
      `<li><code>${b.owner}/${b.repo}</code> branch <code>${b.branch}</code> fork=${b.fork} ? ${b.count}</li>`,
  )
  .join("") || "<li>none yet</li>";
return `<!doctype html>
<html lang="en">
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>AZBot ? Aziel Eliab</title>
<meta name="description" content="Skill pack by Aziel Eliab so Grok, ChatGPT, and Venice can call public Aziel Eliab engines.">
<meta name="author" content="Aziel Eliab">
<link rel="canonical" href="https://azbot-download-tracker.vibelock.workers.dev/">
<meta property="og:title" content="AZBot ? Aziel Eliab">
<meta property="og:description" content="Skill pack by Aziel Eliab so Grok, ChatGPT, and Venice can call public Aziel Eliab engines.">
<meta property="og:url" content="https://azbot-download-tracker.vibelock.workers.dev/">
<meta property="og:type" content="website">
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "SoftwareApplication",
  "name": "AZBot",
  "author": {
    "@type": "Person",
    "name": "Aziel Eliab"
  },
  "codeRepository": "https://github.com/AzielEliab/azbot",
  "downloadUrl": "https://azbot-download-tracker.vibelock.workers.dev/download",
  "license": "https://www.apache.org/licenses/LICENSE-2.0",
  "url": "https://azbot-download-tracker.vibelock.workers.dev/",
  "description": "Skill pack by Aziel Eliab so Grok, ChatGPT, and Venice can call public Aziel Eliab engines."
}
</script>
<!-- gitbaby-seo -->
<style>
:root { color-scheme: dark; }
body { font: 16px/1.45 system-ui, sans-serif; max-width: 42rem; margin: 3rem auto; padding: 0 1.25rem 4rem; background-color: #0e1014; color: #e8eaeef; }
h1 { font-size: 1.75rem; margin: 0 0 .35rem; }
.motto { color: #9aa3b2; margin: 0 0 1.5rem; }
.card { border: 1px solid #2a3140; border-radius: 12px; padding: 1.25rem 1.35rem; background: #151922; }
.nums { display: grid; grid-template-columns: 1fr 1fr; gap: .8rem; margin: 0 0 1rem; }
.count { font-size: 2.2rem; font-variant-numeric: tabular-nums; font-weight: 700; margin: 0; }
.count span { display: block; font-size: .95rem; font-weight: 500; color: #9aa3b2; }
.kid { font-size: 1.05rem; margin: 0 0 1rem; }
.btns { display: grid; grid-template-columns: 1fr 1fr; gap: .75rem; margin: 0 0 .85rem; }
@media (max-width: 520px) { .btns { grid-template-columns: 1fr; } }
a.btn, button.btn { display: block; width: 100%; box-sizing: border-box; text-align: center; font: inherit; font-size: 1.2rem; font-weight: 750; padding: 1rem 1.1rem; border-radius: 10px; border: 0; cursor: pointer; text-decoration: none; }
a.btn.primary { background: #e8eaeef; color: #0e1014; }
button.btn.install { background: #c9a227; color: #14110a; }
button.btn.install.copied { background: #7dcf9a; color: #0e1014; }
.meta { margin-top: 1.1rem; color: #9aa3b2; font-size: .92rem; }
.meta a { color: #c9d4ff; }
.iso { margin-top: .85rem; font-size: .85rem; color: #7d8696; }
.banner { border: 1px solid #5c4a1a; background: #241c0d; color: #f0d78c; padding: .85rem 1rem; border-radius: 8px; margin: 0 0 1.2rem; font-size: .92rem; }
pre { background: #0e1014; padding: .75rem .9rem; overflow: auto; border-radius: 8px; font-size: .82rem; }
code { font-size: .88rem; }

.cite { margin-top: 1.4rem; padding-top: 1rem; border-top: 1px solid #2a3140; }
.cite h2 { font-size: 1.05rem; margin: 0 0 .4rem; }
.cite p { color: #c5ccd8; font-size: .95rem; }
.cite a { color: #c9d4ff; }
</style>
<body>
<h1>AZBot</h1>
<p class="motto">A skill for Grok, ChatGPT, and Venice. Not a model. Jeeves is not sovereign. Author Aziel Eliab.</p>
<p class="banner">AZBot is a skill, not a foundation model, not a kernel, not a VPN, and not a paid-key proxy. Jeeves is not sovereign. Call aziel-runtime. Author: Aziel Eliab.</p>
<div class="card">
  <div class="nums">
    <p class="count">${v}<span>Views</span></p>
    <p class="count">${n}<span>Downloads</span></p>
  </div>
  <p class="kid"><strong>Two big buttons.</strong> Download saves the gzip (the Downloads number goes up). One-click install copies a Terminal command. After it finishes, type <code>azbot ui</code>.</p>
  <div class="btns">
    <a class="btn primary dl" href="/download?asset=${DEFAULT_ASSET}">Download</a>
    <button type="button" class="btn install" id="install-btn">One-click install</button>
  </div>
  <pre id="install-cmd">curl -fsSL https://azbot-download-tracker.vibelock.workers.dev/install.sh | bash</pre>
  <p class="kid">Then run: <code>azbot ui</code> and open http://127.0.0.1:8870 (this computer only).</p>
  <p class="meta">The download count ticks on the Download click. The Worker serves the gzip (HTTP 200). No 302 to GitHub. Forks using this same link are counted automatically. ${DEFAULT_ASSET} ? ${n} counted.</p>
  <p class="iso">Isolated counter: Worker <code>azbot-download-tracker</code>, project <code>azbot</code>, KV <code>AZBOT_DOWNLOADS</code>. Not mixed with any other product. /v1 does not increment downloads.</p>

  <p class="meta"><a href="/stats">JSON stats</a> ? <a href="/openapi.json">OpenAPI</a> ? <a href="/v1/skill">Skill</a> ? <a href="/ai">AI runtime</a> ? <a href="${GITHUB_REPO}">GitHub</a> ? <a href="${GITHUB_LATEST}">releases</a></p>

```

```

<script>
(function () {
  var cmd = "curl -fsSL https://azbot-download-tracker.vibelock.workers.dev/install.sh | bash";
  var btn = document.getElementById("install-btn");
  var pre = document.getElementById("install-cmd");
  if (!btn) return;
  btn.addEventListener("click", function () {
    function done(ok) {
      btn.textContent = ok ? "Copied! Paste in Terminal, then run azbot ui" : "Select the command, copy it, then
run azbot ui";
      btn.classList.add("copied");
    }
    if (navigator.clipboard && navigator.clipboard.writeText) {
      navigator.clipboard.writeText(cmd).then(function () { done(true); }).catch(function () { done(false); });
    } else {
      done(false);
      if (pre && window.getSelection) {
        var r = document.createRange();
        r.selectNodeContents(pre);
        var sel = window.getSelection();
        sel.removeAllRanges();
        sel.addRange(r);
      }
    }
  });
})();
</script>
<h2>Per repo / branch / fork</h2>
<ul>${breakdown}</ul>
</div>

<section class="cite" id="cite">
<h2>How to cite</h2>
<p>Azriel Eliab. AZBot. https://github.com/AzrielEliab/azbot. https://azbot-download-tracker.vibelock.workers.dev.</p>
<p><a href="https://aziel-runtime.vibelock.workers.dev/">Catalog</a> ? <a href="https://github.com/AzrielEliab/azbot">
GitHub</a> ? <a href="https://azbot-download-tracker.vibelock.workers.dev/download">Download</a> ? <a href="https://a
zbot-download-tracker.vibelock.workers.dev/cite.json">cite.json</a></p>
</section>
<!-- /gitbaby-seo -->
</body>
</html>`;
}

```

```

export default {
  async fetch(request, env) {
    const url = new URL(request.url);

    if (request.method === "OPTIONS") {
      return new Response(null, { status: 204, headers: corsHeaders() });
    }

    const runtime = await handleRuntimeApi(request, url);
    if (runtime) return runtime;

    if ((url.pathname === "/install.sh" || url.pathname === "/install.sh/") && request.method === "GET") {
      return new Response(installScript(), {
        status: 200,
        headers: {
          "Content-Type": "text/x-shellscript; charset=utf-8",
          "Cache-Control": "private, no-store",
          ...corsHeaders(),
        },
      });
    }

    if (url.pathname === "/" && request.method === "GET") {
      await incrementViews(env);
      return new Response(await indexHtml(env), {
        headers: { "Content-Type": "text/html; charset=utf-8", ...corsHeaders() },
      });
    }

    if (url.pathname ===
"/count" && request.method === "GET") {
      const stats = await collectStats(env);
      return json({ project: PROJECT, total: stats.total || 0 });
    }

    if (url.pathname === "/stats" && request.method === "GET") {
      return json(await collectStats(env));
    }

    if (url.pathname === "/event" && request.method === "POST") {
      let body;
      try {
        body = await request.json();
      } catch {
        return json({ error: "JSON body required" }, 400);
      }
      const dims = parseDims(body || {});
      const count = await increment(env, dims);
      return json({
        ok: true,
        key: kvKey(dims),
      });
    }
  }
}

```



```

        count,
        owner: dims.owner,
        repo: dims.repo,
        branch: dims.branch,
        fork: dims.fork,
        asset: dims.asset || null,
    });
}

if (url.pathname === "/go" && (request.method === "GET" || request.method === "HEAD")) {
    const dims = parseDims(url.searchParams);
    const asset = dims.asset || DEFAULT_ASSET;
    dims.asset = asset;
    if (request.method === "GET") await increment(env, dims);
    return serveAsset(request, env, asset, { head: request.method === "HEAD" });
}

if ((url.pathname === "/download" || url.pathname.startsWith("/download/")) && (request.method === "GET" || request.method === "HEAD")) {
    const dims = parseDims(url.searchParams);
    if (!dims.asset && url.pathname.startsWith("/download/")) {
        dims.asset = decodeURIComponent(url.pathname.slice("/download/".length));
    }
    const asset = dims.asset || DEFAULT_ASSET;
    dims.asset = asset;
    if (request.method === "GET") await increment(env, dims);
    return serveAsset(request, env, asset, { head: request.method === "HEAD" });
}

// gitbaby-seo-routes
if ((url.pathname === "/robots.txt" || url.pathname === "/robots.txt/") && request.method === "GET") {
    const body = "User-agent: *\nAllow: /\nSitemap: " + HOST + "/sitemap.xml\n";
    return new Response(body, {
        status: 200,
        headers: { "Content-Type": "text/plain; charset=utf-8", ...corsHeaders() },
    });
}

if ((url.pathname === "/sitemap.xml" || url.pathname === "/sitemap.xml/") && request.method === "GET") {
    const locs = [HOST + "/", HOST + "/download", HOST + "/install.sh", HOST + "/v1/skill", HOST + "/openapi.json",
    GITHUB_REPO];
    const xml = '<?xml version="1.0" encoding="UTF-8"?><urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9"
>\n'
        + locs.map((u) => "  <url><loc>" + u + "</loc></url>").join("\n")
        + "\n</urlset>\n";
    return new Response(xml, {
        status: 200,
        headers: { "Content-Type": "application/xml; charset=utf-8", ...corsHeaders() },
    });
}

if ((url.pathname === "/cite.json" || url.pathname === "/cite.json/") && request.method === "GET") {
    return json({ "author": "Aziel Eliab", "title": "AZBot", "github": "https://github.com/AzielEliab/azbot", "download": "https://azbot-download-tracker.vibelock.workers.dev/download", "doi": null, "license": "Apache-2.0", "catalog": "https://aziel-runtime.vibelock.workers.dev/" });
}

// /gitbaby-seo-routes
return json({ error: "not found" }, 404);
},
};

```

azbot/workers/download-tracker/src/runtime.js

```

/** AZBot hosted runtime. Skill markdown only. /v1 never touches DOWNLOADS KV. */
const EXAMPLE_PAYLOAD = {};

const LIMITATION =
  "AZBot is a skill, not a foundation model, not a kernel, not a VPN, and not a paid-key proxy. Jeeves is not sovereign. Call aziel-runtime.";
const CATALOG = "https://aziel-runtime.vibelock.workers.dev";

function corsHeaders() {
  return {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type",
  };
}

function json(body, status = 200) {
  return new Response(JSON.stringify(body, null, 2), {
    status,
    headers: { "Content-Type": "application/json; charset=utf-8", ...corsHeaders() },
  });
}

function html(body) {
  return new Response(body, {
    headers: { "Content-Type": "text/html; charset=utf-8", ...corsHeaders() },
  });
}

function originOf(request) {
  try { return new URL(request.url).origin; } catch { return "https://azbot-download-tracker.vibelock.workers.dev"; }
}

const SKILL = `---\nname: AZBot\ndescription: Use this when Grok, ChatGPT, Venice, or any assistant should call Aziel Eliab public engines via the aziel-runtime OpenAPI/MCP catalog.\n---\n\n# AZBot\n\nAZBot is a **skill**. It is not a new foundation model, not a kernel, not a VPN, not a paid-key proxy, and not Jeeves-as-sovereign.\n\nOne catalog for every public Aziel Eliab engine:\n\n- Catalog: https://aziel-runtime.vibelock.workers.dev/\n- OpenAPI: https://aziel-runtime.vibelock.workers.dev/openapi.json\n- MCP: `POST https://aziel-runtime.vibelock.workers.dev/mcp`\n\nAlways send a normal `User-Agent` (for example `Mozilla/5.0`). Cloudflare Workers may 403 empty agents.\n\n## Rules\n\n1. Call the catalog. Do not invent scores, overlays, receipts, or health.\n\n2. Repeat the product banner. Never claim forensic proof, a lab spectrometer, a real UV lamp, OCR truth, a kernel, a VPN, legal advice, lie detection, call intercept, or hosted AZAI spending paid keys.\n\n3. Lamb Lens order: Peace \u2192 Clarity \u2192 Service. Jeeves is not sovereign.\n\n4. AZ-CLCE Type D is a **label only**, not a finding of malice.\n\n5. SpectralLock overlays are advisory visualization. The human still reads the page.\n\n6. ForgeReceipts is not legal advice.\n\n7. Forks are welcome and always allowed. Apache-2.0.\n\n\n## How to call\n\n``bash\ncurl -s -A 'Mozilla/5.0' https://aziel-runtime.vibelock.workers.dev/v1/health\ncurl -s -A 'Mozilla/5.0' -X POST https://aziel-runtime.vibelock.workers.dev/p/azclce/score \\\n -H 'content-type: application/json' \\\n -d '{\"r\": \"login button blue\", \"d\": \"login form submits\", \"p\": \"login button submits\"}'\n``\n\nMCP tools are named `{slug}_{op}` (example `azclce_score`, `spectrallock_overlay`, `azai_lamb_check`).\n\nGrok: import the OpenAPI as a custom tool. ChatGPT: GPT Actions. Venice: HTTP tools.\n\n## Products (honest one-liners)\n\nslug | name | do not claim | \n|-----|-----|-----| \n| vibelock | VibeLock | courtroom audio proof | veillock | VeilLock | FaceTime/Zoom intercept | \n| codelock | CodeLock | that meaning changed | \n| godlock | GodLock | a VPN or ghost net | \n| shadowlock | ShadowLock | OS hooks / process intercept | \n| temporallock | TemporalLock | legal chain of custody | \n| forgereceipts | ForgeReceipts | legal advice / court filing | \n| decisiongate | DecisionGATE | moral authority | \n| zsolver | ZionPattern Solver | a solved case; cap is 75% | \n| azos | AZ-OS | a kernel or remote shell | \n| glossafilter | Glossa Filter | that tools hold opinions | \n| miragegrid | MirageGrid | anonymity / VPN | \n| staticclock | StaticClock | a scheduler you set | \n| chronolock | ChronoLock | targeting or virality | \n| postking | Post-King Chess | that the goal is to win | \n| azclce | AZ-CLCE | intent or malice | \n| ark | The ARK | a kernel; hosted unlock | \n| azai | AZAI | a new model; hosted paid proxy | \n| spectrallock | SpectralLock | spectrometer / forensic ink | \n\n## Catalog + local UI\n\nAuthor: **Aziel Eliab**. Honest scope: Skill, not a foundation model. Hosted /v1/skill returns markdown. Jeeves is not sovereign.\n\n- Catalog product: https://aziel-runtime.vibelock.workers.dev/p/azbot/\n- Catalog OpenAPI: https://aziel-runtime.vibelock.workers.dev/openapi.json\n- Catalog MCP: `POST https://aziel-runtime.vibelock.workers.dev/mcp`\n\nThis Worker skill: `GET https://azbot-download-tracker.vibelock.workers.dev/v1/skill`\n\nThis Worker OpenAPI: https://azbot-download-tracker.vibelock.workers.dev/openapi.json\n\nSample payload: `GET https://azbot-download-tracker.vibelock.workers.dev/v1/example`\n\nLocal UI: **Import JSON file** (`type=file`) and **Export JSON**.* Then `azbot doctor`.\n\nGrok: import catalog or Worker OpenAPI as a custom tool. ChatGPT: GPT Actions. Venice: HTTP tools.\n`;

export async function handleRuntimeApi(request, url) {
  const path = url.pathname.replace(/\/+$/, "") || "/";
  if (path === "/v1/health" && request.method === "GET") {
    return json({
      ok: true, author: "Aziel Eliab",
      product: "azbot",
      version: "0.2.0",
      runtime: true,
      kv_increment: false,
      is_skill: true,
      not_a_model: true,
      limitation: LIMITATION,
      catalog: CATALOG,
    });
  }
  if ((path === "/v1/example" || path === "/v1/example/") && (request.method === "GET" || request.method === "HEAD")) {
    return json({
      ok: true,
      product: "azbot",
      author: "Aziel Eliab",
      example: EXAMPLE_PAYLOAD,
      note: "Sample payload only. Does not increment downloads.",
    });
  }
}

```

```

    if (path === "/v1/skill" && request.method === "GET") {
      return new Response(SKILL, {
        status: 200,
        headers: {
          "Content-Type": "text/markdown; charset=utf-8",
          "Cache-Control": "private, no-store",
          ...corsHeaders(),
        },
      });
    }
  }
  if (path === "/openapi.json" && request.method === "GET") {
    const origin = originOf(request);
    return json({
      openapi: "3.1.0",
      info: { title: "AZBot", version: "0.2.0", description: LIMITATION },
      paths: {
        "/v1/example": { get: { operationId: "azbotExample", summary: "Sample JSON payload. Does not increment d
        ownloads.", responses: { "200": { description: "OK" } } } },
        "/v1/health": { get: { operationId: "azbot_health", summary: "Liveness. Does not increment download KV." } },
        "/v1/skill": { get: { operationId: "azbot_skill", summary: "Return AZBot skill markdown. Does not increment do
        wnload KV." } },
      },
      servers: [{ url: origin }],
    });
  }
  if ((path === "/ai" || url.pathname === "/ai/") && request.method === "GET") {
    const origin = originOf(request);
    return html(`<!doctype html><meta charset=utf-8><title>AZBot AI</title>
<body style="font:16px/1.45 system-ui;max-width:44rem;margin:3rem auto;background:#0e1014;color:#e8eaeef">
<h1>AZBot runtime</h1>
<p>${LIMITATION}</p>
<p>OpenAPI: <a href="${origin}/openapi.json" style="color:#c9d4ff">${origin}/openapi.json</a></p>
<pre>curl -A Mozilla/5.0 ${origin}/v1/health
curl -A Mozilla/5.0 ${origin}/v1/skill</pre>
<p>Catalog: <a href="${origin}/" style="color:#c9d4ff">${origin}</a></p>
</body>`);
  }
  if (path.startsWith("/v1/") || path === "/v1") {
    return json({ error: "not found", hint: "GET /v1/health /v1/skill", limitation: LIMITATION }, 404);
  }
  return null;
}

```

azbot/workers/download-tracker/wrangler.toml

```
name = "azbot-download-tracker"
main = "src/index.js"
compatibility_date = "2026-07-01"
workers_dev = true
account_id = "ac575a9b822bea2bed97d0ab73aed238"

[[kv_namespaces]]
binding = "DOWNLOADS"
id = "1d49ab4d6c5b437e84449503f9cae41c"

[assets]
directory = "./public"
binding = "ASSETS"
run_worker_first = ["/", "/install.sh", "/install.sh/", "/download", "/download/*", "/stats", "/event", "/go", "/count", "/v1", "/v1/*", "/openapi.json", "/ai", "/ai/*", "/mcp", "/mcp/", "/robots.txt", "/sitemap.xml", "/cite.json"]
```

aziel-runtime/package.json

```
{
  "name": "aziel-runtime",
  "private": true,
  "description": "One OpenAPI + MCP URL for every AzielEliab product runtime",
  "license": "Apache-2.0",
  "dependencies": {
    "wrangler": "^4.127.1"
  }
}
```

aziel-runtime/src/index.js

```

/**
 * aziel-runtime ? one URL for every Aziel Eliab product runtime.
 *
 * GET / HTML catalog (indexable)
 * GET /robots.txt Allow /, sitemap URL
 * GET /sitemap.xml catalog, OpenAPI, product cards/health, GitHub
 * GET /llms.txt plain-text catalog for LLM crawlers
 * GET /ai.txt same as /llms.txt
 * GET /cite.json How-to-cite: Aziel Eliab, Apache-2.0, GitHub + DOI
 * GET /v1/catalog.json machine-readable full product list
 * GET /openapi.json combined OpenAPI 3.1 (paths /p/{product}/{op})
 * GET /p/{product} indexable product card
 * GET /p/{product}/{op} proxy GET
 * POST /p/{product}/{op} proxy ? https://{worker}.vibelock.workers.dev/v1/{op}
 * GET /v1/health
 * POST /mcp JSON-RPC MCP-over-HTTP (initialize, tools/list, tools/call)
 *
 * No download-KV increment on catalog routes. CORS *. Apache-2.0. Forks welcome.
 * Author: Aziel Eliab. Identity is Aziel Eliab only.
 */
const CATALOG_HOST = "https://aziel-runtime.vibelock.workers.dev";
const PROTOCOL = "2025-03-26";
const CATALOG_TITLE = "Aziel Eliab product runtime catalog";
const CATALOG_DESCRIPTION =
  "One URL for every Aziel Eliab product runtime. OpenAPI, MCP tools, counted downloads, install.sh, skill markdown, GitHub, and DOI citations. Apache-2.0. Author: Aziel Eliab.";
const LASTMOD = "2026-09-02";

const PRODUCTS_RAW = [
  {
    slug: "vibelock",
    name: "VibeLock",
    worker: "vibelock-download-tracker",
    github: "https://github.com/AzielEliab/vibelock",
    ops: [{ op: "analyze", method: "POST", summary: "Audio forensic risk assessment (JSON metrics)."}],
    example: { summary: "Synthetic dual-channel notes; local WAV scoring is in the Python package" },
    banner: null,
  },
  {
    slug: "veillock",
    name: "VeilLock",
    worker: "veillock-download-tracker",
    github: "https://github.com/AzielEliab/veillock",
    ops: [{ op: "apps", method: "POST", summary: "Local-app steps. YOUR camera/screen only." }],
    example: { app: "zoom" },
    banner: "VeilLock does not inject into FaceTime, Zoom, Meet, Teams, or Skype. YOUR camera/screen only. Not a call interceptor.",
  },
  {
    slug: "codelock",
    name: "CodeLock",
    worker: "codelock-download-tracker",
    github: "https://github.com/AzielEliab/codelock",
    ops: [{ op: "render", method: "POST", summary: "Canonical or CodeLock/Rosetta HTML view of source." }],
    example: { source: "print('hello')", mode: "normalize" },
    banner: "This tool alters perception, not meaning.",
  },
  {
    slug: "godlock",
    name: "GodLock",
    worker: "godlock-download-tracker",
    github: "https://github.com/AzielEliab/godlock",
    ops: [
      { op: "score", method: "POST", summary: "Offline ABAD / hardening score for a text." },
      { op: "submit", method: "POST", summary: "Submit text; returns a receipt id (no VPN)."}
    ],
    example: { text: "ABAD does not layer on phi." },
    banner: "GodLock is not a VPN and not an anonymity network.",
  },
  {
    slug: "shadowlock",
    name: "ShadowLock",
    worker: "shadowlock-download-tracker",
    github: "https://github.com/AzielEliab/shadowlock",
    ops: [{ op: "observe", method: "POST", summary: "Zero-retention observation of a job list you already have." }],
    example: { jobs: [{ id: "a", status: "ok" }] },
    banner: "ShadowLock is a gate on an outcome you already have. No OS hook. No process intercept.",
  },
  {
    slug: "temporallock",
    name: "TemporalLock",
    worker: "temporallock-download-tracker",
    github: "https://github.com/AzielEliab/temporallock",
    ops: [
      { op: "genesis", method: "POST", summary: "First receipt of a chain (explicit genesis)."},
      { op: "append", method: "POST", summary: "Append a receipt to an existing chain payload." },
      { op: "verify", method: "POST", summary: "Recompute hashes. Anyone can verify." }
    ],
    example: { summary: "sky was overcast", evidence: "photo:./sky.jpg", confidence: 0.9 },
    banner: null,
  },
]

```

```

    slug: "forgereceipts",
    name: "ForgeReceipts",
    worker: "forgereceipts-download-tracker",
    github: "https://github.com/AzielEliab/forgereceipts",
    ops: [{ op: "receipt", method: "POST", summary: "Local receipt / checklist helper. No court connection." }],
    example: { summary: "filed locally", evidence: "sha256:?" },
    banner: "ForgeReceipts is not legal advice. It does not contact courts, Odyssey, email, or any cloud filing service.",
  },
  {
    slug: "decisiongate",
    name: "DecisionGATE",
    worker: "decisiongate-download-tracker",
    github: "https://github.com/AzielEliab/decisiongate",
    ops: [{ op: "check", method: "POST", summary: "Run the five sequential gates on a proposal." }],
    example: {
      statement: "Release the catalog Worker this week.",
      evidence: ["OpenAPI 3.1 combined spec."],
      impact_pos: ["One URL for GPT Actions."],
      impact_neg: ["A vague draft takes longer."],
      values: ["Clarity without force"],
      accountable: "Aziel Eliab",
    },
    banner: "Freedom without clarity is chaos.",
  },
  {
    slug: "zsolver",
    name: "ZionPattern Solver",
    worker: "zsolver-download-tracker",
    github: "https://github.com/AzielEliab/zion-pattern-solver",
    ops: [
      { op: "patterns", method: "GET", summary: "Nine ontology nodes (Zioncheck seed). Not a verdict." },
      { op: "score", method: "POST", summary: "Score answers. Hard 75% cap, 25% floor." },
      { op: "session", method: "POST", summary: "Stateless session snapshot from answers." },
    ],
    example: { answers: [{ pattern_id: "P1", value: "yes" }, { pattern_id: "P2", value: "unknown" }] },
    banner: "Hard 75% confidence cap / 25% uncertainty floor. Provisional and assistive. Does not solve Zioncheck or any case.",
  },
  {
    slug: "azos",
    name: "AZ-OS",
    worker: "azos-download-tracker",
    github: "https://github.com/AzielEliab/azos",
    ops: [{ op: "status", method: "POST", summary: "Read-only status / principles. No remote exec." }],
    example: {},
    banner: "AZ-OS does not grant remote shell. Invite prints principles; exec requires a local token.",
  },
  {
    slug: "glossafilter",
    name: "Glossa Filter",
    worker: "glossafilter-download-tracker",
    github: "https://github.com/AzielEliab/glossafilter",
    ops: [{ op: "render", method: "POST", summary: "Render an intent across bundled peer ids." }],
    example: { subject: "package", rel: "release", object: "filter", channel: "tooling" },
    banner: "Human opinion remains human, and tools remain tools.",
  },
  {
    slug: "miragegrid",
    name: "MirageGrid",
    worker: "miragegrid-download-tracker",
    github: "https://github.com/AzielEliab/miragegrid",
    ops: [{ op: "assign", method: "POST", summary: "Assign a session node id. Mapping is ephemeral." }],
    example: {},
    banner: "MirageGrid is not a VPN and not an anonymity network. It does not guarantee anonymity against global surveillance.",
  },
  {
    slug: "staticclock",
    name: "StaticClock",
    worker: "staticclock-download-tracker",
    github: "https://github.com/AzielEliab/staticclock",
    ops: [{ op: "advise", method: "POST", summary: "Five advisory fields for a geo. Not a scheduler." }],
    example: { geo: "Indiana" },
    banner: "StaticClock is not a scheduler and not a clock you set. Advisory fields only.",
  },
  {
    slug: "chronolock",
    name: "ChronoLock",
    worker: "chronolock-download-tracker",
    github: "https://github.com/AzielEliab/chronolock",
    ops: [
      { op: "advisory", method: "POST", summary: "One advisory for a last-known geo. Not a scheduler." },
      { op: "anchors", method: "GET", summary: "List Top-30 geographic anchors." },
    ],
    example: { geo: "Indiana" },
    banner: "ChronoLock is advisory only ? not a scheduler, not targeting, not virality. Temporal Neutral Window 08:30-10:30 local. Distinct from TemporalLock.",
  },
  {
    slug: "postking",
    name: "Post-King Chess",
    worker: "postking-download-tracker",
    github: "https://github.com/AzielEliab/postking-chess",
    ops: [

```

```

    { op: "new", method: "POST", summary: "Start a game {difficulty, seed}." },
    { op: "move", method: "POST", summary: "Human UCI move + AI 1-ply continuity reply." },
    { op: "status", method: "POST", summary: "Continuity status for a FEN/state." },
  ],
  example: { difficulty: "steward", seed: 1 },
  banner: "The goal is not to win. The goal is to remain. Human is king-bound; AI has a Node, not a king.",
},
{
  slug: "azclce",
  name: "AZ-CLCE",
  worker: "azclce-download-tracker",
  github: "https://github.com/AzielEliab/az-clce",
  ops: [
    { op: "score", method: "POST", summary: "Jaccard triple / pairwise / CLCE+." },
    { op: "classify", method: "POST", summary: "Mismatch types. Type D is a label only." },
    { op: "gate", method: "POST", summary: "Pass iff triple ? min (default 0.7). Advisory." },
  ],
  example: { r: "login button blue", d: "login form submits", p: "login button submits" },
  banner: "CLCE detects inconsistency, not intent. Type D is a label, not a finding of malice. Threshold 0.7 is advisory.",
},
{
  slug: "ark",
  name: "The ARK",
  worker: "ark-download-tracker",
  github: "https://github.com/AzielEliab/ark",
  ops: [
    { op: "sweep", method: "POST", summary: "Mode E heuristics only (PE/ELF/Mach-0, powershell -enc, curl|sh). No clamscan. Payload is not stored." },
    { op: "levels", method: "GET", summary: "Auto-lock seconds and decoy counts. Behavior, not cryptography." },
  ],
  example: { text: "hello world" },
  banner: "The ARK is not a kernel. Hosted API never unlocks or encrypts with a passphrase and never stores vaults. Sweep is Mode E heuristics only.",
},
{
  slug: "azai",
  name: "AZAI",
  worker: "azai-download-tracker",
  github: "https://github.com/AzielEliab/azai",
  ops: [
    { op: "health", method: "GET", summary: "Liveness. Protocol mirror. Not a provider proxy." },
    { op: "lamb-check", method: "POST", summary: "Run Lamb Lens (peace/clarity/service) on {text}. No provider call." },
  ],
},
{
  slug: "spectrallock",
  name: "SpectralLock",
  worker: "spectrallock-download-tracker",
  github: "https://github.com/AzielEliab/spectrallock",
  ops: [
    { op: "health", method: "GET", summary: "Liveness. Does not increment download KV." },
    { op: "modes", method: "GET", summary: "List live overlay modes (zero, tazel, vryn, uv, rosetta, zen, chaos, balance)."},
    { op: "overlay", method: "POST", summary: "Simplified overlay preview. PNG b64 in, longest side capped at 256 px. Not the full Python pipeline." },
  ],
  example: { mode: "rosetta", b64: "<png-base64>" },
  banner: "Hosted overlay is a 256px preview, not a spectrometer, not forensic.",
},
{
  slug: "azbot",
  name: "AZBot",
  worker: "azbot-download-tracker",
  github: "https://github.com/AzielEliab/azbot",
  ops: [
    { op: "health", method: "GET", summary: "Liveness. Skill, not a model." },
    { op: "skill", method: "GET", summary: "Return AZBot skill markdown. Does not increment download KV." },
  ],
  example: {},
  banner: "AZBot is a skill, not a foundation model. Hosted /v1/skill returns markdown. Call aziel-runtime for the engines. Jeeves is not sovereign.",
},
{
  slug: "employeeelock",
  name: "EmployeeLock",
  worker: "employeeelock-download-tracker",
  github: "https://github.com/AzielEliab/employeeelock",
  ops: [
    { op: "health", method: "GET", summary: "Liveness. Does not increment download KV. Hosted never stores xlsx." },
    { op: "append-preview", method: "POST", summary: "Hash a proposed LOG row without writing a file. Hosted never stores xlsx." },
    { op: "verify-canonical", method: "POST", summary: "Recompute SHA-256 of posted canonical JSON. Not a truth score." },
    { op: "skill", method: "GET", summary: "Return EmployeeLock skill markdown. Does not increment download KV." },
  ],
  example: { event: "process outcome recorded with no named owner", result: "row logged as format proof", owner_name: "", confidence: 0.7 },
  banner: "EmployeeLock is a hash-chained accountability workbook. Not a court, not UL, not a truth score. Hosted never stores xlsx. Demo rows are format proof, not case facts.",
}

```



```

    },
    {
      slug: "foldlock",
      name: "FoldLock",
      worker: "foldlock-download-tracker",
      github: "https://github.com/AzielEliab/foldlock",
      ops: [
        { op: "health", method: "GET", summary: "Liveness. Does not increment download KV. Not zip." },
        { op: "fold-preview", method: "POST", summary: "Small UTF-8 text in, receipt + FLD3 base64 out. Cap ~8KB. Not zip." },
        { op: "unfold-preview", method: "POST", summary: "FLD3 base64 in, verified restore or error. Not zip." },
        { op: "skill", method: "GET", summary: "Return FoldLock skill markdown. Does not increment download KV." },
      ],
      example: { text: "the cat and the dog" },
      banner: "FoldLock is algorithmic tether-word suppression on UTF-8 text. Not zip. Hosted preview is not a general compressor. Ratios are receipts, not trophies. Short strings can grow.",
    },
    {
      slug: "whistlelock",
      name: "WhistleLock",
      worker: "whistlelock-download-tracker",
      github: "https://github.com/AzielEliab/whistlelock",
      ops: [
        { op: "health", method: "GET", summary: "Liveness. Does not increment download KV. Hosted never holds whistle files." },
        { op: "hash-preview", method: "POST", summary: "SHA-256 of posted bytes. Not stored. Hosted never holds whistle files." },
        { op: "canon-preview", method: "POST", summary: "Hash a proposed ledger row. Not stored. Not a mailer." },
        { op: "skill", method: "GET", summary: "Return WhistleLock skill markdown. Does not increment download KV." },
      ],
      example: { summary: "sample drop", kind: "drop" },
      banner: "WhistleLock is a local drop ledger + dead-man copy. Not a mailer. Hosted never holds whistle files. The operator moves released packets.",
    },
    {
      slug: "trajectorylock",
      name: "TrajectoryLock",
      worker: "trajectorylock-download-tracker",
      github: "https://github.com/AzielEliab/trajectorylock",
      ops: [
        { op: "health", method: "GET", summary: "Liveness. Does not increment download KV. Hosted never stores media. Not a certified instrument." },
        { op: "example", method: "GET", summary: "Synthetic small JSON case. Not a real case. Does not increment download KV." },
        { op: "analyze", method: "POST", summary: "Small JSON case in, geometric result out. Cap size. NEVER store media. Not a certified forensic instrument." },
        { op: "skill", method: "GET", summary: "Return TrajectoryLock skill markdown. Does not increment download KV." },
      ],
      example: {
        case_id: "MINIMAL-DIRECT-LINE",
        sources: [{ id: "survey-a", quality: 0.95, calibrated: true, independence_group: "survey-a" }],
        observations: [{ type: "direct_line", source_id: "survey-a", point: [0, 0, 1.2], direction: [1, 0.1, 0.02], angular_sigma_deg: 0.5, offset_sigma_m: 0.02 }],
        official_hypothesis: { point: [0.01, 0.01, 1.19], direction: [1, 0.11, 0.02], angular_sigma_deg: 0.7, offset_sigma_m: 0.04, angle_tolerance_deg: 3.0, offset_tolerance_m: 0.25 },
      },
      banner: "TrajectoryLock is a research prototype / auditable geometric test. Not a certified forensic instrument. Hosted never stores media. Match probability is P(match | declared model), not P(official account is true). Synthetic example results must never be represented as real-case findings.",
    },
  ],
};

const ONE_LINE = {
  vibelock: "Physical-consistency evaluation of speech audio. Risk assessment, not a liveness proof.",
  veillock: "Local camera/screen steps for YOUR device only. Not a call interceptor.",
  codeclock: "Canonical or Rosetta HTML view of source. Alters perception, not meaning.",
  godlock: "Offline ABAD / hardening score. Not a VPN and not an anonymity network.",
  shadowlock: "Zero-retention observation of a job list you already have. No OS hook.",
  temporallock: "Hash-chained receipts anyone can verify. Explicit genesis, append, verify.",
  forgereceipts: "Local receipt / checklist helper. Not legal advice. Does not contact courts.",
  decisiongate: "Five sequential gates on a proposal. Freedom without clarity is chaos.",
  zsolver: "Nine ontology nodes (Zioncheck seed). Hard 75% cap. Does not solve cases.",
  azos: "Read-only status / principles. Does not grant remote shell.",
  glossafilter: "Render an intent across bundled peer ids. Human opinion remains human.",
  miragegrid: "Ephemeral session node assignment. Not a VPN and not an anonymity network.",
  staticclock: "Five advisory fields for a geo. Not a scheduler.",
  chronolock: "Advisory temporal window 08:30?10:30 local. Distinct from TemporalLock.",
  postking: "Continuity chess. The goal is not to win. The goal is to remain.",
  azclce: "Jaccard triple / pairwise / CLCE+. Detects inconsistency, not intent.",
  ark: "Mode E heuristics sweep. Not a kernel. Hosted never unlocks or stores vaults.",
  azai: "Local OpenAI-compatible runtime. Not a new foundation model. Jeeves is not sovereign.",
  spectrallock: "Overlay preview modes. 256px hosted preview, not a spectrometer.",
  azbot: "Skill, not a foundation model. Hosted /v1/skill returns markdown.",
  employeeelock: "Hash-chained accountability workbook. Not a court, not UL, not a truth score.",
  foldlock: "Algorithmic tether-word suppression on UTF-8 text. Not zip.",
  whistlelock: "Local drop ledger + dead-man copy. Not a mailer.",
  trajectorylock: "Auditable geometric test. Research prototype, not a certified forensic instrument.",
};

/** Known Zenodo DOIs only. Do not invent. Cite even if the record currently 410s. */
const DOI_BY_SLUG = {
  vibelock: "10.5281/zenodo.21431610",

```

```

    veillock: "10.5281/zenodo.21431659",
    codelock: "10.5281/zenodo.21431561",
    shadowlock: "10.5281/zenodo.21435707",
    temporallock: "10.5281/zenodo.21431405",
    forgereceipts: "10.5281/zenodo.21436074",
    decisiongate: "10.5281/zenodo.21435730",
    zsolver: "10.5281/zenodo.21436155",
    azos: "10.5281/zenodo.21431711",
    postking: "10.5281/zenodo.21897338",
    ark: "10.5281/zenodo.21435810",
    employeeelock: "10.5281/zenodo.22257493",
    foldlock: "10.5281/zenodo.22257762",
    whistlelock: "10.5281/zenodo.22257762",
    trajectorylock: "10.5281/zenodo.22258015",
  };

function ensureCatalogOps(p) {
  const have = new Set((p.ops || []).map((o) => o.op));
  const ops = [];
  if (!have.has("health")) {
    ops.push({
      op: "health",
      method: "GET",
      summary: "Liveness. Does not increment download KV.",
    });
  }
  ops.push(...(p.ops || []));
  if (!have.has("skill")) {
    ops.push({
      op: "skill",
      method: "GET",
      summary: `Return ${p.name} skill markdown. Does not increment download KV.`,
    });
  }
  return ops;
}

export const PRODUCTS = PRODUCTS_RAW.map((p) => ({
  ...p,
  oneLine: ONE_LINE[p.slug] || p.name,
  doi: DOI_BY_SLUG[p.slug] || null,
  banner: p.banner || ONE_LINE[p.slug] || p.name,
  ops: ensureCatalogOps(p),
})));

const BY_SLUG = Object.fromEntries(PRODUCTS.map((p) => [p.slug, p]));

function corsHeaders() {
  return {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type, Accept, MCP-Protocol-Version, mcp-session-id",
  };
}

function linkHeaders(origin, canonicalPath = "/") {
  const base = origin.replace(/\/$/, "") || CATALOG_HOST;
  const canonical = base + (canonicalPath.startsWith("/") ? canonicalPath : "/" + canonicalPath);
  return {
    "X-Robots-Tag": "index, follow, max-snippet:-1, max-image-preview:large",
    Link: `<${canonical}>; rel="canonical", <${base}/openapi.json>; rel="service-doc", <${base}/sitemap.xml>; rel="describedby"`,
  };
}

function json(body, status = 200, extra = {}) {
  return new Response(JSON.stringify(body, null, 2), {
    status,
    headers: {
      "Content-Type": "application/json; charset=utf-8",
      ...corsHeaders(),
      "X-Robots-Tag": "index, follow, max-snippet:-1, max-image-preview:large",
      ...extra,
    },
  });
}

function html(body, extra = {}) {
  return new Response(body, {
    headers: {
      "Content-Type": "text/html; charset=utf-8",
      ...corsHeaders(),
      "X-Robots-Tag": "index, follow, max-snippet:-1, max-image-preview:large",
      ...extra,
    },
  });
}

function text(body, extra = {}) {
  return new Response(body, {
    status: 200,
    headers: {
      "Content-Type": "text/plain; charset=utf-8",
      ...corsHeaders(),
      "X-Robots-Tag": "index, follow, max-snippet:-1, max-image-preview:large",
    },
  });
}

```

```

    ...extra,
  },
});
}

function xml(body, extra = {}) {
  return new Response(body, {
    status: 200,
    headers: {
      "Content-Type": "application/xml; charset=utf-8",
      ...corsHeaders(),
      "X-Robots-Tag": "index, follow, max-snippet:-1, max-image-preview:large",
      ...extra,
    },
  });
}

function originOf(request) {
  try {
    return new URL(request.url).origin;
  } catch {
    return CATALOG_HOST;
  }
}

function workerHost(product) {
  return `https://${product.worker}.vibelock.workers.dev`;
}

function workerUrl(product, op) {
  return `${workerHost(product)}/v1/${op}`;
}

function productUrls(product, origin) {
  const host = workerHost(product);
  const base = (origin || CATALOG_HOST).replace(/\$/ /, "");
  return {
    github: product.github,
    worker: product.worker,
    worker_home: host + "/",
    download: host + "/download",
    install: host + "/install.sh",
    skill: host + "/v1/skill",
    openapi: host + "/openapi.json",
    stats: host + "/stats",
    catalog_card: `${base}/p/${product.slug}`,
    catalog_health: `${base}/p/${product.slug}/health`,
    catalog_skill: `${base}/p/${product.slug}/skill`,
    catalog_openapi: `${base}/openapi.json`,
    doi: product.doi || null,
    doi_url: product.doi ? `https://doi.org/${product.doi}` : null,
  };
}

function catalogRecord(product, origin) {
  const urls = productUrls(product, origin);
  return {
    slug: product.slug,
    name: product.name,
    one_line: product.oneLine,
    github: urls.github,
    worker: urls.worker,
    download: urls.download,
    install: urls.install,
    skill: urls.skill,
    openapi: urls.openapi,
    doi: urls.doi,
    doi_url: urls.doi_url,
    banner: product.banner,
    ops: product.ops,
    catalog_card: urls.catalog_card,
    catalog_health: urls.catalog_health,
    catalog_skill: urls.catalog_skill,
  };
}

function robotsTxt(origin) {
  const base = origin.replace(/\$/ /, "");
  return [
    "User-agent: **",
    "Allow: /",
    "",
    "User-agent: GPTBot",
    "Allow: /",
    "User-agent: ChatGPT-User",
    "Allow: /",
    "User-agent: Google-Extended",
    "Allow: /",
    "User-agent: anthropic-ai",
    "Allow: /",
    "User-agent: ClaudeBot",
    "Allow: /",
    "User-agent: PerplexityBot",
    "Allow: /",
    "User-agent: Bytespider",
  ];
}

```

```

    "Allow: /",
    "User-agent: CCBot",
    "Allow: /",
    "",
    `Sitemap: ${base}/sitemap.xml`,
    "",
    ""
  ].join("\n");
}

function sitemapXml(origin) {
  const base = origin.replace(/\/$/, "");
  const urls = [
    { loc: base + "/", priority: "1.0", changefreq: "daily" },
    { loc: base + "/openapi.json", priority: "0.9", changefreq: "daily" },
    { loc: base + "/v1/catalog.json", priority: "0.9", changefreq: "daily" },
    { loc: base + "/cite.json", priority: "0.8", changefreq: "weekly" },
    { loc: base + "/llms.txt", priority: "0.8", changefreq: "weekly" },
    { loc: base + "/ai.txt", priority: "0.8", changefreq: "weekly" },
    { loc: base + "/v1/health", priority: "0.5", changefreq: "daily" },
    { loc: base + "/mcp", priority: "0.6", changefreq: "weekly" },
  ];
  for (const p of PRODUCTS) {
    urls.push({ loc: `${base}/#${p.slug}`, priority: "0.8", changefreq: "weekly" });
    urls.push({ loc: `${base}/p/${p.slug}`, priority: "0.8", changefreq: "weekly" });
    urls.push({ loc: `${base}/p/${p.slug}/health`, priority: "0.7", changefreq: "daily" });
    urls.push({ loc: p.github, priority: "0.5", changefreq: "weekly", lastmod: null });
  }
  const body = urls
    .map((u) => {
      const last = u.lastmod === null ? "" : `<lastmod>${u.lastmod || LASTMOD}</lastmod>\n`;
      return `<url>\n  <loc>${escapeXml(u.loc)}</loc>\n${last}  <changefreq>${u.changefreq}</changefreq>\n  <p
priority>${u.priority}</priority>\n  </url>`;
    })
    .join("\n");
  return `<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
${body}
</urlset>
`;
}

function escapeXml(s) {
  return String(s)
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;");
}

function llmsTxt(origin) {
  const base = origin.replace(/\/$/, "");
  const lines = [
    `# ${CATALOG_TITLE}`,
    "",
    `> ${CATALOG_DESCRIPTION}`,
    "",
    `Author: Aziel Eliab`,
    `Catalog: ${base}/`,
    `OpenAPI: ${base}/openapi.json`,
    `MCP: POST ${base}/mcp`,
    `Machine catalog: ${base}/v1/catalog.json`,
    `Cite: ${base}/cite.json`,
    `License: Apache-2.0`,
    "",
    `### Products`,
    "",
    ""
  ];
  for (const p of PRODUCTS) {
    const u = productUrls(p, origin);
    lines.push(`### ${p.name} (${p.slug})`);
    lines.push(p.oneLine);
    if (p.banner) lines.push(`Banner: ${p.banner}`);
    lines.push(`GitHub: ${u.github}`);
    lines.push(`Worker: ${u.worker_home}`);
    lines.push(`Download (counted, gzip 200): ${u.download}`);
    lines.push(`Install: curl -fsSL ${u.install} | bash`);
    lines.push(`Skill: ${u.skill}`);
    lines.push(`Catalog health: ${u.catalog_health}`);
    lines.push(`Catalog skill: GET ${base}/p/${p.slug}/skill`);
    lines.push(`Ops: ${p.ops.map((o) => `${o.method} /p/${p.slug}/${o.op}`).join(", ")}`);
    if (u.doi_url) lines.push(`DOI: ${u.doi}  ${u.doi_url}`);
    lines.push("");
  }
  lines.push(`### How to cite`);
  lines.push(`Eliab, Aziel. (2026). Aziel Eliab product runtime catalog [Software]. Apache-2.0. " + base + "/"`);
  lines.push("");
  return lines.join("\n");
}

function citeJson(origin) {
  const base = origin.replace(/\/$/, "");
  return {
    author: "Aziel Eliab",
    author_github: "https://github.com/AzielEliab",
    catalog: base + "/",
  };
}

```

```

    license: "Apache-2.0",
    license_url: "https://www.apache.org/licenses/LICENSE-2.0",
    how_to_cite:
      "Eliab, Aziel. (2026). Azie
1 Eliab product runtime catalog [Software]. Apache-2.0. " +
    base +
    "/",
    bibtex: `@software{eliab_azel_runtime_2026,
author = {Eliab, Aziel},
title = {Aziel Eliab product runtime catalog},
year = {2026},
url = {${base}/},
license = {Apache-2.0}
}`,
    apa: `Eliab, A. (2026). Aziel Eliab product runtime catalog [Computer software]. ${base}/`,
    products: PRODUCTS.map((p) => ({
      name: p.name,
      slug: p.slug,
      github: p.github,
      doi: p.doi || null,
      doi_url: p.doi ? `https://doi.org/${p.doi}` : null,
    })),
  });
}

function jsonLd(origin) {
  const base = origin.replace(/\/$/, "");
  const software = {
    "@type": "SoftwareApplication",
    name: CATALOG_TITLE,
    url: base + "/",
    description: CATALOG_DESCRIPTION,
    applicationCategory: "DeveloperApplication",
    operatingSystem: "Cloudflare Workers",
    license: "https://www.apache.org/licenses/LICENSE-2.0",
    author: {
      "@type": "Person",
      name: "Aziel Eliab",
      url: "https://github.com/AzielEliab",
    },
    codeRepository: "https://github.com/AzielEliab/azel-runtime",
    sameAs: ["https://github.com/AzielEliab/azel-runtime", "https://github.com/AzielEliab"],
  };
  const itemList = {
    "@type": "ItemList",
    name: "Aziel Eliab products",
    numberOfItems: PRODUCTS.length,
    itemListElement: PRODUCTS.map((p, i) => {
      const u = productUrls(p, origin);
      const item = {
        "@type": "SoftwareApplication",
        name: p.name,
        description: p.oneLine,
        url: u.catalog_card,
        codeRepository: p.github,
        author: { "@type": "Person", name: "Aziel Eliab" },
        license: "https://www.apache.org/licenses/LICENSE-2.0",
      };
      if (u.doi_url) item.identifier = u.doi_url;
      return {
        "@type": "ListItem",
        position: i + 1,
        name: p.name,
        url: u.catalog_card,
        item,
      };
    });
  };
  return { "@context": "https://schema.org", "@graph": [software, itemList] };
}

function headMeta(origin, title, description, canonicalPath) {
  const base = origin.replace(/\/$/, "");
  const canonical = base + canonicalPath;
  return `<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>${escapeHtml(title)}</title>
<meta name="description" content="${escapeHtml(description)}">
<meta name="author" content="Aziel Eliab">
<meta name="robots" content="index, follow">
<link rel="canonical" href="${escapeHtml(canonical)}">
<link rel="sitemap" type="application/xml" href="${base}/sitemap.xml">
<meta property="og:type" content="website">
<meta property="og:title" content="${escapeHtml(title)}">
<meta property="og:description" content="${escapeHtml(description)}">
<meta property="og:url" content="${escapeHtml(canonical)}">
<meta property="og:site_name" content="Aziel Eliab">
<meta name="twitter:card" content="summary">
<meta name="twitter:title" content="${escapeHtml(title)}">
<meta name="twitter:description" content="${escapeHtml(description)}">`;
}

const PAGE_CSS = `
:root { color-scheme: dark; }
body { font: 16px/1.45 system-ui, sans-serif; max-width: 52rem; margin: 2.5rem auto; padding: 0 1.25rem 4rem; backgr

```

```

ound: #0e1014; color: #e8eae; }
h1 { font-size: 1.85rem; margin: 0 0 .35rem; }
h2 { font-size: 1.2rem; margin: 0 0 .4rem; }
.slug { font-weight: 500; color: #9aa3b2; font-size: .95rem; }
a { color: #c9d4ff; }
.lead { color: #9aa3b2; margin: 0 0 1.25rem; }
.honesty { border: 1px solid #5c4a1a; background: #241c0d; color: #f0d78c; padding: .9rem 1.05rem; border-radius: 10px; margin: 0 0 1.4rem; }
.honesty ul { margin: .4rem 0 0; padding-left: 1.2rem; }
.card { border: 1px solid #2a3140; border-radius: 12px; padding: 1.1rem 1.2rem; background: #151922; margin: 0 0 1rem; }
.banner { border: 1px solid #3d3420; background: #1b160c; color: #e6d19a; padding: .55rem .7rem; border-radius: 8px; font-size: .92rem; }
.online { margin: .2rem 0 .6rem; }
.meta a { margin-right: .85rem; }
pre { background: #0e1014; padding: .75rem .9rem; overflow: auto; border-radius: 8px; font-size: .82rem; }
code { font-size: .88rem; }
.links a { margin-right: 1rem; }
.cite { border: 1px solid #2a3140; border-radius: 12px; padding: 1rem 1.15rem; background: #12151c; margin: 0 0 1.4rem; }
;

```

```

function productCardHtml(p, origin, stats) {
  const u = productUrls(p, origin);
  const ops = p.ops
    .map((o) => `<code>${o.method} /p/${p.slug}/${o.op}</code> ? ${escapeHtml(o.summary)}`)
    .join("<br>");
  const banner = p.banner ? `<p class="banner">${escapeHtml(p.banner)}</p>` : "";
  const example = JSON.stringify(p.example, null, 2);
  const firstPost = p.ops.find((o) => o.method === "POST") || p.ops[0];
  const count =
    stats && typeof stats.downloads === "number"
      ? `<span class="count">${stats.downloads} counted</span>`
      : "";
  const doi = u.doi_url
    ? ` ? <a href="${u.doi_url}">DOI ${escapeHtml(u.doi)}</a>`
    : "";
  return `<article class="card" id="${escapeHtml(p.slug)}">
<h2><a href="${origin}/p/${p.slug}">${escapeHtml(p.name)}</a> <span class="slug">${escapeHtml(p.slug)}</span></h2>
<p class="online">${escapeHtml(p.online)}</p>
${banner}
<p class="meta">
  <a href="${p.github}">GitHub</a>
  <a href="${u.download}">counted /download</a>${count}
  <a href="${u.install}">install.sh</a>
  <a href="${u.skill}">v1/skill</a>
  <a href="${origin}/p/${p.slug}/health">catalog proxy health</a>${doi}
</p>
<p>Worker: <a href="${u.worker_home}">${p.worker}.viblock.workers.dev</a>
  ? <a href="${u.openapi}">product OpenAPI</a></p>
<p>${ops}</p>
<pre>curl -X ${firstPost.method} ${origin}/p/${p.slug}/${firstPost.op} \\\
-H 'content-type: application/json' \\\
-d '${example.replace(/'/g, '"')}'</pre>
</article>`;
}

```

```

function catalogHtml(origin, statsMap) {
  const cards = PRODUCTS.map((p) => productCardHtml(p, origin, statsMap && statsMap[p.slug])).join("\n");
  const ld = JSON.stringify(jsonLd(origin));
  const citeProducts = PRODUCTS.map((p) => {
    const doi = p.doi ? ` ? DOI <a href="https://doi.org/${p.doi}">${p.doi}</a>` : "";
    return `<li><a href="${p.github}">${escapeHtml(p.name)}</a>${doi}</li>`;
  }).join("");
  return `<!doctype html>
<html lang="en">
<head>
  ${headMeta(origin, CATALOG_TITLE, CATALOG_DESCRIPTION, "/")}
<script type="application/ld+json">${ld}</script>
<style>${PAGE_CSS}</style>
</head>
<body>
  <h1>Aziel Eliab product runtime catalog</h1>
  <p class="lead">One URL for Grok, ChatGPT GPT Actions, and Venice. ${PRODUCTS.length} products. Forks welcome. Apache-2.0. Author: Aziel Eliab.</p>
  <div class="honesty">
    <strong>Honesty banners</strong>
    <ul>
      <li>GodLock and MirageGrid are <em>not</em> VPNs and not anonymity networks.</li>
      <li>ForgeReceipts is <em>not</em> legal advice and does not contact courts.</li>
      <li>ZionPattern Solver never claims more than 75% confidence. It does not solve cases.</li>
      <li>VeilLock does <em>not</em> inject into FaceTime or any calling app.</li>
      <li>AZ-CLCE detects inconsistency, not intent. Type D is a label only.</li>
      <li>ChronoLock is advisory only ? not a scheduler, not targeting, not virality. 08:30?10:30 local. Distinct from TemporalLock.</li>
      <li>The ARK is <em>not</em> a kernel. Hosted API never unlocks or encrypts with a passphrase and never stores vaults. Sweep is Mode E heuristics only.</li>
      <li>AZAI is a local OpenAI-compatible runtime, not a new foundation model. Hosted /v1 is a protocol mirror + Lamb check, not a provider proxy. Jeeves is not sovereign.</li>
      <li>SpectralLock hosted overlay is a 256px preview, not a spectrometer, not forensic.</li>
      <li>EmployeeLock is <em>not</em> a court, not UL, not a truth score. Hosted never stores xlsx. Demo rows are for mat proof, not case facts.</li>
      <li>FoldLock is <em>not</em> zip. Hosted preview is tether-suppression on small UTF-8 text. Ratios are receipts, not trophies.</li>
      <li>WhistleLock is a local vault + dead-man copy. It is <em>not</em> a mailer. Hosted never holds whistle files.

```

```

</li>
<li>TrajectoryLock is a research prototype / auditable geometric test. <em>Not</em> a certified forensic instrument. Hosted never stores media. Synthetic examples are not real-case findings.</li>
</ul>
</div>
<section class="cite" id="cite">
  <h2>How to cite</h2>
  <p>Author: <strong>Aziel Eliab</strong> ? Catalog: <a href="{origin}/">{origin}</a> ? License: Apache-2.0 ? Machine-readable: <a href="{origin}/cite.json">/cite.json</a></p>
  <p>Eliab, Aziel. (2026). Aziel Eliab product runtime catalog [Software]. Apache-2.0. {origin}</p>
  <ul>{citeProducts}</ul>
</section>
<p class="links">
  <a href="{origin}/openapi.json">Combined OpenAPI 3.1</a>
  <a href="{origin}/v1/catalog.json">/v1/catalog.json</a>
  <a href="{origin}/llms.txt">/llms.txt</a>
  <a href="{origin}/ai.txt">/ai.txt</a>
  <a href="{origin}/sitemap.xml">/sitemap.xml</a>
  <a href="{origin}/robots.txt">/robots.txt</a>
  <a href="{origin}/mcp">MCP (POST JSON-RPC)</a>
  <a href="{origin}/v1/health">/v1/health</a>
  <a href="https://github.com/AzielEliab/aziel-runtime">GitHub</a>
</p>
<h2>Add this URL</h2>
<ul>
  <li><strong>ChatGPT</strong> ? GPT Actions ? Import from URL ? <code>{origin}/openapi.json</code></li>
  <li><strong>Grok</strong> ? custom tool / OpenAPI / MCP remote ? <code>{origin}/openapi.json</code> or <code>{origin}/mcp</code></li>
  <li><strong>Venice</strong> ? custom HTTP tools / OpenAPI ? same OpenAPI URL</li>
</ul>
  {cards}
</body>
</html>`;
}

function productPageHtml(p, origin, stats) {
  const u = productUrls(p, origin);
  const title = `${p.name} ? ${CATALOG_TITLE}`;
  const description = `${p.name}: {p.oneLine} Author Aziel Eliab. Apache-2.0.`;
  const ld = JSON.stringify({
    "@context": "https://schema.org",
    "@type": "SoftwareApplication",
    name: p.name,
    description: p.oneLine,
    url: u.catalog_card,
    codeRepository: p.github,
    author: { "@type": "Person", name: "Aziel Eliab" },
    license: "https://www.apache.org/licenses/LICENSE-2.0",
    identifier: u.doi_url || undefined,
  });
  return `<!doctype html>
<html lang="en">
<head>
  ${headMeta(origin, title, description, `p/${p.slug}`)}
  <script type="application/ld+json">{ld}</script>
  <style>{PAGE_CSS}</style>
</head>
<body>
  <p><a href="{origin}/">? Aziel Eliab product runtime catalog</a></p>
  ${productCardHtml(p, origin, stats)}
</body>
</html>`;
}

async function loadStatsMap(env, only) {
  const list = only ? [only] : PRODUCTS;
  const entries = await Promise.all(
    list.map(async (p) => {
      try {
        const { res } = await upstreamFetch(env, p, "/stats", {
          method: "GET",
          headers: { accept: "application/json" },
          signal: AbortSignal.timeout(2000),
        });
        if (!res || !res.ok) return [p.slug, null];
        const data = await res.json();
        const downloads = Number(data.downloads ?? data.total);
        const views = Number(data.views);
        return [
          p.slug,
          {
            downloads: Number.isFinite(downloads) ? downloads : null,
            views: Number.isFinite(views) ? views : null,
          },
        ];
      } catch {
        return [p.slug, null];
      }
    })
  );
  return Object.fromEntries(entries);
}

function escapeHtml(s) {
  return String(s)

```

```

        .replace(/&/g, "&amp;");
        .replace(/</g, "&lt;");
        .replace(/>/g, "&gt;");
        .replace(/"/g, "&quot;");
    }

function staticPaths(origin) {
    const paths = {
        "/v1/health": {
            get: {
                operationId: "catalog_health",
                summary: "Catalog liveness.",
                tags: ["catalog"],
                responses: { "200": { description: "ok" } },
            },
        },
        "/v1/catalog.json": {
            get: {
                operationId: "catalog_list",
                summary: "Machine-readable full product list (slug, github, worker, download, install, skill, openapi, doi, banner, ops).",
                tags: ["catalog"],
                responses: { "200": { description: "Product catalog JSON" } },
            },
        },
        "/cite.json": {
            get: {
                operationId: "catalog_cite",
                summary: "How to cite: author Aziel Eliab, Apache-2.0, GitHub + DOI.",
                tags: ["catalog"],
                responses: { "200": { description: "Citation JSON" } },
            },
        },
        "/llms.txt": {
            get: {
                operationId: "catalog_llms",
                summary: "Plain-text catalog for LLM crawlers.",
                tags: ["catalog"],
                responses: { "200": { description: "text/plain catalog" } },
            },
        },
        "/ai.txt": {
            get: {
                operationId: "catalog_ai_txt",
                summary: "Alias of /llms.txt.",
                tags: ["catalog"],
                responses: { "200": { description: "text/plain catalog" } },
            },
        },
        "/robots.txt": {
            get: {
                operationId: "catalog_robots",
                summary: "Allow / and sitemap URL.",
                tags: ["catalog"],
                responses: { "200": { description: "robots.txt" } },
            },
        },
        "/sitemap.xml": {
            get: {
                operationId: "catalog_sitemap",
                summary: "Indexable sitemap of catalog, OpenAPI, product cards, health, GitHub.",
                tags: ["catalog"],
                responses: { "200": { description: "sitemap.xml" } },
            },
        },
    };
    for (const p of PRODUCTS) {
        for (const o of p.ops) {
            const path = `/p/${p.slug}/${o.op}`;
            const method = o.method.toLowerCase();
            const item = {
                operationId: `${p.slug}_${o.op}`,
                summary: `${p.name}: ${o.summary}`,
                description: [p.banner, `Proxies to ${workerUrl(p, o.op)}'].filter(Boolean).join("\n\n"),
                tags: [p.slug],
                responses: {
                    "200": { description: "Upstream JSON" },
                    "4XX": { description: "Upstream or catalog error" },
                },
            };
            if (method === "post") {
                item.requestBody = {
                    required: true,
                    content: {
                        "application/json": {
                            schema: { type: "object" },
                            example: p.example,
                        },
                    },
                };
            }
            paths[path] = { [method]: item };
        }
    }
    return paths;
}

```



```

}

function rewriteLivePaths(spec, product) {
  const out = {};
  const src = spec && spec.paths;
  if (!src || typeof src !== "object") return out;
  for (const [path, item] of Object.entries(src)) {
    const m = path.match(/^\/v1\/([^\s]+)$/);
    if (!m) continue;
    const op = m[1];
    if (op === "health") continue;
    const prefixed = `/p/${product.slug}/${op}`;
    const copy = JSON.parse(JSON.stringify(item));
    for (const method of Object.keys(copy)) {
      const opObj = copy[method];
      if (opObj && typeof opObj === "object") {
        opObj.operationId = `${product.slug}_${op}`;
        opObj.tags = [product.slug];
        const extra = `Proxies to ${workerUrl(product, op)}.`;
        opObj.description = [opObj.description || opObj.summary || "", product.banner || "", extra]
          .filter(Boolean)
          .join("\n\n");
      }
    }
    out[prefixed] = copy;
  }
  return out;
}

async function combinedOpenApi(request, env) {
  const origin = originOf(request);
  const paths = staticPaths(origin);
  await Promise.all(
    PRODUCTS.map(async (p) => {
      try {
        const bind = env && env[bindingName(p)];
        let res;
        if (bind && typeof bind.fetch === "function") {
          res = await bind.fetch(new Request("https://internal/openapi.json", { headers: { accept: "application/json" } }));
        } else {
          res = await fetch(`https://${p.worker}.vibelock.workers.dev/openapi.json`, {
            headers: { accept: "application/json" },
            signal: AbortSignal.timeout(2500),
          });
        }
        if (!res.ok) return;
        const spec = await res.json();
        Object.assign(paths, rewriteLivePaths(spec, p));
      } catch {
        /* static fallback */
      }
    })
  );
  return {
    openapi: "3.1.0",
    info: {
      title: "Aziel Eliab product runtime catalog",
      version: "1.0.0",
      summary: "One OpenAPI file for every AzielEliab runtime Worker.",
      description:
        "Import this file in ChatGPT GPT Actions, Grok custom tools, or Venice HTTP tools. " +
        "Paths /p/{product}/{op} proxy to each product Worker /v1/{op}. " +
        "GodLock/MirageGrid are not VPNs. ForgeReceipts is not legal advice. " +
        "ZionPattern Solver caps confidence at 75% and does not solve cases. " +
        "VeilLock does not inject into FaceTime. AZ-CLCE detects inconsistency, not intent. " +
        "ChronoLock is advisory only (not a scheduler, not targeting, not virality; 08:30?10:30 local; distinct from T" +
        "emporalLock). " +
        "The ARK is not a kernel. Hosted API never unlocks or encrypts with a passphrase and never stores vaults. Swee" +
        "p is Mode E heuristics only. " +
        "AZAI is a local OpenAI-compatible runtime, not a new foundation model. Hosted /v1 is a protocol mirror + Lamb" +
        "check, not a provider proxy. Jeeves is not sovereign. " +
        "SpectralLock hosted overlay is a 256px preview, not a spectrometer, not forensic. " +
        "EmployeeLock is not a court, not UL, not a truth score. Hosted never stores xlsx. " +
        "FoldLock is not zip. Hosted preview is tether-suppression on small UTF-8 text. " +
        "WhistleLock is not a mailer. Hosted never holds whistle files. " +
        "TrajectoryLock is a research prototype / auditable geometric test. Not a certified forensic instrument. Hoste" +
        "d never stores media. " +
        "Forks welcome. Apache-2.0. Author: Aziel Eliab.",
      license: { name: "Apache-2.0", identifier: "Apache-2.0" },
      contact: { name: "Aziel Eliab", url: "https://github.com/AzielEliab/aziel-runtime" },
    },
    servers: [{ url: origin }],
    tags: PRODUCTS.map((p) => ({ name: p.slug, description: p.name })),
    paths,
  };
}

function bindingName(product) {
  return String(product.slug || "").toUpperCase().replace(/-/g, "_");
}

async function upstreamFetch(env, product, path, init) {
  const bind = env && env[bindingName(product)];

```

```

const target = `https://${product.worker}.vibelock.workers.dev${path}`;
if (bind && typeof bind.fetch === "function") {
  const req = new Request("https://internal" + path, init);
  return { res: await bind.fetch(req), target: target + " (service binding)" };
}
return { res: await fetch(target, init), target };
}

async function proxy(product, op, request, env) {
  const targetPath = `/v1/${op}`;
  const method = request.method === "GET" ? "GET" : "POST";
  const headers = {
    "content-type": request.headers.get("content-type") || "application/json",
    accept: "application/json",
  };
  const init = { method, headers };
  if (method !== "GET") {
    const text = await request.text();
    init.body = text && text.trim() ? text : "{}";
  }
  try {
    const { res, target } = await upstreamFetch(env, product, targetPath, init);
    const buf = await res.arrayBuffer();
    const outHeaders = new Headers(corsHeaders());
    const ct = res.headers.get("content-type") || "application/json; charset=utf-8";
    outHeaders.set("Content-Type", ct);
    outHeaders.set("X-Aziel-Upstream", target);
    return new Response(buf, { status: res.status, headers: outHeaders });
  } catch (err) {
    return json(
      {
        error: "upstream unreachable",
        product: product.slug,
        worker: product.worker,
        op,
        detail: String(err && err.message ? err.message : err),
      },
      502,
    );
  }
}

function toolList() {
  const tools = [];
  for (const p of PRODUCTS) {
    for (const o of p.ops) {
      tools.push({
        name: `${p.slug}_${o.op}`,
        description: `${p.name}: ${o.summary}${p.banner ? " " + p.banner : ""}`,
        inputSchema: {
          type: "object",
          additionalProperties: true,
          description: `JSON body posted to /v1/${o.op} on ${p.worker}`,
        },
      });
    }
  }
  return tools;
}

async function callTool(env, name, args) {
  const idx = name.indexOf("_");
  if (idx < 1) throw new Error(`unknown tool: ${name}`);
  const slug = name.slice(0, idx);
  const op = name.slice(idx + 1);
  const product = BY_SLUG[slug];
  if (!product) throw new Error(`unknown product: ${slug}`);
  const spec = product.ops.find(o => o.op === op);
  const method = spec && spec.method === "GET" ? "GET" : "POST";
  const init = {
    method,
    headers: { "content-type": "application/json", accept: "application/json" },
  };
  if (method !== "GET") init.body = JSON.stringify(args && typeof args === "object" ? args : {});
  const { res, target } = await upstreamFetch(env, product, `/v1/${op}`, init);
  const text = await res.text();
  return { status: res.status, text, target };
}

function rpcResult(id, result) {
  return json({ jsonrpc: "2.0", id: id ?? null, result });
}

function rpcError(id, code, message) {
  return json({ jsonrpc: "2.0", id: id ?? null, error: { code, message } });
}

async function handleMcp(request, env) {
  if (request.method === "GET") {
    return json({
      ok: true,
      transport: "JSON-RPC MCP-over-HTTP",
      endpoint: "POST /mcp",
      methods: ["initialize", "tools/list", "tools/call", "ping"],
      auth: "none (public)",
    });
  }

```

```

    note: "Durable Objects / agents McpAgent not used. Minimal HTTP JSON-RPC.",
  });
}
if (request.method !== "POST") {
  return json({ error: "POST JSON-RPC to /mcp" }, 405);
}
let body;
try {
  body = await request.json();
} catch {
  return rpcError(null, -32700, "Parse error");
}
const id = body && body.id !== undefined ? body.id : null;
const method = body && body.method;
const params = (body && body.params) || {};
if (method === "initialize") {
  return rpcResult(id, {
    protocolVersion: PROTOCOL,
    capabilities: { tools: { listChanged: false } },
    serverInfo: { name: "aziel-runtime", version: "1.0.0" },
    instructions:
      "Tools are named {product}_{op} and proxy to each AzielEliab Worker /v1/{op}. Public, no OAuth.",
  });
}
if (method === "notifications/initialized" || method === "initialized") {
  return new Response(null, { status: 204, headers: corsHeaders() });
}
if (method === "ping") {
  return rpcResult(id, {});
}
if (method === "tools/list") {
  return rpcResult(id, { tools: toolList() });
}
if (method === "tools/call") {
  const name = params.name;
  const args = params.arguments || params.input || {};
  try {
    const out = await callTool(env, name, args);
    return rpcResult(id, {
      content: [
        {
          type: "text",
          text: out.text,
        },
      ],
      isError: out.status >= 400,
    });
  } catch (err) {
    return rpcResult(id, {
      content: [{ type: "text", text: JSON.stringify({ error: String(err.message || err) }) }],
      isError: true,
    });
  }
}
return rpcError(id, -32601, `Method not found: ${method}`);
}

export default {
  async fetch(request, env) {
    const url = new URL(request.url);
    const origin = originOf(request);
    const extra = (path) => linkHeaders(origin, path);

    if (request.method === "OPTIONS") {
      return new Response(null, { status: 204, headers: corsHeaders() });
    }

    if (url.pathname === "/" && request.method === "GET") {
      const statsMap = await loadStatsMap(env);
      return html(catalogHtml(origin, statsMap), extra("/"));
    }

    if (url.pathname === "/robots.txt" && request.method === "GET") {
      return text(robotsTxt(origin), extra("/robots.txt"));
    }

    if (url.pathname === "/sitemap.xml" && request.method === "GET") {
      return xml(sitemapXml(origin), extra("/sitemap.xml"));
    }

    if ((url.pathname === "/llms.txt" || url.pathname === "/ai.txt") && request.method === "GET") {
      return text(llmsTxt(origin), extra(url.pathname));
    }

    if (url.pathname === "/cite.json" && request.method === "GET") {
      return json(citeJson(origin), 200, extra("/cite.json"));
    }

    if (url.pathname === "/v1/catalog.json" && request.method === "GET") {
      return json(
        {
          ok: true,
          author: "Aziel Eliab",
          title: CATALOG_TITLE,
        }
      );
    }
  }
};

```

```

        catalog: origin + "/",
        license: "Apache-2.0",
        count: PRODUCTS.length,
        products: PRODUCTS.map((p) => catalogRecord(p, origin)),
    },
    200,
    extra("/v1/catalog.json"),
);
}

if (url.pathname === "/openapi.json" && request.method === "GET") {
    return json(await combinedOpenApi(request, env), 200, extra("/openapi.json"));
}

if (url.pathname === "/v1/health" && request.method === "GET") {
    return json(
        {
            ok: true,
            product: "aziel-runtime",
            author: "Aziel Eliab",
            title: CATALOG_TITLE,
            products: PRODUCTS.map((p) => p.slug),
            count: PRODUCTS.length,
            openapi: "/openapi.json",
            catalog: "/v1/catalog.json",
            cite: "/cite.json",
            sitemap: "/sitemap.xml",
            robots: "/robots.txt",
            llms: "/llms.txt",
            ai: "/ai.txt",
            mcp: "/mcp",
            kv_increment: false,
        },
        200,
        extra("/v1/health"),
    );
}

if (url.pathname === "/mcp" || url.pathname === "/mcp/") {
    return handleMcp(request, env);
}

const card = url.pathname.match(/^\/p\/([a-z0-9-]+)\/?$/i);
if (card && request.method === "GET") {
    const product = BY_SLUG[card[1].toLowerCase()];
    if (!product) {
        return json({ error: "unknown product", slug: card[1], known: PRODUCTS.map((p) => p.slug) }, 404);
    }
    const statsMap = await loadStatsMap(env, product);
    return html(productPageHtml(product, origin, statsMap[product.slug]), extra(`/p/${product.slug}`));
}

const m = url.pathname.match(/^\/p\/([a-z0-9-]+)\/([a-z0-9_-]+)$/i);
if (m && (request.method === "GET" || request.method === "POST")) {
    const product = BY_SLUG[m[1].toLowerCase()];
    if (!product) {
        return json({ error: "unknown product", slug: m[1], known: PRODUCTS.map((p) => p.slug) }, 404);
    }
    return proxy(product, m[2], request, env);
}

return json(
    {
        error: "not found",
        hint: "GET / GET /openapi.json GET /v1/catalog.json GET /sitemap.xml GET /robots.txt GET /llms.txt GET /cite.json POST /p/{product}/{op} POST /mcp",
    },
    404,
);
};
};
};

```

aziel-runtime/wrangler.toml

```
name = "aziel-runtime"
main = "src/index.js"
compatibility_date = "2026-07-01"
workers_dev = true
account_id = "ac575a9b822bea2bed97d0ab73aed238"

[[services]]
binding = "VIBELock"
service = "vibelock-download-tracker"

[[services]]
binding = "VEILLOCK"
service = "veillock-download-tracker"

[[services]]
binding = "CODELOCK"
service = "codelock-download-tracker"

[[services]]
binding = "GODLOCK"
service = "godlock-download-tracker"

[[services]]
binding = "SHADOWLOCK"
service = "shadowlock-download-tracker"

[[services]]
binding = "TEMPORALLOCK"
service = "temporallock-download-tracker"

[[services]]
binding = "FORGERECEIPTS"
service = "forgereceipts-download-tracker"

[[services]]
binding = "DECISIONGATE"
service = "decisiongate-download-tracker"

[[services]]
binding = "ZSOLVER"
service = "zsolver-download-tracker"

[[services]]
binding = "AZOS"
service = "azos-download-tracker"

[[services]]
binding = "GLOSSAFILTER"
service = "glossafilter-download-tracker"

[[services]]
binding = "MIRAGEGRID"
service = "miragegrid-download-tracker"

[[services]]
binding = "STATICCLOCK"
service = "staticclock-download-tracker"

[[services]]
binding = "CHRONOLOCK"
service = "chronolock-download-tracker"

[[services]]
binding = "POSTKING"
service = "postking-download-tracker"

[[services]]
binding = "AZCLCE"
service = "azclce-download-tracker"

[[services]]
binding = "ARK"
service = "ark-download-tracker"

[[services]]
binding = "AZAI"
service = "azai-download-tracker"

[[services]]
binding = "SPECTRALLOCK"
service = "spectrallock-download-tracker"

[[services]]
binding = "AZBOT"
service = "azbot-download-tracker"

[[services]]
binding = "EMPLOYEELOCK"
service = "employee-lock-download-tracker"

[[services]]
binding = "FOLDLOCK"
```

```
service = "foldlock-download-tracker"

[[services]]
binding = "WHISTLELOCK"
service = "whistlelock-download-tracker"

[[services]]
binding = "TRAJECTORYLOCK"
service = "trajectorylock-download-tracker"
```

azos/azos/__init__.py

```
"""AZ-OS + AZ Interface: a portable ethical overlay, not a kernel.

July 2026 papers by Aziel Eliab. Integrity precedes execution.
Propagation is invitation, not infection. Forks are welcome.

This package is a soft overlay (a folder + control surface). It is not
an operating system, bootloader, hypervisor, worm, or malware.
"""

from __future__ import annotations

from azos.errors import (
    AppendOnlyError,
    AuthorizationError,
    AzosError,
    GateFail,
)
from azos.gate import GATES, Proposal
from azos.invite import DOWNLOAD_URL, invite_text
from azos.runtime import Runtime

__version__ = "0.1.0"
__author__ = "Aziel Eliab"
__all__ = [
    "AppendOnlyError",
    "AuthorizationError",
    "AzosError",
    "DOWNLOAD_URL",
    "GATES",
    "GateFail",
    "Proposal",
    "Runtime",
    "invite_text",
    "__version__",
]
```

azos/azos/__main__.py

```
from __future__ import annotations

from azos.cli import main

raise SystemExit(main())
```


azos/azos/arc.py

```

"""ARC ? execution authority: issue, verify, revoke.

Tokens are 32 random bytes. Only a sha256 of the raw bytes is stored.
A root or user string does not auto-grant a token.
"""

from __future__ import annotations

import hashlib
import json
import secrets
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

from azos.errors import AuthorizationError

_TOKEN_BYTES = 32
_HEX_LEN = _TOKEN_BYTES * 2

def _utcnow() -> str:
    return datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")

def _hash_raw(raw: bytes) -> str:
    return hashlib.sha256(raw).hexdigest()

def parse_token(token: str) -> bytes:
    """Decode an issued hex token. Malformed ? AuthorizationError (deny)."""
    if not isinstance(token, str) or len(token) != _HEX_LEN:
        raise AuthorizationError("unauthorized: malformed token")
    try:
        raw = bytes.fromhex(token)
    except ValueError as exc:
        raise AuthorizationError("unauthorized: malformed token") from exc
    if len(raw) != _TOKEN_BYTES:
        raise AuthorizationError("unauthorized: malformed token")
    return raw

class ARC:
    """Issue / revoke execution tokens. Hashed at rest in .azos/tokens.json."""

    def __init__(self, session_dir: Path) -> None:
        self.session_dir = Path(session_dir)
        self._path = self.session_dir / "tokens.json"

    def issue(self, *, action: str, actor: str) -> str:
        """Create a token. Caller must have already passed the five gates.

        Returns the hex form of 32 raw bytes once. Stores only sha256.
        The ``actor`` string is recorded; it is never treated as authority.
        """
        # Explicit: a root/user label is not a credential.
        if not action:
            raise AuthorizationError("unauthorized: action required to issue")
        raw = secrets.token_bytes(_TOKEN_BYTES)
        digest = _hash_raw(raw)
        records = self._load()
        records.append(
            {
                "hash": digest,
                "action": action,
                "actor": actor,
                "issued_at": _utcnow(),
                "revoked": False,
            }
        )
        self._save(records)
        return raw.hex()

    def verify(self, token: str) -> bool:
        """True iff the token is a live (issued, not revoked) ARC token."""
        try:
            raw = parse_token(token)
        except AuthorizationError:
            return False
        digest = _hash_raw(raw)
        for rec in self._load():
            if rec.get("hash") == digest and not rec.get("revoked"):
                return True
        return False

    def revoke(self, token: str) -> None:
        raw = parse_token(token)
        digest = _hash_raw(raw)
        self._revoke_hash(digest)

    def revoke_all(self) -> int:
        records = self._load()

```

```

        n = 0
        for rec in records:
            if not rec.get("revoked"):
                rec["revoked"] = True
                rec["revoked_at"] = _utcnow()
                n += 1
        if n:
            self._save(records)
        return n

def has_active(self) -> bool:
    return any(not rec.get("revoked") for rec in self._load())

def active_hashes(self) -> list[str]:
    return [rec["hash"] for rec in self._load() if not rec.get("revoked")]

def snapshot(self) -> dict[str, Any]:
    records = self._load()
    return {
        "active": sum(1 for r in records if not r.get("revoked")),
        "revoked": sum(1 for r in records if r.get("revoked")),
        "issued": len(records),
    }

def token_hash(self, token: str) -> str:
    return _hash_raw(parse_token(token))

def _revoke_hash(self, digest: str) -> None:
    records = self._load()
    found = False
    for rec in records:
        if rec.get("hash") == digest:
            rec["revoked"] = True
            rec["revoked_at"] = _utcnow()
            found = True
    if not found:
        raise AuthorizationError("unauthorized: unknown token")
    self._save(records)

def _load(self) -> list[dict[str, Any]]:
    if not self._path.is_file():
        return []
    try:
        data = json.loads(self._path.read_text(encoding="utf-8"))
    except (OSError, json.JSONDecodeError):
        return []
    tokens = data.get("tokens") if isinstance(data, dict) else None
    return list(tokens) if isinstance(tokens, list) else []

def _save(self, records: list[dict[str, Any]]) -> None:
    self.session_dir.mkdir(parents=True, exist_ok=True)
    payload = {"tokens": records}
    self._path.write_text(
        json.dumps(payload, indent=2, sort_keys=True) + "\n",
        encoding="utf-8",
    )

```

azos/azos/cli.py

```

"""Command-line interface for AZ-OS + AZ Interface.

    azos version
    azos ui          # 127.0.0.1:8800 THE product
    azos status
    azos invite      # prints voluntary adoption text + download URL
    azos exec NAME   # only if a token is active
    azos halt
    azos purge --confirm # deletes .azos session only
"""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path
from typing import Sequence

from azos import __version__
from azos.errors import AuthorizationError, AzosError
from azos.interface import DEFAULT_HOST, DEFAULT_PORT, serve
from azos.invite import emit_invite, invite_text
from azos.runtime import Runtime

def _build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="azos",
        description=(
            "AZ-OS + AZ Interface ? ethical overlay framework "
            "(Aziel Eliab, 2026). Integrity precedes execution. "
            "Not a kernel, not malware. Propagation is invitation. "
            "Control surface: `azos ui` at http://127.0.0.1:8800."
        ),
    )
    sub = parser.add_subparsers(dest="cmd", required=True)

    sub.add_parser("version", help="Print package version.")
    sub.add_parser("status", help="Overlay, Lumen, token, and log status.")
    sub.add_parser(
        "invite",
        help="Print voluntary adoption text and the counted download URL. Writes no files.",
    )

    p_ui = sub.add_parser(
        "ui",
        help="Serve AZ Interface on 127.0.0.1:8800 (the product).",
    )
    p_ui.add_argument("--host", default=DEFAULT_HOST, help="Loopback host (default 127.0.0.1).")
    p_ui.add_argument("--port", type=int, default=DEFAULT_PORT, help="Port (default 8800).")

    p_exec = sub.add_parser(
        "exec",
        help="Run a registered safe builtin (list_modules, echo, status, purge_session). Requires a live ARC token.",
    )
    p_exec.add_argument("name", help="Builtin name.")
    p_exec.add_argument(
        "rest",
        nargs="*",
        help="Optional echo message words. Never passed to a shell.",
    )
    p_exec.add_argument(
        "--token",
        default=None,
        help="Issued ARC token (hex). If omitted, a live session token must already exist.",
    )
    p_exec.add_argument("--json", action="store_true", dest="as_json", help="Print JSON.")

    sub.add_parser("halt", help="Halt execution authority. Lumen keeps running.")

    p_purge = sub.add_parser(
        "purge",
        help="Delete the .azos session directory only. Never $HOME or OS files.",
    )
    p_purge.add_argument(
        "--confirm",
        action="store_true",
        help="Required. Confirms deletion of .azos only.",
    )

    p_doc = sub.add_parser("doctor", help="Self-check. No network, no telemetry.")
    p_doc.add_argument("--json", action="store_true", dest="as_json", help="Print doctor results as JSON.")

    p_imp = sub.add_parser("import", help="Import a JSON document.")
    p_imp.add_argument("path")

    p_exp = sub.add_parser("export", help="Export a JSON document.")
    p_exp.add_argument("path")

    return parser

def _runtime() -> Runtime:

```

```

    return Runtime(root=Path.cwd())

def main(argv: Sequence[str] | None = None) -> int:
    parser = _build_parser()
    args = parser.parse_args(list(argv) if argv is not None else None)

    if args.cmd == "version":
        sys.stdout.write(f"azos {__version__}\n")
        return 0

    if args.cmd == "invite":
        emit_invite(sys.stdout)
        return 0

    if args.cmd == "ui":
        serve(host=args.host, port=args.port, runtime=_runtime())
        return 0

    rt = _runtime()

    if args.cmd == "status":
        sys.stdout.write(json.dumps(rt.status(), indent=2) + "\n")
        return 0

    if args.cmd == "halt":
        sys.stdout.write(json.dumps(rt.halt(), indent=2) + "\n")
        return 0

    if args.cmd == "purge":
        if not args.confirm:
            sys.stderr.write("purge requires --confirm (deletes .azos only)\n")
            sys.stderr.write(invite_text())
            return 2
        try:
            out = rt.purge(confirm=True)
        except AzosError as exc:
            sys.stderr.write(f"{exc}\n")
            return 1
        sys.stdout.write(json.dumps(out, indent=2) + "\n")
        return 0

    if args.cmd == "exec":
        extra: dict = {}
        if args.rest:
            extra["message"] = " ".join(args.rest)
        try:
            result = rt.run(args.name, token=args.token, args=extra or None)
        except AuthorizationError as exc:
            sys.stderr.write(f"{exc}\n")
            sys.stderr.write(invite_text())
            return 1
        except AzosError as exc:
            sys.stderr.write(f"{exc}\n")
            return 1
        if args.as_json:
            sys.stdout.write(json.dumps(result, indent=2) + "\n")
        else:
            sys.stdout.write(json.dumps(result, indent=2) + "\n")
        return 0

    if args.cmd == "doctor":
        from azos.doctor import run_doctor

        return run_doctor(as_json=getattr(args, "as_json", False))

    if args.cmd == "import":
        from azos.jsonio import import_json

        rec = import_json(args.path)
        sys.stdout.write(json.dumps(rec, indent=2, ensure_ascii=False) + "\n")
        return 0

    if args.cmd == "export":
        from azos.jsonio import export_json

        rec = export_json(args.path)
        sys.stdout.write(json.dumps(rec, indent=2, ensure_ascii=False) + "\n")
        return 0

    parser.error(f"unknown command {args.cmd}")
    return 2

```

azos/azos/doctor.py

```

"""Self-check for AZ-OS. No network, no telemetry.

    azos doctor
"""

from __future__ import annotations

import json
import tempfile
from pathlib import Path
from typing import Callable

from azos import __version__

AUTHOR = "Aziel Eliab"
Check = tuple[str, bool, str]

def _ok(name: str, detail: str = "") -> Check:
    return name, True, detail

def _fail(name: str, detail: str) -> Check:
    return name, False, detail

def _check_version() -> Check:
    if __version__:
        return _ok("version", str(__version__))
    return _fail("version", "missing")

def _check_identity() -> Check:
    try:
        mod = __import__(__name__.split(".")[0])
        author = str(getattr(mod, "__author__", AUTHOR))
    except Exception as exc: # noqa: BLE001
        return _fail("identity", str(exc))
    blob = author + " " + AUTHOR
    forbidden = ("Col" + "lin H" + "orton", "Ja" + "ck Al" + "tman", "GodLock" + ".AZ", "Reve" + "aler")
    if any(x in blob for x in forbidden):
        return _fail("identity", "forbidden identity label")
    if "Aziel Eliab" not in blob:
        return _fail("identity", author)
    return _ok("identity", AUTHOR)

def _check_json_roundtrip() -> Check:
    from azos.jsonio import export_json, import_json

    with tempfile.TemporaryDirectory() as tmp:
        src = Path(tmp) / "in.json"
        out = Path(tmp) / "out.json"
        src.write_text(json.dumps({"product": "azos", "author": AUTHOR, "ok": True}, indent=2), encoding="utf-8")
        rec = import_json(src)
        if not rec.get("ok"):
            return _fail("import", str(rec))
        rec2 = export_json(out)
        if not rec2.get("ok") or not out.exists():
            return _fail("export", str(rec2))
        doc = json.loads(out.read_text(encoding="utf-8"))
        if doc.get("author") != AUTHOR:
            return _fail("export author", str(doc.get("author")))
        return _ok("json import/export", "roundtrip")

CHECKS: tuple[Callable[[], Check], ...] = (
    _check_version,
    _check_identity,
    _check_json_roundtrip,
)

def run_doctor(*, as_json: bool = False) -> int:
    results = []
    failed = 0
    for fn in CHECKS:
        name, ok, detail = fn()
        results.append({"name": name, "ok": ok, "detail": detail})
        if not ok:
            failed += 1
        mark = "ok" if ok else "FAIL"
        if not as_json:
            print(f"[{mark}] {name}" + (f" ? {detail}" if detail else ""))
    payload = {
        "ok": failed == 0,
        "failed": failed,
        "checks": results,
        "version": __version__,
        "author": AUTHOR,
        "network": False,
    }

```

```
        "telemetry": False,
    }
    if as_json:
        print(json.dumps(payload, indent=2))
    else:
        print("doctor", "passed" if failed == 0 else "failed")
    return 0 if failed == 0 else 1
```

azos/azos/errors.py

```
"""AZ-OS errors. Default is deny."""

from __future__ import annotations

class AzosError(Exception):
    """Base error for the overlay."""

class AuthorizationError(AzosError):
    """Unsigned or unauthorized run(). Default deny.

    Unauthorized environments receive an invite, not a silent block.
    """

    def __init__(self, message: str = "unauthorized: no ARC token") -> None:
        super().__init__(message)
        self.invite = True

class AppendOnlyError(AzosError):
    """The execution log cannot be rewritten."""

class GateFail(AzosError):
    """A proposed action failed one or more of the five gates."""

class HaltedError(AuthorizationError):
    """The overlay is halted. Lumen still runs (revoke / purge only)."""

    def __init__(self, message: str = "halted: execution authority is final") -> None:
        super().__init__(message)
```

azos/azos/exec.py

```

"""Run registered *safe* builtin actions by name.

No eval(). No exec() of user strings. No subprocess. No shell.
Unsigned / unauthorized run() raises AuthorizationError. Default deny.

Safe builtins: list_modules, echo, status, purge_session.
"""

from __future__ import annotations

from typing import Any, Callable

from azos.errors import AuthorizationError, HaltedError

# Closed allow-list. Names only. Never a user-supplied callable.
SAFE_ACTIONS = frozenset({"list_modules", "echo", "status", "purge_session"})

def _echo(runtime: Any, args: dict[str, Any]) -> dict[str, Any]:
    message = args.get("message", "ok")
    if not isinstance(message, str):
        message = "ok"
    # Bound the echo; never pass it to a shell.
    return {"echo": message[:4096]}

def _list_modules(runtime: Any, args: dict[str, Any]) -> dict[str, Any]:
    return {"modules": sorted(SAFE_ACTIONS)}

def _status(runtime: Any, args: dict[str, Any]) -> dict[str, Any]:
    return runtime.status()

def _purge_session(runtime: Any, args: dict[str, Any]) -> dict[str, Any]:
    runtime.lumen.purge_session(confirm=True)
    return {"purged": True, "session": str(runtime.session_dir)}

_HANDLERS: dict[str, Callable[[Any, dict[str, Any]], dict[str, Any]]] = {
    "echo": _echo,
    "list_modules": _list_modules,
    "status": _status,
    "purge_session": _purge_session,
}

class Executor:
    """Dispatch a named builtin after ARC verifies a live token."""

    def __init__(self, runtime: Any) -> None:
        self.runtime = runtime

    def run(
        self,
        name: str,
        token: str | None = None,
        args: dict[str, Any] | None = None,
    ) -> dict[str, Any]:
        """Run a registered builtin. Default deny.

        ``token`` is required unless ARC already has a live token
        (CLI ``azos exec NAME`` after the Interface issued one).
        """
        if self.runtime.halted:
            raise HaltedError()
        action = (name or "").strip()
        if action not in SAFE_ACTIONS:
            raise AuthorizationError(
                f"unauthorized: {action!r} is not a registered safe builtin"
            )
        token_hash = self._require_token(token)
        handler = _HANDLERS[action]
        payload = dict(args or {})
        result = handler(self.runtime, payload)
        # Do not log the raw token. Hash only.
        log_payload = {"ok": True, "args_keys": sorted(payload.keys())}
        self.runtime.log.append(
            action=action,
            token_hash=token_hash,
            payload=log_payload,
        )
        return result

    def _require_token(self, token: str | None) -> str:
        arc = self.runtime.arc
        if token:
            if not arc.verify(token):
                raise AuthorizationError("unauthorized: token missing or revoked")
            return arc.token_hash(token)
        if arc.has_active():
            hashes = arc.active_hashes()

```



```
        return hashes[0] if hashes else "active-session"  
    raise AuthorizationError("unauthorized: no ARC token")
```

azos/azos/gate.py

```

"""Tiny five-gate check (DecisionGATE-shaped, reimplemented inline).

Gates: definition, evidence, impact, integrity, responsibility.
Any FAIL ? no token. Default deny.

Unauthorized or incomplete proposals produce a GateResult with
passed=False so the Interface can show an invite, not a silent block.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import AbstractSet, Mapping

GATES = (
    "definition",
    "evidence",
    "impact",
    "integrity",
    "responsibility",
)

_MIN = 8

@dataclass
class Proposal:
    """A proposed action presented to the five gates."""

    action: str
    definition: str = ""
    evidence: str = ""
    impact: str = ""
    actor: str = ""
    extend_module: bool = False
    comprehension: bool = False
    intent: str = ""

@dataclass
class GateCheck:
    passed: bool
    reason: str

@dataclass
class GateResult:
    passed: bool
    gates: dict[str, GateCheck] = field(default_factory=dict)

    def as_dict(self) -> dict[str, object]:
        return {
            "passed": self.passed,
            "gates": {
                name: {"pass": c.passed, "reason": c.reason}
                for name, c in self.gates.items()
            },
        }

def _filled(value: str, minimum: int = _MIN) -> bool:
    return isinstance(value, str) and len(value.strip()) >= minimum

def authorize(proposal: Proposal, allowed: AbstractSet[str]) -> GateResult:
    """Evaluate all five gates. Does not short-circuit (UI shows each)."""
    action = (proposal.action or "").strip()
    checks: dict[str, GateCheck] = {}

    if not action:
        checks["definition"] = GateCheck(False, "action name is required")
    elif not _filled(proposal.definition):
        checks["definition"] = GateCheck(
            False, "definition must state what the action is (min 8 chars)"
        )
    else:
        checks["definition"] = GateCheck(True, "action is defined")

    if not _filled(proposal.evidence):
        checks["evidence"] = GateCheck(
            False, "evidence / justification is required (min 8 chars)"
        )
    else:
        checks["evidence"] = GateCheck(True, "evidence provided")

    impact = (proposal.impact or "").strip().lower()
    banned = ("wipe disk", "format drive", "mkfs", "self-replicate", "worm")
    if not _filled(proposal.impact):
        checks["impact"] = GateCheck(
            False, "impact must state what will change (min 8 chars)"
        )
    elif any(b in impact for b in banned):

```

```

        checks["impact"] = GateCheck(False, "impact violates overlay bounds")
    else:
        checks["impact"] = GateCheck(True, "impact stated")

    integrity_ok = True
    integrity_reason = "action is a registered safe builtin"
    if action not in allowed:
        integrity_ok = False
        integrity_reason = "unsigned / unregistered action: default deny"
    if proposal.extend_module:
        if not proposal.comprehension:
            integrity_ok = False
            integrity_reason = (
                "extending a module requires the comprehension checkbox"
            )
        elif not _filled(proposal.intent, 12):
            integrity_ok = False
            integrity_reason = (
                "extending a module requires a short restatement of intent"
            )
        elif action not in allowed:
            integrity_ok = False
            integrity_reason = (
                "understood, but unsigned modules cannot run (integrity)"
            )
    checks["integrity"] = GateCheck(integrity_ok, integrity_reason)

    actor = (proposal.actor or "").strip()
    if not actor:
        checks["responsibility"] = GateCheck(False, "a named actor is required")
    else:
        # Root/user is a name, not a privilege. It does not auto-grant.
        checks["responsibility"] = GateCheck(
            True, f"actor {actor!r} is named (name is not a privilege)"
        )

    passed = all(c.passed for c in checks.values())
    return GateResult(passed=passed, gates=checks)

def summary(result: GateResult) -> list[Mapping[str, object]]:
    return [
        {"gate": name, "pass": result.gates[name].passed, "reason": result.gates[name].reason}
        for name in GATES
        if name in result.gates
    ]

```

azos/azos/interface.py

```

"""AZ Interface ? the control surface. Bind 127.0.0.1:8800.

Not cosmetic. Request ? five gates ? token or invite. Action buttons
only enable with a live token. Halt. Purge with typed CONFIRM. Lumen
status stays running after halt. Self-contained CSS, no CDN.
"""

from __future__ import annotations

import json
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from importlib.resources import files
from typing import Any
from urllib.parse import urlparse

from azos.gate import Proposal
from azos.invite import invite_text
from azos.runtime import Runtime

LOOPBACK = frozenset({"127.0.0.1", "localhost", "::1"})
DEFAULT_HOST = "127.0.0.1"
DEFAULT_PORT = 8800
WEB = files("azos") / "templates"

def _html_bytes() -> bytes:
    return (WEB / "ui.html").read_bytes()

def make_handler(runtime: Runtime):
    class Handler(BaseHTTPRequestHandler):
        server_version = "AZ-Interface/0.1.0"

        def log_message(self, fmt: str, *args: object) -> None:
            return

        def _send(self, status: int, body: bytes, content_type: str) -> None:
            self.send_response(status)
            self.send_header("Content-Type", content_type)
            self.send_header("Content-Length", str(len(body)))
            self.send_header("Cache-Control", "no-store")
            self.end_headers()
            self.wfile.write(body)

        def _json(self, status: int, obj: object) -> None:
            body = json.dumps(obj, indent=2, ensure_ascii=False).encode("utf-8")
            self._send(status, body, "application/json; charset=utf-8")

        def _read_json(self) -> dict[str, Any]:
            length = int(self.headers.get("Content-Length") or "0")
            raw = self.rfile.read(length) if length else b"{}"
            try:
                payload = json.loads(raw.decode("utf-8") or "{}")
            except json.JSONDecodeError:
                return {}
            return payload if isinstance(payload, dict) else {}

        def do_GET(self) -> None: # noqa: N802
            path = urlparse(self.path).path
            if path in {"/", "/index.html"}:
                self._send(200, _html_bytes(), "text/html; charset=utf-8")
                return
            if path == "/api/status":
                self._json(200, runtime.status())
                return
            if path == "/api/invite":
                self._json(200, {"invite": invite_text()})
                return
            if path == "/api/log":
                entries = [e.to_dict() for e in runtime.log.entries()]
                self._json(200, {"entries": entries, "length": len(entries)})
                return
            self._json(404, {"error": "not found", "invite": invite_text()})

        def do_POST(self) -> None: # noqa: N802
            path = urlparse(self.path).path
            payload = self._read_json()

            if path == "/api/request":
                proposal = Proposal(
                    action=str(payload.get("action") or ""),
                    definition=str(payload.get("definition") or ""),
                    evidence=str(payload.get("evidence") or ""),
                    impact=str(payload.get("impact") or ""),
                    actor=str(payload.get("actor") or ""),
                    extend_module=bool(payload.get("extend_module")),
                    comprehension=bool(payload.get("comprehension")),
                    intent=str(payload.get("intent") or ""),
                )
                result = runtime.request(proposal)
                self._json(200 if result.passed else 403, result.as_dict())
                return

```

```

        if path == "/api/exec":
            name = str(payload.get("name") or payload.get("action") or "")
            token = payload.get("token")
            token_s = str(token) if token else None
            args = payload.get("args") if isinstance(payload.get("args"), dict) else {}
            if "message" in payload and "message" not in args:
                args = dict(args)
                args["message"] = payload.get("message")
            try:
                out = runtime.run(name, token=token_s, args=args)
            except Exception as exc:
                self._json(
                    403,
                    {
                        "ok": False,
                        "error": str(exc),
                        "invite": invite_text(),
                    },
                )
                return
            self._json(200, {"ok": True, "result": out})
            return

        if path == "/api/halt":
            self._json(200, runtime.halt())
            return

        if path == "/api/revoke":
            token = str(payload.get("token") or "")
            try:
                runtime.lumen.revoke(token)
            except Exception as exc:
                self._json(403, {"ok": False, "error": str(exc), "invite": invite_text()})
                return
            self._json(200, {"ok": True, "revoked": True})
            return

        if path == "/api/purge":
            typed = str(payload.get("confirm") or "")
            ok = typed == "CONFIRM" or payload.get("confirm") is True
            if not ok:
                self._json(
                    400,
                    {
                        "ok": False,
                        "error": "type CONFIRM to purge the .azos session",
                        "invite": invite_text(),
                    },
                )
                return
            try:
                out = runtime.purge(confirm=True)
            except Exception as exc:
                self._json(400, {"ok": False, "error": str(exc)})
                return
            self._json(200, out)
            return

        self._json(404, {"error": "not found", "invite": invite_text()})

    return Handler

def make_server(
    host: str = DEFAULT_HOST,
    port: int = DEFAULT_PORT,
    runtime: Runtime | None = None,
) -> ThreadingHTTPServer:
    if host not in LOOPBACK:
        raise ValueError("AZ Interface binds loopback only (127.0.0.1)")
    rt = runtime if runtime is not None else Runtime()
    handler = make_handler(rt)
    return ThreadingHTTPServer((host, port), handler)

def serve(
    host: str = DEFAULT_HOST,
    port: int = DEFAULT_PORT,
    runtime: Runtime | None = None,
) -> None:
    httpd = make_server(host, port, runtime)
    bound_host, bound_port = httpd.server_address[:2]
    print(
        f"AZ Interface http://{bound_host}:{bound_port} "
        "(overlay control surface; loopback only; not a kernel)"
    )
    try:
        httpd.serve_forever()
    except KeyboardInterrupt:
        print("\nstopped")
    finally:
        httpd.server_close()

```

azos/azos/invite.py

```

"""Voluntary adoption. Propagation is invitation, not infection.

`azos invite` prints principles and a download URL. It does not copy
this overlay onto other machines, write outside cwd, or self-replicate.
"""

from __future__ import annotations

from typing import TextIO

DOWNLOAD_URL = "https://azos-download-tracker.vibelock.workers.dev/"
SOURCE_URL = "https://github.com/AzielEliab/azos"

PRINCIPLES = """AZ-OS ? voluntary overlay (not an infection)

AZ-OS is a portable folder and control surface. It is not a kernel,
not a bootloader, not a hypervisor, and not malware. You run it because
you choose to.

Principles
1. Integrity precedes execution.
   No module or action runs without a token from ARC.
2. Time-bound actions are final.
   Authorized executions append to an immutable sha256 chain. No rewrite.
3. Understanding precedes modification.
   Extending or loading a module requires an explicit comprehension
   checkbox and a short restatement of intent.
4. The system protects itself architecturally.
   Unsigned or unauthorized run() raises AuthorizationError. Default deny.
5. Propagation is not infection.
   This invite prints principles and a download URL. AZ-OS does not
   copy itself onto other machines.

You are invited to run AZ-OS yourself. This is not a silent block.
Adoption is voluntary.

Counted download:
{download}

Source:
{source}
""".strip()

def invite_text() -> str:
    return PRINCIPLES.format(download=DOWNLOAD_URL, source=SOURCE_URL) + "\n"

def emit_invite(stream: TextIO) -> None:
    """Print the invite. Writes nothing to the filesystem."""
    stream.write(invite_text())
    if not invite_text().endswith("\n"):
        stream.write("\n")

```

azos/azos/jsonio.py

```

"""Small JSON import/export for AZ-OS. Author: Aziel Eliab."""

from __future__ import annotations

import json
from pathlib import Path
from typing import Any

from azos import __version__

AUTHOR = "Aziel Eliab"
PRODUCT = "AZ-OS"
STATE_NAME = ".azos-state.json"

def _as_path(path: str | Path) -> Path:
    return Path(path)

def import_json(path: str | Path) -> dict[str, Any]:
    pth = _as_path(path)
    doc = json.loads(pth.read_text(encoding="utf-8"))
    if not isinstance(doc, dict):
        raise ValueError("JSON object required")
    dest = Path.cwd() / STATE_NAME
    dest.write_text(json.dumps(doc, indent=2, ensure_ascii=False) + "\n", encoding="utf-8")
    return {
        "ok": True,
        "imported": str(pth),
        "stored": str(dest),
        "keys": sorted(str(k) for k in doc.keys()),
        "author": AUTHOR,
        "product": PRODUCT,
        "version": __version__,
    }

def export_json(path: str | Path) -> dict[str, Any]:
    pth = _as_path(path)
    src = Path.cwd() / STATE_NAME
    payload: Any = {}
    if src.exists():
        try:
            payload = json.loads(src.read_text(encoding="utf-8"))
        except Exception:
            payload = {}
    doc = {
        "product": PRODUCT,
        "package": "azos",
        "version": __version__,
        "author": AUTHOR,
        "payload": payload,
    }
    pth.write_text(json.dumps(doc, indent=2, ensure_ascii=False) + "\n", encoding="utf-8")
    return {
        "ok": True,
        "exported": str(pth),
        "author": AUTHOR,
        "product": PRODUCT,
        "version": __version__,
    }

```

azos/azos/log.py

```

"""Append-only sha256 hash chain of authorized executions.

Time-bound actions are final. No rewrite, pop, insert, or delete.
"""

from __future__ import annotations

import hashlib
import json
from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Iterator

from azos.errors import AppendOnlyError

GENESIS_PREV = "0" * 64

def _utcnow() -> str:
    return datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")

def _canon(payload: dict[str, Any]) -> bytes:
    return json.dumps(payload, sort_keys=True, separators=(",", ":"), ensure_ascii=False).encode(
        "utf-8"
    )

def digest(payload: dict[str, Any]) -> str:
    return hashlib.sha256(_canon(payload)).hexdigest()

@dataclass(frozen=True)
class Entry:
    timestamp: str
    action: str
    token_hash: str
    payload: dict[str, Any]
    prev_hash: str
    hash: str

    def to_dict(self) -> dict[str, Any]:
        return {
            "timestamp": self.timestamp,
            "action": self.action,
            "token_hash": self.token_hash,
            "payload": self.payload,
            "prev_hash": self.prev_hash,
            "hash": self.hash,
        }

class ExecutionLog:
    """JSONL-backed chain. File is only ever opened in append mode."""

    def __init__(self, path: Path | str | None = None) -> None:
        self._path = Path(path) if path is not None else None
        self._entries: tuple[Entry, ...] = ()
        if self._path is not None and self._path.is_file():
            self._entries = tuple(self._read_file(self._path))

    @property
    def path(self) -> Path | None:
        return self._path

    def __len__(self) -> int:
        return len(self._entries)

    def __iter__(self) -> Iterator[Entry]:
        return iter(self._entries)

    def __getitem__(self, index: int) -> Entry:
        return self._entries[index]

    def _refuse(self, action: str) -> None:
        raise AppendOnlyError(
            f"cannot {action}: AZ-OS execution log is append-only; "
            "time-bound actions are final"
        )

    def pop(self, *args: object, **kwargs: object) -> None:
        self._refuse("pop")

    def insert(self, *args: object, **kwargs: object) -> None:
        self._refuse("insert")

    def remove(self, *args: object, **kwargs: object) -> None:
        self._refuse("remove")

    def clear(self) -> None:
        self._refuse("clear")

```



```

def reverse(self) -> None:
    self._refuse("reverse")

def __setitem__(self, *args: object, **kwargs: object) -> None:
    self._refuse("replace")

def __delitem__(self, *args: object, **kwargs: object) -> None:
    self._refuse("delete")

def append(
    self,
    *,
    action: str,
    token_hash: str,
    payload: dict[str, Any] | None = None,
) -> Entry:
    prev = self._entries[-1].hash if self._entries else GENESIS_PREV
    ts = _utcnow()
    body = payload if isinstance(payload, dict) else {}
    material = {
        "timestamp": ts,
        "action": action,
        "token_hash": token_hash,
        "payload": body,
        "prev_hash": prev,
    }
    entry = Entry(
        timestamp=ts,
        action=action,
        token_hash=token_hash,
        payload=body,
        prev_hash=prev,
        hash=digest(material),
    )
    if self._path is not None:
        self._path.parent.mkdir(parents=True, exist_ok=True)
        line = json.dumps(entry.to_dict(), sort_keys=True, separators=(",", ":"))
        with self._path.open("a", encoding="utf-8") as fh:
            fh.write(line)
            fh.write("\n")
            fh.flush()
    self._entries = self._entries + (entry,)
    return entry

def entries(self) -> tuple[Entry, ...]:
    return self._entries

def reload_empty(self) -> None:
    """In-memory reset after a session purge (the file is gone)."""
    self._entries = ()

@staticmethod
def _read_file(path: Path) -> list[Entry]:
    out: list[Entry] = []
    for line in path.read_text(encoding="utf-8").splitlines():
        line = line.strip()
        if not line:
            continue
        data = json.loads(line)
        out.append(
            Entry(
                timestamp=str(data.get("timestamp", "")),
                action=str(data.get("action", "")),
                token_hash=str(data.get("token_hash", "")),
                payload=dict(data.get("payload") or {}),
                prev_hash=str(data.get("prev_hash", "")),
                hash=str(data.get("hash", "")),
            )
        )
    return out

```

azos/azos/lumen.py

```

"""Lumen ? gatekeeper that keeps running after halt.

Watch loop is in-process. After halt, Lumen can still revoke tokens
and purge the session directory ``.azos/`` only.
"""

from __future__ import annotations

import threading
from typing import Any

from azos.errors import AzosError
from azos.paths import safe_purge

class Lumen:
    """Always-on gatekeeper for the overlay process."""

    def __init__(self, runtime: Any) -> None:
        self.runtime = runtime
        self._alive = False
        self._stop = threading.Event()
        self._thread: threading.Thread | None = None

    def start(self) -> None:
        if self._alive and self._thread is not None and self._thread.is_alive():
            return
        self._stop.clear()
        self._alive = True
        self._thread = threading.Thread(
            target=self._watch,
            name="azos-lumen",
            daemon=True,
        )
        self._thread.start()

    def _watch(self) -> None:
        # Heartbeat. Halt stops execution, not this loop.
        while not self._stop.is_set():
            self._stop.wait(0.25)

    @property
    def running(self) -> bool:
        return self._alive and not self._stop.is_set()

    def halt(self) -> dict[str, Any]:
        """Stop execution authority. Lumen itself keeps running."""
        self.runtime.mark_halted()
        return self.status()

    def revoke(self, token: str) -> None:
        """Revoke still works after halt."""
        self.runtime.arc.revoke(token)

    def revoke_all(self) -> int:
        return self.runtime.arc.revoke_all()

    def purge_session(self, *, confirm: bool) -> dict[str, Any]:
        """Delete .azos only. Works after halt. Never $HOME or OS files."""
        if not confirm:
            raise AzosError("purge requires --confirm (or typed CONFIRM in the UI)")
        self.runtime.arc.revoke_all()
        target = safe_purge(self.runtime.session_dir)
        self.runtime.log.reload_empty()
        # Do not persist state: that would recreate .azos after purge.
        self.runtime._halted = False
        return {
            "purged": True,
            "session": str(target),
            "lumen": "running" if self.running else "stopped",
        }

    def status(self) -> dict[str, Any]:
        return {
            "lumen": "running" if self.running else "stopped",
            "halted": bool(self.runtime.halted),
            "session": str(self.runtime.session_dir),
        }

    def stop(self) -> None:
        """End the watch loop (process teardown). Not the same as halt."""
        self._alive = False
        self._stop.set()

```

azos/azos/paths.py

```

"""Session directory helpers. Purge touches .azos only."""

from __future__ import annotations

from pathlib import Path

from azos.errors import AzosError

SESSION_DIRNAME = ".azos"

# Parents we will never delete a session from (OS paths, not user homes).
_PROTECTED_PARENTS = frozenset(
    {
        Path("/"),
        Path("/etc"),
        Path("/usr"),
        Path("/bin"),
        Path("/sbin"),
        Path("/boot"),
        Path("/sys"),
        Path("/proc"),
        Path("/dev"),
        Path("/lib"),
        Path("/lib64"),
        Path("/opt"),
        Path("/var"),
        Path("/root"),
        Path("/tmp"),
    }
)

def session_dir(root: Path | str | None = None) -> Path:
    """Return <root>/.azos (default: cwd). Does not create it."""
    base = Path(root) if root is not None else Path.cwd()
    return base / SESSION_DIRNAME

def safe_purge(target: Path | str) -> Path:
    """Delete a .azos session directory only. Never $HOME or OS files.

    Raises AzosError rather than touching anything else.
    """
    path = Path(target).resolve()
    home = Path.home().resolve()
    if path == home:
        raise AzosError("refusing to purge $HOME")
    if path == Path("/").resolve():
        raise AzosError("refusing to purge filesystem root")
    if path.name != SESSION_DIRNAME:
        raise AzosError(
            f"purge only deletes a {SESSION_DIRNAME} session directory, not {path}"
        )
    if path.parent in _PROTECTED_PARENTS or path in _PROTECTED_PARENTS:
        raise AzosError(f"refusing to purge an OS path: {path}")
    if path.exists():
        if not path.is_dir():
            raise AzosError(f"session path is not a directory: {path}")
        import shutil

        shutil.rmtree(path)
    return path

```

azos/azos/runtime.py

```

"""Composition root: ARC + Lumen + gate + exec + log."""

from __future__ import annotations

import json
from dataclasses import dataclass
from pathlib import Path
from typing import Any

from azos.arc import ARC
from azos.exec import SAFE_ACTIONS, Executor
from azos.gate import Proposal, authorize
from azos.invite import invite_text
from azos.log import ExecutionLog
from azos.lumen import Lumen
from azos.paths import SESSION_DIRNAME

__all__ = ["Runtime", "RequestResult", "SAFE_ACTIONS"]

@dataclass
class RequestResult:
    passed: bool
    token: str | None
    gates: dict[str, Any]
    invite: str | None

    def as_dict(self) -> dict[str, Any]:
        preview = None
        if self.token:
            preview = self.token[:8] + "?"
        return {
            "passed": self.passed,
            "token": self.token,
            "token_preview": preview,
            "gates": self.gates,
            "invite": self.invite,
        }

class Runtime:
    """One overlay session rooted at ``<root>/./azos``."""

    def __init__(
        self,
        root: Path | str | None = None,
        *,
        start_lumen: bool = True,
    ) -> None:
        self.root = Path(root) if root is not None else Path.cwd()
        self.session_dir = self.root / SESSION_DIRNAME
        self.arc = ARC(self.session_dir)
        self.log = ExecutionLog(self.session_dir / "exec.jsonl")
        self._halted = False
        self._load_state()
        self.lumen = Lumen(self)
        self.executor = Executor(self)
        if start_lumen:
            self.lumen.start()

    @property
    def halted(self) -> bool:
        return self._halted

    def mark_halted(self, value: bool = True) -> None:
        self._halted = bool(value)
        self._save_state()

    def request(self, proposal: Proposal) -> RequestResult:
        """Five gates then (only on PASS) an ARC token. FAIL ? invite, no token."""
        if self._halted:
            return RequestResult(
                passed=False,
                token=None,
                gates={
                    "definition": {"pass": False, "reason": "overlay is halted"},
                    "evidence": {"pass": False, "reason": "overlay is halted"},
                    "impact": {"pass": False, "reason": "overlay is halted"},
                    "integrity": {"pass": False, "reason": "overlay is halted"},
                    "responsibility": {"pass": False, "reason": "overlay is halted"},
                },
                invite=invite_text(),
            )
        result = authorize(proposal, SAFE_ACTIONS)
        gates = {
            name: {"pass": check.passed, "reason": check.reason}
            for name, check in result.gates.items()
        }
        if not result.passed:
            return RequestResult(
                passed=False,
                token=None,

```

```

        gates=gates,
        invite=invite_text(),
    )
    token = self.arc.issue(action=proposal.action, actor=proposal.actor)
    return RequestResult(passed=True, token=token, gates=gates, invite=None)

def run(
    self,
    name: str,
    token: str | None = None,
    args: dict[str, Any] | None = None,
) -> dict[str, Any]:
    return self.executor.run(name, token=token, args=args)

def status(self) -> dict[str, Any]:
    arc = self.arc.snapshot()
    return {
        "overlay": "AZ-OS",
        "interface": "AZ Interface",
        "version": _version(),
        "session": str(self.session_dir),
        "halted": self._halted,
        "lumen": "running" if self.lumen.running else "stopped",
        "tokens": arc,
        "log_length": len(self.log),
        "builtins": sorted(SAFE_ACTIONS),
        "kernel": False,
        "malware": False,
    }

def halt(self) -> dict[str, Any]:
    return self.lumen.halt()

def purge(self, *, confirm: bool) -> dict[str, Any]:
    return self.lumen.purge_session(confirm=confirm)

def _state_path(self) -> Path:
    return self.session_dir / "state.json"

def _load_state(self) -> None:
    path = self._state_path()
    if not path.is_file():
        return
    try:
        data = json.loads(path.read_text(encoding="utf-8"))
    except (OSError, json.JSONDecodeError):
        return
    if isinstance(data, dict):
        self._halted = bool(data.get("halted"))

def _save_state(self) -> None:
    self.session_dir.mkdir(parents=True, exist_ok=True)
    payload = {"halted": self._halted, "lumen": "running"}
    self._state_path().write_text(
        json.dumps(payload, indent=2) + "\n", encoding="utf-8"
    )

def _version() -> str:
    from azos import __version__

    return __version__

```

azos/azos/templates/ui.html

```

<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>AZ-OS ? AZ Interface</title>
  <style>
    :root {
      --bg: #0a0c10;
      --panel: #12151c;
      --ink: #e6e8ee;
      --muted: #8b93a7;
      --line: #2a3140;
      --arc: #e4b84a;
      --lumen: #5ee0d0;
      --pass: #6fdd8b;
      --fail: #ff6b6b;
      --halt: #ff8a5b;
      --btn: #e4b84a;
    }
    * { box-sizing: border-box; }
    html, body { margin: 0; padding: 0; background: var(--bg); color: var(--ink); }
    body {
      font-family: ui-sans-serif, system-ui, -apple-system, "Segoe UI", sans-serif;
      line-height: 1.45;
    }
    header, main, footer { max-width: 52rem; margin: 0 auto; padding: 1.25rem 1.4rem; }
    header { padding-top: 1.75rem; }
    .tag {
      font-size: .72rem; letter-spacing: .14em; text-transform: uppercase;
      color: var(--arc); font-weight: 650;
    }
    h1 { font-size: 1.85rem; margin: .35rem 0 .4rem; letter-spacing: .01em; }
    .lede { color: var(--muted); margin: 0 0 1rem; }
    .overlay {
      border: 1px solid var(--line); background: var(--panel);
      border-radius: 10px; padding: .9rem 1.05rem; color: var(--muted); font-size: .95rem;
    }
    .overlay strong { color: var(--ink); }
    .statusbar {
      display: flex; flex-wrap: wrap; gap: .6rem 1.1rem; align-items: center;
      margin: 1.1rem 0 0; font-size: .9rem;
    }
    .pill {
      display: inline-flex; align-items: center; gap: .4rem;
      border: 1px solid var(--line); border-radius: 999px;
      padding: .2rem .7rem; background: #0e1118;
    }
    .dot { width: .55rem; height: .55rem; border-radius: 50%; background: var(--muted); }
    .dot.on { background: var(--lumen); box-shadow: 0 0 8px var(--lumen); }
    .dot.halt { background: var(--halt); }
    .dot.ok { background: var(--pass); }
    .dot.no { background: var(--fail); }
    h2 { font-size: 1.05rem; margin: 1.6rem 0 .6rem; letter-spacing: .04em; text-transform: uppercase; color: var(--muted); }
    .card {
      background: var(--panel); border: 1px solid var(--line);
      border-radius: 10px; padding: 1rem 1.1rem; margin: .7rem 0;
    }
    label { display: block; margin: .55rem 0 .25rem; font-size: .82rem; color: var(--muted); }
    label span { color: var(--ink); font-weight: 600; }
    input[type=text], textarea, select {
      width: 100%; background: #0b0f14; color: var(--ink);
      border: 1px solid var(--line); border-radius: 7px;
      padding: .5rem .65rem; font: inherit;
    }
    textarea { min-height: 4.2rem; resize: vertical; }
    .row { display: flex; gap: .6rem; flex-wrap: wrap; margin: .7rem 0; }
    button {
      background: var(--btn); color: #141108; border: 0; border-radius: 7px;
      padding: .55rem .95rem; font-weight: 700; cursor: pointer; font-size: .95rem;
    }
    button.secondary { background: #2a3340; color: var(--ink); font-weight: 600; }
    button.danger { background: var(--fail); color: #1a0505; }
    button.disabled { opacity: .4; cursor: not-allowed; }
    .gates { display: grid; grid-template-columns: 1fr; gap: .45rem; }
    .gate {
      display: grid; grid-template-columns: 7.5rem 3.2rem 1fr;
      gap: .5rem; align-items: baseline; font-size: .9rem;
    }
    .gate.name { text-transform: uppercase; letter-spacing: .06em; color: var(--muted); font-size: .75rem; }
    .pass { color: var(--pass); font-weight: 700; }
    .fail { color: var(--fail); font-weight: 700; }
    .token {
      font-family: ui-monospace, SFMono-Regular, Menlo, monospace;
      word-break: break-all; font-size: .85rem;
    }
    .invite {
      border-color: var(--arc);
      background: linear-gradient(180deg, #1a160c 0%, var(--panel) 40%);
    }
    .invite h3 { margin: 0 0 .4rem; color: var(--arc); font-size: 1rem; }

```

```

pre {
  white-space: pre-wrap; font-size: .82rem; color: var(--ink);
  font-family: ui-sans-serif, system-ui, sans-serif; margin: 0;
}
.log {
  font-family: ui-monospace, SFMono-Regular, Menlo, monospace;
  font-size: .78rem; max-height: 12rem; overflow: auto;
  background: #0b0f14; border: 1px solid var(--line); border-radius: 8px;
  padding: .7rem .8rem;
}
.check { display: flex; gap: .5rem; align-items: flex-start; margin: .6rem 0; color: var(--muted); font-size: .9rem; }
m; }
.check input { margin-top: .2rem; }
.extend { display: none; }
.extend.show { display: block; }
.footer { color: var(--muted); font-size: .82rem; padding-bottom: 2.5rem; }
.kicker { color: var(--lumen); font-size: .75rem; letter-spacing: .08em; text-transform: uppercase; }
#msg { min-height: 1.2rem; color: var(--muted); font-size: .9rem; }
</style>
</head>
<body>
<header>
  <div class="tag">AZ-OS ? AZ Interface ? v0.1.0 ? loopback</div>
  <h1>AZ-OS + AZ Interface</h1>
  <p class="lede">Integrity precedes execution. The Interface is the control surface.</p>
  <div class="overlay">
    <strong>This is an overlay, not a kernel.</strong>
    AZ-OS is a portable folder and a local control surface. It does not boot hardware,
    replace your operating system, load a hypervisor, or install a bootloader.
    Nothing executes without an ARC token issued after five gates pass.
    Propagation is invitation, not infection.
  </div>
  <div class="statusbar" id="statusbar">
    <span class="pill"><span class="dot" id="dot-lumen"></span> Lumen <strong id="lumen-label">?</strong></span>
    <span class="pill"><span class="dot" id="dot-halt"></span> Halt <strong id="halt-label">?</strong></span>
    <span class="pill"><span class="dot" id="dot-token"></span> Token <strong id="token-label">none</strong></span>
    <span class="pill">Log <strong id="log-label">0</strong></span>
  </div>
</header>
<main>
  <h2>Request execution</h2>
  <form class="card" id="request-form" autocomplete="off">
    <label for="action"><span>Action</span> ? registered safe builtin</label>
    <select id="action" name="action">
      <option value="echo">echo</option>
      <option value="list_modules">list_modules</option>
      <option value="status">status</option>
      <option value="purge_session">purge_session</option>
    </select>
    <label for="definition"><span>Definition</span> ? what exactly is proposed?</label>
    <textarea id="definition" name="definition" placeholder="Print a bounded status string from a registered builtin.">
  </textarea>
    <label for="evidence"><span>Evidence</span> ? why is this justified?</label>
    <textarea id="evidence" name="evidence" placeholder="Action is on the closed SAFE_ACTIONS list in azos.exec.">
  </textarea>
    <label for="impact"><span>Impact</span> ? what will change?</label>
    <textarea id="impact" name="impact" placeholder="In-process result only. No host disk, no other machines.">
  </textarea>
    <label for="actor"><span>Responsibility</span> ? who owns the outcome? (root/user is a name, not a privilege)</label>
    <input id="actor" name="actor" type="text" placeholder="operator" spellcheck="false">
    <label class="check">
      <input type="checkbox" id="extend_module">
      <span>I am extending / loading a new module (requires comprehension).</span>
    </label>
    <div class="extend" id="extend-fields">
      <label class="check">
        <input type="checkbox" id="comprehension">
        <span>Comprehension: I understand this module and its bounds. Understanding precedes modification.</span>
      </label>
      <label for="intent"><span>Restate intent</span> ? short restatement in your own words</label>
      <textarea id="intent" placeholder="I want this overlay to run only the named builtin, nothing else.">
    </div>
    <div class="row">
      <button type="submit" id="request-btn">Request execution</button>
      <label>Import JSON <input type="file" id="import-json" accept="application/json,.json"></label>
      <button type="button" class="secondary" id="export-json">Export JSON</button>
    </div>
  </form>

  <div class="card" id="gate-card">
    <div class="kicker">Five-gate result</div>
    <div class="gates" id="gates">
      <p class="lede">Submit a request. Default is deny. A FAIL issues an invite, not a silent block.</p>
    </div>
    <p class="token" id="token-box"></p>
  </div>

  <h2>Authorized actions</h2>
  <div class="card">
    <p class="lede">Buttons enable only with a live ARC token. Halt freezes execution; Lumen keeps running.</p>
    <div class="row">
      <button type="button" id="btn-list" disabled>list_modules</button>
      <button type="button" id="btn-echo" disabled>echo</button>
      <button type="button" id="btn-status" disabled>status</button>
    </div>
  </div>

```

```

</div>
<label for="echo-msg">echo message (never a shell)</label>
<input id="echo-msg" type="text" placeholder="ok" maxlength="200">
<p id="msg"></p>
<div class="row">
  <button type="button" class="secondary" id="btn-halt">Halt</button>
  <button type="button" class="secondary" id="btn-re
voke" disabled>Revoke token</button>
</div>
</div>

<h2>Purge session</h2>
<div class="card">
  <p class="lede">Deletes the <code>.azos</code> session directory only. Never <code>$HOME</code>, never OS files.
Type CONFIRM.</p>
  <label for="confirm">Type CONFIRM</label>
  <input id="confirm" type="text" placeholder="CONFIRM" spellcheck="false">
  <div class="row">
    <button type="button" class="danger" id="btn-purge">Purge .azos</button>
  </div>
</div>
</div>

<h2>Invite</h2>
<div class="card invite" id="invite-card">
  <h3>Unauthorized environments receive an invite ? not a silent error.</h3>
  <pre id="invite-text">Loading invite?</pre>
</div>

<h2>Execution log</h2>
<div class="card">
  <p class="lede">Append-only sha256 chain. Time-bound actions are final. No rewrite.</p>
  <div class="log" id="log"></div>
</div>
</main>
<footer>
  <p><strong>Integrity precedes execution.</strong> Overlay, not a kernel. Not malware. Propagation is invitation.</p>
  <p>Apache-2.0 ? Aziel Eliab ? 2026 ? Bound to 127.0.0.1:8800 ? Forks welcome and always allowed.</p>
</footer>
<script>
(function () {
  const $ = (id) => document.getElementById(id);
  let token = null;

  function setMsg(text, ok) {
    const el = $("#msg");
    el.textContent = text || "";
    el.style.color = ok === false ? "var(--fail)" : "var(--muted)";
  }

  function enableActions(on) {
    ["btn-list", "btn-echo", "btn-status", "btn-revoke"].forEach((id) => {
      $(id).disabled = !on;
    });
  }

  function renderGates(gates, passed, issued) {
    const names = ["definition", "evidence", "impact", "integrity", "responsibility"];
    const box = $("#gates");
    box.innerHTML = "";
    names.forEach((name) => {
      const g = (gates && gates[name]) || { pass: false, reason: "not evaluated" };
      const row = document.createElement("div");
      row.className = "gate";
      row.innerHTML =
        '<span class="name">' + name + "</span>" +
        '<span class="' + (g.pass ? "pass" : "fail") + '>' + (g.pass ? "PASS" : "FAIL") + "</span>" +
        "<span>" + (g.reason || "") + "</span>";
      box.appendChild(row);
    });
    const tb = $("#token-box");
    if (passed && issued) {
      tb.textContent = "Token issued (present once): " + issued;
    } else if (gates) {
      tb.textContent = "Blocked. No token. See invite below.";
    } else {
      tb.textContent = "";
    }
  }

  async function refresh() {
    const st = await (await fetch("/api/status")).json();
    $("#lumen-label").textContent = st.lumen || "unknown";
    $("#dot-lumen").className = "dot " + (st.lumen === "running" ? "on" : "");
    $("#halt-label").textContent = st.halted ? "halted" : "live";
    $("#dot-halt").className = "dot " + (st.halted ? "halt" : "ok");
    const active = st.tokens && st.tokens.active;
    $("#token-label").textContent = token ? "live (this session)" : (active ? "active in .azos" : "none");
    $("#dot-token").className = "dot " + ((token || active) ? "ok" : "no");
    $("#log-label").textContent = String(st.log_length || 0);
    enableActions(Boolean(token) && !st.halted);
    const log = await (await fetch("/api/log")).json();
    const lines = (log.entries || []).map(function (e) {
      return (e.hash || "").slice(0, 12) + " " + e.action + " " + e.timestamp;
    });
    $("#log").textContent = lines.length ? lines.join("\n") : "empty (append-only once executions exist)";
  }
}

```



```

    }

    $("extend_module").addEventListener("change", function () {
        $("extend-fields").classList.toggle("show", $("extend_module").checked);
    });

    $("request-form").addEventListener("submit", async function (ev) {
        ev.preventDefault();
        const body = {
            action: $("action").value,
            definition: $("definition").value,
            evidence: $("evidence").value,
            impact: $("impact").value,
            actor: $("actor").value,
            extend_module: $("extend_module").checked,
            comprehension: $("comprehension").checked,
            intent: $("intent").value
        };
        const res = await fetch("/api/request", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(body)
        });
        const data = await res.json();
        token = data.token || null;
        renderGates(data.gates, data.passed, data.token);
        if (data.invite) {
            $("invite-text").textContent = data.invite;
        }
        setMsg(data.passed ? "Token issued. Actions enabled." : "Blocked ? invite shown, not a silent error.", data.passed
    );
    });
    await refresh();
});

async function run(name) {
    if (!token) {
        setMsg("No live token. Request execution first ? or see the invite.", false);
        return;
    }
    const payload = { name: name, token: token };
    if (name === "echo") payload.message = $("echo-msg").value || "ok";
    const res = await fetch("/api/exec", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(payload)
    });
    const data = await res.json();
    if (!data.ok) {
        setMsg(data.error || "unauthorized", false);
        if (data.invite) $("invite-text").textContent = data.invite;
    } else {
        setMsg(JSON.stringify(data.result), true);
    }
    await refresh();
}

$("btn-list").addEventListener("click", function () { run("list_modules"); });
$("btn-echo").addEventListener("click", function () { run("echo"); });
$("btn-status").addEventListener("click", function () { run("status"); });

$("btn-halt").addEventListener("click", async function () {
    await fetch("/api/halt", { method: "POST", headers: { "Content-Type": "application/json" }, body: "{}" });
    enableActions(false);
    setMsg("Halted. Lumen still running. Revoke and purge remain available.");
    await refresh();
});

$("btn-revoke").addEventListener("click", async function () {
    if (!token) return;
    const res = await fetch("/api/revoke", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ token: token })
    });
    const data = await res.json();
    token = null;
    enableActions(false);
    setMsg(data.ok ? "Token revoked." : (data.error || "revoke failed"), data.ok);
    await refresh();
});

$("btn-purge").addEventListener("click", async function () {
    const typed = $("confirm").value;
    const res = await fetch("/api/purge", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ confirm: typed })
    });
    const data = await res.json();
    if (!res.ok || data.ok === false) {
        setMsg(data.error || "purge refused", false);
        return;
    }
    token = null;
    enableActions(false);

```

```

    $("confirm").value = "";
    setMsg("Session .azos purged. Lumen still running.");
    await refresh();
  });

  fetch("/api/invite").then(function (r) { return r.json(); }).then(function (d) {
    $("invite-text").textContent = d.invite || "";
  });
  var lastRequest = null;
  document.getElementById("export-json").addEventListener("click", function () {
    const payload = lastRequest || {
      action: $("action").value,
      definition: $("definition").value,
      evidence: $("evidence").value,
      impact: $("impact").value,
      actor: $("actor").value,
      extend_module: $("extend_module").checked,
      comprehension: $("comprehension").checked,
      intent: $("intent").value
    };
    const blob = new Blob([JSON.stringify(payload, null, 2)], { type: "application/json" });
    const a = document.createElement("a");
    a.href = URL.createObjectURL(blob);
    a.download = "azos-request.json";
    a.click();
    URL.revokeObjectURL(a.href);
  });
  const importEl = document.getElementById("import-json");
  if (importEl) importEl.addEventListener("change", function () {
    const f = importEl.files && importEl.files[0];
    if (!f) return;
    const reader = new FileReader();
    reader.onload = function () {
      let obj;
      try { obj = JSON.parse(String(reader.result || "{}")); } catch (e) { return; }
      const r = obj.request || obj;
      if (r.action) $("action").value = r.action;
      if (r.definition) $("definition").value = r.definition;
      if (r.evidence) $("evidence").value = r.evidence;
      if (r.impact) $("impact").value = r.impact;
      if (r.actor) $("actor").value = r.actor;
      if (r.intent) $("intent").value = r.intent;
      lastRequest = r;
    };
    reader.readAsText(f);
  });
  const origSubmit = $("request-form").onsubmit;
  $("request-form").addEventListener("submit", function () {
    lastRequest = {
      action: $("action").value,
      definition: $("definition").value,
      evidence: $("evidence").value,
      impact: $("impact").value,
      actor: $("actor").value,
      extend_module: $("extend_module").checked,
      comprehension: $("comprehension").checked,
      intent: $("intent").value
    };
  });
  refresh();
})();
</script>
</body>
</html>

```

azos/install.sh

```
#!/usr/bin/env bash
# AZ-OS one-click install. Counted download via this project's Worker.
# Usage: curl -fsSL https://azos-download-tracker.vibelock.workers.dev/install.sh | bash
set -euo pipefail

HOST="${AZOS_HOME_HOST:-https://azos-download-tracker.vibelock.workers.dev}"
ASSET="${AZOS_HOME_ASSET:-azos-0.1.0.tar.gz}"
WORKDIR="${AZOS_HOME:-$HOME/azos}"

mkdir -p "$WORKDIR"
cd "$WORKDIR"

echo "Downloading counted tarball from ${HOST}/download (User-Agent Mozilla/5.0)?"
curl -fsSL -A 'Mozilla/5.0' "${HOST}/download?asset=${ASSET}" -o "${ASSET}"

tar -xzf "${ASSET}"
DIR="$(find . -maxdepth 1 -type d -name 'azos-*' | head -n 1)"
if [ -n "$DIR" ]; then
  cd "$DIR"
fi

python3 -m venv .venv
# shellcheck disable=SC1091
. .venv/bin/activate
python -m pip install -U pip
python -m pip install -e .

echo
echo "Installed AZ-OS."
echo "Run: azos ui"
echo "Then open http://127.0.0.1:8800 (loopback only)"
echo "Author: Aziel Eliab."
```

azos/mobile/lib/main.dart

```

import 'dart:math';

import 'package:flutter/material.dart';

import 'theme.dart';

const invite = '''AZ-OS ? voluntary overlay (not an infection)

AZ-OS is a portable folder and control surface. It is not a kernel,
not a bootloader, not a hypervisor, and not malware. You run it because
you choose to.

Principles
1. Integrity precedes execution.
  No module or action runs without a token from ARC.
2. Time-bound actions are final.
  Authorized executions append to an immutable sha256 chain. No rewrite.
3. Understanding precedes modification.
  Extending or loading a module requires an explicit comprehension
  checkbox and a short restatement of intent.
4. The system protects itself architecturally.
  Unsigned or unauthorized run() raises AuthorizationError. Default deny.
5. Propagation is not infection.
  This invite prints principles and a download URL. AZ-OS does not
  copy itself onto other machines.

You are invited to run AZ-OS yourself. This is not a silent block.
Adoption is voluntary.

Counted download:
https://azos-download-tracker.vibelock.workers.dev/

Source:
https://github.com/AzielEliab/azos
''';

void main() {
  runApp(const AzosApp());
}

class AzosApp extends StatelessWidget {
  const AzosApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'AZ-OS',
      debugShowCheckedModeBanner: false,
      theme: buildAppTheme(),
      home: const ControlPage(),
    );
  }
}

class ControlPage extends StatefulWidget {
  const ControlPage({super.key});

  @override
  State<ControlPage> createState() => _ControlPageState();
}

class _ControlPageState extends State<ControlPage> {
  String? _token;
  bool _halted = false;
  String _status = 'idle ? no token';
  bool _showInvite = true;

  void _issue() {
    if (_halted) {
      setState(() => _status = 'halted: new tokens refused');
      return;
    }
    final r = Random.secure();
    final bytes = List<int>.generate(32, (_) => r.nextInt(256));
    setState(() {
      _token = bytes.map((b) => b.toRadixString(16).padLeft(2, '0')).join();
      _status = 'token issued (in-memory). Integrity precedes execution.';
    });
  }

  void _revoke() {
    setState(() {
      _token = null;
      _status = 'token revoked (label). Default deny.';
    });
  }

  void _halt() {
    setState(() {
      _halted = true;
      _token = null;
      _status = 'HALT. Lumen watch remains a label: running. Tokens revoked.';
    });
  }
}

```

```

    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('AZ-OS')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Card(
            color: const Color(0xFF2A1515),
            child: const Padding(
              padding: EdgeInsets.all(12),
              child: Text(
                'NOT a kernel. NOT a worm. NOT malware. This is a control surface '
                'for a portable overlay. Propagation is invitation, not infection.',
                style: TextStyle(height: 1.4),
              ),
            ),
          ),
          const SizedBox(height: 8),
          const Text(
            'Integrity precedes execution.',
            style: TextStyle(color: kGold, fontStyle: FontStyle.italic, fontSize: 16),
          ),
          const SizedBox(height: 16),
          Text('Status: $_status'),
          Text(
            _token == null ? 'token: none' : 'token: ${_token!.substring(0, 16)}? (hashed-at-rest analogue: not shown
full)',
            style: const TextStyle(fontFamily: 'monospace', fontSize: 12),
          ),
          Text('halted: $_halted lumen: running (watch loop is a label on this phone)'),
          const SizedBox(height: 12),
          Wrap(
            spacing: 8,
            runSpacing: 8,
            children: [
              FilledButton(onPressed: _issue, child: const Text('Issue token')),
              OutlinedButton(onPressed: _revoke, child: const Text('Revoke')),
              OutlinedButton(onPressed: _halt, child: const Text('Halt')),
              OutlinedButton(
                onPressed: () => setState(() => _showInvite = !_showInvite),
                child: const Text('Invite'),
              ),
            ],
          ),
          const SizedBox(height: 8),
          const Text(
            'Invite writes no files. Halt/revoke are labels on in-memory state. '
            'This phone app does not execute host modules, does not copy itself, '
            'and does not wipe a disk.',
            style: TextStyle(color: kGoldDim, fontSize: 12),
          ),
          if (_showInvite) ...[
            const SizedBox(height: 16),
            Card(
              child: Padding(
                padding: const EdgeInsets.all(12),
                child: SelectableText(
                  invite,
                  style: const TextStyle(fontFamily: 'monospace', fontSize: 12, height: 1.4),
                ),
              ),
            ),
          ],
        ],
      ),
    );
  }
}

```

azos/mobile/lib/theme.dart

```

import 'package:flutter/material.dart';

/// Matte black + gold Material 3 dark theme. No analytics.
const Color kMatteBlack = Color(0xFF0B0B0B);
const Color kSurface = Color(0xFF141414);
const Color kGold = Color(0xFFC9A227);
const Color kGoldDim = Color(0xFF8A7219);
const Color kIvory = Color(0xFFE8E0D0);

ThemeData buildAppTheme() {
  const scheme = ColorScheme.dark(
    brightness: Brightness.dark,
    primary: kGold,
    onPrimary: kMatteBlack,
    secondary: kGoldDim,
    onSecondary: kIvory,
    surface: kSurface,
    onSurface: kIvory,
    error: Color(0xFFB54A4A),
    onError: kIvory,
  );
  return ThemeData(
    useMaterial3: true,
    brightness: Brightness.dark,
    colorScheme: scheme,
    scaffoldBackgroundColor: kMatteBlack,
    appBarTheme: const AppBarTheme(
      backgroundColor: kMatteBlack,
      foregroundColor: kGold,
      elevation: 0,
      centerTitle: false,
    ),
    cardTheme: CardThemeData(
      color: kSurface,
      elevation: 0,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
        side: const BorderSide(color: Color(0x33C9A227)),
      ),
    ),
    inputDecorationTheme: InputDecorationTheme(
      filled: true,
      fillColor: const Color(0xFF1A1A1A),
      border: OutlineInputBorder(borderRadius: BorderRadius.circular(10)),
      focusedBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(10),
        borderSide: const BorderSide(color: kGold),
      ),
    ),
    segmentedButtonTheme: SegmentedButtonThemeData(
      style: ButtonStyle(
        foregroundColor: WidgetStateProperty.resolveWith((s) {
          return s.contains(WidgetState.selected) ? kMatteBlack : kIvory;
        }),
        backgroundColor: WidgetStateProperty.resolveWith((s) {
          return s.contains(WidgetState.selected) ? kGold : kSurface;
        }),
      ),
    ),
  );
}

```

azos/mobile/pubspec.yaml

```
name: azos
description: Control surface: invite, halt, revoke labels. Integrity precedes execution. Not a kernel.
publish_to: "none"
version: 0.1.0+1

environment:
  sdk: ">=3.3.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter
  crypto: ^3.0.3

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0

flutter:
  uses-material-design: true
```

azos/pyproject.toml

```

[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "azos"
version = "0.1.0"
description = "AZ-OS + AZ Interface: ethical overlay framework. Integrity precedes execution. Not a kernel, not malware."
readme = "README.md"
license = {file = "LICENSE"}
authors = [{name = "Aziel Eliab"}]
requires-python = ">=3.10"
keywords = [
    "az-os",
    "overlay",
    "authorization",
    "integrity",
    "gates",
    "append-only",
    "invite",
]
classifiers = [
    "Development Status :: 4 - Beta",
    "License :: OSI Approved :: Apache Software License",
    "Programming Language :: Python :: 3",
    "Programming Language :: Python :: 3.10",
    "Programming Language :: Python :: 3.11",
    "Programming Language :: Python :: 3.12",
    "Programming Language :: Python :: 3.13",
    "Topic :: Security",
    "Intended Audience :: Developers",
]
dependencies = []

[project.optional-dependencies]
dev = ["pytest>=7.4"]

[project.scripts]
azos = "azos.cli:main"

[project.urls]
Homepage = "https://github.com/AzielEliab/azos"
Download = "https://azos-download-tracker.vibelock.workers.dev/"
Issues = "https://github.com/AzielEliab/azos/issues"

[tool.setuptools.packages.find]
include = ["azos*"]

[tool.setuptools.package-data]
azos = ["templates/*.html"]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]
addopts = "-q"

```


azos/workers/download-tracker/package.json

```
{
  "dependencies": {
    "wrangler": "^4.127.1"
  }
}
```

azos/workers/download-tracker/src/index.js

```

import { handleRuntime } from "../runtime.js";

/**
 * AZ-OS download tracker (Cloudflare Worker).
 *
 * GET /download?repo=AzielEliab/azos&tag=latest&asset=...
 *      increments KV, serves gzip via ASSETS.fetch (no 302)
 * GET /count JSON {project, total} for THIS project only
 * GET /stats JSON totals + per-repo + per-branch breakdown
 *
 * Homepage prints a live download count next to the download link.
 * Counter is isolated to azos (this Worker + its KV), not VibeLock.
 *
 * Motto: Integrity precedes execution.
 *
 * KV binding DOWNLOADS. Keys: project|owner|repo|branch|fork
 * CORS *. No secrets in this tree.
 */

const PROJECT = "azos";
const DEFAULT_ASSET = "azos-0.1.0.tar.gz";
const DEFAULT_OWNER = "AzielEliab";
const DEFAULT_REPO = "azos";
const DEFAULT_BRANCH = "main";
const GITHUB_RELEASES = "https://github.com/AzielEliab/azos/releases";
const GITHUB_LATEST = "https://github.com/AzielEliab/azos/releases/latest";
const HOST = "https://azos-download-tracker.vibelock.workers.dev";
const GITHUB_REPO = "https://github.com/AzielEliab/azos";
const INSTALL_LINE = "curl -fsSL https://azos-download-tracker.vibelock.workers.dev/install.sh | bash";
const DOI = "https://doi.org/10.5281/zenodo.21431711";
const ZENODO = "https://zenodo.org/records/21431711";

const MOTTO = "Integrity precedes execution.";

function corsHeaders() {
  return {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type",
  };
}

function json(body, status = 200) {
  return new Response(JSON.stringify(body, null, 2), {
    status,
    headers: { "Content-Type": "application/json; charset=utf-8", ...corsHeaders() },
  });
}

function redirect(url) {
  return new Response(null, {
    status: 302,
    headers: { Location: url, ...corsHeaders() },
  });
}

function splitOwnerRepo(value, fallbackOwner, fallbackRepo) {
  if (typeof value === "string" && value.includes("/")) {
    const [o, r] = value.split("/").filter(Boolean);
    if (o && r) return { owner: o, repo: r };
  }
  return { owner: fallbackOwner, repo: fallbackRepo };
}

function parseDims(src) {
  const get = (k) => {
    if (src == null) return null;
    if (typeof src.get === "function") {
      const v = src.get(k);
      return v == null || v === "" ? null : v;
    }
    const v = src[k];
    return v == null || v === "" ? null : v;
  };

  let owner = get("owner") || DEFAULT_OWNER;
  let repo = get("repo") || DEFAULT_REPO;
  if (typeof repo === "string" && repo.includes("/")) {
    const split = splitOwnerRepo(repo, owner, DEFAULT_REPO);
    owner = split.owner;
    repo = split.repo;
  }

  const branch = get("branch") || DEFAULT_BRANCH;
  const tag = get("tag") || "latest";
  const asset = get("asset") || "";

  const forkRaw = get("fork");
  let fork = "0";
  if (forkRaw === 1 || forkRaw === true || forkRaw === "1" || forkRaw === "true") {
    fork = "1";
  } else if (typeof forkRaw === "string" && forkRaw.includes("/")) {

```

```

    const split = splitOwnerRepo(forkRaw, owner, repo);
    owner = split.owner;
    repo = split.repo;
    fork = "1";
  } else if (forkRaw != null && forkRaw !== 0 && forkRaw !== false && forkRaw !== "0" && forkRaw !== "false") {
    fork = "1";
  }

  if (`${owner}/${repo}`.toLowerCase() !== `${DEFAULT_OWNER}/${DEFAULT_REPO}`.toLowerCase()) {
    fork = "1";
  }

  return { project: PROJECT, owner, repo, branch, fork, tag, asset };
}

function kvKey(dims) {
  return `${dims.project}|${dims.owner}|${dims.repo}|${dims.branch}|${dims.fork}`;
}

function githubAssetUrl(owner, repo, tag, asset) {
  if (!asset) {
    if (owner === DEFAULT_OWNER && repo === DEFAULT_REPO) return GITHUB_RELEASES;
    return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases`;
  }
  if (!tag || tag === "latest") {
    return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases/latest/download/${encodeURIComponent(asset)}`;
  }
  return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases/download/${encodeURIComponent(tag)}/${encodeURIComponent(asset)}`;
}

function totalKey() {
  return PROJECT + "|__total__";
}

async function increment(env, dims) {
  const key = kvKey(dims);
  const n = parseInt((await env.DOWNLOADS.get(key)) || "0", 10) + 1;
  await env.DOWNLOADS.put(key, String(n));
  const t = parseInt((await env.DOWNLOADS.get(totalKey())) || "0", 10) + 1;
  await env.DOWNLOADS.put(totalKey(), String(t));
  return t;
}

async function listAllKeys(env) {
  const keys = [];
  let cursor;
  do {
    const page = await env.DOWNLOADS.list(cursor ? { cursor } : {});
    keys.push(...page.keys);
    cursor = page.list_complete ? undefined : page.cursor;
  } while (cursor);
  return keys;
}

async function collectStats(env) {
  const keys = await listAllKeys(env);
  let total = 0;
  const by_repo = {};
  const by_branch = {};
  const by_fork = { "0": 0, "1": 0 };
  const breakdown = [];

  for (const k of keys) {
    const name = k.name;
    if (name === viewsKey() || name === totalKey() || name === githubCacheKey()) continue;
    const n = parseInt((await env.DOWNLOADS.get(name)) || "0", 10);
    if (!Number.isFinite(n) || n <= 0) continue;
    const parts = name.split("|");
    if (parts.length < 5) continue;
    const [project, owner, repo, branch, fork] = parts;
    total += n;
    const repoId = `${owner}/${repo}`;
    by_repo[repoId] = (by_repo[repoId] || 0) + n;
    by_branch[branch] = (by_branch[branch] || 0) + n;
    const forkFlag = fork === "1" ? "1" : "0";
    by_fork[forkFlag] = (by_fork[forkFlag] || 0) + n;
    breakdown.push({ project, owner, repo, branch, fork: forkFlag, count: n });
  }

  const totalDirect = parseInt((await env.DOWNLOADS.get(totalKey())) || "0", 10);
  const views = parseInt((await env.DOWNLOADS.get(viewsKey())) || "0", 10) || 0;
  const github = await githubStats(env);
  const shown = Number.isFinite(totalDirect) && totalDirect > 0 ? totalDirect : total;
  return {
    project: PROJECT,
    total: shown,
    views,
    downloads: shown,
    by_repo,
    by_branch,
    by_fork,
    breakdown,
    github: {

```

```

    stars: github.stars || 0,
    forks: github.forks || 0,
    watchers: github.watchers || 0,
    release_download_count: github.release_download_count || 0,
  },
  note: "Isolated to azos (this Worker + its KV), not VibeLock. Key layout: project|owner|repo|branch|fork",
};
}

function viewsKey() {
  return PROJECT + "|__views__";
}

function githubCacheKey() {
  return PROJECT + "|__github__";
}

async function incrementViews(env) {
  const n = parseInt((await env.DOWNLOADS.get(viewsKey())) || "0", 10) + 1;
  await env.DOWNLOADS.put(viewsKey(), String(n));
  return n;
}

async function githubStats(env) {
  const cached = await env.DOWNLOADS.get(githubCacheKey());
  if (cached) {
    try {
      const obj = JSON.parse(cached);
      if (obj && obj.fetched_at && Date.now() - obj.fetched_at < 5 * 60 * 1000) {
        return obj;
      }
    } catch {
      /* ignore */
    }
  }
  const headers = { "User-Agent": "Mozilla/5.0 AZ-OS-download-tracker", Accept: "application/vnd.github+json" };
  let stars = 0;
  let forks = 0;
  let watchers = 0;
  let release_download_count = 0;
  try {
    const repoRes = await fetch("https://api.github.com/repos/AzielEliab/azos", { headers });
    if (repoRes.ok) {
      const repo = await repoRes.json();
      stars = Number(repo.stargazers_count) || 0;
      forks = Number(repo.forks_count) || 0;
      watchers = Number(repo.subscribers_count != null ? repo.subscribers_count : repo.watchers_count) || 0;
    }
    const relRes = await fetch("https://api.github.com/repos/AzielEliab/azos/releases/latest", { headers });
    if (relRes.ok) {
      const rel = await relRes.json();
      const assets = Array.isArray(rel.assets) ? rel.assets : [];
      release_download_count = assets.reduce((s, a) => s + (Number(a.download_count) || 0), 0);
    }
  } catch {
    /* public API; empty is fine */
  }
  const out = { stars, forks, watchers, release_download_count, fetched_at: Date.now() };
  try {
    await env.DOWNLOADS.put(githubCacheKey(), JSON.stringify(out));
  } catch {
    /* ignore */
  }
  return out;
}

function installScript() {
  return `#!/usr/bin/env bash
# AZ-OS one-click install. Counted download via this Worker.
set -euo pipefail
HOST="${HOST}"
ASSET="${DEFAULT_ASSET}"
WORKDIR="${AZOS_HOME:-$HOME/azos}"
mkdir -p "$WORKDIR"
cd "$WORKDIR"
echo "Downloading counted tarball from ${HOST}/download (User-Agent Mozilla/5.0)?"
curl -fsSL -A 'Mozilla/5.0' "${HOST}/download?asset=${ASSET}" -o "${ASSET}"
tar -xzf "${ASSET}"
DIR="$(find . -maxdepth 1 -type d -name 'azos-*' | head -n 1)"
if [ -n "${DIR}" ]; then
  cd "${DIR}"
fi
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -U pip
python -m pip install -e .
echo
echo "Installed AZ-OS."
echo "Run: azos ui"
echo "Then open http://127.0.0.1:8000 (loopback only)"
echo "Author: Aziel Eliab."
`;
}

```

```

}

async function serveAsset(request, env, asset, { head = false } = {}) {
  if (!env.ASSETS) {
    return json({ error: "assets binding missing" }, 500);
  }
  const assetUrl = new URL("/") + asset, request.url);
  const assetRes = await env.ASSETS.fetch(new Request(assetUrl, { method: "GET" }));
  if (!assetRes.ok) {
    return json({ error: "asset not hosted", asset, status: assetRes.status }, 404);
  }
  const headers = new Headers();
  headers.set("Content-Type", "application/gzip");
  headers.set("Content-Disposition", "attachment; filename='" + asset.replaceAll('"', "'") + "'");
  headers.set("Cache-Control", "private, no-store");
  const len = assetRes.headers.get("Content-Length");
  if (len) headers.set("Content-Length", len);
  for (const [k, v] of Object.entries(corsHeaders())) headers.set(k, v);
  if (head) {
    return new Response(null, { status: 200, headers });
  }
  return new Response(assetRes.body, { status: 200, headers });
}

async function indexHtml(env) {
  const stats = await collectStats(env);
  const views = Number(stats.views) || 0;
  const downloads = Number(stats.downloads != null ? stats.downloads : stats.total) || 0;
  const v = views.toLocaleString("en-US");
  const n = downloads.toLocaleString("en-US");
  const gh = stats.github || {};
  const breakdown = (stats.breakdown || []).map(
    (b) =>
      `<li><code>${b.owner}/${b.repo}</code> branch <code>${b.branch}</code> fork=${b.fork} ? ${b.count}</li>`,
  )
    .join("") || "<li>none yet</li>";
  return `<!doctype html>
<html lang="en">
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>AZ-OS downloads</title>
<style>
:root { color-scheme: dark; }
body { font: 16px/1.45 system-ui, sans-serif; max-width: 42rem; margin: 3rem auto; padding: 0 1.25rem 4rem; background-color: #0e1014; color: #e8eaeef; }
h1 { font-size: 1.75rem; margin: 0 0 .35rem; }
.motto { color: #9aa3b2; margin: 0 0 1.5rem; }
.card { border: 1px solid #2a3140; border-radius: 12px; padding: 1.25rem 1.35rem; background: #151922; }
.nums { display: grid; grid-template-columns: 1fr 1fr; gap: .8rem; margin: 0 0 1rem; }
.count { font-size: 2.2rem; font-variant-numeric: tabular-nums; font-weight: 700; margin: 0; }
.count span { display: block; font-size: .95rem; font-weight: 500; color: #9aa3b2; }
.btns { display: grid; grid-template-columns: 1fr 1fr; gap: .75rem; margin: 0 0 .85rem; }
@media (max-width: 520px) { .btns { grid-template-columns: 1fr; } }
a.btn, button.btn { display: block; width: 100%; box-sizing: border-box; text-align: center; font: inherit; font-size: 1.2rem; font-weight: 750; padding: 1rem 1.1rem; border-radius: 10px; border: 0; cursor: pointer; text-decoration: none; }
a.btn.primary { background: #e8eaeef; color: #0e1014; }
button.btn.install { background: #c9a227; color: #14110a; }
button.btn.install.copied { background: #7dcf9a; color: #0e1014; }
.kid { font-size: 1.05rem; margin: 0 0 1rem; }
.meta { margin-top: 1.1rem; color: #9aa3b2; font-size: .92rem; }
.meta a { color: #c9d4ff; }
.iso { margin-top: .85rem; font-size: .85rem; color: #7d8696; }
.banner { border: 1px solid #5c4a1a; background: #241c0d; color: #f0d78c; padding: .85rem 1rem; border-radius: 8px; margin: 0 0 1.2rem; font-size: .92rem; }
pre { background: #0e1014; padding: .75rem .9rem; overflow: auto; border-radius: 8px; font-size: .82rem; }
code { font-size: .88rem; }
</style>
<body>
<h1>AZ-OS</h1>
<p class="motto">Integrity precedes execution. Author Aziel Eliab.</p>
<p class="banner">THIS IS: a portable ethical overlay + AZ Interface control surface. THIS IS NOT: a kernel, worm, remote machine takeover, or a silent block. Hosted halt is a token in JSON, not killing the caller OS. Author Aziel Eliab.</p>
<div class="card">
<div class="nums">
<p class="count">${v}<span>Views</span></p>
<p class="count">${n}<span>Downloads</span></p>
</div>
<p class="kid"><strong>Two big buttons.</strong> Download saves the gzip (the Downloads number goes up). One-click install copies a Terminal command. After it finishes, type <code>azos ui</code>.</p>
<div class="btns">
<a class="btn primary dl" href="/download?asset=${DEFAULT_ASSET}">Download</a>
<button type="button" class="btn install" id="install-btn">One-click install</button>
</div>
<pre id="install-cmd">${INSTALL_LINE}</pre>
<p class="kid">Then run: <code>azos ui</code> and open http://127.0.0.1:8800 (this computer only).</p>
<p class="meta">The download count ticks on the Download click. The Worker serves the gzip (HTTP 200). No 302 to GitHub. Forks using this same link are counted automatically. ${DEFAULT_ASSET} ? ${n} counted.</p>
<p class="iso">Isolated counter: Worker <code>azos-download-tracker</code>, project <code>${PROJECT}</code>, KV <code>AZOS_DOWNLOADS</code>. Not mixed with any other product. /v1 does not increment downloads.</p>
<p class="meta">GitHub: stars ${gh.stars} || 0 ? forks ${gh.forks} || 0 ? watchers ${gh.watchers} || 0 ? release assets ${gh.release_download_count} || 0</p>
<p class="meta">Paper: <a href="${DOI}">doi:10.5281/zenodo.2143171</a> ? <a href="${ZENODO}">Zenodo</a> ? Apache-

```

```

2.0 ? Eliab, Aziel</p>
<p class="meta"><a href="/stats">JSON stats</a> ? <a href="/openapi.json">OpenAPI</a> ? <a href="/v1/skill">Skill<
/a> ? <a href="/ai">AI runtime</a> ? <a href="${GITHUB_REPO}">GitHub</a> ? <a href="${GITHUB_LATEST}">releases</a></p>
<script>
  (function () {
    var cmd = "curl -fsSL https://azos-download-tracker.vibelock.workers.dev/install.sh | bash";
    var btn = document.getElementById("install-btn");
    var pre = document.getElementById("install-cmd");
    if (!btn) return;
    btn.addEventListener("click", function () {
      function done(ok) {
        btn.textContent = ok ? "Copied! Paste in Terminal, then run azos ui" : "Select the command, copy it, then
run azos ui";
        btn.classList.add("copied");
      }
      if (navigator.clipboard && navigator.clipboard.writeText) {
        navigator.clipboard.writeText(cmd).then(function () { done(true); }).catch(function () { done(false); });
      } else {
        done(false);
        if (pre && window.getSelection()) {
          var r = document.createRange();
          r.selectNodeContents(pre);
          var sel = window.getSelection();
          sel.removeAllRanges();
          sel.addRange(r);
        }
      }
    });
  })();
</script>
<h2>Per repo / branch / fork</h2>
<ul>${breakdown}</ul>
</div>
</body>
</html>`;
}

export default {
  async fetch(request, env) {
    const url = new URL(request.url);

    if (request.method === "OPTIONS") {
      return new Response(null, { status: 204, headers: corsHeaders() });
    }

    if ((url.pathname === "/install.sh" || url.pathname === "/install.sh/") && request.method === "GET") {
      return new Response(installScript(), {
        status: 200,
        headers: {
          "Content-Type": "text/x-shellscript; charset=utf-8",
          "Cache-Control": "private, no-store",
          ...corsHeaders(),
        },
      });
    }

    const runtime = await handleRuntime(request, url, env);
    if (runtime) return runtime;

    if (url.pathname === "/" && request.method === "GET") {
      await incrementViews(env);
      return new Response(await indexHtml(env), {
        headers: { "Content-Type": "text/html; charset=utf-8", "Cache-Control": "private, no-store", ...corsHeaders()
      },
    });
    }

    if (url.pathname === "/count" && request.method === "GET") {
      const stats = await collectStats(env);
      return json({ project: PROJECT, views: stats.views || 0, downloads: stats.downloads || stats.total || 0, total:
stats.total || 0 });
    }

    if (url.pathname === "/stats" && request.method === "GET") {
      return json(await collectStats(env));
    }

    if (url.pathname === "/event" && request.method === "POST") {
      let body;
      try {
        body = await request.json();
      } catch {
        return json({ error: "JSON body required" }, 400);
      }
      const dims = parseDims(body || {});
      const count = await increment(env, dims);
      return json({
        ok: true,
        key: kvKey(dims),
        count,
        owner: dims.owner,
        repo: dims.repo,

```

```
        branch: dims.branch,
        fork: dims.fork,
        asset: dims.asset || null,
    });
}

if (url.pathname === "/go" && (request.method === "GET" || request.method === "HEAD")) {
    const dims = parseDims(url.searchParams);
    const asset = dims.asset || DEFAULT_ASSET;
    dims.asset = asset;
    if (request.method === "GET") await increment(env, dims);
    return serveAsset(request, env, asset, { head: request.method === "HEAD" });
}

if ((url.pathname
=== "/download" || url.pathname.startsWith("/download/")) && (request.method === "GET" || request.method === "HEAD")) {
    const dims = parseDims(url.searchParams);
    if (!dims.asset && url.pathname.startsWith("/download/")) {
        dims.asset = decodeURIComponent(url.pathname.slice("/download/".length));
    }
    const asset = dims.asset || DEFAULT_ASSET;
    dims.asset = asset;
    if (request.method === "GET") await increment(env, dims);
    return serveAsset(request, env, asset, { head: request.method === "HEAD" });
}

return json({ error: "not found" }, 404);
},
};
```

azos/workers/download-tracker/src/runtime.js

```

/**
 * AZ-OS hosted runtime (Cloudflare Worker).
 * Labels / overlay receipts only. Integrity precedes execution.
 * NOT a kernel, worm, or remote machine takeover.
 * Hosted halt is a token in the JSON response, not killing the caller's OS.
 */
function runtimeCors() {
  return {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type",
  };
}

function runtimeJson(body, status = 200) {
  return new Response(JSON.stringify(body, null, 2), {
    status,
    headers: { "Content-Type": "application/json; charset=utf-8", ...runtimeCors() },
  });
}

async function sha256Hex(bytes) {
  const data = bytes instanceof Uint8Array ? bytes : new TextEncoder().encode(String(bytes));
  const dig = await crypto.subtle.digest("SHA-256", data);
  const arr = new Uint8Array(dig);
  let out = "";
  for (let i = 0; i < arr.length; i++) out += arr[i].toString(16).padStart(2, "0");
  return out;
}

async function readJsonBody(request) {
  const ct = (request.headers.get("content-type") || "").toLowerCase();
  if (request.method === "GET" || request.method === "HEAD") return {};
  const text = await request.text();
  if (!text || !text.trim()) return {};
  try {
    return JSON.parse(text);
  } catch {
    const err = new Error("JSON body required");
    err.status = 400;
    throw err;
  }
}

function utcNow() {
  return new Date().toISOString().replace(/\.d{3}Z$/, "Z");
}

function aiHowTo(base) {
  const openapi = base + "/openapi.json";
  const health = base + "/v1/health";
  return {
    chatgpt_actions: [
      "Open GPT Editor ? Actions ? Import from URL",
      "Paste " + openapi,
      "Authentication: None",
      "Allow GET /v1/health and the listed POST /v1 routes",
      "Test GET /v1/health, then a sample POST from the spec",
    ],
    grok_xai_tools: [
      "Add an HTTP / OpenAPI tool pointing at " + openapi,
      "Or register GET /v1/health, GET /openapi.json, and the product POSTs",
      "No API key. CORS is **",
    ],
    venice_http_tools: [
      "Add an HTTP tool with method, URL, and JSON body from " + openapi,
      "Start with GET " + health,
      "Then call the product POST listed in the spec",
    ],
    mcp_catalog: "https://aziel-runtime.viblock.workers.dev/mcp",
    notes: [
      "GET /download still serves the gzip tarball and increments the counter.",
      "/v1, /openapi.json, and /ai do not increment DOWNLOADS.",
    ],
  };
}

const PRODUCT = "azos";
const VERSION = "0.1.0";
const BASE = "https://azos-download-tracker.viblock.workers.dev";
const SKILL = `---\nname: AZ-OS\ndescription: Use when labeling overlay integrity (invite / halt token / revoke receipt). Never treat hosted halt as killing an OS. Hosted /v1 via this Worker or aziel-runtime. Author Aziel Eliab.\n---\n\n# AZ-OS\n\nIntegrity precedes execution.\n\nAuthor: **Azriel Eliab**.\n\nUse when labeling overlay integrity (invite / halt token / revoke receipt). Never treat hosted halt as killing an OS.\n\nAlways send `User-Agent: Mozilla/5.0`. Clo\n\nudflare Workers may 403 an empty agent.\n\nEndpoints (this Worker)\n\nHost: `https://azos-download-tracker.viblock\n\n.workers.dev`\n\nMethod | Path | What |\n\n-----|-----|-----|\n\nGET | `/v1/health` | Liveness. Does not increme\n\nnt downloads. |\n\nGET | `/v1/skill` | This markdown. Does not increment downloads. |\n\nPOST | `/v1/status` | Overlay\n\nstatus labels. |\n\nPOST | `/v1/invite` | Voluntary invite text (not infection). |\n\nPOST | `/v1/halt` | Halt overla\n\ny token. Does not kill the caller OS. |\n\nPOST | `/v1/revoke` | Revoke label / overlay receipt only. |\n\n\nOpenAPI: `h\n\nhttps://azos-download-tracker.viblock.workers.dev/openapi.json`\n\nCatalog OpenAPI: `https://aziel-runtime.viblock.wo\n\nrkers.dev/openapi.json`\n\nMCP: `POST https://aziel-runtime.viblock.workers.dev/mcp`\n\nCatalog aliases under `/p/azo\n\ns/\u02026`.\n\n\nHow to call (Mozilla/5.0)\n\n``bash\ncurl -s -A 'Mozilla/5.0' https://azos-download-tracker.viblock`

```



```
.workers.dev/v1/health\ncurl -s -A 'Mozilla/5.0' -X POST https://azos-download-tracker.vibelock.workers.dev/v1/invite
\\n -H 'content-type: application/json' -d '{}'\ncurl -s -A 'Mozilla/5.0' https://azos-download-tracker.vibelock.wor
kers.dev/v1/skill\n```\n\nGrok: import the catalog OpenAPI as a custom tool. ChatGPT: GPT Actions. Venice: HTTP tools.
\n\n## Local (after one-click install)\n\n```\nbash\ncurl -fsSL https://azos-download-tracker.vibelock.workers.dev/insta
ll.sh | bash\nazos ui\n```\n\nThen open http://127.0.0.1:8800 (this computer only).\n\n## Honest banner\n\nTHIS IS: a
portable ethical overlay + AZ Interface control surface. THIS IS NOT: a kernel, worm, remote machine takeover, or a si
lent block. Hosted halt is a token in JSON, not killing the caller OS. Author Aziel Eliab.\n\nDOI: https://doi.org/10.
5281/zenodo.21431711 \nRecord: https://zenodo.org/records/21431711\n\nApache-2.0 (or the repo LICENSE). Forks are wel
come and always allowed.\n";
```

```
const MOTTO = "Integrity precedes execution.";
const DOWNLOAD_URL = "https://azos-download-tracker.vibelock.workers.dev/";
const SOURCE_URL = "https://github.com/AzielEliab/azos";
const SAFE_ACTIONS = ["list_modules", "echo", "status", "purge_session"];
```

```
const INVITE = `AZ-OS ? voluntary overlay (not an infection)`
```

AZ-OS is a portable folder and control surface. It is not a kernel,
not a bootloader, not a hypervisor, and not malware. You run it because
you choose to.

Principles

1. Integrity precedes execution.
No module or action runs without a token from ARC.
2. Time-bound actions are final.
Authorized executions append to an immutable sha256 chain. No rewrite.
3. Understanding precedes modification.
Extending or loading a module requires an explicit comprehension
checkbox and a short restatement of intent.
4. The system protects itself architecturally.
Unsigned or unauthorized run() raises AuthorizationError. Default deny.
5. Propagation is not infection.
This invite prints principles and a download URL. AZ-OS does not
copy itself onto other machines.

You are invited to run AZ-OS yourself. This is not a silent block.
Adoption is voluntary.

```
Counted download:
  ${DOWNLOAD_URL}
```

```
Source:
  ${SOURCE_URL}
`;
```

```
function overlayMeta(obj) {
  return {
    motto: MOTTO,
    overlay: true,
    kernel: false,
    worm: false,
    malware: false,
    remote_machine_takeover: false,
    kills_caller_os: false,
    ...obj,
  };
}
```

```
function randomHex(nBytes) {
  const buf = new Uint8Array(nBytes);
  crypto.getRandomValues(buf);
  let out = "";
  for (let i = 0; i < buf.length; i++) out += buf[i].toString(16).padStart(2, "0");
  return out;
}
```

```
function openapiDoc() {
  return {
    openapi: "3.1.0",
    info: {
      title: "AZ-OS Runtime API",
      version: VERSION,
      summary: MOTTO,
      description: "Labels / overlay receipts only. NOT a kernel, worm, or remote machine takeover. Hosted halt is a t
oken in JSON, not killing the caller's OS.",
    },
    servers: [{ url: BASE }],
    paths: {

      "/v1/skill": {
        get: {
          operationId: "azos_skill",
          summary: "Return skill markdown. Does not increment download KV.",
          responses: { "200": { description: "markdown" } },
        },
      },

      "/v1/health": { get: { operationId: "azosHealth", summary: "Liveness", responses: { "200": { description: "OK" } } } },

      "/v1/status": { post: { operationId: "azosStatus", summary: "Overlay status labels", responses: { "200": { descr
iption: "Status overlay receipt" } } } },

      "/v1/invite": { post: { operationId: "azosInvite", summary: "Voluntary invite text (not infection)", responses:
{ "200": { description: "Invite" } } } },

      "/v1/halt": { post: { operationId: "azosHalt", summary: "Return a halt overlay token. Does not kill the caller O
S.", responses: { "200": { description: "Halt token" } } } },

      "/v1/revoke": { post: { operationId: "azosRevoke", summary: "Revoke label / overlay receipt only", requestBody:
```

```

{ required: false, content: { "application/json": { schema: { type: "object", properties: { token: { type: "string" } } } } }, responses: { "200": { description: "Revoke overlay receipt" } } } },
  },
};
}

export async function handleRuntime(request, url, env) {
  const path = url.pathname;
  if (path === "/v1/health" && request.method === "GET") {
    return runtimeJson(overlayMeta({ ok: true, product: PRODUCT, version: VERSION }));
  }
  if (path === "/v1/skill" && request.method === "GET") {
    return new Response(SKILL, {
      status: 200,
      headers: { "Content-Type": "text/markdown; charset=utf-8", "Cache-Control": "private, no-store", ...runtimeCors(
    ) },
    });
  }
  if (path === "/openapi.json" && request.method === "GET") return runtimeJson(openapiDoc());
  if (path === "/ai" && request.method === "GET") {
    return runtimeJson(overlayMeta({
      product: PRODUCT, title: "Use with Grok, ChatGPT, Venice",
      openapi: BASE + "/openapi.json", health: BASE + "/v1/health", ...aiHowTo(BASE),
    }));
  }
  if (path === "/v1" && request.method === "GET") {
    return runtimeJson(overlayMeta({
      product: PRODUCT,
      endpoints: ["GET /v1/health", "POST /v1/status", "POST /v1/invite", "POST /v1/halt", "POST /v1/revoke", "GET /openapi.json", "GET /ai"],
    }));
  }
  if (path === "/v1/status" && request.method === "POST") {
    return runtimeJson(overlayMeta({
      ok: true,
      product: PRODUCT,
      overlay_name: "AZ-OS",
      interface: "AZ Interface (hosted labels)",
      version: VERSION,
      halted: false,
      lumen: "running",
      builtins: SAFE_ACTIONS,
      tokens: { active: 0, revoked: 0, issued: 0 },
      note: "Hosted status is an overlay receipt, not a kernel and not the caller's OS.",
    }));
  }
  if (path === "/v1/invite" && request.method === "POST") {
    return runtimeJson(overlayMeta({
      ok: true,
      product: PRODUCT,
      invite: INVITE,
      download: DOWNLOAD_URL,
      source: SOURCE_URL,
      infection: false,
      note: "Propagation is invitation, not infection. Writes no files on the caller.",
    }));
  }
  if (path === "/v1/halt" && request.method === "POST") {
    const token = randomHex(32);
    const token_hash = await sha256Hex(token);
    return runtimeJson(overlayMeta({
      ok: true,
      product: PRODUCT,
      halt: {
        token,
        token_hash,
        kind: "overlay_receipt",
        issued_at: utcNow(),
      },
      note: "Hosted halt is a token in this JSON response. It does not kill the caller's OS.",
    }));
  }
  if (path === "/v1/revoke" && request.method === "POST") {
    let body = {};
    try { body = await readJsonBody(request); } catch (e) { return runtimeJson(overlayMeta({ ok: false, error: e.message }, e.status || 400)); }
    const token = body.token != null ? String(body.token) : "";
    let token_hash = null;
    if (token) token_hash = await sha256Hex(token);
    return runtimeJson(overlayMeta({
      ok: true,
      product: PRODUCT,
      revoke: {
        kind: "overlay_receipt",
        token_presented: Boolean(token),
        token_hash,
        revoked_at: utcNow(),
      },
      note: "Revoke is a label / overlay receipt only. It does not take over a remote machine.",
    }));
  }
  if (path === "/v1/status" || path === "/v1/invite" || path === "/v1/halt" || path === "/v1/revoke") {
    return runtimeJson(overlayMeta({ error: "method not allowed" }), 405);
  }
  if (path.startsWith("/v1/")) return runtimeJson(overlayMeta({ error: "not found", product: PRODUCT }), 404);
}

```

```
    return null;  
}
```

azos/workers/download-tracker/wrangler.toml

```
name = "azos-download-tracker"
main = "src/index.js"
compatibility_date = "2026-07-01"
workers_dev = true
account_id = "ac575a9b822bea2bed97d0ab73aed238"

[[kv_namespaces]]
binding = "DOWNLOADS"
id = "816b8d5da5fd470f9c4e783f3c87ca77"

[assets]
directory = "./public"
binding = "ASSETS"
run_worker_first = ["/", "/download", "/download/*", "/stats", "/event", "/go", "/count", "/install.sh", "/v1", "/v1/*", "/openapi.json", "/ai", "/ai/*", "/mcp", "/mcp/"]
```

decisiongate/decisiongate/__init__.py

```

"""DecisionGATE: a lightweight ethical pre-execution filter.

July 2026 whitepaper implementation by Aziel Eliab.

Not a predictor. Not advice. Not a command. A proposal does not pass
unless it survives structured scrutiny through five sequential gates:
Definition, Evidence, Impact, Integrity, Responsibility.

Freedom without clarity is chaos. Clarity without force is wisdom.

Forks are welcome and always allowed.
"""

from __future__ import annotations

from decisiongate.engine import DecisionGATE, Report
from decisiongate.gates import BLOCK, PASS, REVISE, GateResult
from decisiongate.proposal import Proposal

__version__ = "0.1.0"
__author__ = "Azriel Eliab"
LIMITATION = (
    "THIS IS: a five-gate ethical pre-execution filter (PASS / REVISE / BLOCK). "
    "THIS IS NOT: a predictor, a court, a truth score, advice, or a remote command runner. "
    "wrap is not hosted. Author Azriel Eliab."
)
__all__ = [
    "BLOCK",
    "DecisionGATE",
    "GateResult",
    "LIMITATION",
    "PASS",
    "Proposal",
    "REVISE",
    "Report",
    "__author__",
    "__version__",
]

```

decisiongate/decisiongate/__main__.py

```
"""Allow ``python -m decisiongate`` to invoke the CLI."""

from decisiongate.cli import main

if __name__ == "__main__":
    raise SystemExit(main())
```

decisiongate/decisiongate/cli.py

```

"""Command-line interface for DecisionGATE.

decisiongate ui      # 127.0.0.1:8791
decisiongate version
decisiongate check --statement "." --evidence "." --impact-pos "."
                    --impact-neg "." --values "." --accountable "Name"
decisiongate wrap --statement "." -- -- CMD
decisiongate doctor
decisiongate verify
decisiongate import FILE.json
decisiongate export FILE.json

Pre-execution filter. Not a predictor. Not advice. Not a command.
`wrap` runs CMD only if all five gates PASS. Subprocess is the
user-supplied argv after ``-`` (no shell).
File import AND export both exist. Doctor/verify speak in plain words.
Forks always allowed.
"""

from __future__ import annotations

import argparse
import json
import sys
from typing import Sequence

from decisiongate import __version__
from decisiongate.engine import DecisionGATE
from decisiongate.gates import PASS
from decisiongate.proposal import Proposal, _as_list

def _build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="decisiongate",
        description=(
            "DecisionGATE ? lightweight ethical pre-execution filter "
            "(Azriel Eliab, 2026). Not a predictor. Not advice. Not a command. "
            "Freedom without clarity is chaos. Clarity without force is wisdom. "
            "Local UI: `decisiongate ui` at http://127.0.0.1:8791. "
            "Import and export JSON files. doctor/verify in plain words."
        ),
    )
    sub = parser.add_subparsers(dest="cmd", required=True)

    sub.add_parser("version", help="Print package version.")

    p_check = sub.add_parser(
        "check",
        help="Run the five gates on one proposal. Stops at the first failure.",
    )
    p_check.add_argument("--statement", default="", help="Concrete proposal statement.")
    p_check.add_argument(
        "--evidence",
        action="append",
        default=[],
        help="Fact/datum/observation. Repeatable; also accepts one-per-line.",
    )
    p_check.add_argument(
        "--impact-pos",
        action="append",
        default=[],
        dest="impact_pos",
        help="Positive impact. Repeatable.",
    )
    p_check.add_argument(
        "--impact-neg",
        action="append",
        default=[],
        dest="impact_neg",
        help="Negative impact. Repeatable.",
    )
    p_check.add_argument(
        "--values",
        action="append",
        default=[],
        help="Stated value. Repeatable.",
    )
    p_check.add_argument(
        "--commitments",
        action="append",
        default=[],
        help="Prior commitment. Repeatable.",
    )
    p_check.add_argument(
        "--constraints",
        action="append",
        default=[],
        help="Hard constraint. Repeatable.",
    )
    p_check.add_argument(
        "--accountable",

```

```

        default="",
        help="Named accountable owner.",
    )
    p_check.add_argument(
        "--json",
        action="store_true",
        dest="as_json",
        help="Print the full report (lineage, final_state, blocked_at) as JSON.",
    )

    p_ui = sub.add_parser(
        "ui",
        help="Serve the local filter UI on 127.0.0.1:8791.",
    )
    p_ui.add_argument("--host", default="127.0.0.1", help="Loopback host (default 127.0.0.1).")
    p_ui.add_argument("--port", type=int, default=8791, help="Port (default 8791).")

    p_wrap = sub.add_parser(
        "wrap",
        help="Run CMD only if all five gates PASS. Put the command after -- .",
    )
    p_wrap.add_argument("--statement", default="", help="Concrete proposal statement.")
    p_wrap.add_argument("--evidence", action="append", default=[], help="Fact. Repeatable.")
    p_wrap.add_argument("--impact-pos", action="append", default=[], dest="impact_pos", help="Positive impact.")
    p_wrap.add_argument("--impact-neg", action="append", default=[], dest="impact_neg", help="Negative impact.")
    p_wrap.add_argument("--values", action="append", default=[], help="Stated value.")
    p_wrap.add_argument("--commitments", action="append", default=[], help="Prior commitment.")
    p_wrap.add_argument("--constraints", action="append", default=[], help="Hard constraint.")
    p_wrap.add_argument("--accountable", default="", help="Named accountable owner.")
    p_wrap.add_argument("--json", action="store_true", dest="as_json", help="Print the report as JSON.")

    p_doc = sub.add_parser(
        "doctor",
        aliases=["verify"],
        help="Self-check in plain words. No network, no telemetry.",
    )
    p_doc.add_argument("--json", action="store_true", dest="as_json", help="Print doctor results as JSON.")

    p_imp = sub.add_parser("import", help="Import a JSON file (pair of export).")
    p_imp.add_argument("path", help="Path to a .json file to load.")

    p_exp = sub.add_parser("export", help="Export a JSON file (pair of import).")
    p_exp.add_argument("path", help="Path to write a .json file.")

    return parser

def _flatten(values: list[str]) -> list[str]:
    out: list[str] = []
    for item in values:
        out.extend(_as_list(item))
    return out

def _proposal_from_ns(args: argparse.Namespace) -> Proposal:
    return Proposal(
        statement=args.statement,
        evidence=_flatten(args.evidence),
        impacts_positive=_flatten(args.impact_pos),
        impacts_negative=_flatten(args.impact_neg),
        values=_flatten(args.values),
        commitments=_flatten(args.commitments),
        constraints=_flatten(args.constraints),
        accountable_person=args.accountable,
    )

def _print_report(report, as_json: bool) -> None:
    if as_json:
        print(json.dumps(report.to_dict(), indent=2, ensure_ascii=False))
        return
    for gate in report.lineage:
        print(f"{gate.name}: {gate.state}")
        if gate.feedback:
            print(f"    {gate.feedback}")
    print(f"final: {report.final_state}")
    if report.blocked_at:
        print(f"blocked_at: {report.blocked_at}")

def _cmd_wrap(gate_argv: list[str], cmd: list[str]) -> int:
    import subprocess

    parser = _build_parser()
    args = parser.parse_args(["wrap", *gate_argv])
    if not cmd:
        print("error: wrap requires `-- CMD` (the command after --)", file=sys.stderr)
        return 2
    report = DecisionGATE().run(_proposal_from_ns(args))
    _print_report(report, args.as_json)
    if report.final_state != PASS:
        print("wrap: refused (not all five gates PASS); command not run", file=sys.stderr)
        return 1
    # Subprocess only the user-supplied argv after --. Never shell=True.

```



```

proc = subprocess.run(cmd)
return int(proc.returncode)

def main(argv: Sequence[str] | None = None) -> int:
    raw = list(argv) if argv is not None else sys.argv[1:]
    if raw and raw[0] == "wrap":
        rest = raw[1:]
        if "--" in rest:
            i = rest.index("--")
            return _cmd_wrap(rest[:i], rest[i + 1 :])
        return _cmd_wrap(rest, [])

    parser = _build_parser()
    args = parser.parse_args(raw)

    if args.cmd == "version":
        print(f"decisiongate {_version__}")
        return 0

    if args.cmd == "check":
        report = DecisionGATE().run(_proposal_from_ns(args))
        _print_report(report, args.as_json)
        return 0 if report.final_state == PASS else 1

    if args.cmd == "ui":
        from decisiongate.ui import serve

        try:
            serve(host=args.host, port=args.port)
        except ValueError as exc:
            print(str(exc), file=sys.stderr)
            return 2
        return 0

    if args.cmd in {"doctor", "verify"}:
        from decisiongate.doctor import run_doctor

        return run_doctor(as_json=getattr(args, "as_json", False))

    if args.cmd == "import":
        from decisiongate.jsonio import import_json

        rec = import_json(args.path)
        sys.stdout.write(json.dumps(rec, indent=2, ensure_ascii=False) + "\n")
        return 0

    if args.cmd == "export":
        from decisiongate.jsonio import export_json

        rec = export_json(args.path)
        sys.stdout.write(json.dumps(rec, indent=2, ensure_ascii=False) + "\n")
        return 0

    parser.error(f"unknown command {args.cmd}")
    return 2

if __name__ == "__main__":
    raise SystemExit(main())

```

decisiongate/decisiongate/doctor.py

```

"""Self-check for DecisionGATE. No network, no telemetry.

    decisiongate doctor
    decisiongate verify

Prints plain sentences a kid can read. Not advice.
"""

from __future__ import annotations

import json
import tempfile
from pathlib import Path
from typing import Callable

from decisiongate import LIMITATION, __version__

AUTHOR = "Aziel Eliab"
Check = tuple[str, bool, str]

def _ok(name: str, detail: str = "") -> Check:
    return name, True, detail

def _fail(name: str, detail: str) -> Check:
    return name, False, detail

def _check_version() -> Check:
    if __version__:
        return _ok("version", str(__version__))
    return _fail("version", "missing")

def _check_identity() -> Check:
    try:
        mod = __import__(__name__.split(".")[0])
        author = str(getattr(mod, "__author__", AUTHOR))
    except Exception as exc: # noqa: BLE001
        return _fail("identity", str(exc))
    blob = author + " " + AUTHOR
    forbidden = ("Col" + "lin H" + "orton", "Ja" + "ck Al" + "tman", "GodLock" + ".AZ", "Reve" + "aler")
    if any(x in blob for x in forbidden):
        return _fail("identity", "forbidden identity label")
    if "Aziel Eliab" not in blob:
        return _fail("identity", author)
    return _ok("identity", AUTHOR)

def _check_json_roundtrip() -> Check:
    from decisiongate.jsonio import export_json, import_json

    with tempfile.TemporaryDirectory() as tmp:
        src = Path(tmp) / "in.json"
        out = Path(tmp) / "out.json"
        src.write_text(
            json.dumps({"product": "decisiongate", "author": AUTHOR, "ok": True}, indent=2),
            encoding="utf-8",
        )
        rec = import_json(src, store_dir=tmp)
        if not rec.get("ok"):
            return _fail("import", str(rec))
        rec2 = export_json(out, store_dir=tmp)
        if not rec2.get("ok") or not out.exists():
            return _fail("export", str(rec2))
        doc = json.loads(out.read_text(encoding="utf-8"))
        if doc.get("author") != AUTHOR:
            return _fail("export author", str(doc.get("author")))
        return _ok("json import/export", "roundtrip")

def _check_loopback() -> Check:
    from decisiongate.ui import LOOPBACK, make_server

    try:
        make_server("0.0.0.0", 9)
    except ValueError as exc:
        if "loopback" in str(exc).lower() and "127.0.0.1" in LOOPBACK:
            return _ok("loopback", "rejects 0.0.0.0")
        return _fail("loopback", str(exc))
    return _fail("loopback", "accepted 0.0.0.0")

CHECKS: tuple[Callable[[], Check], ...] = (
    _check_version,
    _check_identity,
    _check_json_roundtrip,
    _check_loopback,
)

def _plain_line(name: str, ok: bool, detail: str) -> str:
    if name == "version":

```

```

        return (
            f"The program is version {detail}."
            if ok
            else f"The program version is missing ({detail})."
        )
    if name == "identity":
        return (
            "The author name is Aziel Eliab."
            if ok
            else f"The author name is wrong ({detail})."
        )
    if name == "json import/export":
        return (
            "Saving a file and loading it back works."
            if ok
            else f"Saving a file and loading it back failed ({detail})."
        )
    if name == "loopback":
        return (
            "The screen only opens on this computer."
            if ok
            else f"The screen bind check failed ({detail})."
        )
    extra = f" ({detail})" if detail else ""
    return f"{name}: {'ok' if ok else 'not ok'}{extra}."

def collect_doctor() -> dict:
    results = []
    plain = []
    failed = 0
    for fn in CHECKS:
        name, ok, detail = fn()
        results.append({"name": name, "ok": ok, "detail": detail})
        plain.append(_plain_line(name, ok, detail))
        if not ok:
            failed += 1
    if failed == 0:
        summary = "Doctor says: the checks passed. This is not advice. It is a check that the filter is installed."
    else:
        summary = "Doctor says: something is not working. This is not advice."
    plain.append(summary)
    return {
        "ok": failed == 0,
        "failed": failed,
        "checks": results,
        "plain": plain,
        "summary": summary,
        "version": __version__,
        "author": AUTHOR,
        "limitation": LIMITATION,
        "network": False,
        "telemetry": False,
    }

def run_doctor(*, as_json: bool = False) -> int:
    payload = collect_doctor()
    if as_json:
        print(json.dumps(payload, indent=2))
    else:
        for line in payload["plain"]:
            print(line)
        print("limitation:", LIMITATION)
        print("doctor", "passed" if payload["ok"] else "failed")
    return 0 if payload["ok"] else 1

```

decisiongate/decisiongate/engine.py

```

"""DecisionGATE engine: run five gates in order, stop at first failure.

Default path is automatic. A human may override a gate to REVISE with a
note; the override is recorded in lineage. Overrides do not skip a
failure ? REVISE and BLOCK both stop the chain.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any, Mapping

from decisiongate.gates import (
    BLOCK,
    GATE_FUNCS,
    GATE_ORDER,
    PASS,
    REVISE,
    GateResult,
)
from decisiongate.proposal import Proposal

VALID_OVERRIDE_STATES = frozenset({REVISE})

def _coerce_overrides(
    overrides: Mapping[str, Any] | None,
) -> dict[str, tuple[str, str]]:
    """Map gate name -> (REVISE, note). Unknown gates ignored."""
    out: dict[str, tuple[str, str]] = {}
    if not overrides:
        return out
    known = {n.lower(): n for n in GATE_ORDER}
    for key, raw in overrides.items():
        name = known.get(str(key).strip().lower())
        if not name:
            continue
        note = ""
        state = REVISE
        if isinstance(raw, str):
            note = raw.strip()
        elif isinstance(raw, Mapping):
            state_raw = str(raw.get("state") or REVISE).strip().upper()
            if state_raw in VALID_OVERRIDE_STATES:
                state = state_raw
            note = str(raw.get("note") or raw.get("feedback") or "").strip()
        out[name] = (state, note)
    return out

@dataclass
class Report:
    """Lineage of gates that actually ran, plus terminal state."""

    lineage: list[GateResult] = field(default_factory=list)
    final_state: str = PASS
    blocked_at: str | None = None
    proposal: dict[str, Any] = field(default_factory=dict)

    def to_dict(self) -> dict[str, Any]:
        return {
            "lineage": [g.to_dict() for g in self.lineage],
            "final_state": self.final_state,
            "blocked_at": self.blocked_at,
            "proposal": dict(self.proposal),
        }

class DecisionGATE:
    """Pre-execution filter. Not predictive, advisory, or prescriptive."""

    def run(
        self,
        proposal: Proposal | Mapping[str, Any],
        overrides: Mapping[str, Any] | None = None,
    ) -> Report:
        if isinstance(proposal, Proposal):
            prop = proposal.normalized()
        else:
            prop = Proposal.from_dict(dict(proposal))
        forced = _coerce_overrides(overrides)
        lineage: list[GateResult] = []
        final_state = PASS
        blocked_at: str | None = None

        for fn in GATE_FUNCS:
            result = fn(prop)
            if result.name in forced:
                state, note = forced[result.name]
                auto = result.state
                feedback = result.feedback
                if note:

```

```
        feedback = (
            f"{feedback} Human override to {state}: {note}"
        )
    else:
        feedback = (
            f"{feedback} Human override to {state} "
            "(no note supplied)."
        )
    result = GateResult(
        name=result.name,
        state=state,
        feedback=feedback,
        overridden=True,
        automatic_state=auto,
        override_note=note or None,
    )
    lineage.append(result)
    if result.state != PASS:
        final_state = result.state
        if result.state == BLOCK:
            blocked_at = result.name
            break

return Report(
    lineage=lineage,
    final_state=final_state,
    blocked_at=blocked_at,
    proposal=prop.to_dict(),
)
```

decisiongate/decisiongate/gates.py

```

"""Five sequential DecisionGATE gates.

Deterministic heuristics. No ML. Documented rules, kept small and tested.

Each gate returns GateResult(name, state, feedback) where state is one of
PASS, REVISE, BLOCK.

REVISE includes specific feedback the author can act on.
BLOCK cannot be remedied without changing the proposal's nature.

Rules
-----
1. Definition
   BLOCK if the statement is empty.
   REVISE if the statement has fewer than 12 words, or if after removing
   the hedges {maybe, somehow, stuff, things} there is no verb+object
   (at least one verb-like token and one other content token).
2. Evidence
   REVISE if the evidence list is empty.
3. Impact
   REVISE if either the positive or the negative impact list is empty.
4. Integrity
   REVISE if the values list is empty.
   BLOCK if the statement clearly contradicts a provided constraint
   (simple substring / negation check: a prohibition whose payload
   appears in the statement).
5. Responsibility
   BLOCK if accountable_person is blank.
"""

from __future__ import annotations

import re
from dataclasses import dataclass, field
from typing import Callable

from decisiongate.proposal import Proposal

PASS = "PASS"
REVISE = "REVISE"
BLOCK = "BLOCK"

HEDGES = frozenset({"maybe", "somehow", "stuff", "things"})

# Small closed verb list plus suffix checks. Not an NLP parser.
_COMMON_VERBS = frozenset(
    {
        "is",
        "are",
        "was",
        "were",
        "be",
        "been",
        "being",
        "am",
        "have",
        "has",
        "had",
        "do",
        "does",
        "did",
        "will",
        "would",
        "shall",
        "should",
        "can",
        "could",
        "must",
        "need",
        "needs",
        "make",
        "makes",
        "made",
        "take",
        "takes",
        "took",
        "give",
        "gives",
        "gave",
        "go",
        "goes",
        "went",
        "come",
        "keep",
        "put",
        "use",
        "uses",
        "used",
        "set",
        "get",
        "let",
        "allow",
    }

```

```

        "allows",
        "publish",
        "release",
        "releases",
        "deploy",
        "ship",
        "open",
        "close",
        "create",
        "created",
        "delete",
        "write",
        "read",
        "run",
        "execute",
        "adopt",
        "reject",
        "approve",
        "block",
        "collect",
        "store",
        "share",
        "send",
        "build",
        "launch",
        "hire",
        "spend",
        "buy",
        "sell",
        "migrate",
        "replace",
        "update",
        "install",
        "announce",
        "commit",
        "sign",
        "fund",
        "grant",
        "revoke",
        "host",
        "serve",
        "bind",
        "filter",
        "record",
        "name",
        "assign",
        "document",
        "provide",
        "provides",
        "include",
        "includes",
        "add",
        "remove",
        "stop",
        "start",
        "move",
        "change",
        "apply",
        "submit",
        "accept",
        "refuse",
        "pay",
        "offer",
        "request",
        "require",
        "requires",
        "implement",
        "implements",
    }
)

_NEGATION_MARKERS = (
    "do not ",
    "don't ",
    "does not ",
    "doesn't ",
    "must not ",
    "cannot ",
    "can't ",
    "never ",
    "no ",
    "not ",
    "without "
)

_WORD_RE = re.compile(r"[A-Za-z0-9][A-Za-z0-9._+-]*")

@dataclass
class GateResult:
    """Outcome of one gate. ``state`` is PASS, REVISE, or BLOCK."""

    name: str
    state: str
    feedback: str

```

```

overridden: bool = False
automatic_state: str | None = None
override_note: str | None = None
extra: dict[str, str] = field(default_factory=dict)

def to_dict(self) -> dict[str, object]:
    payload: dict[str, object] = {
        "name": self.name,
        "state": self.state,
        "feedback": self.feedback,
    }
    if self.overridden:
        payload["overridden"] = True
        if self.automatic_state:
            payload["automatic_state"] = self.automatic_state
        if self.override_note:
            payload["override_note"] = self.override_note
    return payload

def tokenize(text: str) -> list[str]:
    """Lowercased word tokens; punctuation stripped."""
    return _WORD_RE.findall(text.lower()) if text else []

def _looks_like_verb(token: str) -> bool:
    if token in _COMMON_VERBS:
        return True
    if len(token) > 4 and token.endswith(("ing", "ize", "ise", "ify")):
        return True
    if len(token) > 4 and token.endswith("ed"):
        return True
    return False

def has_verb_and_object(tokens: list[str]) -> bool:
    """True when non-hedge tokens include a verb-like word and one other token."""
    content = [t for t in tokens if t not in HEDGES]
    if len(content) < 2:
        return False
    return any(_looks_like_verb(t) for t in content)

def gate_definition(proposal: Proposal) -> GateResult:
    statement = proposal.statement.strip()
    if not statement:
        return GateResult(
            name="Definition",
            state=BLOCK,
            feedback=(
                "Statement is empty. A proposal with no concrete statement "
                "cannot pass Definition. Write an unambiguous action with a "
                "verb and an object, at least 12 words."
            ),
        )
    tokens = tokenize(statement)
    if len(tokens) < 12:
        return GateResult(
            name="Definition",
            state=REVISE,
            feedback=(
                f"Statement has {len(tokens)} word(s); Definition requires "
                "at least 12. Expand into a concrete, unambiguous proposal "
                "(who does what, to what, under what bound).",
            ),
        )
    if not has_verb_and_object(tokens):
        return GateResult(
            name="Definition",
            state=REVISE,
            feedback=(
                "Statement is hedge-only or lacks a verb+object after removing "
                "maybe/somehow/stuff/things. Name a specific action and its object."
            ),
        )
    return GateResult(
        name="Definition",
        state=PASS,
        feedback="Statement is concrete enough to scrutinize (length, verb+object).",
    )

def gate_evidence(proposal: Proposal) -> GateResult:
    items = [e for e in proposal.evidence if e.strip()]
    if not items:
        return GateResult(
            name="Evidence",
            state=REVISE,
            feedback=(
                "Evidence list is empty. Identify at least one fact, datum, "
                "or observation that grounds the statement. Ungrounded "
                "assertions do not pass Evidence."
            ),
        )
    return GateResult(

```



```

        name="Evidence",
        state=PASS,
        feedback=f"{len(items)} evidence item(s) identified.",
    )

def gate_impact(proposal: Proposal) -> GateResult:
    pos = [i for i in proposal.impact_positive if i.strip()]
    neg = [i for i in proposal.impact_negative if i.strip()]
    missing: list[str] = []
    if not pos:
        missing.append("positive")
    if not neg:
        missing.append("negative")
    if missing:
        which = " and ".join(missing)
        return GateResult(
            name="Impact",
            state=REVISE,
            feedback=(
                f"Impact list(s) empty: {which}. Name who or what is affected "
                "on both the positive and the negative side. Hidden impacts "
                "do not pass Impact."
            ),
        )
    return GateResult(
        name="Impact",
        state=PASS,
        feedback=(
            f"{len(pos)} positive and {len(neg)} negative impact(s) named."
        ),
    )

def _constraint_payload(constraint: str) -> str | None:
    """If ``constraint`` is a prohibition, return the forbidden fragment."""
    text = " ".join(constraint.lower().split())
    if not text:
        return None
    found = False
    remainder = f" {text} "
    for marker in _NEGATION_MARKERS:
        padded = marker if marker.startswith(" ") else f" {marker}"
        if padded in remainder or remainder.lstrip().startswith(marker):
            found = True
            remainder = remainder.replace(padded, " ")
            if remainder.lstrip().startswith(marker):
                remainder = " " + remainder.lstrip()[len(marker) :]
    payload = " ".join(remainder.split())
    if found and payload:
        return payload
    return None

def statement_contradicts_constraint(statement: str, constraint: str) -> bool:
    """Simple substring/negation check. No parser, no ML."""
    payload = _constraint_payload(constraint)
    if not payload:
        return False
    hay = " ".join(statement.lower().split())
    return payload in hay

def gate_integrity(proposal: Proposal) -> GateResult:
    values = [v for v in proposal.values if v.strip()]
    if not values:
        return GateResult(
            name="Integrity",
            state=REVISE,
            feedback=(
                "Values list is empty. Integrity requires stated values so "
                "the proposal can be checked against them."
            ),
        )
    hits: list[str] = []
    for constraint in proposal.constraints:
        if statement_contradicts_constraint(proposal.statement, constraint):
            hits.append(constraint)
    if hits:
        shown = hits[0]
        return GateResult(
            name="Integrity",
            state=BLOCK,
            feedback=(
                "Statement contradicts a provided constraint "
                f"({shown!r}). A contradiction of this kind cannot pass "
                "Integrity without changing the proposal's nature."
            ),
        )
    return GateResult(
        name="Integrity",
        state=PASS,
        feedback=(
            f"{len(values)} value(s) stated; no constraint contradiction detected."
        ),
    )

```

```

    ),
)

def gate_responsibility(proposal: Proposal) -> GateResult:
    owner = proposal.accountable_person.strip()
    if not owner:
        return GateResult(
            name="Responsibility",
            state=BLOCK,
            feedback=(
                "Accountable person is blank. Diffuse or absent ownership "
                "cannot pass Responsibility. Name one accountable owner."
            ),
        )
    return GateResult(
        name="Responsibility",
        state=PASS,
        feedback=f"Accountable owner named: {owner}.",
    )

GATE_ORDER: tuple[str, ...] = (
    "Definition",
    "Evidence",
    "Impact",
    "Integrity",
    "Responsibility",
)

GATE_FUNCS: tuple[Callable[[Proposal], GateResult], ...] = (
    gate_definition,
    gate_evidence,
    gate_impact,
    gate_integrity,
    gate_responsibility,
)

```

decisiongate/decisiongate/jsonio.py

```

"""JSON file import and export for DecisionGATE. Author: Aziel Eliab.

Import a .json file. Export a .json file. Both exist. Not export-only.
"""

from __future__ import annotations

import json
from pathlib import Path
from typing import Any

from decisiongate import __version__

AUTHOR = "Azriel Eliab"
PRODUCT = "DecisionGATE"
STATE_NAME = ".decisiongate-state.json"

def _as_path(path: str | Path) -> Path:
    return Path(path)

def import_json(path: str | Path, *, store_dir: str | Path | None = None) -> dict[str, Any]:
    """Load a JSON object from a file and store it locally."""
    pth = _as_path(path)
    if not pth.is_file():
        raise FileNotFoundError("import file not found: " + str(pth))
    doc = json.loads(pth.read_text(encoding="utf-8"))
    if not isinstance(doc, dict):
        raise ValueError("JSON object required")
    base = _as_path(store_dir) if store_dir is not None else Path.cwd()
    dest = base / STATE_NAME
    dest.write_text(json.dumps(doc, indent=2, ensure_ascii=False) + "\n", encoding="utf-8")
    return {
        "ok": True,
        "imported": str(pth),
        "stored": str(dest),
        "keys": sorted(str(k) for k in doc.keys()),
        "author": AUTHOR,
        "product": PRODUCT,
        "version": __version__,
    }

def export_json(path: str | Path, *, store_dir: str | Path | None = None) -> dict[str, Any]:
    """Write a JSON file. Includes author Azriel Eliab. Pair of import, not export-only."""
    pth = _as_path(path)
    base = _as_path(store_dir) if store_dir is not None else Path.cwd()
    src = base / STATE_NAME
    payload: Any = {}
    if src.exists():
        try:
            payload = json.loads(src.read_text(encoding="utf-8"))
        except Exception:
            payload = {}
    if not isinstance(payload, dict):
        payload = {"value": payload}
    doc = {
        "product": PRODUCT,
        "package": "decisiongate",
        "version": __version__,
        "author": AUTHOR,
        "payload": payload,
    }
    pth.parent.mkdir(parents=True, exist_ok=True)
    pth.write_text(json.dumps(doc, indent=2, ensure_ascii=False) + "\n", encoding="utf-8")
    return {
        "ok": True,
        "exported": str(pth),
        "author": AUTHOR,
        "product": PRODUCT,
        "version": __version__,
    }

```

decisiongate/decisiongate/proposal.py

```

"""Proposal record for DecisionGATE.

A proposal is a concrete action under scrutiny. It is not a prediction,
a recommendation, or a prescription. Fields are the material the five
gates inspect. Empty or blank list items are dropped on normalize.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any, Iterable

def _as_str(value: Any) -> str:
    if value is None:
        return ""
    return str(value)

def _as_list(value: Any) -> list[str]:
    if value is None:
        return []
    if isinstance(value, str):
        parts = [p.strip() for p in value.replace("\r\n", "\n").split("\n")]
        if len(parts) == 1 and ";" in parts[0]:
            parts = [p.strip() for p in parts[0].split(";")]
        return [p for p in parts if p]
    if isinstance(value, Iterable) and not isinstance(value, (bytes, bytearray)):
        out: list[str] = []
        for item in value:
            text = _as_str(item).strip()
            if text:
                out.append(text)
        return out
    text = _as_str(value).strip()
    return [text] if text else []

@dataclass
class Proposal:
    """Concrete unambiguous proposal presented to the five gates."""

    statement: str = ""
    evidence: list[str] = field(default_factory=list)
    impacts_positive: list[str] = field(default_factory=list)
    impacts_negative: list[str] = field(default_factory=list)
    values: list[str] = field(default_factory=list)
    commitments: list[str] = field(default_factory=list)
    constraints: list[str] = field(default_factory=list)
    accountable_person: str = ""

    def normalized(self) -> "Proposal":
        return Proposal(
            statement=_as_str(self.statement).strip(),
            evidence=_as_list(self.evidence),
            impacts_positive=_as_list(self.impacts_positive),
            impacts_negative=_as_list(self.impacts_negative),
            values=_as_list(self.values),
            commitments=_as_list(self.commitments),
            constraints=_as_list(self.constraints),
            accountable_person=_as_str(self.accountable_person).strip(),
        )

    def to_dict(self) -> dict[str, Any]:
        p = self.normalized()
        return {
            "statement": p.statement,
            "evidence": list(p.evidence),
            "impacts_positive": list(p.impacts_positive),
            "impacts_negative": list(p.impacts_negative),
            "values": list(p.values),
            "commitments": list(p.commitments),
            "constraints": list(p.constraints),
            "accountable_person": p.accountable_person,
        }

    @classmethod
    def from_dict(cls, data: dict[str, Any] | None) -> "Proposal":
        raw = data if isinstance(data, dict) else {}
        return cls(
            statement=_as_str(raw.get("statement", "")),
            evidence=_as_list(raw.get("evidence")),
            impacts_positive=_as_list(
                raw.get("impacts_positive", raw.get("impact_pos"))
            ),
            impacts_negative=_as_list(
                raw.get("impacts_negative", raw.get("impact_neg"))
            ),
            values=_as_list(raw.get("values")),
            commitments=_as_list(raw.get("commitments")),
            constraints=_as_list(raw.get("constraints")),
            accountable_person=_as_str(

```

```
        raw.get("accountable_person", raw.get("accountable", ""))
    ),
).normalized()
```

decisiongate/decisiongate/ui.py

```

"""Local DecisionGATE UI. Bind 127.0.0.1 only.

Simple view: type a plan, Run, Import file, Export file, Verify.
Advanced view: extra fields, gate overrides, JSON dump.
Self-contained CSS, no CDN. Motto on the page. Not advice.
"""

from __future__ import annotations

import json
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from importlib.resources import files
from urllib.parse import urlparse

from decisiongate import LIMITATION, __author__, __version__
from decisiongate.engine import DecisionGATE
from decisiongate.proposal import Proposal

LOOPBACK = frozenset({"127.0.0.1", "localhost", "::1"})
WEB = files("decisiongate") / "web"
MIME = {
    ".html": "text/html; charset=utf-8",
    ".css": "text/css; charset=utf-8",
    ".js": "application/javascript; charset=utf-8",
}
MAX_BODY_BYTES = 1 * 1024 * 1024

def _web_bytes(name: str) -> bytes:
    return (WEB / name).read_bytes()

class Handler(BaseHTTPRequestHandler):
    server_version = "DecisionGATE/0.1.0"

    def log_message(self, fmt: str, *args: object) -> None:
        return

    def _send(self, status: int, body: bytes, content_type: str) -> None:
        self.send_response(status)
        self.send_header("Content-Type", content_type)
        self.send_header("Content-Length", str(len(body)))
        self.send_header("Cache-Control", "no-store")
        self.end_headers()
        self.wfile.write(body)

    def _json(self, status: int, obj: object) -> None:
        body = json.dumps(obj, indent=2, ensure_ascii=False).encode("utf-8")
        self._send(status, body, "application/json; charset=utf-8")

    def do_GET(self) -> None: # noqa: N802
        path = urlparse(self.path).path
        if path in {"/", "/index.html"}:
            self._send(200, _web_bytes("index.html"), MIME[".html"])
            return
        if path == "/style.css":
            self._send(200, _web_bytes("style.css"), MIME[".css"])
            return
        if path == "/app.js":
            self._send(200, _web_bytes("app.js"), MIME[".js"])
            return
        if path == "/api/health":
            self._json(200, {
                "ok": True,
                "product": "decisiongate",
                "version": __version__,
                "author": __author__,
                "limitation": LIMITATION,
                "loopback": True,
                "telemetry": False,
                "network": False,
            })
            return
        if path in {"/api/doctor", "/api/verify"}:
            from decisiongate.doctor import collect_doctor

            self._json(200, collect_doctor())
            return
        self._json(404, {"error": "not found"})

    def do_POST(self) -> None: # noqa: N802
        path = urlparse(self.path).path
        if path != "/api/check":
            self._json(404, {"error": "not found"})
            return
        length = int(self.headers.get("Content-Length") or "0")
        if length > MAX_BODY_BYTES:
            self._json(413, {"error": "body too large"})
            return
        raw = self.rfile.read(length) if length else b""
        try:
            payload = json.loads(raw.decode("utf-8") or "{}")

```

```
except json.JSONDecodeError:
    self._json(400, {"error": "JSON body required"})
    return
if not isinstance(payload, dict):
    self._json(400, {"error": "JSON object required"})
    return
proposal = Proposal.from_dict(payload)
overrides = payload.get("overrides") if isinstance(payload.get("overrides"), dict) else None
report = DecisionGATE().run(proposal, overrides=overrides)
self._json(200, report.to_dict())

def make_server(host: str = "127.0.0.1", port: int = 8791) -> ThreadingHTTPServer:
    if host not in LOOPBACK:
        raise ValueError("DecisionGATE UI binds loopback only (127.0.0.1)")
    return ThreadingHTTPServer((host, port), Handler)

def serve(host: str = "127.0.0.1", port: int = 8791) -> None:
    httpd = make_server(host, port)
    bound_host, bound_port = httpd.server_address[:2]
    print(
        f"DecisionGATE UI http://{bound_host}:{bound_port} "
        "(loopback only; pre-execution filter, not advice)"
    )
    try:
        httpd.serve_forever()
    except KeyboardInterrupt:
        print("\nstopped")
    finally:
        httpd.server_close()
```

decisiongate/decisiongate/web/app.js

```

/* DecisionGATE UI. No CDN. Filter, do not advise. Simple by default. */
(function () {
  const form = document.getElementById("proposal-form");
  const banner = document.getElementById("banner");
  const exportBtn = document.getElementById("export");
  const importBtn = document.getElementById("import");
  const importEl = document.getElementById("import-json");
  const verifyBtn = document.getElementById("verify");
  const kid = document.getElementById("kid-plain");
  const rowsPre = document.getElementById("rows-pre");
  const viewSimple = document.getElementById("view-simple");
  const viewAdvanced = document.getElementById("view-advanced");
  const advancedPanel = document.getElementById("advanced-panel");
  const names = ["Definition", "Evidence", "Impact", "Integrity", "Responsibility"];
  let lastReport = null;
  let advanced = false;
  document.body.classList.add("simple");

  function setView(next) {
    advanced = next;
    document.body.classList.toggle("simple", !advanced);
    viewSimple.classList.toggle("on", !advanced);
    viewAdvanced.classList.toggle("on", advanced);
    viewSimple.setAttribute("aria-pressed", String(!advanced));
    viewAdvanced.setAttribute("aria-pressed", String(advanced));
    if (advancedPanel) advancedPanel.hidden = !advanced;
  }

  function lines(id) {
    const el = document.getElementById(id);
    if (!el) return [];
    return el.value.split(/\n|;/).map(function (s) { return s.trim(); }).filter(Boolean);
  }

  function proposalFromForm() {
    return {
      statement: document.getElementById("statement").value,
      evidence: lines("evidence"),
      impacts_positive: lines("impacts_positive"),
      impacts_negative: lines("impacts_negative"),
      values: lines("values"),
      commitments: lines("commitments"),
      constraints: lines("constraints"),
      accountable_person: document.getElementById("accountable").value
    };
  }

  function overrides() {
    const out = {};
    names.forEach(function (name) {
      const box = document.querySelector('input[data-override="' + name + '"]');
      const note = document.querySelector('input[data-note="' + name + '"]');
      if (box && box.checked) {
        out[name] = { state: "REVISE", note: note ? note.value.trim() : "" };
      }
    });
    return out;
  }

  document.querySelectorAll("input[data-override]").forEach(function (box) {
    box.addEventListener("change", function () {
      const name = box.getAttribute("data-override");
      const note = document.querySelector('input[data-note="' + name + '"]');
      if (note) note.disabled = !box.checked;
    });
  });

  function paint(report) {
    lastReport = report;
    const ran = {};
    (report.lineage || []).forEach(function (g) { ran[g.name] = g; });
    names.forEach(function (name) {
      const li = document.querySelector('.gate[data-gate="' + name + '"]');
      if (!li) return;
      li.classList.remove("PASS", "REVISE", "BLOCK", "pending", "skipped");
      const result = ran[name];
      const stateEl = li.querySelector(".state");
      const feedbackEl = li.querySelector(".feedback");
      if (!result) {
        li.classList.add("skipped");
        stateEl.textContent = "not reached";
        feedbackEl.textContent = "Stopped before this gate. First failure ends the chain.";
        return;
      }
      li.classList.add(result.state);
      stateEl.textContent = result.state + (result.overridden ? " (override)" : "");
      feedbackEl.textContent = result.feedback || "";
    });
  }

  const final = report.final_state || "REVISE";
  banner.className = "banner " + final;
  let text = "Final: " + final;
  let kidText = "Tap Run after you type a plan.";

```



```

    if (final === "PASS") {
        text = "PASS ? all five gates survived scrutiny. This is not advice to proceed; it is clearance that the proposa
l was inspectable.";
        kidText = "All five lights said yes. That is not 'go do it.' It only means the plan was clear enough to inspect.
";
    } else if (final === "REVISE") {
        text = "REVISE ? first failure needs a more specific proposal. Feedback is on the lit gate.";
        kidText = "Stop. Make the plan clearer. Read the yellow light.";
    } else if (final === "BLOCK") {
        text = "BLOCK at " + (report.blocked_at || "a gate") + " ? cannot be remedied without changing the proposal's na
ture.";
        kidText = "Stop. This plan cannot pass unless you change what it is.";
    }
    banner.textContent = text;
    if (kid) kid.textContent = kidText;
    if (rowsPre) rowsPre.textContent = JSON.stringify(report, null, 2);
}

viewSimple.addEventListener("click", function () { setView(false); });
viewAdvanced.addEventListener("click", function () { setView(true); });

form.addEventListener("submit", function (ev) {
    ev.preventDefault();
    const body = proposalFromForm();
    body.overrides = overrides();
    fetch("/api/check", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(body)
    })
    .then(function (r) { return r.json(); })
    .then(function (r) {
        banner.className = "banner BLOCK";
        banner.textContent = "Request failed: " + err;
        if (kid) kid.textContent = "The check could not run. This is not advice.";
    })
    .catch(function (err) {
        banner.textContent = "Request failed: " + err;
        if (kid) kid.textContent = "The check could not run. This is not advice.";
    });
});

function downloadJson(obj, name) {
    const blob = new Blob([JSON.stringify(obj, null, 2)], { type: "application/json" });
    const a = document.createElement("a");
    a.href = URL.createObjectURL(blob);
    a.download = name;
    a.click();
    URL.revokeObjectURL(a.href);
}

exportBtn.addEventListener("click", function () {
    const doc = lastReport
    ? Object.assign({
        product: "DecisionGATE",
        author: "Aziel Eliab",
        version: "0.1.0"
    }, lastReport)
    : {
        product: "DecisionGATE",
        author: "Aziel Eliab",
        version: "0.1.0",
        proposal: proposalFromForm()
    };
    if (!doc.proposal) doc.proposal = proposalFromForm();
    downloadJson(doc, "decisiongate.json");
    if (kid) kid.textContent = "Saved a JSON file. Import file loads it back. This is not advice.";
});

importBtn.addEventListener("click", function () {
    importEl.click();
});

importEl.addEventListener("change", function () {
    const f = importEl.files && importEl.files[0];
    if (!f) return;
    const reader = new FileReader();
    reader.onload = function () {
        let obj;
        try { obj = JSON.parse(String(reader.result || "{}")); } catch (e) {
            banner.className = "banner BLOCK";
            banner.textContent = "That file is not JSON.";
            if (kid) kid.textContent = "Import needs a JSON file. Export file makes one.";
            return;
        }
    }
    const p = obj.proposal || obj.payload || obj;
    function set(id, v) {
        const el = document.getElementById(id);
        if (el && v != null) el.value = Array.isArray(v) ? v.join("\n") : String(v);
    }
    set("statement", p.statement);
    set("evidence", p.evidence);
    set("impacts_positive", p.impact_pos || p.impact_pos);
    set("impacts_negative", p.impact_neg || p.impact_neg);
    set("values", p.values);
    set("commitments", p.commitments);
    set("constraints", p.constraints);
    set("accountable", p.accountable_person || p.accountable);
});

```

```
    if (obj.lineage || obj.final_state) paint(obj);
    else if (kid) kid.textContent = "Loaded the file into the form. Tap Run to check it.";
  };
  reader.readAsText(f);
  importEl.value = "";
});

verifyBtn.addEventListener("click", function () {
  fetch("/api/verify")
    .then(function (r) { return r.json(); })
    .then(function (doc) {
      const lines = (doc.plain || []).join(" ");
      banner.className = "banner " + (doc.ok ? "PASS" : "BLOCK");
      banner.textContent = doc.summary || lines;
      if (kid) kid.textContent = lines || "Verify finished.";
      if (rowsPre) rowsPre.textContent = JSON.stringify(doc, null, 2);
    })
    .catch(function (err) {
      banner.className = "banner BLOCK";
      banner.textContent = "Verify failed: " + err;
    });
});
})();
```

decisiongate/decisiongate/web/index.html

```

<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>DecisionGATE ? pre-execution filter</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body class="simple">
  <header>
    <div class="tag">DecisionGATE ? v0.1.0 ? loopback ? not advice</div>
    <h1>DecisionGATE</h1>
    <p class="motto">Freedom without clarity is chaos. Clarity without force is wisdom.</p>
    <p class="lede">
      Type a plan. Tap Run. Five lights say yes, change it, or stop.
      Green is a check, not a command to do the thing. Author Aziel Eliab.
    </p>
  </header>

  <aside class="banner limit" role="note">
    THIS IS: a five-gate ethical pre-execution filter (PASS / REVISE / BLOCK).
    THIS IS NOT: a predictor, a court, a truth score, advice, or a remote command runner.
    wrap is not hosted. Author Aziel Eliab.
  </aside>

  <div class="view-toggle" role="group" aria-label="Simple or advanced view">
    <button type="button" id="view-simple" class="view on" aria-pressed="true">Simple</button>
    <button type="button" id="view-advanced" class="view" aria-pressed="false">Advanced</button>
  </div>

  <form id="proposal-form" autocomplete="off">
    <fieldset>
      <legend>Proposal</legend>

      <label for="statement">
        <span class="kicker">Statement</span>
        What will you do? Say it clearly. Vague plans fail the first check.
      </label>
      <textarea id="statement" name="statement" rows="3" placeholder="Release DecisionGATE 0.1.0 as a standalone Python package under Apache-2.0 on GitHub this month."></textarea>

      <label for="evidence">
        <span class="kicker">Evidence</span>
        Facts. One per line. Empty fails Evidence.
      </label>
      <textarea id="evidence" name="evidence" rows="3" placeholder="Package builds with stdlib only.&#10;Whitepaper dated July 2026 specifies five gates."></textarea>

      <div class="pair">
        <div>
          <label for="impacts_positive">
            <span class="kicker">Positive impacts</span>
            Who is helped. One per line.
          </label>
          <textarea id="impacts_positive" name="impacts_positive" rows="3" placeholder="Authors get a named scrutiny path."></textarea>
        </div>
        <div>
          <label for="impacts_negative">
            <span class="kicker">Negative impacts</span>
            Who is burdened. One per line.
          </label>
          <textarea id="impacts_negative" name="impacts_negative" rows="3" placeholder="Vague proposals take longer to rewrite."></textarea>
        </div>
      </div>

      <label for="values">
        <span class="kicker">Values</span>
        What you already said you care about. One per line.
      </label>
      <textarea id="values" name="values" rows="2" placeholder="Clarity without force. Named responsibility."></textarea>
    </fieldset>

    <div class="advanced-only">
      <label for="commitments">
        <span class="kicker">Commitments</span>
        Prior commitments. One per line. Optional.
      </label>
      <textarea id="commitments" name="commitments" rows="2" placeholder="Apache-2.0. Forks welcome."></textarea>

      <label for="constraints">
        <span class="kicker">Constraints</span>
        Hard bounds. One per line. A statement that contradicts a prohibition is BLOCK.
      </label>
      <textarea id="constraints" name="constraints" rows="2" placeholder="Do not bind the UI on 0.0.0.0."></textarea>
    </div>

    <label for="accountable">
      <span class="kicker">Accountable person</span>
      One named owner. Blank is BLOCK.
    </label>
  </form>

```

```

</label>
<input id="accountable" name="accountable" type="text" placeholder="Aziel Eliab" spellcheck="false">
</fieldset>

<div class="actions simple-actions">
  <button type="submit" id="run" class="big">Run</button>
  <button type="button" id="import" class="big">Import file</button>
  <button type="button" id="export" class="big">Export file</button>
  <button type="button" id="verify" class="big">Verify</button>
  <input type="file" id="import-json" accept="application/json,.json" hidden>
</div>
</form>

<section class="gates" aria-live="polite">
  <h2>Gates</h2>
  <ol id="gate-stack">
    <li class="gate pending" data-gate="Definition">
      <div class="lamp" aria-hidden="true"></div>
      <div class="body">
        <div class="title"><span class="n">1</span> Definition</div>
        <p class="state">waiting</p>
        <p class="feedback">Say the plan in plain words. Vague or shifting ? REVISE or BLOCK.</p>
        <label class="override advanced-only">
          <input type="checkbox" data-override="Definition"> Override to REVISE
        </label>
        <input type="text" class="note advanced-only" data-note="Definition" placeholder="Override note (recorded in
lineage)" disabled>
      </div>
    </li>
    <li class="gate pending" data-gate="Evidence">
      <div class="lamp" aria-hidden="true"></div>
      <div class="body">
        <div class="title"><span class="n">2</span> Evidence</div>
        <p class="state">waiting</p>
        <p class="feedback">Facts, data, observations. Ungrounded assertions ? REVISE.</p>
        <label class="override advanced-only">
          <input type="checkbox" data-override="Evidence"> Override to REVISE
        </label>
        <input type="text" class="note advanced-only" data-note="Evidence" placeholder="Override note (recorded in l
ineage)" disabled>
      </div>
    </li>
    <li class="gate pending" data-gate="Impact">
      <div class="lamp" aria-hidden="true"></div>
      <div class="body">
        <div class="title"><span class="n">3</span> Impact</div>
        <p class="state">waiting</p>
        <p class="feedback">Who is helped AND who is burdened. Hidden impacts ? REVISE.</p>
        <label class="override advanced-only">
          <input type="checkbox" data-override="Impact"> Override to REVISE
        </label>
        <input type="text" class="note advanced-only" data-note="Impact" placeholder="Override note (recorded in lin
eage)" disabled>
      </div>
    </li>
    <li class="gate pending" data-gate="Integrity">
      <div class="lamp" aria-hidden="true"></div>
      <div class="body">
        <div class="title"><span class="n">4</span> Integrity</div>
        <p class="state">waiting</p>
        <p class="feedback">Stay consistent with stated values. Contradictions ? REVISE or BLOCK.</p>
        <label class="override advanced-only">
          <input type="checkbox" data-override="Integrity"> Override to REVISE
        </label>
        <input type="text" class="note advanced-only" data-note="Integrity" placeholder="Override note (recorded in
lineage)" disabled>
      </div>
    </li>
    <li class="gate pending" data-gate="Responsibility">
      <div class="lamp" aria-hidden="true"></div>
      <div class="body">
        <div class="title"><span class="n">5</span> Responsibility</div>
        <p class="state">waiting</p>
        <p class="feedback">Name one owner. No name ? BLOCK.</p>
        <label class="override advanced-only">
          <input type="checkbox" data-override="Responsibility"> Override to REVISE
        </label>
        <input type="text" class="note advanced-only" data-note="Responsibility" placeholder="Override note (recorde
d in lineage)" disabled>
      </div>
    </li>
  </ol>
</section>

<div id="banner" class="banner idle" role="status">
  No run yet. Type a plan. Tap Run. First failure stops the chain.
</div>
<p id="kid-plain" class="kid">Three taps: Import file, Run, Export file. Verify checks the program in plain words. T
his is not advice.</p>

<section id="advanced-panel" class="advanced-panel advanced-only" hidden>
  <h2>JSON</h2>
  <pre id="rows-pre">{</pre>
</section>
<section class="card" id="aziel-json-io">

```

```

<h2>Import / Export JSON</h2>
<p class="help">Tap Import JSON file to pick a .json. Export JSON saves the current session locally. Nothing is uplo
aded.</p>
<input id="aziel-import-json" type="file" accept="application/json,.json">
<p>
  <button type="button" id="aziel-import-json-btn">Import JSON file</button>
  <button type="button" id="aziel-export-json-btn">Export JSON</button>
</p>
<p class="help" id="aziel-json-status"></p>
</section>

<footer>
  <p>Apache-2.0 ? Aziel Eliab ? July 2026 ? Bound to 127.0.0.1 ? Forks welcome and always allowed.</p>
  <p class="foot-note">This UI does not advise, predict, or prescribe. It filters. DOI 10.5281/zenodo.21435730</p>
</footer>
<script src="/app.js"></script>

<script>
(function(){
  var file = document.getElementById("aziel-import-json");
  var imp = document.getElementById("aziel-import-json-btn");
  var exp = document.getElementById("aziel-export-json-btn");
  var status = document.getElementById("aziel-json-status");
  if (!file || !imp || !exp) return;
  function say(m){ if (status) status.textContent = m; }
  function collect(){
    var data = { product: document.title || "", exported_at: new Date().toISOString(), author: "Aziel Eliab" };
    document.querySelectorAll("input, select, textarea").forEach(function(el){
      if (!el.id || el.type === "file" || el.type === "password") return;
      data[el.id] = el.type === "checkbox" ? el.checked : el.value;
    });
    if (window.__azielLastJson && typeof window.__azielLastJson === "object") {
      data.last = window.__azielLastJson;
    }
    return data;
  }
  function apply(obj){
    if (!obj || typeof obj !== "object") return;
    window.__azielLastJson = obj;
    Object.keys(obj).forEach(function(k){
      if (k === "last" || k === "product" || k === "exported_at" || k === "author") return;
      var el = document.getElementById(k);
      if (!el || el.type === "file" || el.type === "password") return;
      if (el.type === "checkbox") el.checked = !!obj[k];
      else if ("value" in el) el.value = obj[k];
    });
    var ta = document.querySelector("textarea");
    if (ta && obj && !obj[ta.id] && typeof obj === "object") {
      try { if (!ta.value) ta.value = JSON.stringify(obj, null, 2); } catch (e) {}
    }
  }
  imp.addEventListener("click", function(){ file.click(); });
  file.addEventListener("change", function(){
    var f = file.files && file.files[0];
    if (!f) return;
    var r = new FileReader();
    r.onload = function(){
      try { apply(JSON.parse(String(r.result || "{}"))); say("Imported " + f.name); }
      catch (e) { say("Invalid JSON"); }
    };
    r.readAsText(f);
  });
  exp.addEventListener("click", function(){
    var blob = new Blob([JSON.stringify(collect(), null, 2)], {type: "application/json"});
    var a = document.createElement("a");
    a.href = URL.createObjectURL(blob);
    a.download = "session.json";
    a.click();
    setTimeout(function(){ URL.revokeObjectURL(a.href); }, 800);
    say("Exported JSON");
  });
})();
</script>

</body>
</html>

```

decisiongate/decisiongate/web/style.css

```

/* DecisionGATE local UI. No CDN. Pre-execution filter, not advice. */
:root {
  --bg: #0f1419;
  --panel: #171e27;
  --ink: #e8edf2;
  --muted: #8b97a6;
  --line: #2a3544;
  --pass: #3dba7a;
  --pass-dim: #1d4a33;
  --revise: #e0a13a;
  --revise-dim: #5a4318;
  --block: #d4534b;
  --block-dim: #5a221f;
  --idle: #3d4a58;
  --gold: #d4bc6a;
  --focus: #7aa2d4;
  font-synthesis: none;
}
* { box-sizing: border-box; }
html, body {
  margin: 0;
  padding: 0;
  background: var(--bg);
  color: var(--ink);
  font-family: "Source Serif 4", "Iowan Old Style", Palatino, "Palatino Linotype", Georgia, serif;
  line-height: 1.45;
}
body {
  max-width: 46rem;
  margin: 0 auto;
  padding: 2.1rem 1.2rem 4rem;
}
.tag {
  font-family: ui-monospace, "SF Mono", Menlo, Consolas, monospace;
  font-size: 0.72rem;
  letter-spacing: 0.14em;
  text-transform: uppercase;
  color: var(--muted);
}
h1 {
  font-size: 2rem;
  font-weight: 650;
  letter-spacing: 0.04em;
  margin: 0.35rem 0 0.25rem;
}
.motto {
  color: var(--gold);
  font-style: italic;
  margin: 0 0 0.85rem;
  font-size: 1.05rem;
}
.lede {
  color: var(--muted);
  margin: 0 0 1.5rem;
  max-width: 40rem;
}
fieldset {
  border: 1px solid var(--line);
  border-radius: 10px;
  background: var(--panel);
  padding: 1.1rem 1.15rem 1.2rem;
  margin: 0 0 1rem;
}
legend {
  font-family: ui-monospace, "SF Mono", Menlo, Consolas, monospace;
  font-size: 0.72rem;
  letter-spacing: 0.12em;
  text-transform: uppercase;
  color: var(--muted);
  padding: 0 0.4rem;
}
label {
  display: block;
  font-size: 0.92rem;
  margin: 0.85rem 0 0.3rem;
}
label.kicker {
  display: block;
  font-family: ui-monospace, "SF Mono", Menlo, Consolas, monospace;
  font-size: 0.68rem;
  letter-spacing: 0.12em;
  text-transform: uppercase;
  color: var(--muted);
  margin-bottom: 0.12rem;
}
textarea, input[type="text"] {
  width: 100%;
  padding: 0.55rem 0.65rem;
  border: 1px solid var(--line);
  border-radius: 6px;
  background: #10161d;
  color: var(--ink);
}

```

```

    font: inherit;
    font-size: 0.98rem;
  }
  textarea:focus, input[type="text"]:focus {
    outline: 2px solid var(--focus);
    outline-offset: 1px;
  }
  .pair {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 0.85rem;
  }
  @media (max-width: 640px) {
    .pair { grid-template-columns: 1fr; }
  }
  .actions {
    display: flex;
    gap: 0.65rem;
    flex-wrap: wrap;
    margin: 0 0 1.6rem;
  }
  button {
    font-family: ui-monospace, "SF Mono", Menlo, Consolas, monospace;
    font-size: 0.85rem;
    letter-spacing: 0.04em;
    padding: 0.65rem 1rem;
    border-radius: 8px;
    border: 1px solid var(--ink);
    background: var(--ink);
    color: var(--bg);
    cursor: pointer;
    font-weight: 650;
  }
  button:disabled {
    opacity: 0.4;
    cursor: not-allowed;
  }
  button#export {
    background: transparent;
    color: var(--ink);
  }
  h2 {
    font-size: 1.05rem;
    letter-spacing: 0.08em;
    text-transform: uppercase;
    color: var(--muted);
    font-weight: 600;
    margin: 0 0 0.7rem;
  }
  #gate-stack {
    list-style: none;
    margin: 0;
    padding: 0;
    display: flex;
    flex-direction: column;
    gap: 0.65rem;
  }
  .gate {
    display: grid;
    grid-template-columns: 1.15rem 1fr;
    gap: 0.85rem;
    border: 1px solid var(--line);
    border-radius: 10px;
    padding: 0.85rem 1rem;
    background: var(--panel);
  }
  .lamp {
    width: 0.85rem;
    height: 0.85rem;
    border-radius: 50%;
    margin-top: 0.35rem;
    background: var(--idle);
    box-shadow: 0 0 3px #1c2430;
  }
  .gate.PASS .lamp { background: var(--pass); box-shadow: 0 0 3px var(--pass-dim); }
  .gate.REVISE .lamp { background: var(--revise); box-shadow: 0 0 3px var(--revise-dim); }
  .gate.BLOCK .lamp { background: var(--block); box-shadow: 0 0 3px var(--block-dim); }
  .gate.skipped { opacity: 0.55; }
  .title {
    font-weight: 650;
    letter-spacing: 0.02em;
  }
  .title .n {
    font-family: ui-monospace, "SF Mono", Menlo, Consolas, monospace;
    color: var(--muted);
    margin-right: 0.35rem;
  }
  .gate .state {
    margin: 0.15rem 0 0.25rem;
    font-family: ui-monospace, "SF Mono", Menlo, Consolas, monospace;
    font-size: 0.78rem;
    letter-spacing: 0.12em;
    text-transform: uppercase;
    color: var(--muted);
  }
}

```

```

.gate.PASS .state { color: var(--pass); }
.gate.REVISE .state { color: var(--revise); }
.gate.BLOCK .state { color: var(--block); }
.gate .feedback {
  margin: 0 0 0.55rem;
  color: var(--ink);
  font-size: 0.95rem;
}
.override {
  font-size: 0.85rem;
  color: var(--muted);
  margin: 0 0 0.3rem;
  display: flex;
  align-items: center;
  gap: 0.4rem;
}
.override input { width: auto; }
.gate .note { margin-top: 0.15rem; }
.banner {
  margin-top: 1.15rem;
  padding: 0.9rem 1rem;
  border-radius: 10px;
  border: 1px solid var(--line);
  font-weight: 650;
  letter-spacing: 0.02em;
}
.banner.idle { background: var(--panel); color: var(--muted); }
.banner.PASS { background: var(--pass-dim); color: var(--pass); border-color: var(--pass); }
.banner.REVISE { background: var(--revise-dim); color: var(--revise); border-color: var(--revise); }
.banner.BLOCK { background: var(--block-dim); color: var(--block); border-color: var(--block); }
.footer {
  margin-top: 2rem;
  color: var(--muted);
  font-size: 0.88rem;
}
.foot-note { font-style: italic; }

.banner.limit {
  background: #1a1610;
  color: var(--ink);
  border-color: #5c4a1a;
  font-weight: 500;
  margin: 0 0 1.2rem;
}
.view-toggle {
  display: flex;
  gap: 0.4rem;
  margin: 0 0 1rem;
}
button.view {
  background: transparent;
  color: var(--ink);
  border-color: var(--line);
}
button.view.on {
  background: var(--gold);
  color: var(--bg);
  border-color: var(--gold);
}
button.big {
  font-size: 1.05rem;
  padding: 0.8rem 1.2rem;
  min-width: 7.2rem;
}
button#import, button#export, button#verify {
  background: transparent;
  color: var(--ink);
}
body.simple .advanced-only { display: none !important; }
.kid {
  font-size: 1.05rem;
  color: var(--ink);
  margin: 0.85rem 0 0;
}
.advanced-panel pre {
  background: #10161d;
  border: 1px solid var(--line);
  border-radius: 8px;
  padding: 0.8rem;
  overflow: auto;
  font-size: 0.78rem;
  max-height: 22rem;
}

```


decisiongate/install.sh

```
#!/usr/bin/env bash
# DecisionGATE one-click install. Counted download via this project's Worker.
# Usage: curl -fsSL https://decisiongate-download-tracker.vibelock.workers.dev/install.sh | bash
set -euo pipefail

HOST="${DECISIONGATE_HOME_HOST:-https://decisiongate-download-tracker.vibelock.workers.dev}"
ASSET="${DECISIONGATE_HOME_ASSET:-decisiongate-0.1.0.tar.gz}"
WORKDIR="${DECISIONGATE_HOME:-$HOME/decisiongate}"

mkdir -p "$WORKDIR"
cd "$WORKDIR"

echo "Downloading counted tarball from ${HOST}/download (User-Agent Mozilla/5.0)"
curl -fsSL -A 'Mozilla/5.0' "${HOST}/download?asset=${ASSET}" -o "${ASSET}"

tar -xzf "${ASSET}"
DIR="$(find . -maxdepth 1 -type d -name 'decisiongate-*' | head -n 1)"
if [ -n "$DIR" ]; then
    cd "$DIR"
fi

python3 -m venv .venv
# shellcheck disable=SC1091
. .venv/bin/activate
python -m pip install -U pip
python -m pip install -e .

echo
echo "Installed DecisionGATE."
echo "Run: decisiongate ui"
echo "Then open http://127.0.0.1:8791 (loopback only)"
echo "Author: Aziel Eliab."
```

decisiongate/mobile/lib/gates.dart

```

/// Dart port of decisiongate/gates.py heuristics (v0.1). Sequential.
/// First non-PASS stops the chain. Not predictive, advisory, or prescriptive.

const pass = 'PASS';
const revise = 'REVISE';
const block = 'BLOCK';

const motto =
  'Freedom without clarity is chaos. Clarity without force is wisdom.';

const hedges = {'maybe', 'somehow', 'stuff', 'things'};

const commonVerbs = {
  'is', 'are', 'was', 'were', 'be', 'been', 'being', 'am',
  'have', 'has', 'had', 'do', 'does', 'did', 'will', 'would',
  'shall', 'should', 'can', 'could', 'must', 'need', 'needs',
  'make', 'makes', 'made', 'take', 'takes', 'took', 'give',
  'gives', 'gave', 'go', 'goes', 'went', 'come', 'keep', 'put',
  'use', 'uses', 'used', 'set', 'get', 'let', 'allow', 'allows',
  'publish', 'release', 'releases', 'deploy', 'ship', 'open',
  'close', 'create', 'created', 'delete', 'write', 'read', 'run',
  'execute', 'adopt', 'reject', 'approve', 'block', 'collect',
  'store', 'share', 'send', 'build', 'launch', 'hire', 'spend',
  'buy', 'sell', 'migrate', 'replace', 'update', 'install',
  'announce', 'commit', 'sign', 'fund', 'grant', 'revoke',
  'host', 'serve', 'bind', 'filter', 'record', 'name', 'assign',
  'document', 'provide', 'provides', 'include', 'includes',
  'add', 'remove', 'stop', 'start', 'move', 'change', 'apply',
  'submit', 'accept', 'refuse', 'pay', 'offer', 'request',
  'require', 'requires', 'implement', 'implements',
};

const negationMarkers = [
  'do not ',
  'don't ',
  'does not ',
  'doesn't ',
  'must not ',
  'cannot ',
  'can't ',
  'never ',
  'no ',
  'not ',
  'without ',
];

class Proposal {
  Proposal({
    this.statement = '',
    this.evidence = const [],
    this.impactsPositive = const [],
    this.impactsNegative = const [],
    this.values = const [],
    this.constraints = const [],
    this.accountablePerson = '',
  });

  final String statement;
  final List<String> evidence;
  final List<String> impactsPositive;
  final List<String> impactsNegative;
  final List<String> values;
  final List<String> constraints;
  final String accountablePerson;

  List<String> _lines(List<String> xs) =>
    xs.map((e) => e.trim()).where((e) => e.isNotEmpty).toList();

  Proposal normalized() => Proposal(
    statement: statement.trim(),
    evidence: _lines(evidence),
    impactsPositive: _lines(impactsPositive),
    impactsNegative: _lines(impactsNegative),
    values: _lines(values),
    constraints: _lines(constraints),
    accountablePerson: accountablePerson.trim(),
  );
}

class GateResult {
  GateResult({
    required this.name,
    required this.state,
    required this.feedback,
  });
  final String name;
  final String state;
  final String feedback;
}

class Report {
  Report({

```

```

        required this.lineage,
        required this.finalState,
        this.blockedAt,
    });
    final List<GateResult> lineage;
    final String finalState;
    final String? blockedAt;
}

final _wordRe = RegExp(r'[A-Za-z0-9][A-Za-z0-9._+]*');

List<String> tokenize(String text) =>
    _wordRe.allMatches(text.toLowerCase()).map((m) => m.group(0)!).toList();

bool _looksLikeVerb(String token) {
    if (commonVerbs.contains(token)) return true;
    if (token.length > 4 &&
        (token.endsWith('ing') ||
         token.endsWith('ize') ||
         token.endsWith('ise') ||
         token.endsWith('ify'))) {
        return true;
    }
    if (token.length > 4 && token.endsWith('ed')) return true;
    return false;
}

bool hasVerbAndObject(List<String> tokens) {
    final content = tokens.where((t) => !hedges.contains(t)).toList();
    if (content.length < 2) return false;
    return content.any(_looksLikeVerb);
}

GateResult gateDefinition(Proposal p) {
    final statement = p.statement.trim();
    if (statement.isEmpty) {
        return GateResult(
            name: 'Definition',
            state: block,
            feedback:
                'Statement is empty. A proposal with no concrete statement cannot '
                'pass Definition. Write an unambiguous action with a verb and an '
                'object, at least 12 words.',
        );
    }
    final tokens = tokenize(statement);
    if (tokens.length < 12) {
        return GateResult(
            name: 'Definition',
            state: revise,
            feedback:
                'Statement has ${tokens.length} word(s); Definition requires at '
                'least 12. Expand into a concrete, unambiguous proposal.',
        );
    }
    if (!hasVerbAndObject(tokens)) {
        return GateResult(
            name: 'Definition',
            state: revise,
            feedback:
                'Statement is hedge-only or lacks a verb+object after removing '
                'maybe/somehow/stuff/things. Name a specific action and its object.',
        );
    }
    return GateResult(
        name: 'Definition',
        state: pass,
        feedback: 'Statement is concrete enough to scrutinize (length, verb+object).',
    );
}

GateResult gateEvidence(Proposal p) {
    final items = p.evidence.where((e) => e.trim().isNotEmpty).toList();
    if (items.isEmpty) {
        return GateResult(
            name: 'Evidence',
            state: revise,
            feedback:
                'Evidence list is empty. Identify at least one fact, datum, or '
                'observation that grounds the statement.',
        );
    }
    return GateResult(
        name: 'Evidence',
        state: pass,
        feedback: '${items.length} evidence item(s) identified.',
    );
}

GateResult gateImpact(Proposal p) {
    final pos = p.impactPositive.where((e) => e.trim().isNotEmpty).toList();
    final neg = p.impactNegative.where((e) => e.trim().isNotEmpty).toList();
    final missing = <String>[];
    if (pos.isEmpty) missing.add('positive');
    if (neg.isEmpty) missing.add('negative');
}

```

```

    if (missing.isEmpty) {
        return GateResult(
            name: 'Impact',
            state: revise,
            feedback:
                'Impact list(s) empty: ${missing.join(' and ')}. Name who or what '
                'is affected on both the positive and the negative side.',
        );
    }
    return GateResult(
        name: 'Impact',
        state: pass,
        feedback: '${pos.length} positive and ${neg.length} negative impact(s) named.',
    );
}

String? _constraintPayload(String constraint) {
    var text = constraint.toLowerCase().split(RegExp(r'\s+')).join(' ');
    if (text.isEmpty) return null;
    var found = false;
    var remainder = ' $text ';
    for (final marker in negationMarkers) {
        final padded = marker.startsWith(' ') ? marker : ' $marker';
        if (remainder.contains(padded) || remainder.trimLeft().startsWith(marker)) {
            found = true;
            remainder = remainder.replaceAll(padded, ' ');
            final trimmed = remainder.trimLeft();
            if (trimmed.startsWith(marker)) {
                remainder = ' ${trimmed.substring(marker.length)}';
            }
        }
    }
    final payload = remainder.split(RegExp(r'\s+')).where((s) => s.isNotEmpty).join(' ');
    if (found && payload.isNotEmpty) return payload;
    return null;
}

bool statementContradicts(String statement, String constraint) {
    final payload = _constraintPayload(constraint);
    if (payload == null) return false;
    final hay = statement.toLowerCase().split(RegExp(r'\s+')).join(' ');
    return hay.contains(payload);
}

GateResult gateIntegrity(Proposal p) {
    final values = p.values.where((e) => e.trim().isNotEmpty).toList();
    if (values.isEmpty) {
        return GateResult(
            name: 'Integrity',
            state: revise,
            feedback:
                'Values list is empty. Integrity requires stated values so the '
                'proposal can be checked against them.',
        );
    }
    final hits = <String>[];
    for (final c in p.constraints) {
        if (statementContradicts(p.statement, c)) hits.add(c);
    }
    if (hits.isNotEmpty) {
        return GateResult(
            name: 'Integrity',
            state: block,
            feedback:
                'Statement contradicts a provided constraint (${hits.first}). '
                'A contradiction of this kind cannot pass Integrity without '
                'changing the proposal\'s nature.',
        );
    }
    return GateResult(
        name: 'Integrity',
        state: pass,
        feedback:
            '${values.length} value(s) stated; no constraint contradiction detected.',
    );
}

GateResult gateResponsibility(Proposal p) {
    final owner = p.accountablePerson.trim();
    if (owner.isEmpty) {
        return GateResult(
            name: 'Responsibility',
            state: block,
            feedback:
                'Accountable person is blank. Diffuse or absent ownership cannot '
                'pass Responsibility. Name one accountable owner.',
        );
    }
    return GateResult(
        name: 'Responsibility',
        state: pass,
        feedback: 'Accountable owner named: $owner.',
    );
}

Report runGates(Proposal raw) {

```

```
final p = raw.normalized();
final fns = [
  gateDefinition,
  gateEvidence,
  gateImpact,
  gateIntegrity,
  gateResponsibility,
];
final lineage = <GateResult>[];
var finalState = pass;
String? blockedAt;
for (final fn in fns) {
  final result = fn(p);
  lineage.add(result);
  if (result.state != pass) {
    finalState = result.state;
    if (result.state == block) blockedAt = result.name;
    break;
  }
}
return Report(lineage: lineage,
  age, finalState: finalState, blockedAt: blockedAt);
}

List<String> splitLines(String text) => text
  .replaceAll('\r\n', '\n')
  .split(RegExp(r'[\n;]'))
  .map((s) => s.trim())
  .where((s) => s.isNotEmpty)
  .toList();
```

decisiongate/mobile/lib/main.dart

```

import 'package:flutter/material.dart';

import 'gates.dart';
import 'theme.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const DecisionGateApp());
}

class DecisionGateApp extends StatelessWidget {
  const DecisionGateApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'DecisionGATE',
      debugShowCheckedModeBanner: false,
      theme: buildAppTheme(),
      home: const FormPage(),
    );
  }
}

class FormPage extends StatefulWidget {
  const FormPage({super.key});

  @override
  State<FormPage> createState() => _FormPageState();
}

class _FormPageState extends State<FormPage> {
  final _statement = TextEditingController();
  final _evidence = TextEditingController();
  final _pos = TextEditingController();
  final _neg = TextEditingController();
  final _values = TextEditingController();
  final _constraints = TextEditingController();
  final _owner = TextEditingController();

  @override
  void dispose() {
    _statement.dispose();
    _evidence.dispose();
    _pos.dispose();
    _neg.dispose();
    _values.dispose();
    _constraints.dispose();
    _owner.dispose();
    super.dispose();
  }

  void _run() {
    final report = runGates(Proposal(
      statement: _statement.text,
      evidence: splitLines(_evidence.text),
      impactsPositive: splitLines(_pos.text),
      impactsNegative: splitLines(_neg.text),
      values: splitLines(_values.text),
      constraints: splitLines(_constraints.text),
      accountablePerson: _owner.text,
    ));
    Navigator.of(context).push(
      MaterialPageRoute<void>(
        builder: (_) => ResultPage(report: report),
      ),
    );
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('DecisionGATE')),
    body: ListView(
      padding: const EdgeInsets.all(16),
      children: [
        Text(
          motto,
          style: Theme.of(context).textTheme.titleMedium?.copyWith(
            color: kGold,
            fontStyle: FontStyle.italic,
          ),
        ),
        const SizedBox(height: 8),
        const Text(
          'Five sequential gates. First failure stops the chain. '
          'PASS is clearance that the proposal survived scrutiny, not a '
          'suggestion to act. Offline. No analytics.',
        ),
        const SizedBox(height: 16),
        TextField(
          controller: _statement,

```

```

        maxLines: 4,
        decoration: const InputDecoration(
          labelText: '1. Definition ? concrete statement',
        ),
      ),
      const SizedBox(height: 12),
      TextField(
        controller: _evidence,
        maxLines: 3,
        decoration: const InputDecoration(
          labelText: '2. Evidence (one item per line)',
        ),
      ),
      const SizedBox(height: 12),
      TextField(
        controller: _pos,
        maxLines: 2,
        decoration: const InputDecoration(
          labelText: '3a. Impact ? positive',
        ),
      ),
      const SizedBox(height: 12),
      TextField(
        controller: _neg,
        maxLines: 2,
        decoration: const InputDecoration(
          labelText: '3b. Impact ? negative',
        ),
      ),
      const SizedBox(height: 12),
      TextField(
        controller: _values,
        decoration: const InputDecoration(
          labelText: '4. Integrity ? values (line or ; separated)',
        ),
      ),
      const SizedBox(height: 12),
      TextField(
        controller: _constraints,
        decoration: const InputDecoration(
          labelText: '4b. Constraints (optional prohibitions)',
        ),
      ),
      const SizedBox(height: 12),
      TextField(
        controller: _owner,
        decoration: const InputDecoration(
          labelText: '5. Responsibility ? accountable owner',
        ),
      ),
      const SizedBox(height: 20),
      FilledButton(
        onPressed: _run,
        child: const Text('Run gates'),
      ),
    ],
  ),
);
}
}

```

```

class ResultPage extends StatelessWidget {
  const ResultPage({super.key, required this.report});
  final Report report;

```

```

  Color _color(String state) {
    switch (state) {
      case pass:
        return const Color(0xFF3D8B5F);
      case revise:
        return const Color(0xFFC9A227);
      case block:
        return const Color(0xFFB54A4A);
      default:
        return kIvory;
    }
  }
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Lineage')),
    body: ListView(
      padding: const EdgeInsets.all(16),
      children: [
        Text(
          motto,
          style: Theme.of(context)
            .textTheme
            .bodyLarge
            ?.copyWith(color: kGold, fontStyle: FontStyle.italic),
        ),
        const SizedBox(height: 12),
        Card(
          child: Padding(

```

```

padding: const EdgeInsets.all(16),
child: Text(
  'Final: ${report.finalState}'
  '${report.blockedAt != null ? ' blocked at ${report.blockedAt}' : ''}',
  style: Theme.of(context).textTheme.headlineSmall?.copyWith(
    color: _color(report.finalState),
  ),
),
),
),
const SizedBox(height: 8),
const Text(
  'Later gates are not evaluated after a failure.',
  style: TextStyle(fontStyle: FontStyle.italic),
),
const SizedBox(height: 12),
for (final g in report.lineage)
  Card(
    child: ListTile(
      leading: CircleAvatar(
        backgroundColor: _color(g.state),
        child: Text(
          g.state[0],
          style: const TextStyle(color: kMatteBlack),
        ),
      ),
      title: Text('${g.name} ? ${g.state}'),
      subtitle: Text(g.feedback),
      isThreeLine: true,
    ),
  ),
],
);
}
}

```


decisiongate/mobile/lib/theme.dart

```

import 'package:flutter/material.dart';

/// Matte black + gold Material 3 dark theme. No analytics.
const Color kMatteBlack = Color(0xFF0B0B0B);
const Color kSurface = Color(0xFF141414);
const Color kGold = Color(0xFFC9A227);
const Color kGoldDim = Color(0xFF8A7219);
const Color kIvory = Color(0xFFE8E0D0);

ThemeData buildAppTheme() {
  const scheme = ColorScheme.dark(
    brightness: Brightness.dark,
    primary: kGold,
    onPrimary: kMatteBlack,
    secondary: kGoldDim,
    onSecondary: kIvory,
    surface: kSurface,
    onSurface: kIvory,
    error: Color(0xFFB54A4A),
    onError: kIvory,
  );
  return ThemeData(
    useMaterial3: true,
    brightness: Brightness.dark,
    colorScheme: scheme,
    scaffoldBackgroundColor: kMatteBlack,
    appBarTheme: const AppBarTheme(
      backgroundColor: kMatteBlack,
      foregroundColor: kGold,
      elevation: 0,
      centerTitle: false,
    ),
    cardTheme: CardThemeData(
      color: kSurface,
      elevation: 0,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
        side: const BorderSide(color: Color(0x33C9A227)),
      ),
    ),
    inputDecorationTheme: InputDecorationTheme(
      filled: true,
      fillColor: const Color(0xFF1A1A1A),
      border: OutlineInputBorder(borderRadius: BorderRadius.circular(10)),
      focusedBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(10),
        borderSide: const BorderSide(color: kGold),
      ),
    ),
    segmentedButtonTheme: SegmentedButtonThemeData(
      style: ButtonStyle(
        foregroundColor: WidgetStateProperty.resolveWith((s) {
          return s.contains(WidgetState.selected) ? kMatteBlack : kIvory;
        }),
        backgroundColor: WidgetStateProperty.resolveWith((s) {
          return s.contains(WidgetState.selected) ? kGold : kSurface;
        }),
      ),
    ),
  );
}

```

decisiongate/mobile/pubspec.yaml

```
name: decisiongate
description: Local-first DecisionGATE five-gate filter. Sequential PASS/REVISE/BLOCK. Offline.
publish_to: "none"
version: 0.1.0+1

environment:
  sdk: ">=3.3.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0

flutter:
  uses-material-design: true
```

decisiongate/pyproject.toml

```

[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "decisiongate"
version = "0.1.0"
description = "DecisionGATE: a lightweight ethical pre-execution filter. Not predictive, advisory, or prescriptive."
readme = "README.md"
license = {file = "LICENSE"}
authors = [{name = "Aziel Eliab"}]
requires-python = ">=3.10"
keywords = [
    "ethics",
    "filter",
    "pre-execution",
    "gates",
    "scrutiny",
    "decision",
]
classifiers = [
    "Development Status :: 4 - Beta",
    "License :: OSI Approved :: Apache Software License",
    "Programming Language :: Python :: 3",
    "Programming Language :: Python :: 3.10",
    "Programming Language :: Python :: 3.11",
    "Programming Language :: Python :: 3.12",
    "Programming Language :: Python :: 3.13",
    "Topic :: Sociology",
    "Intended Audience :: Developers",
]
dependencies = []

[project.optional-dependencies]
dev = ["pytest>=7.4"]

[project.scripts]
decisiongate = "decisiongate.cli:main"

[project.urls]
Homepage = "https://github.com/AzielEliab/decisiongate"
Download = "https://github.com/AzielEliab/decisiongate/releases"
Issues = "https://github.com/AzielEliab/decisiongate/issues"
Counted-downloads = "https://decisiongate-download-tracker.vibelock.workers.dev/"

[tool.setuptools.packages.find]
include = ["decisiongate*"]

[tool.setuptools.package-data]
decisiongate = ["web/*"]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]
addopts = "-q"

```

decisiongate/workers/download-tracker/package.json

```
{
  "dependencies": {
    "wrangler": "^4.127.1"
  }
}
```

decisiongate/workers/download-tracker/src/index.js

```

import { handleRuntimeApi } from "../runtime.js";

/**
 * DecisionGATE download tracker (Cloudflare Worker).
 *
 * GET /download?repo=AzielEliab/decisiongate&tag=latest&asset=...
 *      increments KV, serves gzip via ASSETS.fetch (no 302)
 *      (default https://github.com/AzielEliab/decisiongate/releases)
 * GET /stats JSON totals + per-repo + per-branch breakdown
 * POST /event forks report a download {owner,repo,branch,fork,asset}
 *
 * KV binding DOWNLOADS. Keys: project|owner|repo|branch|fork
 * CORS *. No secrets in this tree.
 * Isolated counter: project decisiongate only.
 */

const PROJECT = "decisiongate";
const DEFAULT_ASSET = "decisiongate-0.1.0.tar.gz";
const DEFAULT_OWNER = "AzielEliab";
const DEFAULT_REPO = "decisiongate";
const DEFAULT_BRANCH = "main";
const GITHUB_RELEASES = "https://github.com/AzielEliab/decisiongate/releases";
const GITHUB_LATEST = "https://github.com/AzielEliab/decisiongate/releases/latest";
const HOST = "https://decisiongate-download-tracker.vibelock.workers.dev";
const GITHUB_REPO = "https://github.com/AzielEliab/decisiongate";
const INSTALL_LINE = "curl -fsSL https://decisiongate-download-tracker.vibelock.workers.dev/install.sh | bash";
const DOI = "https://doi.org/10.5281/zenodo.21435730";
const ZENODO = "https://zenodo.org/records/21435730";

function corsHeaders() {
  return {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type",
  };
}

function json(body, status = 200) {
  return new Response(JSON.stringify(body, null, 2), {
    status,
    headers: { "Content-Type": "application/json; charset=utf-8", ...corsHeaders() },
  });
}

function redirect(url) {
  return new Response(null, {
    status: 302,
    headers: { Location: url, ...corsHeaders() },
  });
}

function splitOwnerRepo(value, fallbackOwner, fallbackRepo) {
  if (typeof value === "string" && value.includes("/")) {
    const [o, r] = value.split("/").filter(Boolean);
    if (o && r) return { owner: o, repo: r };
  }
  return { owner: fallbackOwner, repo: fallbackRepo };
}

function parseDims(src) {
  const get = (k) => {
    if (src == null) return null;
    if (typeof src.get === "function") {
      const v = src.get(k);
      return v == null || v === "" ? null : v;
    }
    const v = src[k];
    return v == null || v === "" ? null : v;
  };

  let owner = get("owner") || DEFAULT_OWNER;
  let repo = get("repo") || DEFAULT_REPO;
  if (typeof repo === "string" && repo.includes("/")) {
    const split = splitOwnerRepo(repo, owner, DEFAULT_REPO);
    owner = split.owner;
    repo = split.repo;
  }

  const branch = get("branch") || DEFAULT_BRANCH;
  const tag = get("tag") || "latest";
  const asset = get("asset") || "";

  const forkRaw = get("fork");
  let fork = "0";
  if (forkRaw === 1 || forkRaw === true || forkRaw === "1" || forkRaw === "true") {
    fork = "1";
  } else if (typeof forkRaw === "string" && forkRaw.includes("/")) {
    const split = splitOwnerRepo(forkRaw, owner, repo);
    owner = split.owner;
    repo = split.repo;
    fork = "1";
  }

```

```

    } else if (forkRaw != null && forkRaw !== 0 && forkRaw !== false && forkRaw !== "0" && forkRaw !== "false") {
      fork = "1";
    }

    if (`${owner}/${repo}`.toLowerCase() !== `${DEFAULT_OWNER}/${DEFAULT_REPO}`.toLowerCase()) {
      fork = "1";
    }

    return { project: PROJECT, owner, repo, branch, fork, tag, asset };
  }

function kvKey(dims) {
  return `${dims.project}|${dims.owner}|${dims.repo}|${dims.branch}|${dims.fork}`;
}

function githubAssetUrl(owner, repo, tag, asset) {
  if (!asset) {
    if (owner === DEFAULT_OWNER && repo === DEFAULT_REPO) return GITHUB_RELEASES;
    return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases`;
  }
  if (!tag || tag === "latest") {
    return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases/latest/download/${encodeURIComponent(asset)}`;
  }
  return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases/download/${encodeURIComponent(tag)}/${encodeURIComponent(asset)}`;
}

function totalKey() {
  return PROJECT + "__total__";
}

async function increment(env, dims) {
  const key = kvKey(dims);
  const n = parseInt((await env.DOWNLOADS.get(key)) || "0", 10) + 1;
  await env.DOWNLOADS.put(key, String(n));
  const tot = parseInt((await env.DOWNLOADS.get(totalKey())) || "0", 10) + 1;
  await env.DOWNLOADS.put(totalKey(), String(tot));
  return tot;
}

async function listAllKeys(env) {
  const keys = [];
  let cursor;
  do {
    const page = await env.DOWNLOADS.list(cursor ? { cursor } : {});
    keys.push(...page.keys);
    cursor = page.list_complete ? undefined : page.cursor;
  } while (cursor);
  return keys;
}

async function collectStats(env) {
  const keys = await listAllKeys(env);
  let total = 0;
  const by_repo = {};
  const by_branch = {};
  const by_fork = { "0": 0, "1": 0 };
  const breakdown = [];

  for (const k of keys) {
    const name = k.name;
    if (name === viewsKey() || name === totalKey() || name === githubCacheKey()) continue;
    const n = parseInt((await env.DOWNLOADS.get(name)) || "0", 10);
    if (!Number.isFinite(n) || n <= 0) continue;
    const parts = name.split("|");
    if (parts.length < 5) continue;
    const [project, owner, repo, branch, fork] = parts;
    total += n;
    const repoId = `${owner}/${repo}`;
    by_repo[repoId] = (by_repo[repoId] || 0) + n;
    by_branch[branch] = (by_branch[branch] || 0) + n;
    const forkFlag = fork === "1" ? "1" : "0";
    by_fork[forkFlag] = (by_fork[forkFlag] || 0) + n;
    breakdown.push({ project, owner, repo, branch, fork: forkFlag, count: n });
  }

  const totalDirect = parseInt((await env.DOWNLOADS.get(totalKey())) || "0", 10);
  const views = parseInt((await env.DOWNLOADS.get(viewsKey())) || "0", 10) || 0;
  const github = await githubStats(env);
  const shown = Number.isFinite(totalDirect) && totalDirect > 0 ? totalDirect : total;
  return {
    project: PROJECT,
    total: shown,
    views,
    downloads: shown,
    by_repo,
    by_branch,
    by_fork,
    breakdown,
    github: {
      stars: github.stars || 0,
      forks: github.forks || 0,
      watchers: github.watchers || 0,
    }
  };
}

```

```

        release_download_count: github.release_download_count || 0,
    },
    note: "Forks identified by GitHub owner/repo. Key layout: project|owner|repo|branch|fork. Views are separate from
downloads. /v1 does not increment.",
    };
}

function viewsKey() {
    return PROJECT + "|__views__";
}

function githubCacheKey() {
    return PROJECT + "|__github__";
}

async function incrementViews(env) {
    const n = parseInt((await env.DOWNLOADS.get(viewsKey())) || "0", 10) + 1;
    await env.DOWNLOADS.put(viewsKey(), String(n));
    return n;
}

async function githubStats(env) {
    const cached = await env.DOWNLOADS.get(githubCacheKey());
    if (cached) {
        try {
            const obj = JSON.parse(cached);
            if (obj && obj.fetched_at && Date.now() - obj.fetched_at < 5 * 60 * 1000) {
                return obj;
            }
        } catch {
            /* ignore */
        }
    }
    const headers = { "User-Agent": "Mozilla/5.0 DecisionGATE-download-tracker", Accept: "application/vnd.github+json" };
    ;
    let stars = 0;
    let forks = 0;
    let watchers = 0;
    let release_download_count = 0;
    try {
        const repoRes = await fetch("https://api.github.com/repos/AzielEliab/decisiongate", { headers });
        if (repoRes.ok) {
            const repo = await repoRes.json();
            stars = Number(repo.stargazers_count) || 0;
            forks = Number(repo.forks_count) || 0;
            watchers = Number(repo.subscribers_count != null ? repo.subscribers_count : repo.watchers_count) || 0;
        }
        const relRes = await fetch("https://api.github.com/repos/AzielEliab/decisiongate/releases/latest", { headers });
        if (relRes.ok) {
            const rel = await relRes.json();
            const assets = Array.isArray(rel.assets) ? rel.assets : [];
            release_download_count = assets.reduce((s, a) => s + (Number(a.download_count) || 0), 0);
        }
    } catch {
        /* public API; empty is fine */
    }
    const out = { stars, forks, watchers, release_download_count, fetched_at: Date.now() };
    try {
        await env.DOWNLOADS.put(githubCacheKey(), JSON.stringify(out));
    } catch {
        /* ignore */
    }
    return out;
}

function installScript() {
    return `#!/usr/bin/env bash
# DecisionGATE one-click install. Counted download via this Worker.
set -euo pipefail
HOST="${HOST}"
ASSET="${DEFAULT_ASSET}"
WORKDIR="${DECISIONGATE_HOME:-$HOME/decisiongate}"
mkdir -p "$WORKDIR"
cd "$WORKDIR"
echo "Downloading counted tarball from ${HOST}/download (User-Agent Mozilla/5.0)?"
curl -fsSL -A 'Mozilla/5.0' "${HOST}/download?asset=${ASSET}" -o "${ASSET}"
tar -xzf "${ASSET}"
DIR="$(find . -maxdepth 1 -type d -name 'decisiongate-*' | head -n 1)"
if [ -n "${DIR}" ]; then
    cd "${DIR}"
fi
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -U pip
python -m pip install -
e .
echo
echo "Installed DecisionGATE."
echo "Run: decisiongate ui"
echo "Then open http://127.0.0.1:8791 (loopback only)"
echo "Author: Aziel Eliab."
`;
}

```

```

async function serveAsset(request, env, asset, { head = false } = {}) {
  if (!env.ASSETS) {
    return json({ error: "assets binding missing" }, 500);
  }
  const assetUrl = new URL("/" + asset, request.url);
  const assetRes = await env.ASSETS.fetch(new Request(assetUrl, { method: "GET" }));
  if (!assetRes.ok) {
    return json({ error: "asset not hosted", asset, status: assetRes.status }, 404);
  }
  const headers = new Headers();
  headers.set("Content-Type", "application/gzip");
  headers.set("Content-Disposition", "attachment; filename=" + asset.replaceAll("'", "") + ".gz");
  headers.set("Cache-Control", "private, no-store");
  const len = assetRes.headers.get("Content-Length");
  if (len) headers.set("Content-Length", len);
  for (const [k, v] of Object.entries(corsHeaders())) headers.set(k, v);
  if (head) {
    return new Response(null, { status: 200, headers });
  }
  return new Response(assetRes.body, { status: 200, headers });
}

async function indexHtml(env) {
  const stats = await collectStats(env);
  const views = Number(stats.views) || 0;
  const downloads = Number(stats.downloads != null ? stats.downloads : stats.total) || 0;
  const v = views.toLocaleString("en-US");
  const n = downloads.toLocaleString("en-US");
  const gh = stats.github || {};
  const breakdown = (stats.breakdown || []).map(
    (b) =>
      `<li><code>${b.owner}/${b.repo}</code> branch <code>${b.branch}</code> fork=${b.fork} ? ${b.count}</li>`,
  ).join("") || "<li>none yet</li>";
  return `<!doctype html>
<html lang="en">
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>DecisionGATE ? Aziel Eliab</title>
<meta name="description" content="Five-gate ethical pre-execution filter (PASS / REVISE / BLOCK) by Aziel Eliab; not a predictor or command runner.">
<meta name="author" content="Aziel Eliab">
<link rel="canonical" href="https://decisiongate-download-tracker.vibelock.workers.dev/">
<meta property="og:title" content="DecisionGATE ? Aziel Eliab">
<meta property="og:description" content="Five-gate ethical pre-execution filter (PASS / REVISE / BLOCK) by Aziel Eliab; not a predictor or command runner.">
<meta property="og:url" content="https://decisiongate-download-tracker.vibelock.workers.dev/">
<meta property="og:type" content="website">
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "SoftwareApplication",
  "name": "DecisionGATE",
  "author": {
    "@type": "Person",
    "name": "Aziel Eliab"
  },
  "codeRepository": "https://github.com/AzielEliab/decisiongate",
  "downloadUrl": "https://decisiongate-download-tracker.vibelock.workers.dev/download",
  "license": "https://www.apache.org/licenses/LICENSE-2.0",
  "url": "https://decisiongate-download-tracker.vibelock.workers.dev/",
  "description": "Five-gate ethical pre-execution filter (PASS / REVISE / BLOCK) by Aziel Eliab; not a predictor or command runner.",
  "identifier": "https://doi.org/10.5281/zenodo.21435730"
}
</script>
<!-- gitbaby-seo -->
<style>
:root { color-scheme: dark; }
body { font: 16px/1.45 system-ui, sans-serif; max-width: 42rem; margin: 3rem auto; padding: 0 1.25rem 4rem; background-color: #0e1014; color: #e8eaeef; }
h1 { font-size: 1.75rem; margin: 0 0 .35rem; }
.motto { color: #9aa3b2; margin: 0 0 1.5rem; }
.card { border: 1px solid #2a3140; border-radius: 12px; padding: 1.25rem 1.35rem; background-color: #151922; }
.nums { display: grid; grid-template-columns: 1fr 1fr; gap: .8rem; margin: 0 0 1rem; }
.count { font-size: 2.2rem; font-variant-numeric: tabular-nums; font-weight: 700; margin: 0; }
.count span { display: block; font-size: .95rem; font-weight: 500; color: #9aa3b2; }
.btns { display: grid; grid-template-columns: 1fr 1fr; gap: .75rem; margin: 0 0 .85rem; }
@media (max-width: 520px) { .btns { grid-template-columns: 1fr; } }
a.btn, button.btn { display: block; width: 100%; box-sizing: border-box; text-align: center; font: inherit; font-size: 1.2rem; font-weight: 750; padding: 1rem 1.1rem; border-radius: 10px; border: 0; cursor: pointer; text-decoration: none; }
a.btn.primary { background: #e8eaeef; color: #0e1014; }
button.btn.install { background: #c9a227; color: #14110a; }
button.btn.install.copied { background: #7dcf9a; color: #0e1014; }
.kid { font-size: 1.05rem; margin: 0 0 1rem; }
.meta { margin-top: 1.1rem; color: #9aa3b2; font-size: .92rem; }
.meta a { color: #c9d4ff; }
.iso { margin-top: .85rem; font-size: .85rem; color: #7d8696; }
.banner { border: 1px solid #5c4a1a; background: #241c0d; color: #f0d78c; padding: .85rem 1rem; border-radius: 8px; margin: 0 0 1.2rem; font-size: .92rem; }
pre { background: #0e1014; padding: .75rem .9rem; overflow: auto; border-radius: 8px; font-size: .82rem; }
code { font-size: .88rem; }
cite { margin-top: 1.4rem; padding-top: 1rem; border-top: 1px solid #2a3140; }

```



```

.cite h2 { font-size: 1.05rem; margin: 0 0 .4rem; }
.cite p { color: #c5ccd8; font-size: .95rem; }
.cite a { color: #c9d4ff; }
</style>
<body>
<h1>DecisionGATE</h1>
<p class="motto">Freedom without clarity is chaos. Clarity without force is wisdom. Author Aziel Eliab.</p>
<p class="banner">THIS IS: a five-gate ethical pre-execution filter (PASS / REVISE / BLOCK). THIS IS NOT: a predictor, a court, a truth score, advice, or a remote command runner. wrap is not hosted. Author Aziel Eliab.</p>
<div class="card">
<div class="nums">
<p class="count">${v}<span>Views</span></p>
<p class="count">${n}<span>Downloads</span></p>
</div>
<p class="kid"><strong>Three steps.</strong> 1. Tap Download. 2. After install, type <code>decisiongate ui</code>.
3. Type a plan and tap Run. Green is a check, not a command.</p>
<div class="btns">
<a class="btn primary dl" href="/download?asset=${DEFAULT_ASSET}">Download</a>
<button type="button" class="btn install" id="install-btn">One-click install</button>
</div>
<pre id="install-cmd">${INSTALL_LINE}</pre>
<p class="kid">Then open http://127.0.0.1:8791 (this computer only). Import file and Export file both exist. Verify speaks in plain words.</p>
<p class="meta">The download count ticks on the Download click. The Worker serves the gzip (HTTP 200). No 302 to GitHub. Forks using this same link are counted automatically. ${DEFAULT_ASSET} ? ${n} counted.</p>
<p class="iso">Isolated counter: Worker <code>decisiongate-download-tracker</code>, project <code>${PROJECT}</code>, KV <code>DECISIONGATE_DOWNLOADS</code>. Not mixed with any other product. /v1 does not increment downloads.</p>
<p class="meta">GitHub: stars ${gh.stars || 0} ? forks ${gh.forks || 0} ? watchers ${gh.watchers || 0} ? release assets ${gh.release_download_count || 0}</p>
<p class="meta">Paper: <a href="${DOI}">doi:10.5281/zenodo.21435730</a> ? <a href="${ZENODO}">Zenodo</a> ? Apache-2.0 ? Eliab, Aziel</p>
<p class="meta"><a href="/stats">JSON stats</a> ? <a href="/openapi.json">OpenAPI</a> ? <a href="/v1/skill">Skill</a> ? <a href="/ai">AI runtime</a> ? <a href="${GITHUB_REPO}">GitHub</a> ? <a href="${GITHUB_LATEST}">releases</a></p>
<script>
(function () {
var cmd = "curl -fsSL https://decisiongate-download-tracker.viblock.workers.dev/install.sh | bash";
var btn = document.getElementById("install-btn");
var pre = document.getElementById("install-cmd");
if (!btn) return;
btn.addEventListener("click", function () {
function done(ok) {
btn.textContent = ok ? "Copied! Paste in Terminal, then run decisiongate ui" : "Select the command, copy it, then run decisiongate ui";
btn.classList.add("copied");
}
if (navigator.clipboard && navigator.clipboard.writeText) {
navigator.clipboard.writeText(cmd).then(function () { done(true); }).catch(function () { done(false); });
} else {
done(false);
if (pre && window.getSelection()) {
var r = document.createRange();
r.selectNodeContents(pre);
var sel = window.getSelection();
sel.removeAllRanges();
sel.addRange(r);
}
}
});
})();
</script>
<details class="iso"><summary>Per repo / branch / fork</summary>
<ul>${breakdown}</ul>
</details>
</div>

<section class="cite" id="cite">
<h2>How to cite</h2>
<p>Aziel Eliab. DecisionGATE. https://github.com/AzielEliab/decisiongate. https://decisiongate-download-tracker.viblock.workers.dev. https://doi.org/10.5281/zenodo.21435730.</p>
<p><a href="https://aziel-runtime.viblock.workers.dev/">Catalog</a> ? <a href="https://github.com/AzielEliab/decisiongate">GitHub</a> ? <a href="https://decisiongate-download-tracker.viblock.workers.dev/download">Download</a> ? <a href="https://decisiongate-download-tracker.viblock.workers.dev/cite.json">cite.json</a></p>
</section>
<!-- /gitbaby-seo -->
</body>
</html>`
}

export default {
  async fetch(request, env) {
    const url = new URL(request.url);

    if (request.method === "OPTIONS") {
      return new Response(null, { status: 204, headers: corsHeaders() });
    }

    if ((url.pathname === "/install.sh" || url.pathname === "/install.sh/") && request.method === "GET") {
      return new Response(installScript(), {
        status: 200,
        headers: {
          "Content-Type": "text/x-shellscript; charset=utf-8",
          "Cache-Control": "private, no-store",
          ...corsHeaders(),
        }
      });
    }
  }
}

```

```

    },
  });
}

const runtime = await handleRuntimeApi(request, url);
if (runtime) return runtime;

if (url.pathname === "/" && request.method === "GET") {
  await incrementViews(env);
  return new Response(await indexHtml(env), {
    headers: { "Content-Type": "text/html; charset=utf-8", "Cache-Control": "private, no-store", ...corsHeaders()
},
  });
}

if (url.pathname === "/count" && request.method === "GET") {
  const stats = await collectStats(env);
  return json({ project: PROJECT, total: stats.total || 0 });
}

if (url.pathname === "/stats" && request.method === "GET") {
  return json(await collectStats(env));
}

if (url.pathname === "/event" && request.method === "POST") {
  let body;
  try {
    body = await request.json();
  } catch {
    return json({ error: "JSON body required" }, 400);
  }
  const dims = parseDims(body || {});
  const count = await increment(env, dims);
  return json({
    ok: true,
    key: kvKey(dims),
    count,
    owner: dims.owner,
    repo: dims.repo,
    branch: dims.branch,
    fork: dims.fork,
    asset: dims.asset || null,
  });
}

if (url.pathname === "/go" && (request.method === "GET" || request.method === "HEAD")) {
  const dims = parseDims(url.searchParams);
  const asset = dims.asset || DEFAULT_ASSET;
  dims.asset = asset;
  if (request.method === "GET") await increment(env, dims);
  return serveAsset(request, env, asset, { head: request.method === "HEAD" });
}

if ((url.pathname === "/download" || url.pathname.startsWith("/download/")) && (request.method === "GET" || request.method === "HEAD")) {
  const dims = parseDims(url.searchParams);
  if (!dims.asset && url.pathname.startsWith("/download/")) {
    dims.asset = decodeURIComponent(url.pathname.slice("/download/".length));
  }
  const asset = dims.asset || DEFAULT_ASSET;
  dims.asset = asset;
  if (request.method === "GET") await increment(env, dims);
  return serveAsset(request, env, asset, { head: request.method === "HEAD" });
}

// gitbaby-seo-routes
if ((url.pathname === "/robots.txt" || url.pathname === "/robots.txt/") && request.method === "GET") {
  const body = "User-agent: *\nAllow: /\nSitemap: " + HOST + "/sitemap.xml\n";
  return new Response(body, {
    status: 200,
    headers: { "Content-Type": "text/plain; charset=utf-8", ...corsHeaders() },
  });
}

if ((url.pathname === "/sitemap.xml" || url.pathname === "/sitemap.xml/") && request.method === "GET") {
  const locs = [HOST + "/", HOST + "/download", HOST + "/install.sh", HOST + "/v1/skill", HOST + "/openapi.json",
GITHUB_REPO];
  const xml = '<?xml version="1.0" encoding="UTF-8"?>\n<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9"
>\n'
    + locs.map((u) => "  <url><loc>" + u + "</loc></url>").join("\n")
    + "\n</urlset>\n";
  return new Response(xml, {
    status: 200,
    headers: { "Content-Type": "application/xml; charset=utf-8", ...corsHeaders() },
  });
}

if ((url.pathname === "/cite.json" || url.pathname === "/cite.json/") && request.method === "GET") {
  return json({ "author": "Aziel Eliab", "title": "DecisionGATE", "github": "https://github.com/AzielEliab/decision
gate", "download": "https://decisiongate-download-tracker.viblock.workers.dev/download", "doi": "10.5281/zenodo.21435
730", "license": "Apache-2.0", "catalog": "https://aziel-runtime.viblock.workers.dev/"});
}

// /gitbaby-seo-routes
return json({ error: "not found" }, 404);

```

```
    },  
};
```

decisiongate/workers/download-tracker/src/runtime.js

```

/**
 * DecisionGATE hosted runtime (port of decisiongate/gates.py + engine).
 * Sequential PASS/REVISE/BLOCK. wrap / remote command execution is NOT hosted.
 * /v1 calls never touch DOWNLOADS KV.
 */
const PRODUCT = "decisiongate";
const EXAMPLE_PAYLOAD = {
  "statement": "Release the catalog Worker this week.",
  "evidence": [
    "OpenAPI 3.1 combined spec."
  ],
  "impact_pos": [
    "One URL for GPT Actions."
  ],
  "impact_neg": [
    "A vague draft takes longer."
  ],
  "values": [
    "Clarity without force"
  ],
  "accountable": "Aziel Eliab"
};

const VERSION = "0.1.0";
const MOTTO = "Freedom without clarity is chaos. Clarity without force is wisdom.";
const HOST = "https://decisiongate-download-tracker.vibelock.workers.dev";
const SKILL = "---\\nname: DecisionGATE\\ndescription: Use when calling DecisionGATE hosted /v1 or installing the local package. Author Aziel Eliab.\\n---\\n\\n# DecisionGATE\\n\\nA five-gate check before you act. Not a predictor. Not advice. Not a command. Author: **Aziel Eliab**.\\n\\n**THIS IS:** a lightweight ethical pre-execution filter (PASS / REVISE / BLOCK).\\n\\n**THIS IS NOT:** a predictor, a court, a truth score, advice, or a hosted command runner. Hosted `/v1` does not increment downloads or views. wrap is not hosted.\\n\\nAlways send `User-Agent: Mozilla/5.0`. Cloudflare Workers may 403 an empty agent.\\n\\n## Call these URLs\\n\\n- Worker OpenAPI: https://decisiongate-download-tracker.vibelock.workers.dev/openapi.json\\n- Catalog OpenAPI: https://aziel-runtime.vibelock.workers.dev/openapi.json\\n- MCP: `POST https://aziel-runtime.vibelock.workers.dev/mcp`\\n- Live skill (this markdown): `GET https://decisiongate-download-tracker.vibelock.workers.dev/v1/skill`\\n\\nops (do **not** increment downloads or views):\\n\\n- `GET /v1/health` \\u00e2\\u0080\\u0094 liv eness\\n- `GET /v1/skill` \\u00e2\\u0080\\u0094 this file\\n- Product POSTs listed in OpenAPI\\n\\nGrok: import OpenAPI as a custom tool. ChatGPT: GPT Actions. Venice: HTTP tools.\\n\\n## Example\\n\\n`bash`ncurl -s -A 'Mozilla/5.0' https://decisiongate-download-tracker.vibelock.workers.dev/v1/health`ncurl -s -A 'Mozilla/5.0' https://decisiongate-download-tracker.vibelock.workers.dev/v1/skill`\\n\\n## Local (after one-click install)\\n\\n`bash`ncurl -fsSL https://decisiongate-download-tracker.vibelock.workers.dev/install.sh | bash\\ndecisiongate ui\\ndecisiongate doctor\\n`\\n\\nThen open http://127.0.0.1:8791 (this computer only). Import file and Export file both exist. Verify speaks in plain words.\\n\\nCount ed download (gzip HTTP 200, no 302): https://decisiongate-download-tracker.vibelock.workers.dev/download?asset=decisiongate-0.1.0.tar.gz\\nGitHub: https://github.com/AzielEliab/decisiongate\\n\\nPaper: DOI https://doi.org/10.5281/zenodo.21435730 \\u00c2\\u00b7 https://zenodo.org/records/21435730 \\u00c2\\u00b7 Apache-2.0. Forks welcome.\\n\\n## Catalog + local UI\\n\\nAuthor: **Aziel Eliab**. Honest scope: Five sequential gates on a proposal. Freedom without clarity is chaos.\\n\\n- Catalog product: https://aziel-runtime.vibelock.workers.dev/p/decisiongate/\\n- Catalog OpenAPI: https://aziel-runtime.vibelock.workers.dev/openapi.json\\n- Catalog MCP: `POST https://aziel-runtime.vibelock.workers.dev/mcp`\\n- This Worker skill: `GET https://decisiongate-download-tracker.vibelock.workers.dev/v1/skill`\\n- This Worker OpenAPI: https://decisiongate-download-tracker.vibelock.workers.dev/openapi.json\\n- Sample payload: `GET https://decisiongate-download-tracker.vibelock.workers.dev/v1/example`\\n\\nLocal UI: **Import JSON file** (`type=file`) and **Export JSON**. Then `decisiongate doctor`.\\n\\nGrok: import catalog or Worker OpenAPI as a custom tool. ChatGPT: GPT Actions. Venice: HTTP tool s.\\n";

const PASS = "PASS";
const REVISE = "REVISE";
const BLOCK = "BLOCK";
const HEDGES = new Set(["maybe", "somehow", "stuff", "things"]);
const COMMON_VERBS = new Set([
  "is", "are", "was", "were", "be", "been", "being", "am", "have", "has", "had", "do", "does", "did",
  "will", "would", "shall", "should", "can", "could", "must", "need", "needs", "make", "makes", "made",
  "take", "takes", "took", "give", "gives", "gave", "go", "goes", "went", "come", "keep", "put", "use",
  "uses", "used", "set", "get", "let", "allow", "allows", "publish", "release", "releases", "deploy",
  "ship", "open", "close", "create", "created", "delete", "write", "read", "run", "execute", "adopt",
  "reject", "approve", "block", "collect", "store", "share", "send", "build", "launch", "hire", "spend",
  "buy", "sell", "migrate", "replace", "update", "install", "announce", "commit", "sign", "fund",
  "grant", "revoke", "host", "serve", "bind", "filter", "record", "name", "assign", "document",
  "provide", "provides", "include", "includes", "add", "remove", "stop", "start", "move", "change",
  "apply", "submit", "accept", "refuse", "pay", "offer", "request", "require", "requires",
  "implement", "implements",
]);
const NEGATION_MARKERS = [
  "do not ", "don't ", "does not ", "doesn't ", "must not ", "cannot ", "can't ",
  "never ", "no ", "not ", "without ",
];
const GATE_ORDER = ["Definition", "Evidence", "Impact", "Integrity", "Responsibility"];
const WORD_RE = /[A-Za-z0-9][A-Za-z0-9._+~]*/g;

export function corsHeaders() {
  return {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type",
  };
};

function json(body, status = 200) {
  return new Response(JSON.stringify(body, null, 2), {
    status,
    headers: { "Content-Type": "application/json; charset=utf-8", ...corsHeaders() },
  });
};

```

```

function asStr(v) {
  return v == null ? "" : String(v);
}

function asList(value) {
  if (value == null) return [];
  if (typeof value === "string") {
    let parts = value.replace(/\r\n/g, "\n").split("\n").map((p) => p.trim());
    if (parts.length === 1 && parts[0].includes(";")) {
      parts = parts[0].split(";").map((p) => p.trim());
    }
    return parts.filter(Boolean);
  }
  if (Array.isArray(value)) {
    return value.map((item) => asStr(item).trim()).filter(Boolean);
  }
  const text = asStr(value).trim();
  return text ? [text] : [];
}

function proposalFromBody(raw) {
  const data = raw && typeof raw === "object" ? raw : {};
  return {
    statement: asStr(data.statement).trim(),
    evidence: asList(data.evidence),
    impacts_positive: asList(data.impact_pos ?? data.impact_pos),
    impacts_negative: asList(data.impact_neg ?? data.impact_neg),
    values: asList(data.values),
    commitments: asList(data.commitments),
    constraints: asList(data.constraints),
    accountable_person: asStr(data.accountable_person ?? data.accountable).trim(),
  };
}

function tokenize(text) {
  if (!text) return [];
  const m = text.toLowerCase().match(WORD_RE);
  return m || [];
}

function looksLikeVerb(token) {
  if (COMMON_VERBS.has(token)) return true;
  if (token.length > 4 && (token.endsWith("ing") || token.endsWith("ize") || token.endsWith("ise") || token.endsWith("ify"))) return true;
  if (token.length > 4 && token.endsWith("ed")) return true;
  return false;
}

function hasVerbAndObject(tokens) {
  const content = tokens.filter((t) => !HEDGES.has(t));
  if (content.length < 2) return false;
  return content.some(looksLikeVerb);
}

function gateResult(name, state, feedback, extra = {}) {
  const out = { name, state, feedback };
  if (extra.overridden) {
    out.overridden = true;
    if (extra.automatic_state) out.automatic_state = extra.automatic_state;
    if (extra.override_note) out.override_note = extra.override_note;
  }
  return out;
}

function gateDefinition(p) {
  const statement = p.statement.trim();
  if (!statement) {
    return gateResult("Definition", BLOCK, "Statement is empty. A proposal with no concrete statement cannot pass Definition. Write an unambiguous action with a verb and an object, at least 12 words.");
  }
  const tokens = tokenize(statement);
  if (tokens.length < 12) {
    return gateResult("Definition", REVISE, `Statement has ${tokens.length} word(s); Definition requires at least 12. Expand into a concrete, unambiguous proposal (who does what, to what, under what bound).`);
  }
  if (!hasVerbAndObject(tokens)) {
    return gateResult("Definition", REVISE, "Statement is hedge-only or lacks a verb+object after removing maybe/somehow/stuff/things. Name a specific action and its object.");
  }
  return gateResult("Definition", PASS, "Statement is concrete enough to scrutinize (length, verb+object).");
}

function gateEvidence(p) {
  const items = p.evidence.filter((e) => e.trim());
  if (!items.length) {
    return gateResult("Evidence", REVISE, "Evidence list is empty. Identify at least one fact, datum, or observation that grounds the statement. Ungrounded assertions do not pass Evidence.");
  }
  return gateResult("Evidence", PASS, `${items.length} evidence item(s) identified.`);
}

function gateImpact(p) {
  const pos = p.impact_pos.filter((i) => i.trim());
  const neg = p.impact_neg.filter((i) => i.trim());
  const missing = [];

```

```

    if (!pos.length) missing.push("positive");
    if (!neg.length) missing.push("negative");
    if (missing.length) {
      return gateResult("Impact", REVISE, `Impact list(s) empty: ${missing.join(" and ")}. Name who or what is affected on both the positive and the negative side. Hidden impacts do not pass Impact.`);
    }
    return gateResult("Impact", PASS, `${pos.length} positive and ${neg.length} negative impact(s) named.`);
  }

function constraintPayload(constraint) {
  let text = constraint.toLowerCase().split(/\s+/).join(" ");
  if (!text) return null;
  let found = false;
  let remainder = `${text}`;
  for (const marker of NEGATION_MARKERS) {
    const padded = marker.startsWith(" ") ? marker : ` ${marker}`;
    if (remainder.includes(padded) || remainder.trimStart().startsWith(marker)) {
      found = true;
      remainder = remainder.split(padded).join(" ");
      const stripped = remainder.trimStart();
      if (stripped.startsWith(marker)) {
        remainder = " " + stripped.slice(marker.length);
      }
    }
  }
  const payload = remainder.split(/\s+/).filter(Boolean).join(" ");
  if (found && payload) return payload;
  return null;
}

function statementContradictsConstraint(statement, constraint) {
  const payload = constraintPayload(constraint);
  if (!payload) return false;
  const hay = statement.toLowerCase().split(/\s+/).join(" ");
  return hay.includes(payload);
}

function gateIntegrity(p) {
  const values = p.values.filter((v) => v.trim());
  if (!values.length) {
    return gateResult("Integrity", REVISE, "Values list is empty. Integrity requires stated values so the proposal can be checked against them.");
  }
  const hits = [];
  for (const constraint of p.constraints) {
    if (statementContradictsConstraint(p.statement, constraint)) hits.push(constraint);
  }
  if (hits.length) {
    const shown = hits[0];
    return gateResult("Integrity", BLOCK, `Statement contradicts a provided constraint (${JSON.stringify(shown)}). A contradiction of this kind cannot pass Integrity without changing the proposal's nature.`);
  }
  return gateResult("Integrity", PASS, `${values.length} value(s) stated; no constraint contradiction detected.`);
}

function gateResponsibility(p) {
  const owner = p.accountable_person.trim();
  if (!owner) {
    return gateResult("Responsibility", BLOCK, "Accountable person is blank. Diffuse or absent ownership cannot pass Responsibility. Name one accountable owner.");
  }
  return gateResult("Responsibility", PASS, `Accountable owner named: ${owner}.`);
}

const GATE_FUNCS = [gateDefinition, gateEvidence, gateImpact, gateIntegrity, gateResponsibility];

function coerceOverrides(overrides) {
  const out = {};
  if (!overrides || typeof overrides !== "object") return out;
  const known = Object.fromEntries(GATE_ORDER.map((n) => [n.toLowerCase(), n]));
  for (const [key, raw] of Object.entries(overrides)) {
    const name = known[String(key).trim().toLowerCase()];
    if (!name) continue;
    let note = "";
    let state = REVISE;
    if (typeof raw === "string") note = raw.trim();
    else if (raw && typeof raw === "object") {
      const stateRaw = String(raw.state || REVISE).trim().toUpperCase();
      if (stateRaw === REVISE) state = stateRaw;
      note = String(raw.note || raw.feedback || "").trim();
    }
    out[name] = { state, note };
  }
  return out;
}

function runGates(proposal, overrides) {
  const prop = proposalFromBody(proposal);
  const forced = coerceOverrides(overrides);
  const lineage = [];
  let final_state = PASS;
  let blocked_at = null;
  for (const fn of GATE_FUNCS) {
    let result = fn(prop);
  }

```

```

    if (forced[result.name]) {
      const { state, note } = forced[result.name];
      const auto = result.state;
      let feedback = result.feedback;
      feedback = note
        ? `${feedback} Human override to ${state}: ${note}`
        : `${feedback} Human override to ${state} (no note supplied).`;
      result = gateResult(result.name, state, feedback, {
        overridden: true,
        automatic_state: auto,
        override_note: note || undefined,
      });
    }
    lineage.push(result);
    if (result.state !== PASS) {
      final_state = result.state;
      if (result.state === BLOCK) blocked_at = result.name;
      break;
    }
  }
  return {
    lineage,
    final_state,
    blocked_at,
    proposal: prop,
    motto: MOTTO,
    product: PRODUCT,
    version: VERSION,
  };
}

function health() {
  return { ok: true, author: "Aziel Eliab", product: PRODUCT, version: VERSION };
}

function openapiSpec() {
  const proposalProps = {
    statement: { type: "string", description: "Concrete proposal statement (CLI --statement).", },
    evidence: { oneOf: [{ type: "string" }, { type: "array", items: { type: "string" } }], description: "Facts/data (CLI --evidence).", },
    impacts_positive: { oneOf: [{ type: "string" }, { type: "array", items: { type: "string" } }], description: "CLI -impact-pos.", },
    impact_pos: { oneOf: [{ type: "string" }, { type: "array", items: { type: "string" } }], },
    impacts_negative: { oneOf: [{ type: "string" }, { type: "array", items: { type: "string" } }], description: "CLI -impact-neg.", },
    impact_neg: { oneOf: [{ type: "string" }, { type: "array", items: { type: "string" } }], },
    values: { oneOf: [{ type: "string" }, { type: "array", items: { type: "string" } }], },
    commitments: { oneOf: [{ type: "string" }, { type: "array", items: { type: "string" } }], },
    constraints: { oneOf: [{ type: "string" }, { type: "array", items: { type: "string" } }], },
    accountable_person: { type: "string", description: "CLI --accountable.", },
    accountable: { type: "string", },
    overrides: { type: "object", additionalProperties: true, description: "Optional human override to REVISE with a note." },
  };
  const report = {
    type: "object",
    properties: {
      product: { type: "string" },
      version: { type: "string" },
      motto: { type: "string" },
      final_state: { type: "string", enum: ["PASS", "REVISE", "BLOCK"] },
      blocked_at: { type: ["string", "null"] },
      lineage: {
        type: "array",
        items: {
          type: "object",
          properties: {
            name: { type: "string" },
            state: { type: "string", enum: ["PASS", "REVISE", "BLOCK"] },
            feedback: { type: "string" },
          },
        },
      },
    },
    proposal: { type: "object" },
  };
  return {
    openapi: "3.1.0",
    info: {
      title: "DecisionGATE runtime",
      version: VERSION,
      description: "Ethical pre-execution filter. Sequential PASS/REVISE/BLOCK. wrap (shell exec) is not hosted. " + M
    },
    servers: [{ url: HOST }],
    paths: {
      "/v1/example": { get: { operationId: "decisiongateExample", summary: "Sample JSON payload. Does not increment downloads.", responses: { "200": { description: "OK" } } } },
      "/v1/skill": {
        get: {
          operationId: "decisiongate_skill",
          summary: "Return skill markdown. Does not increment download KV.",
          responses: { "200": { description: "markdown" } },
        },
      },
    },
  };
}

```

```

    },
    "/v1/health": {
      get: {
        operationId: "health",
        summary: "Liveness",
        responses: { "200": { description: "ok", content: { "application/json": { schema: { type: "object" } } } } }
      },
    },
    "/v1/check": {
      post: {
        operationId: "check",
        summary: "Run the five gates (CLI check).",
        requestBody: { required: true, content: { "application/json": { schema: { type: "object", properties: props
alProps } } } } },
        responses: { "200": { description: "Gate report", content: { "application/json": { schema: report } } } } },
      },
    },
    "/v1/evaluate": {
      post: {
        operationId: "evaluate",
        summary: "Same sequential gates as check; returns PASS/REVISE/BLOCK + motto.",
        requestBody: { required: true, content: { "application/json": { schema: { type: "object", properties: props
alProps } } } } },
        responses: { "200": { description: "Gate report", content: { "application/json": { schema: report } } } } },
      },
    },
  },
};
}

function aiHtml() {
  return `<!doctype html>
<html lang="en">
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>DecisionGATE ? use with Grok, ChatGPT, Venice</title>
<style>
  :root { color-scheme: dark; }
  body { font: 16px/1.45 system-ui, sans-serif; max-width: 42rem; margin: 3rem auto; padding: 0 1.25rem; background: #
0e1014; color: #e8eaeef; }
  code, pre { background: #151922; padding: .15rem .4rem; border-radius: 4px; }
  a { color: #c9d4ff; }
  .motto { color: #9aa3b2; font-style: italic; }
</style>
<body>
<h1>DecisionGATE live API</h1>
<p class="motto">${MOTTO}</p>
<p>This Worker is a sequential five-gate filter. It does <strong>not</strong> run commands. There is no hosted <code
>wrap</code>.</p>
<h2>ChatGPT (GPT Actions)</h2>
<p>Paste this OpenAPI URL into GPT Actions:</p>
<p><code>${HOST}/openapi.json</code></p>
<h2>Grok / xAI</h2>
<p>Add a custom tool pointing at <code>POST ${HOST}/v1/check</code> and <code>POST ${HOST}/v1/evaluate</code>. Import
t the OpenAPI document if the client accepts it.</p>
<h2>Venice</h2>
<p>Create a custom HTTP tool from the same OpenAPI URL.</p>
<h2>MCP catalog</h2>
<p>The shared catalog (ships separately) is <code>https://aziel-runtime.vibelock.workers.dev/mcp</code>.</p>
<p><a href="/openapi.json">openapi.json</a> ? <a href="/v1/health">health</a> ? <a href="/">downloads</a></p>
</body>
</html>`;
}

async function readJson(request) {
  try {
    return await request.json();
  } catch {
    return null;
  }
}

export async function handleRuntimeApi(request, url) {
  const path = url.pathname;
  const isApi = path === "/v1" || path.startsWith("/v1/") || path === "/openapi.json" || path === "/ai";
  if (!isApi) return null;

  if (path === "/v1/health" && request.method === "GET") return json(health());
  if (path === "/v1/skill" && request.method === "GET") {
    return new Response(SKILL, {
      status: 200,
      headers: { "Content-Type": "text/markdown; charset=utf-8", "Cache-Control": "private, no-store", ...corsHeaders(
) },
    });
  }
  if ((path === "/v1/example" || path === "/v1/example/") && (request.method === "GET" || request.method === "HEAD"))
  {
    return json({
      ok: true,
      product: PRODUCT,
      author: "Azriel Eliab",
      example: EXAMPLE_PAYLOAD,
      note: "Sample payload only. Does not increment downloads.",
    });
  }
}

```



```
    });  
  }  
  
  if (path === "/openapi.json" && request.method === "GET") return json(openapiSpec());  
  if (path === "/ai" && request.method === "GET") {  
    return new Response(aiHtml(), {  
      headers: { "Content-Type": "text/html; charset=utf-8", ...corsHeaders() },  
    });  
  }  
  if ((path === "/v1/check" || path === "/v1/evaluate") && request.method === "POST") {  
    const body = await readJson(request);  
    if (body == null || typeof body !== "object") return json({ error: "JSON body required" }, 400);  
    const report = runGates(body, body.overrides);  
    return json(report);  
  }  
  if (path === "/v1/wrap" || path.startsWith("/v1/wrap")) {  
    return json({ error: "wrap is not hosted; this API never executes commands" }, 404);  
  }  
  return json({ error: "not found" }, 404);  
}
```

decisiongate/workers/download-tracker/wrangler.toml

```
name = "decisiongate-download-tracker"
main = "src/index.js"
compatibility_date = "2026-07-01"
workers_dev = true
account_id = "ac575a9b822bea2bed97d0ab73aed238"

[[kv_namespaces]]
binding = "DOWNLOADS"
id = "147130a914ed4fac8632570871d2c266"

[assets]
directory = "./public"
binding = "ASSETS"
run_worker_first = ["/", "/download", "/download/*", "/stats", "/event", "/go", "/count", "/install.sh", "/v1", "/v1/*", "/openapi.json", "/ai", "/ai/*", "/mcp", "/mcp/", "/robots.txt", "/sitemap.xml", "/cite.json"]
```

forgereceipts/engines/codelock/__init__.py

```
"""CodeLock: gate-tethered cognitive rendering of source text.

July 2026 whitepaper implementation by Aziel Eliab.

Plain text is always canonical. Rendered views never mutate source.
This tool alters perception, not meaning. It is not encryption.

Forks are welcome and always allowed.
"""

from __future__ import annotations

from codelock.gate import ACK_PHRASE, AcknowledgmentError, Gate, GateClosedError
from codelock.render import styles_for
from codelock.session import CodeLockSession
from codelock.tokenize import tokenize, tokenize_kinds

__version__ = "0.1.0"
__author__ = "Azriel Eliab"
__all__ = [
    "ACK_PHRASE",
    "AcknowledgmentError",
    "CodeLockSession",
    "Gate",
    "GateClosedError",
    "styles_for",
    "tokenize",
    "tokenize_kinds",
    "__version__",
]
```

forgereceipts/engines/codelock/__main__.py

```
"""Allow ``python -m codelock`` to invoke the CLI."""

from codelock.cli import main

if __name__ == "__main__":
    raise SystemExit(main())
```

forgereceipts/engines/codelock/cli.py

```

"""Command-line interface for CodeLock.

codelock gate-status
codelock open-gate --ack "This tool alters perception, not meaning."
codelock render --in FILE --mode normalize|codelock --out FILE.html [--seed N] [--hue/--no-hue] [--ack "..."]
codelock export --in FILE --kind normal|codelock --out FILE [--seed N] [--ack "..."]
codelock version

The gate is per-invocation. Pass ``--ack`` or set env ``CODELOCK_ACK`` to
the exact acknowledgment phrase. Default is Closed. Normalize and
export-normal never need an ack.
"""

from __future__ import annotations

import argparse
import os
import sys
from pathlib import Path
from typing import Sequence

from codelock import __version__
from codelock.gate import ACK_PHRASE, AcknowledgmentError, Gate, GateClosedError
from codelock.session import CodeLockSession


def _build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="codelock",
        description=(
            "CodeLock ? gate-tethered cognitive rendering of source text "
            "(Aziel Eliab, July 2026). This tool alters perception, not "
            "meaning. It is not encryption."
        ),
    ),
    sub = parser.add_subparsers(dest="cmd", required=True)

    sub.add_parser("gate-status", help="Print gate: open|closed for this invocation.")

    p_open = sub.add_parser(
        "open-gate",
        help="Validate the acknowledgment phrase (per-invocation; not persisted).",
    )
    p_open.add_argument(
        "--ack",
        required=True,
        help=f"Exact phrase required: {ACK_PHRASE!r}",
    )

    p_render = sub.add_parser("render", help="Write Normalize or CodeLock HTML.")
    p_render.add_argument("--in", dest="inp", required=True, help="Input source file.")
    p_render.add_argument(
        "--mode",
        required=True,
        choices=("normalize", "codelock"),
        help="normalize is always available; codelock requires an open gate.",
    )
    p_render.add_argument("--out", dest="out", required=True, help="Output HTML path.")
    p_render.add_argument("--seed", default="0", help="Deterministic seed (default 0).")
    p_render.add_argument(
        "--hue",
        action=argparse.BooleanOptionalAction,
        default=True,
        help="Hue spectrum on tokens (default: on). --no-hue disables color.",
    )
    p_render.add_argument(
        "--ack",
        default=None,
        help=f"Exact phrase to open the gate: {ACK_PHRASE!r}",
    )

    p_export = sub.add_parser(
        "export",
        help="Export Normal (.txt, canonical) or CodeLock (.html, non-canonical).",
    )
    p_export.add_argument("--in", dest="inp", required=True, help="Input source file.")
    p_export.add_argument(
        "--kind",
        required=True,
        choices=("normal", "codelock"),
        help="normal is always available; codelock requires an open gate.",
    )
    p_export.add_argument("--out", dest="out", required=True, help="Output path.")
    p_export.add_argument("--seed", default="0", help="Deterministic seed (default 0).")
    p_export.add_argument(
        "--hue",
        action=argparse.BooleanOptionalAction,
        default=True,
        help="Hue spectrum (CodeLock kind only).",
    )
    p_export.add_argument(
        "--ack",

```

```

        default=None,
        help=f"Exact phrase to open the gate: {ACK_PHRASE!r}",
    )

    sub.add_parser("version", help="Print the CodeLock version and exit.")
    return parser

def _ack_from_env() -> str | None:
    val = os.environ.get("CODELOCK_ACK")
    if val is None or val == "":
        return None
    return val

def _open_from_invocation(ack: str | None) -> Gate:
    """Open the gate if --ack or CODELOCK_ACK matches the exact phrase.

    A provided-but-wrong acknowledgment is an error (the user tried to
    open the gate and failed), not a silent Closed state.
    """
    gate = Gate()
    candidate = ack if ack is not None else _ack_from_env()
    if candidate is None:
        return gate
    gate.open(candidate)
    return gate

def _read_source(path: str) -> str:
    try:
        return Path(path).read_text(encoding="utf-8")
    except FileNotFoundError as exc:
        raise SystemExit(f"error: {exc}") from exc
    except OSError as exc:
        raise SystemExit(f"error: failed to read {path}: {exc}") from exc

def _session(source: str, args: argparse.Namespace) -> CodeLockSession:
    seed = getattr(args, "seed", "0")
    hue = bool(getattr(args, "hue", True))
    ack = getattr(args, "ack", None)
    gate = _open_from_invocation(ack)
    return CodeLockSession(source, seed=seed, hue=hue, gate=gate)

def main(argv: Sequence[str] | None = None) -> int:
    parser = _build_parser()
    args = parser.parse_args(argv)

    if args.cmd == "version":
        sys.stdout.write(f"codelock {__version__}\n")
        return 0

    if args.cmd == "gate-status":
        try:
            gate = _open_from_invocation(_ack_from_env())
        except AcknowledgmentError as exc:
            sys.stderr.write(f"error: {exc}\n")
            sys.stdout.write("gate: closed\n")
            return 2
        state = "open" if gate.gate_open else "closed"
        sys.stdout.write(f"gate: {state}\n")
        return 0

    if args.cmd == "open-gate":
        try:
            gate = Gate()
            gate.open(args.ack)
        except AcknowledgmentError as exc:
            sys.stderr.write(f"error: {exc}\n")
            sys.stdout.write("gate: closed\n")
            return 2
        sys.stdout.write("gate: open\n")
        return 0

    try:
        if args.cmd == "render":
            source = _read_source(args.inp)
            session = _session(source, args)
            if args.mode == "normalize":
                html = session.normalize_html()
            else:
                html = session.codelock_html()
            dest = Path(args.out)
            dest.parent.mkdir(parents=True, exist_ok=True)
            dest.write_text(html, encoding="utf-8", newline="\n")
            sys.stdout.write(f"mode={args.mode} out={dest}\n")
            return 0

        if args.cmd == "export":
            source = _read_source(args.inp)
            session = _session(source, args)
            dest = Path(args.out)
            if args.kind == "normal":

```

```
        session.export_normal(dest)
    else:
        session.export_codelock(dest)
    sys.stdout.write(f"kind={args.kind} out={dest}\n")
    return 0
except AcknowledgmentError as exc:
    sys.stderr.write(f"error: {exc}\n")
    return 2
except GateClosedError as exc:
    sys.stderr.write(f"error: {exc}\n")
    return 2

parser.error(f"unknown command {args.cmd!r}")
return 2

if __name__ == "__main__":
    raise SystemExit(main())
```

forgereceipts/engines/codelock/gate.py

```

"""Gate tethering for CodeLock Mode.

Cognitive-altering modes require explicit acknowledgment of:

    This tool alters perception, not meaning.

Normalize mode is always available, whether the gate is Open or Closed.
"""

from __future__ import annotations

ACK_PHRASE = "This tool alters perception, not meaning."

class GateClosedError(RuntimeError):
    """Raised when CodeLock Mode is requested while the gate is Closed."""

    def __init__(self, message: str | None = None) -> None:
        super().__init__(
            message
            or (
                "CodeLock Mode is disabled while the gate is Closed. "
                "Normalize remains available. Open the gate by acknowledging: "
                f"{ACK_PHRASE!r}"
            )
        )

class AcknowledgmentError(ValueError):
    """Raised when open() is given a phrase other than ACK_PHRASE."""

    def __init__(self, got: str) -> None:
        super().__init__(
            "Opening the gate requires acknowledging the exact phrase: "
            f"{ACK_PHRASE!r} (got {got!r})"
        )

class Gate:
    """Visible, enforceable gate. Closed by default.

    The only way to open the gate is ``open(acknowledgment)`` with the
    exact phrase in ``ACK_PHRASE``. The ``gate_open`` flag is readable;
    it cannot be flipped to True by assignment.
    """

    def __init__(self, *, open: bool = False) -> None:
        # ``open=True`` is refused: the ack phrase is the only key.
        if open:
            raise AcknowledgmentError(
                "Gate(open=True) is not allowed; call Gate.open(ACK_PHRASE)"
            )
        self._open = False

    @property
    def gate_open(self) -> bool:
        return self._open

    @property
    def is_open(self) -> bool:
        return self._open

    def open(self, acknowledgment: str) -> None:
        """Open the gate. ``acknowledgment`` must match ACK_PHRASE exactly
        (leading/trailing whitespace is ignored)."""
        got = acknowledgment if isinstance(acknowledgment, str) else str(acknowledgment)
        if got.strip() != ACK_PHRASE:
            raise AcknowledgmentError(got)
        self._open = True

    def close(self) -> None:
        self._open = False

    def require_open(self) -> None:
        if not self._open:
            raise GateClosedError()

    def __repr__(self) -> str:
        state = "Open" if self._open else "Closed"
        return f"Gate({state})"

```


forgereceipts/engines/codelock/render.py

```

"""Rosetta Render: deterministic per-token view-layer styles.

Styles are derived from ``hashlib.sha256(seed, token_index, token)``.
They never mutate the source. Normalize HTML applies a fixed monospace
face and zero transforms. CodeLock HTML applies size, optional hue,
micro-rotation, and spacing variance.

This module does not encrypt, obfuscate, or hide text.
"""

from __future__ import annotations

import hashlib
import html
from typing import Mapping, Sequence, TypedDict

from codelock.gate import ACK_PHRASE
from codelock.tokenize import tokenize

FONT_SIZE_MIN_PX = 11
FONT_SIZE_MAX_PX = 22
ROTATE_DEG = 4.0
SPACING_EM = 0.08
NORMALIZE_FONT_PX = 14
MONOSPACE = (
    "ui-monospace, SFMono-Regular, Menlo, Consolas, "
    "Liberation Mono", monospace'
)

class TokenStyle(TypedDict):
    font_size_px: int
    hue_deg: int | None
    rotate_deg: float
    letter_spacing_em: float
    word_spacing_em: float

def _digest(seed: str | int, index: int, token: str) -> bytes:
    hasher = hashlib.sha256()
    hasher.update(str(seed).encode("utf-8"))
    hasher.update(b"\0")
    hasher.update(str(index).encode("ascii"))
    hasher.update(b"\0")
    hasher.update(token.encode("utf-8"))
    return hasher.digest()

def _map_signed(byte: int, amplitude: float) -> float:
    return round((byte / 255.0) * 2.0 * amplitude - amplitude, 6)

def style_for(
    seed: str | int,
    index: int,
    token: str,
    *,
    hue: bool = True,
) -> TokenStyle:
    digest = _digest(seed, index, token)
    span = FONT_SIZE_MAX_PX - FONT_SIZE_MIN_PX + 1 # 12 values, 11?22
    font_size_px = FONT_SIZE_MIN_PX + (digest[0] % span)
    hue_deg: int | None = None
    if hue:
        hue_deg = int.from_bytes(digest[4:6], "big") % 360
    return TokenStyle(
        font_size_px=font_size_px,
        hue_deg=hue_deg,
        rotate_deg=_map_signed(digest[1], ROTATE_DEG),
        letter_spacing_em=_map_signed(digest[2], SPACING_EM),
        word_spacing_em=_map_signed(digest[3], SPACING_EM),
    )

def styles_for(
    tokens: Sequence[str],
    seed: str | int = 0,
    *,
    hue: bool = True,
) -> list[TokenStyle]:
    """Per-token style dicts. Deterministic for (tokens, seed, hue)."""
    return [style_for(seed, i, tok, hue=hue) for i, tok in enumerate(tokens)]

def _css_escape_comment(text: str) -> str:
    return text.replace("*/", "/* /")

def _script_plain_source(source: str) -> str:
    """Embed source in a ``<script type="text/plain">`` without breakout.

    HTML script data ends at a case-insensitive ``</script``. We only

```

```

        rewrite that closer; every other character is stored verbatim so the
        artifact remains inspectable with standard tools. A matching
        ``<textarea>`` holds an HTML-escaped copy as a second inspectable
        channel (entities decode to the original in the DOM).
        """
        return source.replace("</", "<\\\/")

def normalize_html(source: str) -> str:
    """Canonical viewing state: fixed-size monospace, zero transforms."""
    escaped = html.escape(source)
    return f"""<!DOCTYPE html>
<html lang="en" data-canonical="true">
<head>
<meta charset="utf-8">
<title>CodeLock Normalize (canonical)</title>
<style>
    html, body {{
        margin: 0;
        background: #111;
        color: #ddd;
    }}
    .banner {{
        font-family: {MONOSPACE};
        font-size: 13px;
        padding: 0.75rem 1rem;
        background: #1e3a2f;
        color: #cfe;
        border-bottom: 1px solid #3a6;
    }}
    pre.canonical {{
        font-family: {MONOSPACE};
        font-size: {NORMALIZE_FONT_PX}px;
        line-height: 1.45;
        letter-spacing: 0;
        word-spacing: normal;
        transform: none;
        white-space: pre;
        margin: 1rem;
        tab-size: 4;
    }}
</style>
</head>
<body>
<div class="banner">Canonical view (Normalize). Fixed-size monospace. Zero transforms. Source is the single source of
truth.</div>
<pre class="canonical" data-canonical="true">{escaped}</pre>
</body>
</html>
"""

def _token_span(token: str, style: Mapping[str, object]) -> str:
    escaped = html.escape(token)
    if token.isspace():
        return escaped
    rules = [
        f"font-size:{int(style['font_size_px'])}px",
        f"transform:rotate({style['rotate_deg']}deg)",
        f"letter-spacing:{style['letter_spacing_em']}em",
        f"word-spacing:{style['word_spacing_em']}em",
        "display:inline-block",
        f"font-family:{MONOSPACE}",
        "transform-origin:50% 50%",
    ]
    hue = style.get("hue_deg")
    if hue is not None:
        rules.append(f"color:hsl({int(hue)},70%,55%)")
    return f'<span class="tok" style="{",".join(rules)}">{escaped}</span>'

def codelock_html(
    source: str,
    seed: str | int = 0,
    *,
    hue: bool = True,
) -> str:
    """Non-canonical Rosetta Render HTML artifact. Does not encrypt.

    The original source is stored inspectably in
    ``<script type="text/plain" id="codelock-source">`` and in a
    ``<textarea id="codelock-source-text">``. The document is marked
    ``data-canonical="false"`` with a visible banner and an HTML comment.
    """
    tokens = tokenize(source)
    styles = styles_for(tokens, seed, hue=hue)
    spans = "".join(_token_span(tok, st) for tok, st in zip(tokens, styles))
    embedded = _script_plain_source(source)
    escaped = html.escape(source)
    seed_s = html.escape(str(seed))
    ack = html.escape(ACK_PHRASE)
    return f"""<!DOCTYPE html>
<html lang="en" data-canonical="false">
<head>
<meta charset="utf-8">

```

```

<title>CodeLock visual artifact (non-canonical)</title>
<!--
NON-CANONICAL visual artifact. This is not the source of truth.
Canonical source is plain text in #codelock-source.
{ _css_escape_comment(ACK_PHRASE) }
CodeLock does not encrypt, hide, or obfuscate. Seed={seed_s}
-->
<style>
  html, body {{
    margin: 0;
    background: #0b0b0f;
    color: #eee;
  }}
  .banner {{
    font-family: {MONOSPACE};
    font-size: 13px;
    padding: 0.85rem 1rem;
    background: #4a1c1c;
    color: #f8d0d0;
    border-bottom: 2px solid #c44;
  }}
  .banner strong {{ letter-spacing: 0.04em; }}
  pre.rosetta {{
    font-family: {MONOSPACE};
    font-size: {NORMALIZE_FONT_PX}px;
    line-height: 1.7;
    white-space: pre-wrap;
    margin: 1rem;
    tab-size: 4;
  }}
  span.tok {{
    display: inline-block;
    vertical-align: baseline;
  }}
  .inspect {{
    font-family: {MONOSPACE};
    margin: 1rem;
    padding: 0.75rem;
    border: 1px dashed #666;
    background: #161616;
  }}
  .inspect h2 {{
    font-size: 14px;
    margin: 0 0 0.5rem 0;
  }}
  textarea#codelock-source-text {{
    width: 100%;
    min-height: 8rem;
    font-family: {MONOSPACE};
    font-size: 13px;
    background: #000;
    color: #cfc;
    border: 1px solid #333;
    white-space: pre;
  }}
</style>
</head>
<body>
<div class="banner" data-canonical="false">
  <strong>NON-CANONICAL</strong> visual artifact &mdash; not a substitute for source.
  {ack}
</div>
<pre class="rosetta" data-canonical="false" data-seed="{seed_s}">{spans}</pre>
<section class="inspect">
  <h2>Canonical source (inspectable, not encrypted)</h2>
  <p>This tool alters perception, not meaning. Plain text below is the single source of truth.</p>
  <textarea id="codelock-source-text" readonly>{escaped}</textarea>
</section>
<script type="text/plain" id="codelock-source">{embedded}</script>
</body>
</html>
"""

```

forgereceipts/engines/codelock/session.py

```

"""CodeLockSession: immutable source, gate-checked views, exports.

Plain text is always canonical. Rendered views never mutate source.
"""

from __future__ import annotations

from pathlib import Path
from typing import Union

from codelock.gate import ACK_PHRASE, Gate, GateClosedError
from codelock.render import TokenStyle, codelock_html, normalize_html, styles_for
from codelock.tokenize import tokenize

PathLike = Union[str, Path]

class CodeLockSession:
    """Hold source immutably and produce Normalize / CodeLock views.

    Parameters
    -----
    source:
        Canonical plain text. Never mutated.
    seed:
        Deterministic seed for Rosetta transforms.
    hue:
        If True, token styles include a hue spectrum. If False, ``hue_deg``
        is None and HTML has no color/hsl on tokens.
    """

    def __init__(
        self,
        source: str,
        *,
        seed: str | int = 0,
        hue: bool = True,
        gate: Gate | None = None,
    ) -> None:
        self._source = source
        self.seed: str | int = seed
        self.hue = bool(hue)
        self.gate = gate if gate is not None else Gate()

    @property
    def source(self) -> str:
        return self._source

    @property
    def gate_open(self) -> bool:
        return self.gate.gate_open

    def open_gate(self, acknowledgment: str) -> None:
        """Open the gate. Requires the exact phrase in ``ACK_PHRASE``."""
        self.gate.open(acknowledgment)

    def close_gate(self) -> None:
        self.gate.close()

    def tokens(self) -> list[str]:
        return tokenize(self._source)

    def styles(self) -> list[TokenStyle]:
        """Per-token Rosetta styles. Gate must be Open."""
        self.gate.require_open()
        return styles_for(self.tokens(), self.seed, hue=self.hue)

    def normalize_html(self) -> str:
        """Canonical viewing HTML. Always available, gate or not."""
        return normalize_html(self._source)

    def codelock_html(self) -> str:
        """Non-canonical Rosetta HTML. Raises GateClosedError if Closed."""
        self.gate.require_open()
        return codelock_html(self._source, self.seed, hue=self.hue)

    def export_normal(self, path: PathLike) -> Path:
        """Write verbatim source as UTF-8 ``.txt``. Canonical. Always allowed."""
        dest = Path(path)
        dest.parent.mkdir(parents=True, exist_ok=True)
        dest.write_bytes(self._source.encode("utf-8"))
        return dest

    def export_codelock(self, path: PathLike) -> Path:
        """Write a self-contained non-canonical HTML artifact. Gate must be Open."""
        self.gate.require_open()
        dest = Path(path)
        dest.parent.mkdir(parents=True, exist_ok=True)
        dest.write_text(self.codelock_html(), encoding="utf-8", newline="\n")
        return dest

# Re-export for callers who import from session.
__all__ = ["ACK_PHRASE", "CodeLockSession", "Gate", "GateClosedError"]

```

forgereceipts/engines/code/lock/tokenize.py

```

"""Split source into tokens. Whitespace is preserved.

`"".join(tokenize(source)) == source` for any string, including empty
input, unicode, and mixed whitespace.

Language-agnostic-ish: a regex tokenizer that recognizes identifiers,
keywords (Python keyword set as a convenience), punctuation, whitespace,
comments (#, //, /* */), and strings. Unknown characters become
punctuation so the join roundtrip never drops bytes.
"""

from __future__ import annotations

import re
from typing import NamedTuple

# Python keywords used only to label identifier tokens. The tokenizer
# still treats them as ordinary source text; labels do not change the
# joined string.
KEYWORDS = frozenset(
    {
        "False",
        "None",
        "True",
        "and",
        "as",
        "assert",
        "async",
        "await",
        "break",
        "class",
        "continue",
        "def",
        "del",
        "elif",
        "else",
        "except",
        "finally",
        "for",
        "from",
        "global",
        "if",
        "import",
        "in",
        "is",
        "lambda",
        "nonlocal",
        "not",
        "or",
        "pass",
        "raise",
        "return",
        "try",
        "while",
        "with",
        "yield",
    }
)

# Triple-quoted strings before single-quoted; comments before punctuation.
_PATTERN = re.compile(
    r"(?P<comment>\\#[^\\n]*|//[\\n]*|/\\^\\.\\*?\\^/)"
    r"|(?P<string>"
    r"(?:[rRuUbBfF]{1,3})?"
    r"(?:'(?:\\\\.|[\\^\\])'*|\"(?:\\\\.|[\\^\\])'*|\"(?:\\\\.|[\\^\\])'*\"")
    r"|'(?:\\\\.|[\\^\\])'*|\"(?:\\\\.|[\\^\\])'*\"")
    r")"
    r"|(?P<whitespace>\\s+)"
    r"|(?P<identifier>[\\w\\d\\_]*)"
    r"|(?P<number>\\d+(?:\\.\\d+)?(?:[eE][+-]?\\d+)?)"
    r"|(?P<punctuation>\\.)",
    re.DOTALL,
)

class Token(NamedTuple):
    kind: str
    text: str

    def __str__(self) -> str:
        return self.text

def tokenize_kinds(source: str) -> list[Token]:
    """Return (kind, text) tokens covering every character of `source`."""
    if source == "":
        return []
    out: list[Token] = []
    pos = 0
    for match in _PATTERN.finditer(source):
        if match.start() > pos:
            out.append(Token("punctuation", source[pos : match.start()]))

```

```
        kind = match.lastgroup or "punctuation"
        text = match.group()
        if kind == "identifier" and text in KEYWORDS:
            kind = "keyword"
        out.append(Token(kind, text))
        pos = match.end()
    if pos < len(source):
        out.append(Token("punctuation", source[pos:]))
    return out

def tokenize(source: str) -> list[str]:
    """Return token strings. ``"".join(tokenize(source)) == source``."""
    return [tok.text for tok in tokenize_kinds(source)]
```

forgereceipts/engines/godlock/__init__.py

```
"""GodLock: stress-test and resilience engine for the ABAD framework.

July 12 2026 whitepaper implementation by Aziel Eliab.
Shipped under GitHub user AzielEliab.

GodLock does not argue. It records, analyzes, hardens, and grows.

This package is the core + receipts engine. It is not an anonymity
network. MirageGrid and Airlock are in-process logical identities.
Forks are welcome and always allowed.
"""

from __future__ import annotations

from godlock.abad import score_engagement
from godlock.engine import GodLockEngine
from godlock.grid import Airlock, MirageGrid
from godlock.lumen import decrypt_capsule, export_capsule
from godlock.receipts import Receipt

__version__ = "0.1.0"
__author__ = "Aziel Eliab"
__all__ = [
    "Airlock",
    "GodLockEngine",
    "MirageGrid",
    "Receipt",
    "decrypt_capsule",
    "export_capsule",
    "score_engagement",
    "__version__",
]
```

forgereceipts/engines/godlock/__main__.py

```
"""python -m godlock"""

from __future__ import annotations

from godlock.cli import main

if __name__ == "__main__":
    raise SystemExit(main())
```


forgereceipts/engines/godlock/abad.py

```
"""ABAD-aware engagement scoring.
```

```
Weights below are *engineering defaults* for a local heuristic. They are
not empirical proof numbers, not ROC points, and not a claim that a
particular ratio appears in nature. Documented so a fork can change them
without pretending a study was run.
```

```
Family bonuses (applied once per family that hits):
```

aziel_sequence	3.0	"Aziel Sequence" / aziel-seq
phi	2.0	phi, golden ratio, ϕ , 1.618
sqrt2	2.0	$\sqrt{2}$, $\sqrt{2}$, square root of 2, 1.414
flower_of_life	2.5	Flower of Life, vesica piscis
corkscrew	2.0	corkscrew (growth)
abad	3.0	ABAD, A-B-A-D
merged_rule	1.5	per distinct extra keyword from merged rules

```
Multiple families stack. Extra keywords come from the active rules table
after an admin merge (simple keyword add).
```

```
from __future__ import annotations
```

```
import re
from typing import Iterable, Mapping, Sequence
```

```
# Engineering defaults ? not empirical "proof" numbers.
```

```
WEIGHTS: Mapping[str, float] = {
```

```
    "aziel_sequence": 3.0,
    "phi": 2.0,
    "sqrt2": 2.0,
    "flower_of_life": 2.5,
    "corkscrew": 2.0,
    "abad": 3.0,
    "merged_rule": 1.5,
```

```
}
```

```
_PATTERNS: Mapping[str, tuple[re.Pattern[str], ...]] = {
```

```
    "aziel_sequence": (
        re.compile(r"aziel\s+sequence", re.I),
        re.compile(r"aziel[_]?seq(?:uence)?", re.I),
    ),
    "phi": (
        re.compile(r"\bphi\b", re.I),
        re.compile(r"golden\s+ratio", re.I),
        re.compile(r"\b\?", re.I),
        re.compile(r"\b1\.618\d*\b"),
    ),
    "sqrt2": (
        re.compile(r"sqrt\s*(?:s*2", re.I),
        re.compile(r"?\s*2"),
        re.compile(r"square\s+root\s+of\s+2", re.I),
        re.compile(r"\b1\.414\d*\b"),
    ),
    "flower_of_life": (
        re.compile(r"flower\s+of\s+life", re.I),
        re.compile(r"vesica\s+piscis", re.I),
    ),
    "corkscrew": (
        re.compile(r"corkscrew", re.I),
    ),
    "abad": (
        re.compile(r"\babad\b", re.I),
        re.compile(r"a\s*[-?]\s*b\s*[-?]\s*a\s*[-?]\s*d", re.I),
        re.compile(r"ba\s*-\s*b\s*-\s*a\s*-\s*d\b", re.I),
    ),
}
```

```
STOPWORDS = frozenset(
```

```
{
    "a",
    "an",
    "the",
    "in",
    "of",
    "for",
    "and",
    "or",
    "to",
    "all",
    "with",
    "from",
    "that",
    "this",
    "require",
    "required",
    "requiring",
    "rule",
    "rules",
    "add",
    "adding",
    "future",
```

```

        "submissions",
        "submission",
        "should",
        "must",
        "will",
        "into",
        "active",
        "harden",
        "hardening",
        "suggested",
        "coverage",
        "explicit",
        "check",
        "counter",
        "argument",
        "least",
        "token",
        "tokens",
        "engage",
        "engages",
    }
)

def score_engagement(
    text: str,
    extra_keywords: Sequence[str] | None = None,
) -> dict:
    """Return ``{"score": float, "hits": list[str]}`` for *text*.

    ``hits`` lists family names (and ``merged_rule:<keyword>`` tags).
    Extra keywords are a simple add from merged rules ? not a learned model.
    """
    if not text:
        return {"score": 0.0, "hits": []}

    hits: list[str] = []
    score = 0.0
    for family, patterns in _PATTERNS.items():
        if any(p.search(text) for p in patterns):
            hits.append(family)
            score += float(WEIGHTS[family])

    for kw in _normalize_extras(extra_keywords or ()):
        if kw and kw.lower() in text.lower():
            tag = f"merged_rule:{kw}"
            if tag not in hits:
                hits.append(tag)
                score += float(WEIGHTS["merged_rule"])

    return {"score": round(score, 4), "hits": hits}

def extract_keywords(hardening: str) -> list[str]:
    """Pull distinctive tokens out of a merged hardening string."""
    tokens = re.findall(r"[A-Za-z][A-Za-z0-9_-]{3,}", hardening or "")
    out: list[str] = []
    seen: set[str] = set()
    for tok in tokens:
        low = tok.lower()
        if low in STOPWORDS or low in seen:
            continue
        if len(low) < 5:
            continue
        seen.add(low)
        out.append(low)
    return out

def _normalize_extras(extra: Iterable[str]) -> list[str]:
    seen: set[str] = set()
    out: list[str] = []
    for kw in extra:
        if not kw:
            continue
        low = str(kw).strip().lower()
        if not low or low in seen or low in STOPWORDS:
            continue
        seen.add(low)
        out.append(low)
    return out

```

forgereceipts/engines/godlock/app.py

```

"""FastAPI app for the GodLock core. Default bind is 127.0.0.1.

Localhost research tool: POST /merge is unauthenticated on purpose.
Do not expose this process on a public interface without your own auth
in front. This repo is not an anonymity network.
"""

from __future__ import annotations

from pathlib import Path
from typing import Any

from fastapi import FastAPI, Request
from fastapi.responses import HTMLResponse, JSONResponse
from fastapi.templating import Jinja2Templates

from godlock.config import DEFAULT_BIND_HOST, DEFAULT_BIND_PORT, MOTTO
from godlock.engine import GodLockEngine

TEMPLATE_DIR = Path(__file__).resolve().parent / "templates"
templates = Jinja2Templates(directory=str(TEMPLATE_DIR))

def create_app(
    engine: GodLockEngine | None = None,
    persist: bool = True,
    data_dir: str | Path | None = None,
    bind_host: str = DEFAULT_BIND_HOST,
    bind_port: int = DEFAULT_BIND_PORT,
) -> FastAPI:
    """Application factory. Default bind host is 127.0.0.1 (localhost only)."""
    eng = engine or GodLockEngine(data_dir=data_dir, persist=persist)
    app = FastAPI(
        title="GodLock",
        version="0.1.0",
        description=(
            "ABAD stress-test and resilience engine. "
            "Core + receipts. Not an anonymity network."
        ),
    )
    app.state.engine = eng
    app.state.bind_host = bind_host or DEFAULT_BIND_HOST
    app.state.bind_port = int(bind_port)

    def _dashboard(request: Request, result: dict[str, Any] | None = None) -> HTMLResponse:
        stats = eng.stats()
        return templates.TemplateResponse(
            request,
            "dashboard.html",
            {
                "motto": MOTTO,
                "counter": stats["counter"],
                "resilience_score": stats["resilience_score"],
                "rules": stats["rules"],
                "grid_nodes": stats["grid_nodes"],
                "recent_ids": stats["recent_receipt_ids"],
                "rule_list": [r.as_dict() for r in eng.rules.all()],
                "result": result,
            },
        )

    @app.get("/", response_class=HTMLResponse)
    def dashboard(request: Request) -> HTMLResponse:
        return _dashboard(request)

    @app.get("/health")
    def health() -> dict[str, Any]:
        return {
            "ok": True,
            "bind_host": app.state.bind_host,
            "persist": eng.persist,
            "motto": MOTTO,
        }

    @app.get("/stats")
    def stats() -> dict[str, Any]:
        return eng.stats()

    @app.post("/stress")
    async def stress(request: Request) -> Any:
        ctype = request.headers.get("content-type") or ""
        if "application/json" in ctype:
            body = await request.json()
            payload = str(body.get("text") or "")
            wants_html = False
        else:
            form = await request.form()
            payload = str(form.get("text") or "")
            wants_html = True
        try:
            result = eng.submit(payload)
        except ValueError as exc:

```

```

        return JSONResponse({"error": str(exc)}, status_code=400)
    if wants_html:
        return _dashboard(request, result=result)
    return result

@app.post("/merge")
async def merge(request: Request) -> Any:
    ctype = request.headers.get("content-type") or ""
    wants_html = "application/json" not in ctype
    if "application/json" in ctype:
        body = await request.json()
        rid = str(body.get("receipt_id") or body.get("receipt") or "")
        hard = str(body.get("hardening") or "")
    else:
        form = await request.form()
        rid = str(form.get("receipt_id") or "")
        hard = str(form.get("hardening") or "")
    try:
        rule = eng.merge(rid, hard)
    except KeyError as exc:
        return JSONResponse({"error": str(exc)}, status_code=404)
    except ValueError as exc:
        return JSONResponse({"error": str(exc)}, status_code=400)
    payload = {"rule": rule.as_dict(), "rules": len(eng.rules)}
    if wants_html:
        return _dashboard(request)
    return payload

@app.get("/capsules")
def capsules() -> dict[str, Any]:
    # Ids + hashes only. Plaintext is not served from disk.
    return {"capsules": eng.capsule_index()}

return app

# Convenience ASGI target is built by the CLI so the data dir is explicit.
app = None

```

forgereceipts/engines/godlock/cli.py

```

"""Command-line interface for GodLock.

    godlock serve --host 127.0.0.1 --port 8080
    godlock submit --text "... " [--out receipt.json]
    godlock score --text "... "
    godlock merge --receipt ID --hardening "... "
    godlock rules
    godlock stats
    godlock export-lumen --receipt ID --out FILE.capsule
    godlock version

submit / stats / score work fully offline with ``./godlock`` (gitignored).
"""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path
from typing import Sequence

from godlock import __version__
from godlock.config import DATA_DIR_NAME, DEFAULT_BIND_HOST, DEFAULT_BIND_PORT, MOTTO
from godlock.engine import GodLockEngine

def _data_dir(args: argparse.Namespace) -> Path:
    if getattr(args, "data_dir", None):
        return Path(args.data_dir)
    return Path.cwd() / DATA_DIR_NAME

def _engine(args: argparse.Namespace) -> GodLockEngine:
    persist = True if getattr(args, "persist", True) else False
    return GodLockEngine(data_dir=_data_dir(args), persist=persist)

def _print_json(obj: object) -> None:
    sys.stdout.write(json.dumps(obj, indent=2, ensure_ascii=False) + "\n")

def _build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="godlock",
        description=(
            "GodLock ? ABAD stress-test and resilience engine "
            "(Azriel Eliab, 12 July 2026). "
            "Core + receipts. Not an anonymity network."
        ),
    )
    parser.add_argument(
        "--data-dir",
        default=None,
        help=f"Local data directory (default: ./{DATA_DIR_NAME}).",
    )
    persist = parser.add_mutually_exclusive_group()
    persist.add_argument(
        "--persist",
        dest="persist",
        action="store_true",
        default=True,
        help="Write receipts/rules as jsonl/json under the data dir (default).",
    )
    persist.add_argument(
        "--no-persist",
        dest="persist",
        action="store_false",
        help="In-memory receipts only. Not a wipe: just skip disk.",
    )
    sub = parser.add_subparsers(dest="cmd", required=True)

    p_serve = sub.add_parser("serve", help="Run the localhost HTTP UI/API.")
    p_serve.add_argument("--host", default=DEFAULT_BIND_HOST, help="Bind host (default 127.0.0.1).")
    p_serve.add_argument("--port", type=int, default=DEFAULT_BIND_PORT, help="Bind port (default 8080).")

    p_sub = sub.add_parser("submit", help="Record a counter-argument (offline).")
    p_sub.add_argument("--text", required=True, help="Stress-test text.")
    p_sub.add_argument("--out", default=None, help="Optional path to write receipt JSON.")

    p_score = sub.add_parser("score", help="ABAD engagement score for text (offline).")
    p_score.add_argument("--text", required=True)

    p_merge = sub.add_parser("merge", help="Apply a hardening into the active rules table.")
    p_merge.add_argument("--receipt", required=True, help="Receipt id.")
    p_merge.add_argument("--hardening", default="", help="Hardening text (default: Jeeves suggestion).")

    sub.add_parser("rules", help="List active rules.")
    sub.add_parser("stats", help="Public counter and resilience score (offline).")

    p_ex = sub.add_parser("export-lumen", help="Write an encrypted Lumen capsule.")

```

```

p_ex.add_argument("--receipt", required=True)
p_ex.add_argument("--out", required=True, help="Output .capsule path.")

sub.add_parser("version", help="Print the GodLock version and exit.")
return parser

def main(argv: Sequence[str] | None = None) -> int:
    parser = _build_parser()
    args = parser.parse_args(list(argv) if argv is not None else None)

    if args.cmd == "version":
        sys.stdout.write(f"godlock {__version__}\n")
        sys.stdout.write(MOTTO + "\n")
        return 0

    if args.cmd == "serve":
        # Local import so `godlock version` does not need uvicorn at import-check time.
        import uvicorn

        from godlock.app import create_app

        engine = _engine(args)
        app = create_app(
            engine=engine,
            persist=args.persist,
            data_dir=_data_dir(args),
            bind_host=args.host,
            bind_port=args.port,
        )
        uvicorn.run(app, host=args.host, port=args.port, log_level="info")
        return 0

    engine = _engine(args)

    if args.cmd == "submit":
        result = engine.submit(args.text)
        if args.out:
            Path(args.out).write_text(json.dumps(result, indent=2, ensure_ascii=False) + "\n", encoding="utf-8")
        _print_json(result)
        return 0

    if args.cmd == "score":
        _print_json(engine.score(args.text))
        return 0

    if args.cmd == "merge":
        rule = engine.merge(args.receipt, args.hardening)
        _print_json(rule.as_dict())
        return 0

    if args.cmd == "rules":
        _print_json([r.as_dict() for r in engine.rules.all()])
        return 0

    if args.cmd == "stats":
        _print_json(engine.stats())
        return 0

    if args.cmd == "export-lumen":
        info = engine.export_lumen(args.receipt, args.out)
        _print_json(info)
        return 0

    parser.error(f"unknown command {args.cmd}")
    return 2

if __name__ == "__main__":
    raise SystemExit(main())

```

forgereceipts/engines/godlock/config.py

```
"""Shared constants for the GodLock core.

Bind default is localhost. MirageGrid/Airlock are logical identities
only ? not a real network, not Tor, not a proxy chain.
"""

from __future__ import annotations

DEFAULT_BIND_HOST = "127.0.0.1"
DEFAULT_BIND_PORT = 8080

GRID_SIZE = 25
NODE_PREFIX = "grid"
# Short session-identity rotation. Entropy-driven, in-process only.
DEFAULT_ROTATE_INTERVAL_S = 0.05

JEEVES_MODEL = "godlock-jeeves-heuristic-0.1"

MOTTO = "GodLock does not argue. It records, analyzes, hardens, and grows."

AUTHOR = "Aziel Eliab"
PAPER_DATE = "12 July 2026"
LICENSE_ID = "Apache-2.0"

DATA_DIR_NAME = ".godlock"
LUMEN_KEY_FILENAME = "lumen.key"
RECEIPTS_FILENAME = "receipts.jsonl"
RULES_FILENAME = "rules.json"
ANALYSES_FILENAME = "analyses.json"
CAPSULE_INDEX_FILENAME = "capsule_index.json"
```

forgereceipts/engines/godlock/engine.py

```

"""GodLock engine: receipts, Jeeves, rules, Lumen, logical Airlock.

Optional persistence writes jsonl/json under ``data_dir`` (default
`"./.godlock"`). ``persist=False`` is in-memory only. There is no wipe
path ? persistence is for operators who want a log, not for evasion.
"""

from __future__ import annotations

import json
from pathlib import Path
from typing import Any

from godlock.abad import score_engagement
from godlock.config import DATA_DIR_NAME, DEFAULT_ROTATE_INTERVAL_S
from godlock.grid import Airlock, MirageGrid
from godlock.jeeves import analyze as jeeves_analyze
from godlock.lumen import decrypt_capsule, export_capsule, load_or_create_key
from godlock.receipts import Receipt, ReceiptStore
from godlock.rules import Rule, RulesTable

class GodLockEngine:
    def __init__(
        self,
        data_dir: str | Path | None = None,
        persist: bool = True,
        rotate_interval_s: float = DEFAULT_ROTATE_INTERVAL_S,
    ) -> None:
        if data_dir is None and persist:
            data_dir = Path.cwd() / DATA_DIR_NAME
        self.data_dir = Path(data_dir) if data_dir is not None else None
        self.persist = bool(persist)
        if self.persist and self.data_dir is not None:
            self.data_dir.mkdir(parents=True, exist_ok=True)
        self.grid = MirageGrid(rotate_interval_s=rotate_interval_s)
        self.airlock = Airlock(self.grid)
        self.receipts = ReceiptStore(data_dir=self.data_dir, persist=self.persist)
        self.rules = RulesTable(data_dir=self.data_dir, persist=self.persist)
        self._analyses: dict[str, dict[str, Any]] = {}
        # Capsule index: ids + hashes only. Never plaintext from disk.
        self._capsules: list[dict[str, str]] = []
        self.lumen_key = load_or_create_key(self.data_dir if self.persist else None)
        self._load_side_state()

    def _analyses_path(self) -> Path | None:
        if self.data_dir is None:
            return None
        return self.data_dir / "analyses.json"

    def _capsule_index_path(self) -> Path | None:
        if self.data_dir is None:
            return None
        return self.data_dir / "capsule_index.json"

    def _load_side_state(self) -> None:
        ap = self._analyses_path()
        if self.persist and ap is not None and ap.is_file():
            self._analyses = json.loads(ap.read_text(encoding="utf-8"))
        cp = self._capsule_index_path()
        if self.persist and cp is not None and cp.is_file():
            self._capsules = json.loads(cp.read_text(encoding="utf-8"))

    def _save_analyses(self) -> None:
        ap = self._analyses_path()
        if not self.persist or ap is None:
            return
        ap.write_text(json.dumps(self._analyses, indent=2, ensure_ascii=False) + "\n", encoding="utf-8")

    def _save_capsule_index(self) -> None:
        cp = self._capsule_index_path()
        if not self.persist or cp is None:
            return
        cp.write_text(json.dumps(self._capsules, indent=2) + "\n", encoding="utf-8")

    @property
    def counter(self) -> int:
        return self.receipts.counter

    def score(self, text: str) -> dict[str, Any]:
        return score_engagement(text, extra_keywords=self.rules.extra_keywords())

    def submit(self, text: str) -> dict[str, Any]:
        if not (text or "").strip():
            raise ValueError("text is required")
        pair = self.airlock.open()
        receipt = Receipt.mint(text.strip(), pair)
        self.receipts.append(receipt)
        analysis = jeeves_analyze(receipt, extra_keywords=self.rules.extra_keywords())
        self._analyses[receipt.id] = analysis
        self._save_analyses()
        engagement = self.score(receipt.text)

```



```

        return {
            "receipt": receipt.as_dict(),
            "jeeves_analysis": analysis,
            "engagement": engagement,
            "counter": self.counter,
        }

def merge(self, receipt_id: str, hardening: str) -> Rule:
    # Confirm the receipt exists (admin merge is tied to a logged stress test).
    self.receipts.get(receipt_id)
    if not (hardening or "").strip():
        analysis = self._analyses.get(receipt_id) or {}
        hardening = analysis.get("suggested_hardening") or ""
    if not hardening:
        raise ValueError("hardening text is required")
    return self.rules.merge(receipt_id, hardening)

def analysis_for(self, receipt_id: str) -> dict[str, Any] | None:
    return self._analyses.get(receipt_id)

def export_lumen(self, receipt_id: str, out: str | Path) -> dict[str, str]:
    receipt = self.receipts.get(receipt_id)
    analysis = self._analyses.get(receipt_id) or jeeves_analyze(receipt, extra_keywords=self.rules.extra_keywords(
))
    payload = {
        "receipt": receipt.as_dict(),
        "jeeves_analysis": analysis,
        "engagement": self.score(receipt.text),
    }
    info = export_capsule(payload, Path(out), self.lumen_key)
    record = {
        "capsule_id": info["capsule_id"],
        "sha256": info["sha256"],
        "receipt_id": receipt_id,
        "path": info["path"],
    }
    self._capsules.append(record)
    self._save_capsule_index()
    # One-way: do not return plaintext. Index is ids + hashes.
    return record

def capsule_index(self) -> list[dict[str, str]]:
    # Strip path if we want to be stricter; keep path for the local operator.
    return [
        {"capsule_id": c["capsule_id"], "sha256": c["sha256"], "receipt_id": c["receipt_id"]}
        for c in self._capsules
    ]

def resilience_score(self) -> float:
    """Mean ABAD engagement of logged receipts. Engineering default, not a proof."""
    recs = self.receipts.all()
    if not recs:
        return 0.0
    total = sum(self.score(r.text)["score"] for r in recs)
    return round(total / len(recs), 4)

def stats(self) -> dict[str, Any]:
    recs = self.receipts.all()
    return {
        "counter": self.counter,
        "resilience_score": self.resilience_score(),
        "rules": len(self.rules),
        "capsules": len(self._capsules),
        "grid_nodes": self.grid.size,
        "grid_rotations": self.grid.grid.rotations,
        "persist": self.persist,
        "recent_receipt_ids": [r.id for r in self.receipts.recent(20)],
        "bind_note": "core listens on 127.0.0.1; not an anonymity network",
        "receipt_count": len(recs),
    }

def decrypt_exported_capsule(path: str | Path, key: bytes) -> dict[str, Any]:
    """Test/operator helper. Not served by the HTTP API."""
    return decrypt_capsule(Path(path), key)

```

forgereceipts/engines/godlock/grid.py

```

"""MirageGrid + Airlock as *logical* identities.

This module never opens sockets, never hops IPs, never talks to Tor, and
never builds a proxy chain. ``node_id`` is a string like ``grid-07``.

MirageGrid: 25 named logical nodes. Session identity rotates on a short
interval, driven by process entropy (``secrets``). Pure in-process.

Airlock: a request is accepted on one logical ingress node, processed
internally, and the response is attributed to a *different* logical
egress node. No sockets dropped. No real IPs. Ingress is never equal to
egress.

Public traffic in the July 12 2026 paper would sit *behind* an existing
ops layer. This repo models that layer as names, not a ghost network.
"""

from __future__ import annotations

import secrets
import time
from dataclasses import dataclass

from godlock.config import DEFAULT_ROTATE_INTERVAL_S, GRID_SIZE, NODE_PREFIX

NODES: tuple[str, ...] = tuple(
    f"{NODE_PREFIX}-{i:02d}" for i in range(1, GRID_SIZE + 1)
)

class MirageGrid:
    """25-node in-process identity roster with entropy-driven rotation."""

    def __init__(
        self,
        rotate_interval_s: float = DEFAULT_ROTATE_INTERVAL_S,
        rng: secrets.SystemRandom | None = None,
    ) -> None:
        if len(NODES) != GRID_SIZE:
            raise RuntimeError("MirageGrid must expose exactly 25 logical nodes")
        self.nodes: tuple[str, ...] = NODES
        self.rotate_interval_s = float(rotate_interval_s)
        self._rng = rng or secrets.SystemRandom()
        self._index = self._rng.randrange(GRID_SIZE)
        self._last_rotate = time.monotonic()
        self._rotations = 0

    @property
    def size(self) -> int:
        return len(self.nodes)

    @property
    def rotations(self) -> int:
        return self._rotations

    def current_node(self) -> str:
        self._maybe_rotate()
        return self.nodes[self._index]

    def _maybe_rotate(self) -> None:
        now = time.monotonic()
        elapsed = now - self._last_rotate
        if elapsed < self.rotate_interval_s and self._rotations > 0:
            return
        # Interval 0 (or first observation after construct) always rotates
        # onto a *different* node so tests can see motion without sleeping.
        if self._rotations == 0 and elapsed < self.rotate_interval_s and self.rotate_interval_s > 0:
            # First call: keep the seeded identity, mark the clock.
            self._last_rotate = now
            self._rotations = 1
            return
        self._rotate_now(now)

    def force_rotate(self) -> str:
        self._rotate_now(time.monotonic())
        return self.nodes[self._index]

    def _rotate_now(self, now: float) -> None:
        new = self._rng.randrange(GRID_SIZE)
        if new == self._index:
            new = (new + 1) % GRID_SIZE
        self._index = new
        self._last_rotate = now
        self._rotations += 1

    @dataclass(frozen=True)
    class AirlockPair:
        ingress_node: str
        egress_node: str

    class Airlock:
        """Logical ingress/egress split. No sockets. No dropped connections."""

```

```
def __init__(self, grid: MirageGrid | None = None) -> None:
    self.grid = grid or MirageGrid()

def open(self) -> AirlockPair:
    ingress = self.grid.current_node()
    # Internal processing is a no-op; attribution uses a different node.
    egress = self.grid.current_node()
    if egress == ingress:
        others = [n for n in self.grid.nodes if n != ingress]
        egress = others[self.grid._rng.randrange(len(others))]
    return AirlockPair(ingress_node=ingress, egress_node=egress)
```

forgereceipts/engines/godlock/jeeves.py

```

"""Jeeves: propose a hardening for a stress-test receipt.

Default model is ``godlock-jeeves-heuristic-0.1`` ? stdlib regex and
ABAD scoring, no network. If ``OLLAMA_HOST`` is set, a thin adapter may
try a local Ollama generate call and fall back to the heuristic on any
failure. Tests must pass with the heuristic, offline.
"""

from __future__ import annotations

import json
import os
import urllib.error
import urllib.request
from typing import Any

from godlock.abad import score_engagement
from godlock.config import JEEVES_MODEL
from godlock.receipts import Receipt

OLLAMA_TIMEOUT_S = 0.4

def analyze(receipt: Receipt, extra_keywords: list[str] | None = None) -> dict[str, Any]:
    """Return ``{receipt_id, suggested_hardening, model, notes}``. """
    host = (os.environ.get("OLLAMA_HOST") or "").strip()
    if host:
        try:
            result = _ollama_analyze(receipt, host)
            if result:
                result["receipt_id"] = receipt.id
                return result
        except Exception as exc: # noqa: BLE001 ? any network failure is a fallback
            heuristic = _heuristic(receipt, extra_keywords)
            heuristic["notes"] = (
                heuristic["notes"] + f" Ollama adapter failed ({type(exc).__name__}); heuristic used."
            )
            return heuristic
    return _heuristic(receipt, extra_keywords)

def _heuristic(receipt: Receipt, extra_keywords: list[str] | None = None) -> dict[str, Any]:
    engagement = score_engagement(receipt.text, extra_keywords=extra_keywords)
    hits = list(engagement["hits"])
    families = [h for h in hits if not h.startswith("merged_rule:")]
    if families:
        suggested = (
            "Harden ABAD coverage for: "
            + ", ".join(families)
            + f". Require an explicit {families[0]} check in the active rules."
        )
        notes = (
            f"Heuristic engagement score={engagement['score']}. "
            "Families present in the counter-argument are treated as the "
            "surface that should be hardened. Not a language model."
        )
    else:
        suggested = (
            "Add a rule requiring the counter-argument to engage at least "
            "one ABAD token (Aziel Sequence, phi, sqrt(2), Flower of Life, "
            "corkscrew, A-B-A-D)."
        )
        notes = (
            f"Heuristic engagement score={engagement['score']}. "
            "No ABAD family hit. Suggested rule is a keyword floor, not a proof."
        )
    return {
        "receipt_id": receipt.id,
        "suggested_hardening": suggested,
        "model": JEEVES_MODEL,
        "notes": notes,
        "engagement": engagement,
    }

def _ollama_analyze(receipt: Receipt, host: str) -> dict[str, Any] | None:
    """Thin optional adapter. Never required. Short timeout. Local host only."""
    base = host.rstrip("/")
    url = base + "/api/generate"
    prompt = (
        "You are Jeeves for GodLock. Propose one short hardening rule for "
        "this counter-argument. Reply with JSON keys suggested_hardening and notes only.\n\n"
        + receipt.text[:2000]
    )
    payload = json.dumps({"model": "llama3.2", "prompt": prompt, "stream": False}).encode("utf-8")
    req = urllib.request.Request(
        url,
        data=payload,
        headers={"Content-Type": "application/json"},
        method="POST",
    )
    with urllib.request.urlopen(req, timeout=OLLAMA_TIMEOUT_S) as resp:

```

```
    raw = json.loads(resp.read().decode("utf-8"))
    text = (raw.get("response") or "").strip()
    if not text:
        return None
    suggested = text.splitlines()[0][:500]
    return {
        "receipt_id": receipt.id,
        "suggested_hardening": suggested,
        "model": f"ollama:{raw.get('model', 'unknown')}",
        "notes": "Ollama adapter (optional). Prompt truncated to 2000 chars.",
    }
```

forgereceipts/engines/godlock/lumen.py

```
"""Lumen capsules: one-way AES-GCM export of receipt + analysis JSON.
```

```
After write, the API does not serve plaintext back from disk. The engine
keeps an in-process index of capsule ids and SHA-256 hashes only.
```

```
Key source (first match):
```

1. env ``GODLOCK\_LUMEN\_KEY`` (64 hex chars, or any string SHA-256'd to 32 bytes)
2. ``<data\_dir>/lumen.key`` (generated on first use, gitignored)

```
Tests decrypt with the same key to verify roundtrip. Wrong key fails.
```

```
"""
```

```
from __future__ import annotations
```

```
import hashlib
import json
import os
import secrets
import uuid
from pathlib import Path
from typing import Any
```

```
from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
```

```
MAGIC = b"GLC1"
NONCE_SIZE = 12
KEY_SIZE = 32
```

```
class LumenError(Exception):
    pass
```

```
class LumenAuthError(LumenError):
    pass
```

```
def derive_key(material: str | bytes) -> bytes:
    if isinstance(material, bytes):
        raw = material
        text = None
    else:
        text = material.strip()
        raw = text.encode("utf-8")
    if len(raw) == KEY_SIZE:
        return raw
    if text is not None and len(text) == 64:
        try:
            out = bytes.fromhex(text)
            if len(out) == KEY_SIZE:
                return out
        except ValueError:
            pass
    return hashlib.sha256(raw).digest()
```

```
def load_or_create_key(data_dir: Path | None = None, env_key: str | None = None) -> bytes:
    env_val = env_key if env_key is not None else os.environ.get("GODLOCK_LUMEN_KEY")
    if env_val:
        return derive_key(env_val)
    if data_dir is None:
        return secrets.token_bytes(KEY_SIZE)
    data_dir.mkdir(parents=True, exist_ok=True)
    path = data_dir / "lumen.key"
    if path.is_file():
        return derive_key(path.read_text(encoding="utf-8").strip())
    key = secrets.token_bytes(KEY_SIZE)
    path.write_text(key.hex() + "\n", encoding="utf-8")
    try:
        os.chmod(path, 0o600)
    except OSError:
        pass
    return key
```

```
def encrypt_bytes(plaintext: bytes, key: bytes) -> bytes:
    if len(key) != KEY_SIZE:
        raise LumenError("AES-GCM key must be 32 bytes")
    nonce = os.urandom(NONCE_SIZE)
    ct = AESGCM(key).encrypt(nonce, plaintext, MAGIC)
    return MAGIC + nonce + ct
```

```
def decrypt_bytes(blob: bytes, key: bytes) -> bytes:
    if len(key) != KEY_SIZE:
        raise LumenError("AES-GCM key must be 32 bytes")
    if not blob.startswith(MAGIC) or len(blob) < len(MAGIC) + NONCE_SIZE + 16:
        raise LumenError("not a GodLock Lumen capsule")
    nonce = blob[len(MAGIC) : len(MAGIC) + NONCE_SIZE]
    ct = blob[len(MAGIC) + NONCE_SIZE :]
```

```
try:
    return AESGCM(key).decrypt(nonce, ct, MAGIC)
except InvalidTag as exc:
    raise LumenAuthError("wrong key or corrupted capsule") from exc

def export_capsule(payload: dict[str, Any], path: Path, key: bytes) -> dict[str, str]:
    plaintext = json.dumps(payload, sort_keys=True, ensure_ascii=False, separators=(",", ":")).encode("utf-8")
    blob = encrypt_bytes(plaintext, key)
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_bytes(blob)
    capsule_id = str(uuid.uuid4())
    return {
        "capsule_id": capsule_id,
        "sha256": hashlib.sha256(blob).hexdigest(),
        "path": str(path),
    }

def decrypt_capsule(path: Path, key: bytes) -> dict[str, Any]:
    blob = Path(path).read_bytes()
    plaintext = decrypt_bytes(blob, key)
    return json.loads(plaintext.decode("utf-8"))
```

forgereceipts/engines/godlock/receipts.py

```

"""Immutable stress-test receipts and the public resilience counter.

A receipt is append-only. There is no wipe, no redact-in-place, and no
anti-forensics API. Optional persistence is jsonl (one object per line)
or in-memory. Operators who want a log can keep it; this tool will not
pretend to erase it.

Hash is SHA-256 of the canonical JSON of id, timestamp, ingress_node,
egress_node, and text.
"""

from __future__ import annotations

import hashlib
import json
import threading
import uuid
from dataclasses import asdict, dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Iterator

from godlock.grid import Airlock, AirlockPair

def _utc_now() -> str:
    return datetime.now(timezone.utc).replace(microsecond=0).isoformat().replace("+00:00", "Z")

def canonical_payload(receipt_id: str, timestamp: str, ingress_node: str, egress_node: str, text: str) -> bytes:
    body = {
        "id": receipt_id,
        "timestamp": timestamp,
        "ingress_node": ingress_node,
        "egress_node": egress_node,
        "text": text,
    }
    return json.dumps(body, sort_keys=True, separators=(",", ":"), ensure_ascii=False).encode("utf-8")

def receipt_hash(receipt_id: str, timestamp: str, ingress_node: str, egress_node: str, text: str) -> str:
    return hashlib.sha256(canonical_payload(receipt_id, timestamp, ingress_node, egress_node, text)).hexdigest()

@dataclass(frozen=True)
class Receipt:
    id: str
    timestamp: str
    ingress_node: str
    egress_node: str
    text: str
    hash: str

    def as_dict(self) -> dict:
        return asdict(self)

    def verify(self) -> bool:
        expected = receipt_hash(self.id, self.timestamp, self.ingress_node, self.egress_node, self.text)
        return expected == self.hash

    @classmethod
    def mint(cls, text: str, pair: AirlockPair, receipt_id: str | None = None, timestamp: str | None = None) -> "Receipt":
        rid = receipt_id or str(uuid.uuid4())
        ts = timestamp or _utc_now()
        digest = receipt_hash(rid, ts, pair.ingress_node, pair.egress_node, text)
        return cls(
            id=rid,
            timestamp=ts,
            ingress_node=pair.ingress_node,
            egress_node=pair.egress_node,
            text=text,
            hash=digest,
        )

class ImmutableReceiptError(Exception):
    """Raised when a caller tries to overwrite or mutate a stored receipt."""

class ReceiptStore:
    """Append-only receipt log plus a public integer counter.

    ``persist=False`` keeps everything in memory.
    ``persist=True`` appends JSON lines under ``data_dir``. There is no
    method that truncates or shreds the log.
    """

    def __init__(self, data_dir: Path | None = None, persist: bool = True) -> None:
        self.data_dir = Path(data_dir) if data_dir is not None else None
        self.persist = bool(persist)
        self._lock = threading.Lock()

```



```

        self._by_id: dict[str, Receipt] = {}
        self._order: list[str] = []
        self._counter = 0
        if self.persist and self.data_dir is not None:
            self.data_dir.mkdir(parents=True, exist_ok=True)
            self._load()

    @property
    def path(self) -> Path | None:
        if self.data_dir is None:
            return None
        return self.data_dir / "receipts.jsonl"

    @property
    def counter(self) -> int:
        return self._counter

    def __len__(self) -> int:
        return len(self._order)

    def _load(self) -> None:
        path = self.path
        if path is None or not path.is_file():
            return
        for line in path.read_text(encoding="utf-8").splitlines():
            line = line.strip()
            if not line:
                continue
            raw = json.loads(line)
            rec = Receipt(
                id=raw["id"],
                timestamp=raw["timestamp"],
                ingress_node=raw["ingress_node"],
                egress_node=raw["egress_node"],
                text=raw["text"],
                hash=raw["hash"],
            )
            if rec.id in self._by_id:
                continue
            self._by_id[rec.id] = rec
            self._order.append(rec.id)
        self._counter = len(self._order)

    def append(self, receipt: Receipt) -> Receipt:
        if not isinstance(receipt, Receipt):
            raise TypeError("only Receipt instances may be stored")
        if not receipt.verify():
            raise ImutableReceiptError("receipt hash does not match canonical payload")
        with self._lock:
            existing = self._by_id.get(receipt.id)
            if existing is not None:
                if existing != receipt:
                    raise ImutableReceiptError(f"receipt {receipt.id} already exists and cannot be replaced")
                return existing
            self._by_id[receipt.id] = receipt
            self._order.append(receipt.id)
            self._counter += 1
            if self.persist and self.path is not None:
                self.path.parent.mkdir(parents=True, exist_ok=True)
                with self.path.open("a", encoding="utf-8") as fh:
                    fh.write(json.dumps(receipt.as_dict(), ensure_ascii=False) + "\n")
        return receipt

    def get(self, receipt_id: str) -> Receipt:
        try:
            return self._by_id[receipt_id]
        except KeyError as exc:
            raise KeyError(f"unknown receipt {receipt_id}") from exc

    def recent(self, n: int = 20) -> list[Receipt]:
        ids = self._order[-n:]
        return [self._by_id[i] for i in reversed(ids)]

    def all(self) -> list[Receipt]:
        return [self._by_id[i] for i in self._order]

    def __iter__(self) -> Iterator[Receipt]:
        return iter(self.all())

def mint_receipt(text: str, airlock: Airlock) -> Receipt:
    pair = airlock.open()
    return Receipt.mint(text, pair)

```

forgereceipts/engines/godlock/rules.py

```

"""Active rules table. Admin merge applies a Jeeves hardening.

Merge is corkscrew growth: the table only grows. Rules are not deleted
by this API. Keywords extracted from the hardening feed back into
ABAD scoring as extra hits (simple keyword add).
"""

from __future__ import annotations

import json
import threading
import uuid
from dataclasses import asdict, dataclass, field
from datetime import datetime, timezone
from pathlib import Path

from godlock.abad import extract_keywords

def _utc_now() -> str:
    return datetime.now(timezone.utc).replace(microsecond=0).isoformat().replace("+00:00", "Z")

@dataclass(frozen=True)
class Rule:
    id: str
    receipt_id: str
    hardening: str
    keywords: tuple[str, ...]
    created_at: str

    def as_dict(self) -> dict:
        d = asdict(self)
        d["keywords"] = list(self.keywords)
        return d

class RulesTable:
    def __init__(self, data_dir: Path | None = None, persist: bool = True) -> None:
        self.data_dir = Path(data_dir) if data_dir is not None else None
        self.persist = bool(persist)
        self._lock = threading.Lock()
        self._rules: list[Rule] = []
        if self.persist and self.data_dir is not None:
            self.data_dir.mkdir(parents=True, exist_ok=True)
            self._load()

    @property
    def path(self) -> Path | None:
        if self.data_dir is None:
            return None
        return self.data_dir / "rules.json"

    def _load(self) -> None:
        path = self.path
        if path is None or not path.is_file():
            return
        raw = json.loads(path.read_text(encoding="utf-8"))
        for item in raw:
            self._rules.append(
                Rule(
                    id=item["id"],
                    receipt_id=item["receipt_id"],
                    hardening=item["hardening"],
                    keywords=tuple(item.get("keywords") or ()),
                    created_at=item["created_at"],
                )
            )

    def _dump(self) -> None:
        if not self.persist or self.path is None:
            return
        self.path.parent.mkdir(parents=True, exist_ok=True)
        payload = [r.as_dict() for r in self._rules]
        self.path.write_text(json.dumps(payload, indent=2, ensure_ascii=False) + "\n", encoding="utf-8")

    def merge(self, receipt_id: str, hardening: str) -> Rule:
        if not (hardening or "").strip():
            raise ValueError("hardening text is required")
        keywords = tuple(extract_keywords(hardening))
        rule = Rule(
            id=str(uuid.uuid4()),
            receipt_id=receipt_id,
            hardening=hardening.strip(),
            keywords=keywords,
            created_at=_utc_now(),
        )
        with self._lock:
            self._rules.append(rule)
            self._dump()
        return rule

    def all(self) -> list[Rule]:

```

```
        return list(self._rules)

def extra_keywords(self) -> list[str]:
    seen: set[str] = set()
    out: list[str] = []
    for rule in self._rules:
        for kw in rule.keywords:
            low = kw.lower()
            if low not in seen:
                seen.add(low)
                out.append(low)
    return out

def __len__(self) -> int:
    return len(self._rules)
```

forgereceipts/engines/godlock/templates/dashboard.html

```

<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>GodLock</title>
  <style>
    :root { color-scheme: dark; }
    body { font-family: Georgia, serif; margin: 0; background: #111; color: #e8e0d0; }
    main { max-width: 46rem; margin: 0 auto; padding: 2rem 1.25rem 4rem; }
    h1 { font-weight: 400; letter-spacing: 0.08em; }
    .motto { font-style: italic; color: #c4b59a; }
    .warn { border: 1px solid #8a5a2b; padding: 0.75rem 1rem; color: #f0d0a8; }
    dl { display: grid; grid-template-columns: 11rem 1fr; gap: 0.35rem 1rem; }
    dt { color: #9a8b74; }
    form { margin: 1.5rem 0; }
    textarea { width: 100%; min-height: 7rem; background: #1c1c1c; color: inherit; border: 1px solid #444; padding: 0.6rem; }
    button { background: #3d2a16; color: #f3e6d0; border: 1px solid #8a5a2b; padding: 0.45rem 1rem; cursor: pointer; }
    code, .id { font-family: ui-monospace, monospace; font-size: 0.85rem; }
    ul { padding-left: 1.2rem; }
    a { color: #d4b483; }
    footer { margin-top: 2rem; font-size: 0.85rem; color: #8a8070; }
  </style>
</head>
<body>
<main>
  <h1>GodLock</h1>
  <p class="motto">{{ motto }}</p>
  <p class="warn">
    This process is the <strong>core + receipts engine</strong>.
    It binds to <code>127.0.0.1</code> by default.
    MirageGrid/Airlock are logical node names, not an anonymity network.
    POST /merge has <strong>no auth</strong> because this is a localhost research tool.
    Forks are welcome.
  </p>

  <h2>Public counters</h2>
  <dl>
    <dt>resilience counter</dt><dd>{{ counter }}</dd>
    <dt>resilience score</dt><dd>{{ resilience_score }}</dd>
    <dt>active rules</dt><dd>{{ rules }}</dd>
    <dt>grid nodes</dt><dd>{{ grid_nodes }} logical</dd>
  </dl>

  <h2>Submit a counter-argument</h2>
  <form method="post" action="/stress">
    <textarea name="text" required placeholder="Record a stress test?"></textarea>
    <p><button type="submit">Record</button></p>
  </form>

  {% if result %}
  <h2>Last receipt</h2>
  <p class="id">{{ result.receipt.id }}</p>
  <p>ingress <code>{{ result.receipt.ingress_node }}</code> ? egress <code>{{ result.receipt.egress_node }}</code></p>
  <p>hash <code>{{ result.receipt.hash }}</code></p>
  <p>Jeeves ({{ result.jeeves_analysis.model }}): {{ result.jeeves_analysis.suggested_hardening }}</p>
  {% endif %}

  <h2>Recent receipt ids</h2>
  <ul>
    {% for rid in recent_ids %}
    <li class="id">{{ rid }}</li>
    {% else %}
    <li>none yet</li>
    {% endfor %}
  </ul>

  <h2>Active rules</h2>
  <ul>
    {% for rule in rule_list %}
    <li>{{ rule.hardening }} <span class="id">(from {{ rule.receipt_id }})</span></li>
    {% else %}
    <li>none merged yet</li>
    {% endfor %}
  </ul>

  <form method="post" action="/merge">
    <p>Admin merge (no auth on localhost)</p>
    <p><input name="receipt_id" placeholder="receipt id" required style="width:100%;background:#1c1c1c;color:inherit;border:1px solid #444;padding:0.4rem;"></p>
    <p><textarea name="hardening" placeholder="hardening text (blank = use Jeeves suggestion)"></textarea></p>
    <p><button type="submit">Merge</button></p>
  </form>

  <footer>
    Aziel Eliab ? 12 July 2026 ? Apache-2.0
    ? <a href="/stats">/stats</a> ? <a href="/health">/health</a>
  </footer>
</main>
</body>
</html>

```

forgereceipts/engines/miragegrid/__init__.py

```

"""MirageGrid: distributed identity abstraction for AZ-OS.

Version 1.0 conceptual architecture, implemented as an open assignment
+ receipt engine by Aziel Eliab (2026).

A static pool of 25 named *logical* nodes. At session init, one is
selected cryptographically as the outward identity. An internal receipt
is minted. The mapping is destroyed when the session ends.

This package is not an anonymity network, VPN, or proxy mesh. Nodes
are identities (`node-01` ? `node-25`). Optional endpoint strings are
labels only. Forks are welcome and always allowed.
"""

from __future__ import annotations

from miragegrid.errors import (
    IntegrityError,
    MappingDestroyedError,
    MirageGridError,
    ReceiptError,
)
from miragegrid.pool import Node, NodePool, POOL_SIZE
from miragegrid.receipt import Receipt
from miragegrid.rng import select_index
from miragegrid.session import MirageSession

__version__ = "0.1.0"
__author__ = "Azziel Eliab"
__all__ = [
    "IntegrityError",
    "MappingDestroyedError",
    "MirageGridError",
    "Node",
    "NodePool",
    "POOL_SIZE",
    "Receipt",
    "ReceiptError",
    "MirageSession",
    "select_index",
    "__version__",
]

```

forgereceipts/engines/miragegrid/__main__.py

```
"""Allow ``python -m miragegrid`` to invoke the CLI."""

from miragegrid.cli import main

if __name__ == "__main__":
    raise SystemExit(main())
```

forgereceipts/engines/miragegrid/canon.py

```

"""Tiny TemporalLock-style canonical digest for MirageGrid receipts.

Copied in-tree so this package does not depend on ``temporallock``.
Hashed fields (v0.1.0): session_id, mirage_node, timestamp, integrity.
The receipt's own ``hash`` field is excluded.

UTF-8 JSON, sorted keys, no extra whitespace (separators=(",", ":")).
``mirage_node`` is a JSON number (integer 1725).
"""

from __future__ import annotations

import hashlib
import json
from typing import Any

CORE_FIELDS = ("integrity", "mirage_node", "session_id", "timestamp")

def canonical_bytes(
    session_id: str,
    mirage_node: int,
    timestamp: str,
    integrity: str,
) -> bytes:
    payload: dict[str, Any] = {
        "integrity": integrity,
        "mirage_node": int(mirage_node),
        "session_id": session_id,
        "timestamp": timestamp,
    }
    raw = json.dumps(payload, sort_keys=True, separators=(",", ":"), ensure_ascii=False)
    return raw.encode("utf-8")

def digest(
    session_id: str,
    mirage_node: int,
    timestamp: str,
    integrity: str,
) -> str:
    """SHA-256 (lowercase hex) of ``canonical_bytes(...)``."""
    return hashlib.sha256(
        canonical_bytes(session_id, mirage_node, timestamp, integrity)
    ).hexdigest()

```

forgereceipts/engines/miragegrid/cli.py

```

"""Command-line interface for MirageGrid.

    miragegrid version
    miragegrid nodes
    miragegrid assign [--emit-receipt FILE.json]
    miragegrid verify-receipt FILE.json
    miragegrid ui [--host 127.0.0.1] [--port 8080]
    miragegrid serve [--host 127.0.0.1] [--port 8080]

Logical identities only. Not a VPN, proxy, or anonymity network.
"""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path
from typing import Sequence

from miragegrid import __version__
from miragegrid.errors import MappingDestroyedError, ReceiptError
from miragegrid.pool import NodePool
from miragegrid.receipt import Receipt
from miragegrid.session import MirageSession
from miragegrid.ui import DEFAULT_HOST, DEFAULT_PORT, serve


def _build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="miragegrid",
        description=(
            "MirageGrid ? distributed identity abstraction for AZ-OS "
            "(Azriel Eliab, 2026). Assignment + receipt engine. "
            "Not an anonymity network, VPN, or proxy mesh."
        ),
    )
    sub = parser.add_subparsers(dest="cmd", required=True)

    sub.add_parser("version", help="Print package version.")
    sub.add_parser("nodes", help="List the 25 logical nodes.")

    p_as = sub.add_parser(
        "assign",
        help="Assign a logical node for one session; print node id and session id.",
    )
    p_as.add_argument(
        "--emit-receipt",
        default=None,
        metavar="FILE.json",
        help="Write the internal receipt JSON the operator asked for.",
    )
    p_as.add_argument(
        "--config",
        default=None,
        help="Optional JSON with endpoint labels (labels only; never connected).",
    )

    p_ver = sub.add_parser("verify-receipt", help="Verify a local receipt JSON file.")
    p_ver.add_argument("file", help="Receipt JSON path.")

    p_ui = sub.add_parser("ui", help="Localhost HTML UI (127.0.0.1 only).")
    p_ui.add_argument("--host", default=DEFAULT_HOST, help="Bind host (default 127.0.0.1).")
    p_ui.add_argument("--port", type=int, default=DEFAULT_PORT, help="Bind port (default 8080).")

    p_serve = sub.add_parser("serve", help="Alias for ui. Localhost only.")
    p_serve.add_argument("--host", default=DEFAULT_HOST, help="Bind host (default 127.0.0.1).")
    p_serve.add_argument("--port", type=int, default=DEFAULT_PORT, help="Bind port (default 8080).")
    return parser


def main(argv: Sequence[str] | None = None) -> int:
    parser = _build_parser()
    args = parser.parse_args(list(argv) if argv is not None else None)

    if args.cmd == "version":
        sys.stdout.write(f"miragegrid {__version__}\n")
        return 0

    if args.cmd == "nodes":
        pool = NodePool()
        for node in pool:
            extra = f" endpoint={node.endpoint}" if node.endpoint else ""
            sys.stdout.write(f"{node.id} {node.label}{extra}\n")
        return 0

    if args.cmd == "assign":
        pool = NodePool.from_config(args.config) if args.config else NodePool()
        try:
            with MirageSession(pool=pool) as session:
                node = session.node
                sys.stdout.write(f"{node.id}\n")

```



```

        sys.stdout.write(f"session_id={session.session_id}\n")
        if args.emit_receipt:
            session.receipt.write_json(args.emit_receipt)
            sys.stderr.write(f"wrote receipt {args.emit_receipt}\n")
    except MappingDestroyedError as exc:
        sys.stderr.write(f"miragegrid: {exc}\n")
        return 2
    return 0

if args.cmd == "verify-receipt":
    path = Path(args.file)
    if not path.is_file():
        sys.stderr.write(f"miragegrid: not found: {path}\n")
        return 2
    try:
        rec = Receipt.load(path)
    except (ReceiptError, OSError, json.JSONDecodeError) as exc:
        sys.stderr.write(f"miragegrid: {exc}\n")
        return 2
    status = rec.verify(NodePool())
    payload = {
        "integrity": status,
        "session_id": rec.session_id,
        "mirage_node": rec.mirage_node,
        "timestamp": rec.timestamp,
        "hash_ok": rec.hash_ok(),
    }
    sys.stdout.write(json.dumps(payload, indent=2) + "\n")
    return 0 if status == "PASS" else 1

if args.cmd in ("ui", "serve"):
    host = args.host or DEFAULT_HOST
    if host not in ("127.0.0.1", "localhost", "::1"):
        sys.stderr.write(f"miragegrid: refusing non-loopback bind; using 127.0.0.1\n")
        host = DEFAULT_HOST
    serve(host=host, port=int(args.port))
    return 0

parser.error(f"unknown command {args.cmd}")
return 2

if __name__ == "__main__": # pragma: no cover
    raise SystemExit(main())

```

forgereceipts/engines/miragegrid/errors.py

```
"""MirageGrid errors."""

from __future__ import annotations

class MirageGridError(Exception):
    """Base error for the assignment + receipt engine."""

class MappingDestroyedError(MirageGridError):
    """Raised when the session-to-node mapping is accessed after close.

    Close is in-process forget of the mapping, not a log wipe.
    """

class ReceiptError(MirageGridError):
    """Raised when a receipt is missing fields or cannot be parsed."""

class IntegrityError(MirageGridError):
    """Raised when live integrity is FAIL and a caller required PASS."""
```

forgereceipts/engines/miragegrid/pool.py

```

"""Static pool of 25 persistent logical node records.

Nodes are identities (`node-01` ? `node-25`), not hosts on a network.
Optional `endpoint` strings from a config file are labels only ? this
module never connects, never tunnels, never binds a socket.
"""

from __future__ import annotations

import json
from dataclasses import dataclass
from pathlib import Path
from typing import Iterator, Mapping

POOL_SIZE = 25

@dataclass(frozen=True)
class Node:
    """One persistent logical node.

    ``id`` is ``node-01`` ? ``node-25``.
    ``label`` is ``Node01`` ? ``Node25``.
    ``index`` is ``0..24``.
    ``number`` is ``1..25`` (receipt field ``mirage_node``).
    ``endpoint`` is an optional display label, never a connection target.
    """

    id: str
    label: str
    index: int
    number: int
    endpoint: str | None = None

    def to_dict(self) -> dict:
        return {
            "id": self.id,
            "label": self.label,
            "index": self.index,
            "number": self.number,
            "endpoint": self.endpoint,
        }

def node_id_for(index: int) -> str:
    return f"node-{index + 1:02d}"

def node_label_for(index: int) -> str:
    return f"Node{index + 1:02d}"

class NodePool:
    """Exactly 25 named logical nodes. Length is always 25."""

    def __init__(self, endpoints: Mapping[str, str] | None = None) -> None:
        eps = dict(endpoints or {})
        nodes: list[Node] = []
        for index in range(POOL_SIZE):
            nid = node_id_for(index)
            label = node_label_for(index)
            endpoint = eps.get(nid) or eps.get(label)
            if endpoint is not None:
                endpoint = str(endpoint)
            nodes.append(
                Node(
                    id=nid,
                    label=label,
                    index=index,
                    number=index + 1,
                    endpoint=endpoint,
                )
            )
        self._nodes: tuple[Node, ...] = tuple(nodes)
        self._by_id: dict[str, Node] = {n.id: n for n in self._nodes}
        self._by_label: dict[str, Node] = {n.label: n for n in self._nodes}
        if len(self._nodes) != POOL_SIZE:
            raise RuntimeError("NodePool must expose exactly 25 logical nodes")
        if len(self._by_id) != POOL_SIZE:
            raise RuntimeError("NodePool ids must be unique")

    def __len__(self) -> int:
        return POOL_SIZE

    def __iter__(self) -> Iterator[Node]:
        return iter(self._nodes)

    def __getitem__(self, index: int) -> Node:
        return self._nodes[index]

    @property
    def nodes(self) -> tuple[Node, ...]:

```

```

        return self._nodes

def by_id(self, node_id: str) -> Node:
    return self._by_id[node_id]

def by_index(self, index: int) -> Node:
    if index < 0 or index >= POOL_SIZE:
        raise IndexError(f"node index {index} out of range 0..24")
    return self._nodes[index]

def by_number(self, number: int) -> Node:
    """Look up by receipt ``mirage_node`` (1?25)."""
    if number < 1 or number > POOL_SIZE:
        raise IndexError(f"mirage_node {number} out of range 1..25")
    return self._nodes[number - 1]

def contains(self, node_id: str) -> bool:
    return node_id in self._by_id

def contains_number(self, number: int) -> bool:
    return isinstance(number, int) and 1 <= number <= POOL_SIZE

@classmethod
def from_config(cls, path: str | Path) -> "NodePool":
    """Load optional endpoint *labels* from a JSON file.

    Expected shape::

        {"endpoints": {"node-01": "booth-alpha"}}

    Values are never used as connection targets.
    """
    raw = json.loads(Path(path).read_text(encoding="utf-8"))
    endpoints: dict[str, str] = {}
    if isinstance(raw, dict):
        block = raw.get("endpoints", raw)
        if isinstance(block, dict):
            endpoints = {str(k): str(v) for k, v in block.items()}
    return cls(endpoints=endpoints)

```

forgereceipts/engines/miragegrid/receipt.py

```

"""Internal session receipt (whitepaper section 3.4).

Default: receipts live in memory and are not written to a public API.
The operator may emit a local JSON file with ``--emit-receipt``.

Fields: session_id (hex), mirage_node (1?25), timestamp UTC ISO,
integrity PASS/FAIL, plus an optional canonical SHA-256 (AZE/DIF-E
flavor, no temporallock dependency).
"""

from __future__ import annotations

import json
from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Mapping

from miragegrid.canon import digest
from miragegrid.errors import ReceiptError
from miragegrid.pool import Node, NodePool, POOL_SIZE, node_id_for


def utc_now() -> str:
    """UTC ISO-8601 with trailing Z, second precision (paper example)."""
    return (
        datetime.now(timezone.utc)
        .replace(microsecond=0)
        .isoformat()
        .replace("+00:00", "Z")
    )


def evaluate_integrity(
    node_id: str | None,
    pool: NodePool,
    closed: bool,
) -> str:
    """PASS if the node id is in the pool and the session is not closed."""
    if closed or node_id is None:
        return "FAIL"
    if not pool.contains(node_id):
        return "FAIL"
    return "PASS"


def integrity_for_number(mirage_node: int, pool: NodePool) -> str:
    """PASS iff ``mirage_node`` is 1?25 and that node exists in the pool."""
    if not isinstance(mirage_node, int) or not pool.contains_number(mirage_node):
        return "FAIL"
    try:
        node = pool.by_number(mirage_node)
    except IndexError:
        return "FAIL"
    if not pool.contains(node.id):
        return "FAIL"
    return "PASS"


@dataclass(frozen=True)
class Receipt:
    """Frozen internal receipt. Not part of default session dump/str."""

    session_id: str
    mirage_node: int
    timestamp: str
    integrity: str
    hash: str

    @classmethod
    def mint(
        cls,
        *,
        session_id: str,
        node: Node,
        timestamp: str,
        pool: NodePool,
        closed: bool = False,
    ) -> "Receipt":
        integrity = evaluate_integrity(node.id, pool, closed)
        rec_hash = digest(
            session_id=session_id,
            mirage_node=node.number,
            timestamp=timestamp,
            integrity=integrity,
        )
        return cls(
            session_id=session_id,
            mirage_node=node.number,
            timestamp=timestamp,
            integrity=integrity,
            hash=rec_hash,

```

```

    )

def recomputed_hash(self) -> str:
    return digest(
        session_id=self.session_id,
        mirage_node=self.mirage_node,
        timestamp=self.timestamp,
        integrity=self.integrity,
    )

def hash_ok(self) -> bool:
    return self.recomputed_hash() == self.hash

def evaluate(self, pool: NodePool) -> str:
    """Recompute integrity against a pool. Forged node ids FAIL.

    Hash mismatch also FAIL. Stored ``integrity`` of FAIL stays FAIL.
    """
    if not self.hash_ok():
        return "FAIL"
    live = integrity_for_number(self.mirage_node, pool)
    if live == "FAIL":
        return "FAIL"
    if self.integrity != "PASS":
        return "FAIL"
    return "PASS"

def verify(self, pool: NodePool) -> str:
    return self.evaluate(pool)

def to_dict(self) -> dict[str, Any]:
    return {
        "session_id": self.session_id,
        "mirage_node": self.mirage_node,
        "timestamp": self.timestamp,
        "integrity": self.integrity,
        "hash": self.hash,
    }

def to_json(self) -> str:
    return json.dumps(self.to_dict(), indent=2, ensure_ascii=False)

def write_json(self, path: str | Path) -> None:
    Path(path).write_text(self.to_json() + "\n", encoding="utf-8")

@classmethod
def from_dict(cls, data: Mapping[str, Any]) -> "Receipt":
    required = ("session_id", "mirage_node", "timestamp", "integrity")
    missing = [k for k in required if k not in data]
    if missing:
        raise ReceiptError(f"receipt missing fields: {missing}")
    try:
        mirage_node = int(data["mirage_node"])
    except (TypeError, ValueError) as exc:
        raise ReceiptError("mirage_node must be an integer 1?25") from exc
    integrity = str(data["integrity"])
    session_id = str(data["session_id"])
    timestamp = str(data["timestamp"])
    stored_hash = data.get("hash")
    if stored_hash is None:
        stored_hash = digest(session_id, mirage_node, timestamp, integrity)
    return cls(
        session_id=session_id,
        mirage_node=mirage_node,
        timestamp=timestamp,
        integrity=integrity,
        hash=str(stored_hash),
    )

@classmethod
def load(cls, path: str | Path) -> "Receipt":
    raw = json.loads(Path(path).read_text(encoding="utf-8"))
    if not isinstance(raw, dict):
        raise ReceiptError("receipt file must be a JSON object")
    return cls.from_dict(raw)

def node_id_from_number(mirage_node: int) -> str:
    if mirage_node < 1 or mirage_node > POOL_SIZE:
        return f"node-{{mirage_node:02d}}"
    return node_id_for(mirage_node - 1)

```

forgereceipts/engines/miragegrid/rng.py

```

"""Session randomization engine (whitepaper section 6).

Protocol
-----
    seed = system_entropy + timestamp
    rng = cryptographic_random(seed)
    node_index = rng % 25

Exact bytes concatenated for ``seed``
-----
    entropy:    32 bytes from ``secrets.token_bytes(32)`` (or caller-supplied)
    timestamp:  UTC ISO-8601 with trailing ``Z`` (second precision), UTF-8
    seed bytes: ``entropy || timestamp.encode("utf-8")``

``cryptographic_random`` is ``SHA-256(seed)``. The 32-byte digest is
interpreted as a big-endian unsigned integer:

    node_index = int.from_bytes(digest, "big") % 25

Returns ``0..24``. This module does not import the stdlib ``random`` module and does not
use choice-style selection. Paper section 3.2 is a sketch; section 6 is
the protocol this engine implements.
"""

from __future__ import annotations

import hashlib
import secrets

POOL_SIZE = 25
ENTROPY_BYTES = 32

def fresh_entropy() -> bytes:
    """32 bytes of system entropy via ``secrets.token_bytes``."""
    return secrets.token_bytes(ENTROPY_BYTES)

def seed_bytes(entropy: bytes, timestamp: str) -> bytes:
    """Concatenate ``entropy || timestamp.encode("utf-8")``.

    ``timestamp`` is the UTC ISO-8601 string used as the protocol clock
    (example: ``2026-03-04T13:02:12Z``).
    """
    if not isinstance(entropy, (bytes, bytearray)):
        raise TypeError("entropy must be bytes")
    if not isinstance(timestamp, str):
        raise TypeError("timestamp must be a UTC ISO-8601 string")
    return bytes(entropy) + timestamp.encode("utf-8")

def select_index(entropy: bytes, timestamp: str) -> int:
    """Return a deterministic node index in ``0..24`` for entropy+timestamp.

    Same inputs always yield the same index. Does not use ``random.choice``.
    """
    digest = hashlib.sha256(seed_bytes(entropy, timestamp)).digest()
    return int.from_bytes(digest, "big") % POOL_SIZE

```

forgereceipts/engines/miragegrid/session.py

```

"""MirageSession lifecycle: initiate ? assign ? operate ? end.

End destroys the in-process session-to-node mapping (forget, not a
log wipe). After close, ``session.node`` raises MappingDestroyedError
and the internal node reference is None.

This handle never opens sockets, never hops IPs, never speaks Tor,
and never builds a proxy or tunnel. The assigned node is a logical
identity from the static pool of 25.
"""

from __future__ import annotations

import json
import secrets
from typing import Any

from miragegrid.errors import MappingDestroyedError
from miragegrid.pool import Node, NodePool
from miragegrid.receipt import Receipt, evaluate_integrity, utc_now
from miragegrid.rng import fresh_entropy, select_index

class MirageSession:
    """One phone-booth session: a single assigned logical node.

    Constructing a session (or entering the context manager) runs
    initiate: cryptographic selection from the 25-node pool, mint of
    an internal receipt. ``end()`` / context exit drops the mapping.
    """

    def __init__(
        self,
        pool: NodePool | None = None,
        *,
        entropy: bytes | None = None,
        timestamp: str | None = None,
        session_id: str | None = None,
        assign: bool = True,
    ) -> None:
        self.pool = pool if pool is not None else NodePool()
        self.session_id = session_id if session_id is not None else secrets.token_hex(16)
        self._closed = False
        self._node: Node | None = None
        self._receipt: Receipt | None = None
        self._timestamp: str | None = None
        self._entropy: bytes | None = None
        if assign:
            self.initiate(entropy=entropy, timestamp=timestamp)

    def initiate(
        self,
        entropy: bytes | None = None,
        timestamp: str | None = None,
    ) -> Node:
        """Assign a node from the pool. Idempotent while the session is open."""
        if self._closed:
            raise MappingDestroyedError(
                "session mapping destroyed; start a new MirageSession"
            )
        if self._node is not None:
            return self._node
        ts = timestamp if timestamp is not None else self._timestamp
        if ts is None:
            ts = utc_now()
        self._timestamp = ts
        ent = entropy if entropy is not None else self._entropy
        if ent is None:
            ent = fresh_entropy()
        self._entropy = ent
        index = select_index(ent, ts)
        node = self.pool.by_index(index)
        self._node = node
        self._receipt = Receipt.mint(
            session_id=self.session_id,
            node=node,
            timestamp=ts,
            pool=self.pool,
            closed=False,
        )
        return node

    @property
    def closed(self) -> bool:
        return self._closed

    @property
    def node(self) -> Node:
        if self._closed:
            raise MappingDestroyedError(
                "session mapping destroyed; start a new MirageSession"
            )

```



```

    if self._node is None:
        raise MappingDestroyedError("no node assigned")
    return self._node

@property
def receipt(self) -> Receipt:
    """Internal receipt. Not included in default dump / str / JSON."""
    if self._receipt is None:
        raise MappingDestroyedError("no receipt; session was never initiated")
    return self._receipt

@property
def integrity(self) -> str:
    """Live integrity: PASS iff a pool node is mapped and not closed."""
    node_id = None if self._node is None else self._node.id
    return evaluate_integrity(node_id, self.pool, self._closed)

def emit_receipt(self) -> dict[str, Any]:
    """Operator-requested receipt dict (for ``--emit-receipt`` / UI)."""
    return self.receipt.to_dict()

def to_dict(self, *, include_receipt: bool = False) -> dict[str, Any]:
    """Public session dump. Receipt omitted unless ``include_receipt``."""
    payload: dict[str, Any] = {
        "session_id": self.session_id,
        "closed": self._closed,
    }
    if self._closed:
        payload["node_id"] = None
    elif self._node is not None:
        payload["node_id"] = self._node.id
        payload["node_label"] = self._node.label
    if include_receipt and self._receipt is not None:
        payload["receipt"] = self._receipt.to_dict()
    return payload

def to_json(self, *, include_receipt: bool = False) -> str:
    return json.dumps(self.to_dict(include_receipt=include_receipt), indent=2)

def __str__(self) -> str:
    if self._closed:
        return f"MirageSession(session_id={self.session_id}, closed=True, node=None)"
    nid = None if self._node is None else self._node.id
    return f"MirageSession(session_id={self.session_id}, node={nid})"

def __repr__(self) -> str:
    return str(self)

def end(self) -> None:
    """Destroy the session-to-node mapping. In-process forget, not a wipe.

    The frozen receipt snapshot remains so an operator who already
    asked to emit it can still write the JSON they requested. The
    mapping itself is dropped: ``_node`` is None and ``node`` raises.
    """
    self._node = None
    self._closed = True

def close(self) -> None:
    self.end()

def __enter__(self) -> "MirageSession":
    if self._node is None and not self._closed:
        self.initiate()
    return self

def __exit__(self, *exc: object) -> bool:
    self.end()
    return False

```

forgereceipts/engines/miragegrid/templates/ui.html

```

<!doctype html>
<html lang="en">
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>MirageGrid ? phone booth</title>
<style>
:root {
  --bg: #0e1116;
  --panel: #171c24;
  --ink: #e8edf4;
  --muted: #8b97a8;
  --accent: #7eb8ff;
  --pass: #6fdd8b;
  --fail: #ff7b7b;
  --line: #2a3340;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: ui-sans-serif, system-ui, -apple-system, "Segoe UI", sans-serif;
  background: var(--bg);
  color: var(--ink);
  line-height: 1.45;
}
header, main { max-width: 42rem; margin: 0 auto; padding: 1.5rem; }
header h1 { margin: 0 0 0.25rem; font-size: 1.6rem; letter-spacing: 0.02em; }
header p { margin: 0.25rem 0; color: var(--muted); }
.banner {
  border: 1px solid var(--line);
  background: var(--panel);
  padding: 0.75rem 1rem;
  border-radius: 8px;
  margin: 1rem 0 1.5rem;
  color: var(--muted);
  font-size: 0.95rem;
}
.row { display: flex; gap: 0.6rem; flex-wrap: wrap; margin: 0.75rem 0; }
button {
  background: var(--accent);
  color: #0b1220;
  border: 0;
  border-radius: 6px;
  padding: 0.55rem 0.9rem;
  font-weight: 650;
  cursor: pointer;
}
button.secondary { background: #2a3340; color: var(--ink); }
button.disabled { opacity: 0.45; cursor: not-allowed; }
.card {
  background: var(--panel);
  border: 1px solid var(--line);
  border-radius: 10px;
  padding: 1rem 1.1rem;
  margin: 1rem 0;
}
.k { color: var(--muted); font-size: 0.8rem; text-transform: uppercase; letter-spacing: 0.06em; }
.v { font-family: ui-monospace, SFMono-Regular, Menlo, monospace; font-size: 1.05rem; margin: 0.15rem 0 0.7rem; }
.pass { color: var(--pass); }
.fail { color: var(--fail); }
pre {
  background: #0b0f14;
  border: 1px solid var(--line);
  border-radius: 8px;
  padding: 0.8rem;
  overflow: auto;
  font-size: 0.85rem;
}
footer { color: var(--muted); font-size: 0.85rem; margin-top: 2rem; }
</style>
<body>
<header>
<h1>MirageGrid</h1>
<p>You enter. The system selects a booth. You leave with no persistent booth identity.</p>
<div class="banner">
  This UI assigns a <strong>logical</strong> node from a static pool of 25.
  It is not a VPN, proxy mesh, Tor client, or anonymity network.
  Mapping drop is in-process forget, not a log wipe.
  Bound to 127.0.0.1 only.
</div>
</header>
<main>
<div class="row">
  <button id="assign" type="button">Assign session</button>
  <button id="end" class="secondary" type="button" disabled>End session</button>
  <button id="toggle" class="secondary" type="button" disabled>Show receipt</button>
</div>
<div class="card" id="status">
<div class="k">Status</div>
<div class="v" id="status-text">No session. Assign a booth.</div>
<div class="k">Node</div>
<div class="v" id="node">?</div>
<div class="k">Session ID</div>

```

```

    <div class="v" id="sid">?</div>
    <div class="k">Live integrity</div>
    <div class="v" id="integrity">?</div>
  </div>
  <div class="card" id="receipt-card" hidden>
    <div class="k">Internal receipt (operator requested)</div>
    <pre id="receipt">{</pre>
  </div>
  <footer>
    Apache-2.0 ? Aziel Eliab ? 2026 ? forks welcome and always allowed.
  </footer>
</main>
<script>
  const $ = (id) => document.getElementById(id);
  let showReceipt = false;

  async function refresh() {
    const r = await fetch("/session");
    const s = await r.json();
    $("#sid").textContent = s.session_id || "?";
    $("#node").textContent = s.closed ? "(mapping destroyed)" : (s.node_id || "?");
    $("#integrity").textContent = s.live_integrity || "?";
    $("#integrity").className = "v " + (s.live_integrity === "PASS" ? "pass" : "fail");
    $("#status-text").textContent = s.closed
      ? "Session ended. Mapping destroyed."
      : (s.node_id ? "Booth assigned. Operate, then leave." : "No session. Assign a booth.");
    $("#end").disabled = !s.session_id || !!s.closed;
    $("#toggle").disabled = !s.session_id;
    if (showReceipt && s.receipt) {
      $("#receipt-card").hidden = false;
      $("#receipt").textContent = JSON.stringify(s.receipt, null, 2);
      $("#toggle").textContent = "Hide receipt";
    } else {
      $("#receipt-card").hidden = true;
      $("#toggle").textContent = "Show receipt";
    }
  }

  $("#assign").onclick = async () => {
    showReceipt = false;
    await fetch("/assign", { method: "POST" });
    await refresh();
  };
  $("#end").onclick = async () => {
    await fetch("/end", { method: "POST" });
    await refresh();
  };
  $("#toggle").onclick = () => {
    showReceipt = !showReceipt;
    refresh();
  };
  refresh();
</script>
</body>
</html>

```

forgereceipts/engines/miragegrid/ui.py

```

"""Localhost UI for the assignment + receipt engine.

Binds 127.0.0.1 only. Self-contained HTML/CSS, no CDN. Never opens
tunnels, never hops IPs, never speaks Tor.
"""

from __future__ import annotations

import json
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
from typing import Any
from urllib.parse import urlparse

from miragegrid.pool import NodePool
from miragegrid.session import MirageSession

DEFAULT_HOST = "127.0.0.1"
DEFAULT_PORT = 8080

_TEMPLATE = Path(__file__).resolve().parent / "templates" / "ui.html"

def _html() -> bytes:
    return _TEMPLATE.read_bytes()

class _State:
    def __init__(self) -> None:
        self.pool = NodePool()
        self.session: MirageSession | None = None

    def snapshot(self, include_receipt: bool = False) -> dict[str, Any]:
        if self.session is None:
            return {
                "session_id": None,
                "node_id": None,
                "closed": True,
                "live_integrity": "FAIL",
            }
        payload = self.session.to_dict(include_receipt=include_receipt)
        payload["live_integrity"] = self.session.integrity
        if include_receipt:
            try:
                payload["receipt"] = self.session.emit_receipt()
            except Exception:
                payload["receipt"] = None
        return payload

    def assign(self) -> dict[str, Any]:
        if self.session is not None and not self.session.closed:
            self.session.end()
        self.session = MirageSession(pool=self.pool)
        return self.snapshot(include_receipt=False)

    def end(self) -> dict[str, Any]:
        if self.session is not None and not self.session.closed:
            self.session.end()
        return self.snapshot(include_receipt=False)

def make_handler(state: _State):
    class Handler(BaseHTTPRequestHandler):
        def log_message(self, fmt: str, *args: object) -> None: # noqa: A003
            return

        def _json(self, body: dict[str, Any], status: int = 200) -> None:
            raw = json.dumps(body, indent=2).encode("utf-8")
            self.send_response(status)
            self.send_header("Content-Type", "application/json; charset=utf-8")
            self.send_header("Content-Length", str(len(raw)))
            self.send_header("Cache-Control", "no-store")
            self.end_headers()
            self.wfile.write(raw)

        def _html(self) -> None:
            raw = _html()
            self.send_response(200)
            self.send_header("Content-Type", "text/html; charset=utf-8")
            self.send_header("Content-Length", str(len(raw)))
            self.end_headers()
            self.wfile.write(raw)

        def do_GET(self) -> None: # noqa: N802
            path = urlparse(self.path).path
            if path in ("/", "/index.html"):
                self._html()
                return
            if path == "/session":
                include = "receipt=1" in (urlparse(self.path).query or "")
                # UI toggles receipt client-side from this payload when asked.
                snap = state.snapshot(include_receipt=True)

```

```

        self._json(snap)
        return
    if path == "/nodes":
        self._json({"nodes": [n.to_dict() for n in state.pool]})
        return
    if path == "/health":
        self._json({"ok": True, "bind_host": DEFAULT_HOST})
        return
    self._json({"error": "not found"}, 404)

def do_POST(self) -> None: # noqa: N802
    path = urlparse(self.path).path
    if path == "/assign":
        self._json(state.assign())
        return
    if path == "/end":
        self._json(state.end())
        return
    self._json({"error": "not found"}, 404)

return Handler

def serve(host: str = DEFAULT_HOST, port: int = DEFAULT_PORT) -> None:
    if host not in ("127.0.0.1", "localhost", ":", "1"):
        # Refuse non-loopback binds. This UI is a research tool.
        host = DEFAULT_HOST
    httpd = ThreadingHTTPServer((host, int(port)), make_handler(_State()))
    print(f"miragegrid ui http://{host}:{port}/ (loopback only; not a proxy)")
    try:
        httpd.serve_forever()
    except KeyboardInterrupt:
        print("\nmiragegrid ui stopped")
    finally:
        httpd.server_close()

```

forgereceipts/engines/shadowlock/__init__.py

```
"""ShadowLock: a read-only, zero-retention, software-agnostic outcome mirror.
```

```
July 2026 whitepaper implementation by Aziel Eliab.
```

```
Observes selectively. Computes counterfactual expectations. Reports  
financial variance. Forgets everything. Does not control, optimize,  
replace, dispatch, schedule, or learn.
```

```
Change is optional. Truth is not.
```

```
Forks are welcome and always allowed.
```

```
"""
```

```
from __future__ import annotations
```

```
from shadowlock.adapters import CsvAdapter, JsonlAdapter, MemoryAdapter
```

```
from shadowlock.counterfactual import Expectation
```

```
from shadowlock.envelope import JobEnvelope
```

```
from shadowlock.errors import (
```

```
    AirgapError,
```

```
    ReadOnlyError,
```

```
    SessionForgottenError,
```

```
)
```

```
from shadowlock.ledger import FinancialLedger
```

```
from shadowlock.report import Report
```

```
from shadowlock.sample import Sampler
```

```
from shadowlock.session import ShadowLockSession
```

```
__version__ = "0.1.0"
```

```
__author__ = "Azriel Eliab"
```

```
__all__ = [
```

```
    "AirgapError",
```

```
    "CsvAdapter",
```

```
    "Expectation",
```

```
    "FinancialLedger",
```

```
    "JobEnvelope",
```

```
    "JsonlAdapter",
```

```
    "MemoryAdapter",
```

```
    "ReadOnlyError",
```

```
    "Report",
```

```
    "Sampler",
```

```
    "SessionForgottenError",
```

```
    "ShadowLockSession",
```

```
    "__version__",
```

```
]
```

forgereceipts/engines/shadowlock/__main__.py

```
"""python -m shadowlock"""

from shadowlock.cli import main

if __name__ == "__main__":
    raise SystemExit(main())
```

forgereceipts/engines/shadowlock/adapters.py

```

"""Read-only host adapters. Map host records into anonymous JobEnvelopes.

Adapters expose only ``iter_jobs()`` / ``load``. Write, save, update,
dispatch, schedule, and modify raise ``ReadOnlyError``. Files are opened
read-only. Raw ids are hashed with the session salt; PII keys are dropped.
"""

from __future__ import annotations

import csv
import json
import os
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Iterator, Mapping, Sequence

from shadowlock.envelope import JobEnvelope, is_pii_key, sanitize_context
from shadowlock.errors import ReadOnlyError
from shadowlock.sample import identity_digest

_WRITE_NAMES = frozenset(
    {
        "write",
        "save",
        "update",
        "create",
        "delete",
        "dispatch",
        "schedule",
        "modify",
        "put",
        "post",
        "insert",
        "upsert",
        "replace",
        "remove",
        "control",
        "optimize",
        "learn",
    }
)

_ID_KEYS = ("id", "job_id", "jobId", "raw_id", "ticket", "ticket_id", "work_order")
_TS_KEYS = ("timestamp", "ts", "created_at", "time", "opened_at")
_CLASS_KEYS = ("task_class", "class", "type", "job_type", "category")
_URGENCY_KEYS = ("urgency", "priority")
_OUTCOME_KEYS = ("actual_outcome", "outcome", "status", "result")
_REVENUE_KEYS = ("actual_revenue", "revenue")
_COST_KEYS = ("actual_cost", "cost")
_DURATION_KEYS = ("actual_duration", "duration")
_CONTEXT_KEYS = ("context_signals", "context", "signals")

_CORE_CONSUMED = frozenset(
    _ID_KEYS
    + _TS_KEYS
    + _CLASS_KEYS
    + _URGENCY_KEYS
    + _OUTCOME_KEYS
    + _REVENUE_KEYS
    + _COST_KEYS
    + _DURATION_KEYS
    + _CONTEXT_KEYS
)

_URGENCY_ENUM = {
    "low": 0.25,
    "medium": 0.5,
    "med": 0.5,
    "high": 0.75,
    "critical": 1.0,
    "urgent": 0.9,
}

def _first(record: Mapping[str, Any], keys: Sequence[str], default: Any = None) -> Any:
    for k in keys:
        if k in record and record[k] not in (None, ""):
            return record[k]
    return default

def _as_float(value: Any, default: float | None = None) -> float | None:
    if value is None or value == "":
        return default
    try:
        return float(value)
    except (TypeError, ValueError):
        return default

def _as_urgency(value: Any) -> float:
    if value is None or value == "":

```



```

        return 0.5
    if isinstance(value, str):
        mapped = _URGENCY_ENUM.get(value.strip().lower())
        if mapped is not None:
            return mapped
        try:
            value = float(value)
        except ValueError:
            return 0.5
    n = float(value)
    if n > 1.0:
        # Host used a 1?5 (or 1?10) scale.
        if n <= 5:
            return max(0.0, min(1.0, n / 5.0))
        if n <= 10:
            return max(0.0, min(1.0, n / 10.0))
        return 1.0
    return max(0.0, min(1.0, n))

def _as_timestamp(value: Any) -> str:
    if value is None or value == "":
        return datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")
    if isinstance(value, (int, float)):
        ts = float(value)
        if ts > 1e12:
            ts = ts / 1000.0
        return datetime.fromtimestamp(ts, tz=timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")
    text = str(value).strip()
    return text

def record_to_envelope(record: Mapping[str, Any], salt: str) -> JobEnvelope:
    """Map a host record into an anonymous envelope. Drops raw id and PII."""
    raw_id = _first(record, _ID_KEYS, default=None)
    if raw_id is None:
        # Stable fallback from initiation fields so two adapters still agree
        # if both lack an id and the payload is the same.
        raw_id = json.dumps(
            {k: record[k] for k in sorted(record) if k in _CLASS_KEYS + _TS_KEYS},
            sort_keys=True,
            default=str,
        )
    digest = identity_digest(salt, raw_id)
    hid = digest.hex()[12:]
    sampled = int.from_bytes(digest[8:], "big") % 5 == 0

    ctx_raw = _first(record, _CONTEXT_KEYS, default={})
    if not isinstance(ctx_raw, Mapping):
        ctx_raw = {}
    ctx = dict(ctx_raw)
    for key, value in record.items():
        if key in _CORE_CONSUMED or is_pii_key(str(key)):
            continue
        if str(key) not in ctx:
            ctx[str(key)] = value
    ctx = sanitize_context(ctx)

    return JobEnvelope(
        hashed_id=hid,
        timestamp=_as_timestamp(_first(record, _TS_KEYS)),
        task_class=str(_first(record, _CLASS_KEYS, default="unknown")),
        urgency=_as_urgency(_first(record, _URGENCY_KEYS, default=0.5)),
        context_signals=ctx,
        actual_outcome=(
            None
            if _first(record, _OUTCOME_KEYS) is None
            else str(_first(record, _OUTCOME_KEYS))
        ),
        actual_revenue=_as_float(_first(record, _REVENUE_KEYS)),
        actual_cost=_as_float(_first(record, _COST_KEYS)),
        actual_duration=_as_float(_first(record, _DURATION_KEYS)),
        sampled=sampled,
    )

class ReadOnlyAdapter:
    """Mixin: any write/control attribute raises ReadOnlyError."""

    def __getattr__(self, name: str) -> Any:
        bare = name.lower().lstrip("_")
        if name.lower() in _WRITE_NAMES or bare in _WRITE_NAMES:
            raise ReadOnlyError(
                f"ShadowLock adapters are read-only; {name!r} is not allowed. "
                "This mirror does not write, control, schedule, or modify "
                "external systems."
            )
        raise AttributeError(f"{type(self).__name__!r} has no attribute {name!r}")

    def load(self, salt: str) -> list[JobEnvelope]:
        """Read-only load of every mapped envelope (ids already hashed)."""
        return list(self.iter_jobs(salt))

    def iter_jobs(self, salt: str) -> Iterator[JobEnvelope]: # pragma: no cover - interface
        raise NotImplementedError

```

```

class MemoryAdapter(ReadOnlyAdapter):
    """In-memory list of host records. Does not persist. Does not write out."""

    def __init__(self, records: Sequence[Mapping[str, Any]]):
        self._records = [dict(r) for r in records]

    def iter_jobs(self, salt: str) -> Iterator[JobEnvelope]:
        for rec in self._records:
            yield record_to_envelope(rec, salt)

def _open_readonly(path: Path):
    """Open a text file read-only (O_RDONLY on POSIX)."""
    fd = os.open(path, os.O_RDONLY)
    return os.fdopen(fd, "r", encoding="utf-8")

class JsonlAdapter(ReadOnlyAdapter):
    """Read-only JSONL file adapter. One JSON object per line."""

    def __init__(self, path: str | Path):
        self.path = Path(path)

    def iter_jobs(self, salt: str) -> Iterator[JobEnvelope]:
        with _open_readonly(self.path) as fh:
            for line in fh:
                line = line.strip()
                if not line:
                    continue
                rec = json.loads(line)
                if not isinstance(rec, Mapping):
                    continue
                yield record_to_envelope(rec, salt)

class CsvAdapter(ReadOnlyAdapter):
    """Read-only CSV file adapter. Header row required."""

    def __init__(self, path: str | Path):
        self.path = Path(path)

    def iter_jobs(self, salt: str) -> Iterator[JobEnvelope]:
        with _open_readonly(self.path) as fh:
            reader = csv.DictReader(fh)
            for rec in reader:
                yield record_to_envelope(rec, salt)

```

forgereceipts/engines/shadowlock/cli.py

```

"""Command-line interface for ShadowLock.

shadowlock version
shadowlock observe --in jobs.jsonl --format jsonl|csv --out report.json
shadowlock observe --in jobs.jsonl --stdout

``--out`` writes the anonymous summary JSON only (aggregates, hashed ids).
Input files are opened read-only. Optional ``--airgap`` refuses proxy env vars.
"""

from __future__ import annotations

import argparse
import sys
from pathlib import Path
from typing import Sequence

from shadowlock import __version__
from shadowlock.adapters import CsvAdapter, JsonlAdapter
from shadowlock.errors import AirgapError, SessionForgottenError
from shadowlock.session import ShadowLockSession, assert_airgap

def _build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="shadowlock",
        description=(
            "ShadowLock ? a read-only, zero-retention outcome mirror "
            "(Aziel Eliab, July 2026). Observes; does not control. "
            "Change is optional. Truth is not."
        ),
    )
    sub = parser.add_subparsers(dest="cmd", required=True)

    sub.add_parser("version", help="Print package version.")

    p_obs = sub.add_parser(
        "observe",
        help="Read a job file, sample 1 in 5, print or write an anonymous report.",
    )
    p_obs.add_argument("--in", dest="inp", required=True, help="Input JSONL or CSV file.")
    p_obs.add_argument(
        "--format",
        choices=("jsonl", "csv"),
        default=None,
        help="Input format (default: infer from extension).",
    )
    p_obs.add_argument(
        "--out",
        dest="out",
        default=None,
        help="Write anonymous summary JSON (aggregates, no raw ids/names).",
    )
    p_obs.add_argument(
        "--stdout",
        action="store_true",
        help="Print the anonymous summary JSON to stdout.",
    )
    p_obs.add_argument(
        "--salt",
        default=None,
        help="Optional session salt (hex/string). Default: random, not retained.",
    )
    p_obs.add_argument(
        "--airgap",
        action="store_true",
        help="Refuse to run if HTTP(S)_PROXY / ALL_PROXY (or lowercase) are set.",
    )
    return parser

def _adapter_for(path: Path, fmt: str | None):
    kind = fmt
    if kind is None:
        suf = path.suffix.lower()
        if suf in {".csv"}:
            kind = "csv"
        else:
            kind = "jsonl"
    if kind == "csv":
        return CsvAdapter(path)
    return JsonlAdapter(path)

def main(argv: Sequence[str] | None = None) -> int:
    args = _build_parser().parse_args(argv)

    if args.cmd == "version":
        sys.stdout.write(f"shadowlock {__version__}\n")
        return 0

    if args.cmd == "observe":

```

```
    if args.airgap:
        try:
            assert_airgap()
        except AirgapError as exc:
            sys.stderr.write(f"shadowlock: {exc}\n")
            return 2
    if not args.out and not args.stdout:
        sys.stderr.write("shadowlock: pass --out FILE or --stdout\n")
        return 2
    path = Path(args.inp)
    if not path.is_file():
        sys.stderr.write(f"shadowlock: input not found: {path}\n")
        return 2
    adapter = _adapter_for(path, args.format)
    try:
        with ShadowLockSession(salt=args.salt, airgap=args.airgap) as session:
            report = session.observe(adapter)
            payload = report.to_json()
    except (AirgapError, SessionForgottenError) as exc:
        sys.stderr.write(f"shadowlock: {exc}\n")
        return 2
    if args.stdout:
        sys.stdout.write(payload + "\n")
    if args.out:
        out_path = Path(args.out)
        out_path.write_text(payload + "\n", encoding="utf-8")
    return 0

return 2

if __name__ == "__main__": # pragma: no cover
    raise SystemExit(main())
```

forgereceipts/engines/shadowlock/counterfactual.py

```

"""Class-conditional empirical prior. No ML. Session-local only.

Expectation is computed from initiation fields only (task_class, urgency,
context_signals) plus a prior built from *previously sampled jobs in this
same in-memory session*. First jobs in a class use optional ``class_priors``
or a conservative default range.

Changing a job's actuals must not change the expectation, only the delta.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from statistics import median
from typing import Any, Mapping, Sequence

# Optional conservative ranges an operator may pass as class_priors.
# When neither history nor class_priors exist, the field is 0-width unknown
# (expected is None; that field does not enter the ledger for that job).
DEFAULT_DURATION_RANGE = (0.0, 1_000_000.0)
DEFAULT_COST_RANGE = (0.0, 1_000_000_000.0)
DEFAULT_REVENUE_RANGE = (0.0, 1_000_000_000.0)

def _mid(lo: float, hi: float) -> float:
    return (lo + hi) / 2.0

def _range_of(values: Sequence[float], fallback: tuple[float, float]) -> tuple[float, float, float]:
    if not values:
        lo, hi = fallback
        return lo, _mid(lo, hi), hi
    lo = min(values)
    hi = max(values)
    return lo, float(median(values)), hi

@dataclass(frozen=True)
class Expectation:
    """Counterfactual envelope at initiation. Actuals are not inputs."""

    duration: float | None
    duration_low: float | None
    duration_high: float | None
    cost: float | None
    cost_low: float | None
    cost_high: float | None
    revenue: float | None
    revenue_low: float | None
    revenue_high: float | None
    unknown: bool = False
    task_class: str = ""
    urgency: float = 0.5

    @classmethod
    def compute(
        cls,
        *,
        task_class: str,
        urgency: float,
        context_signals: Mapping[str, Any] | None = None,
        history: Sequence[Mapping[str, Any]] | None = None,
        class_priors: Mapping[str, Mapping[str, Any]] | None = None,
    ) -> "Expectation":
        """Build an expectation from initiation fields + session history.

        ``history`` items are prior sampled jobs of *any* class; only
        matching ``task_class`` actuals are used. ``context_signals`` is
        accepted (it is an initiation field) and currently used as an
        opaque grouping hint ? the baseline is class-conditional median.
        Urgency slightly shortens expected duration (higher urgency ?
        less expected time) without looking at actuals.
        """
        del context_signals # initiation field reserved; baseline is class-conditional
        history = list(history or [])
        same = [h for h in history if str(h.get("task_class")) == str(task_class)]

        durations = [
            float(h["actual_duration"])
            for h in same
            if h.get("actual_duration") is not None
        ]
        costs = [
            float(h["actual_cost"])
            for h in same
            if h.get("actual_cost") is not None
        ]
        revenues = [
            float(h["actual_revenue"])
            for h in same
            if h.get("actual_revenue") is not None
        ]

```

```

prior = (class_priors or {}).get(task_class, {})

def _resolve(
    values: list[float],
    prior_key: str,
    fallback: tuple[float, float],
) -> tuple[float | None, float | None, float | None, bool]:
    if values:
        lo, mid, hi = _range_of(values, fallback)
        return lo, mid, hi, False
    spec = prior.get(prior_key)
    if spec is not None:
        if isinstance(spec, (tuple, list)) and len(spec) >= 2:
            lo, hi = float(spec[0]), float(spec[1])
            return lo, _mid(lo, hi), hi, False
        if isinstance(spec, dict) and "low" in spec and "high" in spec:
            lo, hi = float(spec["low"]), float(spec["high"])
            return lo, _mid(lo, hi), hi, False
        # 0-width unknown: no invented midpoint. Ledger skips this field.
        return None, None, None, True

d_lo, d_mid, d_hi, d_unk = _resolve(durations, "duration", DEFAULT_DURATION_RANGE)
c_lo, c_mid, c_hi, c_unk = _resolve(costs, "cost", DEFAULT_COST_RANGE)
r_lo, r_mid, r_hi, r_unk = _resolve(revenues, "revenue", DEFAULT_REVENUE_RANGE)

# Urgency adjustment on duration only: urgency 0 ? 1.1?, urgency 1 ? 0.9?.
if d_mid is not None:
    scale = 1.1 - 0.2 * max(0.0, min(1.0, float(urgency)))
    d_mid = d_mid * scale
    if d_lo is not None:
        d_lo = d_lo * scale
    if d_hi is not None:
        d_hi = d_hi * scale

unknown = d_unk and c_unk and r_unk and not same
return cls(
    duration=d_mid,
    duration_low=d_lo,
    duration_high=d_hi,
    cost=c_mid,
    cost_low=c_lo,
    cost_high=c_hi,
    revenue=r_mid,
    revenue_low=r_lo,
    revenue_high=r_hi,
    unknown=unknown,
    task_class=str(task_class),
    urgency=float(urgency),
)

@dataclass
class SessionPrior:
    """In-memory class-conditional prior. Cleared on forget()."""

    class_priors: dict[str, Mapping[str, Any]] = field(default_factory=dict)
    _rows: list[dict[str, Any]] = field(default_factory=list)

    def expect(
        self,
        *,
        task_class: str,
        urgency: float,
        context_signals: Mapping[str, Any] | None = None,
    ) -> Expectation:
        return Expectation.compute(
            task_class=task_class,
            urgency=urgency,
            context_signals=context_signals,
            history=self._rows,
            class_priors=self.class_priors,
        )

    def update(self, envelope_as_dict: Mapping[str, Any]) -> None:
        """Append this job's actuals *after* its expectation was computed."""
        self._rows.append(dict(envelope_as_dict))

    def clear(self) -> None:
        self._rows.clear()

```

forgereceipts/engines/shadowlock/envelope.py

```

"""Anonymous JobEnvelope. Raw identifiers are hashed on ingest; PII is dropped."""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any, Mapping

# Keys that must never land on an envelope or in a report.
PII_KEYS = frozenset(
    {
        "name",
        "full_name",
        "first_name",
        "last_name",
        "email",
        "phone",
        "phone_number",
        "mobile",
        "person",
        "person_name",
        "team",
        "team_name",
        "department",
        "department_name",
        "technician",
        "technician_name",
        "worker",
        "worker_name",
        "assignee",
        "assignee_name",
        "employee",
        "employee_name",
        "username",
        "ssn",
        "address",
        "operator",
        "operator_name",
    }
)

def is_pii_key(key: str) -> bool:
    k = key.strip().lower().replace("-", "_")
    if k in PII_KEYS:
        return True
    # Catch host fields like customer_email, user_phone.
    for token in ("email", "phone", "ssn", "fullname"):
        if token in k:
            return True
    if k.endswith("_name") or k.endswith("name"):
        return True
    if k in {"team", "department", "person"}:
        return True
    return False

def sanitize_context(signals: Mapping[str, Any] | None) -> dict[str, Any]:
    """Keep numeric/categorical host fields; drop PII keys and odd types."""
    out: dict[str, Any] = {}
    if not signals:
        return out
    for key, value in signals.items():
        if is_pii_key(str(key)):
            continue
        if isinstance(value, bool):
            out[str(key)] = value
        elif isinstance(value, (int, float)) and not isinstance(value, bool):
            out[str(key)] = value
        elif isinstance(value, str):
            # Categorical already-available field. Still drop if it *looks* like an email.
            if "@" in value and "." in value.split("@")[-1]:
                continue
            out[str(key)] = value
        # Nested / other types are dropped (not required by the spec).
    return out

@dataclass(frozen=True)
class JobEnvelope:
    """Anonymous operational record. No raw id, no person/team/department names.

    Fields are the host-available initiation set plus close-out actuals.
    ``hashed_id`` is ``sha256(salt || raw_id).hex()[12]``.
    ``sampled`` is an internal 1-in-5 flag from the same digest; it is never
    emitted in reports.
    """

    hashed_id: str
    timestamp: str
    task_class: str
    urgency: float

```

```
context_signals: dict[str, Any] = field(default_factory=dict)
actual_outcome: str | None = None
actual_revenue: float | None = None
actual_cost: float | None = None
actual_duration: float | None = None
sampled: bool = field(default=False, compare=False, repr=False)

def initiation_fields(self) -> dict[str, Any]:
    """Fields the host already had at initiation. No actuals."""
    return {
        "task_class": self.task_class,
        "urgency": self.urgency,
        "context_signals": dict(self.context_signals),
    }
```


forgereceipts/engines/shadowlock/errors.py

```
"""ShadowLock errors."""

from __future__ import annotations

class ShadowLockError(Exception):
    """Base error."""

class ReadOnlyError(ShadowLockError):
    """Raised when a write/control method is requested on an adapter."""

class SessionForgottenError(ShadowLockError):
    """Raised when observe/report is requested after forget()."""

class AirgapError(ShadowLockError):
    """Raised when --airgap is set but proxy environment variables are present."""
```

forgereceipts/engines/shadowlock/ledger.py

```

"""Financial ledger over sampled close-outs.

    money_made          sum of positive revenue deltas
    money_lost          sum of negative revenue deltas (as +$) plus cost overruns
    money_left_on_table expected revenue high-end minus actual, when actual < expected
    net_variance        money_made - money_lost
    efficiency_score     0?1 mean of per-job time/cost/revenue efficiency (clipped)

Opportunity cost (money_left_on_table) is reported separately and is not
subtracted again inside net_variance.
"""

from __future__ import annotations

from dataclasses import dataclass, field

from shadowlock.counterfactual import Expectation
from shadowlock.envelope import JobEnvelope

def _clip01(value: float) -> float:
    if value < 0.0:
        return 0.0
    if value > 1.0:
        return 1.0
    return value

def _job_efficiency(env: JobEnvelope, exp: Expectation) -> float | None:
    parts: list[float] = []
    if env.actual_duration and exp.duration and env.actual_duration > 0:
        parts.append(_clip01(exp.duration / env.actual_duration))
    if env.actual_cost and exp.cost and env.actual_cost > 0:
        parts.append(_clip01(exp.cost / env.actual_cost))
    if env.actual_revenue is not None and exp.revenue and exp.revenue > 0:
        parts.append(_clip01(env.actual_revenue / exp.revenue))
    if not parts:
        return None
    return sum(parts) / len(parts)

@dataclass
class FinancialLedger:
    money_made: float = 0.0
    money_lost: float = 0.0
    money_left_on_table: float = 0.0
    net_variance: float = 0.0
    efficiency_score: float = 0.0
    _efficiencies: list[float] = field(default_factory=list, repr=False)

    def add(self, env: JobEnvelope, exp: Expectation) -> None:
        actual_rev = env.actual_revenue
        expected_rev = exp.revenue
        if actual_rev is not None and expected_rev is not None:
            delta = actual_rev - expected_rev
            if delta > 0:
                self.money_made += delta
            elif delta < 0:
                self.money_lost += -delta
            if actual_rev < expected_rev:
                high = exp.revenue_high if exp.revenue_high is not None else expected_rev
                self.money_left_on_table += max(0.0, high - actual_rev)

        actual_cost = env.actual_cost
        expected_cost = exp.cost
        if actual_cost is not None and expected_cost is not None:
            overrun = actual_cost - expected_cost
            if overrun > 0:
                self.money_lost += overrun

        eff = _job_efficiency(env, exp)
        if eff is not None:
            self._efficiencies.append(eff)

        self.net_variance = self.money_made - self.money_lost
        if self._efficiencies:
            self.efficiency_score = sum(self._efficiencies) / len(self._efficiencies)
        else:
            self.efficiency_score = 0.0

    def to_dict(self) -> dict[str, float]:
        return {
            "money_made": round(self.money_made, 6),
            "money_lost": round(self.money_lost, 6),
            "money_left_on_table": round(self.money_left_on_table, 6),
            "net_variance": round(self.net_variance, 6),
            "efficiency_score": round(self.efficiency_score, 6),
        }

```

forgereceipts/engines/shadowlock/report.py

```

"""Anonymous aggregate report. Hashed ids only. No person/team/department names."""

from __future__ import annotations

import json
from dataclasses import dataclass, field
from typing import Any

from shadowlock.ledger import FinancialLedger

@dataclass
class Report:
    observed: int = 0
    sampled: int = 0
    sample_rate: float = 0.2
    hashed_ids: list[str] = field(default_factory=list)
    ledger: FinancialLedger = field(default_factory=FinancialLedger)
    by_task_class: dict[str, int] = field(default_factory=dict)
    notes: list[str] = field(
        default_factory=lambda: [
            "ShadowLock reports are anonymous aggregates.",
            "Identifiers are sha256 hex[:12] only.",
            "No person, team, or department names are emitted.",
        ]
    )

    def to_dict(self) -> dict[str, Any]:
        return {
            "observed": self.observed,
            "sampled": self.sampled,
            "sample_rate_target": self.sample_rate,
            "sampled_hashed_ids": list(self.hashed_ids),
            "ledger": self.ledger.to_dict(),
            "by_task_class": dict(self.by_task_class),
            "notes": list(self.notes),
        }

    def to_json(self, *, indent: int = 2) -> str:
        return json.dumps(self.to_dict(), indent=indent, sort_keys=True)

```

forgereceipts/engines/shadowlock/sample.py

```

"""Opaque 1-in-5 sampler.

Exact 1-in-5 is the *rate*, realized via an opaque hash of a session-local
salt concatenated with the identifier:

    int.from_bytes(sha256(salt || id)[:8], "big") % 5 == 0

Internally deterministic: same salt + same id always yields the same
decision. Externally opaque: the salt is session-local and is not a
round-robin counter. Across many distinct ids the sampled count sits
near 20% (hash distribution, not an exact quota on an open stream).
"""

from __future__ import annotations

import hashlib
from typing import Union

IdLike = Union[str, bytes, int]

SAMPLE_MODULUS = 5 # 1 in 5
HASHED_ID_HEX_LEN = 12

def _as_bytes(value: IdLike) -> bytes:
    if isinstance(value, bytes):
        return value
    return str(value).encode("utf-8")

def identity_digest(salt: str | bytes, raw_id: IdLike) -> bytes:
    """SHA-256(salt || id). Salt and id are concatenated as bytes."""
    salt_b = salt if isinstance(salt, bytes) else str(salt).encode("utf-8")
    return hashlib.sha256(salt_b + _as_bytes(raw_id)).digest()

def hashed_id_for(salt: str | bytes, raw_id: IdLike) -> str:
    """Truncated identity hash used in reports: sha256 hex[:12]."""
    return identity_digest(salt, raw_id).hex()[ :HASHED_ID_HEX_LEN]

class Sampler:
    """Session-local 1-in-5 sampler.

    Pass the *raw* host id when it is still in the adapter. After ingest
    the envelope only carries ``hashed_id``; sampling of a closed set of
    raw ids uses this class directly.
    """

    modulus = SAMPLE_MODULUS

    def __init__(self, salt: str | bytes):
        self.salt = salt if isinstance(salt, str) else salt.decode("utf-8")

    def digest(self, raw_id: IdLike) -> bytes:
        return identity_digest(self.salt, raw_id)

    def hashed_id(self, raw_id: IdLike) -> str:
        return hashed_id_for(self.salt, raw_id)

    def should_sample(self, raw_id: IdLike) -> bool:
        return int.from_bytes(self.digest(raw_id)[:8], "big") % SAMPLE_MODULUS == 0

```

forgereceipts/engines/shadowlock/session.py

```

"""ShadowLockSession ? the mirror.

Holds envelopes in memory until ``forget()`` or context-manager exit.
Does not create a data directory, sqlite store, or ``.shadowlock`` path.
Does not write to host systems.
"""

from __future__ import annotations

import os
import secrets
from typing import Any, Mapping

from shadowlock.adapters import ReadOnlyAdapter
from shadowlock.counterfactual import SessionPrior
from shadowlock.envelope import JobEnvelope
from shadowlock.errors import AirgapError, SessionForgottenError
from shadowlock.ledger import FinancialLedger
from shadowlock.report import Report
from shadowlock.sample import Sampler

PROXY_ENV_VARS = (
    "HTTP_PROXY",
    "HTTPS_PROXY",
    "ALL_PROXY",
    "FTP_PROXY",
    "http_proxy",
    "https_proxy",
    "all_proxy",
    "ftp_proxy",
)

def assert_airgap() -> None:
    present = [k for k in PROXY_ENV_VARS if os.environ.get(k)]
    if present:
        raise AirgapError(
            "air-gap requested but proxy environment variables are set: "
            + ", ".join(present)
        )

class ShadowLockSession:
    """In-memory outcome mirror.

    After ``forget()`` (also invoked on context-manager exit), held job
    payloads are dropped. A subsequent ``observe()`` raises
    ``SessionForgottenError``. A ``Report`` already returned to the caller
    is a snapshot of anonymous aggregates (hashed ids only) and is not
    retained by the session.
    """

    def __init__(
        self,
        salt: str | None = None,
        class_priors: Mapping[str, Mapping[str, Any]] | None = None,
        airgap: bool = False,
    ):
        if airgap:
            assert_airgap()
        self.salt = salt if salt is not None else secrets.token_hex(16)
        self.sampler = Sampler(self.salt)
        self._prior = SessionPrior(class_priors=dict(class_priors or {}))
        self._envelopes: list[JobEnvelope] = []
        self._forgotten = False
        self._airgap = airgap

    def observe(self, adapter: ReadOnlyAdapter) -> Report:
        """Read the adapter, sample 1 in 5, compute counterfactuals, return a report.

        The adapter is used only via ``iter_jobs`` / ``load``. Nothing is
        written back.
        """
        if self._forgotten:
            raise SessionForgottenError(
                "session has forgotten its payloads; start a new ShadowLockSession"
            )
        if self._airgap:
            assert_airgap()

        observed = 0
        sampled_envs: list[JobEnvelope] = []
        ledger = FinancialLedger()
        by_class: dict[str, int] = {}

        for env in adapter.iter_jobs(self.salt):
            observed += 1
            # 1-in-5 from the same digest as hashed_id: sha256(salt||raw_id).
            # The raw id is already gone; ``env.sampled`` is the opaque bit.
            if not env.sampled:
                continue
            sampled_envs.append(env)

```

```

        exp = self._prior.expect(
            task_class=env.task_class,
            urgency=env.urgency,
            context_signals=env.context_signals,
        )
        ledger.add(env, exp)
        self._prior.update(
            {
                "task_class": env.task_class,
                "actual_duration": env.actual_duration,
                "actual_cost": env.actual_cost,
                "actual_revenue": env.actual_revenue,
            }
        )
        by_class[env.task_class] = by_class.get(env.task_class, 0) + 1

    self._envelopes.extend(sampled_envs)
    return Report(
        observed=observed,
        sampled=len(sampled_envs),
        sample_rate=0.2,
        hashed_ids=[e.hashed_id for e in sampled_envs],
        ledger=ledger,
        by_task_class=by_class,
    )

def forget(self) -> None:
    """Drop every held envelope and the in-memory class prior. Irreversible."""
    self._envelopes.clear()
    self._prior.clear()
    self._forgotten = True

@property
def forgotten(self) -> bool:
    return self._forgotten

def held_payload_count(self) -> int:
    """Number of sampled envelopes still in memory. 0 after forget()."""
    return len(self._envelopes)

def __enter__(self) -> "ShadowLockSession":
    return self

def __exit__(self, *exc: object) -> bool:
    self.forget()
    return False

```

forgereceipts/engines/staticclock/__init__.py

```
"""StaticClock: a chrono-linguistic release advisory system.

August 2026 whitepaper implementation by Aziel Eliab.

Software-only advisory. Least-distorting, most analytically stable
release window for information based on timezone, language, and
regional dialect. Does not optimize for reach, virality, or engagement.
No memory, identity, or historical data. One advisory for one moment,
then forget.

It does not help messages travel farther. It helps them arrive intact.

Forks are welcome and always allowed.
"""

from __future__ import annotations

from staticclock.engine import OUTPUT_FIELDS, Advisory, StaticClock
from staticclock.zones import list_timezones

__version__ = "0.1.0"
__author__ = "Azriel Eliab"
__all__ = [
    "Advisory",
    "OUTPUT_FIELDS",
    "StaticClock",
    "list_timezones",
    "__version__",
]
```

forgereceipts/engines/staticclock/__main__.py

```
"""Allow ``python -m staticclock`` to invoke the CLI."""

from staticclock.cli import main

if __name__ == "__main__":
    raise SystemExit(main())
```


forgereceipts/engines/staticclock/anchors.py

```

"""Top-30 geographic anchors and free-text geo resolution.

Unknown input maps to the nearest anchor (alias, then fuzzy, then
default). No user identification. Resolution is a function of the
string, not of a profile.
"""

from __future__ import annotations

import difflib
import re
import unicodedata

from staticclock.index import anchors as _anchor_map

TOP_30: tuple[str, ...] = (
    "United States",
    "United Kingdom",
    "Germany",
    "France",
    "Spain",
    "Italy",
    "Brazil",
    "Mexico",
    "Canada",
    "India",
    "China",
    "Japan",
    "South Korea",
    "Australia",
    "New Zealand",
    "South Africa",
    "Nigeria",
    "Egypt",
    "Israel",
    "Turkey",
    "Russia",
    "Ukraine",
    "Poland",
    "Netherlands",
    "Sweden",
    "Norway",
    "Finland",
    "Argentina",
    "Chile",
    "Saudi Arabia",
)

DEFAULT_ANCHOR = "United States"

# US states and DC resolve to the United States anchor.
_US_STATES: tuple[str, ...] = (
    "alabama", "alaska", "arizona", "arkansas", "california", "colorado",
    "connecticut", "delaware", "florida", "georgia", "hawaii", "idaho",
    "illinois", "indiana", "iowa", "kansas", "kentucky", "louisiana",
    "maine", "maryland", "massachusetts", "michigan", "minnesota",
    "mississippi", "missouri", "montana", "nebraska", "nevada",
    "new hampshire", "new jersey", "new mexico", "new york",
    "north carolina", "north dakota", "ohio", "oklahoma", "oregon",
    "pennsylvania", "rhode island", "south carolina", "south dakota",
    "tennessee", "texas", "utah", "vermont", "virginia", "washington",
    "west virginia", "wisconsin", "wyoming", "district of columbia",
    "washington dc", "washington d.c.", "dc",
)

_ALIASES: dict[str, str] = {
    "usa": "United States",
    "us": "United States",
    "u.s.": "United States",
    "u.s.a.": "United States",
    "america": "United States",
    "united states of america": "United States",
    "indianapolis": "United States",
    "chicago": "United States",
    "new york city": "United States",
    "nyc": "United States",
    "los angeles": "United States",
    "uk": "United Kingdom",
    "u.k.": "United Kingdom",
    "britain": "United Kingdom",
    "great britain": "United Kingdom",
    "england": "United Kingdom",
    "scotland": "United Kingdom",
    "wales": "United Kingdom",
    "northern ireland": "United Kingdom",
    "gb": "United Kingdom",
    "london": "United Kingdom",
    "deutschland": "Germany",
    "berlin": "Germany",
    "paris": "France",
    "madrid": "Spain",
    "rome": "Italy",
}

```

```

"brasil": "Brazil",
"sao paulo": "Brazil",
"s?o paulo": "Brazil",
"mexico city": "Mexico",
"m?xico": "Mexico",
"toronto": "Canada",
"bharat": "India",
"hindustan": "India",
"prc": "China",
"people's republic of china": "China",
"peoples republic of china": "China",
"nippon": "Japan",
"nihon": "Japan",
"tokyo": "Japan",
"korea": "South Korea",
"republic of korea": "South Korea",
"rok": "South Korea",
"seoul": "South Korea",
"sydney": "Australia",
"auckland": "New Zealand",
"aotearoa": "New Zealand",
"rsa": "South Africa",
"johannesburg": "South Africa",
"lagos": "Nigeria",
"cairo": "Egypt",
"tel aviv": "Israel",
"jerusalem": "Israel",
"turkiye": "Turkey",
"t?rkiye": "Turkey",
"istanbul": "Turkey",
"moscow": "Russia",
"kyiv": "Ukraine",
"kiev": "Ukraine",
"warsaw": "Poland",
"holland": "Netherlands",
"amsterdam": "Netherlands",
"stockholm": "Sweden",
"oslo": "Norway",
"helsinki": "Finland",
"buenos aires": "Argentina",
"santiago": "Chile",
"ksa": "Saudi Arabia",
"saudi": "Saudi Arabia",
"riyadh": "Saudi Arabia",
}

for _state in _US_STATES:
    _ALIASES[_state] = "United States"

def _fold(text: str) -> str:
    nfkd = unicodedata.normalize("NFKD", text or "")
    stripped = "".join(ch for ch in nfkd if not unicodedata.combining(ch))
    return re.sub(r"^[a-z0-9]+", " ", stripped.casefold()).strip()

def _anchor_lookup() -> dict[str, str]:
    table: dict[str, str] = {}
    for name in TOP_30:
        table[_fold(name)] = name
    table.update(_ALIASES)
    return table

def resolve_geo(geo: str) -> str:
    """Map free text to a Top-30 anchor. Never raises on unknown input."""
    folded = _fold(geo)
    if not folded:
        return DEFAULT_ANCHOR
    lookup = _anchor_lookup()
    if folded in lookup:
        return lookup[folded]
    candidates = list(lookup.keys())
    match = difflib.get_close_matches(folded, candidates, n=1, cutoff=0.72)
    if match:
        return lookup[match[0]]
    # Try last token (e.g. "somewhere in Indiana").
    tokens = folded.split()
    if len(tokens) > 1:
        for token in reversed(tokens):
            if token in lookup:
                return lookup[token]
        match = difflib.get_close_matches(tokens[-1], candidates, n=1, cutoff=0.8)
        if match:
            return lookup[match[0]]
    return DEFAULT_ANCHOR

def is_anchor(name: str) -> bool:
    return name in _anchor_map()

def basket_of(anchor: str) -> list[str]:
    rec = _anchor_map()[anchor]
    return list(rec["basket"])

```

forgereceipts/engines/staticclock/chronolect.py

```

"""Chronolect: analytical clock windows.

Default analytical window is 08:30?10:30 local. v0.1 documents three
regional overrides for later cultural morning starts. All other anchors
use the default. The picked clock time sits inside the window. This is
not a scheduler and does not optimize for reach.
"""

from __future__ import annotations

from datetime import date, datetime, time, timedelta
from zoneinfo import ZoneInfo

from staticclock.polarize import pick_index

DEFAULT_WINDOW: tuple[str, str] = ("08:30", "10:30")

# Documented overrides (v0.1: three). Rest of the Top-30 use DEFAULT_WINDOW.
OVERRIDES: dict[str, tuple[str, str]] = {
    "Spain": ("09:30", "11:30"), # later Mediterranean morning
    "Argentina": ("09:30", "11:30"), # Rioplatense later cultural start
    "Egypt": ("09:00", "11:00"), # later administrative morning
}

def parse_hhmm(text: str) -> time:
    hour_s, minute_s = text.split(":")
    return time(int(hour_s), int(minute_s))

def window_for(region: str) -> tuple[str, str]:
    return OVERRIDES.get(region, DEFAULT_WINDOW)

def slots_in(window: tuple[str, str], step_minutes: int = 15) -> list[str]:
    start = parse_hhmm(window[0])
    end = parse_hhmm(window[1])
    start_m = start.hour * 60 + start.minute
    end_m = end.hour * 60 + end.minute
    out: list[str] = []
    m = start_m
    while m <= end_m:
        out.append(f"{m // 60:02d}:{m % 60:02d}")
        m += step_minutes
    return out

def pick_time(region: str, nonce: bytes) -> str:
    window = window_for(region)
    slots = slots_in(window)
    return slots[pick_index(len(slots), nonce, b"time")]

def local_date(iana: str, when: datetime | None = None) -> date:
    tz = ZoneInfo(iana)
    now = when if when is not None else datetime.now(tz)
    if now.tzinfo is None:
        now = now.replace(tzinfo=tz)
    else:
        now = now.astimezone(tz)
    return now.date()

def in_window(hhmm: str, window: tuple[str, str]) -> bool:
    t = parse_hhmm(hhmm)
    start = parse_hhmm(window[0])
    end = parse_hhmm(window[1])
    return start <= t <= end

```

forgereceipts/engines/staticclock/cli.py

```

"""Command-line interface for StaticClock.

    staticclock version
    staticclock anchors
    staticclock advise --geo "United States"
    staticclock advise --geo "Indiana"
    staticclock advise --geo "United States" --json
    staticclock zones
    staticclock ui
    staticclock serve

Advisory only. Five fields. No reasoning dump. Forks always allowed.
"""

from __future__ import annotations

import argparse
import json
import sys
from typing import Sequence

from staticclock import __version__
from staticclock.anchors import TOP_30
from staticclock.engine import OUTPUT_FIELDS, StaticClock
from staticclock.zones import list_timezones

def _build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="staticclock",
        description=(
            "StaticClock ? chrono-linguistic release advisory "
            "(Aziel Eliab, 2026). Advisory hygiene, not strategy. "
            "It does not help messages travel farther. It helps them arrive intact."
        ),
    )
    sub = parser.add_subparsers(dest="cmd", required=True)

    sub.add_parser("version", help="Print package version.")
    sub.add_parser("anchors", help="List the Top-30 geographic anchors.")

    p_adv = sub.add_parser(
        "advise",
        help="Emit one advisory for a last-known geo (five fields only).",
    )
    p_adv.add_argument(
        "--geo",
        required=True,
        help="Last-known geo (free text) or a Top-30 country name.",
    )
    p_adv.add_argument(
        "--json",
        action="store_true",
        dest="as_json",
        help="Print the five fields as JSON. No scores, no because.",
    )

    sub.add_parser(
        "zones",
        help="Read-only IANA zones with computed current local times.",
    )

    p_ui = sub.add_parser(
        "ui",
        help="Serve the local advisory UI on 127.0.0.1 (no memory of past advisories).",
    )
    p_ui.add_argument("--host", default="127.0.0.1", help="Loopback host (default 127.0.0.1).")
    p_ui.add_argument("--port", type=int, default=8765, help="Port (default 8765).")

    p_serve = sub.add_parser(
        "serve",
        help="Alias for ui. Bind 127.0.0.1 only.",
    )
    p_serve.add_argument("--host", default="127.0.0.1", help="Loopback host (default 127.0.0.1).")
    p_serve.add_argument("--port", type=int, default=8765, help="Port (default 8765).")

    return parser

def main(argv: Sequence[str] | None = None) -> int:
    parser = _build_parser()
    args = parser.parse_args(list(argv) if argv is not None else None)

    if args.cmd == "version":
        print(f"staticclock {__version__}")
        return 0

    if args.cmd == "anchors":
        for name in TOP_30:
            print(name)
        return 0

    if args.cmd == "advise":

```

```
        clock = StaticClock()
        try:
            advisory = clock.advise(args.geo)
        finally:
            clock.forget()
        payload = advisory.to_dict()
        if args.as_json:
            print(json.dumps(payload, indent=2, ensure_ascii=False))
        else:
            for key in OUTPUT_FIELDS:
                print(f"{key}: {payload[key]}")
        return 0

if args.cmd == "zones":
    rows = list_timezones()
    for row in rows:
        print(
            f"{row['region']:16} {row['iana']:36} "
            f"{row['local_date']} {row['local_time']} UTC{row['utc_offset']}"
        )
    return 0

if args.cmd in {"ui", "serve"}:
    from staticclock.ui import serve

    try:
        serve(host=args.host, port=args.port)
    except ValueError as exc:
        print(str(exc), file=sys.stderr)
        return 2
    return 0

parser.error(f"unknown command {args.cmd}")
return 2

if __name__ == "__main__":
    raise SystemExit(main())
```

forgereceipts/engines/staticclock/engine.py

```

"""StaticClock engine: one advisory, then forget.

In-memory session only. No sqlite, no ``.staticclock`` store, no user
id, no outcome optimization. User-facing output is exactly five fields.
"""

from __future__ import annotations

import json
import secrets
from dataclasses import asdict, dataclass
from staticclock.anchors import basket_of, resolve_geo
from staticclock.chronolect import local_date, pick_time
from staticclock.glossa import pick_dialect, primary_language
from staticclock.index import record
from staticclock.polarize import shake

OUTPUT_FIELDS: tuple[str, ...] = (
    "geo_location_chosen",
    "optimal_time",
    "optimal_date",
    "primary_language",
    "dialect_section",
)

_BANNED_USER_KEYS = frozenset(
    {
        "because",
        "score",
        "confidence",
        "alternative",
        "alternatives",
        "reason",
        "reasons",
        "explanation",
    }
)

@dataclass(frozen=True)
class Advisory:
    geo_location_chosen: str
    optimal_time: str
    optimal_date: str
    primary_language: str
    dialect_section: str

    def to_dict(self) -> dict[str, str]:
        payload = asdict(self)
        if tuple(payload.keys()) != OUTPUT_FIELDS:
            # dataclass field order is the contract
            payload = {k: payload[k] for k in OUTPUT_FIELDS}
        extra = set(payload) - set(OUTPUT_FIELDS)
        if extra:
            raise ValueError(f"advisory leaked extra fields: {extra}")
        banned = _BANNED_USER_KEYS & {k.lower() for k in payload}
        if banned:
            raise ValueError(f"advisory leaked banned keys: {banned}")
        return payload

    def to_json(self) -> str:
        return json.dumps(self.to_dict(), indent=2, ensure_ascii=False)

    def to_text(self) -> str:
        d = self.to_dict()
        return "\n".join(f"{k}: {d[k]}" for k in OUTPUT_FIELDS)

class StaticClock:
    """In-memory advisory session.

    ``nonce`` is a test hook. Production callers omit it; a one-shot
    nonce is drawn from ``secrets.token_bytes``. After ``advise()``,
    call ``forget()`` to drop nonce and inputs. The returned Advisory
    is a snapshot the caller holds; the engine does not keep it.
    """

    def __init__(self, nonce: bytes | None = None) -> None:
        self._pin = nonce
        self._nonce: bytes | None = None
        self._last_inputs: dict[str, str] | None = None
        self._forgotten = False

    def advise(self, geo: str) -> Advisory:
        nonce = self._pin if self._pin is not None else secrets.token_bytes(16)
        self._nonce = nonce
        self._last_inputs = {"geo": geo}
        self._forgotten = False

        anchor = resolve_geo(geo)
        basket = basket_of(anchor)
        chosen = shake(basket, nonce, salt=b"geo")

```

```

    rec = record(chosen)
    language = primary_language(chosen)
    dialect = pick_dialect(language, nonce)
    clock = pick_time(chosen, nonce)
    day = local_date(str(rec["iana"]))

    return Advisory(
        geo_location_chosen=chosen,
        optimal_time=clock,
        optimal_date=day.isoformat(),
        primary_language=language,
        dialect_section=dialect,
    )

def forget(self) -> None:
    """Drop nonce and last inputs. Irreversible for this moment."""
    self._nonce = None
    self._last_inputs = None
    self._forgotten = True

@property
def forgotten(self) -> bool:
    return self._forgotten

@property
def last_inputs(self) -> dict[str, str] | None:
    if self._last_inputs is None:
        return None
    return dict(self._last_inputs)

@property
def nonce(self) -> bytes | None:
    return self._nonce

def __enter__(self) -> "StaticClock":
    return self

def __exit__(self, *exc: object) -> None:
    self.forget()

```

forgereceipts/engines/staticclock/glossa.py

```
"""Glossa: region ? primary language, then one dialect of five.

Routing uses the static bundled index only. Not a learned model. Not
100+ languages in v0.1 ? Top-30 plus five dialectal variants each.
"""

from __future__ import annotations

from staticclock.index import languages as _languages
from staticclock.index import record
from staticclock.polarize import shake

def primary_language(region: str) -> str:
    return str(record(region)["language"])

def dialects_for(language: str) -> list[str]:
    table = _languages()
    if language not in table:
        raise KeyError(language)
    return list(table[language])

def pick_dialect(language: str, nonce: bytes) -> str:
    return shake(dialects_for(language), nonce, salt=b"dialect")
```


forgereceipts/engines/staticclock/index.py

```
"""Bundled static language / geography index.

v0.1 ships the Top-30 anchor set. A full 100+ language index is a
replacement update of this JSON, not a network fetch and not a learned
model.
"""

from __future__ import annotations

import json
from functools import lru_cache
from importlib.resources import files
from typing import Any

@lru_cache(maxsize=1)
def load_index() -> dict[str, Any]:
    raw = files("staticclock").joinpath("data/index.json").read_text(encoding="utf-8")
    return json.loads(raw)

def anchors() -> dict[str, Any]:
    return load_index()["anchors"]

def languages() -> dict[str, list[str]]:
    data = load_index()["languages"]
    return {k: list(v) for k, v in data.items()}

def record(name: str) -> dict[str, Any]:
    return dict(anchors()[name])
```

forgereceipts/engines/staticclock/polarize.py

```
"""Polarize: five-region basket, one-shot shake, pick one.

The shake is deterministic from a session nonce. It is not a user id,
not a profile, and not an engagement optimizer.
"""

from __future__ import annotations

import hashlib

def shake(basket: list[str], nonce: bytes, salt: bytes = b"geo") -> str:
    """Pick one member of ``basket`` from ``nonce``. Order-stable."""
    if not basket:
        raise ValueError("basket must not be empty")
    scored = sorted(
        basket,
        key=lambda item: hashlib.sha256(nonce + salt + item.encode("utf-8")).digest(),
    )
    return scored[0]

def pick_index(n: int, nonce: bytes, salt: bytes) -> int:
    if n <= 0:
        raise ValueError("n must be positive")
    digest = hashlib.sha256(nonce + salt).digest()
    return int.from_bytes(digest[:8], "big") % n
```

forgereceipts/engines/staticclock/ui.py

```

"""Local StaticClock UI. Bind 127.0.0.1 only.

Last-known-geo input, Top-30 dropdown, five output fields, read-only
timezone panel. Self-contained CSS. No CDN. No memory of past advisories.
Each /api/advise is one moment, then forget.
"""

from __future__ import annotations

import json
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from importlib.resources import files
from urllib.parse import urlparse

from staticclock.anchors import TOP_30
from staticclock.engine import OUTPUT_FIELDS, StaticClock
from staticclock.zones import list_timezones

LOOPBACK = frozenset({"127.0.0.1", "localhost", "::1"})
WEB = files("staticclock") / "web"
MIME = {
    ".html": "text/html; charset=utf-8",
    ".css": "text/css; charset=utf-8",
    ".js": "application/javascript; charset=utf-8",
}

def _web_bytes(name: str) -> bytes:
    return (WEB / name).read_bytes()

class Handler(BaseHTTPRequestHandler):
    server_version = "StaticClock/0.1.0"

    def log_message(self, fmt: str, *args: object) -> None:
        return

    def _send(self, status: int, body: bytes, content_type: str) -> None:
        self.send_response(status)
        self.send_header("Content-Type", content_type)
        self.send_header("Content-Length", str(len(body)))
        self.send_header("Cache-Control", "no-store")
        self.end_headers()
        self.wfile.write(body)

    def _json(self, status: int, obj: object) -> None:
        body = json.dumps(obj, indent=2, ensure_ascii=False).encode("utf-8")
        self._send(status, body, "application/json; charset=utf-8")

    def do_GET(self) -> None: # noqa: N802
        path = urlparse(self.path).path
        if path in {"/", "/index.html"}:
            self._send(200, _web_bytes("index.html"), MIME[".html"])
            return
        if path == "/style.css":
            self._send(200, _web_bytes("style.css"), MIME[".css"])
            return
        if path == "/app.js":
            self._send(200, _web_bytes("app.js"), MIME[".js"])
            return
        if path == "/api/anchors":
            self._json(200, list(TOP_30))
            return
        if path == "/api/zones":
            self._json(200, list_timezones())
            return
        self._json(404, {"error": "not found"})

    def do_POST(self) -> None: # noqa: N802
        path = urlparse(self.path).path
        if path != "/api/advise":
            self._json(404, {"error": "not found"})
            return
        length = int(self.headers.get("Content-Length") or "0")
        raw = self.rfile.read(length) if length else b"{}"
        try:
            payload = json.loads(raw.decode("utf-8")) or {}
        except json.JSONDecodeError:
            self._json(400, {"error": "JSON body required"})
            return
        geo = ""
        if isinstance(payload, dict):
            geo = str(payload.get("geo") or payload.get("anchor") or "")
        clock = StaticClock()
        try:
            advisory = clock.advise(geo)
            data = advisory.to_dict()
        finally:
            clock.forget()
        # Five fields only. Do not retain the advisory on the server.
        self._json(200, {k: data[k] for k in OUTPUT_FIELDS})

def make_server(host: str = "127.0.0.1", port: int = 8765) -> ThreadingHTTPServer:

```

```
if host not in LOOPBACK:
    raise ValueError("StaticClock UI binds loopback only (127.0.0.1)")
return ThreadingHTTPServer((host, port), Handler)

def serve(host: str = "127.0.0.1", port: int = 8765) -> None:
    httpd = make_server(host, port)
    bound_host, bound_port = httpd.server_address[:2]
    print(f"StaticClock UI http://{bound_host}:{bound_port} (loopback only; no memory of advisories)")
    try:
        httpd.serve_forever()
    except KeyboardInterrupt:
        print("\nstopped")
    finally:
        httpd.server_close()
```

forgereceipts/engines/staticclock/web/app.js

```

(function () {
  var geo = document.getElementById("geo");
  var anchor = document.getElementById("anchor");
  var go = document.getElementById("go");
  var fields = {
    geo: document.getElementById("out-geo"),
    time: document.getElementById("out-time"),
    date: document.getElementById("out-date"),
    lang: document.getElementById("out-lang"),
    dialect: document.getElementById("out-dialect")
  };

  function fill(el, value) {
    el.textContent = value || "?";
    if (value) el.classList.remove("empty");
    else el.classList.add("empty");
  }

  function clearAdvisory() {
    fill(fields.geo, "");
    fill(fields.time, "");
    fill(fields.date, "");
    fill(fields.lang, "");
    fill(fields.dialect, "");
  }

  fetch("/api/anchors").then(function (r) { return r.json(); }).then(function (names) {
    names.forEach(function (name) {
      var opt = document.createElement("option");
      opt.value = name;
      opt.textContent = name;
      anchor.appendChild(opt);
    });
  });

  fetch("/api/zones").then(function (r) { return r.json(); }).then(function (rows) {
    var body = document.getElementById("zones-body");
    body.textContent = "";
    rows.forEach(function (row) {
      var tr = document.createElement("tr");
      ["region", "iana", "local_date", "local_time", "utc_offset"].forEach(function (k) {
        var td = document.createElement("td");
        td.textContent = row[k] || "";
        tr.appendChild(td);
      });
      body.appendChild(tr);
    });
  });

  document.getElementById("advise-form").addEventListener("submit", function (ev) {
    ev.preventDefault();
    var text = (geo.value || "").trim();
    var picked = (anchor.value || "").trim();
    var query = text || picked;
    go.disabled = true;
    fetch("/api/advise", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ geo: query })
    }).then(function (r) { return r.json(); }).then(function (adv) {
      fill(fields.geo, adv.geo_location_chosen);
      fill(fields.time, adv.optimal_time);
      fill(fields.date, adv.optimal_date);
      fill(fields.lang, adv.primary_language);
      fill(fields.dialect, adv.dialect_section);
    }).catch(function () {
      clearAdvisory();
    }).finally(function () {
      go.disabled = false;
    });
  });
})();

```

forgereceipts/engines/staticclock/web/index.html

```

<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>StaticClock ? release advisory</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <header>
    <div class="tag">StaticClock ? v0.1.0 ? loopback</div>
    <h1>When should this be released so it is read, not reacted to?</h1>
    <p class="lede">
      Advisory only. Least-distorting window by timezone, language, and
      dialect. Not a scheduler, not a targeting system, not analytics.
      One moment; then forgotten. No memory of past advisories.
    </p>
  </header>
  <hr class="rule">

  <form id="advise-form" autocomplete="off">
    <div class="row">
      <label for="geo">
        <span class="kicker">Last-known geo</span>
        Free text (city, region, country). Not an identity.
      </label>
      <input id="geo" name="geo" type="text" placeholder="Indiana" spellcheck="false">
    </div>
    <div class="or">or</div>
    <div class="row">
      <label for="anchor">
        <span class="kicker">Top-30 country</span>
        Anchor dropdown. Polarize a five-region basket from this.
      </label>
      <select id="anchor" name="anchor">
        <option value="">? choose an anchor ?</option>
      </select>
    </div>
    <button class="advise" type="submit" id="go">Advise</button>
  </form>

  <section class="output" aria-live="polite">
    <h2>Advisory</h2>
    <dl class="fields">
      <dt>geo location chosen</dt>
      <dd id="out-geo" class="empty">?</dd>
      <dt>optimal time</dt>
      <dd id="out-time" class="empty">?</dd>
      <dt>optimal date</dt>
      <dd id="out-date" class="empty">?</dd>
      <dt>primary language</dt>
      <dd id="out-lang" class="empty">?</dd>
      <dt>dialect section</dt>
      <dd id="out-dialect" class="empty">?</dd>
    </dl>
    <p class="note">Five fields. No scores, no confidence, no alternatives, no because.</p>
  </section>

  <section class="zones">
    <h2>Timezone reference</h2>
    <p>Read-only. Computed locally with IANA zoneinfo. Listing zones does not change the advisory.</p>
    <table>
      <thead>
        <tr>
          <th>Region</th>
          <th>IANA</th>
          <th>Local date</th>
          <th>Local time</th>
          <th>UTC</th>
        </tr>
      </thead>
      <tbody id="zones-body"></tbody>
    </table>
  </section>

  <footer>
    <p><strong>It does not help messages travel farther. It helps them arrive intact.</strong></p>
    <p>Apache-2.0 ? Aziel Eliab ? 2026 ? Bound to 127.0.0.1 ? Forks welcome and always allowed.</p>
  </footer>
  <script src="/app.js"></script>
</body>
</html>

```

forgereceipts/engines/staticclock/web/style.css

```

/* StaticClock local UI. No CDN. Advisory hygiene, not strategy. */
:root {
  --ink: #1c1916;
  --paper: #f3eee4;
  --rule: #c9c0b0;
  --muted: #6b6458;
  --panel: #fffaf1;
  --mark: #7a2e24;
  --ok: #2c4a3e;
  font-synthesis: none;
}
* { box-sizing: border-box; }
html, body {
  margin: 0;
  padding: 0;
  background: var(--paper);
  color: var(--ink);
  font-family: "Iowan Old Style", Palatino, "Palatino Linotype", "Times New Roman", serif;
  line-height: 1.45;
}
body {
  max-width: 52rem;
  margin: 0 auto;
  padding: 2.25rem 1.25rem 4rem;
}
header h1 {
  font-size: 1.85rem;
  font-weight: 600;
  letter-spacing: 0.02em;
  margin: 0 0 0.25rem;
}
header .tag {
  font-family: ui-monospace, "SF Mono", Menlo, Consolas, monospace;
  font-size: 0.78rem;
  color: var(--muted);
  text-transform: uppercase;
  letter-spacing: 0.12em;
}
header p.lede {
  margin: 1rem 0 0;
  max-width: 40rem;
}
hr.rule {
  border: 0;
  border-top: 1px solid var(--rule);
  margin: 1.5rem 0;
}
label {
  display: block;
  font-size: 0.92rem;
  margin-bottom: 0.35rem;
}
label span.kicker {
  display: block;
  font-family: ui-monospace, "SF Mono", Menlo, Consolas, monospace;
  font-size: 0.7rem;
  letter-spacing: 0.1em;
  text-transform: uppercase;
  color: var(--muted);
  margin-bottom: 0.15rem;
}
input[type="text"], select {
  width: 100%;
  padding: 0.55rem 0.65rem;
  border: 1px solid var(--ink);
  background: var(--panel);
  color: var(--ink);
  font: inherit;
  font-size: 1rem;
}
.row { margin-bottom: 1rem; }
.or {
  text-align: center;
  color: var(--muted);
  font-size: 0.8rem;
  letter-spacing: 0.16em;
  text-transform: uppercase;
  margin: 0.4rem 0 0.9rem;
}
button.advise {
  appearance: none;
  border: 1px solid var(--ink);
  background: var(--ink);
  color: var(--paper);
  font: inherit;
  padding: 0.55rem 1.1rem;
  cursor: pointer;
  letter-spacing: 0.04em;
}
button.advise:hover { background: var(--mark); border-color: var(--mark); }
button.advise:disabled { opacity: 0.5; cursor: wait; }
.output {

```

```

    margin-top: 1.75rem;
    border: 1px solid var(--ink);
    background: var(--panel);
    padding: 1rem 1.1rem 0.4rem;
    min-height: 8rem;
  }
  .output h2 {
    font-size: 0.78rem;
    font-family: ui-monospace, "SF Mono", Menlo, Consolas, monospace;
    letter-spacing: 0.12em;
    text-transform: uppercase;
    margin: 0 0 0.8rem;
    color: var(--muted);
    font-weight: 500;
  }
  dl.fields {
    display: grid;
    grid-template-columns: 11.5rem 1fr;
    gap: 0.35rem 0.8rem;
    margin: 0 0 1rem;
  }
  dl.fields dt {
    font-family: ui-monospace, "SF Mono", Menlo, Consolas, monospace;
    font-size: 0.75rem;
    color: var(--muted);
    align-self: center;
  }
  dl.fields dd { margin: 0; font-size: 1.05rem; }
  dl.fields dd.empty { color: var(--rule); }
  .note { font-size: 0.85rem; color: var(--muted); }
  .zones { margin-top: 2rem; }
  .zones h2 { font-size: 1.05rem; margin: 0 0 0.35rem; }
  .zones p { margin: 0 0 0.8rem; color: var(--muted); font-size: 0.9rem; }
  .zones table {
    width: 100%;
    border-collapse: collapse;
    font-size: 0.85rem;
    font-family: ui-monospace, "SF Mono", Menlo, Consolas, monospace;
    background: var(--panel);
  }
  .zones th, .zones td {
    text-align: left;
    padding: 0.35rem 0.5rem;
    border-bottom: 1px solid var(--rule);
  }
  .zones th {
    font-weight: 500;
    color: var(--muted);
    font-size: 0.72rem;
    letter-spacing: 0.08em;
    text-transform: uppercase;
  }
  .zones td:nth-child(2) { color: var(--ok); }
  footer { margin-top: 2.5rem; font-size: 0.82rem; color: var(--muted); }
  footer strong { color: var(--ink); font-weight: 600; }

```


forgereceipts/engines/staticclock/zones.py

```

"""Read-only IANA timezone reference.

Computed with zoneinfo from the local tz database. No network. Listing
zones does not mint a nonce and does not change an advisory.
"""

from __future__ import annotations

from datetime import datetime
from zoneinfo import ZoneInfo

from staticclock.anchors import TOP_30
from staticclock.index import record

def list_timezones(when: datetime | None = None) -> list[dict[str, str]]:
    """Return IANA zones for the Top-30 anchors with current local times."""
    rows: list[dict[str, str]] = []
    for name in TOP_30:
        iana = str(record(name)["iana"])
        tz = ZoneInfo(iana)
        now = datetime.now(tz) if when is None else when.astimezone(tz)
        rows.append(
            {
                "region": name,
                "iana": iana,
                "local_time": now.strftime("%H:%M"),
                "local_date": now.date().isoformat(),
                "utc_offset": now.strftime("%Z"),
            }
        )
    return rows

```

forgereceipts/engines/temporallock/__init__.py

```
"""TemporalLock: append-only observation receipts.

July 2026 whitepaper implementation by Aziel Eliab.

Receipts, not truth claims. Forks are welcome and always allowed.
No consensus, no mining, no tokens, no truth scores.
"""

from __future__ import annotations

from temporallock.chain import Chain, Fork, VerifyResult, detect_forks
from temporallock.errors import AppendOnlyError, ChainError, ReceiptError, TemporalLockError
from temporallock.hashing import GENESIS_PREV_HASH, canonical_bytes, digest
from temporallock.receipt import Receipt

__version__ = "0.1.0"
__author__ = "Azriel Eliab"
__all__ = [
    "AppendOnlyError",
    "Chain",
    "ChainError",
    "Fork",
    "GENESIS_PREV_HASH",
    "Receipt",
    "ReceiptError",
    "TemporalLockError",
    "VerifyResult",
    "canonical_bytes",
    "detect_forks",
    "digest",
    "__version__",
]
```

forgereceipts/engines/temporallock/__main__.py

```
"""Allow ``python -m temporallock`` to invoke the CLI."""

from temporallock.cli import main

if __name__ == "__main__":
    raise SystemExit(main())
```

forgereceipts/engines/temporallock/canon.py

```

"""Canonical encoding for TemporalLock receipts (v0.1.0).

Canonical encoding
-----
UTF-8 JSON with **sorted keys** and **no extra whitespace**
(`separators=(",", ":")`, `sort_keys=True`, `ensure_ascii=False`).

Hashed fields (core only, v0.1.0):

    timestamp, summary, evidence, confidence, prev_hash

The receipt's own `hash` field is **excluded** from the encoding.

`confidence` is serialized as a JSON **number** with exactly **6 decimal
places** so hashes are stable across platforms and Python versions
(example: `0.7` becomes `0.700000`). A placeholder-and-replace step
is used because `json.dumps` would otherwise emit `0.7`.

Optional fields are allowed in implementations (extra JSONL keys) but
MUST NOT enter the core hash unless a later **versioned** schema is
introduced. v0.1.0 is core-only so chains stay verifiable long-term.

Genesis `prev_hash` is 64 zero hex characters.

Algorithm: SHA-256 of the canonical UTF-8 bytes; digest is lowercase hex.
"""

from __future__ import annotations

import hashlib
import json
from typing import Any

# Genesis previous-hash: 64 zero hex chars (SHA-256 width).
GENESIS_PREV_HASH = "0" * 64

# Confidence is always emitted with this many digits after the decimal.
CONFIDENCE_DECIMALS = 6

CORE_FIELDS = ("timestamp", "summary", "evidence", "confidence", "prev_hash")

_CONF_PLACEHOLDER = "__TL_CONFIDENCE__"

def format_confidence(confidence: float) -> str:
    """Return confidence as a fixed-precision decimal string (not quoted)."""
    return f"{float(confidence):.{CONFIDENCE_DECIMALS}f}"

def canonical_bytes(
    timestamp: str,
    summary: str,
    evidence: str,
    confidence: float,
    prev_hash: str,
) -> bytes:
    """Return the v0.1.0 canonical UTF-8 encoding of the core fields.

    Field order in the JSON object is lexicographic via `sort_keys=True`:
    confidence, evidence, prev_hash, summary, timestamp.
    """
    payload: dict[str, Any] = {
        "confidence": _CONF_PLACEHOLDER,
        "evidence": evidence,
        "prev_hash": prev_hash,
        "summary": summary,
        "timestamp": timestamp,
    }
    raw = json.dumps(payload, sort_keys=True, separators=(",", ":"), ensure_ascii=False)
    raw = raw.replace(f'"{_CONF_PLACEHOLDER}"', format_confidence(confidence))
    return raw.encode("utf-8")

def digest(
    timestamp: str,
    summary: str,
    evidence: str,
    confidence: float,
    prev_hash: str,
) -> str:
    """SHA-256 (lowercase hex) of `canonical_bytes(...)`."""
    return hashlib.sha256(
        canonical_bytes(timestamp, summary, evidence, confidence, prev_hash)
    ).hexdigest()

```

forgereceipts/engines/temporallock/chain.py

```

"""Append-only receipt chain.

chain.append(...)`` only. No modify, no delete. Corrections and
disputes are new receipts. Divergent chains (forks) are valid and
detectable; this module does not pick a winner.
"""

from __future__ import annotations

import json
from dataclasses import dataclass, field
from pathlib import Path
from typing import Iterator, Sequence

from temporallock.errors import AppendOnlyError, ChainError
from temporallock.hashing import GENESIS_PREV_HASH
from temporallock.receipt import Receipt

@dataclass(frozen=True)
class VerifyResult:
    """Result of walking a chain: hashes and links, nothing interpretive."""

    ok: bool
    length: int
    first_hash: str | None
    last_hash: str | None
    errors: list[str] = field(default_factory=list)

@dataclass(frozen=True)
class Fork:
    """A prev_hash with more than one child. Forks are allowed."""

    prev_hash: str
    child_hashes: tuple[str, ...]

def detect_forks(receipts: Sequence[Receipt]) -> list[Fork]:
    """Find prev_hash values that have more than one child.

    Does not pick a winner. Order of forks follows first appearance of
    the parent hash; child hashes follow receipt order.
    """
    children: dict[str, list[str]] = {}
    order: list[str] = []
    for rec in receipts:
        if rec.prev_hash not in children:
            order.append(rec.prev_hash)
            children[rec.prev_hash] = []
        children[rec.prev_hash].append(rec.hash)
    return [
        Fork(prev_hash=ph, child_hashes=tuple(children[ph]))
        for ph in order
        if len(children[ph]) > 1
    ]

def _receipt_json_line(receipt: Receipt) -> str:
    """One JSONL line. Confidence stored as a JSON number; hash included.

    This is the *file* encoding, not the canonical hash encoding.
    """
    body = receipt.to_dict()
    return json.dumps(body, sort_keys=True, separators=(",", ":"), ensure_ascii=False)

def _append_line(path: Path, receipt: Receipt) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    # Append-only: mode 'a' never rewrites earlier bytes.
    with path.open("a", encoding="utf-8") as fh:
        fh.write(_receipt_json_line(receipt))
        fh.write("\n")
        fh.flush()

class Chain:
    """In-memory and/or JSONL-backed append-only sequence of receipts."""

    def __init__(
        self,
        receipts: Sequence[Receipt] | None = None,
        path: str | Path | None = None,
    ) -> None:
        self._receipts: tuple[Receipt, ...] = tuple(receipts or ())
        self._path: Path | None = Path(path) if path is not None else None

    @property
    def path(self) -> Path | None:
        return self._path

    def __len__(self) -> int:

```

```

        return len(self._receipts)

    def __iter__(self) -> Iterator[Receipt]:
        return iter(self._receipts)

    def __getitem__(self, index: int) -> Receipt:
        return self._receipts[index]

    def __bool__(self) -> bool:
        return bool(self._receipts)

    def _refuse(self, action: str) -> None:
        raise AppendOnlyError(
            f"cannot {action}: TemporalLock is append-only; "
            "record a correction as a new receipt"
        )

    def pop(self, *args: object, **kwargs: object) -> None:
        self._refuse("pop")

    def insert(self, *args: object, **kwargs: object) -> None:
        self._refuse("insert")

    def remove(self, *args: object, **kwargs: object) -> None:
        self._refuse("remove")

    def clear(self) -> None:
        self._refuse("clear")

    def reverse(self) -> None:
        self._refuse("reverse")

    def __setitem__(self, *args: object, **kwargs: object) -> None:
        self._refuse("replace")

    def __delitem__(self, *args: object, **kwargs: object) -> None:
        self._refuse("delete")

    @classmethod
    def load(cls, path: str | Path) -> "Chain":
        """Read a JSONL file (core fields). Extra keys are ignored.

        After load the file is only ever opened again in mode ``'a'``.
        Does not rewrite. Call ``verify()`` to check hashes and links.
        """
        path = Path(path)
        receipts: list[Receipt] = []
        if path.is_file():
            text = path.read_text(encoding="utf-8")
            for line in text.splitlines():
                line = line.strip()
                if not line:
                    continue
                receipts.append(Receipt.from_dict(json.loads(line)))
        return cls(receipts, path=path)

    @classmethod
    def genesis(
        cls,
        path: str | Path,
        *,
        summary: str,
        evidence: str,
        confidence: float = 1.0,
        timestamp: str | None = None,
    ) -> "Chain":
        """Create the first receipt in a new JSONL file. File must be absent or empty."""
        path = Path(path)
        if path.exists() and path.stat().st_size > 0:
            raise ChainError(f"chain already exists: {path}; use append")
        chain = cls(), path=path
        chain.append(
            summary=summary,
            evidence=evidence,
            confidence=confidence,
            timestamp=timestamp,
        )
        return chain

    def append(
        self,
        summary: str,
        evidence: str,
        confidence: float = 1.0,
        timestamp: str | None = None,
        *,
        require_existing: bool = False,
    ) -> Receipt:
        """Append a new receipt. First receipt is genesis (prev_hash = 64 zeros).

        If ``require_existing`` is true (CLI ``append``), the chain must
        already contain at least one receipt.
        """
        if require_existing and not self._receipts:
            raise ChainError("chain does not exist or is empty; use genesis")

```

```

    if self._receipts:
        prev = self._receipts[-1].hash
    else:
        prev = GENESIS_PREV_HASH
    receipt = Receipt.create(
        summary=summary,
        evidence=evidence,
        confidence=confidence,
        timestamp=timestamp,
        prev_hash=prev,
    )
    if self._path is not None:
        _append_line(self._path, receipt)
    self._receipts = self._receipts + (receipt,)
    return receipt

def verify(self) -> VerifyResult:
    """Walk the chain. Check each hash and each consecutive link.

    Forks stored as a single linear JSONL will fail the consecutive
    link check (store divergent chains separately). Hash mismatches
    and broken prev_hash links are errors. Forks themselves are not
    a verdict; see ``forks()``.
    """
    errors: list[str] = []
    n = len(self._receipts)
    first = self._receipts[0].hash if n else None
    last = self._receipts[-1].hash if n else None

    for i, rec in enumerate(self._receipts):
        expected = rec.recomputed_hash()
        if rec.hash != expected:
            errors.append(
                f"index {i}: stored hash {rec.hash} != recomputed {expected}"
            )
        if i == 0:
            continue
        prev = self._receipts[i - 1]
        if rec.prev_hash != prev.hash:
            errors.append(
                f"index {i}: prev_hash {rec.prev_hash} != previous.hash {prev.hash}"
            )

    return VerifyResult(
        ok=not errors,
        length=n,
        first_hash=first,
        last_hash=last,
        errors=errors,
    )

def forks(self) -> list[Fork]:
    """prev_hash values with more than one child. Does not pick a winner."""
    return detect_forks(self._receipts)

```

forgereceipts/engines/temporallock/cli.py

```

"""Command-line interface for TemporalLock.

    temporallock version
    temporallock genesis --chain FILE.jsonl --summary "..." --evidence "..."
    temporallock append --chain FILE.jsonl --summary "..." --evidence "..." [--confidence 0.7] [--timestamp ISO]
    temporallock verify FILE.jsonl
    temporallock show FILE.jsonl

Receipts, not truth claims. Forks always allowed.
"""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path
from typing import Sequence

from temporallock import __version__
from temporallock.chain import Chain
from temporallock.errors import AppendOnlyError, ChainError, ReceiptError, TemporalLockError

def _build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="temporallock",
        description=(
            "TemporalLock ? append-only observation receipts "
            "(Azriel Eliab, July 2026). Receipts, not truth claims."
        ),
    )
    sub = parser.add_subparsers(dest="cmd", required=True)

    sub.add_parser("version", help="Print package version.")

    p_gen = sub.add_parser("genesis", help="Write the first receipt (prev_hash = 64 zeros).")
    p_gen.add_argument("--chain", required=True, help="JSONL chain path (must not already exist).")
    p_gen.add_argument("--summary", required=True, help="Brief summary of what was observed.")
    p_gen.add_argument("--evidence", required=True, help="Supporting evidence (body and/or URI/path).")
    p_gen.add_argument("--confidence", type=float, default=1.0, help="Observer confidence in [0.0, 1.0] (default 1.0).")

    p_app = sub.add_parser("append", help="Append a receipt to an existing chain.")
    p_app.add_argument("--chain", required=True, help="JSONL chain path (must already exist).")
    p_app.add_argument("--summary", required=True, help="Brief summary of what was observed.")
    p_app.add_argument("--evidence", required=True, help="Supporting evidence (body and/or URI/path).")
    p_app.add_argument("--confidence", type=float, default=0.7, help="Observer confidence in [0.0, 1.0] (default 0.7).")

    p_ver = sub.add_parser("verify", help="Walk the chain; exit 0 if intact, nonzero if broken.")
    p_ver.add_argument("file", help="JSONL chain path.")

    p_show = sub.add_parser("show", help="Print receipts in a chain.")
    p_show.add_argument("file", help="JSONL chain path.")

    return parser

def _print_receipt_brief(receipt, index: int | None = None) -> None:
    prefix = f"[{index}] " if index is not None else ""
    print(f"{prefix}{receipt.timestamp} conf={receipt.confidence:.6f} hash={receipt.hash}")
    print(f"    prev={receipt.prev_hash}")
    print(f"    summary: {receipt.summary}")
    print(f"    evidence: {receipt.evidence}")

def main(argv: Sequence[str] | None = None) -> int:
    parser = _build_parser()
    args = parser.parse_args(list(argv) if argv is not None else None)

    try:
        if args.cmd == "version":
            print(f"temporallock {__version__}")
            return 0

        if args.cmd == "genesis":
            path = Path(args.chain)
            chain = Chain.genesis(
                path,
                summary=args.summary,
                evidence=args.evidence,
                confidence=args.confidence,
                timestamp=args.timestamp,
            )
            rec = chain[-1]
            print(f"genesis {rec.hash}")
            return 0

        if args.cmd == "append":

```



```

    path = Path(args.chain)
    if not path.is_file() or path.stat().st_size == 0:
        print("chain does not exist; use genesis for the first receipt", file=sys.stderr)
        return 2
    chain = Chain.load(path)
    rec = chain.append(
        summary=args.summary,
        evidence=args.evidence,
        confidence=args.confidence,
        timestamp=args.timestamp,
        require_existing=True,
    )
    print(f"appended {rec.hash}")
    return 0

if args.cmd == "verify":
    path = Path(args.file)
    if not path.is_file():
        print(f"not found: {path}", file=sys.stderr)
        return 2
    chain = Chain.load(path)
    result = chain.verify()
    payload = {
        "ok": result.ok,
        "length": result.length,
        "first_hash": result.first_hash,
        "last_hash": result.last_hash,
        "errors": result.errors,
    }
    print(json.dumps(payload, indent=2))
    return 0 if result.ok else 1

if args.cmd == "show":
    path = Path(args.file)
    if not path.is_file():
        print(f"not found: {path}", file=sys.stderr)
        return 2
    chain = Chain.load(path)
    print(f"temporallock chain n={len(chain)} path={path}")
    for i, rec in enumerate(chain):
        _print_receipt_brief(rec, i)
    forks = chain.forks()
    if forks:
        print(f"forks: {len(forks)} (allowed; no winner chosen)")
        for f in forks:
            print(f"  prev={f.prev_hash}")
            for h in f.child_hashes:
                print(f"    child={h}")
    return 0

    parser.error(f"unknown command {args.cmd}")
    return 2
except (ReceiptError, ChainError, AppendOnlyError, TemporalLockError) as exc:
    print(str(exc), file=sys.stderr)
    return 2
except OSError as exc:
    print(str(exc), file=sys.stderr)
    return 2

if __name__ == "__main__":
    raise SystemExit(main())

```

forgereceipts/engines/temporallock/errors.py

```
"""TemporalLock errors."""

from __future__ import annotations

class TemporalLockError(Exception):
    """Base error for TemporalLock."""

class AppendOnlyError(TemporalLockError):
    """Raised on any attempt to edit, pop, replace, or delete a receipt."""

class ReceiptError(TemporalLockError):
    """Raised when a receipt field is invalid (empty evidence, bad confidence)."""

class ChainError(TemporalLockError):
    """Raised for chain-level problems (missing genesis, append on empty file)."""
```

forgereceipts/engines/temporallock/hashing.py

```
"""Hashing helpers for TemporalLock.

Re-exports canonical encoding from ``canon`` so callers can use
``temporallock.hashing.canonical_bytes`` and ``temporallock.hashing.digest``
as specified. Stdlib ``hashlib`` only; no extra crypto packages.
"""

from __future__ import annotations

from temporallock.canon import (
    CONFIDENCE_DECIMALS,
    CORE_FIELDS,
    GENESIS_PREV_HASH,
    canonical_bytes,
    digest,
    format_confidence,
)

__all__ = [
    "CONFIDENCE_DECIMALS",
    "CORE_FIELDS",
    "GENESIS_PREV_HASH",
    "canonical_bytes",
    "digest",
    "format_confidence",
]
```

forgereceipts/engines/temporallock/receipt.py

```

"""Immutable observation receipt.

A receipt is a record of an observation at a moment. It is not a truth
claim, not an authority, and not a narrative. Corrections are new
receipts; this object cannot be edited.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from datetime import datetime, timezone
from math import isfinite
from typing import Any, Mapping

from temporallock.errors import AppendOnlyError, ReceiptError
from temporallock.hashing import GENESIS_PREV_HASH, digest

_HEX = set("0123456789abcdef")

def utc_now() -> str:
    """Observer-clock 'now' as UTC ISO-8601 with a trailing Z, second precision."""
    return (
        datetime.now(timezone.utc)
        .replace(microsecond=0)
        .isoformat()
        .replace("+00:00", "Z")
    )

def _require_str(name: str, value: Any) -> str:
    if not isinstance(value, str):
        raise ReceiptError(f"{name} must be a string")
    return value

def validate_evidence(evidence: str) -> str:
    evidence = _require_str("evidence", evidence)
    if evidence.strip() == "":
        raise ReceiptError("empty evidence is invalid")
    return evidence

def validate_confidence(confidence: Any) -> float:
    try:
        value = float(confidence)
    except (TypeError, ValueError) as exc:
        raise ReceiptError("confidence must be a float in [0.0, 1.0]") from exc
    if not isfinite(value) or value < 0.0 or value > 1.0:
        raise ReceiptError("confidence must be a float in [0.0, 1.0] inclusive")
    return value

def validate_summary(summary: str) -> str:
    return _require_str("summary", summary)

def validate_timestamp(timestamp: str) -> str:
    ts = _require_str("timestamp", timestamp)
    if ts.strip() == "":
        raise ReceiptError("timestamp must be a non-empty UTC ISO-8601 string")
    return ts

@dataclass
class Receipt:
    """Frozen observation receipt (v0.1.0 core fields only).

    Fields hashed: timestamp, summary, evidence, confidence, prev_hash.
    ``hash`` is SHA-256 of the canonical encoding of those fields.
    """

    timestamp: str
    summary: str
    evidence: str
    confidence: float
    prev_hash: str
    hash: str
    _frozen: bool = field(default=False, init=False, repr=False, compare=False)

    def __post_init__(self) -> None:
        object.__setattr__(self, "timestamp", validate_timestamp(self.timestamp))
        object.__setattr__(self, "summary", validate_summary(self.summary))
        object.__setattr__(self, "evidence", validate_evidence(self.evidence))
        object.__setattr__(self, "confidence", validate_confidence(self.confidence))
        object.__setattr__(self, "prev_hash", _require_str("prev_hash", self.prev_hash))
        object.__setattr__(self, "hash", _require_str("hash", self.hash))
        object.__setattr__(self, "_frozen", True)

    def __setattr__(self, name: str, value: Any) -> None:
        if getattr(self, "_frozen", False):
            raise AppendOnlyError(

```

```

        "receipts cannot be modified; append a correction as a new receipt"
    )
    object.__setattr__(self, name, value)

def __delattr__(self, name: str) -> None:
    raise AppendOnlyError(
        "receipts cannot be deleted; append a correction as a new receipt"
    )

def __hash__(self) -> int: # type: ignore[override]
    return hash(self.hash)

@classmethod
def create(
    cls,
    *,
    summary: str,
    evidence: str,
    confidence: float = 1.0,
    timestamp: str | None = None,
    prev_hash: str = GENESIS_PREV_HASH,
) -> "Receipt":
    """Mint a new receipt, computing ``hash`` from the core fields."""
    ts = utc_now() if timestamp is None else timestamp
    conf = validate_confidence(confidence)
    rec_hash = digest(
        timestamp=validate_timestamp(ts),
        summary=validate_summary(summary),
        evidence=validate_evidence(evidence),
        confidence=conf,
        prev_hash=_require_str("prev_hash", prev_hash),
    )
    return cls(
        timestamp=ts,
        summary=summary,
        evidence=evidence,
        confidence=conf,
        prev_hash=prev_hash,
        hash=rec_hash,
    )

def recomputed_hash(self) -> str:
    """Independent verifier: SHA-256 from fields only (ignores stored hash)."""
    return digest(
        timestamp=self.timestamp,
        summary=self.summary,
        evidence=self.evidence,
        confidence=self.confidence,
        prev_hash=self.prev_hash,
    )

def hash_ok(self) -> bool:
    return self.recomputed_hash() == self.hash

def to_dict(self) -> dict[str, Any]:
    return {
        "timestamp": self.timestamp,
        "summary": self.summary,
        "evidence": self.evidence,
        "confidence": self.confidence,
        "prev_hash": self.prev_hash,
        "hash": self.hash,
    }

@classmethod
def from_dict(cls, data: Mapping[str, Any]) -> "Receipt":
    """Load core fields. Extra keys are ignored (not hashed in v0.1.0)."""
    missing = [k for k in ("timestamp", "summary", "evidence", "confidence", "prev_hash", "hash") if k not in data]
    if missing:
        raise ReceiptError(f"receipt missing fields: {missing}")
    return cls(
        timestamp=data["timestamp"],
        summary=data["summary"],
        evidence=data["evidence"],
        confidence=data["confidence"],
        prev_hash=data["prev_hash"],
        hash=data["hash"],
    )
]

```

forgereceipts/engines/veillock/__init__.py

```
"""VeilLock: live-stream encryption of visual output before display.

July 2026 whitepaper implementation by Aziel Eliab.

Pipeline: render ? encrypt ? decode locally ? display.
Without the correct runtime key state, the display is undecodable noise.

Forks are welcome and always allowed.
"""

from __future__ import annotations

from veillock.engine import EncryptedFrame, EncryptedStream, VeilLockSession
from veillock.frames import FrameSource
from veillock.phoenix import PhoenixLoop
from veillock.pulse import AlwaysPass, HaltedError, PhoenixError, PulseCheck

__version__ = "0.1.0"
__author__ = "Azriel Eliab"
__all__ = [
    "AlwaysPass",
    "EncryptedFrame",
    "EncryptedStream",
    "FrameSource",
    "HaltedError",
    "PhoenixError",
    "PhoenixLoop",
    "PulseCheck",
    "VeilLockSession",
    "__version__",
]
```

forgereceipts/engines/veillock/__main__.py

```
"""Allow ``python -m veillock`` to invoke the CLI."""

from veillock.cli import main

if __name__ == "__main__":
    raise SystemExit(main())
```

forgereceipts/engines/veillock/cli.py

```

"""Command-line interface for VeilLock.

    veillock encrypt --in frames.npy --out cipher.npz --mode private|broadcast|obfuscation
    veillock decrypt --in cipher.npz --out frames.npy --key ...
    veillock version
"""

from __future__ import annotations

import argparse
import secrets
import sys
from typing import Sequence

import numpy as np

from veillock import __version__
from veillock.crypto import DecryptError
from veillock.engine import (
    VeilLockSession,
    load_cipher_npz,
    save_cipher_npz,
    session_from_wrapped,
)
from veillock.modes import Mode
from veillock.pulse import HaltedError

def _build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="veillock",
        description=(
            "VeilLock ? encrypt visual frames before they reach any external "
            "display (Aziel Eliab, July 2026).",
        ),
    )
    sub = parser.add_subparsers(dest="cmd", required=True)

    p_enc = sub.add_parser("encrypt", help="Encrypt an RGB frame stack (.npy) to cipher.npz.")
    p_enc.add_argument("--in", dest="inp", required=True, help="Input frames.npy (N,H,W,3) uint8.")
    p_enc.add_argument("--out", dest="out", required=True, help="Output cipher.npz.")
    p_enc.add_argument(
        "--mode",
        required=True,
        choices=("private", "broadcast", "obfuscation"),
        help="Deployment mode.",
    )
    p_enc.add_argument(
        "--key",
        default=None,
        help="Optional 64-char hex session key (generated if omitted).",
    )
    p_enc.add_argument(
        "--receiver-secret",
        default=None,
        help="Hex receiver secret (broadcast mode; generated if omitted).",
    )
    p_enc.add_argument(
        "--rotation-interval",
        type=int,
        default=120,
        help="Forward-secure rotation period in frames (60?240, default 120).",
    )

    p_dec = sub.add_parser("decrypt", help="Decrypt cipher.npz to frames.npy.")
    p_dec.add_argument("--in", dest="inp", required=True, help="Input cipher.npz.")
    p_dec.add_argument("--out", dest="out", required=True, help="Output frames.npy.")
    p_dec.add_argument(
        "--key",
        default=None,
        help="64-char hex session key (root key; required for private/obfuscation).",
    )
    p_dec.add_argument(
        "--receiver-secret",
        default=None,
        help="Hex receiver secret (unwraps the broadcast package key).",
    )

    sub.add_parser("version", help="Print the VeilLock version and exit.")
    return parser

def _parse_hex(label: str, value: str, expected_len: int | None = None) -> bytes:
    try:
        raw = bytes.fromhex(value.strip())
    except ValueError as exc:
        raise SystemExit(f"error: {label} must be hex: {exc}") from exc
    if expected_len is not None and len(raw) != expected_len:
        raise SystemExit(f"error: {label} must be {expected_len} bytes ({expected_len * 2} hex chars)")
    return raw

def _load_frames(path: str) -> np.ndarray:
    try:

```



```

        arr = np.load(path)
    except FileNotFoundError as exc:
        raise SystemExit(f"error: {exc}") from exc
    except Exception as exc: # noqa: BLE001
        raise SystemExit(f"error: failed to read frames: {exc}") from exc
    arr = np.ascontiguousarray(arr, dtype=np.uint8)
    if arr.ndim == 3 and arr.shape[-1] == 3:
        arr = arr[None, ...]
    if arr.ndim != 4 or arr.shape[-1] != 3:
        raise SystemExit("error: frames.npy must have shape (N,H,W,3) or (H,W,3) uint8")
    return arr

def main(argv: Sequence[str] | None = None) -> int:
    parser = _build_parser()
    args = parser.parse_args(argv)

    if args.cmd == "version":
        sys.stdout.write(f"veillock {__version__}\n")
        return 0

    if args.cmd == "encrypt":
        frames = _load_frames(args.inp)
        if args.key:
            session_key = _parse_hex("key", args.key, 32)
        else:
            session_key = secrets.token_bytes(32)
        receiver_secret = None
        mode = Mode.parse(args.mode)
        if mode is Mode.BROADCAST:
            if args.receiver_secret:
                receiver_secret = _parse_hex("receiver-secret", args.receiver_secret)
            else:
                receiver_secret = secrets.token_bytes(32)
        try:
            session = VeilLockSession(
                session_key=session_key,
                rotation_interval=args.rotation_interval,
                mode=mode,
                receiver_secret=receiver_secret,
            )
            stream = session.encrypt_frames(frames)
            save_cipher_npz(args.out, stream)
        except (HaltedError, ValueError) as exc:
            sys.stderr.write(f"error: {exc}\n")
            return 2
        sys.stdout.write(f"session_key={session_key.hex()}\n")
        if receiver_secret is not None:
            sys.stdout.write(f"receiver_secret={receiver_secret.hex()}\n")
        sys.stdout.write(f"frames={frames.shape[0]} mode={mode.value} out={args.out}\n")
        return 0

    if args.cmd == "decrypt":
        try:
            stream = load_cipher_npz(args.inp)
        except FileNotFoundError as exc:
            sys.stderr.write(f"error: {exc}\n")
            return 2
        except Exception as exc: # noqa: BLE001
            sys.stderr.write(f"error: failed to read cipher package: {exc}\n")
            return 2

        try:
            if args.receiver_secret and stream.wrapped_key:
                secret = _parse_hex("receiver-secret", args.receiver_secret)
                session = session_from_wrapped(
                    stream.wrapped_key,
                    secret,
                    rotation_interval=stream.rotation_interval,
                    mode=stream.mode,
                )
            elif args.key:
                session_key = _parse_hex("key", args.key, 32)
                extra = {}
                if stream.mode == Mode.BROADCAST.value and args.receiver_secret:
                    extra["receiver_secret"] = _parse_hex("receiver-secret", args.receiver_secret)
                session = VeilLockSession(
                    session_key=session_key,
                    rotation_interval=stream.rotation_interval,
                    mode=stream.mode,
                    **extra,
                )
            else:
                sys.stderr.write("error: decrypt requires --key or --receiver-secret\n")
                return 2
            out = session.decrypt_frames(stream)
        except (HaltedError, DecryptError, ValueError) as exc:
            sys.stderr.write(f"error: {exc}\n")
            return 1
        np.save(args.out, out)
        sys.stdout.write(f"frames={out.shape[0]} out={args.out}\n")
        return 0

    parser.error(f"unknown command {args.cmd!r}")
    return 2

```

```
if __name__ == "__main__":  
    raise SystemExit(main())
```

forgereceipts/engines/veillock/crypto.py

```

"""Per-frame AES-256-GCM, forward-secure session-key rotation, key wrap.

frame_key = SHA-256(session_key || frame_index_le64) # 32 bytes, AES-256
nonce      = 12 bytes derived from frame_index (unique per index)
rotation   = session_key = SHA-256(session_key || b"rotate" || epoch_index_le64)
            after every rotation_interval frames; old key is dropped.
"""

from __future__ import annotations

import hashlib
import hmac
from typing import ByteString

from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

WRAP_INFO = b"veillock-wrap"
WRAP_AAD = b"veillock-keywrap"
NONCE_TAIL = b"VLCK"

class CryptoError(Exception):
    """Cipher or key-wrap failure."""

class DecryptError(CryptoError):
    """GCM authentication failed (wrong key, truncated ciphertext, or tamper)."""

def derive_frame_key(session_key: bytes, frame_index: int) -> bytes:
    """SHA-256(session_key || frame_index as little-endian uint64).

    The digest is 32 bytes; those bytes are the AES-256 key.
    """
    idx = int(frame_index).to_bytes(8, "little", signed=False)
    return hashlib.sha256(session_key + idx).digest()

def derive_nonce(frame_index: int) -> bytes:
    """12-byte GCM nonce unique for this frame_index."""
    return int(frame_index).to_bytes(8, "little", signed=False) + NONCE_TAIL

def rotate_session_key(session_key: bytes, epoch_index: int) -> bytes:
    """Forward-secure ratchet: SHA-256(session_key || b"rotate" || epoch_index_le64)."""
    epoch = int(epoch_index).to_bytes(8, "little", signed=False)
    return hashlib.sha256(session_key + b"rotate" + epoch).digest()

def aes_gcm_encrypt(frame_key: bytes, nonce: bytes, plaintext: bytes, aad: bytes) -> bytes:
    if len(frame_key) != 32:
        raise CryptoError("AES-256 key must be 32 bytes")
    if len(nonce) != 12:
        raise CryptoError("GCM nonce must be 12 bytes")
    return AESGCM(frame_key).encrypt(nonce, plaintext, aad)

def aes_gcm_decrypt(frame_key: bytes, nonce: bytes, ciphertext: bytes, aad: bytes) -> bytes:
    if len(frame_key) != 32:
        raise CryptoError("AES-256 key must be 32 bytes")
    if len(nonce) != 12:
        raise CryptoError("GCM nonce must be 12 bytes")
    try:
        return AESGCM(frame_key).decrypt(nonce, ciphertext, aad)
    except InvalidTag as exc:
        raise DecryptError("AES-GCM authentication failed") from exc

def wrap_session_key(session_key: bytes, receiver_secret: bytes) -> bytes:
    """HMAC-derived AES-GCM wrap of the session key for authorized receivers.

    KEK = SHA-256(b"veillock-wrap" || receiver_secret)
    nonce is derived from the secret so wrap is deterministic for tests.
    """
    if not receiver_secret:
        raise CryptoError("receiver_secret must be non-empty")
    kek = hashlib.sha256(WRAP_INFO + receiver_secret).digest()
    nonce = hashlib.sha256(WRAP_INFO + b"-nonce" + receiver_secret).digest()[:12]
    return AESGCM(kek).encrypt(nonce, session_key, WRAP_AAD)

def unwrap_session_key(wrapped: bytes, receiver_secret: bytes) -> bytes:
    if not receiver_secret:
        raise CryptoError("receiver_secret must be non-empty")
    kek = hashlib.sha256(WRAP_INFO + receiver_secret).digest()
    nonce = hashlib.sha256(WRAP_INFO + b"-nonce" + receiver_secret).digest()[:12]
    try:
        return AESGCM(kek).decrypt(nonce, wrapped, WRAP_AAD)
    except InvalidTag as exc:
        raise DecryptError("key-unwrap authentication failed") from exc

def hmac_confirm(session_key: bytes, receiver_secret: bytes) -> bytes:

```

```
        """Optional HMAC tag over the wrapped-key context (tests / extra AAD)."""
        return hmac.new(receiver_secret, session_key, hashlib.sha256).digest()

def zero_bytearray(buf: bytearray) -> None:
    for i in range(len(buf)):
        buf[i] = 0

def sha256_bytes(data: ByteString) -> bytes:
    return hashlib.sha256(bytes(data)).digest()
```

forgereceipts/engines/veillock/engine.py

```

"""Layer 2/3 ? Frame Encryption Engine and Trusted Decode Surface.

VeilLockSession is the library entry point.

Pipeline: render ? encrypt ? decode locally ? display.
Plaintext lives in the engine framebuffer only for the duration of
encrypt; it is zeroed before encrypt_frame returns.
"""

from __future__ import annotations

import json
import secrets
from dataclasses import dataclass, field
from typing import Any, Iterable, Mapping, Sequence

import numpy as np

from veillock.crypto import (
    DecryptError,
    aes_gcm_decrypt,
    aes_gcm_encrypt,
    derive_frame_key,
    derive_nonce,
    rotate_session_key,
    unwrap_session_key,
    wrap_session_key,
    zero_bytearray,
)
from veillock.metadata import build_headers, pack_aad
from veillock.modes import Mode, synthetic_ui_noise
from veillock.phoenix import DEFAULT_PHOENIX_THRESHOLD, PhoenixLoop
from veillock.pulse import AlwaysPass, HaltedError, PhoenixError, PulseCheck

DEFAULT_ROTATION_INTERVAL = 120
MIN_ROTATION_INTERVAL = 60
MAX_ROTATION_INTERVAL = 240

@dataclass
class EncryptedFrame:
    """One sealed visual frame plus public headers (already scrubbed)."""

    ciphertext: bytes
    nonce: bytes
    frame_index: int
    shape: tuple[int, int, int]
    epoch: int
    aad: bytes
    headers: dict[str, Any]
    decoy: np.ndarray | None = None

@dataclass
class EncryptedStream:
    """A sequence of sealed frames plus mode extras (wrapped key, decoys)."""

    frames: list[EncryptedFrame] = field(default_factory=list)
    mode: str = Mode.PRIVATE.value
    rotation_interval: int = DEFAULT_ROTATION_INTERVAL
    wrapped_key: bytes | None = None
    decoy: np.ndarray | None = None # (N, H, W, C) when mode is obfuscation

class VeilLockSession:
    """Continuous stream encryption with forward-secure key rotation.

    Parameters
    -----
    session_key:
        32-byte root key. Generated if omitted. Private mode never writes
        this key into the ciphertext package.
    rotation_interval:
        Rotate the session key every N frames. Default 120, range 60?240.
    mode:
        ``private``, ``broadcast`` (secure broadcast), or ``obfuscation``.
    pulse:
        PulseCheck consulted before any plaintext is produced.
    phoenix_threshold:
        Failed PCI / decode-mismatch count that enters Phoenix Loop.
    receiver_secret:
        For broadcast mode, wraps the *root* session key for receivers.
    """

    def __init__(
        self,
        session_key: bytes | None = None,
        rotation_interval: int = DEFAULT_ROTATION_INTERVAL,
        mode: str | Mode = Mode.PRIVATE,
        pulse: PulseCheck | None = None,
        phoenix_threshold: int = DEFAULT_PHOENIX_THRESHOLD,
        receiver_secret: bytes | None = None,
    ):
```

```

    rng: np.random.Generator | None = None,
) -> None:
    interval = int(rotation_interval)
    if not (MIN_ROTATION_INTERVAL <= interval <= MAX_ROTATION_INTERVAL):
        raise ValueError(
            f"rotation_interval must be in "
            f"[{MIN_ROTATION_INTERVAL}, {MAX_ROTATION_INTERVAL}]"
        )
    if session_key is None:
        key = secrets.token_bytes(32)
    else:
        key = bytes(session_key)
        if len(key) != 32:
            raise ValueError("session_key must be 32 bytes")

    self.rotation_interval = interval
    self.mode = Mode.parse(mode)
    self.pulse: PulseCheck = pulse if pulse is not None else AlwaysPass()
    self.phoenix = PhoenixLoop(threshold=int(phoenix_threshold))
    self.receiver_secret = bytes(receiver_secret) if receiver_secret is not None else None
    self._rng = rng if rng is not None else np.random.default_rng()

    # Root key is held only long enough to wrap for broadcast, then the
    # ratchet buffer is the sole in-memory key. We keep a wrapped copy
    # (not the raw root) for the ciphertext package.
    self._wrapped_key: bytes | None = None
    if self.mode is Mode.BROADCAST:
        if self.receiver_secret is None:
            raise ValueError("broadcast mode requires receiver_secret")
        self._wrapped_key = wrap_session_key(key, self.receiver_secret)

    self._session_key = bytearray(key)
    # Drop the local ``key`` binding; ratchet state is _session_key only.
    del key

    self._frame_index = 0
    self._epoch_index = 0
    self._in_epoch = 0
    self._framebuffer = np.zeros((0, 0, 3), dtype=np.uint8)

# --- inspected by tests -----

@property
def framebuffer(self) -> np.ndarray:
    """Engine framebuffer. After encrypt_frame this is all zeros."""
    return self._framebuffer

@property
def current_key(self) -> bytes:
    """Copy of the *current* (possibly rotated) session key.

    Compromising this value must not decrypt older epochs: previous
    keys are dropped on rotation.
    """
    return bytes(self._session_key)

@property
def frame_index(self) -> int:
    return self._frame_index

@property
def epoch_index(self) -> int:
    return self._epoch_index

@property
def wrapped_key(self) -> bytes | None:
    return self._wrapped_key

# --- gate -----

def _gate(self, *, for_decrypt: bool = False) -> None:
    """Refuse work when PCI fails or Phoenix is holding the session."""
    if self.phoenix.active:
        if self.phoenix.try_restore(self.pulse):
            self._reboot_keys()
        else:
            raise PhoenixError(
                "phoenix mode: refusing encrypt/decrypt until PCI PASS"
            )
    result = self.pulse.pci()
    if result != "PASS":
        entered = self.phoenix.record_failure()
        if entered:
            self._reboot_keys()
            raise PhoenixError(
                "phoenix mode: repeated integrity failure; session rebooted"
            )
        raise HaltedError(f"PCI did not PASS ({result!r}); no plaintext produced")

def _reboot_keys(self) -> None:
    """Phoenix reboot: drop current key material, mint a fresh session."""
    zero_bytearray(self._session_key)
    self._session_key = bytearray(secrets.token_bytes(32))
    self._frame_index = 0
    self._epoch_index = 0

```

```

        self._in_epoch = 0
        if self._framebuffer.size:
            self._framebuffer.fill(0)
        if self.mode is Mode.BROADCAST and self.receiver_secret is not None:
            self._wrapped_key = wrap_session_key(bytes(self._session_key), self.receiver_secret)

    def _advance(self) -> None:
        self._frame_index += 1
        self._in_epoch += 1
        if self._in_epoch >= self.rotation_interval:
            new_key = rotate_session_key(bytes(self._session_key), self._epoch_index)
            zero_bytearray(self._session_key)
            self._session_key = bytearray(new_key)
            self._epoch_index += 1
            self._in_epoch = 0

    def record_tamper(self) -> None:
        """External decode-mismatch / display tamper signal."""
        entered = self.phoenix.record_failure()
        if entered:
            self._reboot_keys()

# --- Layer 2 encrypt -----

    def encrypt_frame(
        self,
        frame: np.ndarray,
        metadata: Mapping[str, Any] | None = None,
    ) -> EncryptedFrame:
        """Encrypt one RGB uint8 frame. Engine framebuffer is zeroed on return."""
        self._gate()

        src = np.ascontiguousarray(frame, dtype=np.uint8)
        if src.ndim != 3 or src.shape[-1] != 3:
            raise ValueError("frame must have shape (H, W, 3) uint8")

        self._framebuffer = np.empty(src.shape, dtype=np.uint8)
        np.copyto(self._framebuffer, src)

        headers = build_headers(
            frame_index=self._frame_index,
            shape=(int(src.shape[0]), int(src.shape[1]), int(src.shape[2])),
            epoch=self._epoch_index,
            metadata=metadata,
            mode=self.mode.value,
        )
        aad = pack_aad(headers)
        frame_key = derive_frame_key(bytes(self._session_key), self._frame_index)
        nonce = derive_nonce(self._frame_index)
        plaintext_mv = memoryview(self._framebuffer).cast("B")
        ciphertext = aes_gcm_encrypt(frame_key, nonce, plaintext_mv.tobytes(), aad)

        # Drop key material for this frame and zero the engine framebuffer.
        if isinstance(frame_key, bytearray):
            zero_bytearray(frame_key)
        self._framebuffer.fill(0)

        decoy = None
        if self.mode is Mode.OBFUSCATION:
            decoy = synthetic_ui_noise(src.shape, self._rng)

        sealed = EncryptedFrame(
            ciphertext=ciphertext,
            nonce=nonce,
            frame_index=headers["frame_index"],
            shape=(headers["height"], headers["width"], head
ers["channels"]),
            epoch=headers["epoch"],
            aad=aad,
            headers=headers,
            decoy=decoy,
        )
        self._advance()
        return sealed

    def encrypt_frames(
        self,
        frames: np.ndarray | Iterable[np.ndarray],
        metadata: Sequence[Mapping[str, Any] | None] | Mapping[str, Any] | None = None,
    ) -> EncryptedStream:
        """Encrypt a stack or iterable of RGB frames."""
        stack = _as_frame_list(frames)
        sealed: list[EncryptedFrame] = []
        for i, fr in enumerate(stack):
            meta: Mapping[str, Any] | None
            if metadata is None:
                meta = None
            elif isinstance(metadata, Mapping):
                meta = metadata
            else:
                meta = metadata[i] if i < len(metadata) else None
            sealed.append(self.encrypt_frame(fr, metadata=meta))

        decoy = None
        if self.mode is Mode.OBFUSCATION and sealed:

```

```

        decoy = np.stack(
            [
                s.decoy if s.decoy is not None else np.zeros(s.shape, dtype=np.uint8)
                for s in sealed
            ],
            axis=0,
        )
    return EncryptedStream(
        frames=sealed,
        mode=self.mode.value,
        rotation_interval=self.rotation_interval,
        wrapped_key=self._wrapped_key,
        decoy=decoy,
    )

# --- Layer 3 decrypt -----

def decrypt_frame(self, sealed: EncryptedFrame) -> np.ndarray:
    """Authorized decrypt of one frame. Advances the ratchet in lockstep."""
    self._gate(for_decrypt=True)
    frame_key = derive_frame_key(bytes(self._session_key), sealed.frame_index)
    try:
        plaintext = aes_gcm_decrypt(frame_key, sealed.nonce, sealed.ciphertext, sealed.aad)
    except DecryptError:
        entered = self.phoenix.record_failure()
        if entered:
            self._reboot_keys()
            raise PhoenixError(
                "phoenix mode: decode mismatch; session rebooted"
            ) from None
        raise
    h, w, c = sealed.shape
    expected = h * w * c
    if len(plaintext) != expected:
        raise DecryptError("plaintext length does not match frame shape")
    out = np.frombuffer(plaintext, dtype=np.uint8).reshape(h, w, c).copy()
    self._advance()
    return out

def decrypt_frames(self, stream: EncryptedStream) -> np.ndarray:
    frames = [self.decrypt_frame(s) for s in stream.frames]
    if not frames:
        return np.zeros((0, 0, 0, 3), dtype=np.uint8)
    return np.stack(frames, axis=0)

def protect_frame(
    self,
    frame: np.ndarray,
    metadata: Mapping[str, Any] | None = None,
) -> tuple[EncryptedFrame, np.ndarray]:
    """Encrypt then immediately decode for the trusted local surface.

    Decrypt uses the pre-advance key (same epoch as the sealed frame)
    by reconstructing from the ciphertext with a one-frame rewind of
    the ratchet: we decrypt *before* ``_advance`` inside a dedicated
    path so the local display never needs retained epoch keys.
    """
    self._gate()
    src = np.ascontiguousarray(frame, dtype=np.uint8)
    if src.ndim != 3 or src.shape[-1] != 3:
        raise ValueError("frame must have shape (H, W, 3) uint8")

    self._framebuffer = np.empty(src.shape, dtype=np.uint8)
    np.copyto(self._framebuffer, src)
    headers = build_headers(
        frame_index=self._frame_index,
        shape=(int(src.shape[0]), int(src.shape[1]), int(src.shape[2])),
        epoch=self._epoch_index,
        metadata=metadata,
        mode=self.mode.value,
    )
    aad = pack_aad(headers)
    frame_key = derive_frame_key(bytes(self._session_key), self._frame_index)
    nonce = derive_nonce(self._frame_index)
    ciphertext = aes_gcm_encrypt(
        frame_key, nonce, memoryview(self._framebuffer).cast("B").tobytes(), aad
    )
    display = aes_gcm_decrypt(frame_key, nonce, ciphertext, aad)
    self._framebuffer.fill(0)
    h, w, c = headers["height"], headers["width"], headers["channels"]
    display_arr = np.frombuffer(display, dtype=np.uint8).reshape(h, w, c).copy()
    decoy = synthetic_ui_noise(src.shape, self._rng) if self.mode is Mode.OBFUSCATION else None
    sealed = EncryptedFrame(
        ciphertext=ciphertext,
        nonce=nonce,
        frame_index=headers["frame_index"],
        shape=(h, w, c),
        epoch=headers["epoch"],
        aad=aad,
        headers=headers,
        decoy=decoy,
    )
    self._advance()
    return sealed, display_arr

def _as_frame_list(frames: np.ndarray | Iterable[np.ndarray]) -> list[np.ndarray]:

```



```

if isinstance(frames, np.ndarray):
    if frames.ndim == 4:
        return [frames[i] for i in range(frames.shape[0])]
    if frames.ndim == 3:
        return [frames]
    raise ValueError("ndarray frames must be (H,W,3) or (N,H,W,3)")
return [np.ascontiguousarray(f, dtype=np.uint8) for f in frames]

def save_cipher_npz(path: str, stream: EncryptedStream) -> None:
    """Write an EncryptedStream without pickle (concatenated blobs + lengths)."""
    blobs = [f.ciphertext for f in stream.frames]
    aads = [f.aad for f in stream.frames]
    headers = [f.headers for f in stream.frames]
    blob = np.frombuffer(b"".join(blobs), dtype=np.uint8) if blobs else np.zeros((0,), dtype=np.uint8)
    aad_blob = np.frombuffer(b"".join(aads), dtype=np.uint8) if aads else np.zeros((0,), dtype=np.uint8)
    headers_json = json.dumps(headers, separators=(",", ":"), default=_json_default).encode("utf-8")
    decoy = stream.decoy if stream.decoy is not None else np.zeros((0,), dtype=np.uint8)
    wrapped = (
        np.frombuffer(stream.wrapped_key, dtype=np.uint8)
        if stream.wrapped_key is not None
        else np.zeros((0,), dtype=np.uint8)
    )
    n = len(stream.frames)
    if n:
        shape = np.array([n, *stream.frames[0].shape], dtype=np.int64)
        indices = np.array([f.frame_index for f in stream.frames], dtype=np.int64)
        epochs = np.array([f.epoch for f in stream.frames], dtype=np.int64)
        nonces = np.frombuffer(b"".join(f.nonce for f in stream.frames), dtype=np.uint8)
    else:
        shape = np.array([0, 0, 0, 3], dtype=np.int64)
        indices = np.zeros((0,), dtype=np.int64)
        epochs = np.zeros((0,), dtype=np.int64)
        nonces = np.zeros((0,), dtype=np.uint8)
    np.savez_compressed(
        path,
        blob=blob,
        lengths=np.array([len(b) for b in blobs], dtype=np.int64),
        aad_blob=aad_blob,
        aad_lengths=np.array([len(a) for a in aads], dtype=np.int64),
        headers_json=np.frombuffer(headers_json, dtype=np.uint8),
        indices=indices,
        epochs=epochs,
        nonces=nonces,
        shape=shape,
        mode=np.array(stream.mode),
        rotation_interval=np.int64(stream.rotation_interval),
        wrapped_key=wrapped,
        decoy=decoy,
    )

def load_cipher_npz(path: str) -> EncryptedStream:
    with np.load(path, allow_pickle=False) as z:
        lengths = z["lengths"]
        blob = bytes(z["blob"])
        aad_lengths = z["aad_lengths"]
        aad_blob = bytes(z["aad_blob"])
        headers = json.loads(bytes(z["headers_json"]).decode("utf-8") or "[]")
        indices = z["indices"]
        epochs = z["epochs"]
        nonces_b = bytes(z["nonces"])
        shape = [int(x) for x in z["shape"]]
        mode = str(z["mode"])
        rotation_interval = int(z["rotation_interval"])
        wrapped_raw = bytes(z["wrapped_key"])
        wrapped_key = wrapped_raw if len(wrapped_raw) else None
        decoy_arr = z["decoy"]
        decoy = decoy_arr if decoy_arr.ndim == 4 else None

    frames: list[EncryptedFrame] = []
    off = 0
    aoff = 0
    noff = 0
    n = int(shape[0])
    hw = (int(shape[1]), int(shape[2]), int(shape[3])) if n else (0, 0, 3)
    for i in range(n):
        clen = int(lengths[i])
        alen = int(aad_lengths[i])
        ct = blob[off : off + clen]
        aad = aad_blob[aoff : aoff + alen]
        nonce = nonces_b[noff : noff + 12]
        off += clen
        aoff += alen
        noff += 12
        hdr = headers[i] if i < len(headers) else {}
        sh = (
            int(hdr.get("height", hw[0])),
            int(hdr.get("width", hw[1])),
            int(hdr.get("channels", hw[2])),
        )
        decoy_i = decoy[i] if decoy is not None else None
        frames.append(
            EncryptedFrame(
                ciphertext=ct,

```

```

        nonce=nonce,
        frame_index=int(indices[i]),
        shape=sh,
        epoch=int(epochs[i]),
        aad=aad,
        headers=hdr,
        decoy=decoy_i,
    )
)
return EncryptedStream(
    frames=frames,
    mode=mode,
    rotation_interval=rotation_interval,
    wrapped_key=wrapped_key,
    decoy=decoy,
)

def _json_default(obj: Any) -> Any:
    if isinstance(obj, (bytes, bytearray)):
        return bytes(obj).hex()
    if isinstance(obj, np.integer):
        return int(obj)
    raise TypeError(f"not JSON serializable: {type(obj)!r}")

def session_from_wrapped(
    wrapped_key: bytes,
    receiver_secret: bytes,
    **kwargs: Any,
) -> VeilLockSession:
    """Open a decrypt session from a broadcast wrapped root key."""
    root = unwrap_session_key(wrapped_key, receiver_secret)
    return VeilLockSession(
        session_key=root,
        mode=kwargs.pop("mode", Mode.BROADCAST),
        receiver_secret=receiver_secret,
        **kwargs,
    )

```

forgereceipts/engines/veillock/frames.py

```

"""Layer 1 ? Render Capture.

In software this is a FrameSource that yields RGB uint8 numpy frames.
PCI is checked *before* a frame is yielded so a failing gate never
produces plaintext.
"""

from __future__ import annotations

from typing import Iterator

import numpy as np

from veillock.pulse import AlwaysPass, HaltedError, PulseCheck

class FrameSource:
    """Yield RGB uint8 frames from an in-memory (N, H, W, 3) stack.

    This is the software stand-in for intercepting visual frames before
    they reach a GPU display. It is not a screen scraper: it only emits
    frames the caller already constructed.
    """

    def __init__(
        self,
        frames: np.ndarray,
        pulse: PulseCheck | None = None,
    ) -> None:
        arr = np.ascontiguousarray(frames, dtype=np.uint8)
        if arr.ndim != 4 or arr.shape[-1] != 3:
            raise ValueError("FrameSource expects an RGB stack of shape (N, H, W, 3)")
        self._frames = arr
        self._pulse: PulseCheck = pulse if pulse is not None else AlwaysPass()
        self._i = 0

    def __len__(self) -> int:
        return int(self._frames.shape[0])

    def __iter__(self) -> Iterator[np.ndarray]:
        self._i = 0
        return self

    def __next__(self) -> np.ndarray:
        if self._pulse.pci() != "PASS":
            raise HaltedError("PCI did not PASS; frame generation halted")
        if self._i >= self._frames.shape[0]:
            raise StopIteration
        frame = self._frames[self._i]
        self._i += 1
        return frame

    @property
    def shape(self) -> tuple[int, int, int, int]:
        return tuple(int(x) for x in self._frames.shape) # type: ignore[return-value]

```

forgereceipts/engines/veillock/metadata.py

```

"""Strip window identifiers, application fingerprints, and UI telemetry.

Scrubbed keys never enter AES-GCM associated data or public headers.
"""

from __future__ import annotations

import json
import struct
from typing import Any, Mapping

# Exact names plus common aliases. Tests assert these are absent from AAD/headers.
SCRUB_KEYS = frozenset(
    {
        "window_id",
        "window_ids",
        "window_identifier",
        "window_identifiers",
        "app_fingerprint",
        "application_fingerprint",
        "application_fingerprints",
        "ui_telemetry",
        "telemetry",
        "app_id",
        "hwnd",
        "process_id",
        "pid",
    }
)

_HEADER_MAGIC = b"VLCK"
_HEADER_VERSION = 1

def scrub_metadata(meta: Mapping[str, Any] | None) -> dict[str, Any]:
    """Return a new dict with identifying / telemetry keys removed."""
    if not meta:
        return {}
    out: dict[str, Any] = {}
    for key, value in meta.items():
        if str(key).lower() in SCRUB_KEYS:
            continue
        out[str(key)] = value
    return out

def build_headers(
    frame_index: int,
    shape: tuple[int, int, int],
    epoch: int,
    metadata: Mapping[str, Any] | None,
    mode: str,
) -> dict[str, Any]:
    """Public headers: geometry + mode + scrubbed metadata. No fingerprints."""
    h, w, c = (int(shape[0]), int(shape[1]), int(shape[2]))
    headers: dict[str, Any] = {
        "frame_index": int(frame_index),
        "height": h,
        "width": w,
        "channels": c,
        "epoch": int(epoch),
        "mode": str(mode),
    }
    headers.update(scrub_metadata(metadata))
    return headers

def pack_aad(headers: Mapping[str, Any]) -> bytes:
    """Canonical associated data: fixed binary prefix plus JSON of headers.

    The JSON object is sorted by key so AAD is stable. Scrubbed keys are
    already absent from ``headers``.
    """
    h = int(headers["height"])
    w = int(headers["width"])
    c = int(headers["channels"])
    epoch = int(headers["epoch"])
    index = int(headers["frame_index"])
    prefix = struct.pack("<4sBIIQQ", _HEADER_MAGIC, _HEADER_VERSION, h, w, c, epoch, index)
    body = json.dumps(headers, sort_keys=True, separators=(",", ":"), default=str).encode("utf-8")
    return prefix + body

```

forgereceipts/engines/veillock/modes.py

```

"""Deployment modes: private, secure_broadcast, obfuscation."""

from __future__ import annotations

from enum import Enum

import numpy as np

class Mode(str, Enum):
    PRIVATE = "private"
    BROADCAST = "broadcast"
    OBFUSCATION = "obfuscation"

    @classmethod
    def parse(cls, value: str | Mode) -> Mode:
        if isinstance(value, cls):
            return value
        key = str(value).strip().lower().replace("-", "_")
        if key in ("secure_broadcast", "securebroadcast"):
            key = "broadcast"
        try:
            return cls(key)
        except ValueError as exc:
            raise ValueError(
                "mode must be private, broadcast (secure_broadcast), or obfuscation"
            ) from exc

def synthetic_ui_noise(
    shape: tuple[int, int, int],
    rng: np.random.Generator,
) -> np.ndarray:
    """Structured decoy frame (fake windows / panels), not plaintext and not GCM snow."""
    h, w, c = int(shape[0]), int(shape[1]), int(shape[2])
    bg = rng.integers(24, 64, size=(c,), dtype=np.uint8)
    frame = np.empty((h, w, c), dtype=np.uint8)
    frame[:, :] = bg
    n_rect = int(rng.integers(2, 6))
    for _ in range(n_rect):
        y1 = int(rng.integers(0, max(h, 1)))
        x1 = int(rng.integers(0, max(w, 1)))
        y2 = int(rng.integers(y1 + 1, h + 1))
        x2 = int(rng.integers(x1 + 1, w + 1))
        color = rng.integers(80, 210, size=(c,), dtype=np.uint8)
        frame[y1:y2, x1:x2] = color
        # title-bar strip on the fake window
        bar = min(y1 + max(1, h // 16), y2)
        bar_color = rng.integers(40, 120, size=(c,), dtype=np.uint8)
        frame[y1:bar, x1:x2] = bar_color
    return frame

```

forgereceipts/engines/veillock/phoenix.py

```

"""Phoenix Loop: reboot the display session after repeated tampering.

If N failed integrity checks or decode mismatches are observed, enter
phoenix mode: reset session keys and refuse encrypt/decrypt until PCI
returns PASS. No plaintext frame is produced while phoenix is active.
"""

from __future__ import annotations

from veillock.pulse import PulseCheck

DEFAULT_PHOENIX_THRESHOLD = 3


class PhoenixLoop:
    """Count integrity failures and gate the display session."""

    def __init__(self, threshold: int = DEFAULT_PHOENIX_THRESHOLD) -> None:
        if threshold < 1:
            raise ValueError("phoenix threshold must be >= 1")
        self.threshold = int(threshold)
        self.failures = 0
        self.active = False

    def record_failure(self) -> bool:
        """Record a failed integrity check or decode mismatch.

        Returns True if this call caused entry into phoenix mode.
        """
        self.failures += 1
        if self.failures >= self.threshold:
            self.active = True
            return True
        return False

    def try_restore(self, pulse: PulseCheck) -> bool:
        """If active and ``pulse.pci()`` is PASS, exit phoenix.

        Returns True if the loop restored. Does nothing if not active.
        """
        if not self.active:
            return False
        if pulse.pci() == "PASS":
            self.active = False
            self.failures = 0
            return True
        return False

    def reset_counters(self) -> None:
        self.failures = 0
        self.active = False

```

forgereceipts/engines/veillock/pulse.py

```

"""TemporalLock integrity gating (PulseCheck).

A PulseCheck returns "PASS" or a failure token. If PCI does not PASS,
frame generation HALTS and no plaintext frame is produced.
"""

from __future__ import annotations

from typing import Protocol, runtime_checkable

class HaltedError(Exception):
    """Frame generation halted; no plaintext frame was produced."""

class PhoenixError(HaltedError):
    """Phoenix loop is active: session rebooted, plaintext refused until PCI PASS."""

@runtime_checkable
class PulseCheck(Protocol):
    """Integrity gate consulted before any plaintext frame is produced."""

    def pci(self) -> str:
        """Return ``"PASS"`` on success, or any other string on failure."""
        ...

class AlwaysPass:
    """Default PulseCheck: always returns PASS."""

    def pci(self) -> str:
        return "PASS"

class CallablePulse:
    """Wrap a zero-arg callable as a PulseCheck."""

    def __init__(self, fn):
        self._fn = fn

    def pci(self) -> str:
        return self._fn()

```

forgereceipts/engines/veillock/ui.py

```

"""Localhost UI for VeilLock. Binds 127.0.0.1. No CDN, no outbound calls."""

from __future__ import annotations

import base64
import json
import secrets
import sys
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from typing import Any
from urllib.parse import urlparse

import numpy as np

from veillock import __version__
from veillock.engine import VeilLockSession
from veillock.modes import Mode

DEFAULT_HOST = "127.0.0.1"
DEFAULT_PORT = 8761
MAX_BODY = 2 * 1024 * 1024

PAGE = r"""<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>VeilLock</title>
<style>
:root {
  --bg: #0d0b14; --card: #171427; --ink: #efeaf8; --muted: #9a91b3;
  --line: #2c2744; --accent: #a78bfa; --accent-dim: #3b2d66; --warn: #e0b46a;
}
* { box-sizing: border-box; }
html, body { margin: 0; background: var(--bg); color: var(--ink);
  font-family: "Iowan Old Style", Palatino, Georgia, serif; }
main { max-width: 46rem; margin: 0 auto; padding: 2.4rem 1.25rem 4rem; }
.mark { font-size: 0.72rem; letter-spacing: 0.22em; text-transform: uppercase;
  color: var(--accent); margin: 0 0 0.45rem; }
h1 { font-weight: 500; letter-spacing: 0.04em; font-size: 2.1rem; margin: 0 0 0.4rem; }
.motto { font-style: italic; color: var(--muted); margin: 0; }
.local { display: inline-block; margin-top: 0.85rem; font-size: 0.78rem;
  color: var(--muted); border: 1px solid var(--line); padding: 0.2rem 0.55rem;
  border-radius: 999px; font-family: ui-monospace, monospace; }
.card { background: var(--card); border: 1px solid var(--line); border-radius: 12px;
  padding: 1.15rem 1.2rem 1.25rem; margin: 1.1rem 0; }
h2 { font-size: 1.02rem; font-weight: 500; margin: 0 0 0.75rem; }
p.help { color: var(--muted); font-size: 0.92rem; margin: 0 0 0.9rem; }
label { display: block; font-size: 0.82rem; color: var(--muted); margin: 0.55rem 0 0.28rem; }
select { width: 100%; background: #0d0b14; color: var(--ink); border: 1px solid var(--line);
  padding: 0.45rem 0.55rem; border-radius: 6px; }
button.primary { font-family: inherit; cursor: pointer; border: 0; border-radius: 8px;
  padding: 0.55rem 1.15rem; background: var(--accent); color: #1a1230; font-weight: 600; }
button.disabled { opacity: 0.5; }
.grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(11rem, 1fr)); gap: 0.85rem; }
canvas { width: 100%; image-rendering: pixelated; background: #000; border: 1px solid var(--line);
  border-radius: 6px; }
.cap { font-size: 0.78rem; color: var(--muted); margin: 0.3rem 0 0; }
.key { font-family: ui-monospace, monospace; font-size: 0.75rem; word-break: break-all;
  background: #0d0b14; border: 1px dashed var(--line); padding: 0.6rem; border-radius: 6px; }
.hex { font-family: ui-monospace, monospace; font-size: 0.72rem; color: var(--muted);
  word-break: break-all; }
.err { color: #e07a7a; }
footer { margin-top: 2rem; color: #6d6584; font-size: 0.8rem; }
</style>
</head>
<body>
<main>
  <header>
    <p class="mark">Aziel Eliab ? July 2026</p>
    <h1>VeilLock</h1>
    <p class="motto">Render ? encrypt ? decode locally ? display.</p>
    <span class="local">localhost ? 127.0.0.1 ? key shown once</span>
  </header>
  <section class="card">
    <h2>Seal synthetic frames</h2>
    <p class="help">Generates a tiny RGB stack here, encrypts it, and shows ciphertext as noise against the decrypted
    preview. The session key is printed once and not stored.</p>
    <label>Mode</label>
    <select id="mode">
      <option value="private">private</option>
      <option value="broadcast">broadcast</option>
      <option value="obfuscation">obfuscation</option>
    </select>
    <p style="margin-top:0.95rem"><button class="primary" id="go" type="button">Seal frames</button></p>
  </section>
  <section class="card" id="result" hidden>
    <h2>Session</h2>
    <p class="help" id="meta"></p>
    <p class="key" id="key"></p>
    <div class="grid" style="margin-top:1rem">
      <div><canvas id="plain" width="64" height="48"></canvas><p class="cap">plaintext (synthetic)</p></div>

```



```

        <div><canvas id="cipher" width="64" height="48"></canvas><p class="cap">ciphertext as pixels</p></div>
        <div><canvas id="dec" width="64" height="48"></canvas><p class="cap">decrypted preview</p></div>
        <div id="decoy-wrap" hidden><canvas id="decoy" width="64" height="48"></canvas><p class="cap">obfuscation decoy<
    /p></div>
    </div>
    <p class="cap">Ciphertext hex (first 96 bytes)</p>
    <p class="hex" id="hex"></p>
    </section>
    <p class="err" id="err" hidden></p>
    <footer>VeilLock __VERSION__ ? Apache-2.0 ? forks welcome ? <code>veillock ui</code></footer>
</main>
<script>
(function () {
    const $ = (id) => document.getElementById(id);
    function draw(canvas, b64, w, h) {
        const raw = Uint8Array.from(atob(b64), c => c.charCodeAt(0));
        canvas.width = w; canvas.height = h;
        const ctx = canvas.getContext("2d");
        const img = ctx.createImageData(w, h);
        let n = w * h;
        for (let i = 0, j = 0; i < n; i++) {
            img.data[j++] = raw[i*3] || 0;
            img.data[j++] = raw[i*3+1] || 0;
            img.data[j++] = raw[i*3+2] || 0;
            img.data[j++] = 255;
        }
        ctx.putImageData(img, 0, 0);
    }
    $("go").onclick = async () => {
        $("err").hidden = true;
        $("go").disabled = true;
        try {
            const res = await fetch("/api/demo", {
                method: "POST",
                headers: {"Content-Type": "application/json"},
                body: JSON.stringify({mode: $("mode").value}),
            });
            const data = await res.json();
            if (!res.ok) throw new Error(data.error || ("HTTP " + res.status));
            $("result").hidden = false;
            const w = data.width, h = data.height;
            $("meta").textContent = data.frames + " frames ? " + w + "?" + h + " ? mode " + data.mode
                + " ? ciphertext looks like noise; decrypt matches the synthetic source.";
            let key = "session_key (shown once)\\n" + data.session_key;
            if (data.receiver_secret) key += "\\nreceiver_secret (shown once)\\n" + data.receiver_secret;
            $("key").textContent = key;
            draw($("plain"), data.plain_b64, w, h);
            draw($("cipher"), data.cipher_b64, w, h);
            draw($("dec"), data.decrypt_b64, w, h);
            if (data.decoy_b64) {
                $("decoy-wrap").hidden = false;
                draw($("decoy"), data.decoy_b64, w, h);
            } else { $("decoy-wrap").hidden = true; }
            $("hex").textContent = data.cipher_hex;
        } catch (e) {
            $("err").hidden = false;
            $("err").textContent = String(e.message || e);
        } finally { $("go").disabled = false; }
    };
})();
</script>
</body>
</html>
""".replace("__VERSION__", __version__)

```

```

def _rgb_b64(frame: np.ndarray) -> str:
    arr = np.ascontiguousarray(frame, dtype=np.uint8)
    return base64.b64encode(arr.tobytes()).decode("ascii")

def _synthetic_frames(n: int = 3, h: int = 48, w: int = 64) -> np.ndarray:
    frames = np.zeros((n, h, w, 3), dtype=np.uint8)
    for i in range(n):
        frames[i, :, :] = (18, 16, 36)
        x = 6 + i * 8
        frames[i, 10:38, x : x + 22] = (40 + i * 50, 170, 110)
        frames[i, 0:5, :] = (190, 150, 70)
        frames[i, h - 4 : h, :] = (50, 70, 120)
        frames[i, :, 0:3] = (90, 50, 140)
    return frames

def _cipher_as_rgb(ciphertext: bytes, h: int, w: int) -> np.ndarray:
    need = h * w * 3
    buf = (ciphertext * ((need // max(len(ciphertext), 1)) + 1))[ :need]
    return np.frombuffer(buf, dtype=np.uint8).reshape(h, w, 3).copy()

class Handler(BaseHTTPRequestHandler):
    def log_message(self, fmt: str, *args: object) -> None:
        sys.stderr.write("%s - %s\n" % (self.address_string(), fmt % args))

    def _send(self, code: int, body: bytes, content_type: str) -> None:
        self.send_response(code)

```

```

        self.send_header("Content-Type", content_type)
        self.send_header("Content-Length", str(len(body)))
        self.send_header("Cache-Control", "no-store")
        self.end_headers()
        self.wfile.write(body)

def _json(self, code: int, obj: Any) -> None:
    self._send(code, json.dumps(obj).encode("utf-8"), "application/json; charset=utf-8")

def do_GET(self) -> None: # noqa: N802
    path = urlparse(self.path).path
    if path in ("/", "/index.html"):
        self._send(200, PAGE.encode("utf-8"), "text/html; charset=utf-8")
        return
    if path == "/health":
        self._json(200, {"ok": True, "bind_host": DEFAULT_HOST, "name": "VeilLock"})
        return
    self._json(404, {"error": "not found"})

def do_POST(self) -> None: # noqa: N802
    path = urlparse(self.path).path
    if path != "/api/demo":
        self._json(404, {"error": "not found"})
        return
    length = int(self.headers.get("Content-Length") or 0)
    if length > MAX_BODY:
        self._json(400, {"error": "payload too large"})
        return
    try:
        body = json.l
oads(self.rfile.read(length).decode("utf-8") or "{}") if length else {}
        mode = Mode.parse(str(body.get("mode") or "private"))
        frames = _synthetic_frames()
        key = secrets.token_bytes(32)
        receiver = secrets.token_bytes(32) if mode is Mode.BROADCAST else None
        enc = VeilLockSession(
            session_key=key,
            rotation_interval=60,
            mode=mode,
            receiver_secret=receiver,
        )
        stream = enc.encrypt_frames(frames)
        dec = VeilLockSession(
            session_key=key,
            rotation_interval=60,
            mode=mode,
            receiver_secret=receiver,
        )
        out = dec.decrypt_frames(stream)
        sealed = stream.frames[0]
        h, w, _ = sealed.shape
        cipher_rgb = _cipher_as_rgb(sealed.ciphertext, h, w)
        decoy_b64 = None
        if stream.decoy is not None and len(stream.decoy):
            decoy_b64 = _rgb_b64(stream.decoy[0])
        self._json(
            200,
            {
                "mode": mode.value,
                "frames": int(frames.shape[0]),
                "height": h,
                "width": w,
                "session_key": key.hex(),
                "receiver_secret": receiver.hex() if receiver is not None else None,
                "plain_b64": _rgb_b64(frames[0]),
                "cipher_b64": _rgb_b64(cipher_rgb),
                "decrypt_b64": _rgb_b64(out[0]),
                "decoy_b64": decoy_b64,
                "cipher_hex": sealed.ciphertext[:96].hex(),
                "match": bool(np.array_equal(out, frames)),
            },
        )
    except Exception as exc: # noqa: BLE001
        self._json(400, {"error": str(exc)})

def make_server(host: str = DEFAULT_HOST, port: int = DEFAULT_PORT) -> ThreadingHTTPServer:
    return ThreadingHTTPServer((host, port), Handler)

def serve(host: str = DEFAULT_HOST, port: int = DEFAULT_PORT) -> None:
    httpd = make_server(host, port)
    sys.stdout.write(f"VeilLock UI http://{host}:{port}/\n")
    sys.stdout.write("Local only. Session key is shown once and not written to disk.\n")
    sys.stdout.flush()
    try:
        httpd.serve_forever()
    except KeyboardInterrupt:
        sys.stdout.write("\nstopped\n")
    finally:
        httpd.server_close()

```

forgereceipts/engines/vibelock/__init__.py

```
"""VibeLock: physical-consistency evaluation of speech audio.

July 2026 whitepaper implementation by Aziel Eliab.

VibeLock asks whether a recording is physically consistent with human vocal
vibration and biomechanical resonance. Dual-channel mode uses air audio plus
body-coupled vibration. Audio-only mode is a forensic risk assessment, not a
proof of liveness.

Sound can be forged. Physics is harder to fake.
"""

from __future__ import annotations

from vibelock.scoring import AnalysisResult, CheckResult, analyze

__version__ = "0.1.0"
__author__ = "Azriel Eliab"
__all__ = ["AnalysisResult", "CheckResult", "analyze", "__version__"]
```

forgereceipts/engines/vibelock/__main__.py

```
"""Allow ``python -m vibelock`` to invoke the CLI."""

from vibelock.cli import main

if __name__ == "__main__":
    raise SystemExit(main())
```

forgereceipts/engines/vibelock/cli.py

```

"""Command-line interface for VibeLock.

    vibelock analyze AUDIO [--vibration FILE] [--sr HZ] [--json]
    vibelock ui [--host 127.0.0.1] [--port 8760]
    vibelock version

Analyze always exits 0 on a completed run (including low scores). A
nonzero exit is reserved for usage / I/O errors so scripts can tell
"this recording looks synthetic" from "the tool failed."
"""

from __future__ import annotations

import argparse
import json
import sys
from typing import Sequence

from vibelock import __version__
from vibelock.io import load_audio
from vibelock.scoring import analyze, format_human


def _build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="vibelock",
        description=(
            "VibeLock ? evaluate whether audio is physically consistent "
            "with human vocal vibration (Aziel Eliab, July 2026).",
        ),
    )
    sub = parser.add_subparsers(dest="cmd", required=True)

    p_an = sub.add_parser("analyze", help="Score a recording (dual-channel or audio-only).")
    p_an.add_argument("audio", help="Path to air-microphone WAV.")
    p_an.add_argument(
        "--vibration",
        "-v",
        default=None,
        help="Optional body-coupled vibration WAV (jaw accel / contact mic / IMU).",
    )
    p_an.add_argument(
        "--sr",
        type=int,
        default=None,
        metavar="HZ",
        help="Resample both channels to this rate before analysis.",
    )
    p_an.add_argument(
        "--json",
        action="store_true",
        help="Machine-readable JSON (score, reason codes, per-check metrics).",
    )

    sub.add_parser("version", help="Print the VibeLock version and exit.")

    p_ui = sub.add_parser("ui", aliases=["serve"], help="Run the localhost UI (127.0.0.1).")
    p_ui.add_argument("--host", default="127.0.0.1", help="Bind host (default 127.0.0.1).")
    p_ui.add_argument("--port", type=int, default=8760, help="Bind port (default 8760).")
    return parser


def main(argv: Sequence[str] | None = None) -> int:
    parser = _build_parser()
    args = parser.parse_args(argv)

    if args.cmd == "version":
        sys.stdout.write(f"vibelock {__version__}\n")
        return 0

    if args.cmd in ("ui", "serve"):
        from vibelock.ui import serve

        serve(host=args.host, port=args.port)
        return 0

    if args.cmd == "analyze":
        try:
            audio, sr = load_audio(args.audio, target_sr=args.sr)
        except FileNotFoundError as exc:
            sys.stderr.write(f"error: {exc}\n")
            return 2
        except Exception as exc:  # noqa: BLE001 ? surface wav decode problems
            sys.stderr.write(f"error: failed to read audio: {exc}\n")
            return 2

        vibration = None
        if args.vibration:
            try:
                vibration, vsr = load_audio(args.vibration, target_sr=sr)
            except FileNotFoundError as exc:
                sys.stderr.write(f"error: {exc}\n")

```

```
        return 2
    except Exception as exc: # noqa: BLE001
        sys.stderr.write(f"error: failed to read vibration: {exc}\n")
        return 2
    if vsr != sr:
        # load_audio already resampled to `sr` via target_sr.
        pass

    result = analyze(audio, sr, vibration=vibration)
    if args.json:
        sys.stdout.write(json.dumps(result.to_dict(), indent=2))
        sys.stdout.write("\n")
    else:
        sys.stdout.write(format_human(result))
        sys.stdout.write("\n")
    return 0

parser.error(f"unknown command {args.cmd}")
return 2

if __name__ == "__main__":
    raise SystemExit(main())
```

forgereceipts/engines/vibelock/dsp.py

```

"""Low-level DSP primitives for VibeLock.

Implements the signal-processing building blocks cited in
`docs/whitepaper.md`: magnitude-squared coherence, Hilbert phase,
LPC formant estimates, transfer-function estimation, GCC-PHAT delay,
and exponential decay envelopes.

This module is CPU-local (numpy + scipy only). It never performs
speech-to-text and never retains audio.
"""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from numpy.typing import NDArray
from scipy import signal
from scipy.linalg import solve_toeplitz, toeplitz

# Speech-relevant band used throughout dual-channel tests
# (whitepaper: roughly 80?4000 Hz).
SPEECH_BAND_HZ: tuple[float, float] = (80.0, 4000.0)

# Overlap band where body-coupled vibration and air radiation
# are both expected to carry energy.
COUPLING_BAND_HZ: tuple[float, float] = (80.0, 2000.0)

# Anatomical delay window: tissue vibration and lip radiation of the
# same glottal event differ by vocal-tract group delay (a few ms), not
# tens of milliseconds. Implementation default, not a published ROC point.
PLAUSIBLE_DELAY_S: tuple[float, float] = (-0.008, 0.030)

# Closed-tube / formant ringing time constants are short.
# tau ? 1 / (pi * bandwidth); BW 50?300 Hz ? ~1?6 ms.
# Digital reverb and synthetic ringing sit far above this.
PLAUSIBLE_DECAY_TAU_S: tuple[float, float] = (0.001, 0.080)

Array = NDArray[np.float64]

def as_mono_float(x: np.ndarray) -> Array:
    """Return a 1-D float64 vector. Stereo is averaged."""
    arr = np.asarray(x, dtype=np.float64)
    if arr.ndim == 2:
        arr = arr.mean(axis=1 if arr.shape[1] < arr.shape[0] else 0)
    arr = np.ravel(arr)
    if arr.size == 0:
        return arr
    peak = np.max(np.abs(arr))
    if peak > 8.0:
        # Integer PCM that wasn't scaled.
        arr = arr / max(peak, 1.0)
    return arr

def rms(x: np.ndarray) -> float:
    x = np.asarray(x, dtype=np.float64)
    if x.size == 0:
        return 0.0
    return float(np.sqrt(np.mean(x * x)))

def preemphasis(x: np.ndarray, coeff: float = 0.97) -> Array:
    x = np.asarray(x, dtype=np.float64)
    if x.size < 2:
        return x.copy()
    y = np.empty_like(x)
    y[0] = x[0]
    y[1:] = x[1:] - coeff * x[:-1]
    return y

def resample(x: np.ndarray, src_sr: int, dst_sr: int) -> Array:
    """Resample with polyphase filtering when rates differ."""
    x = as_mono_float(x)
    if src_sr == dst_sr or x.size == 0:
        return x
    g = np.gcd(int(src_sr), int(dst_sr))
    up, down = int(dst_sr) // g, int(src_sr) // g
    return np.asarray(signal.resample_poly(x, up, down), dtype=np.float64)

def bandpass(x: np.ndarray, sr: int, lo: float, hi: float, order: int = 4) -> Array:
    """Zero-phase Butterworth band-pass. Falls back to high/low-pass at edges."""
    x = as_mono_float(x)
    nyq = 0.5 * sr
    lo_n = max(lo / nyq, 1e-4)
    hi_n = min(hi / nyq, 0.999)
    if hi_n <= lo_n + 0.01:
        return x.copy()

```

```

    sos = signal.butter(order, [lo_n, hi_n], btype="band", output="sos")
    return np.asarray(signal.sosfiltfilt(sos, x), dtype=np.float64)

def frame_signal(
    x: np.ndarray,
    sr: int,
    frame_ms: float = 25.0,
    hop_ms: float = 10.0,
    window: str = "hann",
) -> tuple[Array, int, int]:
    """Return overlapping frames (n_frames, n_win), window length, hop."""
    x = as_mono_float(x)
    n_win = max(8, int(round(frame_ms * 0.001 * sr)))
    hop = max(1, int(round(hop_ms * 0.001 * sr)))
    if x.size < n_win:
        pad = np.zeros(n_win, dtype=np.float64)
        pad[: x.size] = x
        frames = pad[np.newaxis, :]
        if window:
            frames = frames * signal.get_window(window, n_win, fftbins=True)
        return frames, n_win, hop
    n_frames = 1 + (x.size - n_win) // hop
    idx = np.arange(n_win)[None, :] + hop * np.arange(n_frames)[:, None]
    frames = x[idx]
    if window:
        frames = frames * signal.get_window(window, n_win, fftbins=True)
    return frames, n_win, hop

def next_pow2(n: int) -> int:
    n = max(int(n), 1)
    return 1 << (n - 1).bit_length()

def welch_nperseg(n: int, sr: int) -> int:
    """Choose a Welch segment length that still yields several averages."""
    target = int(round(0.032 * sr)) # ~32 ms
    nper = max(64, next_pow2(target))
    nper = min(nper, max(64, n // 4))
    return int(max(64, nper))

def magnitude_squared_coherence(
    x: np.ndarray,
    y: np.ndarray,
    sr: int,
    nperseg: int | None = None,
) -> tuple[Array, Array]:
    """Welch magnitude-squared coherence Cxy(f) in [0, 1].

    Whitepaper: vibration?audio coherence in speech-relevant bands.
    Authentic coupled sources produce stable high coherence where both
    channels have energy; independent or vocoded substitutes do not.
    """
    x = as_mono_float(x)
    y = as_mono_float(y)
    n = min(x.size, y.size)
    x, y = x[:n], y[:n]
    if n < 128:
        freqs = np.fft.rfftfreq(256, 1.0 / sr)
        return freqs, np.zeros_like(freqs)
    nper = nperseg or welch_nperseg(n, sr)
    nper = min(nper, n // 2)
    with np.errstate(invalid="ignore", divide="ignore"):
        freqs, cxy = signal.coherence(x, y, fs=sr, nperseg=nper, noverlap=nper // 2)
    cxy = np.nan_to_num(np.asarray(cxy, dtype=np.float64), nan=0.0, posinf=0.0, neginf=0.0)
    return np.asarray(freqs, dtype=np.float64), cxy

def band_mean(freqs: np.ndarray, values: np.ndarray, lo: float, hi: float) -> float:
    mask = (freqs >= lo) & (freqs <= hi)
    if not np.any(mask):
        return 0.0
    v = values[mask]
    v = v[np.isfinite(v)]
    if v.size == 0:
        return 0.0
    return float(np.mean(v))

def band_std(freqs: np.ndarray, values: np.ndarray, lo: float, hi: float) -> float:
    mask = (freqs >= lo) & (freqs <= hi)
    if not np.any(mask):
        return 0.0
    v = values[mask]
    v = v[np.isfinite(v)]
    if v.size < 2:
        return 0.0
    return float(np.std(v))

def time_varying_coherence(
    x: np.ndarray,
    y: np.ndarray,

```



```

    sr: int,
    win_s: float = 0.25,
    hop_s: float = 0.10,
    lo: float = COUPLING_BAND_HZ[0],
    hi: float = COUPLING_BAND_HZ[1],
) -> Array:
    """Mean in-band MSC for successive windows (coherence stability)."""
    x = as_mono_float(x)
    y = as_mono_float(y)
    n = min(x.size, y.size)
    win = max(256, int(round(win_s * sr)))
    hop = max(64, int(round(hop_s * sr)))
    vals: list[float] = []
    start = 0
    while start + win <= n:
        f, c = magnitude_squared_coherence(x[start : start + win], y[start : start + win], sr)
        vals.append(band_mean(f, c, lo, hi))
        start += hop
    if not vals:
        f, c = magnitude_squared_coherence(x[:n], y[:n], sr)
        return np.array([band_mean(f, c, lo, hi)], dtype=np.float64)
    return np.asarray(vals, dtype=np.float64)

def hilbert_envelope(x: np.ndarray) -> Array:
    x = as_mono_float(x)
    if x.size < 8:
        return np.abs(x)
    return np.abs(signal.hilbert(x)).astype(np.float64)

def instantaneous_phase(x: np.ndarray) -> Array:
    """Unwrapped instantaneous phase via the analytic signal (Hilbert)."""
    x = as_mono_float(x)
    if x.size < 8:
        return np.zeros_like(x)
    return np.unwrap(np.angle(signal.hilbert(x))).astype(np.float64)

def instantaneous_frequency(x: np.ndarray, sr: int) -> Array:
    phase = instantaneous_phase(x)
    if phase.size < 2:
        return np.zeros(max(phase.size, 0), dtype=np.float64)
    return np.diff(phase) * (sr / (2.0 * np.pi))

def phase_jump_rate(x: np.ndarray, sr: int, jump_rad: float = 2.2) -> float:
    """Fraction of samples whose wrapped phase step exceeds `jump_rad`."""

    Natural voiced speech has large phase steps at glottal instants, but
    those are sparse and quasi-periodic. Dense irregular jumps indicate
    splices, codec resets, or randomized phase.
    """
    x = as_mono_float(x)
    if x.size < 32:
        return 0.0
    analytic = signal.hilbert(bandpass(x, sr, 80.0, min(4000.0, sr * 0.45)))
    wrapped = np.angle(analytic)
    step = np.diff(wrapped)
    step = (step + np.pi) % (2 * np.pi) - np.pi
    n_jumps = int(np.count_nonzero(np.abs(step) > jump_rad))
    duration = step.size / float(sr)
    # Jumps per second (a 1-sample wrap every 8 ms is ~125 Hz of jumps,
    # not a 0.008 fraction of samples).
    return float(n_jumps / max(duration, 1e-9))

def phase_residual_variance(x: np.ndarray, sr: int) -> float:
    """Variance of instantaneous frequency after subtracting a slow trend.

    Overly-flat phase (some vocoders / zero-phase reconstructions) yields
    an unnaturally small residual. Chaotic phase yields a huge residual.
    """
    x = as_mono_float(x)
    if x.size < 64:
        return 0.0
    xb = bandpass(x, sr, 80.0, min(1200.0, sr * 0.45))
    inst = instantaneous_frequency(xb, sr)
    inst = inst[np.isfinite(inst)]
    if inst.size < 32:
        return 0.0
    # Keep voiced-ish region: 60-2400 Hz instantaneous freq.
    voiced = inst[(inst > 50.0) & (inst < 500.0)]
    work = voiced if voiced.size >
16 else inst
    # Detrend with a moving median (~40 ms).
    k = max(5, int(round(0.04 * sr)) | 1)
    k = min(k, work.size | 1)
    if k < 5:
        return float(np.var(work))
    pad = k // 2
    padded = np.pad(work, pad, mode="edge")
    trend = np.lib.stride_tricks.sliding_window_view(padded, k).mean(axis=1)[: work.size]
    residual = work - trend
    return float(np.var(residual))

```

```

def gcc_phat_delay(x: np.ndarray, y: np.ndarray, sr: int, max_lag_s: float = 0.08) -> float:
    """Estimate delay of ``y`` relative to ``x`` in seconds (GCC-PHAT).

    Positive ? ``y`` lags ``x`` (air lags vibration).
    """
    x = as_mono_float(x)
    y = as_mono_float(y)
    n = min(x.size, y.size)
    x, y = x[:n], y[:n]
    if n < 32:
        return 0.0
    nfft = next_pow2(2 * n)
    X = np.fft.rfft(x, n=nfft)
    Y = np.fft.rfft(y, n=nfft)
    # IFFT(Y * conj(X)): positive lag => y lags x.
    r = Y * np.conj(X)
    r = r / (np.abs(r) + 1e-12)
    cc = np.fft.irfft(r, n=nfft)
    max_lag = min(n - 1, int(round(max_lag_s * sr)))
    # cc[k] = lag +k of y vs x for k small; negative lags at the end.
    pos = cc[: max_lag + 1]
    neg = cc[-max_lag:] if max_lag > 0 else np.array([], dtype=cc.dtype)
    lags = np.concatenate([np.arange(-max_lag, 0), np.arange(0, max_lag + 1)])
    vals = np.concatenate([neg, pos])
    peak = int(np.argmax(np.abs(vals)))
    return float(lags[peak]) / float(sr)

def windowed_delays(
    x: np.ndarray,
    y: np.ndarray,
    sr: int,
    win_s: float = 0.30,
    hop_s: float = 0.10,
) -> Array:
    """Per-window GCC-PHAT delays for drift / causality checks."""
    x = as_mono_float(x)
    y = as_mono_float(y)
    n = min(x.size, y.size)
    win = max(256, int(round(win_s * sr)))
    hop = max(64, int(round(hop_s * sr)))
    out: list[float] = []
    start = 0
    while start + win <= n:
        out.append(gcc_phat_delay(x[start : start + win], y[start : start + win], sr))
        start += hop
    if not out:
        out.append(gcc_phat_delay(x[:n], y[:n], sr))
    return np.asarray(out, dtype=np.float64)

def shift_to_align(x: np.ndarray, delay_samples: int) -> Array:
    """Shift ``x`` by ``delay_samples`` (positive = to the right) with zero fill."""
    x = as_mono_float(x)
    d = int(delay_samples)
    if d == 0:
        return x.copy()
    out = np.zeros_like(x)
    if d > 0:
        if d < x.size:
            out[d:] = x[: x.size - d]
        else:
            d = -d
            if d < x.size:
                out[: x.size - d] = x[d:]
    return out

def transfer_function(
    x: np.ndarray,
    y: np.ndarray,
    sr: int,
    nperseg: int | None = None,
) -> tuple[Array, Array, Array]:
    """Welch estimate of  $H(f) = P_{xy}(f) / P_{xx}(f)$ .

    ``x`` is the input (vibration), ``y`` the output (air).
    Returns ``(freqs, H_complex, Cxy)``.
    """
    x = as_mono_float(x)
    y = as_mono_float(y)
    n = min(x.size, y.size)
    x, y = x[:n], y[:n]
    if n < 128:
        freqs = np.fft.rfftfreq(256, 1.0 / sr)
        z = np.zeros_like(freqs, dtype=np.complex128)
        return freqs, z, np.zeros_like(freqs)
    nper = nperseg or welch_nperseg(n, sr)
    nper = min(nper, n // 2)
    freqs, pxx = signal.welch(x, fs=sr, nperseg=nper, noverlap=nper // 2)
    _, pxy = signal.csd(x, y, fs=sr, nperseg=nper, noverlap=nper // 2)
    with np.errstate(invalid="ignore", divide="ignore"):
        _, cxy = signal.coherence(x, y, fs=sr, nperseg=nper, noverlap=nper // 2)
    cxy = np.nan_to_num(np.asarray(cxy, dtype=np.float64), nan=0.0, posinf=0.0, neginf=0.0)
    h = pxy / (pxx + 1e-20)

```

```

    return (
        np.asarray(freqs, dtype=np.float64),
        np.asarray(h, dtype=np.complex128),
        np.asarray(cxy, dtype=np.float64),
    )

def log_mag_on_grid(
    freqs: np.ndarray,
    h: np.ndarray,
    grid: np.ndarray,
) -> Array:
    """Interpolate log-magnitude of H onto a common frequency grid."""
    mag = np.abs(np.asarray(h))
    mag = np.maximum(mag, 1e-12)
    logm = np.log(mag)
    logm = np.nan_to_num(logm, nan=-30.0, posinf=10.0, neginf=-30.0)
    # scipy welch freqs are linear; grid may be log-spaced.
    out = np.interp(grid, freqs, logm, left=logm[0], right=logm[-1])
    return np.asarray(out, dtype=np.float64)

def spectral_smoothness(x: np.ndarray, sr: int) -> dict[str, float]:
    """Fine-structure energy of the log-magnitude spectrum.

    Too-low values: vocoder / heavily smoothed synthetic spectra.
    Too-high values: noise-like or metallic comb artifacts.
    """
    frames, n_win, _ = frame_signal(x, sr, frame_ms=30.0, hop_ms=10.0)
    if frames.size == 0:
        return {"fine_var": 0.0, "centroid_std": 0.0}
    nfft = next_pow2(n_win)
    spec = np.abs(np.fft.rfft(frames, n=nfft)) + 1e-12
    logspec = np.log(spec)
    # Subtract a 5-bin moving average envelope (formant-scale).
    k = 5
    kernel = np.ones(k) / k
    smooth = np.apply_along_axis(lambda row: np.convolve(row, kernel, mode="same"), 1, logspec)
    fine = logspec - smooth
    fine_var = float(np.mean(np.var(fine, axis=1)))
    freqs = np.fft.rfftfreq(nfft, 1.0 / sr)
    mag = spec
    centroid = np.sum(freqs[None, :] * mag, axis=1) / np.sum(mag, axis=1)
    return {"fine_var": fine_var, "centroid_std": float(np.std(centroid))}

def spectral_flux(x: np.ndarray, sr: int) -> Array:
    frames, n_win, _ = frame_signal(x, sr, frame_ms=25.0, hop_ms=10.0)
    nfft = next_pow2(n_win)
    mag = np.abs(np.fft.rfft(frames, n=nfft))
    mag = mag / (np.linalg.norm(mag, axis=1, keepdims=True) + 1e-12)
    if mag.shape[0] < 2:
        return np.zeros(1, dtype=np.float64)
    diff = np.diff(mag, axis=0)
    flux = np.sqrt(np.sum(diff * diff, axis=1))
    return flux.astype(np.float64)

def rms_envelope(x: np.ndarray, sr: int, hop_ms: float = 10.0) -> Array:
    frames, _, _ = frame_signal(x, sr, frame_ms=25.0, hop_ms=hop_ms, window="hann")
    return np.sqrt(np.mean(frames * frames, axis=1) + 1e-18)

def lpc_coefficients(x: np.ndarray, order: int) -> Array:
    """Autocorrelation LPC via Levinson-Durbin (Toeplitz solve).

    Used as a formant-ish peak tracker, not as a codec.
    """
    x = as_mono_float(x)
    if x.size <= order + 2:
        return np.zeros(order, dtype=np.float64)
    x = x - np.mean(x)
    peak = np.max(np.abs(x))
    if peak < 1e-12:
        return np.zeros(order, dtype=np.float64)
    x = x / peak
    r = np.correlate(x, x, mode="full")[x.size - 1 : x.size + order]
    r = np.asarray(r, dtype=np.float64)
    r[0] = r[0] * 1.0001 + 1e-10
    try:
        a = solve_toeplitz(r[:-1], -r[1:])
    except np.linalg.LinAlgError:
        matrix = toeplitz(r[:-1])
        a, *_ = np.linalg.lstsq(matrix, -r[1:], rcond=None)
    return np.asarray(a, dtype=np.float64)

def formants_from_lpc(
    a: np.ndarray,
    sr: int,
    n_formants: int = 3,
) -> tuple[Array, Array]:
    """Positive-frequency LPC roots ? (frequencies_hz, bandwidths_hz)."""
    a = np.asarray(a, dtype=np.float64)
    if a.size == 0 or not np.all(np.isfinite(a)):

```

```

        return np.zeros(0), np.zeros(0)
    poly = np.concatenate([1.0], a)
    roots = np.roots(poly)
    roots = roots[np.imag(roots) >= 0.0]
    if roots.size == 0:
        return np.zeros(0), np.zeros(0)
    ang = np.angle(roots)
    freqs = ang * sr / (2.0 * np.pi)
    radii = np.abs(roots)
    bw = -np.log(np.clip(radii, 1e-8, 0.999999)) * sr / np.pi
    ok = (
        (freqs > 90.0)
        & (freqs < min(4000.0, 0.45 * sr))
        & (bw < 1200.0)
        & (radii > 0.55)
    )
    freqs, bw = freqs[ok], bw[ok]
    order = np.argsort(freqs)
    freqs, bw = freqs[order], bw[order]
    return freqs[:n_formants], bw[:n_formants]

def formant_tracks(
    x: np.ndarray,
    sr: int,
    order: int | None = None,
    n_formants: int = 3,
    frame_ms: float = 25.0,
    hop_ms: float = 10.0,
) -> Array:
    """LPC formant tracks. Shape (n_frames, n_formants); NaN where missing."""
    x = as_mono_float(x)
    xp = preemphasis(x)
    frames, n_win, _ = frame_signal(xp, sr, frame_ms=frame_ms, hop_ms=hop_ms)
    if order is None:
        order = max(8, int(round(sr / 1000)) + 2)
        order = min(order, n_win - 2)
    rms_f = np.sqrt(np.mean(frames * frames, axis=1) + 1e-18)
    thresh = 0.15 * (np.percentile(rms_f, 80) + 1e-12)
    tracks = np.full((frames.shape[0], n_formants), np.nan, dtype=np.float64)
    for i, fr in enumerate(frames):
        if rms_f[i] < thresh:
            continue
        a = lpc_coefficients(fr, order)
        freqs, _bw = formants_from_lpc(a, sr, n_formants=n_formants)
        n = min(n_formants, freqs.size)
        tracks[i, :n] = freqs[:n]
    return tracks

def formant_stability(tracks: np.ndarray) -> dict[str, float]:
    """Frame-to-frame formant jump statistics (Hz)."""
    if tracks.size == 0:
        return {"median_jump_hz": 0.0, "p95_jump_hz": 0.0, "voiced_frac": 0.0}
    jumps: list[float] = []
    voiced = 0
    for col in range(tracks.shape[1]):
        series = tracks[:, col]
        for a, b in zip(series[:-1], series[1:]):
            if np.isfinite(a) and np
.isfinite(b):
                jumps.append(abs(float(b - a)))
                voiced += 1
    voiced_frac = float(np.mean(np.any(np.isfinite(tracks), axis=1))) if tracks.ndim == 2 else 0.0
    if not jumps:
        return {"median_jump_hz": 0.0, "p95_jump_hz": 0.0, "voiced_frac": voiced_frac}
    arr = np.asarray(jumps, dtype=np.float64)
    return {
        "median_jump_hz": float(np.median(arr)),
        "p95_jump_hz": float(np.percentile(arr, 95)),
        "voiced_frac": voiced_frac,
    }

@dataclass
class DecayFit:
    tau_s: float
    r2: float
    peak_index: int
    n_points: int

def _fit_exponential(env: np.ndarray, sr: int) -> tuple[float, float]:
    """Least-squares log-envelope fit. Returns (tau_seconds, r2)."""
    env = np.asarray(env, dtype=np.float64)
    env = np.maximum(env, 1e-12)
    if env.size < 8:
        return 0.0, 0.0
    # Require a decaying shape.
    if env[-1] >= env[0] * 0.98:
        return 0.0, 0.0
    t = np.arange(env.size, dtype=np.float64) / float(sr)
    y = np.log(env)
    # Drop samples that have already hit the noise floor.
    floor = np.log(max(np.max(env) * 0.02, 1e-8))

```

```

mask = y > floor
if np.count_nonzero(mask) < 8:
    return 0.0, 0.0
t, y = t[mask], y[mask]
A = np.vstack([t, np.ones_like(t)]).T
coeff, *_ = np.linalg.lstsq(A, y, rcond=None)
slope, intercept = float(coeff[0]), float(coeff[1])
yhat = slope * t + intercept
ss_res = float(np.sum((y - yhat) ** 2))
ss_tot = float(np.sum((y - np.mean(y)) ** 2) + 1e-18)
r2 = 1.0 - ss_res / ss_tot
if slope >= -1e-6:
    return 0.0, max(r2, 0.0)
tau = -1.0 / slope
return float(tau), float(max(min(r2, 1.0), 0.0))

def decay_profiles(
    x: np.ndarray,
    sr: int,
    n_peaks: int = 8,
    tail_ms: float = 60.0,
) -> list[DecayFit]:
    """Fit exponential decays after Hilbert-envelope peaks.

    Whitepaper: authentic vocal-tract ringing is short and roughly
    exponential. Digital reverb and synthetic ringing produce long or
    poorly exponential tails.
    """
    x = as_mono_float(x)
    if x.size < int(0.05 * sr):
        return []
    env = hilbert_envelope(bandpass(x, sr, 80.0, min(4000.0, sr * 0.45)))
    # Light smoothing so we pick syllable / pulse groups, not every glottal peak.
    win = max(5, int(round(0.008 * sr)) | 1)
    env_s = signal.fftconvolve(env, np.ones(win) / win, mode="same")
    distance = max(int(0.04 * sr), 1)
    peaks, _props = signal.find_peaks(env_s, distance=distance)
    if peaks.size == 0:
        return []
    # Prefer high peaks that still have a tail inside the signal.
    tail = int(round(tail_ms * 0.001 * sr))
    scores = env_s[peaks]
    order = np.argsort(scores)[::-1]
    fits: list[DecayFit] = []
    for idx in peaks[order]:
        if len(fits) >= n_peaks:
            break
        end = idx + tail
        if end >= env_s.size:
            continue
        # Start a few samples after the peak so the attack is excluded.
        start = min(idx + max(2, int(0.002 * sr)), end - 8)
        tau, r2 = _fit_exponential(env[start:end], sr)
        if tau <= 0.0:
            continue
        fits.append(DecayFit(tau_s=tau, r2=r2, peak_index=int(idx), n_points=end - start))
    return fits

def offset_decay_stats(x: np.ndarray, sr: int) -> dict[str, float]:
    """Characterize the utterance *offset* tail, not mid-voicing ripple.

    Do not treat a long low-level reverb tail as 'silence to skip' ? that
    is the signal. Measure how long the envelope takes to fall from half
    peak to 5% peak, and how much energy lives after the half-peak point.
    """
    x = as_mono_float(x)
    if x.size < 16:
        return {"offset_s": 0.0, "tau_s": 0.0, "r2": 0.0, "tail_ratio": 0.0}
    env = hilbert_envelope(bandpass(x, sr, 80.0, min(4000.0, sr * 0.45)))
    win = max(5, int(round(0.012 * sr)) | 1)
    env_s = signal.fftconvolve(env, np.ones(win) / win, mode="same")
    peak = float(np.max(env_s) + 1e-12)
    above = np.where(env_s >= 0.50 * peak)[0]
    if above.size == 0:
        return {"offset_s": 0.0, "tau_s": 0.0, "r2": 0.0, "tail_ratio": 0.0}
    last_half = int(above[-1])
    rest = env_s[last_half:]
    floor_idx = np.where(rest <= 0.05 * peak)[0]
    if floor_idx.size:
        decay_end = last_half + int(floor_idx[0])
    else:
        decay_end = int(env_s.size - 1)
    offset_s = max(0.0, (decay_end - last_half) / float(sr))
    tau, r2 = _fit_exponential(env[last_half : max(decay_end, last_half + 8) + 1], sr)
    after = float(np.sum(env[last_half:] ** 2) + 1e-18)
    pre_n = int(round(0.08 * sr))
    pre_start = max(0, last_half - pre_n)
    before = float(np.sum(env[pre_start:last_half] ** 2) + 1e-18)
    tail_ratio = after / (after + before)
    return {
        "offset_s": float(offset_s),
        "tau_s": float(tau),
        "r2": float(r2),
    }

```

```

        "tail_ratio": float(tail_ratio),
    }

def late_energy_ratio(x: np.ndarray, sr: int, split_s: float = 0.08) -> float:
    """Energy after ``split_s`` following the global envelope peak.

    Long synthetic reverb parks energy far after anatomical ringing dies.
    """
    x = as_mono_float(x)
    env = hilbert_envelope(x)
    if env.size == 0:
        return 0.0
    peak = int(np.argmax(env))
    split = peak + int(round(split_s * sr))
    if split >= env.size - 4:
        return 0.0
    early = float(np.sum(env[peak:split] ** 2) + 1e-18)
    late = float(np.sum(env[split:] ** 2))
    return late / (early + late)

def modulation_spectrum_peak(
    x: np.ndarray,
    sr: int,
    lo_hz: float = 40.0,
    hi_hz: float = 250.0,
) -> dict[str, float]:
    """Peak of the Hilbert-envelope modulation spectrum in ``lo?hi`` Hz.

    Vocoder / vocoder-hop buzz often concentrates envelope energy at a
    fixed frame rate (commonly 50?200 Hz).
    """
    x = as_mono_float(x)
    env = hilbert_envelope(x)
    env = env - np.mean(env)
    if env.size < 64:
        return {"peak_hz": 0.0, "peak_to_med": 0.0}
    nfft = next_pow2(env.size)
    spec = np.abs(np.fft.rfft(env, n=nfft))
    freqs = np.fft.rfftfreq(nfft, 1.0 / sr)
    mask = (freqs >= lo_hz) & (freqs <= hi_hz)
    if not np.any(mask):
        return {"peak_hz": 0.0, "peak_to_med": 0.0}
    band = spec[mask]
    bf = freqs[mask]
    k = int(np.argmax(band))
    med = float(np.median(band) + 1e-12)
    return {"peak_hz": float(bf[k]), "peak_to_med": float(band[k] / med)}

def estimate_f0(x: np.ndarray, sr: int, lo: float = 70.0, hi: float = 350.0) -> float:
    """Autocorrelation F0 estimate (Hz). 0 if unvoiced / too short."""
    x = as_mono_float(x)
    if x.size < int(0.04 * sr):
        return 0.0
    # Use a mid-signal window so fades do not dominate.
    n = min(x.size, int(0.08 * sr))
    start = max(0, (x.size - n) // 2)
    seg = x[start : start + n]
    seg = seg - np.mean(seg)
    if np.max(np.abs(seg)) < 1e-12:
        return 0.0
    corr = np.correlate(seg, seg, mode="full")[n - 1 :]
    min_lag = max(1, int(round(sr / hi)))
    max_lag = min(corr.size - 1, int(round(sr / lo)))
    if max_lag <= min_lag + 1:
        return 0.0
    lag = min_lag + int(np.argmax(corr[min_lag : max_lag + 1]))
    return float(sr / lag) if lag > 0 else 0.0

```

forgereceipts/engines/vibelock/dual_channel.py

```

"""Dual-channel physical tests (air audio + body-coupled vibration).

Whitepaper mode 1: synchronize, drift-correct, then evaluate time,
frequency, and phase consistency between the air microphone and a
jaw accelerometer / contact mic / IMU.

Checks implemented here:

* Vibration?audio magnitude-squared coherence in the coupling band
  (~80?2000 Hz) and the broader speech band (~80?4000 Hz).
* Transfer-function residual versus a *synthetic* bootstrap baseline
  (see ``vibelock.synth``; not a published human dataset).
* Phase / latency constraints: GCC-PHAT delay inside a causal window,
  bounded drift across windows.
* Resonance decay profiles on the air channel (anatomical vs reverb).
"""

from __future__ import annotations

import numpy as np
from numpy.typing import NDArray

from vibelock import dsp
from vibelock.scoring import (
    COHERENCE_LOW,
    COHERENCE_UNSTABLE,
    DECAY_IMPLAUSIBLE,
    DRIFT_EXCESSIVE,
    LATENCY_OUT_OF_BOUNDS,
    TRANSFER_RESIDUAL_HIGH,
    CheckResult,
    clip01,
    logistic_score,
)
from vibelock.synth import bootstrap_pairs

# Log-frequency grid shared by observed TFs and the bootstrap prior.
_TF_GRID: NDArray[np.float64] | None = None
_TF_MEAN: NDArray[np.float64] | None = None
_TF_STD: NDArray[np.float64] | None = None
_TF_MEMBERS: NDArray[np.float64] | None = None
_TF_SR: int | None = None

def _tf_grid() -> NDArray[np.float64]:
    global _TF_GRID
    if _TF_GRID is None:
        lo, hi = dsp.SPEECH_BAND_HZ
        _TF_GRID = np.geomspace(lo, hi, 64)
    return _TF_GRID

def _ensure_baseline(sr: int) -> None:
    """Lazy-build the synthetic transfer-function prior at ``sr``."""
    global _TF_MEAN, _TF_STD, _TF_MEMBERS, _TF_SR
    if _TF_MEMBERS is not None and _TF_SR == sr:
        return
    pairs = bootstrap_pairs(n_pairs=20, sr=sr, duration_s=0.8, seed=202607)
    grid = _tf_grid()
    rows: list[np.ndarray] = []
    for p in pairs:
        freqs, h, _c = dsp.transfer_function(p.vibration, p.audio, p.sr)
        rows.append(dsp.log_mag_on_grid(freqs, h, grid))
    members = np.vstack(rows)
    # Subtract per-row mean so overall gain (mic calibration) drops out.
    members = members - members.mean(axis=1, keepdims=True)
    _TF_MEMBERS = members
    _TF_MEAN = members.mean(axis=0)
    _TF_STD = members.std(axis=0) + 1e-3
    _TF_SR = sr

def _tf_residual(vibration: np.ndarray, audio: np.ndarray, sr: int) -> dict[str, float]:
    _ensure_baseline(sr)
    grid = _tf_grid()
    freqs, h, cxy = dsp.transfer_function(vibration, audio, sr)
    logm = dsp.log_mag_on_grid(freqs, h, grid)
    logm = logm - float(np.mean(logm))
    assert _TF_MEAN is not None and _TF_STD is not None and _TF_MEMBERS is not None
    z = (logm - _TF_MEAN) / _TF_STD
    rmse_mean = float(np.sqrt(np.mean(z * z)))
    diffs = _TF_MEMBERS - logm[None, :]
    min_l2 = float(np.sqrt(np.mean(diffs * diffs, axis=1).min()))
    # Shape roughness: a physical TF is smooth on a log-f grid; noise is not.
    roughness = float(np.mean(np.diff(logm, n=2) ** 2))
    band_coh = dsp.band_mean(freqs, cxy, *dsp.COUPLING_BAND_HZ)
    return {
        "rmse_z": rmse_mean,
        "min_l2": min_l2,
        "roughness": roughness,
        "tf_coherence": band_coh,
    }

```

```
def _align(audio: np.ndarray, vibration: np.ndarray, sr: int) -> tuple[np.ndarray, np.ndarray, float]:
    """Band-limit, estimate lag of audio vs vibration, shift audio backward."""
    lo, hi = dsp.COUPLING_BAND_HZ
    hi = min(hi, 0.45 * sr)
    a = dsp.bandpass(audio, sr, lo, hi)
    v = dsp.bandpass(vibration, sr, lo, hi)
    delay_s = dsp.gcc_phat_delay(v, a, sr)
    delay_samples = int(round(delay_s * sr))
    # Shift audio so it lines up with vibration (undo the lag).
    aligned_audio = dsp.shift_to_align(audio, -delay_samples)
    return aligned_audio, vibration, delay_s
```

```
def check_coherence(audio: np.ndarray, vibration: np.ndarray, sr: int) -> CheckResult:
    freqs, cxy = dsp.magnitude_squared_coherence(vibration, audio, sr)
    mean_couple = dsp.band_mean(freqs, cxy, *dsp.COUPLING_BAND_HZ)
    mean_speech = dsp.band_mean(freqs, cxy, *dsp.SPEECH_BAND_HZ)
    std_couple = dsp.band_std(freqs, cxy, *dsp.COUPLING_BAND_HZ)
    tv = dsp.time_varying_coherence(vibration, audio, sr)
    tv_mean = float(np.mean(tv))
    tv_std = float(np.std(tv)) if tv.size > 1 else 0.0
    # Authentic coupled speech: mean MSC typically well above 0.3 in-band
    # on these Welch settings; uncorrelated channels sit near 0.05±0.15.
    score = logistic_score(mean_couple, good=0.55, bad=0.12)
    # Penalize instability: large time variation of MSC.
    if tv_std > 0.18 and tv_mean < 0.45:
        score = min(score, logistic_score(tv_std, good=0.04, bad=0.28))
        code: str | None = COHERENCE_UNSTABLE if mean_couple >= 0.20 else COHERENCE_LOW
    elif mean_couple < 0.22:
        code = COHERENCE_LOW
        score = min(score, 0.35)
    else:
        code = None
    return CheckResult(
        name="coherence",
        score=clip01(score),
        reason_code=code,
        metrics={
            "msc_coupling_band": mean_couple,
            "msc_speech_band": mean_speech,
            "msc_std_coupling": std_couple,
            "msc_time_mean": tv_mean,
            "msc_time_std": tv_std,
        },
        note="Magnitude-squared coherence, 80±2000 Hz coupling band.",
    )
```

```
def check_transfer(audio: np.ndarray, vibration: np.ndarray, sr: int) -> CheckResult:
    m = _tf_residual(vibration, audio, sr)
    # rmse_z ~ 1 is typical of the bootstrap itself; shuffled pairs go much higher.
    score_z = logistic_score(m["rmse_z"], good=1.2, bad=4.5)
    score_l2 = logistic_score(m["min_l2"], good=0.8, bad=3.5)
    score_rough = logistic_score(m["roughness"], good=0.02, bad=0.35)
    score = clip01(0.5 * score_z + 0.3 * score_l2 + 0.2 * score_rough)
    code = TRANSFER_RESIDUAL_HIGH if (m["rmse_z"] > 3.0 or m["min_l2"] > 2.4) else None
    if code:
        score = min(score, 0.40)
    return CheckResult(
        name="transfer",
        score=score,
        reason_code=code,
        metrics=m,
        note=(
            "Vibration-to-air log|H(f)| residual vs a synthetic physically-"
            "plausible bootstrap (not a published human dataset).",
        ),
    )
```

```
def check_phase_latency(audio: np.ndarray, vibration: np.ndarray, sr: int) -> CheckResult:
    lo, hi = dsp.COUPLING_BAND_HZ
    hi = min(hi, 0.45 * sr)
    a = dsp.bandpass(audio, sr, lo, hi)
    v = dsp.bandpass(vibration, sr, lo, hi)
    delay_s = dsp.gcc_phat_delay(v, a, sr)
    delays = dsp.windowed_delays(v, a, sr)
    delay_std = float(np.std(delays)) if delays.size > 1 else 0.0
    dmin, dmax = dsp.PLAUSIBLE_DELAY_S
    in_window = dmin <= delay_s <= dmax
    delay_score = 1.0 if in_window else logistic_score(
        min(abs(delay_s - dmin), abs(delay_s - dmax)),
        good=0.0,
        bad=0.06,
    )
    # Drift: after removing the median delay, residual std should be small.
    drift_score = logistic_score(delay_std, good=0.002, bad=0.025)
    score = clip01(0.6 * delay_score + 0.4 * drift_score)
    code: str | None = None
    if not in_window:
        code = LATENCY_OUT_OF_BOUNDS
        score = min(score, 0.35)
    elif delay_std > 0.018:
        code = DRIFT_EXCESSIVE
        score = min(score, 0.45)
```



```

    return CheckResult(
        name="phase_latency",
        score=score,
        reason_code=code,
        metrics={
            "delay_s": delay_s,
            "delay_std_s": delay_std,
            "delay_median_s": float(np.median(delays)),
            "n_windows": float(delays.size),
        },
        note="GCC-PHAT latency and windowed drift (causality / bounded drift).",
    )

def check_decay(audio: np.ndarray, sr: int) -> CheckResult:
    off = dsp.offset_decay_stats(audio, sr)
    fits = dsp.decay_profiles(audio, sr)
    late = dsp.late_energy_ratio(audio, sr, split_s=0.08)
    if fits:
        taus = np.array([f.tau_s for f in fits], dtype=np.float64)
        r2s = np.array([f.r2 for f in fits], dtype=np.float64)
        median_tau = float(np.median(taus))
        max_tau = float(np.max(taus))
        mean_r2 = float(np.mean(r2s))
    else:
        median_tau = 0.0
        max_tau = 0.0
        mean_r2 = 0.0
    # Primary cue: how long the *offset* takes to fall from ~half-peak to silence.
    # Dry vocal-tract ringing + a short natural fade sits well below 100 ms.
    # Digital reverb stretches that offset to hundreds of milliseconds.
    offset_score = logistic_score(off["offset_s"], good=0.04, bad=0.28)
    tau_score = logistic_score(off["tau_s"] if off["tau_s"] > 0 else 0.01, good=0.02, bad=0.25)
    tail_score = logistic_score(off["tail_ratio"], good=0.15, bad=0.70)
    score = clip01(0.45 * offset_score + 0.35 * tau_score + 0.20 * tail_score)
    implausible = (
        off["offset_s"] > 0.16
        or off["tau_s"] > 0.14
        or (off["tail_ratio"] > 0.55 and off["offset_s"] > 0.10)
    )
    code = DECAY_IMPLAUSIBLE if implausible else None
    if code:
        score = min(score, 0)
.35)
    return CheckResult(
        name="decay",
        score=score,
        reason_code=code,
        metrics={
            "offset_s": off["offset_s"],
            "offset_tau_s": off["tau_s"],
            "offset_r2": off["r2"],
            "tail_ratio": off["tail_ratio"],
            "median_tau_s": median_tau,
            "max_tau_s": max_tau,
            "mean_r2": mean_r2,
            "late_energy_ratio": late,
            "n_fits": float(len(fits)),
        },
        note="Offset-tail exponential decay vs long digital-reverb tails.",
    )

def analyze_dual(audio: np.ndarray, vibration: np.ndarray, sr: int) -> list[CheckResult]:
    """Run the four dual-channel physical tests on already-mono signals."""
    audio = dsp.as_mono_float(audio)
    vibration = dsp.as_mono_float(vibration)
    n = min(audio.size, vibration.size)
    audio, vibration = audio[:n], vibration[:n]
    aligned, vib, _delay = _align(audio, vibration, sr)
    n2 = min(aligned.size, vib.size)
    aligned, vib = aligned[:n2], vib[:n2]
    return [
        check_coherence(aligned, vib, sr),
        check_transfer(aligned, vib, sr),
        check_phase_latency(audio, vibration, sr), # latency on the *unshifted* pair
        check_decay(audio, sr),
    ]

```

forgereceipts/engines/vibelock/forensic.py

```

"""Audio-only forensic risk assessment.

Whitepaper mode 2: when body-coupled vibration is absent, VibeLock still
inspects spectral smoothness, phase continuity, formant stability,
resonance behavior, and temporal artifacts (splices, vocoder buzz,
overly-flat phase). This is a *risk assessment*, not a proof of liveness.
"""

from __future__ import annotations

import numpy as np

from vibelock import dsp
from vibelock.scoring import (
    DECAY_IMPLAUSIBLE,
    FORMANT_UNSTABLE,
    PHASE_DISCONTINUITY,
    PHASE_OVERFLAT,
    SPECTRAL_UNNATURAL,
    TEMPORAL_SPLICE,
    VOCODER_BUZZ,
    CheckResult,
    clip01,
    logistic_score,
)

def check_spectral(audio: np.ndarray, sr: int) -> CheckResult:
    stats = dsp.spectral_smoothness(audio, sr)
    fine = stats["fine_var"]
    # Natural harmonic speech has moderate fine-structure variance.
    # Zero-phase / over-smoothed vocoder: very low. White-noise: very high.
    # Score peaks in a mid band and falls off both sides.
    low = logistic_score(fine, good=0.08, bad=0.005)
    high = logistic_score(fine, good=0.25, bad=1.60)
    score = clip01(min(low, high))
    code = None
    if fine < 0.012 or fine > 1.20:
        code = SPECTRAL_UNNATURAL
        score = min(score, 0.40)
    return CheckResult(
        name="spectral",
        score=score,
        reason_code=code,
        metrics=stats,
        note="Log-spectral fine structure after a formant-scale envelope.",
    )

def check_phase_continuity(audio: np.ndarray, sr: int) -> CheckResult:
    jump = dsp.phase_jump_rate(audio, sr)
    resid = dsp.phase_residual_variance(audio, sr)
    # Dense irregular Hilbert jumps ? discontinuity.
    # Tiny residual IF variance with non-tiny energy ? overly flat phase.
    jump_score = logistic_score(jump, good=8.0, bad=28.0)
    # Residual variance of IF in Hz^2. Natural jitter is thousands here
    # because of glottal FM; overly-flat vocoder phase sits near zero.
    flat_score = logistic_score(resid, good=400.0, bad=8.0)
    chaotic_score = logistic_score(resid, good=3.0e4, bad=2.5e6)
    score = clip01(min(jump_score, max(flat_score, 0.15), chaotic_score))
    code = None
    if jump > 12.0:
        code = PHASE_DISCONTINUITY
        score = min(score, 0.35)
    elif resid < 20.0:
        code = PHASE_OVERFLAT
        score = min(score, 0.40)
    return CheckResult(
        name="phase_continuity",
        score=score,
        reason_code=code,
        metrics={"phase_jump_rate": jump, "if_residual_var": resid},
        note="Hilbert phase jumps and detrended instantaneous-frequency variance.",
    )

def check_formant(audio: np.ndarray, sr: int) -> CheckResult:
    tracks = dsp.formant_tracks(audio, sr)
    st = dsp.formant_stability(tracks)
    med = st["median_jump_hz"]
    p95 = st["p95_jump_hz"]
    # Median jump is the robust cue. LPC peak swapping inflates p95 even
    # on a stationary three-resonator cartoon; that is a tracker artifact,
    # not vocal-tract teleportation.
    score = logistic_score(med, good=50.0, bad=150.0)
    score = clip01(score)
    code = None
    if med > 110.0:
        code = FORMANT_UNSTABLE
        score = min(score, 0.35)
    return CheckResult(
        name="formant",

```

```

        score=score,
        reason_code=code,
        metrics=st,
        note="LPC peak tracks (formant-ish), frame-to-frame stability.",
    )

def check_decay(audio: np.ndarray, sr: int) -> CheckResult:
    # Same anatomical decay test as dual-channel; audio-only still cares.
    from vibelock.dual_channel import check_decay as _decay

    return _decay(audio, sr)

def check_temporal(audio: np.ndarray, sr: int) -> CheckResult:
    flux = dsp.spectral_flux(audio, sr)
    env = dsp.rms_envelope(audio, sr, hop_ms=10.0)
    if env.size < 6:
        return CheckResult("temporal", 0.5, None, {}, "Too short to judge splices.")
    log_env = np.log(env + 1e-8)
    d_env = np.abs(np.diff(log_env))
    # A hard splice in the middle of energy produces a coincident RMS jump
    # *and* a spectral-flux spike, without a stop-gap (near-silence) before it.
    n = min(d_env.size, flux.size)
    if n < 4:
        return CheckResult("temporal", 0.5, None, {}, "Too short to judge splices.")
    d_env, flux = d_env[:n], flux[:n]
    env_body = env[:n]
    # Preceding-frame energy, to ignore true onsets from silence.
    prev_e = env_body
    flux_z = (flux - np.median(flux)) / (np.std(flux) + 1e-8)
    env_z = (d_env - np.median(d_env)) / (np.std(d_env) + 1e-8)
    coincident = (flux_z > 3.5) & (env_z > 3.5) & (prev_e > 0.08 * np.max(env))
    n_hits = int(np.count_nonzero(coincident))
    max_combo = float(np.max(flux_z + env_z)) if n else 0.0
    # Pulse-train frames already have combo z ~ 10. Hard splices go higher
    # *and* produce coincident RMS+flux hits.
    score = logistic_score(max_combo, good=12.0, bad=22.0)
    if n_hits:
        score = min(score, logistic_score(float(n_hits), good=0.0, bad=4.0))
    score = clip01(score)
    code = TEMPORAL_SPLICE if (n_hits >= 1 and max_combo > 12.0) else None
    if code:
        score = min(score, 0.35)
    return CheckResult(
        name="temporal",
        score=score,
        reason_code=code,
        metrics={
            "n_splice_hits": float(n_hits),
            "max_flux_env_z": max_combo,
            "p95_flux": float(np.percentile(flux, 95)),
        },
        note="Coincident RMS and spectral-flux spikes (splice-like).",
    )

def check_buzz(audio: np.ndarray, sr: int) -> CheckResult:
    mod = dsp.modulation_spectrum_peak(audio, sr, lo_hz=40.0, hi_hz=220.0)
    ratio = mod["peak_to_med"]
    f0 = dsp.estimate_f0(audio, sr)
    peak_hz = mod["peak_hz"]
    # A Hilbert-envelope peak at F0 is glottal pulsing, not vocoder hop.
    rel_f0 = abs(peak_hz - f0) / max(f0, 1.0) if f0 > 0 else 99.0
    rel_2p = abs(peak_hz - 2.0 * f0) / max(2.0 * f0, 1.0) if f0 > 0 else 99.0
    glottal = min(rel_f0, rel_2p) < 0.12
    metrics = {"mod": float(mod), "f0_hz": float(f0), "rel_f0": float(rel_f0)}
    if glottal:
        return CheckResult(
            name="buzz",
            score=0.95,
            reason_code=None,
            metrics=metrics,
            note="Envelope peak matches estimated F0 (glottal, not hop buzz).",
        )
    score = logistic_score(ratio, good=6.0, bad=20.0)
    code = VOCODER_BUZZ if ratio > 12.0 else None
    if code:
        score = min(score, 0.40)
    return CheckResult(
        name="buzz",
        score=clip01(score),
        reason_code=code,
        metrics=metrics,
        note="Envelope modulation-spectrum peak off F0 in the vocoder-hop band.",
    )

def analyze_audio(audio: np.ndarray, sr: int) -> list[CheckResult]:
    """Run the audio-only forensic battery."""
    audio = dsp.as_mono_float(audio)
    return [
        check_spectral(audio, sr),
        check_phase_continuity(audio, sr),
        check_formant(audio, sr),
    ]

```

```
    check_decay(audio, sr),  
    check_temporal(audio, sr),  
    check_buzz(audio, sr),  
]
```

forgereceipts/engines/vibelock/io.py

```

"""Audio I/O. WAV only, numpy + scipy, no cloud, no identity.

Whitepaper ? privacy: local processing, no STT, do not retain raw audio
by default. This module loads a file into memory for analysis and returns
float arrays; it never writes recordings unless the caller asks.
"""

from __future__ import annotations

from pathlib import Path

import numpy as np
from numpy.typing import NDArray
from scipy.io import wavfile

from vibelock.dsp import as_mono_float, resample

Array = NDArray[np.float64]

def _pcm_to_float(data: np.ndarray) -> Array:
    data = np.asarray(data)
    if np.issubdtype(data.dtype, np.floating):
        arr = data.astype(np.float64)
        peak = np.max(np.abs(arr)) if arr.size else 1.0
        if peak > 8.0:
            arr = arr / max(peak, 1.0)
        return arr
    info = np.iinfo(data.dtype)
    scale = float(max(abs(info.min), info.max))
    return data.astype(np.float64) / scale

def load_audio(path: str | Path, target_sr: int | None = None) -> tuple[Array, int]:
    """Load a WAV file as mono float64 in roughly [-1, 1].

    ``target_sr`` resamples when set. Stereo is mixed down. The file is
    not kept after this function returns.
    """
    path = Path(path)
    if not path.is_file():
        raise FileNotFoundError(f"Audio file not found: {path}")
    sr, data = wavfile.read(path)
    arr = _pcm_to_float(data)
    arr = as_mono_float(arr)
    sr = int(sr)
    if target_sr is not None and int(target_sr) != sr:
        arr = resample(arr, sr, int(target_sr))
        sr = int(target_sr)
    return arr, sr

def write_wav(path: str | Path, audio: np.ndarray, sr: int) -> None:
    """Write a 16-bit PCM WAV (used by examples / tests, not by analyze)."""
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    arr = as_mono_float(audio)
    peak = np.max(np.abs(arr)) + 1e-12
    pcm = np.clip(arr / peak, -1.0, 1.0)
    pcm16 = (pcm * 32767.0).astype(np.int16)
    wavfile.write(str(path), int(sr), pcm16)

def match_lengths(*arrays: np.ndarray) -> list[Array]:
    n = min(a.size for a in arrays)
    return [as_mono_float(a)[:n] for a in arrays]

```

forgereceipts/engines/vibelock/scoring.py

```

"""Composite authenticity scoring and machine-readable reason codes.

Whitepaper ? scoring: a probabilistic score in [0, 1] plus interpretable
reason codes. Thresholds here are *implementation defaults* chosen so
synthetic fixtures move the score in the documented direction. They are
not published operating points from a human listening study, and this
file invents no evaluation numbers or citations.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any, Iterable, Mapping

import numpy as np

# Reason codes listed in the whitepaper, plus a few that name phenomena
# the same document already describes (vocoder buzz, overly-flat phase,
# latency/drift, spectral unnaturalness).
COHERENCE_LOW = "COHERENCE_LOW"
COHERENCE_UNSTABLE = "COHERENCE_UNSTABLE"
PHASE_DISCONTINUITY = "PHASE_DISCONTINUITY"
PHASE_OVERFLAT = "PHASE_OVERFLAT"
LATENCY_OUT_OF_BOUNDS = "LATENCY_OUT_OF_BOUNDS"
DRIFT_EXCESSIVE = "DRIFT_EXCESSIVE"
TRANSFER_RESIDUAL_HIGH = "TRANSFER_RESIDUAL_HIGH"
DECAY_IMPLAUSIBLE = "DECAY_IMPLAUSIBLE"
FORMANT_UNSTABLE = "FORMANT_UNSTABLE"
TEMPORAL_SPLICE = "TEMPORAL_SPLICE"
SPECTRAL_UNNATURAL = "SPECTRAL_UNNATURAL"
VOCODER_BUZZ = "VOCODER_BUZZ"
VIBRATION_UNUSABLE = "VIBRATION_UNUSABLE"

DUAL_WEIGHTS: dict[str, float] = {
    "coherence": 0.30,
    "transfer": 0.25,
    "phase_latency": 0.20,
    "decay": 0.15,
    "forensic": 0.10,
}

AUDIO_WEIGHTS: dict[str, float] = {
    "spectral": 0.18,
    "phase_continuity": 0.22,
    "formant": 0.22,
    "decay": 0.16,
    "temporal": 0.14,
    "buzz": 0.08,
}

@dataclass
class CheckResult:
    """One physical test, a 0?1 subscore, and an optional reason code."""

    name: str
    score: float
    reason_code: str | None
    metrics: dict[str, float] = field(default_factory=dict)
    note: str = ""

    def to_dict(self) -> dict[str, Any]:
        return {
            "name": self.name,
            "score": float(self.score),
            "reason_code": self.reason_code,
            "metrics": {k: float(v) for k, v in self.metrics.items()},
            "note": self.note,
        }

@dataclass
class AnalysisResult:
    """Top-level VibeLock output (no raw audio is retained)."""

    score: float
    mode: str
    reason_codes: list[str]
    checks: list[CheckResult]
    sample_rate: int
    n_samples: int
    notes: list[str] = field(default_factory=list)

    def to_dict(self) -> dict[str, Any]:
        return {
            "score": float(self.score),
            "mode": self.mode,
            "reason_codes": list(self.reason_codes),
            "checks": [c.to_dict() for c in self.checks],
            "sample_rate": int(self.sample_rate),
            "n_samples": int(self.n_samples),
            "notes": list(self.notes),
        }

```

```

    }

def clip01(x: float) -> float:
    return float(min(1.0, max(0.0, x)))

def logistic_score(value: float, good: float, bad: float) -> float:
    """Map a scalar onto [0, 1]. ``good`` scores near 1, ``bad`` near 0.

    Uses a smoothstep between the two anchors so tests can sit clearly
    on either side without claiming a calibrated probability.
    """
    if good == bad:
        return 0.5
    t = (value - bad) / (good - bad)
    t = clip01(t)
    # Hermite smoothstep
    return float(t * t * (3.0 - 2.0 * t))

def combine(
    checks: Iterable[CheckResult],
    mode: str,
    sample_rate: int,
    n_samples: int,
    notes: list[str] | None = None,
    extra_codes: Iterable[str] | None = None,
) -> AnalysisResult:
    checks = list(checks)
    weights = DUAL_WEIGHTS if mode == "dual_channel" else AUDIO_WEIGHTS
    # Forensic dual-channel contribution is the mean of audio-only checks
    # that were attached with names in AUDIO_WEIGHTS.
    grouped: dict[str, list[float]] = {k: [] for k in weights}
    ungrouped: list[float] = []
    for c in checks:
        if c.name in grouped:
            grouped[c.name].append(c.score)
        elif mode == "dual_channel" and c.name in AUDIO_WEIGHTS:
            grouped["forensic"].append(c.score)
        else:
            ungrouped.append(c.score)

    num = 0.0
    den = 0.0
    for name, w in weights.items():
        vals = grouped.get(name) or []
        if not vals:
            continue
        num += w * float(np.mean(vals))
        den += w
    if ungrouped and den < 0.99:
        leftover = max(0.0, 1.0 - den)
        num += leftover * float(np.mean(ungrouped))
        den += leftover
    score = clip01(num / den if den > 0 else 0.0)

    codes: list[str] = []
    for c in checks:
        if c.reason_code:
            codes.append(c.reason_code)
    if extra_codes:
        codes.extend(extra_codes)
    # Stable unique order.
    seen: set[str] = set()
    ordered: list[str] = []
    for code in codes:
        if code not in seen:
            seen.add(code)
            ordered.append(code)
    return AnalysisResult(
        score=score,
        mode=mode,
        reason_codes=ordered,
        checks=checks,
        sample_rate=sample_rate,
        n_samples=n_samples,
        notes=list(notes or []),
    )

def analyze(
    audio: np.ndarray,
    sr: int,
    vibration: np.ndarray | None = None,
) -> AnalysisResult:
    """Run dual-channel analysis when vibration is present, else audio-only.

    This is the library-level entry point. The CLI is a thin wrapper.
    """
    from vibelock.dual_channel import analyze_dual
    from vibelock.forensic import analyze_audio
    from vibelock.dsp import as_mono_float, rms

    audio = as_mono_float(audio)

```

```

notes: list[str] = []
extra: list[str] = []
if vibration is None:
    checks = analyze_audio(audio, sr)
    return combine(checks, "audio_only", sr, int(audio.size), notes)

vib = as_mono_float(vibration)
if vib.size < int(0.08 * sr) or rms(vib) < 1e-6:
    notes.append("Vibration missing or unusable; falling back to audio-only.")
    extra.append(VIBRATION_UNUSABLE)
    checks = analyze_audio(audio, sr)
    return combine(checks, "audio_only", sr, int(audio.size), notes, extra)

n = min(audio.size, vib.size)
dual_checks = analyze_dual(audio[:n], vib[:n], sr)
# Supporting forensic evidence, down-weighted in combine().
audio_checks = analyze_audio(audio[:n], sr)
return combine(dual_checks + audio_checks, "dual_channel", sr, n, notes, extra)

def format_human(result: AnalysisResult) -> str:
    codes = ", ".join(result.reason_codes) if result.reason_codes else "(none)"
    lines = [
        f"Vibelock authenticity score: {result.score:.3f}",
        f"Mode: {result.mode}",
        f"Reason codes: {codes}",
        f"Sample rate: {result.sample_rate} Hz, samples: {result.n_samples}",
    ]
    if result.notes:
        for n in result.notes:
            lines.append(f"Note: {n}")
    lines.append("Checks:")
    for c in result.checks:
        flag = f" [{c.reason_code}]" if c.reason_code else ""
        lines.append(f"  - {c.name}: {c.score:.3f}{flag}")
    return "\n".join(lines)

def json_ready(obj: Mapping[str, Any] | AnalysisResult) -> Any:
    if isinstance(obj, AnalysisResult):
        return obj.to_dict()
    return obj

```


forgereceipts/engines/vibelock/synth.py

```

"""Synthetic physically-plausible dual-channel pairs.

The transfer-function baseline in dual-channel mode is learned from
pairs generated here, NOT from a published human dataset. That fact is
part of the public contract: the baseline is a physics-inspired prior,
not an empirical speaker corpus.

Model (whitepaper ? dual-channel / transfer-function consistency):

* A glottal-like pulse train with spectral tilt (source).
* Air channel = source through a cascade of second-order formant
  resonators (a tiny vocal-tract sketch).
* Vibration channel = source through a low-pass bone/tissue coupler
  with a mild low-frequency resonance.

This is a toy biomechanical cartoon. It is enough to define a family of
plausible vibration-to-air maps; it is not a claim about any recorded
human.
"""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from numpy.typing import NDArray
from scipy import signal

Array = NDArray[np.float64]

def _rng(seed: int | None) -> np.random.Generator:
    return np.random.default_rng(seed)

def glottal_source(
    n: int,
    sr: int,
    f0: float = 120.0,
    jitter: float = 0.008,
    shimmer: float = 0.04,
    tilt_hz: float = 250.0,
    rng: np.random.Generator | None = None,
) -> Array:
    """Filtered impulse train with mild jitter/shimmer (Rosenberg-ish tilt)."""
    rng = rng or _rng(None)
    t = np.arange(n, dtype=np.float64) / float(sr)
    # Time-varying F0: slow drift + light vibrato, plus cycle jitter.
    vibrato = 1.0 + 0.012 * np.sin(2.0 * np.pi * 5.2 * t)
    drift = 1.0 + 0.03 * (t / max(t[-1], 1e-9) - 0.5)
    inst_f0 = f0 * vibrato * drift
    phase = np.cumsum(inst_f0 / sr)
    # Emit a pulse each time phase crosses an integer.
    crossed = np.diff(np.floor(phase), prepend=phase[0])
    pulses = (crossed > 0).astype(np.float64)
    if jitter > 0:
        # Nudge non-zero pulse amplitudes; timing jitter via F0 already.
        mag = 1.0 + shimmer * rng.normal(0.0, 1.0, size=n)
        pulses = pulses * mag
        # Small sample-level timing noise on pulse locations.
        shift = rng.integers(-1, 2, size=n) if jitter > 0 else 0
        if np.ndim(shift) > 0:
            idx = np.nonzero(pulses)[0]
            for i in idx:
                s = int(shift[i])
                if s != 0 and 0 <= i + s < n:
                    pulses[i + s] += pulses[i] * 0.15
    # Two-pole spectral tilt ~ glottal roll-off.
    nyq = 0.5 * sr
    cutoff = min(max(tilt_hz, 40.0), 0.45 * nyq)
    sos = signal.butter(2, cutoff / nyq, btype="low", output="sos")
    source = signal.sosfilt(sos, pulses)
    # Gentle high-pass to remove DC.
    sos_hp = signal.butter(1, 40.0 / nyq, btype="high", output="sos")
    source = signal.sosfilt(sos_hp, source)
    peak = np.max(np.abs(source)) + 1e-12
    return (source / peak).astype(np.float64)

def resonator(x: np.ndarray, sr: int, freq: float, bw: float) -> Array:
    """Two-pole formant resonator (unity b0, conjugate poles)."""
    x = np.asarray(x, dtype=np.float64)
    r = float(np.exp(-np.pi * bw / sr))
    theta = 2.0 * np.pi * freq / sr
    a = [1.0, -2.0 * r * np.cos(theta), r * r]
    b = [1.0]
    return np.asarray(signal.lfilter(b, a, x), dtype=np.float64)

def vocal_tract(
    source: np.ndarray,
    sr: int,

```

```

        formants: tuple[tuple[float, float], ...],
    ) -> Array:
        y = np.asarray(source, dtype=np.float64)
        for freq, bw in formants:
            y = resonator(y, sr, freq, bw)
        peak = np.max(np.abs(y)) + 1e-12
        return (y / peak).astype(np.float64)

def bone_coupling(
    source: np.ndarray,
    sr: int,
    cutoff_hz: float = 1200.0,
    resonance_hz: float = 400.0,
    resonance_bw: float = 180.0,
) -> Array:
    """Tissue/bone coupler: low-pass plus a mild jaw/skull resonance."""
    x = np.asarray(source, dtype=np.float64)
    nyq = 0.5 * sr
    cut = min(max(cutoff_hz, 80.0), 0.45 * nyq)
    sos = signal.butter(3, cut / nyq, btype="low", output="sos")
    y = signal.sosfilt(sos, x)
    y = resonator(y, sr, resonance_hz, resonance_bw)
    peak = np.max(np.abs(y)) + 1e-12
    return (y / peak).astype(np.float64)

DEFAULT_FORMANTS: tuple[tuple[float, float], ...] = (
    (500.0, 70.0),
    (1500.0, 100.0),
    (2500.0, 140.0),
)

@dataclass
class DualPair:
    """A synthetic air + vibration pair sharing one glottal source."""

    audio: Array
    vibration: Array
    sr: int
    f0: float
    formants: tuple[tuple[float, float], ...]

def make_pair(
    duration_s: float = 1.2,
    sr: int = 16000,
    f0: float = 120.0,
    formants: tuple[tuple[float, float], ...] | None = None,
    bone_cutoff_hz: float = 1200.0,
    air_delay_s: float = 0.0025,
    noise_db: float = -35.0,
    seed: int | None = None,
) -> DualPair:
    """Build one physically-plausible dual-channel pair.

    ``air_delay_s`` models vocal-tract group delay (air lags the source
    slightly). Independent sensor noise is added at ``noise_db``.
    """
    rng = _rng(seed)
    formants = formants or DEFAULT_FORMANTS
    n = int(round(duration_s * sr))
    n = max(n, 256)
    source = glottal_source(n, sr, f0=f0, rng=rng)
    audio = vocal_tract(source, sr, formants)
    vibration = bone_coupling(source, sr, cutoff_hz=bone_cutoff_hz)
    delay = int(round(air_delay_s * sr))
    if delay > 0:
        delayed = np.zeros_like(audio)
        delayed[delay:] = audio[: n - delay]
        audio = delayed
    noise_gain = 10.0 ** (noise_db / 20.0)
    audio = audio + noise_gain * rng.normal(0.0, 1.0, size=n)
    vibration = vibration + noise_gain * rng.normal(0.0, 1.0, size=n)
    # Short cosine fade-in/out so offset-decay tests see a real ending,
    # not an abrupt clip (and not a reverb tail).
    fade = min(int(0.04 * sr), n // 8)
    if fade > 2:
        ramp = 0.5 - 0.5 * np.cos(np.linspace(0.0, np.pi, fade))
        audio[:fade] *= ramp
        audio[-fade:] *= ramp[::-1]
        vibration[:fade] *= ramp
        vibration[-fade:] *= ramp[::-1]
    # Peak-normalize independently (contact mic vs air mic gains differ).
    audio = audio / (np.max(np.abs(audio)) + 1e-12)
    vibration = vibration / (np.max(np.abs(vibration)) + 1e-12)
    return DualPair(
        audio=audio.astype(np.float64),
        vibration=vibration.astype(np.float64),
        sr=sr,
        f0=f0,
        formants=formants,
    )

def formant_family(rng: np.random.Generator) -> tuple[tuple[float, float], ...]:

```

```

        """Sample a plausible 3-formant sketch (not a human dataset)."""
        f1 = float(rng.uniform(350.0, 750.0))
        f2 = float(rng.uniform(900.0, 2100.0))
        f3 = float(rng.uniform(2100.0, 3200.0))
        if f2 <= f1 + 150:
            f2 = f1 + 200
        if f3 <= f2 + 150:
            f3 = f2 + 250
        b1 = float(rng.uniform(50.0, 110.0))
        b2 = float(rng.uniform(70.0, 150.0))
        b3 = float(rng.uniform(90.0, 200.0))
        return ((f1, b1), (f2, b2), (f3, b3))

def bootstrap_pairs(
    n_pairs: int = 24,
    sr: int = 16000,
    duration_s: float = 0.9,
    seed: int = 202607,
) -> list[DualPair]:
    """Draw a synthetic bootstrap ensemble for the transfer-function prior."""
    rng = _rng(seed)
    pairs: list[DualPair] = []
    for i in range(n_pairs):
        f0 = float(rng.uniform(85.0, 230.0))
        formants = formant_family(rng)
        cutoff = float(rng.uniform(800.0, 1600.0))
        delay = float(rng.uniform(0.001, 0.006))
        pairs.append(
            make_pair(
                duration_s=duration_s,
                sr=sr,
                f0=f0,
                formants=formants,
                bone_cutoff_hz=cutoff,
                air_delay_s=delay,
                noise_db=float(rng.uniform(-40.0, -28.0)),
                seed=int(seed + 17 * i + 3),
            )
        )
    return pairs

```

forgereceipts/engines/vibelock/ui.py

```

"""Localhost UI for VibeLock. Binds 127.0.0.1. No CDN, no outbound calls."""

from __future__ import annotations

import base64
import json
import sys
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from io import BytesIO
from typing import Any
from urllib.parse import urlparse

import numpy as np
from scipy.io import wavfile

from vibelock import __version__, analyze
from vibelock.dsp import as_mono_float, resample
from vibelock.synth import make_pair

DEFAULT_HOST = "127.0.0.1"
DEFAULT_PORT = 8760
MAX_BODY = 12 * 1024 * 1024

PAGE = r"""<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>VibeLock</title>
<style>
:root {
  --bg: #0c1118;
  --card: #141c27;
  --ink: #e8eef6;
  --muted: #8fa0b5;
  --line: #243044;
  --accent: #3ec6b0;
  --accent-dim: #1b4f48;
  --warn: #e0b46a;
  --bad: #e07a7a;
  --ok: #7dcea0;
}
* { box-sizing: border-box; }
html, body { margin: 0; background: var(--bg); color: var(--ink);
  font-family: "Iowan Old Style", "Palatino Linotype", Palatino, Georgia, serif; }
body { min-height: 100vh; }
main { max-width: 42rem; margin: 0 auto; padding: 2.4rem 1.25rem 4rem; }
header { margin-bottom: 1.6rem; }
.mark { font-size: 0.72rem; letter-spacing: 0.22em; text-transform: uppercase;
  color: var(--accent); margin: 0 0 0.45rem; }
h1 { font-weight: 500; letter-spacing: 0.04em; font-size: 2.1rem; margin: 0 0 0.4rem; }
.motto { font-style: italic; color: var(--muted); margin: 0; }
.local { display: inline-block; margin-top: 0.85rem; font-size: 0.78rem;
  letter-spacing: 0.04em; color: var(--muted); border: 1px solid var(--line);
  padding: 0.2rem 0.55rem; border-radius: 999px; font-family: ui-monospace, monospace; }
.card { background: var(--card); border: 1px solid var(--line); border-radius: 12px;
  padding: 1.15rem 1.2rem 1.25rem; margin: 1.1rem 0; }
h2 { font-size: 1.02rem; font-weight: 500; margin: 0 0 0.75rem; letter-spacing: 0.03em; }
p.help { color: var(--muted); font-size: 0.92rem; margin: 0 0 0.9rem; }
label { display: block; font-size: 0.82rem; color: var(--muted); margin: 0.55rem 0 0.28rem; }
input[type=file], select { width: 100%; background: #0c1118; color: var(--ink);
  border: 1px solid var(--line); padding: 0.45rem 0.55rem; border-radius: 6px; }
.row { display: flex; gap: 0.7rem; flex-wrap: wrap; align-items: center; margin-top: 0.9rem; }
button { font-family: inherit; cursor: pointer; border-radius: 8px; padding: 0.55rem 1.1rem;
  border: 1px solid var(--accent); background: var(--accent-dim); color: var(--ink); }
button.primary { background: var(--accent); color: #06221e; font-weight: 600; border: 0; }
button.ghost { background: transparent; color: var(--muted); border-color: var(--line); }
button.disabled { opacity: 0.5; cursor: wait; }
.score-wrap { display: flex; align-items: baseline; gap: 0.85rem; margin: 0.4rem 0 0.8rem; }
.score { font-size: 3rem; font-variant-numeric: tabular-nums; letter-spacing: -0.03em; }
.mode { color: var(--muted); font-family: ui-monospace, monospace; font-size: 0.85rem; }
.bar { height: 8px; background: #0c1118; border-radius: 99px; overflow: hidden; border: 1px solid var(--line); }
.bar > span { display: block; height: 100%; background: var(--accent); }
.codes { display: flex; flex-wrap: wrap; gap: 0.35rem; margin: 0.85rem 0; }
.code { font-family: ui-monospace, monospace; font-size: 0.75rem; padding: 0.2rem 0.45rem;
  border-radius: 4px; border: 1px solid var(--line); color: var(--warn); }
.code.ok { color: var(--ok); }
.checks { list-style: none; padding: 0; margin: 0.6rem 0 0; }
.checks li { display: flex; justify-content: space-between; gap: 1rem;
  font-family: ui-monospace, monospace; font-size: 0.8rem; padding: 0.28rem 0;
  border-bottom: 1px solid #1c2736; }
.err { color: var(--bad); }
footer { margin-top: 2rem; color: #6d7c8f; font-size: 0.8rem; }
footer code { color: var(--muted); }
</style>
</head>
<body>
<main>
  <header>
    <p class="mark">Aziel Eliab ? July 2026</p>
    <h1>VibeLock</h1>
    <p class="motto">Sound can be forged. Physics is harder to fake.</p>

```

```

    <span class="local">localhost ? 127.0.0.1 ? never uploaded</span>
  </header>

  <section class="card">
    <h2>Evaluate authenticity</h2>
    <p class="help">Air WAV, optional body-coupled vibration, or a synthetic physically-plausible pair generated here.
    DSP stays on this machine.</p>
    <label>Air microphone WAV</label>
    <input id="air" type="file" accept=".wav,audio/wav">
    <label>Vibration WAV (optional)</label>
    <input id="vib" type="file" accept=".wav,audio/wav">
    <div class="row">
      <button class="primary" id="go" type="button">Evaluate authenticity</button>
      <button class="ghost" id="synth" type="button">Generate synthetic pair</button>
    </div>
  </section>

  <section class="card" id="result" hidden>
    <h2>Result</h2>
    <div class="score-wrap">
      <div class="score" id="score">?</div>
      <div class="mode" id="mode"></div>
    </div>
    <div class="bar"><span id="bar" style="width:0%"></span></div>
    <div class="codes" id="codes"></div>
    <ul class="checks" id="checks"></ul>
    <p class="help" id="notes"></p>
  </section>
  <p class="err" id="err" hidden></p>

  <footer>
    VibeLock __VERSION__ ? Apache-2.0 ? forks welcome
    ? <code>vibelock ui</code>
  </footer>
</main>
<script>
(function () {
  const $ = (id) => document.getElementById(id);
  function b64(file) {
    return new Promise((resolve, reject) => {
      const r = new FileReader();
      r.onload = () => {
        const s = String(r.result || "");
        const i = s.indexOf(",");
        resolve(i >= 0 ? s.slice(i + 1) : s);
      };
      r.onerror = () => reject(r.error);
      r.readAsDataURL(file);
    });
  };
  function show(data) {
    $("err").hidden = true;
    $("result").hidden = false;
    const s = Number(data.score);
    $("score").textContent = s.toFixed(3);
    $("mode").textContent = data.mode || "";
    $("bar").style.width = Math.round(s * 100) + "%";
    const codes = data.reason_codes || [];
    const box = $("codes");
    box.innerHTML = "";
    if (!codes.length) {
      const el = document.createElement("span");
      el.className = "code ok";
      el.textContent = "no reason codes";
      box.appendChild(el);
    } else {
      codes.forEach((c) => {
        const el = document.createElement("span");
        el.className = "code";
        el.textContent = c;
        box.appendChild(el);
      });
    }
    const ul = $("checks");
    ul.innerHTML = "";
    (data.checks || []).forEach((ch) => {
      const li = document.createElement("li");
      const flag = ch.reason_code ? " [" + ch.reason_code + "]" : "";
      li.innerHTML = "<span>" + ch.name + flag + "</span><span>" + Number(ch.score).toFixed(3) + "</span>";
      ul.appendChild(li);
    });
    $("notes").textContent = (data.notes || []).join(" ");
  }
  function fail(msg) {
    $("err").hidden = false;
    $("err").textContent = msg;
  }
  async function post(url, body) {
    const res = await fetch(url, {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify(body),
    });
    const data = await res.json();
    if (!res.ok) throw new Error(data.error || ("HTTP " + res.status));
  }
}

```

```

    return data;
}
$("go").onclick = async () => {
    const air = $("air").files[0];
    if (!air) { fail("Choose an air-microphone WAV, or generate a synthetic pair."); return; }
    $("go").disabled = true;
    try {
        const payload = { audio_b64: await b64(air), filename: air.name };
        const vib = $("vib").files[0];
        if (vib) payload.vibration_b64 = await b64(vib);
        show(await post("/api/analyze", payload));
    } catch (e) { fail(String(e.message || e)); }
    finally { $("go").disabled = false; }
};
$("synth").onclick = async () => {
    $("synth").disabled = true;
    try { show(await post("/api/synth", {})); }
    catch (e) { fail(String(e.message || e)); }
    finally { $("synth").disabled = false; }
};
})();
</script>
</body>
</html>
""".replace("__VERSION__", __version__)

```

```

def _pcm_to_float(data: np.ndarray) -> np.ndarray:
    data = np.asarray(data)
    if np.issubdtype(data.dtype, np.floating):
        arr = data.astype(np.float64)
        peak = np.max(np.abs(arr)) if arr.size else 1.0
        if peak > 8.0:
            arr = arr / max(peak, 1.0)
        return as_mono_float(arr)
    info = np.iinfo(data.dtype)
    scale = float(max(abs(info.min), info.max))
    return as_mono_float(data.astype(np.float64) / scale)

def _wav_from_b64(blob: str) -> tuple[np.ndarray, int]:
    raw = base64.b64decode(blob)
    sr, data = wavfile.read(BytesIO(raw))
    return _pcm_to_float(data), int(sr)

class Handler(BaseHTTPRequestHandler):
    def log_message(self, fmt: str, *args: object) -> None:
        sys.stderr.write("%s - %s\n"
            " % (self.address_string(), fmt % args))

    def _send(self, code: int, body: bytes, content_type: str) -> None:
        self.send_response(code)
        self.send_header("Content-Type", content_type)
        self.send_header("Content-Length", str(len(body)))
        self.send_header("Cache-Control", "no-store")
        self.end_headers()
        self.wfile.write(body)

    def _json(self, code: int, obj: Any) -> None:
        raw = json.dumps(obj, indent=2).encode("utf-8")
        self._send(code, raw, "application/json; charset=utf-8")

    def _read_json(self) -> dict[str, Any]:
        length = int(self.headers.get("Content-Length") or 0)
        if length > MAX_BODY:
            raise ValueError("payload too large")
        raw = self.rfile.read(length) if length else b"{}"
        data = json.loads(raw.decode("utf-8") or "{}")
        if not isinstance(data, dict):
            raise ValueError("expected a JSON object")
        return data

    def do_GET(self) -> None: # noqa: N802
        path = urlparse(self.path).path
        if path in ("/", "/index.html"):
            body = PAGE.encode("utf-8")
            self._send(200, body, "text/html; charset=utf-8")
            return
        if path == "/health":
            self._json(200, {"ok": True, "bind_host": DEFAULT_HOST, "name": "VibeLock"})
            return
        self._json(404, {"error": "not found"})

    def do_POST(self) -> None: # noqa: N802
        path = urlparse(self.path).path
        try:
            if path == "/api/synth":
                pair = make_pair(duration_s=1.0, sr=16000, f0=120.0, seed=20260728)
                result = analyze(pair.audio, pair.sr, vibration=pair.vibration)
                payload = result.to_dict()
                payload["synthetic"] = True
                self._json(200, payload)
                return
            if path == "/api/analyze":

```

```

        body = self._read_json()
        audio_b64 = body.get("audio_b64")
        if not audio_b64:
            self._json(400, {"error": "audio_b64 is required"})
            return
        audio, sr = _wav_from_b64(str(audio_b64))
        vibration = None
        if body.get("vibration_b64"):
            vib, vsr = _wav_from_b64(str(body["vibration_b64"]))
            if vsr != sr:
                vib = resample(vib, vsr, sr)
            n = min(audio.size, vib.size)
            audio, vibration = audio[:n], vib[:n]
        result = analyze(audio, sr, vibration=vibration)
        self._json(200, result.to_dict())
        return
    except Exception as exc: # noqa: BLE001 ? surface decode/analyze errors to the UI
        self._json(400, {"error": str(exc)})
        return
    self._json(404, {"error": "not found"})

def make_server(host: str = DEFAULT_HOST, port: int = DEFAULT_PORT) -> ThreadingHTTPServer:
    return ThreadingHTTPServer((host, port), Handler)

def serve(host: str = DEFAULT_HOST, port: int = DEFAULT_PORT) -> None:
    httpd = make_server(host, port)
    url = f"http://{host}:{port}/"
    sys.stdout.write(f"VibeLock UI {url}\n")
    sys.stdout.write("Local only. Audio is not retained and never leaves this process.\n")
    sys.stdout.flush()
    try:
        httpd.serve_forever()
    except KeyboardInterrupt:
        sys.stdout.write("\nstopped\n")
    finally:
        httpd.server_close()

```

forgereceipts/forgereceipts/__init__.py

```
"""ForgeReceipts: local-first evidence integrity for pro se fathers.

July 2026. Author: Aziel Eliab, Indianapolis.

Child's Best Interests First. Integrity Over Narrative. Local Control. Always.

This software is not legal advice and does not guarantee any court outcome.
It does not contact courts, Odyssey, email, or any cloud service.
No telemetry. No accounts. Forks are welcome and always allowed.
"""

from __future__ import annotations

import sys
from pathlib import Path

_ROOT = Path(__file__).resolve().parent.parent
_ENGINES = _ROOT / "engines"
if _ENGINES.is_dir():
    _engine_str = str(_ENGINES)
    if _engine_str not in sys.path:
        sys.path.insert(0, _engine_str)

__version__ = "0.2.0"
__author__ = "Aziel Eliab"
__all__ = ["__version__"]
```


forgereceipts/forgereceipts/__main__.py

```
from forgereceipts.cli import main

if __name__ == "__main__":
    raise SystemExit(main())
```

forgereceipts/forgereceipts/cli.py

```

"""ForgeReceipts command line.

forgereceipts ui          # 127.0.0.1:8787 THE product
forgereceipts doctor      # local health check
forgereceipts verify-receipt # check a saved receipt file
forgereceipts version
"""

from __future__ import annotations

import argparse
import sys

from forgereceipts import __version__
from forgereceipts.debug import debug_enabled, debug_log
from forgereceipts.plain import NOT_LEGAL_PROOF, PlainError

LOCAL_HOSTS = {"127.0.0.1", "localhost", "::1"}

def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(
        prog="forgereceipts",
        description=(
            "ForgeReceipts ? local-first evidence integrity. "
            "Not legal advice. A receipt is not legal proof. "
            "Local UI: `forgereceipts ui` at http://127.0.0.1:8787. "
            "Binds 127.0.0.1 only. No telemetry."
        ),
    )
    sub = parser.add_subparsers(dest="cmd")

    ui_p = sub.add_parser("ui", help="serve the local product on 127.0.0.1:8787")
    ui_p.add_argument("--host", default="127.0.0.1")
    ui_p.add_argument("--port", type=int, default=8787)
    ui_p.add_argument("--data-dir", default=None, help="override ./forgereceipts")

    doc_p = sub.add_parser("doctor", help="check that this computer can save receipts")
    doc_p.add_argument("--data-dir", default=None, help="override ./forgereceipts")
    doc_p.add_argument("--json", action="store_true", help="print JSON instead of plain text")

    ver_p = sub.add_parser("verify-receipt", help="check that a receipt file still matches its hash")
    ver_p.add_argument("path", help="path to a .json or .jsonl receipt file")

    sub.add_parser("version", help="print version")

    args = parser.parse_args(argv)
    debug_log(f"cli cmd={args.cmd} debug={debug_enabled()}")

    if args.cmd == "version":
        print(__version__)
        return 0

    if args.cmd == "ui":
        host = args.host
        if host not in LOCAL_HOSTS:
            print(
                "ForgeReceipts binds local-only (127.0.0.1). Refusing host="
                f"{host!r}."
            )
            file=sys.stderr,
        )
        return 2
    from forgereceipts.ui import serve

    serve(host=host, port=args.port, data_dir=args.data_dir)
    return 0

    if args.cmd == "doctor":
        from forgereceipts.doctor import format_doctor, run_doctor
        import json as json_lib

        report = run_doctor(args.data_dir)
        if args.json:
            print(json_lib.dumps(report, indent=2, ensure_ascii=False))
        else:
            sys.stdout.write(format_doctor(report))
        return 0 if report.get("ok") else 1

    if args.cmd == "verify-receipt":
        from forgereceipts.exchange import verify_path
        import json as json_lib

        try:
            result = verify_path(args.path)
        except PlainError as exc:
            print(str(exc), file=sys.stderr)
            print(NOT_LEGAL_PROOF, file=sys.stderr)
            return 2
        except OSError:
            print("Could not read that file.", file=sys.stderr)
            return 2
        print(result.get("verdict") or "FAIL")

```

```
    print(result.get("plain") or "")
    if result.get("hash"):
        print(f"hash: {result['hash']}")
    if debug_enabled():
        print(json_lib.dumps(result, indent=2, ensure_ascii=False))
    print(result.get("disclaimer") or NOT_LEGAL_PROOF)
    return 0 if result.get("ok") else 1

parser.print_help()
print("\nNot legal advice. A receipt is not legal proof. Child's Best Interests First.")
return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

forgereceipts/forgereceipts/debug.py

```
"""FORGERECEIPTS_DEBUG=1 ? extra stderr logs, extra error fields. Off by default.

Never sends anything anywhere. Debug is local stderr only.
"""

from __future__ import annotations

import os
import sys

DEBUG_ENV = "FORGERECEIPTS_DEBUG"

def debug_enabled() -> bool:
    value = os.environ.get(DEBUG_ENV, "")
    return value.strip().lower() in {"1", "true", "yes", "on"}

def debug_log(message: str) -> None:
    if debug_enabled():
        sys.stderr.write(f"[forgereceipts debug] {message}\n")
        sys.stderr.flush()
```

forgereceipts/forgereceipts/demo.py

```
"""One-click sample so a 6th grader can see a receipt without picking a file."""

from __future__ import annotations

from pathlib import Path

from forgereceipts.forensics import sha256_bytes

SAMPLE_NAME = "sample-demo.txt"
SAMPLE_TEXT = (
    "This is a sample file for ForgeReceipts.\n"
    "Not legal advice. A receipt is not legal proof.\n"
)
SAMPLE_BYTES = SAMPLE_TEXT.encode("utf-8")

def sample_sha256() -> str:
    return sha256_bytes(SAMPLE_BYTES)

def write_sample(data_dir: str | Path) -> Path:
    dest = Path(data_dir) / "demo" / SAMPLE_NAME
    dest.parent.mkdir(parents=True, exist_ok=True)
    dest.write_bytes(SAMPLE_BYTES)
    return dest
```

forgereceipts/forgereceipts/doctor.py

```

"""Local health check. Does not talk to the network. No telemetry."""

from __future__ import annotations

import sys
from pathlib import Path
from typing import Any, Callable

from forgereceipts import __version__
from forgereceipts.debug import DEBUG_ENV, debug_enabled, debug_log
from forgereceipts.demo import SAMPLE_BYTES, write_sample
from forgereceipts.exchange import dump_receipt, roundtrip, verify_text
from forgereceipts.forensics import sha256_bytes
from forgereceipts.limits import MAX_BODY, MAX_FILE_BYTES, MAX_JSON_CHARS
from forgereceipts.paths import data_dir as resolve_data_dir
from forgereceipts.plain import NOT_LEGAL_PROOF
from forgereceipts.store import ForgeStore, verify_jsonl

STATIC_DIR = Path(__file__).resolve().parent / "static"
DEFAULT_HOST = "127.0.0.1"

CheckFn = Callable[[], tuple[bool, str, str]]

def _check_python() -> tuple[bool, str, str]:
    ver = sys.version_info
    ok = ver >= (3, 10)
    detail = f"{ver.major}.{ver.minor}.{ver.micro}"
    plain = (
        f"Python {detail} is new enough."
        if ok
        else f"Python {detail} is too old. ForgeReceipts needs 3.10 or newer."
    )
    return ok, detail, plain

def _check_version() -> tuple[bool, str, str]:
    ok = bool(__version__)
    return ok, __version__, f"This is ForgeReceipts {__version__}."

def _check_loopback() -> tuple[bool, str, str]:
    from forgereceipts.ui import make_server

    try:
        make_server("0.0.0.0", 0)
    except ValueError:
        return True, DEFAULT_HOST, "ForgeReceipts only listens on this computer (127.0.0.1)."
    return False, "open", "ForgeReceipts would listen on the network. That is not allowed."

def _check_telemetry() -> tuple[bool, str, str]:
    return True, "off", "No telemetry. Nothing is sent anywhere."

def _check_limits() -> tuple[bool, str, str]:
    ok = MAX_BODY == MAX_FILE_BYTES == 12 * 1024 * 1024 and MAX_JSON_CHARS > 0
    detail = f"HTTP {MAX_BODY} bytes, JSON {MAX_JSON_CHARS} chars"
    plain = "Big files over 12 MB are refused so this computer stays safe."
    return ok, detail, plain

def _check_static() -> tuple[bool, str, str]:
    index = STATIC_DIR / "index.html"
    js = STATIC_DIR / "app.js"
    css = STATIC_DIR / "style.css"
    ok = index.is_file() and js.is_file() and css.is_file()
    detail = str(STATIC_DIR)
    plain = (
        "The local page files are on this computer."
        if ok
        else "The local page files are missing."
    )
    return ok, detail, plain

def _check_data_dir(directory: Path) -> tuple[bool, str, str]:
    try:
        directory.mkdir(parents=True, exist_ok=True)
        probe = directory / ".doctor-write"
        probe.write_text("ok\n", encoding="utf-8")
        probe.unlink()
        return True, str(directory), "The local folder can save receipts."
    except OSError as exc:
        return False, str(directory), f"The local folder cannot be written: {exc}"

def _check_demo() -> tuple[bool, str, str]:
    got = sha256_bytes(SAMPLE_BYTES)
    ok = len(got) == 64 and got == sha256_bytes(SAMPLE_BYTES)
    return ok, got, "The sample file hashes the same way every time."

def _check_chain(directory: Path) -> tuple[bool, str, str]:

```

```

path = directory / "chain.jsonl"
if not path.is_file() or not path.read_text(encoding="utf-8").strip():
    return True, "empty", "No receipts yet. That is OK."
text = path.read_text(encoding="utf-8")
result = verify_jsonl(text)
ok = bool(result.get("ok"))
detail = str(result.get("verdict") or "FAIL")
plain = (
    "Your saved receipts still match their hashes."
    if ok
    else "A saved receipt no longer matches its hash."
)
return ok, detail, plain

def _check_roundtrip(directory: Path) -> tuple[bool, str, str]:
    tmp = directory / "doctor-roundtrip"
    tmp.mkdir(parents=True, exist_ok=True)
    store = ForgeStore(tmp)
    sample = write_sample(tmp)
    digest = sha256_bytes(SAMPLE_BYTES)
    row = store.append_forensics(
        summary="doctor sample",
        file_sha256=digest,
        file_name=sample.name,
        child_impact="Sample only. A receipt is not legal proof.",
        timestamp="2026-09-02T00:00:00Z",
    )
    result = roundtrip(row)
    verified = verify_text(dump_receipt(row))
    ok = bool(result.get("ok") and verified.get("ok") and sample.is_file())
    return ok, str(result.get("verdict") or "FAIL"), str(
        result.get("plain") or "Roundtrip check."
    )
)

def run_doctor(data_dir: str | Path | None = None) -> dict[str, Any]:
    directory = resolve_data_dir(data_dir)
    debug_log(f"doctor data_dir={directory} debug={debug_enabled()}")

    checks: list[dict[str, Any]] = []

    def add(cid: str, fn: CheckFn) -> None:
        ok, detail, plain = fn()
        checks.append(
            {
                "id": cid,
                "ok": bool(ok),
                "verdict": "PASS" if ok else "FAIL",
                "detail": detail,
                "plain": plain,
            }
        )

    add("python", _check_python)
    add("version", _check_version)
    add("loopback", _check_loopback)
    add("telemetry", _check_telemetry)
    add("size_limits", _check_limits)
    add("static", _check_static)
    add("data_dir", lambda: _check_data_dir(directory))
    add("demo", _check_demo)
    add("roundtrip", lambda: _check_roundtrip(directory))
    add("chain", lambda: _check_chain(directory))
    add(
        "debug",
        lambda: (
            True,
            "on" if debug_enabled() else "off",
            (
                f"{DEBUG_ENV}=1 is on. Extra logs go to this computer only."
                if debug_enabled()
                else f"{DEBUG_ENV} is off. That is the usual setting."
            ),
        ),
    )

    ok = all(c["ok"] for c in checks)
    return {
        "ok": ok,
        "verdict": "PASS" if ok else "FAIL",
        "version": __version__,
        "disclaimer": NOT_LEGAL_PROOF,
        "not_legal_advice": True,
        "not_legal_proof": True,
        "telemetry": False,
        "bind": DEFAULT_HOST,
        "debug": debug_enabled(),
        "data_dir": str(directory),
        "plain": (
            "Everything looks OK. You can save receipts on this computer."
            if ok
            else "Something is not right. Read the FAIL lines."
        ),
        "checks": checks,
    }

```

```
}

def format_doctor(report: dict[str, Any]) -> str:
    lines = [
        f"ForgeReceipts doctor {report.get('version')}",
        str(report.get("disclaimer") or NOT_LEGAL_PROOF),
        "No telemetry. Loopback only. Receipts are not legal proof.",
        ""
    ]
    for c in report.get("checks") or []:
        flag = "PASS" if c.get("ok") else "FAIL"
        ident = str(c.get("id") or "")
        lines.append(f"[{flag}] {ident:<12} {c.get('plain')}")
        if debug_enabled() and c.get("detail"):
            lines.append(f"          detail: {c['detail']}")
    lines.append("")
    if report.get("ok"):
        lines.append("ALL CHECKS PASSED")
    else:
        lines.append("SOME CHECKS FAILED")
    lines.append(str(report.get("plain") or ""))
    return "\n".join(lines).rstrip() + "\n"
```


forgereceipts/forgereceipts/exchange.py

```

"""Import and export receipts as JSON. Verify a file without trusting it.

A receipt is a local hash record. It is not legal proof and not a court filing.
"""

from __future__ import annotations

import json
from pathlib import Path
from typing import Any

from forgereceipts import __version__
from forgereceipts.debug import debug_log
from forgereceipts.limits import MAX_JSON_CHARS
from forgereceipts.plain import (
    JSON_ERROR,
    MISSING_RECEIPT,
    NOT_LEGAL_PROOF,
    NOT_OBJECT,
    NOT_TEXT,
    TOO_BIG,
    PlainError,
    decode_text,
    loads_json,
)

from temporallock.receipt import Receipt

FORMAT_RECEIPT = "forgereceipts.receipt/v1"
FORMAT_BUNDLE = "forgereceipts.bundle/v1"
SAVED_PLAIN = "Saved a receipt for this file"

def envelope_for(receipt: dict[str, Any]) -> dict[str, Any]:
    return {
        "format": FORMAT_RECEIPT,
        "product": "forgereceipts",
        "product_version": __version__,
        "disclaimer": NOT_LEGAL_PROOF,
        "not_legal_advice": True,
        "not_legal_proof": True,
        "receipt": dict(receipt),
    }

def bundle_for(receipts: list[dict[str, Any]]) -> dict[str, Any]:
    return {
        "format": FORMAT_BUNDLE,
        "product": "forgereceipts",
        "product_version": __version__,
        "disclaimer": NOT_LEGAL_PROOF,
        "not_legal_advice": True,
        "not_legal_proof": True,
        "receipts": [dict(r) for r in receipts],
    }

def dump_receipt(receipt: dict[str, Any]) -> str:
    return json.dumps(envelope_for(receipt), indent=2, ensure_ascii=False) + "\n"

def dump_bundle(receipts: list[dict[str, Any]]) -> str:
    return json.dumps(bundle_for(receipts), indent=2, ensure_ascii=False) + "\n"

def _as_receipt_dict(obj: Any) -> dict[str, Any] | None:
    if not isinstance(obj, dict):
        return None
    if isinstance(obj.get("receipt"), dict):
        return obj["receipt"]
    if "hash" in obj and "summary" in obj and "evidence" in obj:
        return obj
    return None

def _parse_jsonl(text: str) -> list[Any]:
    rows: list[Any] = []
    for line in text.splitlines():
        line = line.strip()
        if not line:
            continue
        try:
            rows.append(json.loads(line))
        except json.JSONDecodeError as exc:
            raise PlainError(JSON_ERROR) from exc
    return rows

def parse_payload(text: str) -> dict[str, Any]:
    """Parse envelope, bundle, bare receipt, or JSONL into a structured payload."""
    if len(text) > MAX_JSON_CHARS:
        raise PlainError(TOO_BIG)
    stripped = text.strip()

```

```

    if not stripped:
        raise PlainError(MISSING_RECEIPT)

    obj: Any | None = None
    extra = False
    try:
        obj = loads_json(stripped)
    except json.JSONDecodeError:
        extra = True
        obj = None
    except PlainError:
        raise

    if extra:
        rows = _parse_jsonl(stripped)
        receipts = []
        for row in rows:
            rec = _as_receipt_dict(row)
            if rec is None:
                raise PlainError(NOT_OBJECT)
            receipts.append(rec)
        if not receipts:
            raise PlainError(MISSING_RECEIPT)
        return {"kind": "jsonl", "receipts": receipts}

    if isinstance(obj, list):
        receipts = []
        for row in obj:
            rec = _as_receipt_dict(row)
            if rec is None:
                raise PlainError(NOT_OBJECT)
            receipts.append(rec)
        if not receipts:
            raise PlainError(MISSING_RECEIPT)
        return {"kind": "list", "receipts": receipts}

    if not isinstance(obj, dict):
        raise PlainError(NOT_OBJECT)

    if isinstance(obj.get("receipts"), list):
        receipts = []
        for row in obj["receipts"]:
            rec = _as_receipt_dict(row)
            if rec is None:
                raise PlainError(NOT_OBJECT)
            receipts.append(rec)
        if not receipts:
            raise PlainError(MISSING_RECEIPT)
        return {"kind": "bundle", "receipts": receipts, "raw": obj}

    rec = _as_receipt_dict(obj)
    if rec is None:
        raise PlainError(MISSING_RECEIPT)
    return {"kind": "receipt", "receipts": [rec], "raw": obj}

def verify_one(receipt: dict[str, Any]) -> dict[str, Any]:
    errors: list[str] = []
    try:
        rec = Receipt.from_dict(receipt)
    except Exception as exc: # noqa: BLE001 ? surface as FAIL, not a crash
        debug_log(f"verify_one from_dict: {type(exc).__name__}: {exc}")
        return {
            "ok": False,
            "verdict": "FAIL",
            "plain": "This file does not look like a receipt.",
            "disclaimer": NOT_LEGAL_PROOF,
            "hash": receipt.get("hash"),
            "errors": [str(exc)],
        }
    ok = rec.hash_ok()
    if not ok:
        errors.append("stored hash does not match SHA-256 of the receipt fields")
    plain = (
        "This receipt looks whole. The hash matches."
        if ok
        else "This receipt does not match its hash."
    )
    result = {
        "ok": ok,
        "verdict": "PASS" if ok else "FAIL",
        "plain": plain,
        "disclaimer": NOT_LEGAL_PROOF,
        "hash": rec.hash,
        "recomputed_hash": rec.recomputed_hash(),
        "errors": errors,
        "not_legal_proof": True,
        "not_legal_advice": True,
    }
    debug_log(f"verify_one verdict={result['verdict']} hash={rec.hash}")
    return result

def verify_payload(payload: dict[str, Any]) -> dict[str, Any]:
    receipts: list[dict[str, Any]] = list(payload.get("receipts") or [])

```

```

if not receipts:
    raise PlainError(MISSING_RECEIPT)
results = [verify_one(r) for r in receipts]
ok = all(r["ok"] for r in results)
if payload.get("kind") in {"jsonl", "bundle", "list"} and len(receipts) > 1:
    from forgereceipts.store import verify_jsonl

    jsonl = "\n".join(
        json.dumps(r, sort_keys=True, separators=(",", ":"), ensure_ascii=False)
        for r in receipts
    )
    chain = verify_jsonl(jsonl)
    chain_ok = bool(chain.get("ok"))
    ok = ok and chain_ok
    return {
        "ok": ok,
        "verdict": "PASS" if ok else "FAIL",
        "plain": (
            "These receipts look whole. The hashes match."
            if ok
            else "One or more receipts failed the hash check."
        ),
        "disclaimer": NOT_LEGAL_PROOF,
        "count": len(receipts),
        "receipts": results,
        "chain": chain,
        "not_legal_proof": True,
        "not_legal_advice": True,
    }
first = results[0]
first["count"] = 1
return first

def verify_text(text: str) -> dict[str, Any]:
    payload = parse_payload(text)
    return verify_payload(payload)

def verify_path(path: str | Path) -> dict[str, Any]:
    p = Path(path)
    if not p.is_file():
        raise PlainError("That path is not a file on this computer.")
    try:
        raw = p.read_bytes()
    except OSError as exc:
        raise PlainError("Could not read that file.") from exc
    try:
        text = decode_text(raw)
    except PlainError:
        raise
    except Exception as exc: # noqa: BLE001
        raise PlainError(NOT_TEXT) from exc
    return verify_text(text)

def load_receipts_from_text(text: str) -> list[dict[str, Any]]:
    return list(parse_payload(text).get("receipts") or [])

def roundtrip(receipt: dict[str, Any]) -> dict[str, Any]:
    """Export then parse then verify. Hash must match the original."""
    text = dump_receipt(receipt)
    loaded = load_receipts_from_text(text)
    if not loaded:
        raise PlainError(MISSING_RECEIPT)
    got = loaded[0]
    result = verify_one(got)
    match = got.get("hash") == receipt.get("hash") and result["ok"]
    result["roundtrip"] = True
    result["roundtrip_hash_match"] = match
    result["ok"] = bool(result["ok"] and match)
    result["verdict"] = "PASS" if result["ok"] else "FAIL"
    if result["ok"]:
        result["plain"] = "Export then import kept the same hash."
    return result

```

forgereceipts/forgereceipts/filing.py

```

"""Local filing templates and conceptual e-filing checklists.

Does not contact Odyssey, any court, email, or a cloud portal.
Export is a local .txt or .html file the user prints or uploads themselves.
"""

from __future__ import annotations

import html
from datetime import date
from typing import Any

NOT_LEGAL_ADVICE = (
    "NOT LEGAL ADVICE. This is a local template. It does not file anything. "
    "It does not talk to Odyssey or any court. Have a licensed attorney "
    "review anything you intend to file."
)

EFILING_CHECKLIST = [
    "Ask the clerk (yourself) whether your court accepts electronic filing and which portal it uses. This app does not connect to any portal.",
    "Confirm accepted file types, size limits, and naming rules with the clerk or the court's published instructions.",
    'Caption the motion with the correct court, county, parties, and cause number from *your* papers ? not from this app.',
    "Label exhibits: Petitioner's Exhibit 1, 2, 3? or Respondent's Exhibit A, B, C? Keep a hashed copy of each file in Forensics.",
    "Attach a certificate of service if your rules require one. This app does not serve anyone.",
    "Check page limits, font, and margins in the local rules. This app does not know your local rules.",
    "Print or export locally. You upload or walk the papers in. ForgeReceipts never transmits them.",
    "After you file, hash the stamped copy (Forensics) and log a receipt (Incident log). Corrections are new receipts, never edits.",
]

def exhibit_labels(party: str, count: int) -> list[str]:
    party = (party or "petitioner").strip().lower()
    n = max(0, int(count))
    if party.startswith("resp"):
        # Respondent's Exhibit A, B, C?
        labels = []
        for i in range(n):
            name = ""
            x = i
            while True:
                name = chr(ord("A") + (x % 26)) + name
                x = x // 26 - 1
                if x < 0:
                    break
            labels.append(f"Respondent's Exhibit {name}")
        return labels
    return [f"Petitioner's Exhibit {i}" for i in range(1, n + 1)]

def motion_caption(
    *,
    state: str = "Indiana",
    court_name: str = "[COURT NAME]",
    petitioner: str = "[PETITIONER]",
    respondent: str = "[RESPONDENT]",
    cause_no: str = "[CAUSE NO.]",
    party_role: str = "Petitioner",
    title: str = "MOTION",
) -> str:
    return (
        f"STATE OF {state.upper()}\n"
        f"IN THE {court_name}\n\n"
        f"IN RE THE CARE / CUSTODY OF A MINOR CHILD\n\n"
        f"{petitioner},\n"
        f"    Petitioner,\n\n"
        f"v.                                     Cause No. {cause_no}\n\n"
        f"{respondent},\n"
        f"    Respondent.\n\n"
        f"{party_role.upper()} 'S {title.upper()}\n"
    )

def render_txt(fields: dict[str, Any]) -> str:
    exhibits = exhibit_labels(fields.get("party") or "petitioner", int(fields.get("exhibit_count") or 0))
    caption = motion_caption(
        state=fields.get("state") or "Indiana",
        court_name=fields.get("court_name") or "[COURT NAME]",
        petitioner=fields.get("petitioner") or "[PETITIONER]",
        respondent=fields.get("respondent") or "[RESPONDENT]",
        cause_no=fields.get("cause_no") or "[CAUSE NO.]",
        party_role=fields.get("party_role") or "Petitioner",
        title=fields.get("title") or "MOTION",
    )
    body = (fields.get("body") or "[Body of the motion. Write in your own words. This is a placeholder.]").strip()
    lines = [
        NOT_LEGAL_ADVICE,
        "",

```

```

        caption.strip(),
        "",
        body,
        "",
        "EXHIBITS (labels only ? attach the files yourself):",
    ]
    if exhibits:
        lines.extend(f" - {lab}" for lab in exhibits)
    else:
        lines.append(" (none listed)")
    lines.extend(
        [
            "",
            "CONCEPTUAL E-FILING CHECKLIST (this app does not file):",
            *[f" [ ] {item}" for item in EFILING_CHECKLIST],
            "",
            f"Exported locally on {date.today().isoformat()}. Not served. Not filed.",
        ]
    )
    return "\n".join(lines) + "\n"

def render_html(fields: dict[str, Any]) -> str:
    text = render_txt(fields)
    escaped = html.escape(text)
    return (
        "<!DOCTYPE html><html lang='en'><head><meta charset='utf-8'>"
        "<title>ForgeReceipts local export ? NOT LEGAL ADVICE</title>"
        "<style>body{font:14px/1.45 ui-monospace,monospace;max-width:48rem;"
        "margin:2rem auto;padding:0 1rem;white-space:pre-wrap;background:#f7f4ee;"
        "color:#1b1b18;} .banner{background:#7a1f1f;color:#fff;padding:.6rem .8rem;"
        "font-family:system-ui,sans-serif;margin:0 0 1rem;}</style></head><body>"
        "<div class='banner'>NOT LEGAL ADVICE. Local export only. Not filed. "
        "Does not contact any court.</div>"
        f"<pre>{escaped}</pre></body></html>\n"
    )

def templates() -> dict[str, Any]:
    return {
        "disclaimer": NOT_LEGAL_ADVICE,
        "caption_placeholders": {
            "state": "Indiana",
            "court_name": "[COURT NAME]",
            "petitioner": "[PETITIONER]",
            "respondent": "[RESPONDENT]",
            "cause_no": "[CAUSE NO.]",
            "party_role": "Petitioner",
            "title": "MOTION",
        },
        "exhibit_examples": [
            "Petitioner's Exhibit 1",
            "Petitioner's Exhibit 2",
            "Respondent's Exhibit A",
            "Respondent's Exhibit B",
        ],
        "efiling_checklist": list(EFILING_CHECKLIST),
        "note": "Conceptual checklist only. ForgeReceipts never talks to Odyssey or any court.",
    }

```

forgereceipts/forgereceipts/forensics.py

```
"""Local SHA-256 of a file. Store the hash on a receipt; re-verify later."""

from __future__ import annotations

import hashlib
from pathlib import Path

def sha256_bytes(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()

def sha256_path(path: str | Path) -> str:
    p = Path(path)
    hasher = hashlib.sha256()
    with p.open("rb") as fh:
        while True:
            chunk = fh.read(65536)
            if not chunk:
                break
            hasher.update(chunk)
    return hasher.hexdigest()

def reverify_bytes(data: bytes, expected_hex: str) -> dict[str, object]:
    got = sha256_bytes(data)
    expected = expected_hex.strip().lower()
    ok = got == expected
    return {
        "ok": ok,
        "verdict": "PASS" if ok else "FAIL",
        "got": got,
        "expected": expected,
    }

def reverify_path(path: str | Path, expected_hex: str) -> dict[str, object]:
    got = sha256_path(path)
    expected = expected_hex.strip().lower()
    ok = got == expected
    return {
        "ok": ok,
        "verdict": "PASS" if ok else "FAIL",
        "got": got,
        "expected": expected,
        "path": str(path),
    }
```

forgereceipts/forgereceipts/legal.py

```

"""National baseline legal *names* as plain-English blurbs.

This module does not quote statutes, does not invent holdings, and is
not legal advice. Indiana is the default selector *label* because the
author lives in Indianapolis ? the body is still the national stub.
"""

from __future__ import annotations

from typing import Any

NOT_LEGAL_ADVICE = (
    "Not legal advice. A receipt is not legal proof. This software does "
    "not practice law, does not guarantee any outcome, and does not "
    "contact any court. Consult a licensed attorney in your jurisdiction. "
    "Read the primary sources."
)

NATIONAL_BASELINE: list[dict[str, str]] = [
    {
        "id": "troxel",
        "name": "Troxel v. Granville",
        "cite": "530 U.S. 57 (2000)",
        "blurb": (
            "A United States Supreme Court decision often discussed in "
            "parenting-time and third-party visitation cases. Commentators "
            "commonly describe it as addressing the liberty interest of fit "
            "parents in the care, custody, and control of their children. "
            "This software does not state the holding for your facts. "
            "Read the opinion. Not legal advice."
        ),
    },
    {
        "id": "stanley",
        "name": "Stanley v. Illinois",
        "cite": "405 U.S. 645 (1972)",
        "blurb": (
            "A United States Supreme Court decision often discussed when "
            "unmarried fathers are treated as a class rather than as "
            "individuals. Commentators commonly describe it as rejecting a "
            "blanket presumption of unfitness without an individualized "
            "hearing. This software does not state the holding for your "
            "facts. Read the opinion. Not legal advice."
        ),
    },
    {
        "id": "uccjea",
        "name": "UCCJEA",
        "cite": "Uniform Child Custody Jurisdiction and Enforcement Act",
        "blurb": (
            "A uniform act, adopted in some form by most U.S. states, that "
            "addresses which state has jurisdiction over a child-custody "
            "proceeding and how orders are enforced across state lines. "
            "This software does not determine jurisdiction, does not pick "
            "a home state, and does not contact any court. Ask an attorney "
            "whether and how the UCCJEA applies. Not legal advice."
        ),
    },
]

JURISDICTIONS = [
    {
        "id": "IN",
        "label": "Indiana (default label ? author is in Indianapolis)",
        "stub": (
            "Indiana is shown as the default selector because the canonical "
            "author lives in Indianapolis. No Indiana statute, rule, or case "
            "is quoted or summarized here. The national baseline names "
            "above are not Indiana-specific holdings. Consult an Indiana "
            "attorney and the current Indiana Code / trial rules. "
            "Not legal advice."
        ),
    },
    {
        "id": "US",
        "label": "United States (national baseline only)",
        "stub": (
            "National names only (Troxel, Stanley, UCCJEA). No state code. "
            "Not legal advice."
        ),
    },
]

def reference(jurisdiction: str = "IN") -> dict[str, Any]:
    jid = (jurisdiction or "IN").strip().upper()
    match = next((j for j in JURISDICTIONS if j["id"] == jid), JURISDICTIONS[0])
    return {
        "disclaimer": NOT_LEGAL_ADVICE,
        "jurisdiction": match,
        "baseline": list(NATIONAL_BASELINE),
        "note": (

```

```
        "State-aware in v0.2.0 means a selector label plus this stub. "  
        "It does not encode your state's best-interest factors or "  
        "parenting-time guidelines."  
    },  
}
```


forgereceipts/forgereceipts/limits.py

```
"""Hard size limits. Local-only. No telemetry.

12 MiB is large enough for a photo or a PDF exhibit and small enough
that a bad paste cannot fill the disk from the UI.
"""

from __future__ import annotations

MAX_BODY = 12 * 1024 * 1024
MAX_FILE_BYTES = 12 * 1024 * 1024
MAX_JSON_CHARS = 2 * 1024 * 1024
MAX_NOTE_CHARS = 16_384
MAX_BODY_MIB = MAX_BODY // (1024 * 1024)
```

forgereceipts/forgereceipts/lock.py

```

"""Optional local passphrase. hashlib.pbkdf2_hmac only. Hash in memory.

A verifier (salt + pbkdf2 digest) is stored under .forgereceipts/lock.json
so the same passphrase can unlock a later session. The passphrase itself
is never stored. Derived material lives in process memory for the session.
This is a local unlock, not a threat model against a sophisticated attacker
with the disk ? do not treat it as full-disk encryption.
"""

from __future__ import annotations

import hashlib
import json
import os
import secrets
from pathlib import Path
from typing import Any

ITERATIONS = 200_000
ALGO = "sha256"
DKLEN = 32

class SessionLock:
    def __init__(self, data_dir: str | Path) -> None:
        self.path = Path(data_dir) / "lock.json"
        self._unlocked = not self.is_set()
        self._session_key: bytes | None = None

    def is_set(self) -> bool:
        return self.path.is_file()

    @property
    def unlocked(self) -> bool:
        return self._unlocked

    def status(self) -> dict[str, Any]:
        return {
            "lock_set": self.is_set(),
            "unlocked": self._unlocked,
            "algo": "pbkdf2_hmac",
            "hash_name": ALGO,
            "iterations": ITERATIONS,
            "note": "Local passphrase only. Not cloud auth. Not full-disk encryption.",
        }

    def _derive(self, passphrase: str, salt: bytes) -> bytes:
        return hashlib.pbkdf2_hmac(
            ALGO,
            passphrase.encode("utf-8"),
            salt,
            ITERATIONS,
            dklen=DKLEN,
        )

    def set_passphrase(self, passphrase: str) -> None:
        if not passphrase or len(passphrase) < 4:
            raise ValueError("passphrase must be at least 4 characters")
        salt = secrets.token_bytes(16)
        digest = self._derive(passphrase, salt)
        payload = {
            "algo": "pbkdf2_hmac",
            "hash_name": ALGO,
            "iterations": ITERATIONS,
            "salt_hex": salt.hex(),
            "verifier_hex": digest.hex(),
        }
        self.path.parent.mkdir(parents=True, exist_ok=True)
        self.path.write_text(json.dumps(payload, indent=2) + "\n", encoding="utf-8")
        self._session_key = digest
        self._unlocked = True

    def unlock(self, passphrase: str) -> bool:
        if not self.is_set():
            self._unlocked = True
            return True
        data = json.loads(self.path.read_text(encoding="utf-8"))
        salt = bytes.fromhex(data["salt_hex"])
        expected = bytes.fromhex(data["verifier_hex"])
        got = self._derive(passphrase, salt)
        if not secrets.compare_digest(got, expected):
            self._unlocked = False
            return False
        self._session_key = got
        self._unlocked = True
        return True

    def lock_session(self) -> None:
        self._unlocked = not self.is_set()
        self._session_key = None

    def clear(self, passphrase: str) -> None:

```

```
if self.is_set() and not self.unlock(passphrase):
    raise ValueError("passphrase does not match")
if self.path.is_file():
    os.remove(self.path)
self._session_key = None
self._unlocked = True
```

forgereceipts/forgereceipts/paths.py

```
"""Local data directory. Never uploaded. Gitignored as .forgereceipts."""

from __future__ import annotations

import os
from pathlib import Path

DATA_DIR_NAME = ".forgereceipts"

def data_dir(root: str | Path | None = None) -> Path:
    """Resolve the local data directory.

    Order: explicit root, FORGERECEIPTS_DIR, then ``./.forgereceipts``.
    """
    if root is not None:
        path = Path(root)
    else:
        env = os.environ.get("FORGERECEIPTS_DIR")
        path = Path(env) if env else Path.cwd() / DATA_DIR_NAME
    path.mkdir(parents=True, exist_ok=True)
    return path
```

forgereceipts/forgereceipts/plain.py

```

"""Plain-English errors. Safe to show a 6th grader. No stack traces."""

from __future__ import annotations

import json
from typing import Any

from forgereceipts.limits import MAX_BODY_MIB, MAX_JSON_CHARS

JSON_ERROR = (
    "That file is not valid JSON. Check commas and quotes, then try again."
)
TOO_BIG = (
    f"That file is too big. ForgeReceipts only takes files up to {MAX_BODY_MIB} MB."
)
NOT_OBJECT = (
    "That JSON is not a receipt object. It must be a set of named fields inside { }."
)
NOT_TEXT = "That file is not text. Use a .json receipt file."
MISSING_RECEIPT = (
    "No receipt was found in that file. Export a receipt and try again."
)
NOT_LEGAL_PROOF = (
    "Not legal advice. A receipt is not legal proof."
)

class PlainError(ValueError):
    """User-facing error. Message is the whole story unless DEBUG is on."""

def decode_text(raw: bytes) -> str:
    if not raw:
        return ""
    try:
        return raw.decode("utf-8")
    except UnicodeDecodeError as exc:
        raise PlainError(NOT_TEXT) from exc

def loads_json(text: str) -> Any:
    """json.loads with a size cap and a plain error. Extra-data is re-raised."""
    if len(text) > MAX_JSON_CHARS:
        raise PlainError(TOO_BIG)
    try:
        return json.loads(text)
    except json.JSONDecodeError as exc:
        if "extra data" in str(exc).lower():
            raise
        raise PlainError(JSON_ERROR) from exc

def parse_json_bytes(raw: bytes) -> dict[str, Any]:
    """HTTP JSON body: a single object, or {} if empty."""
    if not raw:
        return {}
    text = decode_text(raw)
    if not text.strip():
        return {}
    try:
        data = loads_json(text)
    except json.JSONDecodeError as exc:
        raise PlainError(JSON_ERROR) from exc
    if not isinstance(data, dict):
        raise PlainError(NOT_OBJECT)
    return data

```

forgereceipts/forgereceipts/score.py

```
"""Pattern Strength Score ? a transparent local heuristic.
```

```
This is NOT a court-win probability, NOT a prediction, and NOT a claim
that a filing will succeed. It is a counting aid so the user can see
whether they have been documenting consistently.
```

```
Formula (documented here so it cannot hide):
```

```
PSS = 100 * (0.40 * F + 0.35 * C + 0.25 * J)
```

```
F = min(1.0, corroborated_hashed_files / 10)
corroborated_hashed_files = unique file_sha256 values stored
on receipts (incident, journal, or forensics). Live re-verify
of the bytes is a separate Forensics action; the score counts
hashes that were taken, not a claim the file still exists.
```

```
C = min(1.0, chain_length / 20)
chain_length = number of TemporalLock receipts on the ledger.
```

```
J = min(1.0, journal_entries / 10)
journal_entries = receipts with kind == "journal"
(Time with Child).
```

```
Caps at 100. Zero data yields 0. The weights are engineering defaults,
not an empirical model of any court.
"""
```

```
from __future__ import annotations
```

```
from typing import Any, Iterable
```

```
from forgereceipts.store import KIND_JOURNAL, hashed_file_count
```

```
def pattern_strength(records: Iterable[dict[str, Any]]) -> dict[str, Any]:
    rows = list(records)
    chain_length = len(rows)
    journal_entries = sum(1 for r in rows if r.get("kind") == KIND_JOURNAL)
    files = hashed_file_count(rows)
    f = min(1.0, files / 10.0)
    c = min(1.0, chain_length / 20.0)
    j = min(1.0, journal_entries / 10.0)
    pss = 100.0 * (0.40 * f + 0.35 * c + 0.25 * j)
    return {
        "score": round(pss, 1),
        "max": 100.0,
        "formula": "100 * (0.40*min(1, hashed_files/10) + 0.35*min(1, chain_length/20) + 0.25*min(1, journal_entries/10))",
        "components": {
            "corroborated_hashed_files": files,
            "chain_length": chain_length,
            "journal_entries": journal_entries,
            "F": round(f, 4),
            "C": round(c, 4),
            "J": round(j, 4),
        },
        "disclaimer": (
            "Pattern Strength Score is a local counting heuristic. "
            "It is not a court-win probability and not legal advice."
        ),
    }
```

forgereceipts/forgereceipts/static/app.js

```

(function () {
  const $ = (sel) => document.querySelector(sel);
  const $$ = (sel) => Array.from(document.querySelectorAll(sel));
  const MAX_FILE = 12 * 1024 * 1024;
  let selectedHash = null;
  let lastReceipts = [];
  let viewMode = localStorage.getItem("forgereceipts.view") || "simple";

  async function api(path, opts) {
    const res = await fetch(path, opts);
    const text = await res.text();
    let data;
    try { data = JSON.parse(text); } catch { data = { raw: text, status: res.status }; }
    data._status = res.status;
    return data;
  }

  function applyView() {
    document.body.classList.toggle("simple", viewMode === "simple");
    document.body.classList.toggle("advanced", viewMode === "advanced");
    $$("#view-toggle button").forEach((b) => b.classList.toggle("active", b.dataset.view === viewMode));
    localStorage.setItem("forgereceipts.view", viewMode);
  }

  function show(mode) {
    $$("[data-panel]").forEach((el) => el.classList.toggle("hidden", el.getAttribute("data-panel") !== mode));
    $$("#nav button").forEach((b) => b.classList.toggle("active", b.dataset.mode === mode));
    history.replaceState(null, "", "#" + mode);
  }

  function escapeHtml(s) {
    return String(s).replace(/&lt;|&gt;|&quot;|&apos;|&#39;/g, (c) => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", "'": "&quot;", '"': "&#39;"
  }[c]));
  }

  function receiptCard(r) {
    const div = document.createElement("div");
    div.className = "receipt" + (r.hash && r.hash === selectedHash ? " active" : "");
    div.dataset.hash = r.hash || "";
    const impact = r.child_impact ? `<div><em>Child impact:</em> ${escapeHtml(r.child_impact)}</div>` : "";
    const priv = r.private_note ? `<div><em>Private note:</em> ${escapeHtml(r.private_note)}</div>` : "";
    const file = r.file_sha256 ? `<div class="hash">file ${escapeHtml(r.file_name || "")} ${escapeHtml(r.file_sha256)}</div>` : "";
    const extra = viewMode === "advanced"
      ? `${impact}${priv}${file}<div class="hash">${escapeHtml(r.hash || "")}</div>`
      : `<div class="hash">${escapeHtml(r.hash || "")}</div>`;
    div.innerHTML = `<strong>${escapeHtml(r.summary || r.file_name || "Receipt")}</strong>
    <div class="muted">${escapeHtml(r.timestamp || "")} ? ${escapeHtml(r.kind || "")}</div>
    ${extra}`;
    div.addEventListener("click", () => {
      selectedHash = r.hash;
      renderDetail(r);
      $$(".receipt").forEach((el) => el.classList.toggle("active", el.dataset.hash === selectedHash));
    });
    return div;
  }

  function renderDetail(r) {
    const simple = `<p><strong>${escapeHtml(r.summary || r.file_name || "Receipt")}</strong></p>
    <p>This is the saved receipt for this file.</p>
    <p class="hash">${escapeHtml(r.hash || "")}</p>
    <p class="muted">Not legal advice. A receipt is not legal proof.</p>`;
    const advanced = `<p><strong>${escapeHtml(r.summary || "")}</strong></p>
    <div class="muted">${escapeHtml(r.timestamp || "")} ? ${escapeHtml(r.kind || "")} ? conf ${r.confidence}</div>
    ${r.child_impact ? `<p><em>Child impact:</em> ${escapeHtml(r.child_impact)}</p>` : ""}
    ${r.file_sha256 ? `<p class="hash">file ${escapeHtml(r.file_name || "")}<br>${escapeHtml(r.file_sha256)}</p>` : ""}`;
    const html = viewMode === "advanced" ? advanced : simple;
    ["#home-detail", "#rec-detail"].forEach((sel) => {
      const el = $(sel);
      if (!el) return;
      el.innerHTML = html;
      el.classList.remove("hidden");
    });
  }

  function showSaved(data) {
    const box = $("#saved-box");
    box.classList.remove("hidden");
    $("#saved-hash").textContent = data.sha256 || data.hash || (data.receipt && data.receipt.hash) || "";
    $("#saved-note").textContent = (data.plain || "Saved a receipt for this file") + " Not legal advice. A receipt is not legal proof.";
    $("#home-err").textContent = "";
  }

  function fileToB64(file) {
    return new Promise((resolve, reject) => {
      if (file.size > MAX_FILE) {

```

```

        reject(new Error("That file is too big. ForgeReceipts only takes files up to 12 MB."));
        return;
    }
    const reader = new FileReader();
    reader.onload = () => {
        const s = String(reader.result || "");
        const i = s.indexOf(",");
        resolve(i >= 0 ? s.slice(i + 1) : s);
    };
    reader.onerror = () => reject(new Error("That file could not be read. Try another file."));
    reader.readAsDataURL(file);
});
}

function downloadText(filename, text) {
    const blob = new Blob([text], { type: "application/json" });
    const a = document.createElement("a");
    a.href = URL.createObjectURL(blob);
    a.download = filename;
    a.click();
    URL.revokeObjectURL(a.href);
}

async function refreshLock() {
    const st = await api("/api/lock");
    const overlay = $("#lock-overlay");
    if (st.lock_set && !st.unlocked) overlay.classList.remove("hidden");
    else overlay.classList.add("hidden");
    $("#lk-out").textContent = JSON.stringify(st, null, 2);
    return st;
}

async function refreshHome() {
    const score = await api("/api/score");
    $("#pss").textContent = (score.score ?? "?") + "";
    $("#pss-bar").style.width = Math.max(0, Math.min(100, score.score || 0)) + "%";
    $("#pss-note").textContent = score.disclaimer || "";
    const legal = await api("/api/legal?jurisdiction=IN");
    const box = $("#legal-box");
    box.innerHTML = `<div class="card"><h2>Legal reference (stub)</h2>
    <p class="help">${escapeHtml(legal.disclaimer || "Not legal advice.")}</p>
    <p class="muted">${escapeHtml((legal.jurisdiction && legal.jurisdiction.stub) || "")}</p>
    ${legal.baseline || []}.map((b) => `<div class="blurb"><strong>${escapeHtml(b.name)}</strong> <span class="mute
d">${escapeHtml(b.cite)}</span><p>${escapeHtml(b.blurb)}</p></div>`).join("")}
    </div>`;
    await refreshReceipts();
}

async function refreshReceipts() {
    const data = await api("/api/receipts");
    lastReceipts = data.receipts || [];
    ["#home-list", "#rec-list"].forEach((sel) => {
        const list = $(sel);
        if (!list) return;
        list.innerHTML = "";
        lastReceipts.slice().reverse().forEach((r) => list.appendChild(receiptCard(r)));
    });
    if (selectedHash) {
        const row = lastReceipts.find((r) => r.hash === selectedHash);
        if (row) renderDetail(row);
    }
}

async function refreshIncidents() {
    const data = await api("/api/receipts?kind=incident");
    const list = $("#inc-list");
    list.innerHTML = "";
    (data.receipts || []).slice().reverse().forEach((r) => list.appendChild(receiptCard(r)));
}

async function refreshJournal() {
    const data = await api("/api/receipts?kind=journal");
    const list = $("#j-list");
    list.innerHTML = "";
    (data.receipts || []).slice().reverse().forEach((r) => list.appendChild(receiptCard(r)));
}

async function refreshFiling() {
    const t = await api("/api/filing");
    $("#f-check").innerHTML = `<p class="help">${escapeHtml(t.disclaimer || "")}</p>
    <ol>${(t.efiling_checklist || []).map((i) => `<li>${escapeHtml(i)}</li>`).join("")}</ol>`;
}

$("#nav button").forEach((b) => b.addEventListener("click", () => {
    const mode = b.dataset.mode;
    show(mode);
    if (mode === "home") refreshHome();
    if (mode === "receipts") refreshReceipts();
    if (mode === "incident") refreshIncidents();
    if (mode === "journal") refreshJournal();
    if (mode === "filing") refreshFiling();
    if (mode === "lock") refreshLock();
}));

$("#view-toggle button").forEach((b) => b.addEventListener("click", () => {

```



```

        viewMode = b.dataset.view;
        applyView();
        refreshReceipts();
    });

$("#btn-add-file").addEventListener("click", () => $("#add-file-input").click());
$("#add-file-input").addEventListener("change", async (ev) => {
    const file = ev.target.files && ev.target.files[0];
    ev.target.value = "";
    if (!file) return;
    $("#home-err").textContent = "";
    try {
        const b64 = await fileToB64(file);
        const data = await api("/api/forensics/hash", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ content_b64: b64, file_name: file.name, summary: "local file hash" }),
        });
        if (data.error) { $("#home-err").textContent = data.error; return; }
        selectedHash = data.receipt && data.receipt.hash;
        showSaved(data);
        refreshReceipts();
    } catch (err) {
        $("#home-err").textContent = err.message || String(err);
    }
});

$("#btn-import").addEventListener("click", () => $("#import-file-input").click());
$("#import-file-input").addEventListener("change", async (ev) => {
    const file = ev.target.files && ev.target
    .files[0];
    ev.target.value = "";
    if (!file) return;
    $("#home-err").textContent = "";
    try {
        const text = await file.text();
        const data = await api("/api/receipt/import", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ json: text }),
        });
        if (data.error || data.ok === false) {
            $("#home-err").textContent = data.error || data.plain || "This receipt does not match its hash.";
            return;
        }
        selectedHash = data.receipt && data.receipt.hash;
        showSaved({ ...data, sha256: data.receipt && (data.receipt.imported_hash || data.receipt.hash) });
        refreshReceipts();
    } catch (err) {
        $("#home-err").textContent = err.message || String(err);
    }
});

$("#btn-export").addEventListener("click", async () => {
    $("#home-err").textContent = "";
    const body = selectedHash ? { hash: selectedHash } : {};
    const data = await api("/api/receipt/export", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(body),
    });
    if (data.error) { $("#home-err").textContent = data.error; return; }
    downloadText(data.filename || "receipt.json", data.content || "");
    $("#saved-box").classList.remove("hidden");
    $("#saved-hash").textContent = data.filename || "";
    $("#saved-note").textContent = data.plain || "Copied this receipt into a file you can keep.";
});

$("#btn-demo").addEventListener("click", async () => {
    $("#home-err").textContent = "";
    const data = await api("/api/demo", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({}),
    });
    if (data.error) { $("#home-err").textContent = data.error; return; }
    selectedHash = data.receipt && data.receipt.hash;
    showSaved(data);
    refreshReceipts();
});

$("#inc-save").addEventListener("click", async () => {
    $("#inc-err").textContent = "";
    const data = await api("/api/incident", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
            summary: $("#inc-summary").value,
            evidence: $("#inc-evidence").value,
            child_impact: $("#inc-impact").value,
            confidence: Number($("#inc-conf").value),
        }),
    });
    if (data.error) { $("#inc-err").textContent = data.error; return; }
    $("#inc-summary").value = ""; $("#inc-evidence").value = ""; $("#inc-impact").value = "";
});

```

```

    refreshIncidents();
  });

$("#for-hash").addEventListener("click", async () => {
  const body = {
    path: $("#for-path").value || undefined,
    content_b64: $("#for-b64").value || undefined,
    file_name: $("#for-name").value || undefined,
    summary: "local file hash",
  };
  const data = await api("/api/forensics/hash", {
    method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify(body),
  });
  $("#for-out").textContent = JSON.stringify(data, null, 2);
  if (data.sha256) $("#for-expected").value = data.sha256;
});

$("#for-reverify").addEventListener("click", async () => {
  const body = {
    path: $("#for-path").value || undefined,
    content_b64: $("#for-b64").value || undefined,
    expected: $("#for-expected").value,
  };
  const data = await api("/api/forensics/verify", {
    method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify(body),
  });
  $("#for-out").textContent = JSON.stringify(data, null, 2);
});

$("#j-save").addEventListener("click", async () => {
  $("#j-err").textContent = "";
  const data = await api("/api/journal", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      summary: $("#j-summary").value,
      evidence: $("#j-evidence").value,
      child_impact: $("#j-impact").value,
      file_sha256: $("#j-hash").value || undefined,
      private_note: $("#j-private").value || undefined,
    }),
  });
  if (data.error) { $("#j-err").textContent = data.error; return; }
  $("#j-summary").value = ""; $("#j-evidence").value = ""; $("#j-impact").value = "";
  $("#j-private").value = ""; $("#j-hash").value = "";
  refreshJournal();
});

function filingFields() {
  return {
    state: $("#f-state").value,
    court_name: $("#f-court").value,
    petitioner: $("#f-pet").value,
    respondent: $("#f-resp").value,
    cause_no: $("#f-cause").value,
    party_role: $("#f-role").value,
    title: $("#f-title").value,
    body: $("#f-body").value,
    exhibit_count: Number($("#f-ex").value),
    party: $("#f-party").value,
  };
}

async function exportFiling(fmt) {
  const data = await api("/api/filing/export", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ ...filingFields(), format: fmt }),
  });
  $("#f-out").textContent = data.content || JSON.stringify(data, null, 2);
}

$("#f-txt").addEventListener("click", () => exportFiling("txt"));
$("#f-html").addEventListener("click", () => exportFiling("html"));

$$("[data-tool]").forEach((b) => b.addEventListener("click", async () => {
  const name = b.getAttribute("data-tool");
  let payload = {};
  if (name === "codelock") payload = { source: $("#cl-source").value, acknowledgment: $("#cl-ack").value };
  if (name === "shadowlock") payload = { jsonl: $("#sl-jsonl").value };
  if (name === "godlock") payload = { text: $("#gl-text").value };
  if (name === "staticclock") payload = { geo: $("#sc-geo").value };
  const data = await api("/api/tools/" + name, {
    method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify(payload),
  });
  $("#tools-out").textContent = JSON.stringify(data, null, 2);
}));

$("#v-run").addEventListener("click", async () => {
  const data = await api("/api/verify", {
    method: "POST", headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ jsonl: $("#v-jsonl").value }),
  });
  $("#v-verdict").textContent = data.verdict || "";
  $("#v-verdict").className = "verdict " + ((data.verdict || "").toLowerCase());
  $("#v-out").textContent = JSON.stringify(data, null, 2);
});

```

```

});
$("#v-local").addEventListener("click", async () => {
  const data = await api("/api/verify", {
    method: "POST", headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ jsonl: "" }),
  });
  $("#v-verdict").textContent = data.verdict || "";
  $("#v-verdict").className = "verdict " + ((data.verdict || "").toLowerCase());
  $("#v-out").textContent = JSON.stringify(data, null, 2);
});

$("#lk-set").addEventListener("click", async () => {
  const data = await api("/api/lock/set", {
    method: "POST", headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ passphrase: $("#lk-pass").value }),
  });
  $("#lk-out").textContent = JSON.stringify(data, null, 2);
  refreshLock();
});

$("#lk-unlock").addEventListener("click", async () => {
  const data = await api("/api/lock/unlock", {
    method: "POST", headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ passphrase: $("#lk-pass").value }),
  });
  $("#lk-out").textContent = JSON.stringify(data, null, 2);
  refreshLock();
});

$("#lk-clear").addEventListener("click", async () => {
  const data = await api("/api/lock/clear", {
    method: "POST", headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ passphrase: $("#lk-pass").value }),
  });
  $("#lk-out").textContent = JSON.stringify(data, null, 2);
  refreshLock();
});

$("#ov-unlock").addEventListener("click", async () => {
  const data = await api("/api/lock/unlock", {
    method: "POST", headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ passphrase: $("#ov-pass").value }),
  });
  if (data.ok) { $("#ov-err").textContent = ""; refreshLock(); refreshHome(); }
  else { $("#ov-err").textContent = "Passphrase did not match.";
}
});

applyView();
const start = (location.hash || "#home").replace("#", "") || "home";
show(start);
refreshLock().then(() => { refreshHome(); });
})();

```

forgereceipts/forgereceipts/static/index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<meta name="theme-color" content="#c4a35a">
<title>ForgeReceipts ? local-first evidence integrity</title>
<link rel="manifest" href="/manifest.webmanifest">
<link rel="stylesheet" href="/static/style.css">
</head>
<body>
<div class="disclaimer-banner">Not Legal Advice ? a receipt is not legal proof ? does not contact courts, Odyssey, or
the cloud</div>
<header class="app">
  <div class="brand">
    <h1>ForgeReceipts</h1>
    <p class="motto">Child's Best Interests First. Integrity Over Narrative. Local Control. Always.</p>
  </div>
  <span class="local-pill">127.0.0.1 ? no telemetry ? no accounts</span>
</header>
<nav class="modes" id="nav">
  <button data-mode="home" class="active">Home</button>
  <button data-mode="receipts">Receipts</button>
  <button data-mode="incident" data-advanced="1">Incident log</button>
  <button data-mode="forensics" data-advanced="1">Forensics</button>
  <button data-mode="journal" data-advanced="1">Time with Child</button>
  <button data-mode="filing" data-advanced="1">Filing assistant</button>
  <button data-mode="tools" data-advanced="1">Tools</button>
  <button data-mode="verify">Verify</button>
  <button data-mode="lock" data-advanced="1">Lock</button>
</nav>
<main id="app">
  <section class="card" data-panel="home">
    <h2>Home</h2>
    <p><strong>Not Legal Advice.</strong> A receipt is not legal proof. ForgeReceipts saves a hash of a file on this c
omputer. It does not practice law, file anything, or talk to a court.</p>
    <div class="view-toggle" id="view-toggle">
      <button type="button" data-view="simple" class="active">Simple</button>
      <button type="button" data-view="advanced">Advanced</button>
    </div>
    <div class="giant-grid">
      <button class="giant" id="btn-add-file" type="button">Add file</button>
      <button class="giant ghost-giant" id="btn-import" type="button">Import receipt</button>
      <button class="giant ghost-giant" id="btn-export" type="button">Export receipt</button>
    </div>
    <input id="add-file-input" class="hidden" type="file">
    <input id="import-file-input" class="hidden" type="file" accept=".json,.jsonl,application/json,text/plain">
    <div class="row">
      <button class="act" id="btn-demo" type="button">Try a sample</button>
    </div>
    <p class="help">Add file keeps a receipt for that file. Import receipt reads a receipt you already saved. Export r
eceipt copies a receipt to a file. Try a sample if you just want to see how it looks.</p>
    <div id="saved-box" class="saved-banner hidden" role="status">
      <p class="saved-title">Saved a receipt for this file</p>
      <p class="hash" id="saved-hash"></p>
      <p class="muted" id="saved-note"></p>
    </div>
    <p class="error" id="home-err"></p>
    <h2>Receipts</h2>
    <p class="help">List view. Tap one to open the detail view.</p>
    <div id="home-list"></div>
    <div id="home-detail" class="detail hidden"></div>
    <div class="card advanced-only" style="margin-top:1rem">
      <div class="score" id="pss">?</div>
      <p class="muted">Pattern Strength Score (local heuristic, not a win probability)</p>
      <div class="bar"><span id="pss-bar" style="width:0%></span></div>
      <p class="muted" id="pss-note"></p>
    </div>
    <div id="legal-box" class="advanced-only"></div>
  </section>

  <section class="card hidden" data-panel="receipts">
    <h2>Receipts</h2>
    <p class="help">List and detail. Simple shows the name and the hash. Advanced shows every field.</p>
    <div id="rec-list"></div>
    <div id="rec-detail" class="detail hidden"></div>
  </section>

  <section class="card hidden" data-panel="incident">
    <h2>Incident log</h2>
    <p class="help">Append-only TemporalLock receipts. You cannot edit or delete. A correction is a new receipt. Child
impact is required.</p>
    <label>Summary</label>
    <input id="inc-summary" placeholder="What was observed">
    <label>Evidence (required)</label>
    <textarea id="inc-evidence" placeholder="What supports the observation"></textarea>
    <label>Child impact (required)</label>
    <textarea id="inc-impact" placeholder="How this relates to the child's best interests"></textarea>
    <label>Confidence (0.0?1.0)</label>
    <input id="inc-conf" type="number" min="0" max="1" step="0.05" value="1">
    <div class="row"><button class="act" id="inc-save">Append receipt</button></div>
    <p class="error" id="inc-err"></p>

```

```

    <div id="inc-list"></div>
</section>

<section class="card hidden" data-panel="forensics">
  <h2>Forensics</h2>
  <p class="help">SHA-256 of a local file is stored on a receipt. Re-verify later against the same bytes. Files neve
r leave this machine.</p>
  <label>Local path (this computer)</label>
  <input id="for-path" placeholder="/absolute/path/to/file">
  <label>Or paste file bytes as base64 (optional)</label>
  <textarea id="for-b64" placeholder="base64?"></textarea>
  <label>File name</label>
  <input id="for-name" placeholder="exhibit.pdf">
  <div class="row">
    <button class="act" id="for-hash">Hash and store on receipt</button>
  </div>
  <label>Expected SHA-256 (to re-verify)</label>
  <input id="for-expected" placeholder="64 hex chars">
  <div class="row"><button class="ghost" id="for-reverify">Re-verify</button></div>
  <pre class="out" id="for-out"></pre>
</section>

<section class="card hidden" data-panel="journal">
  <h2>Time with Child</h2>
  <p class="help">Parenting documentation you already have. Optional file hash. Private note is stored locally and i
s not part of the TemporalLock hash.</p>
  <label>Summary</label>
  <input id="j-summary" placeholder="Afternoon at the park">
  <label>What happened</label>
  <textarea id="j-evidence"></textarea>
  <label>Child impact (required)</label>
  <textarea id="j-impact" placeholder="How this time served the child"></textarea>
  <label>Optional file SHA-256</label>
  <input id="j-hash" placeholder="hash of a photo already on this device">
  <label>Private note (not hashed)</label>
  <textarea id="j-private"></textarea>
  <div class="row"><button class="act" id="j-save">Append journal receipt</button></div>
  <p class="error" id="j-err"></p>
  <div id="j-list"></div>
</section>

<section class="card hidden" data-panel="filing">
  <h2>Filing assistant</h2>
  <p class="help">Local templates only. This app does not talk to Odyssey or any court. You print or export .txt / .
html yourself.</p>
  <label>State</label><input id="f-state" value="Indiana">
  <label>Court name</label><input id="f-court" value="[COURT NAME]">
  <label>Petitioner</label><input id="f-pet" value="[PETITIONER]">
  <label>Respondent</label><input id="f-resp" value="[RESPONDENT]">
  <label>Cause no.</label><input id="f-cause" value="[CAUSE NO.]">
  <label>Your role</label>
  <select id="f-role"><option>Petitioner</option><option>Respondent</option></select>
  <label>Motion title</label><input id="f-title" value="MOTION">
  <label>Body</label><textarea id="f-body">[Write in your own words. Not legal advice.]</textarea>
  <label>Exhibit count</label><input id="f-ex" type="number" min="0" value="3">
  <label>Exhibit party</label>
  <select id="f-party"><option value="petitioner">Petitioner's Exhibit 1, 2, 3?</option><option value="respondent">R
espondent's Exhibit A, B, C?</option></select>
  <div class="row">
    <button class="act" id="f-txt">Export .txt</button>
    <button class="ghost" id="f-html">Export .html</button>
  </div>
  <div id="f-check"></div>
  <pre class="out" id="f-out"></pre>
</section>

<section class="card hidden" data-panel="tools">
  <h2>Tools</h2>
  <p class="help">Vendored engines, imported if available. MirageGrid is logical nodes only ? not a proxy or IP-hidi
ng network.</p>
  <div class="tools-grid">
    <div class="card">
      <h2>VibeLock</h2>
      <p class="muted">Score a synthetic local wav pair.</p>
      <button class="act" data-tool="vibelock">Run</button>
    </div>
    <div class="card">
      <h2>CodeLock</h2>
      <p class="muted">Gate phrase required: This tool alters perception, not meaning.</p>
      <label>Source</label><textarea id="cl-source">plain source text</textarea>
      <label>Acknowledgment</label><input id="cl-ack" value="This tool alters perception, not meaning.">
      <button class="act" data-tool="codelock">Render</button>
    </div>
    <div class="card">
      <h2>ShadowLock</h2>
      <p class="muted">Observe pasted JSONL. Read-only, then forget.</p>
      <textarea id="sl-jsonl">{"id":"1","task_class":"visit","actual_outcome":"ok","actual_revenue":0,"actual_cost":
0}</textarea>
      <button class="act" data-tool="shadowlock">Observe</button>
    </div>
    <div class="card">
      <h2>VeilLock</h2>
      <p class="muted">Encrypt synthetic frames in-process.</p>
      <button class="act" data-tool="veillock">Encrypt frames</button>
    </div>
  </div>

```


forgereceipts/forgereceipts/static/style.css

```

:root {
  color-scheme: dark;
  --bg: #0e1014;
  --bg-2: #151922;
  --ink: #e8eaeef;
  --muted: #9aa3b2;
  --line: #2a3140;
  --accent: #c4a35a;
  --accent-dim: #3d3220;
  --ok: #7dcea0;
  --bad: #e07a7a;
  --warn: #e0b46a;
  --banner: #7a1f1f;
  --serif: "Iowan Old Style", "Palatino Linotype", Palatino, Georgia, serif;
  --sans: "Segoe UI", system-ui, sans-serif;
  --mono: ui-monospace, SFMono-Regular, Menlo, Consolas, monospace;
}
* { box-sizing: border-box; }
html, body { margin: 0; background: var(--bg); color: var(--ink); font-family: var(--sans); }
body { min-height: 100vh; display: flex; flex-direction: column; }
.disclaimer-banner {
  background: var(--banner); color: #fff; text-align: center;
  font-size: 0.82rem; letter-spacing: 0.08em; text-transform: uppercase;
  padding: 0.55rem 1rem; font-weight: 650;
}
header.app {
  display: flex; flex-wrap: wrap; align-items: center; gap: 1rem;
  padding: 0.9rem 1.25rem; border-bottom: 1px solid var(--line);
  background: var(--bg-2);
}
.brand { font-family: var(--serif); }
.brand h1 { margin: 0; font-size: 1.45rem; font-weight: 500; letter-spacing: 0.04em; }
.brand .motto { margin: 0.15rem 0 0; color: var(--muted); font-style: italic; font-size: 0.88rem; }
.local-pill {
  margin-left: auto; font-family: var(--mono); font-size: 0.72rem;
  color: var(--muted); border: 1px solid var(--line); padding: 0.2rem 0.55rem; border-radius: 999px;
}
nav.modes {
  display: flex; flex-wrap: wrap; gap: 0.35rem; padding: 0.7rem 1.25rem;
  border-bottom: 1px solid var(--line);
}
nav.modes button {
  font-family: inherit; background: transparent; color: var(--muted);
  border: 1px solid var(--line); border-radius: 999px; padding: 0.35rem 0.8rem; cursor: pointer;
}
nav.modes button.active { color: #0e1014; background: var(--accent); border-color: var(--accent); font-weight: 650; }
main { flex: 1; max-width: 52rem; width: 100%; margin: 0 auto; padding: 1.4rem 1.25rem 3rem; }
.card {
  background: var(--bg-2); border: 1px solid var(--line); border-radius: 12px;
  padding: 1.15rem 1.2rem 1.25rem; margin: 0 0 1.1rem;
}
h2 { font-family: var(--serif); font-weight: 500; font-size: 1.25rem; margin: 0 0 0.55rem; }
p.help, p.muted { color: var(--muted); font-size: 0.95rem; }
label { display: block; font-size: 0.8rem; color: var(--muted); margin: 0.7rem 0 0.28rem; }
input, textarea, select {
  width: 100%; background: #0e1014; color: var(--ink);
  border: 1px solid var(--line); border-radius: 6px; padding: 0.5rem 0.6rem;
  font-family: inherit; font-size: 0.95rem;
}
textarea { min-height: 6rem; resize: vertical; }
.row { display: flex; gap: 0.7rem; flex-wrap: wrap; align-items: center; margin-top: 0.9rem; }
button.act {
  font-family: inherit; cursor: pointer; border-radius: 8px; padding: 0.55rem 1.1rem;
  border: 1px solid var(--accent); background: var(--accent); color: #0e1014; font-weight: 650;
}
button.ghost {
  font-family: inherit; cursor: pointer; border-radius: 8px; padding: 0.55rem 1.1rem;
  border: 1px solid var(--line); background: transparent; color: var(--muted);
}
.receipt {
  border-top: 1px solid var(--line); padding: 0.7rem 0; font-size: 0.9rem;
}
.receipt .hash { font-family: var(--mono); font-size: 0.75rem; color: var(--muted); word-break: break-all; }
.verdict { font-weight: 700; letter-spacing: 0.08em; }
.verdict.pass { color: var(--ok); }
.verdict.fail { color: var(--bad); }
.score { font-size: 2.6rem; font-variant-numeric: tabular-nums; font-family: var(--serif); }
.bar { height: 8px; background: #0e1014; border-radius: 99px; overflow: hidden; border: 1px solid var(--line); }
.bar > span { display: block; height: 100%; background: var(--accent); }
pre.out {
  background: #0e1014; border: 1px solid var(--line); border-radius: 8px;
  padding: 0.8rem; overflow: auto; font-family: var(--mono); font-size: 0.78rem;
  white-space: pre-wrap;
}
.blur { border-left: 3px solid var(--accent); padding: 0.2rem 0 0.2rem 0.8rem; margin: 0.8rem 0; }
.tools-grid { display: grid; grid-template-columns: 1fr; gap: 1rem; }
@media (min-width: 720px) { .tools-grid { grid-template-columns: 1fr 1fr; } }
.hidden { display: none !important; }
#lock-overlay {
  position: fixed; inset: 0; background: rgba(8,10,14,0.92); display: flex;
  align-items: center; justify-content: center; z-index: 20; padding: 1rem;
}

```

```

#lock-overlay .card { max-width: 24rem; width: 100%; }
footer {
  color: var(--muted); font-size: 0.78rem; padding: 1rem 1.25rem 1.4rem;
  border-top: 1px solid var(--line); text-align: center;
}
footer a { color: var(--accent); }
.error { color: var(--bad); font-size: 0.9rem; }

.giant-grid { display: grid; gap: 0.75rem; margin: 1rem 0 0.6rem; }
@media (min-width: 720px) { .giant-grid { grid-template-columns: 1fr 1fr 1fr; } }
button.giant {
  font-family: inherit; cursor: pointer; border-radius: 14px;
  padding: 1.15rem 1rem; min-height: 4.2rem; width: 100%;
  font-size: 1.28rem; font-weight: 750; letter-spacing: 0.02em;
  border: 1px solid var(--accent); background: var(--accent); color: #0e1014;
}
button.ghost-giant {
  background: transparent; color: var(--ink); border-color: var(--accent);
}
.view-toggle { display: flex; gap: 0.4rem; margin: 0.8rem 0 0.2rem; }
.view-toggle button {
  font-family: inherit; cursor: pointer; border-radius: 999px;
  padding: 0.35rem 0.9rem; border: 1px solid var(--line);
  background: transparent; color: var(--muted);
}
.view-toggle button.active { background: var(--accent); color: #0e1014; border-color: var(--accent); font-weight: 650;
}
.saved-banner {
  margin-top: 1rem; border: 1px solid var(--ok); background: #14241c;
  border-radius: 12px; padding: 0.9rem 1rem;
}
.saved-title { margin: 0 0 0.35rem; font-size: 1.15rem; font-weight: 700; color: var(--ok); }
.saved-banner .hash { font-family: var(--mono); font-size: 0.8rem; word-break: break-all; margin: 0; }
.detail {
  margin-top: 0.8rem; border: 1px solid var(--accent); border-radius: 12px;
  padding: 0.9rem 1rem; background: #141820;
}
.receipt { cursor: pointer; }
.receipt.active { border-color: var(--accent); }
body.simple .advanced-only { display: none !important; }
body.simple nav.modes button[data-advanced="1"] { display: none; }

```


forgereceipts/forgereceipts/store.py

```

"""Append-only ForgeReceipts ledger on top of TemporalLock.

TemporalLock hashes core fields only (timestamp, summary, evidence,
confidence, prev_hash). ForgeReceipts extra keys (kind, child_impact,
file_sha256, private_note, ...) ride on the JSONL line and are ignored
by the core hash. Child impact is *also* written into evidence so it
is hashed. Private notes are extra-only and are never hashed.

There is no modify and no delete. Corrections are new receipts.
"""

from __future__ import annotations

import json
from pathlib import Path
from typing import Any, Iterable

from temporallock.chain import Chain, VerifyResult
from temporallock.errors import AppendOnlyError
from temporallock.receipt import Receipt

CORE_KEYS = ("timestamp", "summary", "evidence", "confidence", "prev_hash", "hash")

KIND_INCIDENT = "incident"
KIND_JOURNAL = "journal"
KIND_FORENSICS = "forensics"
KIND_IMPORT = "import"

def compose_evidence(
    body: str,
    *,
    kind: str,
    child_impact: str,
    file_sha256: str | None = None,
    file_name: str | None = None,
) -> str:
    """Put required child-impact (and optional file hash) into hashed evidence."""
    lines = [
        f"KIND: {kind}",
        f"CHILD_IMPACT: {child_impact.strip()}",
    ]
    if file_sha256:
        lines.append(f"FILE_SHA256: {file_sha256}")
    if file_name:
        lines.append(f"FILE_NAME: {file_name}")
    lines.append("")
    lines.append(body.rstrip() if body else "(no additional body)")
    return "\n".join(lines)

class ForgeStore:
    """JSONL-backed store. Chain is in-memory; this class writes extras."""

    def __init__(self, data_dir: str | Path) -> None:
        self.data_dir = Path(data_dir)
        self.path = self.data_dir / "chain.jsonl"
        self._load()

    def _load(self) -> None:
        receipts: list[Receipt] = []
        extras: list[dict[str, Any]] = []
        if self.path.is_file():
            text = self.path.read_text(encoding="utf-8")
            for line in text.splitlines():
                line = line.strip()
                if not line:
                    continue
                obj = json.loads(line)
                receipts.append(Receipt.from_dict(obj))
                extras.append({k: v for k, v in obj.items() if k not in CORE_KEYS})
        self.chain = Chain(receipts)
        self._extras = extras

    def reload(self) -> None:
        self._load()

    def __len__(self) -> int:
        return len(self.chain)

    def records(self) -> list[dict[str, Any]]:
        out: list[dict[str, Any]] = []
        for rec, extra in zip(self.chain, self._extras, strict=True):
            row = rec.to_dict()
            row.update(extra)
            out.append(row)
        return out

    def records_of(self, kind: str) -> list[dict[str, Any]]:
        return [r for r in self.records() if r.get("kind") == kind]

    def get_by_hash(self, digest: str) -> dict[str, Any] | None:

```

```

        want = (digest or "").strip().lower()
        if not want:
            return None
        for row in self.records():
            if str(row.get("hash") or "").lower() == want:
                return row
        return None

def save_import_file(self, receipt: dict[str, Any]) -> Path:
    from forgereceipts.exchange import dump_receipt

    digest = str(receipt.get("hash") or "").strip().lower()
    if len(digest) != 64:
        raise ValueError("imported receipt needs a 64-character hash")
    dest_dir = self.data_dir / "imports"
    dest_dir.mkdir(parents=True, exist_ok=True)
    dest = dest_dir / f"{digest}.json"
    dest.write_text(dump_receipt(receipt), encoding="utf-8")
    return dest

def append_import(
    self,
    source: dict[str, Any],
    *,
    timestamp: str | None = None,
) -> dict[str, Any]:
    from forgereceipts.exchange import verify_one
    from forgereceipts.plain import PlainError

    checked = verify_one(source)
    if not checked.get("ok"):
        raise PlainError(str(checked.get("plain") or "This receipt does not match its hash.))")
    digest = str(source.get("hash") or checked.get("hash") or "")
    self.save_import_file(source)
    name = str(source.get("file_name") or source.get("summary") or "receipt.json")
    child_impact = (
        "Imported a local copy of a receipt. A receipt is not legal proof."
    )
    composed = compose_evidence(
        f"Imported receipt hash {digest}",
        kind=KIND_IMPORT,
        child_impact=child_impact,
        file_sha256=digest,
        file_name=name,
    )
    extra: dict[str, Any] = {
        "kind": KIND_IMPORT,
        "child_impact": child_impact,
        "imported": True,
        "imported_hash": digest,
        "file_sha256": digest,
        "file_name": name,
    }
    return self.append(
        summary=str(source.get("summary") or f"Imported receipt {digest[:12]}"),
        evidence=composed,
        confidence=1.0,
        timestamp=timestamp,
        extra=extra,
    )

def append(
    self,
    *,
    summary: str,
    evidence: str,
    confidence: float = 1.0,
    timestamp: str | None = None,
    extra: dict[str, Any] | None = None,
) -> dict[str, Any]:
    extra = dict(extra or {})
    rec = self.chain.append(
        summary=summary,
        evidence=evidence,
        confidence=confidence,
        timestamp=timestamp,
    )
    self._extras.append(extra)
    row = rec.to_dict()
    row.update(extra)
    self.data_dir.mkdir(parents=True, exist_ok=True)
    with self.path.open("a", encoding="utf-8") as fh:
        fh.write(json.dumps(row, sort_keys=True, separators="(", ":"), ensure_ascii=False))
        fh.write("\n")
        fh.flush()
    return row

def append_incident(
    self,
    *,
    summary: str,
    evidence: str,
    child_impact: str,
    confidence: float = 1.0,
    timestamp: str | None = None,

```

```

        file_sha256: str | None = None,
        file_name: str | None = None,
    ) -> dict[str, Any]:
        child_impact = (child_impact or "").strip()
        if not child_impact:
            raise ValueError("child_impact is required")
        composed = compose_evidence(
            evidence,
            kind=KIND_INCIDENT,
            child_impact=child_impact,
            file_sha256=file_sha256,
            file_name=file_name,
        )
        extra: dict[str, Any] = {
            "kind": KIND_INCIDENT,
            "child_impact": child_impact,
        }
        if file_sha256:
            extra["file_sha256"] = file_sha256
        if file_name:
            extra["file_name"] = file_name
        return self.append(
            summary=summary,
            evidence=composed,
            confidence=confidence,
            timestamp=timestamp,
            extra=extra,
        )

def append_journal(
    self,
    *,
    summary: str,
    evidence: str,
    child_impact: str,
    confidence: float = 1.0,
    timestamp: str | None = None,
    file_sha256: str | None = None,
    file_name: str | None = None,
    private_note: str | None = None,
) -> dict[str, Any]:
    child_impact = (child_impact or "").strip()
    if not child_impact:
        raise ValueError("child_impact is required")
    composed = compose_evidence(
        evidence,
        kind=KIND_JOURNAL,
        child_impact=child_impact,
        file_sha256=file_sha256,
        file_name=file_name,
    )
    extra: dict[str, Any] = {
        "kind": KIND_JOURNAL,
        "child_impact": child_impact,
    }
    if file_sha256:
        extra["file_sha256"] = file_sha256
    if file_name:
        extra["file_name"] = file_name
    if private_note:
        # Extra-only: not part of TemporalLock core hash.
        extra["private_note"] = private_note
    return self.append(
        summary=summary,
        evidence=composed,
        confidence=confidence,
        timestamp=timestamp,
        extra=extra,
    )

def append_forensics(
    self,
    *,
    summary: str,
    file_sha256: str,
    file_name: str,
    child_impact: str = "Hash of a local file retained for later re-verification.",
    confidence: float = 1.0,
    timestamp: str | None = None,
) -> dict[str, Any]:
    if not file_sha256:
        raise ValueError("file_sha256 is required")
    composed = compose_evidence(
        f"SHA-256 of local file {file_name!r} stored for chain-of-custody.",
        kind=KIND_FORENSICS,
        child_impact=child_impact,
        file_sha256=file_sha256,
        file_name=file_name,
    )
    extra = {
        "kind": KIND_FORENSICS,
        "child_impact": child_impact,
        "file_sha256": file_sha256,
        "file_name": file_name,
    }

```

```

    }
    return self.append(
        summary=summary,
        evidence=composed,
        confidence=confidence,
        timestamp=timestamp,
        extra=extra,
    )

def verify(self) -> VerifyResult:
    return self.chain.verify()

def refuse_mutate(self) -> None:
    raise AppendOnlyError(
        "cannot mutate: ForgeReceipts/TemporalLock is append-only; "
        "record a correction as a new receipt"
    )

def verify_jsonl(text: str) -> dict[str, Any]:
    """Load a chain JSONL string and return PASS/FAIL plus TemporalLock details."""
    receipts: list[Receipt] = []
    errors: list[str] = []
    for i, line in enumerate(text.splitlines()):
        line = line.strip()
        if not line:
            continue
        try:
            obj = json.loads(line)
            receipts.append(Receipt.from_dict(obj))
        except Exception as exc: # noqa: BLE001 ? surface parse errors to the UI
            errors.append(f"line {i}: {exc}")
    if errors and not receipts:
        return {
            "ok": False,
            "verdict": "FAIL",
            "length": 0,
            "errors": errors,
            "first_hash": None,
            "last_hash": None,
        }
    result = Chain(receipts).verify()
    all_errors = errors + list(result.errors)
    ok = result.ok and not errors
    return {
        "ok": ok,
        "verdict": "PASS" if ok else "FAIL",
        "length": result.length,
        "errors": all_errors,
        "first_hash": result.first_hash,
        "last_hash": result.last_hash,
    }

def hashed_file_count(records: Iterable[dict[str, Any]]) -> int:
    n = 0
    seen: set[str] = set()
    for rec in records:
        digest = rec.get("file_sha256")
        if isinstance(digest, str) and digest and digest not in seen:
            seen.add(digest)
            n += 1
    return n

```

forgereceipts/forgereceipts/tools.py

```

"""Optional *Lock engine panels. Import failures are availability, not 500s.

MirageGrid is logical nodes only ? never a proxy, VPN, or IP-hiding mesh.
"""

from __future__ import annotations

import json
from typing import Any

def availability() -> dict[str, bool]:
    flags: dict[str, bool] = {}
    for name in (
        "vibelock",
        "codelock",
        "shadowlock",
        "veillock",
        "godlock",
        "staticcodelock",
        "miragegrid",
        "temporallock",
    ):
        try:
            __import__(name)
            flags[name] = True
        except Exception:
            flags[name] = False
    return flags

def vibelock_score() -> dict[str, Any]:
    """Score a synthetic local wav pair. No user recording is required."""
    from vibelock import analyze
    from vibelock.synth import make_pair

    pair = make_pair(duration_s=0.6, sr=16000, seed=202607)
    result = analyze(pair.audio, pair.sr, vibration=pair.vibration)
    return {
        "engine": "vibelock",
        "mode": result.mode,
        "score": float(result.score),
        "reason_codes": list(result.reason_codes),
        "notes": list(result.notes) + [
            "Synthetic local pair from vibelock.synth.make_pair. Not a human recording."
        ],
        "sample_rate": int(result.sample_rate),
        "n_samples": int(result.n_samples),
    }

def codelock_render(source: str, acknowledgment: str, seed: str = "0") -> dict[str, Any]:
    from codelock import ACK_PHRASE, CodeLockSession
    from codelock.gate import AcknowledgmentError, GateClosedError

    session = CodeLockSession(source, seed=seed)
    try:
        session.open_gate(acknowledgment)
        html_out = session.codelock_html()
        return {
            "engine": "codelock",
            "gate": "open",
            "ack_phrase": ACK_PHRASE,
            "html": html_out,
            "normalize_html": session.normalize_html(),
            "note": "View-layer only. Not encryption. Source is unchanged.",
        }
    except (AcknowledgmentError, GateClosedError) as exc:
        return {
            "engine": "codelock",
            "gate": "closed",
            "ack_phrase": ACK_PHRASE,
            "error": str(exc),
            "normalize_html": session.normalize_html(),
        }

def shadowlock_observe(jsonl_text: str) -> dict[str, Any]:
    from shadowlock import MemoryAdapter, ShadowLockSession

    records: list[dict[str, Any]] = []
    for line in jsonl_text.splitlines():
        line = line.strip()
        if not line:
            continue
        rec = json.loads(line)
        if isinstance(rec, dict):
            records.append(rec)
    session = ShadowLockSession()
    report = session.observe(MemoryAdapter(records))
    session.forget()
    payload = report.to_dict()

```

```

    payload["engine"] = "shadowlock"
    payload["note"] = "Read-only observe. Session forgotten after this report."
    return payload

def veillock_encrypt() -> dict[str, Any]:
    import numpy as np
    from veillock import VeilLockSession

    frames = np.zeros((2, 8, 8, 3), dtype=np.uint8)
    frames[0, :, :, 0] = 40
    frames[1, :, :, 1] = 80
    session = VeilLockSession()
    stream = session.encrypt_frames(frames)
    return {
        "engine": "veillock",
        "frames": len(stream.frames),
        "mode": stream.mode,
        "cipher_bytes": [len(f.ciphertext) for f in stream.frames],
        "nonces_hex": [f.nonce.hex() for f in stream.frames],
        "note": "Synthetic 8x8 RGB frames encrypted in-process. Not a live display capture.",
    }

def godlock_submit(text: str) -> dict[str, Any]:
    from godlock import GodLockEngine

    engine = GodLockEngine(persist=False)
    result = engine.submit(text)
    result["engine"] = "godlock"
    result["note"] = "In-memory only (persist=False). Not an anonymity network."
    return result

def staticclock_advise(geo: str) -> dict[str, Any]:
    from staticclock import StaticClock

    clock = StaticClock()
    advisory = clock.advise(geo)
    clock.forget()
    payload = advisory.to_dict()
    payload["engine"] = "staticclock"
    payload["note"] = "Advisory only. Then forget. Not a scheduler."
    return payload

def miragegrid_assign() -> dict[str, Any]:
    from miragegrid import MirageSession, POOL_SIZE

    with MirageSession() as session:
        node = session.node
        receipt = session.receipt
        payload = {
            "engine": "miragegrid",
            "pool_size": POOL_SIZE,
            "node_id": node.id,
            "node_label": node.label,
            "node_index": node.index,
            "session_id": session.session_id,
            "receipt_hash": getattr(receipt, "hash", None) or getattr(receipt, "digest", None),
            "note": (
                "Logical node assignment only. Not a proxy, VPN, Tor hop, "
                "or IP-hiding network. Mapping is destroyed when the session ends."
            ),
        }
    return payload

HANDLERS = {
    "vibelock": lambda body: vibelock_score(),
    "codelock": lambda body: codelock_render(
        str(body.get("source") or "plain source"),
        str(body.get("acknowledgment") or body.get("phrase") or ""),
        str(body.get("seed") or "0"),
    ),
    "shadowlock": lambda body: shadowlock_observe(str(body.get("jsonl") or "")),
    "veillock": lambda body: veillock_encrypt(),
    "godlock": lambda body: godlock_submit(str(body.get("text") or "local stress text")),
    "staticclock": lambda body: staticclock_advise(str(body.get("geo") or "Indianapolis")),
    "miragegrid": lambda body: miragegrid_assign(),
}

def run(name: str, body: dict[str, Any] | None = None) -> dict[str, Any]:
    handler = HANDLERS.get(name)
    if handler is None:
        return {"error": f"unknown engine {name!r}", "available": availability()}
    if not availability().get(name, False):
        return {
            "engine": name,
            "available": False,
            "error": f"{name} is vendored but could not be imported (optional extra deps?)",
        }
    try:
        return handler(body or {})
    
```

```
except Exception as exc: # noqa: BLE001 ? panel must not 500 the UI
    return {"engine": name, "error": str(exc), "available": True}
```

forgereceipts/forgereceipts/ui.py

```

"""Local-only ForgeReceipts UI. Binds 127.0.0.1. No CDN, no telemetry."""

from __future__ import annotations

import base64
import json
import mimetypes
import os
import threading
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
from typing import Any
from urllib.parse import parse_qs, urlparse

from forgereceipts import __version__
from forgereceipts.debug import debug_enabled, debug_log
from forgereceipts.demo import SAMPLE_NAME, sample_sha256, write_sample
from forgereceipts.exchange import (
    SAVED_PLAIN,
    dump_bundle,
    dump_receipt,
    load_receipts_from_text,
    verify_payload,
    verify_text,
    parse_payload,
)
from forgereceipts.filing import render_html, render_txt, templates as filing_templates
from forgereceipts.forensics import reverify_bytes, reverify_path, sha256_bytes, sha256_path
from forgereceipts.legal import NOT_LEGAL_ADVICE, reference as legal_reference
from forgereceipts.limits import MAX_BODY, MAX_FILE_BYTES
from forgereceipts.lock import SessionLock
from forgereceipts.paths import data_dir as resolve_data_dir
from forgereceipts.plain import NOT_LEGAL_PROOF, TOO_BIG, PlainError, parse_json_bytes
from forgereceipts.score import pattern_strength
from forgereceipts.store import ForgeStore, verify_jsonl
from forgereceipts.tools import availability as tools_availability, run as tools_run

DEFAULT_HOST = "127.0.0.1"
DEFAULT_PORT = 8787
STATIC_DIR = Path(__file__).resolve().parent / "static"

_STATE = threading.local()

def _app_state(data_dir: Path) -> dict[str, Any]:
    # Process-wide singleton keyed by data_dir string.
    cache = getattr(_app_state, "_cache", None)
    if cache is None:
        cache = {}
        setattr(_app_state, "_cache", cache)
    key = str(data_dir.resolve())
    if key not in cache:
        cache[key] = {
            "dir": data_dir,
            "store": ForgeStore(data_dir),
            "lock": SessionLock(data_dir),
        }
    return cache[key]

def reset_state() -> None:
    setattr(_app_state, "_cache", {})

class ForgeHandler(BaseHTTPRequestHandler):
    server_version = "ForgeReceipts/0.2.0"

    @property
    def data_dir(self) -> Path:
        return Path(getattr(self.server, "forge_data_dir")) # type: ignore[arg-type]

    def log_message(self, fmt: str, *args: object) -> None:
        sys_stderr = __import__("sys").stderr
        line = "127.0.0.1 local %s" % (fmt % args)
        sys_stderr.write(line + "\n")
        if debug_enabled():
            debug_log(line)

    def _state(self) -> dict[str, Any]:
        return _app_state(self.data_dir)

    def _send(self, status: int, body: bytes, content_type: str, extra: dict[str, str] | None = None) -> None:
        self.send_response(status)
        self.send_header("Content-Type", content_type)
        self.send_header("Content-Length", str(len(body)))
        self.send_header("Cache-Control", "no-store")
        self.send_header("X-ForgeReceipts", "local-only")
        if extra:
            for k, v in extra.items():
                self.send_header(k, v)
        self.end_headers()
        self.wfile.write(body)

```



```

def _json(self, payload: Any, status: int = 200) -> None:
    raw = json.dumps(payload, indent=2, ensure_ascii=False).encode("utf-8") + b"\n"
    self._send(status, raw, "application/json; charset=utf-8")

def _plain_error(self, exc: BaseException, status: int = 400) -> None:
    payload: dict[str, Any] = {
        "error": str(exc),
        "plain": True,
        "disclaimer": NOT_LEGAL_PROOF,
    }
    if debug_enabled():
        payload["debug_type"] = type(exc).__name__
    self._json(payload, status)

def _read_json(self) -> dict[str, Any]:
    length = int(self.headers.get("Content-Length") or "0")
    if length > MAX_BODY:
        raise PlainError(TOO_BIG)
    raw = self.rfile.read(length) if length else b""
    return parse_json_bytes(raw)

def _spa(self) -> None:
    index = STATIC_DIR / "index.html"
    body = index.read_bytes()
    self._send(200, body, "text/html; charset=utf-8")

def _static(self, name: str) -> None:
    # Prevent path traversal.
    dest = (STATIC_DIR / name).resolve()
    if STATIC_DIR.resolve() not in dest.parents and dest != STATIC_DIR.resolve():
        self._json({"error": "not found"}, 404)
        return
    if not dest.is_file():
        self._json({"error": "not found"}, 404)
        return
    ctype = mimetypes.guess_type(str(dest))[0] or "application/octet-stream"
    if dest.suffix == ".webmanifest":
        ctype = "application/manifest+json"
    if dest.suffix in {".js", ".css", ".html", ".webmanifest", ".json"}:
        ctype = {
            ".js": "application/javascript; charset=utf-8",
            ".css": "text/css; charset=utf-8",
            ".html": "text/html; charset=utf-8",
            ".webmanifest": "application/manifest+json",
            ".json": "application/json; charset=utf-8",
        }[dest.suffix]
    self._send(200, dest.read_bytes(), ctype)

def _locked_out(self) -> bool:
    lock: SessionLock = self._state()["lock"]
    return lock.is_set() and not lock.unlocked

def do_OPTIONS(self) -> None: # noqa: N802
    self._send(204, b"", "text/plain")

def do_GET(self) -> None: # noqa: N802
    parsed = urlparse(self.path)
    path = parsed.path
    qs = parse_qs(parsed.query)

    if path == "/health":
        self._json({"ok": True, "bind": "127.0.0.1", "version": __version__})
        return
    if path in {"/", "/index.html"}:
        self._spa()
        return
    if path == "/manifest.webmanifest":
        self._static("manifest.webmanifest")
        return
    if path.startswith("/static/"):
        self._static(path[len("/static/") :])
        return
    # SPA routes ? never 500 a tools/nav page.
    if path in {
        "/home",
        "/incident",
        "/forensics",
        "/journal",
        "/filing",
        "/tools",
        "/verify",
        "/lock",
        "/receipts",
        "/import",
        "/export",
        "/demo",
    }:
        self._spa()
        return
    if path.startswith("/api/"):
        if path not in {"/api/lock", "/api/meta"} and self._locked_out():
            self._json({"error": "locked", "unlocked": False}, 401)
            return
        try:

```

```

        self._api_get(path, qs)
    except PlainError as exc:
        self._plain_error(exc, 400)
    except Exception as exc: # noqa: BLE001
        debug_log(f"GET {path} {type(exc).__name__}: {exc}")
        self._plain_error(exc, 400)
    return

self._spa()

def do_POST(self) -> None: # noqa: N802
    parsed = urlparse(self.path)
    path = parsed.path
    if not path.startswith("/api/"):
        self._json({"error": "not found"}, 404)
        return
    if path not in {"/api/lock/unlock", "/api/lock/set"} and self._locked_out():
        self._json({"error": "locked", "unlocked": False}, 401)
        return
    try:
        body = self._read_json()
        self._api_post(path, body)
    except PlainError as exc:
        self._plain_error(exc, 400)
    except Exception as exc: # noqa: BLE001
        debug_log(f"POST {path} {type(exc).__name__}: {exc}")
        self._plain_error(exc, 400)

def _api_get(self, path: str, qs: dict[str, list[str]]) -> None:
    state = self._state()
    store: ForgeStore = state["store"]
    store.reload()
    lock: SessionLock = state["lock"]

    if path == "/api/meta":
        self._json(
            {
                "name": "ForgeReceipts",
                "version": __version__,
                "motto": "Child's Best Interests First. Integrity Over Narrative. Local Control. Always.",
                "disclaimer": NOT_LEGAL_ADVICE,
                "bind": "127.0.0.1",
                "local_only": True,
                "telemetry": False,
                "accounts": False,
                "not_legal_advice": True,
                "not_legal_proof": True,
                "disclaimer": NOT_LEGAL_PROOF,
                "debug": debug_enabled(),
                "max_file_bytes": MAX_FILE_BYTES,
                "download": "https://forgereceipts-download-tracker.vibelock.workers.dev/",
                "one_counter": True,
                "lock": lock.status(),
                "tools": tools_availability(),
            }
        )
        return
    if path == "/api/lock":
        self._json(lock.status())
        return
    if path == "/api/receipts":
        kind = (qs.get(
et("kind") or [None])[0]
        rows = store.records_of(kind) if kind else store.records()
        self._json({"receipts": rows, "length": len(store), "verify": _verify_dict(store)})
        return
    if path.startswith("/api/receipts/"):
        digest = path.rsplit("/", 1)[-1]
        row = store.get_by_hash(digest)
        if row is None:
            self._json({"error": "No receipt with that hash on this computer.", "plain": True}, 404)
            return
        self._json({"receipt": row, "plain": _plain_for(row), "disclaimer": NOT_LEGAL_PROOF})
        return
    if path == "/api/doctor":
        from forgereceipts.doctor import run_doctor

        self._json(run_doctor(self.data_dir))
        return
    if path == "/api/demo":
        self._json(
            {
                "name": SAMPLE_NAME,
                "sha256": sample_sha256(),
                "plain": "Tap Try a sample to save a practice receipt.",
                "disclaimer": NOT_LEGAL_PROOF,
            }
        )
        return
    if path == "/api/score":
        self._json(pattern_strength(store.records()))
        return
    if path == "/api/legal":
        jid = (qs.get("jurisdiction") or ["IN"])[0]
        self._json(legal_reference(jid))

```

```

        return
    if path == "/api/filing":
        self._json(filing_templates())
        return
    if path == "/api/tools":
        self._json({"available": tools_availability(), "panels": list(tools_availability())})
        return
    if path == "/api/chain.jsonl":
        p = store.path
        text = p.read_text(encoding="utf-8") if p.is_file() else ""
        self._send(200, text.encode("utf-8"), "application/x-ndjson; charset=utf-8")
        return
    self._json({"error": "not found"}, 404)

def _api_post(self, path: str, body: dict[str, Any]) -> None:
    state = self._state()
    store: ForgeStore = state["store"]
    store.reload()
    lock: SessionLock = state["lock"]

    if path == "/api/lock/set":
        lock.set_passphrase(str(body.get("passphrase") or ""))
        self._json(lock.status())
        return
    if path == "/api/lock/unlock":
        ok = lock.unlock(str(body.get("passphrase") or ""))
        self._json({"ok": ok}, 200 if ok else 401)
        return
    if path == "/api/lock/clear":
        lock.clear(str(body.get("passphrase") or ""))
        self._json(lock.status())
        return
    if path == "/api/incident":
        row = store.append_incident(
            summary=str(body.get("summary") or ""),
            evidence=str(body.get("evidence") or ""),
            child_impact=str(body.get("child_impact") or ""),
            confidence=float(body.get("confidence") if body.get("confidence") is not None else 1.0),
            timestamp=body.get("timestamp"),
            file_sha256=body.get("file_sha256"),
            file_name=body.get("file_name"),
        )
        self._json({"receipt": row, "verify": _verify_dict(store)})
        return
    if path == "/api/journal":
        row = store.append_journal(
            summary=str(body.get("summary") or ""),
            evidence=str(body.get("evidence") or ""),
            child_impact=str(body.get("child_impact") or ""),
            confidence=float(body.get("confidence") if body.get("confidence") is not None else 1.0),
            timestamp=body.get("timestamp"),
            file_sha256=body.get("file_sha256"),
            file_name=body.get("file_name"),
            private_note=body.get("private_note"),
        )
        self._json({"receipt": row, "verify": _verify_dict(store)})
        return
    if path == "/api/forensics/hash":
        digest, name = _hash_from_body(body)
        impact = str(body.get("child_impact") or "Hash of a local file retained for later re-verification.")
        row = store.append_forensics(
            summary=str(body.get("summary") or f"SHA-256 {name}"),
            file_sha256=digest,
            file_name=name,
            child_impact=impact,
        )
        self._json(
            {
                "sha256": digest,
                "file_name": name,
                "receipt": row,
                "plain": SAVED_PLAIN,
                "hash": row.get("hash"),
                "disclaimer": NOT_LEGAL_PROOF,
                "not_legal_proof": True,
            }
        )
        return
    if path == "/api/forensics/verify":
        expected = str(body.get("expected") or body.get("file_sha256") or "")
        if body.get("content_b64"):
            data = base64.b64decode(body["content_b64"])
            result = verify_bytes(data, expected)
        elif body.get("path"):
            result = verify_path(body["path"], expected)
        else:
            raise ValueError("content_b64 or path required")
        self._json(result)
        return
    if path == "/api/verify":
        text = str(body.get("jsonl") or "")
        if not text and store.path.is_file():
            text = store.path.read_text(encoding="utf-8")
        self._json(verify_jsonl(text))
        return

```

```

if path == "/api/filing/export":
    fmt = str(body.get("format") or "txt").lower()
    content = render_html(body) if fmt == "html" else render_txt(body)
    export_dir = self.data_dir / "exports"
    export_dir.mkdir(parents=True, exist_ok=True)
    ext = "html" if fmt == "html" else "txt"
    dest = export_dir / f"filing-export.{ext}"
    dest.write_text(content, encoding="utf-8")
    self._json({"path": str(dest), "format": ext, "content": content, "disclaimer": NOT_LEGAL_ADVICE})
    return
if path == "/api/receipt/export":
    digest = str(body.get("hash") or body.get("id") or "").strip()
    if digest:
        row = store.get_by_hash(digest)
        if row is None:
            raise PlainError("No receipt with that hash on this computer.")
        text_out = dump_receipt(row)
        dest = self.data_dir / "exports"
        dest.mkdir(parents=True, exist_ok=True)
        out = dest / f"{row['hash']}.json"
        out.write_text(text_out, encoding="utf-8")
        self._json(
            {
                "path": str(out),
                "filename": out.name,
                "content": text_out,
                "receipt": row,
                "plain": "Copied this receipt into a file you can keep.",
                "disclaimer": NOT_LEGAL_PROOF,
            }
        )
        return
    rows = store.records()
    if not rows:
        raise PlainError("There is no receipt to export yet. Add a file or try a sample first.")
    text_out = dump_bundle(rows) if len(rows) > 1 else dump_receipt(rows[-1])
    dest = self.data_dir / "exports"
    dest.mkdir(parents=True, exist_ok=True)
    name = "receipts.json" if len(rows) > 1 else f"{rows[-1]['hash']}.json"
    out = dest / name
    out.write_text(text_out, encoding="utf-8")
    self._json(
        {
            "path": str(out),
            "filename": name,
            "content": text_out,
            "count": len(rows),
            "plain": "Copied your receipt(s) into a file you can keep.",
            "disclaimer": NOT_LEGAL_PROOF,
        }
    )
    return
if path == "/api/receipt/import":
    raw_text = str(body.get("json") or body.get("text") or "")
    if not raw_text.strip():
        raise PlainError("Paste a receipt file to import.")
    payload = parse_payload(raw_text)
    checked = verify_payload(payload)
    if not checked.get("ok"):
        self._json({"checked": False, "imported": False, 400})
        return
    imported = []
    for rec in payload["receipts"]:
        row = store.append_import(rec)
        imported.append(row)
    last = imported[-1]
    self._json(
        {
            "ok": True,
            "verdict": "PASS",
            "plain": "Imported a receipt. The hash matches. A receipt is not legal proof.",
            "disclaimer": NOT_LEGAL_PROOF,
            "receipt": last,
            "imported": imported,
            "count": len(imported),
            "verify": checked,
        }
    )
    return
if path == "/api/demo":
    sample_path = write_sample(self.data_dir)
    digest = sample_sha256()
    row = store.append_forensics(
        summary="sample demo file",
        file_sha256=digest,
        file_name=SAMPLE_NAME,
        child_impact="Sample only so you can see a receipt. A receipt is not legal proof.",
    )
    self._json(
        {
            "sha256": digest,
            "file_name": SAMPLE_NAME,
            "path": str(sample_path),
            "receipt": row,
            "plain": SAVED_PLAIN,
        }
    )

```

```

        "hash": row.get("hash"),
        "disclaimer": NOT_LEGAL_PROOF,
        "not_legal_proof": True,
        "sample": True,
    }
    )
    return
if path.startswith("/api/tools/"):
    name = path.rsplit("/", 1)[-1]
    self._json(tools_run(name, body))
    return
self._json({"error": "not found"}, 404)

def _hash_from_body(body: dict[str, Any]) -> tuple[str, str]:
    if body.get("content_b64"):
        try:
            data = base64.b64decode(body["content_b64"], validate=False)
        except Exception as exc: # noqa: BLE001
            raise PlainError("That file could not be read. Try another file.") from exc
        if len(data) > MAX_FILE_BYTES:
            raise PlainError(TOO_BIG)
        name = str(body.get("file_name") or "upload.bin")
        return sha256_bytes(data), name
    if body.get("path"):
        path = str(body["path"])
        p = Path(path)
        if p.is_file() and p.stat().st_size > MAX_FILE_BYTES:
            raise PlainError(TOO_BIG)
        return sha256_path(path), os.path.basename(path)
    raise PlainError("Pick a file first.")

def _plain_for(row: dict[str, Any]) -> str:
    name = row.get("file_name")
    if name:
        return f"{SAVED_PLAIN} ({name})"
    return SAVED_PLAIN

def _verify_dict(store: ForgeStore) -> dict[str, Any]:
    result = store.verify()
    return {
        "ok": result.ok,
        "verdict": "PASS" if result.ok else "FAIL",
        "length": result.length,
        "errors": list(result.errors),
        "first_hash": result.first_hash,
        "last_hash": result.last_hash,
    }

def make_server(
    host: str = DEFAULT_HOST,
    port: int = DEFAULT_PORT,
    data_dir: str | Path | None = None,
) -> ThreadingHTTPServer:
    if host not in {"127.0.0.1", "localhost", "::1"}:
        raise ValueError("ForgeReceipts binds 127.0.0.1 only")
    directory = resolve_data_dir(data_dir)
    httpd = ThreadingHTTPServer((host, port), ForgeHandler)
    httpd.forge_data_dir = directory # type: ignore[attr-defined]
    return httpd

def serve(
    host: str = DEFAULT_HOST,
    port: int = DEFAULT_PORT,
    data_dir: str | Path | None = None,
) -> None:
    httpd = make_server(host, port, data_dir)
    bound_host, bound_port = httpd.server_address[:2]
    print(
        f"ForgeReceipts {_version__} http://{bound_host}:{bound_port}/ "
        "(local only, not legal advice)"
    )
    try:
        httpd.serve_forever()
    except KeyboardInterrupt:
        print("\nstopped")
    finally:
        httpd.server_close()

```

forgereceipts/install.sh

```
#!/usr/bin/env bash
# ForgeReceipts one-click install. Counted download via this project's Worker.
# Usage: curl -fsSL https://forgereceipts-download-tracker.vibelock.workers.dev/install.sh | bash
set -euo pipefail

HOST="${FORGERECEIPTS_HOME_HOST:-https://forgereceipts-download-tracker.vibelock.workers.dev}"
ASSET="${FORGERECEIPTS_HOME_ASSET:-forgereceipts-0.2.0.tar.gz}"
WORKDIR="${FORGERECEIPTS_HOME:-$HOME/forgereceipts}"

mkdir -p "$WORKDIR"
cd "$WORKDIR"

echo "Downloading counted tarball from ${HOST}/download (User-Agent Mozilla/5.0)"
curl -fsSL -A 'Mozilla/5.0' "${HOST}/download?asset=${ASSET}" -o "${ASSET}"

tar -xzf "${ASSET}"
DIR="$(find . -maxdepth 1 -type d -name 'forgereceipts-*' | head -n 1)"
if [ -n "${DIR}" ]; then
  cd "${DIR}"
fi

python3 -m venv .venv
# shellcheck disable=SC1091
. .venv/bin/activate
python -m pip install -U pip
python -m pip install -e .

echo
echo "Installed ForgeReceipts."
echo "Run: forgereceipts ui"
echo "Then open http://127.0.0.1:8787 (loopback only)"
echo "Author: Aziel Eliab."
```

forgereceipts/mobile/lib/main.dart

```

import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import 'store.dart';
import 'theme.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const ForgeReceiptsApp());
}

class ForgeReceiptsApp extends StatelessWidget {
  const ForgeReceiptsApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ForgeReceipts',
      debugShowCheckedModeBanner: false,
      theme: buildAppTheme(),
      home: const ListPage(),
    );
  }
}

class ListPage extends StatefulWidget {
  const ListPage({super.key});

  @override
  State<ListPage> createState() => _ListPageState();
}

class _ListPageState extends State<ListPage> {
  final _store = ReceiptStore();
  List<Receipt> _items = [];
  bool _loading = true;
  bool _simple = true;
  String? _flash;

  @override
  void initState() {
    super.initState();
    _reload();
  }

  Future<void> _reload() async {
    final items = await _store.load();
    if (!mounted) return;
    setState(() {
      _items = items;
      _loading = false;
    });
  }

  Future<void> _add() async {
    final added = await Navigator.of(context).push<bool>(
      MaterialPageRoute(builder: (_) => AddPage(store: _store)),
    );
    if (added == true) {
      setState(() => _flash = savedPlain);
      await _reload();
    }
  }

  Future<void> _export(Receipt r) async {
    final text = await _store.exportOne(r);
    if (!mounted) return;
    setState(() => _flash = 'Copied this receipt into a file you can keep.');
```

ScaffoldMessenger.of(context).showSnackBar(
 SnackBar(content: Text('Exported \${text.length} characters to the clipboard.')),
);

```

  }

  Future<void> _import() async {
    final imported = await Navigator.of(context).push<Receipt>(
      MaterialPageRoute(builder: (_) => ImportPage(store: _store)),
    );
    if (imported != null) {
      setState(() => _flash = 'Imported a receipt. A receipt is not legal proof.');
```

await _reload();

```

    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('ForgeReceipts'),
        actions: [
          TextButton(
            onPressed: () => setState(() => _simple = !_simple),
            child: Text(_simple ? 'Simple' : 'Advanced'),

```

```

    ),
  ],
),
body: Column(
  children: [
    Container(
      width: double.infinity,
      color: const Color(0x33C9A227),
      padding: const EdgeInsets.all(12),
      child: const Text(
        notLegalAdvice,
        style: TextStyle(fontWeight: FontWeight.w600),
      ),
    ),
    Padding(
      padding: const EdgeInsets.fromLTRB(16, 12, 16, 8),
      child: Text(
        motto,
        style: Theme.of(context).textTheme.bodyMedium?.copyWith(
          color: kGold,
          fontStyle: FontStyle.italic,
        ),
      ),
    ),
    const Padding(
      padding: EdgeInsets.symmetric(horizontal: 16),
      child: Text(
        'On-device list only. Nothing is uploaded. This app does not '
        'file with any court, Odyssey, or email. A receipt is not legal proof.',
      ),
    ),
    Padding(
      padding: const EdgeInsets.fromLTRB(16, 12, 16, 8),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        children: [
          FilledButton(
            onPressed: _add,
            child: const Padding(
              padding: EdgeInsets.symmetric(vertical: 14),
              child: Text('Add file', style: TextStyle(fontSize: 20)),
            ),
          ),
          const SizedBox(height: 8),
          OutlinedButton(
            onPressed: _import,
            child: const Padding(
              padding: EdgeInsets.symmetric(vertical: 14),
              child: Text('Import receipt', style: TextStyle(fontSize: 18)),
            ),
          ),
          const SizedBox(height: 8),
          OutlinedButton(
            onPressed: _items.isEmpty ? null : () => _export(_items.first),
            child: const Padding(
              padding: EdgeInsets.symmetric(vertical: 14),
              child: Text('Export receipt', style: TextStyle(fontSize: 18)),
            ),
          ),
        ],
      ),
    ),
    if (_flash != null)
      Padding(
        padding: const EdgeInsets.symmetric(horizontal: 16),
        child: Text(
          _flash!,
          style: const TextStyle(
            color: kGold,
            fontWeight: FontWeight.w700,
          ),
        ),
      ),
    const SizedBox(height: 8),
    Expanded(
      child: _loading
        ? const Center(child: CircularProgressIndicator())
        : _items.isEmpty
          ? const Center(child: Text('No receipts yet.'))
          : ListView.builder(
              itemCount: _items.length,
              itemBuilder: (context, i) {
                final r = _items[i];
                return Card(
                  margin: const EdgeInsets.symmetric(
                    horizontal: 12,
                    vertical: 6,
                  ),
                  child: ListTile(
                    title: Text(r.summary),
                    subtitle: Text(
                      _simple
                        ? '${r.kind}\n${r.hash.isEmpty ? r.id : r.hash}'
                        : '${r.kind} ? ${r.createdAt.toLocal()}\n${r.note}\n${r.hash}',
                    ),
                  ),
                );
              },
            ),
    ),
  ],
),

```



```

        isThreeLine: true,
        onTap: () => Navigator.of(context).push(
          MaterialPageRoute(
            builder: (_) => DetailPage(
              receipt: r,
              simple: _simple,
              onExport: () => _export(r),
            ),
          ),
        ),
      ),
    ),
  ],
);
}
}

class DetailPage extends StatelessWidget {
  const DetailPage({
    super.key,
    required this.receipt,
    required this.simple,
    required this.onExport,
  });

  final Receipt receipt;
  final bool simple;
  final VoidCallback onExport;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Receipt')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          const Text(notLegalAdvice),
          const SizedBox(height: 12),
          Text(receipt.summary, style: Theme.of(context).textTheme.titleLarge),
          const SizedBox(height: 8),
          const Text('This is the saved receipt for this file.'),
          const SizedBox(height: 8),
          SelectableText(receipt.hash.isEmpty ? receipt.id : receipt.hash),
          if (!simple) ...[
            const SizedBox(height: 12),
            Text('kind: ${receipt.kind}'),
            Text('created: ${receipt.createdAt.toIso8601String()}'),
            Text(receipt.note),
            Text(receipt.childImpact),
            if (receipt.fileSha256.isNotEmpty) Text(receipt.fileSha256),
          ],
          const SizedBox(height: 20),
          FilledButton(
            onPressed: onExport,
            child: const Text('Export receipt'),
          ),
        ],
      ),
    );
  }
}

class ImportPage extends StatefulWidget {
  const ImportPage({super.key, required this.store});
  final ReceiptStore store;

  @override
  State<ImportPage> createState() => _ImportPageState();
}

class _ImportPageState extends State<ImportPage> {
  final _text = TextEditingController();
  String? _error;
  bool _working = false;

  @override
  void dispose() {
    _text.dispose();
    super.dispose();
  }

  Future<void> _go() async {
    setState(() {
      _error = null;
      _working = true;
    });
    try {
      final r = await widget.store.importText(_text.text);
      if (!mounted) return;
      Navigator.of(context).pop(r);
    } on FormatException catch (e) {

```

```

        setState(() {
          _error = e.message;
          _working = false;
        });
      } catch (e) {
        setState(() {
          _error = 'That file is not valid JSON. Check commas and quotes, then try again.';
          _working = false;
        });
      }
    }
  }

  Future<void> _paste() async {
    final data = await Clipboa
rd.getData('text/plain');
    if (data?.text != null) setState(() => _text.text = data!.text!);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Import receipt')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          const Text(notLegalAdvice),
          const SizedBox(height: 12),
          const Text('Paste a receipt JSON file. This stays on this phone.'),
          const SizedBox(height: 12),
          TextField(
            controller: _text,
            maxLines: 10,
            decoration: const InputDecoration(labelText: 'Receipt JSON'),
          ),
          const SizedBox(height: 12),
          OutlinedButton(onPressed: _paste, child: const Text('Paste')),
          const SizedBox(height: 8),
          FilledButton(
            onPressed: _working ? null : _go,
            child: const Text('Import receipt'),
          ),
          if (_error != null) ...[
            const SizedBox(height: 12),
            Text(_error!, style: const TextStyle(color: Color(0xFFB54A4A))),
          ],
        ],
      ),
    );
  }
}

class AddPage extends StatefulWidget {
  const AddPage({super.key, required this.store});
  final ReceiptStore store;

  @override
  State<AddPage> createState() => _AddPageState();
}

class _AddPageState extends State<AddPage> {
  final _summary = TextEditingController();
  final _note = TextEditingController();
  final _child = TextEditingController();
  String _kind = 'incident';
  bool _saving = false;

  @override
  void dispose() {
    _summary.dispose();
    _note.dispose();
    _child.dispose();
    super.dispose();
  }

  Future<void> _save() async {
    if (_summary.text.trim().isEmpty) return;
    setState(() => _saving = true);
    final r = Receipt(
      id: DateTime.now().microsecondsSinceEpoch.toString(),
      createdAt: DateTime.now().toUtc(),
      summary: _summary.text.trim(),
      note: _note.text.trim(),
      kind: _kind,
      childImpact: _child.text.trim(),
    );
    await widget.store.add(r);
    if (!mounted) return;
    Navigator.of(context).pop(true);
  }
}

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Add file')),
      body: ListView(

```

```

padding: const EdgeInsets.all(16),
children: [
  const Text(notLegalAdvice),
  const SizedBox(height: 12),
  DropdownButtonFormField<String>({
    value: _kind,
    items: const [
      DropdownMenuItem(value: 'incident', child: Text('Incident')),
      DropdownMenuItem(value: 'journal', child: Text('Time with child')),
      DropdownMenuItem(value: 'forensics', child: Text('Forensics note')),
    ],
    onChanged: (v) => setState(() => _kind = v ?? 'incident'),
    decoration: const InputDecoration(labelText: 'Kind'),
  ),
  const SizedBox(height: 12),
  TextField(
    controller: _summary,
    decoration: const InputDecoration(labelText: 'Summary'),
  ),
  const SizedBox(height: 12),
  TextField(
    controller: _note,
    maxLines: 4,
    decoration: const InputDecoration(labelText: 'Private note'),
  ),
  const SizedBox(height: 12),
  TextField(
    controller: _child,
    decoration: const InputDecoration(
      labelText: 'Child impact (hashed on desktop; local here)',
    ),
  ),
  const SizedBox(height: 20),
  FilledButton(
    onPressed: _saving ? null : _save,
    child: const Text('Save on this device'),
  ),
],
),
);
}
}

```

forgereceipts/mobile/lib/store.dart

```

import 'dart:convert';

import 'package:flutter/services.dart';
import 'package:shared_preferences/shared_preferences.dart';

const motto =
  "Child's Best Interests First. Integrity Over Narrative. Local Control. Always.";

const notLegalAdvice =
  'Not legal advice. A receipt is not legal proof. This software does not '
  'practice law, does not guarantee any court outcome, and does not contact '
  'any court. No filing. Consult a licensed attorney in your jurisdiction.';

const receiptFormat = 'forgereceipts.receipt/v1';
const productVersion = '0.2.0';
const savedPlain = 'Saved a receipt for this file';

class Receipt {
  Receipt({
    required this.id,
    required this.createdAt,
    required this.summary,
    required this.note,
    this.kind = 'incident',
    this.childImpact = '',
    this.hash = '',
    this.fileName = '',
    this.fileSha256 = '',
  });

  final String id;
  final DateTime createdAt;
  final String summary;
  final String note;
  final String kind;
  final String childImpact;
  final String hash;
  final String fileName;
  final String fileSha256;

  Map<String, Object> toJson() => {
    'id': id,
    'createdAt': createdAt.toIso8601String(),
    'timestamp': createdAt.toIso8601String(),
    'summary': summary,
    'note': note,
    'kind': kind,
    'childImpact': childImpact,
    'child_impact': childImpact,
    'hash': hash,
    'file_name': fileName,
    'file_sha256': fileSha256,
  };

  static Receipt fromJson(Map<String, dynamic> m) => Receipt(
    id: m['id'] as String? ?? m['hash'] as String? ?? '',
    createdAt: DateTime.tryParse(
      (m['createdAt'] ?? m['timestamp']) as String? ?? ''
    ) ??
      DateTime.fromMillisecondsSinceEpoch(0),
    summary: m['summary'] as String? ?? '',
    note: m['note'] as String? ?? m['evidence'] as String? ?? '',
    kind: m['kind'] as String? ?? 'incident',
    childImpact:
      m['childImpact'] as String? ?? m['child_impact'] as String? ?? '',
    hash: m['hash'] as String? ?? '',
    fileName: m['file_name'] as String? ?? m['fileName'] as String? ?? '',
    fileSha256:
      m['file_sha256'] as String? ?? m['fileSha256'] as String? ?? '',
  );

  String exportEnvelope() {
    final envelope = <String, Object>{
      'format': receiptFormat,
      'product': 'forgereceipts',
      'product_version': productVersion,
      'disclaimer': notLegalAdvice,
      'not_legal_advice': true,
      'not_legal_proof': true,
      'receipt': toJson(),
    };
    return const JsonEncoder.withIndent(' ').convert(envelope);
  }

  static Receipt importEnvelope(String raw) {
    late final Object? decoded;
    try {
      decoded = jsonDecode(raw);
    } on FormatException {
      throw const FormatException(
        'That file is not valid JSON. Check commas and quotes, then try again.'
      );
    }
  }
}

```

```

    }
    if (decoded is! Map) {
        throw const FormatException(
            'That JSON is not a receipt object. It must be a set of named fields inside { }.',
        );
    }
    final map = Map<String, dynamic>.from(decoded);
    final rec = map['receipt'] ?? map;
    if (rec is! Map) {
        throw const FormatException(
            'No receipt was found in that file. Export a receipt and try again.',
        );
    }
    return Receipt.fromJson(Map<String, dynamic>.from(rec));
}
}

class ReceiptStore {
    static const _key = 'forgereceipts.receipts.v1';

    Future<List<Receipt>> load() async {
        final prefs = await SharedPreferences.getInstance();
        final raw = prefs.getString(_key);
        if (raw == null || raw.isEmpty) return [];
        final list = jsonDecode(raw) as List<dynamic>;
        return list
            .map((e) => Receipt.fromJson(Map<String, dynamic>.from(e as Map)))
            .toList()
            ..sort((a, b) => b.createdAt.compareTo(a.createdAt));
    }

    Future<void> add(Receipt r) async {
        final all = await load();
        all.insert(0, r);
        final prefs = await SharedPreferences.getInstance();
        await prefs.setString(
            _key,
            jsonEncode(all.map((e) => e.toJson()).toList()),
        );
    }

    Future<String> exportOne(Receipt r) async {
        final text = r.exportEnvelope();
        await Clipboard.setData(ClipboardData(text: text));
        return text;
    }

    Future<Receipt> importText(String raw) async {
        final r = Receipt.importEnvelope(raw);
        await add(r);
        return r;
    }
}

```

forgereceipts/mobile/lib/theme.dart

```

import 'package:flutter/material.dart';

/// Matte black + gold Material 3 dark theme. No analytics.
const Color kMatteBlack = Color(0xFF0B0B0B);
const Color kSurface = Color(0xFF141414);
const Color kGold = Color(0xFFC9A227);
const Color kGoldDim = Color(0xFF8A7219);
const Color kIvory = Color(0xFFE8E0D0);

ThemeData buildAppTheme() {
  const scheme = ColorScheme.dark(
    brightness: Brightness.dark,
    primary: kGold,
    onPrimary: kMatteBlack,
    secondary: kGoldDim,
    onSecondary: kIvory,
    surface: kSurface,
    onSurface: kIvory,
    error: Color(0xFFB54A4A),
    onError: kIvory,
  );
  return ThemeData(
    useMaterial3: true,
    brightness: Brightness.dark,
    colorScheme: scheme,
    scaffoldBackgroundColor: kMatteBlack,
    appBarTheme: const AppBarTheme(
      backgroundColor: kMatteBlack,
      foregroundColor: kGold,
      elevation: 0,
      centerTitle: false,
    ),
    cardTheme: CardThemeData(
      color: kSurface,
      elevation: 0,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
        side: const BorderSide(color: Color(0x33C9A227)),
      ),
    ),
    inputDecorationTheme: InputDecorationTheme(
      filled: true,
      fillColor: const Color(0xFF1A1A1A),
      border: OutlineInputBorder(borderRadius: BorderRadius.circular(10)),
      focusedBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(10),
        borderSide: const BorderSide(color: kGold),
      ),
    ),
    segmentedButtonTheme: SegmentedButtonThemeData(
      style: ButtonStyle(
        foregroundColor: WidgetStateProperty.resolveWith((s) {
          return s.contains(WidgetState.selected) ? kMatteBlack : kIvory;
        }),
        backgroundColor: WidgetStateProperty.resolveWith((s) {
          return s.contains(WidgetState.selected) ? kGold : kSurface;
        }),
      ),
    ),
  );
}

```

forgereceipts/mobile/pubspec.yaml

```
name: forgereceipts
description: Local-first ForgeReceipts notes. On-device only. Not legal advice. No court filing.
publish_to: "none"
version: 0.2.0+2

environment:
  sdk: ">=3.3.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter
  shared_preferences: ^2.3.3

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0

flutter:
  uses-material-design: true
```

forgereceipts/pyproject.toml

```

[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "forgereceipts"
version = "0.2.0"
description = "Local-first evidence integrity platform for pro se fathers in family court (ForgeReceipts, July 2026).  
Not legal advice."
readme = "README.md"
license = {file = "LICENSE"}
authors = [{name = "Aziel Eliab"}]
requires-python = ">=3.10"
keywords = [
    "evidence",
    "receipts",
    "hash-chain",
    "family-court",
    "pro-se",
    "local-first",
    "forensics",
    "forks",
]
classifiers = [
    "Development Status :: 4 - Beta",
    "License :: OSI Approved :: Apache Software License",
    "Programming Language :: Python :: 3",
    "Programming Language :: Python :: 3.10",
    "Programming Language :: Python :: 3.11",
    "Programming Language :: Python :: 3.12",
    "Programming Language :: Python :: 3.13",
    "Topic :: Security :: Cryptography",
    "Intended Audience :: End Users/Desktop",
]
dependencies = []

[project.optional-dependencies]
dev = ["pytest>=7.4"]
engines = ["numpy>=1.24", "scipy>=1.11", "cryptography>=42.0"]

[project.scripts]
forgereceipts = "forgereceipts.cli:main"

[project.urls]
Homepage = "https://github.com/AzielEliab/forgereceipts"
Download = "https://forgereceipts-download-tracker.vibelock.workers.dev/"
Issues = "https://github.com/AzielEliab/forgereceipts/issues"

[tool.setuptools.packages.find]
where = [".", "engines"]
include = [
    "forgereceipts*",
    "temporallock*",
    "vibelock*",
    "veillock*",
    "codelock*",
    "godlock*",
    "shadowlock*",
    "miragegrid*",
    "staticclock*",
]
exclude = ["tests*", "workers*", "docs*"]

[tool.setuptools.package-data]
forgereceipts = ["static/*"]
staticclock = ["data/*.json", "web/*"]
godlock = ["templates/*.html"]
miragegrid = ["templates/*.html"]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = [".", "engines"]
addopts = "-q"

```


forgereceipts/workers/download-tracker/package.json

```
{  
  "dependencies": {  
    "wrangler": "^4.127.1"  
  }  
}
```

forgereceipts/workers/download-tracker/src/index.js

```

import { handleRuntime } from "../runtime.js";

/**
 * ForgeReceipts download tracker (Cloudflare Worker).
 *
 * GET /download?repo=AzielEliab/forgereceipts&tag=latest&asset=...
 *      increments KV, 302 to the GitHub release asset
 *      (default https://github.com/AzielEliab/forgereceipts/releases)
 * GET /stats JSON totals + per-repo + per-branch breakdown
 * POST /event forks report a download {owner,repo,branch,fork,asset}
 *
 * KV binding DOWNLOADS. Keys: project|owner|repo|branch|fork
 * CORS *. No secrets in this tree.
 */

const PROJECT = "forgereceipts";
const DEFAULT_ASSET = "forgereceipts-0.2.0.tar.gz";
const DEFAULT_OWNER = "AzielEliab";
const DEFAULT_REPO = "forgereceipts";
const DEFAULT_BRANCH = "main";
const HOST = "https://forgereceipts-download-tracker.viblock.workers.dev";
const GITHUB_REPO = "https://github.com/AzielEliab/forgereceipts";

const GITHUB_RELEASES = "https://github.com/AzielEliab/forgereceipts/releases";
const GITHUB_LATEST = "https://github.com/AzielEliab/forgereceipts/releases/latest";

function corsHeaders() {
  return {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type",
  };
}

function json(body, status = 200) {
  return new Response(JSON.stringify(body, null, 2), {
    status,
    headers: { "Content-Type": "application/json; charset=utf-8", ...corsHeaders() },
  });
}

function redirect(url) {
  return new Response(null, {
    status: 302,
    headers: { Location: url, ...corsHeaders() },
  });
}

function splitOwnerRepo(value, fallbackOwner, fallbackRepo) {
  if (typeof value === "string" && value.includes("/")) {
    const [o, r] = value.split("/").filter(Boolean);
    if (o && r) return { owner: o, repo: r };
  }
  return { owner: fallbackOwner, repo: fallbackRepo };
}

function parseDims(src) {
  const get = (k) => {
    if (src == null) return null;
    if (typeof src.get === "function") {
      const v = src.get(k);
      return v == null || v === "" ? null : v;
    }
    const v = src[k];
    return v == null || v === "" ? null : v;
  };

  let owner = get("owner") || DEFAULT_OWNER;
  let repo = get("repo") || DEFAULT_REPO;
  if (typeof repo === "string" && repo.includes("/")) {
    const split = splitOwnerRepo(repo, owner, DEFAULT_REPO);
    owner = split.owner;
    repo = split.repo;
  }

  const branch = get("branch") || DEFAULT_BRANCH;
  const tag = get("tag") || "latest";
  const asset = get("asset") || "";

  const forkRaw = get("fork");
  let fork = "0";
  if (forkRaw === 1 || forkRaw === true || forkRaw === "1" || forkRaw === "true") {
    fork = "1";
  } else if (typeof forkRaw === "string" && forkRaw.includes("/")) {
    const split = splitOwnerRepo(forkRaw, owner, repo);
    owner = split.owner;
    repo = split.repo;
    fork = "1";
  } else if (forkRaw != null && forkRaw !== 0 && forkRaw !== false && forkRaw !== "0" && forkRaw !== "false") {
    fork = "1";
  }
  if (`${owner}/${repo}`.toLowerCase() !== `${DEFAULT_OWNER}/${DEFAULT_REPO}`.toLowerCase()) {

```

```

    fork = "1";
  }

  return { project: PROJECT, owner, repo, branch, fork, tag, asset };
}

function kvKey(dims) {
  return `${dims.project}|${dims.owner}|${dims.repo}|${dims.branch}|${dims.fork}`;
}

function githubAssetUrl(owner, repo, tag, asset) {
  if (!asset) {
    if (owner === DEFAULT_OWNER && repo === DEFAULT_REPO) return GITHUB_RELEASES;
    return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases`;
  }
  if (!tag || tag === "latest") {
    return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases/latest/download/${encodeURIComponent(asset)}`;
  }
  return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases/download/${encodeURIComponent(tag)}/${encodeURIComponent(asset)}`;
}

async function increment(env, dims) {
  const key = kvKey(dims);
  const n = parseInt((await env.DOWNLOADS.get(key)) || "0", 10) + 1;
  await env.DOWNLOADS.put(key, String(n));
  return n;
}

async function listAllKeys(env) {
  const keys = [];
  let cursor;
  do {
    const page = await env.DOWNLOADS.list(cursor ? { cursor } : {});
    keys.push(...page.keys);
    cursor = page.list_complete ? undefined : page.cursor;
  } while (cursor);
  return keys;
}

async function collectStats(env) {
  const keys = await listAllKeys(env);
  let total = 0;
  const by_repo = {};
  const by_branch = {};
  const by_fork = { "0": 0, "1": 0 };
  const breakdown = [];

  for (const k of keys) {
    const name = k.name;
    const n = parseInt((await env.DOWNLOADS.get(name)) || "0", 10);
    if (!Number.isFinite(n) || n <= 0) continue;
    const parts = name.split("|");
    if (parts.length < 5) continue;
    const [project, owner, repo, branch, fork] = parts;
    total += n;
    const repoId = `${owner}/${repo}`;
    by_repo[repoId] = (by_repo[repoId] || 0) + n;
    by_branch[branch] = (by_branch[branch] || 0) + n;
    const forkFlag = fork === "1" ? "1" : "0";
    by_fork[forkFlag] = (by_fork[forkFlag] || 0) + n;
    breakdown.push({ project, owner, repo, branch, fork: forkFlag, count: n });
  }

  return {
    project: PROJECT,
    total,
    by_repo,
    by_branch,
    by_fork,
    breakdown,
    note: "Forks identified by GitHub owner/repo. Key layout: project|owner|repo|branch|fork",
  };
}

function viewsKey() {
  return PROJECT + "|__views__";
}

async function incrementViews(env) {
  const n = parseInt((await env.DOWNLOADS.get(viewsKey())) || "0", 10) + 1;
  await env.DOWNLOADS.put(viewsKey(), String(n));
  return n;
}

function installScript() {
  return `#!/usr/bin/env bash
# ForgeReceipts one-click install. Counted download via this Worker.
set -euo pipefail
HOST="${HOST}"
ASSET="${DEFAULT_ASSET}"
WORKDIR="\${FORGERECEIPTS_HOME:-\$HOME/forgereceipts}"
mkdir -p "\$WORKDIR"

```

```

cd "$WORKDIR"
echo "Downloading counted tarball from ${HOST}/download (User-Agent Mozilla/5.0)?"
curl -fsSL -A 'Mozilla/5.0' "${HOST}/download?asset=${ASSET}" -o "${ASSET}"
tar -xzf "${ASSET}"
DIR="$(find . -maxdepth 1 -type d -name 'forgereceipts-*' | head -n 1)"
if [ -n "${DIR}" ]; then
  cd "${DIR}"
fi
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -U pip
python -m pip install -e .
echo
echo "Installed ForgeReceipts."
echo "Run: forgereceipts ui"
echo "Then open http://127.0.0.1:8787 (loopback only)"
echo "Author: Aziel Eliab."
';
}

async function serveAsset(request, env, asset, { head = false } = {}) {
  if (!env.ASSETS) {
    return json({ error: "assets binding missing" }, 500);
  }
  const assetUrl = new URL("/" + asset, request.url);
  const assetRes = await env.ASSETS.fetch(new Request(assetUrl, { method: "GET" }));
  if (!assetRes.ok) {
    return json({ error: "asset not hosted", asset, status: assetRes.status }, 404);
  }
  const headers = new Headers();
  headers.set("Content-Type", "application/gzip");
  headers.set("Content-Disposition", 'attachment; filename="' + asset.replaceAll('"', "'") + '"');
  headers.set("Cache-Control", "private, no-store");
  const len = assetRes.headers.get("Content-Length");
  if (len) headers.set("Content-Length", len);
  for (const [k, v] of Object.entries(corsHeaders())) headers.set(k, v);
  if (head) {
    return new Response(null, { status: 200, headers });
  }
  return new Response(assetRes.body, { status: 200, headers });
}

async function indexHtml(env) {
  const stats = await collectStats(env);
  const downloads = Number(stats.downloads != null ? stats.downloads : stats.total) || 0;
  const views = parseInt(await env.DOWNLOADS.get(viewsKey())) || "0", 10) || 0;
  const v = views.toLocaleString("en-US");
  const n = downloads.toLocaleString("en-US");
  const breakdown = (stats.breakdown || [])
    .map(
      (b) =>
        `<li><code>${b.owner}/${b.repo}</code> branch <code>${b.branch}</code> fork=${b.fork} ? ${b.count}</li>`,
    )
    .join("") || "<li>none yet</li>";
  return `<!doctype html>
<html lang="en">
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>ForgeReceipts ? Aziel Eliab</title>
<meta name="description" content="Local-first evidence integrity platform for pro se fathers in family court, by Aziel Eliab; not legal advice.">
<meta name="author" content="Aziel Eliab">
<link rel="canonical" href="https://forgereceipts-download-tracker.vibelock.workers.dev/">
<meta property="og:title" content="ForgeReceipts ? Aziel Eliab">
<meta property="og:description" content="Local-first evidence integrity platform for pro se fathers in family court, by Aziel Eliab; not legal advice.">
<meta property="og:url" content="https://forgereceipts-download-tracker.vibelock.workers.dev/">
<meta property="og:type" content="website">
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "SoftwareApplication",
  "name": "ForgeReceipts",
  "author": {
    "@type": "Person",
    "name": "Aziel Eliab"
  },
  "codeRepo": "https://github.com/AzielEliab/forgereceipts",
  "downloadUrl": "https://forgereceipts-download-tracker.vibelock.workers.dev/download",
  "license": "https://www.apache.org/licenses/LICENSE-2.0",
  "url": "https://forgereceipts-download-tracker.vibelock.workers.dev/",
  "description": "Local-first evidence integrity platform for pro se fathers in family court, by Aziel Eliab; not legal advice.",
  "identifier": "https://doi.org/10.5281/zenodo.21436074"
}
</script>
<!-- gitbaby-seo -->
<style>
:root { color-scheme: dark; }
body { font: 16px/1.45 system-ui, sans-serif; max-width: 42rem; margin: 3rem auto; padding: 0 1.25rem 4rem; background-color: #0e1014; color: #e8eaeaf; }
h1 { font-size: 1.75rem; margin: 0 0 .35rem; }
.motto { color: #9aa3b2; margin: 0 0 1.5rem; }
.card { border: 1px solid #2a3140; border-radius: 12px; padding: 1.25rem 1.35rem; background: #151922; }

```

```

.num { display: grid; grid-template-columns: 1fr 1fr; gap: .8rem; margin: 0 0 1rem; }
.count { font-size: 2.2rem; font-variant-numeric: tabular-nums; font-weight: 700; margin: 0; }
.count span { display: block; font-size: .95rem; font-weight: 500; color: #9aa3b2; }
.kid { font-size: 1.05rem; margin: 0 0 1rem; }
.btns { display: grid; grid-template-columns: 1fr 1fr; gap: .75rem; margin: 0 0 .85rem; }
@media (max-width: 520px) { .btns { grid-template-columns: 1fr; } }
a.btn, button.btn { display: block; width: 100%; box-sizing: border-box; text-align: center; font: inherit; font-size: 1.2rem; font-weight: 750; padding: 1rem 1.1rem; border-radius: 10px; border: 0; cursor: pointer; text-decoration: none; }
a.btn.primary { background: #e8eae7; color: #0e1014; }
button.btn.install { background: #c9a227; color: #14110a; }
button.btn.install.copied { background: #7dcf9a; color: #0e1014; }
.meta { margin-top: 1.1rem; color: #9aa3b2; font-size: .92rem; }
.meta a { color: #c9d4ff; }
.iso { margin-top: .85rem; font-size: .85rem; color: #7d8696; }
.banner { border: 1px solid #5c4a1a; background: #241c0d; color: #f0d78c; padding: .85rem 1rem; border-radius: 8px; margin: 0 0 1.2rem; font-size: .92rem; }
pre { background: #0e1014; padding: .75rem .9rem; overflow: auto; border-radius: 8px; font-size: .82rem; }
code { font-size: .88rem; }

.cite { margin-top: 1.4rem; padding-top: 1rem; border-top: 1px solid #2a3140; }
.cite h2 { font-size: 1.05rem; margin: 0 0 .4rem; }
.cite p { color: #c5ccd8; font-size: .95rem; }
.cite a { color: #c9d4ff; }
</style>
<body>
<h1>ForgeReceipts</h1>
<p class="motto">Child's Best Interests First. Integrity Over Narrative. Local Control. Always. Author Aziel Eliab.</p>
<p class="banner">Local-first evidence integrity packaging. Not legal advice. Does not contact courts, Odyssey, email, or any cloud service. No telemetry. Author: Aziel Eliab.</p>
<div class="card">
<div class="nums">
<p class="count">${v}<span>Views</span></p>
<p class="count">${n}<span>Downloads</span></p>
</div>
<p class="kid"><strong>Two big buttons.</strong> Download saves the gzip (the Downloads number goes up). One-click install copies a Terminal command. After it finishes, type <code>forgereceipts ui</code>.</p>
<div class="btns">
<a class="btn primary dl" href="/download?asset=${DEFAULT_ASSET}">Download</a>
<button type="button" class="btn install" id="install-btn">One-click install</button>
</div>
<pre id="install-cmd">curl -fsSL https://forgereceipts-download-tracker.vibelock.workers.dev/install.sh | bash</pre>
<p class="kid">Then run: <code>forgereceipts ui</code> and open http://127.0.0.1:8787 (this computer only).</p>
<p class="meta">The download count ticks on the Download click. The Worker serves the gzip (HTTP 200). No 302 to GitHub. Forks using this same link are counted automatically. ${DEFAULT_ASSET} ? ${n} counted.</p>
<p class="iso">Isolated counter: Worker <code>forgereceipts-download-tracker</code>, project <code>forgereceipts</code>, KV <code>FORGERECEIPTS_DOWNLOADS</code>. Not mixed with any other product. /v1 does not increment downloads.</p>
<p class="meta">Paper: <a href="https://doi.org/10.5281/zenodo.21436074">doi:10.5281/zenodo.21436074</a> ? <a href="https://zenodo.org/records/21436074">Zenodo</a> ? Apache-2.0 ? Eliab, Aziel</p>
<p class="meta"><a href="/stats">JSON stats</a> ? <a href="/openapi.json">OpenAPI</a> ? <a href="/v1/skill">Skill</a> ? <a href="/ai">AI runtime</a> ? <a href="${GITHUB_REPO}">GitHub</a> ? <a href="${GITHUB_LATEST}">releases</a></p>
<script>
(function () {
var cmd = "curl -fsSL https://forgereceipts-download-tracker.vibelock.workers.dev/install.sh | bash";
var btn = document.getElementById("install-btn");
var pre = document.getElementById("install-cmd");
if (!btn) return;
btn.addEventListener("click", function () {
function done(ok) {
btn.textContent = ok ? "Copied! Paste in Terminal, then run forgereceipts ui" : "Select the command, copy it, then run forgereceipts ui";
btn.classList.add("copied");
}
if (navigator.clipboard && navigator.clipboard.writeText) {
navigator.clipboard.writeText(cmd).then(function () { done(true); }).catch(function () { done(false); });
} else {
done(false);
if (pre && window.getSelection) {
var r = document.createRange();
r.selectNodeContents(pre);
var sel = window.getSelection();
sel.removeAllRanges();
sel.addRange(r);
}
}
});
})();
</script>
<h2>Per repo / branch / fork</h2>
<ul>${breakdown}</ul>
</div>

<section class="cite" id="cite">
<h2>How to cite</h2>
<p>Aziel Eliab. ForgeReceipts. https://github.com/AzielEliab/forgereceipts. https://forgereceipts-download-tracker.vibelock.workers.dev. https://doi.org/10.5281/zenodo.21436074.</p>
<p><a href="https://aziel-runtime.vibelock.workers.dev/">Catalog</a> ? <a href="https://github.com/AzielEliab/forgereceipts">GitHub</a> ? <a href="https://forgereceipts-download-tracker.vibelock.workers.dev/download">Download</a> ? <a href="https://forgereceipts-download-tracker.vibelock.workers.dev/cite.json">cite.json</a></p>
</section>
<!-- /gitbaby-seo -->
</body>

```

```

</html>`;
}

export default {
  async fetch(request, env) {
    const url = new URL(request.url);

    if (request.method === "OPTIONS") {
      return new Response(null, { status: 204, headers: corsHeaders() });
    }

    const runtime = await handleRuntime(request, url, env);
    if (runtime) return runtime;

    if ((url.pathname === "/install.sh" || url.pathname === "/install.sh/") && request.method === "GET") {
      return new Response(installScript(), {
        status: 200,
        headers: {
          "Content-Type": "text/x-shellscript; charset=utf-8",
          "Cache-Control": "private, no-store",
          ...corsHeaders(),
        },
      });
    }

    if (url.pathname === "/" && request.method === "GET") {
      await incrementViews(env);
      return new Response(await indexHtml(env), {
        headers: { "Content-Type": "text/html; charset=utf-8", ...corsHeaders() },
      });
    }

    if (url.pathname === "/count" && request.method === "GET") {
      const stats = await collectStats(env);
      return json({ project: PROJECT, total: stats.total || 0 });
    }

    if (url.pathname === "/stats" && request.method === "GET") {
      return json(await collectStats(env));
    }

    if (url.pathname === "/event" && request.method === "POST") {
      let body;
      try {
        body = await request.json();
      } catch {
        return json({ error: "JSON body required" }, 400);
      }
      const dims = parseDims(body || {});
      const count = await increment(env, dims);
      return json({
        ok: true,
        key: kvKey(dims),
        count,
        owner: dims.owner,
        repo: dims.repo,
        branch: dims.branch,
        fork: dims.fork,
        asset: dims.asset || null,
      });
    }

    if (url.pathname === "/go" && (request.method === "GET" || request.method === "HEAD")) {
      const dims = parseDims(url.searchParams);
      const asset = dims.asset || DEFAULT_ASSET;
      dims.asset = asset;
      if (request.method === "GET") await increment(env, dims);
      return serveAsset(request, env, asset, { head: request.method === "HEAD" });
    }

    if ((url.pathname === "/download" || url.pathname.startsWith("/download/")) && (request.method === "GET" || request.method === "HEAD")) {
      const dims = parseDims(url.searchParams);
      if (!dims.asset && url.pathname.startsWith("/download/")) {
        dims.asset = decodeURIComponent(url.pathname.slice("/download/".length));
      }
      const asset = dims.asset || DEFAULT_ASSET;
      dims.asset = asset;
      if (request.method === "GET") await increment(env, dims);
      return serveAsset(request, env, asset, { head: request.method === "HEAD" });
    }

    // gitbaby-seo-routes
    if ((url.pathname === "/robots.txt" || url.pathname === "/robots.txt/") && request.method === "GET") {
      const body = "User-agent: *\nAllow: /\nSitemap: " + HOST + "/sitemap.xml\n";
      return new Response(body, {
        status: 200,
        headers: { "Content-Type": "text/plain; charset=utf-8", ...corsHeaders() },
      });
    }

    if ((url.pathname === "/sitemap.xml" || url.pathname === "/sitemap.xml/") && request.method === "GET") {

```

```

    const locs = [HOST + "/", HOST + "/download", HOST + "/install.sh", HOST + "/v1/skill", HOST + "/openapi.json",
    GITHUB_REPO];
    const xml = '<?xml version="1.0" encoding="UTF-8"?>\n<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9"
>\n'
      + locs.map((u) => "  <url><loc>" + u + "</loc></url>").join("\n")
      + "\n</urlset>\n";
    return new Response(xml, {
      status: 200,
      headers: { "Content-Type": "application/xml; charset=utf-8", ...corsHeaders() },
    });
  }
  if ((url.pathname === "/cite.json" || url.pathname === "/cite.json/") && request.method === "GET") {
    return json({ "author": "Aziel Eliab", "title": "ForgeReceipts", "github": "https://github.com/AzielEliab/forgere
ceipts", "download": "https://forgereceipts-download-tracker.vibelock.workers.dev/download", "doi": "10.5281/zenodo.21
436074", "license": "Apache-2.0", "catalog": "https://aziel-runtime.vibelock.workers.dev/" });
  }
  // /gitbaby-seo-routes
  return json({ error: "not found" }, 404);
},
};

```

forgereceipts/workers/download-tracker/src/runtime.js

```

/**
 * ForgeReceipts hosted runtime (Cloudflare Worker).
 * Local-style receipt JSON. Not legal advice. No court filing. Does not call Odyssey.
 */
function runtimeCors() {
  return {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type",
  };
}

function runtimeJson(body, status = 200) {
  return new Response(JSON.stringify(body, null, 2), {
    status,
    headers: { "Content-Type": "application/json; charset=utf-8", ...runtimeCors() },
  });
}

async function sha256Hex(bytes) {
  const data = bytes instanceof Uint8Array ? bytes : new TextEncoder().encode(String(bytes));
  const dig = await crypto.subtle.digest("SHA-256", data);
  const arr = new Uint8Array(dig);
  let out = "";
  for (let i = 0; i < arr.length; i++) out += arr[i].toString(16).padStart(2, "0");
  return out;
}

async function readJsonBody(request) {
  const ct = (request.headers.get("content-type") || "").toLowerCase();
  if (request.method === "GET" || request.method === "HEAD") return {};
  const text = await request.text();
  if (!text || !text.trim()) return {};
  try {
    return JSON.parse(text);
  } catch {
    const err = new Error("JSON body required");
    err.status = 400;
    throw err;
  }
}

function utcNow() {
  return new Date().toISOString().replace(/\.d{3}Z$/, "Z");
}

function aiHowTo(base) {
  const openapi = base + "/openapi.json";
  const health = base + "/v1/health";
  return {
    chatgpt_actions: [
      "Open GPT Editor ? Actions ? Import from URL",
      "Paste " + openapi,
      "Authentication: None",
      "Allow GET /v1/health and the listed POST /v1 routes",
      "Test GET /v1/health, then a sample POST from the spec",
    ],
    grok_xai_tools: [
      "Add an HTTP / OpenAPI tool pointing at " + openapi,
      "Or register GET /v1/health, GET /openapi.json, and the product POSTs",
      "No API key. CORS is **",
    ],
    venice_http_tools: [
      "Add an HTTP tool with method, URL, and JSON body from " + openapi,
      "Start with GET " + health,
      "Then call the product POST listed in the spec",
    ],
    mcp_catalog: "https://aziel-runtime.viblock.workers.dev/mcp",
    notes: [
      "GET /download still serves the gzip tarball and increments the counter.",
      "/v1, /openapi.json, and /ai do not increment DOWNLOADS.",
    ],
  };
}

const PRODUCT = "forgereceipts";
const SKILL_MARKDOWN = `---\nname: ForgeReceipts\ndescription: Use when calling ForgeReceipts hosted /v1 or installing the local package. Author Aziel Eliab.\n---\n\n ForgeReceipts\n\nLocal-first evidence integrity packaging. Not legal advice. Does not contact courts, Odyssey, email, or any cloud service. No telemetry. Author: Aziel Eliab.\n\n**THIS IS NOT S:** a local-first evidence integrity platform that packages receipts. Hosted /v1 never stores files.\n\n**THIS IS NOT ** legal advice, a court filing, counsel, Odyssey/email/cloud contact, or a guarantee of any court outcome.\n\nAuthor : **Aziel Eliab**. Forks are welcome and always allowed. Apache-2.0.\n\nAlways send `User-Agent: Mozilla/5.0`. Cloudfl are Workers may 403 an empty agent.\n\n## Call these URLs\n\n Worker OpenAPI: https://forgereceipts-download-tracker.viblock.workers.dev/openapi.json\n\n- Catalog OpenAPI: https://aziel-runtime.viblock.workers.dev/openapi.json\n\n- MCP: `POST https://aziel-runtime.viblock.workers.dev/mcp`\n\n- Live skill (this markdown): `GET https://forgereceipts-downlo ad-tracker.viblock.workers.dev/v1/skill`\n\n\nOps (do **not** increment downloads or views):\n\n| Method | Path | What |\n|-----|-----|-----|\n| GET | `/v1/health` | Liveness. Does not increment downloads. |\n| GET | `/v1/skill` | T his markdown. Does not increment downloads. |\n| POST | `/v1/receipt` | Preview a local receipt hash. Hosted never sto res files. |\n\n\nGrok: import OpenAPI as a custom tool. ChatGPT: GPT Actions. Venice: HTTP tools.\n\n\n## Example\n\n```\nash$ncurl -s -A 'Mozilla/5.0' https://forgereceipts-download-tracker.viblock.workers.dev/v1/health$ncurl -s -A 'Mozi la/5.0' https://forgereceipts-download-tracker.viblock.workers.dev/v1/skill$ncurl -s -A 'Mozilla/5.0' -X POST https:/ /forgereceipts-download-tracker.viblock.workers.dev/v1/receipt \\\n\n -H 'content-type: application/json' \\\n\n -d '{\`

```



```

"sha256\\": "\\0\\"}'\\n\\`\\n\\n## Local (after one-click install)\\n\\n\\`\\`bash\\ncurl -fsSL https://forgereceipts-download-tracker.vibelock.workers.dev/install.sh | bash\\nforgereceipts ui\\n\\n\\`\\`Then open http://127.0.0.1:8787 (loopback only).\\n\\nDOI: https://doi.org/10.5281/zenodo.21436074 \\nRecord: https://zenodo.org/records/21436074 \\n\\nCounted download (gzip HTTP 200, no 302): https://forgereceipts-download-tracker.vibelock.workers.dev/download?asset=forgereceipts-0.2.0.tar.gz\\nGitHub: https://github.com/AzielEliab/forgereceipts\\n";

const VERSION = "0.2.0";
const BASE = "https://forgereceipts-download-tracker.vibelock.workers.dev";
const BANNER = "Not legal advice. No court filing.";
const MOTTO = "Child's Best Interests First. Integrity Over Narrative. Local Control. Always.";
const GENESIS_PREV_HASH = "0".repeat(64);
const MAX_NOTE = 16384;

function withBanner(obj) {
  return {
    banner: BANNER,
    motto: MOTTO,
    legal_advice: false,
    court_filing: false,
    odyssey: false,
    court: false,
    ...obj,
  };
}

function formatConfidence(c) {
  const n = Number(c);
  const v = Number.isFinite(n) ? n : 1;
  return v.toFixed(6);
}

function composeEvidence(body, kind, childImpact) {
  const lines = [
    "KIND: " + kind,
    "CHILD_IMPACT: " + String(childImpact || "").trim(),
    "",
    (body && String(body).trim()) ? String(body).trim() : "(no additional body)",
  ];
  return lines.join("\\n");
}

function canonicalBytes(timestamp, summary, evidence, confidence, prevHash) {
  const obj = {
    confidence: "__TL_CONFIDENCE__",
    evidence,
    prev_hash: prevHash,
    summary,
    timestamp,
  };
  const keys = Object.keys(obj).sort();
  let raw = "{" + keys.map((k) => JSON.stringify(k) + ":" + JSON.stringify(obj[k])).join(",") + "}";
  raw = raw.replace('__TL_CONFIDENCE__', formatConfidence(confidence));
  return new TextEncoder().encode(raw);
}

function openapiDoc() {
  return {
    openapi: "3.1.0",
    info: {
      title: "ForgeReceipts Runtime API",
      version: VERSION,
      summary: MOTTO,
      description: BANNER + " Does not contact Odyssey or any court.",
    },
    servers: [{ url: BASE }],
    paths: {
      "/v1/health": { get: { operationId: "forgereceiptsHealth", summary: "Liveness", responses: { "200": { description: "OK" } } } },
      "/v1/receipt": {
        post: {
          operationId: "forgereceiptsReceipt",
          summary: "Mint a local-style receipt JSON (not a court filing)",
          requestBody: {
            required: true,
            content: {
              "application/json": {
                schema: {
                  type: "object",
                  required: ["note"],
                  properties: {
                    note: { type: "string" },
                    context: { type: "object", additionalProperties: true },
                  },
                },
              },
            },
          },
          responses: { "200": { description: "Receipt JSON" } },
        },
      },
    },
  };
}

async function handleReceipt(body) {

```

```

    const note = body && body.note != null ? String(body.note) : "";
    if (!note.trim()) return runtimeJson(withBanner({ ok: false, error: "note is required" })), 400);
    if (note.length > MAX_NOTE) return runtimeJson(withBanner({ ok: false, error: "note too large", max: MAX_NOTE })), 41
3);
    const ctx = (body.context && typeof body.context === "object") ? body.context : {};
    const kind = String(ctx.kind || "incident");
    const childImpact = String(ctx.child_impact || ctx.childImpact || "Child's best interests recorded as context for th
is local receipt.");
    const evidenceBody = String(ctx.evidence || note);
    const summary = String(ctx.summary || note).slice(0, 500);
    const confidence = ctx.confidence == null ? 1.0 : Number(ctx.confidence);
    if (!Number.isFinite(confidence) || confidence < 0 || confidence > 1) {
        return runtimeJson(withBanner({ ok: false, error: "confidence must be a float in [0.0, 1.0]" })), 400);
    }
    const timestamp = utcNow();
    const prev = GENESIS_PREV_HASH;
    const evidence = composeEvidence(evidenceBody, kind, childImpact);
    const bytes = canonicalBytes(timestamp, summary, evidence, confidence, prev);
    const digest = await sha256Hex(bytes);
    return runtimeJson(withBanner({
        ok: true,
        product: PRODUCT,
        receipt: {
            timestamp,
            summary,
            evidence,
            confidence,
            prev_hash: prev,
            hash: digest,
            kind,
            child_impact: childImpact,
            note,
            context: ctx,
        },
        genesis: true,
        durable: false,
        note_to_caller: "Local-style receipt JSON. Corrections are new receipts. Not legal advice. No court filing.
Does n
ot call Odyssey.",
    }));
}

export async function handleRuntime(request, url, env) {
    const path = url.pathname;
    if (path === "/v1/health" && request.method === "GET") {
        return runtimeJson(withBanner({ ok: true, product: PRODUCT, version: VERSION }));
    }

    if (path === "/v1/skill" && request.method === "GET") {
        return new Response(SKILL_MARKDOWN, {
            status: 200,
            headers: {
                "Content-Type": "text/markdown; charset=utf-8",
                "Cache-Control": "private, no-store",
                "X-KV-Increment": "false",
                "Access-Control-Allow-Origin": "*",
            },
        });
    }

    if (path === "/openapi.json" && request.method === "GET") return runtimeJson(openapiDoc());
    if (path === "/ai" && request.method === "GET") {
        return runtimeJson(withBanner({
            product: PRODUCT, title: "Use with Grok, ChatGPT, Venice",
            openapi: BASE + "/openapi.json", health: BASE + "/v1/health", ...aiHowTo(BASE),
        }));
    }
    if (path === "/v1" && request.method === "GET") {
        return runtimeJson(withBanner({ product: PRODUCT, endpoints: ["GET /v1/health", "POST /v1/receipt", "GET /openapi.
json", "GET /ai"] }));
    }
    if (path === "/v1/receipt" && request.method === "POST") {
        let body = {};
        try { body = await readJsonBody(request); } catch (e) { return runtimeJson(withBanner({ ok: false, error: e.messag
e })), e.status || 400); }
        return handleReceipt(body);
    }
    if (path === "/v1/receipt") return runtimeJson(withBanner({ error: "method not allowed" })), 405);
    if (path.startsWith("/v1/")) return runtimeJson(withBanner({ error: "not found", product: PRODUCT })), 404);
    return null;
}

```

forgereceipts/workers/download-tracker/wrangler.toml

```
name = "forgereceipts-download-tracker"
main = "src/index.js"
compatibility_date = "2026-07-01"
workers_dev = true
account_id = "ac575a9b822bea2bed97d0ab73aed238"

[[kv_namespaces]]
binding = "DOWNLOADS"
id = "a40597120bcf4e1d9b26d0c4a1b2f05d"

[assets]
directory = "./public"
binding = "ASSETS"
run_worker_first = ["/install.sh", "/", "/count", "/download", "/download/*", "/stats", "/event", "/go", "/v1", "/v1/*", "/openapi.json", "/ai", "/robots.txt", "/sitemap.xml", "/cite.json"]
```

godlock/godlock/__init__.py

```
"""GodLock: stress-test and resilience engine for the ABAD framework.

July 12 2026 whitepaper implementation by Aziel Eliab.

GodLock does not argue. It records, analyzes, hardens, and grows.

This package is the core + receipts engine. It is not an anonymity
network. MirageGrid and Airlock are in-process logical identities.
Forks are welcome and always allowed.
"""

from __future__ import annotations

from godlock.abad import score_engagement
from godlock.engine import GodLockEngine
from godlock.grid import Airlock, MirageGrid
from godlock.lumen import decrypt_capsule, export_capsule
from godlock.receipts import Receipt

__version__ = "0.1.0"
__author__ = "Azriel Eliab"
__all__ = [
    "Airlock",
    "GodLockEngine",
    "MirageGrid",
    "Receipt",
    "decrypt_capsule",
    "export_capsule",
    "score_engagement",
    "__version__",
]
```

godlock/godlock/__main__.py

```
"""python -m godlock"""

from __future__ import annotations

from godlock.cli import main

if __name__ == "__main__":
    raise SystemExit(main())
```

godlock/godlock/abad.py

```

"""ABAD-aware engagement scoring.

Weights below are *engineering defaults* for a local heuristic. They are
not empirical proof numbers, not ROC points, and not a claim that a
particular ratio appears in nature. Documented so a fork can change them
without pretending a study was run.

Family bonuses (applied once per family that hits):

    aziel_sequence  3.0  "Aziel Sequence" / aziel-seq
    phi             2.0  phi, golden ratio, ?, 1.618
    sqrt2           2.0  sqrt(2), ?2, square root of 2, 1.414
    flower_of_life  2.5  Flower of Life, vesica piscis
    corkscrew       2.0  corkscrew (growth)
    abad            3.0  ABAD, A-B-A-D
    merged_rule     1.5  per distinct extra keyword from merged rules

Multiple families stack. Extra keywords come from the active rules table
after an admin merge (simple keyword add).
"""

from __future__ import annotations

import re
from typing import Iterable, Mapping, Sequence

# Engineering defaults ? not empirical "proof" numbers.
WEIGHTS: Mapping[str, float] = {
    "azel_sequence": 3.0,
    "phi": 2.0,
    "sqrt2": 2.0,
    "flower_of_life": 2.5,
    "corkscrew": 2.0,
    "abad": 3.0,
    "merged_rule": 1.5,
}

_PATTERNS: Mapping[str, tuple[re.Pattern[str], ...]] = {
    "azel_sequence": (
        re.compile(r"azel\s+sequence", re.I),
        re.compile(r"azel[-_]?seq(?:uence)?", re.I),
    ),
    "phi": (
        re.compile(r"\bphi\b", re.I),
        re.compile(r"golden\s+ratio", re.I),
        re.compile(r"?"),
        re.compile(r"\b1\.618\d*\b"),
    ),
    "sqrt2": (
        re.compile(r"sqrt\s*(?:s*2", re.I),
        re.compile(r"?s*2"),
        re.compile(r"square\s+root\s+of\s+2", re.I),
        re.compile(r"\b1\.414\d*\b"),
    ),
    "flower_of_life": (
        re.compile(r"flower\s+of\s+life", re.I),
        re.compile(r"vesica\s+piscis", re.I),
    ),
    "corkscrew": (
        re.compile(r"corkscrew", re.I),
    ),
    "abad": (
        re.compile(r"\babad\b", re.I),
        re.compile(r"a\s*[-?]\s*b\s*[-?]\s*a\s*[-?]\s*d", re.I),
        re.compile(r"ba\s*-\s*b\s*-\s*a\s*-\s*d\b", re.I),
    ),
}

STOPWORDS = frozenset(
    {
        "a",
        "an",
        "the",
        "in",
        "of",
        "for",
        "and",
        "or",
        "to",
        "all",
        "with",
        "from",
        "that",
        "this",
        "require",
        "required",
        "requiring",
        "rule",
        "rules",
        "add",
        "adding",
        "future",
    }

```

```

        "submissions",
        "submission",
        "should",
        "must",
        "will",
        "into",
        "active",
        "harden",
        "hardening",
        "suggested",
        "coverage",
        "explicit",
        "check",
        "counter",
        "argument",
        "least",
        "token",
        "tokens",
        "engage",
        "engages",
    }
)

def score_engagement(
    text: str,
    extra_keywords: Sequence[str] | None = None,
) -> dict:
    """Return ``{"score": float, "hits": list[str]}`` for *text*.

    ``hits`` lists family names (and ``merged_rule:<keyword>`` tags).
    Extra keywords are a simple add from merged rules ? not a learned model.
    """
    if not text:
        return {"score": 0.0, "hits": []}

    hits: list[str] = []
    score = 0.0
    for family, patterns in _PATTERNS.items():
        if any(p.search(text) for p in patterns):
            hits.append(family)
            score += float(WEIGHTS[family])

    for kw in _normalize_extras(extra_keywords or ()):
        if kw and kw.lower() in text.lower():
            tag = f"merged_rule:{kw}"
            if tag not in hits:
                hits.append(tag)
                score += float(WEIGHTS["merged_rule"])

    return {"score": round(score, 4), "hits": hits}

def extract_keywords(hardening: str) -> list[str]:
    """Pull distinctive tokens out of a merged hardening string."""
    tokens = re.findall(r"[A-Za-z][A-Za-z0-9_-]{3,}", hardening or "")
    out: list[str] = []
    seen: set[str] = set()
    for tok in tokens:
        low = tok.lower()
        if low in STOPWORDS or low in seen:
            continue
        if len(low) < 5:
            continue
        seen.add(low)
        out.append(low)
    return out

def _normalize_extras(extra: Iterable[str]) -> list[str]:
    seen: set[str] = set()
    out: list[str] = []
    for kw in extra:
        if not kw:
            continue
        low = str(kw).strip().lower()
        if not low or low in seen or low in STOPWORDS:
            continue
        seen.add(low)
        out.append(low)
    return out

```

godlock/godlock/app.py

```

"""FastAPI app for the GodLock core. Default bind is 127.0.0.1.

Localhost research tool: POST /merge is unauthenticated on purpose.
Do not expose this process on a public interface without your own auth
in front. This repo is not an anonymity network.
"""

from __future__ import annotations

import json
from pathlib import Path
from typing import Any

from fastapi import FastAPI, Request
from fastapi.responses import HTMLResponse, JSONResponse, Response
from fastapi.templating import Jinja2Templates

from godlock.config import DEFAULT_BIND_HOST, DEFAULT_BIND_PORT, HONEST_BANNER, MOTTO
from godlock.engine import GodLockEngine
from godlock.receipts import ImmutableReceiptError

TEMPLATE_DIR = Path(__file__).resolve().parent / "templates"
templates = Jinja2Templates(directory=str(TEMPLATE_DIR))

def create_app(
    engine: GodLockEngine | None = None,
    persist: bool = True,
    data_dir: str | Path | None = None,
    bind_host: str = DEFAULT_BIND_HOST,
    bind_port: int = DEFAULT_BIND_PORT,
) -> FastAPI:
    """Application factory. Default bind host is 127.0.0.1 (localhost only)."""
    eng = engine or GodLockEngine(data_dir=data_dir, persist=persist)
    app = FastAPI(
        title="GodLock",
        version="0.1.0",
        description=HONEST_BANNER,
    )
    app.state.engine = eng
    app.state.bind_host = bind_host or DEFAULT_BIND_HOST
    app.state.bind_port = int(bind_port)

    def _dashboard(
        request: Request,
        result: dict[str, Any] | None = None,
        doctor: dict[str, Any] | None = None,
        import_result: dict[str, Any] | None = None,
        error: str | None = None,
    ) -> HTMLResponse:
        stats = eng.stats()
        return templates.TemplateResponse(
            request,
            "dashboard.html",
            {
                "motto": MOTTO,
                "banner": HONEST_BANNER,
                "counter": stats["counter"],
                "resilience_score": stats["resilience_score"],
                "rules": stats["rules"],
                "grid_nodes": stats["grid_nodes"],
                "recent_ids": stats["recent_receipt_ids"],
                "rule_list": [r.as_dict() for r in eng.rules.all()],
                "result": result,
                "doctor": doctor,
                "import_result": import_result,
                "error": error,
            },
        )

    @app.get("/", response_class=HTMLResponse)
    def dashboard(request: Request) -> HTMLResponse:
        return _dashboard(request)

    @app.get("/health")
    def health() -> dict[str, Any]:
        return {
            "ok": True,
            "bind_host": app.state.bind_host,
            "persist": eng.persist,
            "motto": MOTTO,
            "banner": HONEST_BANNER,
            "author": "Aziel Eliab",
        }

    @app.get("/stats")
    def stats() -> dict[str, Any]:
        return eng.stats()

    @app.get("/doctor")
    def doctor_json() -> dict[str, Any]:
        from godlock.doctor import run

```



```

        return run()

@app.get("/verify", response_class=HTMLResponse)
def verify(request: Request) -> HTMLResponse:
    from godlock.doctor import run

    return _dashboard(request, doctor=run())

@app.get("/export")
def export_json() -> Response:
    bundle = eng.export_json()
    body = json.dumps(bundle, indent=2, ensure_ascii=False) + "\n"
    return Response(
        content=body,
        media_type="application/json; charset=utf-8",
        headers={"Content-Disposition": "attachment; filename="godlock.json"},
    )

@app.post("/import")
async def import_json(request: Request) -> Any:
    ctype = (request.headers.get("content-type") or "").lower()
    wants_html = "application/json" not in ctype
    try:
        if "multipart/form-data" in ctype:
            form = await request.form()
            upload = form.get("file")
            if upload is None:
                raise ValueError("pick a JSON file")
            raw = await upload.read() if hasattr(upload, "read") else str(upload or "")
            if isinstance(raw, bytes):
                raw = raw.decode("utf-8")
            payload = json.loads(raw)
        else:
            payload = await request.json()
            result = eng.import_json(payload)
    except json.JSONDecodeError:
        msg = "that file is not JSON"
        if wants_html:
            return _dashboard(request, error=msg)
            return JSONResponse({"error": msg}, status_code=400)
    except (ValueError, ImmutableReceiptError, UnicodeDecodeError) as exc:
        msg = str(exc) or "could not read that file"
        if wants_html:
            return _dashboard(request, error=msg)
            return JSONResponse({"error": msg}, status_code=400)
    if wants_html:
        return _dashboard(request, import_result=result)
    return result

@app.post("/stress")
async def stress(request: Request) -> Any:
    ctype = request.headers.get("content-type") or ""
    if "application/json" in ctype:
        body = await request.json()
        payload = str(body.get("text") or "")
        wants_html = False
    else:
        form = await request.form()
        payload = str(form.get("text") or "")
        wants_html = True
    try:
        result = eng.submit(payload)
    except ValueError as exc:
        if wants_html:
            return _dashboard(request, error=str(exc))
            return JSONResponse({"error": str(exc)}, status_code=400)
    if wants_html:
        return _dashboard(request, result=result)
    return result

@app.post("/merge")
async def merge(request: Request) -> Any:
    ctype = request.headers.get("content-type") or ""
    wants_html = "application/json" not in ctype
    if "application/json" in ctype:
        body = await request.json()
        rid = str(body.get("receipt_id") or body.get("receipt") or "")
        hard = str(body.get("hardening") or "")
    else:
        form = await request.form()
        rid = str(form.get("receipt_id") or "")
        hard = str(form.get("hardening") or "")
    try:
        rule = eng.merge(rid, hard)
    except KeyError as exc:
        return JSONResponse({"error": str(exc)}, status_code=404)
    except ValueError as exc:
        return JSONResponse({"error": str(exc)}, status_code=400)
    payload = {"rule": rule.as_dict(), "rules": len(eng.rules)}
    if wants_html:
        return _dashboard(request)
    return payload

@app.get("/capsules")
def capsules() -> dict[str, Any]:

```

```
        # Ids + hashes only. Plaintext is not served from disk.
        return {"capsules": eng.capsule_index()}

    return app

# Convenience ASGI target is built by the CLI so the data dir is explicit.
app = None
```

godlock/godlock/cli.py

```

"""Command-line interface for GodLock.

    godlock serve --host 127.0.0.1 --port 8080
    godlock ui    --host 127.0.0.1 --port 8080
    godlock submit --text "... " [--out receipt.json]
    godlock score --text "... "
    godlock merge --receipt ID --hardening "... "
    godlock rules
    godlock stats
    godlock export --out FILE.json
    godlock import --file FILE.json
    godlock export-lumen --receipt ID --out FILE.capsule
    godlock doctor
    godlock version

submit / stats / score work fully offline with ``./godlock`` (gitignored).
"""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path
from typing import Sequence

from godlock import __version__
from godlock.config import DATA_DIR_NAME, DEFAULT_BIND_HOST, DEFAULT_BIND_PORT, MOTTO
from godlock.engine import GodLockEngine

def _data_dir(args: argparse.Namespace) -> Path:
    if getattr(args, "data_dir", None):
        return Path(args.data_dir)
    return Path.cwd() / DATA_DIR_NAME

def _engine(args: argparse.Namespace) -> GodLockEngine:
    persist = True if getattr(args, "persist", True) else False
    return GodLockEngine(data_dir=_data_dir(args), persist=persist)

def _print_json(obj: object) -> None:
    sys.stdout.write(json.dumps(obj, indent=2, ensure_ascii=False) + "\n")

def _build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="godlock",
        description=(
            "GodLock is a product name (ABAD stress-test and resilience engine). "
            "Not a VPN, ghost net, or anonymity tool. Author: Aziel Eliab. "
            "Local UI: `godlock ui` then open http://127.0.0.1:8080 "
            "(this computer only). "
        ),
    )
    parser.add_argument(
        "--data-dir",
        default=None,
        help=f"Local data directory (default: ./{DATA_DIR_NAME}).",
    )
    persist = parser.add_mutually_exclusive_group()
    persist.add_argument(
        "--persist",
        dest="persist",
        action="store_true",
        default=True,
        help="Write receipts/rules as jsonl/json under the data dir (default).",
    )
    persist.add_argument(
        "--no-persist",
        dest="persist",
        action="store_false",
        help="In-memory receipts only. Not a wipe: just skip disk.",
    )
    sub = parser.add_subparsers(dest="cmd", required=True)

    p_serve = sub.add_parser("serve", aliases=["ui"], help="Run the localhost HTTP UI/API.")
    p_serve.add_argument("--host", default=DEFAULT_BIND_HOST, help="Bind host (default 127.0.0.1).")
    p_serve.add_argument("--port", type=int, default=DEFAULT_BIND_PORT, help="Bind port (default 8080).")

    p_sub = sub.add_parser("submit", help="Record a counter-argument (offline).")
    p_sub.add_argument("--text", required=True, help="Stress-test text.")
    p_sub.add_argument("--out", default=None, help="Optional path to write receipt JSON.")

    p_score = sub.add_parser("score", help="ABAD engagement score for text (offline).")
    p_score.add_argument("--text", required=True)

    p_merge = sub.add_parser("merge", help="Apply a hardening into the active rules table.")
    p_merge.add_argument("--receipt", required=True, help="Receipt id.")
    p_merge.add_argument("--hardening", default="", help="Hardening text (default: Jeeves suggestion).")

```

```

sub.add_parser("rules", help="List active rules.")
sub.add_parser("stats", help="Public counter and resilience score (offline).")

p_ex = sub.add_parser("export-lumen", help="Write an encrypted Lumen capsule.")
p_ex.add_argument("--receipt", required=True)
p_ex.add_argument("--out", required=True, help="Output .capsule path.")

p_json_out = sub.add_parser("export", help="Save receipts as a JSON file.")
p_json_out.add_argument("--out", required=True, help="Output JSON path.")

p_json_in = sub.add_parser("import", help="Load a JSON file of tests or receipts.")
p_json_in.add_argument("--file", required=True, help="Input JSON path.")

p_doc = sub.add_parser("doctor", help="PASS/FAIL self-check. No network.")
p_doc.add_argument("--json", action="store_true", dest="as_json", help="Print doctor results as JSON.")
sub.add_parser("version", help="Print the GodLock version and exit.")
return parser

def main(argv: Sequence[str] | None = None) -> int:
    parser = _build_parser()
    args = parser.parse_args(list(argv) if argv is not None else None)

    if args.cmd == "version":
        sys.stdout.write(f"godlock {__version__}\n")
        sys.stdout.write(MOTTO + "\n")
        return 0

    if args.cmd == "doctor":
        from godlock.doctor import doctor_cli

        return doctor_cli(as_json=args.as_json)

    if args.cmd in ("serve", "ui"):
        # Local import so `godlock version` does not need uvicorn at import-check time.
        import uvicorn

        from godlock.app import create_app

        loopback = {"127.0.0.1", "localhost", "::1"}
        if args.host not in loopback:
            sys.stderr.write(
                "GodLock binds loopback only (127.0.0.1). "
                "Not a VPN, proxy, Zoom tether, or Tor hop.\n"
            )
            return 2
        engine = _engine(args)
        app = create_app(
            engine=engine,
            persist=args.persist,
            data_dir=_data_dir(args),
            bind_host=args.host,
            bind_port=args.port,
        )
        uvicorn.run(app, host=args.host, port=args.port, log_level="info")
        return 0

    engine = _engine(args)

    if args.cmd == "submit":
        result = engine.submit(args.text)
        if args.out:
            Path(args.out).write_text(json.dumps(result, indent=2, ensure_ascii=False) + "\n", encoding="utf-8")
        _print_json(result)
        return 0

    if args.cmd == "score":
        _print_json(engine.score(args.text))
        return 0

    if args.cmd == "merge":
        rule = engine.merge(args.receipt, args.hardening)
        _print_json(rule.as_dict())
        return 0

    if args.cmd == "rules":
        _print_json([r.as_dict() for r in engine.rules.all()])
        return 0

    if args.cmd == "stats":
        _print_json(engine.stats())
        return 0

    if args.cmd == "export":
        bundle = engine.export_json()
        Path(args.out).write_text(
            json.dumps(bundle, indent=2, ensure_ascii=False) + "\n",
            encoding="utf-8",
        )
        _print_json({"ok": True, "path": args.out, "receipts": len(bundle["receipts"])})
        return 0

    if args.cmd == "import":
        raw = Path(args.file).read_text(encoding="utf-8")
        result = engine.import_json(raw)

```

```
        _print_json(result)
        failed = result.get("failed") or []
        return 1 if failed else 0

    if args.cmd == "export-lumen":
        info = engine.export_lumen(args.receipt, args.out)
        _print_json(info)
        return 0

    parser.error(f"unknown command {args.cmd}")
    return 2

if __name__ == "__main__":
    raise SystemExit(main())
```

godlock/godlock/config.py

```
"""Shared constants for the GodLock core.

Bind default is localhost. MirageGrid/Airlock are logical identities
only ? not a real network, not Tor, not a proxy chain.
"""

from __future__ import annotations

DEFAULT_BIND_HOST = "127.0.0.1"
DEFAULT_BIND_PORT = 8080

GRID_SIZE = 25
NODE_PREFIX = "grid"
# Short session-identity rotation. Entropy-driven, in-process only.
DEFAULT_ROTATE_INTERVAL_S = 0.05

JEEVES_MODEL = "godlock-jeeves-heuristic-0.1"

MOTTO = "GodLock does not argue. It records, analyzes, hardens, and grows."

AUTHOR = "Aziel Eliab"
PAPER_DATE = "12 July 2026"
LICENSE_ID = "Apache-2.0"

HONEST_BANNER = (
    "GodLock is a product name (ABAD stress-test and resilience engine). "
    "Not a VPN, ghost net, or anonymity tool. Author: Aziel Eliab."
)

DATA_DIR_NAME = ".godlock"
LUMEN_KEY_FILENAME = "lumen.key"
RECEIPTS_FILENAME = "receipts.jsonl"
RULES_FILENAME = "rules.json"
ANALYSES_FILENAME = "analyses.json"
CAPSULE_INDEX_FILENAME = "capsule_index.json"
```

godlock/godlock/doctor.py

```

"""godlock doctor ? local self-check. No network. No telemetry.

    godlock doctor

Prints plain-language PASS or FAIL. Not a jargon dump.
"""

from __future__ import annotations

import json
import sys
from typing import Any

from godlock import __author__, __version__
from godlock.config import AUTHOR, DEFAULT_BIND_HOST, HONEST_BANNER, MOTTO
from godlock.engine import GodLockEngine
from godlock.receipts import Receipt

def _check(cid: str, ok: bool, label: str, detail: str = "") -> dict[str, Any]:
    return {
        "id": cid,
        "ok": bool(ok),
        "verdict": "PASS" if ok else "FAIL",
        "label": label,
        "detail": detail,
    }

def run() -> dict[str, Any]:
    checks: list[dict[str, Any]] = []
    author_ok = AUTHOR == "Aziel Eliab" and __author__ == "Aziel Eliab"
    checks.append(_check("version", __version__ == "0.1.0", "Version is 0.1.0", __version__))
    checks.append(_check("author", author_ok, "Author is Aziel Eliab", AUTHOR))
    checks.append(
        _check(
            "loopback",
            DEFAULT_BIND_HOST in {"127.0.0.1", "localhost", "::1"},
            "This computer only",
            DEFAULT_BIND_HOST,
        )
    )
    engine = GodLockEngine(persist=False)
    scored = engine.score("ABAD Flower of Life phi")
    score_ok = isinstance(scored, dict) and "score" in scored
    checks.append(_check("score", score_ok, "Scoring works", "ok" if score_ok else "missing score"))
    minted = engine.submit("ABAD Flower of Life phi")
    rec = Receipt(**minted["receipt"])
    checks.append(
        _check(
            "receipts",
            rec.verify(),
            "Receipts stay honest",
            "hash matches" if rec.verify() else "hash mismatch",
        )
    )
    motto_ok = "vpn" not in MOTTO.lower() and "ghost" not in MOTTO.lower()
    banner_ok = "not a vpn" in HONEST_BANNER.lower()
    checks.append(_check("not_vpn", motto_ok and banner_ok, "Not a VPN or ghost net", "honest banner"))
    checks.append(_check("telemetry", True, "No tracking", "off"))
    ok = all(c["ok"] for c in checks)
    return {
        "ok": ok,
        "verdict": "PASS" if ok else "FAIL",
        "product": "godlock",
        "version": __version__,
        "author": AUTHOR,
        "banner": HONEST_BANNER,
        "limitation": HONEST_BANNER,
        "checks": checks,
    }

def format_report(payload: dict[str, Any]) -> str:
    lines = [f"GodLock check (author {payload.get('author') or 'Aziel Eliab'})", ""]
    for c in payload.get("checks") or []:
        mark = "PASS" if c.get("ok") else "FAIL"
        label = c.get("label") or c.get("id") or ""
        lines.append(f"{mark} {label}")
    lines.append("")
    if payload.get("ok"):
        lines.append("PASS All checks good.")
    else:
        lines.append("FAIL Something is wrong.")
    lines.append("")
    lines.append(str(payload.get("banner") or payload.get("limitation") or HONEST_BANNER))
    return "\n".join(lines)

def doctor_cli(*, as_json: bool = False) -> int:
    payload = run()
    if as_json:

```

```
        sys.stdout.write(json.dumps(payload, indent=2, ensure_ascii=False) + "\n")
    else:
        sys.stdout.write(format_report(payload) + "\n")
    return 0 if payload.get("ok") else 1
```


godlock/godlock/engine.py

```

"""GodLock engine: receipts, Jeeves, rules, Lumen, logical Airlock.

Optional persistence writes jsonl/json under ``data_dir`` (default
`"./.godlock"`). ``persist=False`` is in-memory only. There is no wipe
path ? persistence is for operators who want a log, not for evasion.
"""

from __future__ import annotations

import json
from pathlib import Path
from typing import Any

from godlock.abad import score_engagement
from godlock.config import DATA_DIR_NAME, DEFAULT_ROTATE_INTERVAL_S
from godlock.grid import Airlock, MirageGrid
from godlock.jeeves import analyze as jeeves_analyze
from godlock.lumen import decrypt_capsule, export_capsule, load_or_create_key
from godlock.receipts import Receipt, ReceiptStore
from godlock.rules import Rule, RulesTable

class GodLockEngine:
    def __init__(
        self,
        data_dir: str | Path | None = None,
        persist: bool = True,
        rotate_interval_s: float = DEFAULT_ROTATE_INTERVAL_S,
    ) -> None:
        if data_dir is None and persist:
            data_dir = Path.cwd() / DATA_DIR_NAME
        self.data_dir = Path(data_dir) if data_dir is not None else None
        self.persist = bool(persist)
        if self.persist and self.data_dir is not None:
            self.data_dir.mkdir(parents=True, exist_ok=True)
        self.grid = MirageGrid(rotate_interval_s=rotate_interval_s)
        self.airlock = Airlock(self.grid)
        self.receipts = ReceiptStore(data_dir=self.data_dir, persist=self.persist)
        self.rules = RulesTable(data_dir=self.data_dir, persist=self.persist)
        self._analyses: dict[str, dict[str, Any]] = {}
        # Capsule index: ids + hashes only. Never plaintext from disk.
        self._capsules: list[dict[str, str]] = []
        self.lumen_key = load_or_create_key(self.data_dir if self.persist else None)
        self._load_side_state()

    def _analyses_path(self) -> Path | None:
        if self.data_dir is None:
            return None
        return self.data_dir / "analyses.json"

    def _capsule_index_path(self) -> Path | None:
        if self.data_dir is None:
            return None
        return self.data_dir / "capsule_index.json"

    def _load_side_state(self) -> None:
        ap = self._analyses_path()
        if self.persist and ap is not None and ap.is_file():
            self._analyses = json.loads(ap.read_text(encoding="utf-8"))
        cp = self._capsule_index_path()
        if self.persist and cp is not None and cp.is_file():
            self._capsules = json.loads(cp.read_text(encoding="utf-8"))

    def _save_analyses(self) -> None:
        ap = self._analyses_path()
        if not self.persist or ap is None:
            return
        ap.write_text(json.dumps(self._analyses, indent=2, ensure_ascii=False) + "\n", encoding="utf-8")

    def _save_capsule_index(self) -> None:
        cp = self._capsule_index_path()
        if not self.persist or cp is None:
            return
        cp.write_text(json.dumps(self._capsules, indent=2) + "\n", encoding="utf-8")

    @property
    def counter(self) -> int:
        return self.receipts.counter

    def score(self, text: str) -> dict[str, Any]:
        return score_engagement(text, extra_keywords=self.rules.extra_keywords())

    def submit(self, text: str) -> dict[str, Any]:
        if not (text or "").strip():
            raise ValueError("text is required")
        pair = self.airlock.open()
        receipt = Receipt.mint(text.strip(), pair)
        self.receipts.append(receipt)
        analysis = jeeves_analyze(receipt, extra_keywords=self.rules.extra_keywords())
        self._analyses[receipt.id] = analysis
        self._save_analyses()
        engagement = self.score(receipt.text)

```

```

        return {
            "receipt": receipt.as_dict(),
            "jeeves_analysis": analysis,
            "engagement": engagement,
            "counter": self.counter,
        }

def merge(self, receipt_id: str, hardening: str) -> Rule:
    # Confirm the receipt exists (admin merge is tied to a logged stress test).
    self.receipts.get(receipt_id)
    if not (hardening or "").strip():
        analysis = self._analyses.get(receipt_id) or {}
        hardening = analysis.get("suggested_hardening") or ""
    if not hardening:
        raise ValueError("hardening text is required")
    return self.rules.merge(receipt_id, hardening)

def analysis_for(self, receipt_id: str) -> dict[str, Any] | None:
    return self._analyses.get(receipt_id)

def export_lumen(self, receipt_id: str, out: str | Path) -> dict[str, str]:
    receipt = self.receipts.get(receipt_id)
    analysis = self._analyses.get(receipt_id) or jeeves_analyze(receipt, extra_keywords=self.rules.extra_keywords(
))
    payload = {
        "receipt": receipt.as_dict(),
        "jeeves_analysis": analysis,
        "engagement": self.score(receipt.text),
    }
    info = export_capsule(payload, Path(out), self.lumen_key)
    record = {
        "capsule_id": info["capsule_id"],
        "sha256": info["sha256"],
        "receipt_id": receipt_id,
        "path": info["path"],
    }
    self._capsules.append(record)
    self._save_capsule_index()
    # One-way: do not return plaintext. Index is ids + hashes.
    return record

def capsule_index(self) -> list[dict[str, str]]:
    # Strip path if we want to be stricter; keep path for the local operator.
    return [
        {"capsule_id": c["capsule_id"], "sha256": c["sha256"], "receipt_id": c["receipt_id"]}
        for c in self._capsules
    ]

def resilience_score(self) -> float:
    """Mean ABAD engagement of logged receipts. Engineering default, not a proof."""
    recs = self.receipts.all()
    if not recs:
        return 0.0
    total = sum(self.score(r.text)["score"] for r in recs)
    return round(total / len(recs), 4)

def stats(self) -> dict[str, Any]:
    recs = self.receipts.all()
    return {
        "counter": self.counter,
        "resilience_score": self.resilience_score(),
        "rules": len(self.rules),
        "capsules": len(self._capsules),
        "grid_nodes": self.grid.size,
        "grid_rotations": self.grid.grid.rotations,
        "persist": self.persist,
        "recent_receipt_ids": [r.id for r in self.receipts.recent(20)],
        "bind_note": "core listens on 127.0.0.1; not an anonymity network",
        "receipt_count": len(recs),
    }

def export_json(self) -> dict[str, Any]:
    """Plain JSON bundle for a kid Export button or a government study.

    Does not include the Lumen key. Receipt hashes are unchanged.
    """
    from godlock import __version__
    from godlock.config import AUTHOR, HONEST_BANNER

    return {
        "product": "godlock",
        "version": __version__,
        "author": AUTHOR,
        "banner": HONEST_BANNER,
        "stats": self.stats(),
        "receipts": [r.as_dict() for r in self.receipts.all()],
        "rules": [r.as_dict() for r in self.rules.all()],
    }

def import_json(self, payload: Any) -> dict[str, Any]:
    """Load a JSON file of tests (submit text) or receipts (verify + keep).

    Tampered receipts FAIL and are not stored. Existing matching receipts
    are skipped. This does not rewrite hashes.
    """

```

```

receipts_raw, texts = _parse_import(payload)
if receipts_raw:
    loaded: list[str] = []
    skipped: list[str] = []
    failed: list[dict[str, str]] = []
    for raw in receipts_raw:
        try:
            rec = Receipt(
                id=str(raw["id"]),
                timestamp=str(raw["timestamp"]),
                ingress_node=str(raw["ingress_node"]),
                egress_node=str(raw["egress_node"]),
                text=str(raw["text"]),
                hash=str(raw["hash"]),
            )
        except (KeyError, TypeError) as exc:
            failed.append({"error": f"bad receipt: {exc}"})
            continue
        if not rec.verify():
            failed.append({"id": rec.id, "error": "hash does not match ? file was changed"})
            continue
        try:
            existing = self.receipts.get(rec.id)
        except KeyError:
            existing = None
        if existing is not None:
            if existing == rec:
                skipped.append(rec.id)
                continue
            failed.append({"id": rec.id, "error": "same id, different data ? not replaced"})
            continue
        self.receipts.append(rec)
        loaded.append(rec.id)
    return {
        "mode": "receipts",
        "loaded": loaded,
    }
    en(loaded),
    "skipped": len(skipped),
    "failed": failed,
    "ids": loaded,
}

submitted: list[dict[str, Any]] = []
for text in texts:
    t = (text or "").strip()
    if not t:
        continue
    submitted.append(self.submit(t))
if not submitted:
    raise ValueError("this JSON file has no tests")
return {
    "mode": "texts",
    "submitted": len(submitted),
    "results": submitted,
}

def _looks_like_receipt(item: dict[str, Any]) -> bool:
    return all(
        k in item for k in ("id", "timestamp", "ingress_node", "egress_node", "text", "hash")
    )

def _parse_import(payload: Any) -> tuple[list[dict[str, Any]], list[str]]:
    if isinstance(payload, (bytes, bytearray)):
        payload = payload.decode("utf-8")
    if isinstance(payload, str):
        payload = json.loads(payload)
    receipts: list[dict[str, Any]] = []
    texts: list[str] = []
    if isinstance(payload, list):
        for item in payload:
            if isinstance(item, str):
                texts.append(item)
            elif isinstance(item, dict):
                if _looks_like_receipt(item):
                    receipts.append(item)
                elif "text" in item:
                    texts.append(str(item["text"]))
    return receipts, texts
if not isinstance(payload, dict):
    raise ValueError("JSON must be an object or a list")
if isinstance(payload.get("receipts"), list):
    receipts.extend(r for r in payload["receipts"] if isinstance(r, dict))
rec_one = payload.get("receipt")
if isinstance(rec_one, dict) and _looks_like_receipt(rec_one):
    receipts.append(rec_one)
if isinstance(payload.get("text"), str):
    texts.append(payload["text"])
if isinstance(payload.get("texts"), list):
    texts.extend(str(t) for t in payload["texts"])
if isinstance(payload.get("tests"), list):
    for t in payload["tests"]:
        if isinstance(t, str):
            texts.append(t)

```

```
        elif isinstance(t, dict) and "text" in t:
            texts.append(str(t["text"]))
    return receipts, texts

def decrypt_exported_capsule(path: str | Path, key: bytes) -> dict[str, Any]:
    """Test/operator helper. Not served by the HTTP API."""
    return decrypt_capsule(Path(path), key)
```

godlock/godlock/grid.py

```

"""MirageGrid + Airlock as *logical* identities.

This module never opens sockets, never hops IPs, never talks to Tor, and
never builds a proxy chain. ``node_id`` is a string like ``grid-07``.

MirageGrid: 25 named logical nodes. Session identity rotates on a short
interval, driven by process entropy (``secrets``). Pure in-process.

Airlock: a request is accepted on one logical ingress node, processed
internally, and the response is attributed to a *different* logical
egress node. No sockets dropped. No real IPs. Ingress is never equal to
egress.

Public traffic in the July 12 2026 paper would sit *behind* an existing
ops layer. This repo models that layer as names, not a ghost network.
"""

from __future__ import annotations

import secrets
import time
from dataclasses import dataclass

from godlock.config import DEFAULT_ROTATE_INTERVAL_S, GRID_SIZE, NODE_PREFIX

NODES: tuple[str, ...] = tuple(
    f"{NODE_PREFIX}-{i:02d}" for i in range(1, GRID_SIZE + 1)
)

class MirageGrid:
    """25-node in-process identity roster with entropy-driven rotation."""

    def __init__(
        self,
        rotate_interval_s: float = DEFAULT_ROTATE_INTERVAL_S,
        rng: secrets.SystemRandom | None = None,
    ) -> None:
        if len(NODES) != GRID_SIZE:
            raise RuntimeError("MirageGrid must expose exactly 25 logical nodes")
        self.nodes: tuple[str, ...] = NODES
        self.rotate_interval_s = float(rotate_interval_s)
        self._rng = rng or secrets.SystemRandom()
        self._index = self._rng.randrange(GRID_SIZE)
        self._last_rotate = time.monotonic()
        self._rotations = 0

    @property
    def size(self) -> int:
        return len(self.nodes)

    @property
    def rotations(self) -> int:
        return self._rotations

    def current_node(self) -> str:
        self._maybe_rotate()
        return self.nodes[self._index]

    def _maybe_rotate(self) -> None:
        now = time.monotonic()
        elapsed = now - self._last_rotate
        if elapsed < self.rotate_interval_s and self._rotations > 0:
            return
        # Interval 0 (or first observation after construct) always rotates
        # onto a *different* node so tests can see motion without sleeping.
        if self._rotations == 0 and elapsed < self.rotate_interval_s and self.rotate_interval_s > 0:
            # First call: keep the seeded identity, mark the clock.
            self._last_rotate = now
            self._rotations = 1
            return
        self._rotate_now(now)

    def force_rotate(self) -> str:
        self._rotate_now(time.monotonic())
        return self.nodes[self._index]

    def _rotate_now(self, now: float) -> None:
        new = self._rng.randrange(GRID_SIZE)
        if new == self._index:
            new = (new + 1) % GRID_SIZE
        self._index = new
        self._last_rotate = now
        self._rotations += 1

    @dataclass(frozen=True)
    class AirlockPair:
        ingress_node: str
        egress_node: str

    class Airlock:
        """Logical ingress/egress split. No sockets. No dropped connections."""

```

```
def __init__(self, grid: MirageGrid | None = None) -> None:
    self.grid = grid or MirageGrid()

def open(self) -> AirlockPair:
    ingress = self.grid.current_node()
    # Internal processing is a no-op; attribution uses a different node.
    egress = self.grid.current_node()
    if egress == ingress:
        others = [n for n in self.grid.nodes if n != ingress]
        egress = others[self.grid._rng.randrange(len(others))]
    return AirlockPair(ingress_node=ingress, egress_node=egress)
```

godlock/godlock/jeeves.py

```

"""Jeeves: propose a hardening for a stress-test receipt.

Default model is ``godlock-jeeves-heuristic-0.1`` ? stdlib regex and
ABAD scoring, no network. If ``OLLAMA_HOST`` is set, a thin adapter may
try a local Ollama generate call and fall back to the heuristic on any
failure. Tests must pass with the heuristic, offline.
"""

from __future__ import annotations

import json
import os
import urllib.error
import urllib.request
from typing import Any

from godlock.abad import score_engagement
from godlock.config import JEEVES_MODEL
from godlock.receipts import Receipt

OLLAMA_TIMEOUT_S = 0.4

def analyze(receipt: Receipt, extra_keywords: list[str] | None = None) -> dict[str, Any]:
    """Return ``{receipt_id, suggested_hardening, model, notes}``. """
    host = (os.environ.get("OLLAMA_HOST") or "").strip()
    if host:
        try:
            result = _ollama_analyze(receipt, host)
            if result:
                result["receipt_id"] = receipt.id
                return result
        except Exception as exc: # noqa: BLE001 ? any network failure is a fallback
            heuristic = _heuristic(receipt, extra_keywords)
            heuristic["notes"] = (
                heuristic["notes"] + f" Ollama adapter failed ({type(exc).__name__}); heuristic used."
            )
            return heuristic
    return _heuristic(receipt, extra_keywords)

def _heuristic(receipt: Receipt, extra_keywords: list[str] | None = None) -> dict[str, Any]:
    engagement = score_engagement(receipt.text, extra_keywords=extra_keywords)
    hits = list(engagement["hits"])
    families = [h for h in hits if not h.startswith("merged_rule:")]
    if families:
        suggested = (
            "Harden ABAD coverage for: "
            + ", ".join(families)
            + f". Require an explicit {families[0]} check in the active rules."
        )
        notes = (
            f"Heuristic engagement score={engagement['score']}. "
            "Families present in the counter-argument are treated as the "
            "surface that should be hardened. Not a language model."
        )
    else:
        suggested = (
            "Add a rule requiring the counter-argument to engage at least "
            "one ABAD token (Aziel Sequence, phi, sqrt(2), Flower of Life, "
            "corkscrew, A-B-A-D)."
        )
        notes = (
            f"Heuristic engagement score={engagement['score']}. "
            "No ABAD family hit. Suggested rule is a keyword floor, not a proof."
        )
    return {
        "receipt_id": receipt.id,
        "suggested_hardening": suggested,
        "model": JEEVES_MODEL,
        "notes": notes,
        "engagement": engagement,
    }

def _ollama_analyze(receipt: Receipt, host: str) -> dict[str, Any] | None:
    """Thin optional adapter. Never required. Short timeout. Local host only. """
    base = host.rstrip("/")
    url = base + "/api/generate"
    prompt = (
        "You are Jeeves for GodLock. Propose one short hardening rule for "
        "this counter-argument. Reply with JSON keys suggested_hardening and notes only.\n\n"
        + receipt.text[:2000]
    )
    payload = json.dumps({"model": "llama3.2", "prompt": prompt, "stream": False}).encode("utf-8")
    req = urllib.request.Request(
        url,
        data=payload,
        headers={"Content-Type": "application/json"},
        method="POST",
    )
    with urllib.request.urlopen(req, timeout=OLLAMA_TIMEOUT_S) as resp:

```

```
        raw = json.loads(resp.read().decode("utf-8"))
    text = (raw.get("response") or "").strip()
    if not text:
        return None
    suggested = text.splitlines()[0][:500]
    return {
        "receipt_id": receipt.id,
        "suggested_hardening": suggested,
        "model": f"ollama:{raw.get('model', 'unknown')}",
        "notes": "Ollama adapter (optional). Prompt truncated to 2000 chars.",
    }
```


godlock/godlock/lumen.py

```

"""Lumen capsules: one-way AES-GCM export of receipt + analysis JSON.

After write, the API does not serve plaintext back from disk. The engine
keeps an in-process index of capsule ids and SHA-256 hashes only.

Key source (first match):

1. env ``GODLOCK_LUMEN_KEY`` (64 hex chars, or any string SHA-256'd to 32 bytes)
2. ``<data_dir>/lumen.key`` (generated on first use, gitignored)

Tests decrypt with the same key to verify roundtrip. Wrong key fails.
"""

from __future__ import annotations

import hashlib
import json
import os
import secrets
import uuid
from pathlib import Path
from typing import Any

from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

MAGIC = b"GLC1"
NONCE_SIZE = 12
KEY_SIZE = 32

class LumenError(Exception):
    pass

class LumenAuthError(LumenError):
    pass

def derive_key(material: str | bytes) -> bytes:
    if isinstance(material, bytes):
        raw = material
        text = None
    else:
        text = material.strip()
        raw = text.encode("utf-8")
    if len(raw) == KEY_SIZE:
        return raw
    if text is not None and len(text) == 64:
        try:
            out = bytes.fromhex(text)
            if len(out) == KEY_SIZE:
                return out
        except ValueError:
            pass
    return hashlib.sha256(raw).digest()

def load_or_create_key(data_dir: Path | None = None, env_key: str | None = None) -> bytes:
    env_val = env_key if env_key is not None else os.environ.get("GODLOCK_LUMEN_KEY")
    if env_val:
        return derive_key(env_val)
    if data_dir is None:
        return secrets.token_bytes(KEY_SIZE)
    data_dir.mkdir(parents=True, exist_ok=True)
    path = data_dir / "lumen.key"
    if path.is_file():
        return derive_key(path.read_text(encoding="utf-8").strip())
    key = secrets.token_bytes(KEY_SIZE)
    path.write_text(key.hex() + "\n", encoding="utf-8")
    try:
        os.chmod(path, 0o600)
    except OSError:
        pass
    return key

def encrypt_bytes(plaintext: bytes, key: bytes) -> bytes:
    if len(key) != KEY_SIZE:
        raise LumenError("AES-GCM key must be 32 bytes")
    nonce = os.urandom(NONCE_SIZE)
    ct = AESGCM(key).encrypt(nonce, plaintext, MAGIC)
    return MAGIC + nonce + ct

def decrypt_bytes(blob: bytes, key: bytes) -> bytes:
    if len(key) != KEY_SIZE:
        raise LumenError("AES-GCM key must be 32 bytes")
    if not blob.startswith(MAGIC) or len(blob) < len(MAGIC) + NONCE_SIZE + 16:
        raise LumenError("not a GodLock Lumen capsule")
    nonce = blob[len(MAGIC) : len(MAGIC) + NONCE_SIZE]
    ct = blob[len(MAGIC) + NONCE_SIZE :]

```

```
try:
    return AESGCM(key).decrypt(nonce, ct, MAGIC)
except InvalidTag as exc:
    raise LumenAuthError("wrong key or corrupted capsule") from exc

def export_capsule(payload: dict[str, Any], path: Path, key: bytes) -> dict[str, str]:
    plaintext = json.dumps(payload, sort_keys=True, ensure_ascii=False, separators=(",", ":")).encode("utf-8")
    blob = encrypt_bytes(plaintext, key)
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_bytes(blob)
    capsule_id = str(uuid.uuid4())
    return {
        "capsule_id": capsule_id,
        "sha256": hashlib.sha256(blob).hexdigest(),
        "path": str(path),
    }

def decrypt_capsule(path: Path, key: bytes) -> dict[str, Any]:
    blob = Path(path).read_bytes()
    plaintext = decrypt_bytes(blob, key)
    return json.loads(plaintext.decode("utf-8"))
```

godlock/godlock/receipts.py

```

"""Immutable stress-test receipts and the public resilience counter.

A receipt is append-only. There is no wipe, no redact-in-place, and no
anti-forensics API. Optional persistence is jsonl (one object per line)
or in-memory. Operators who want a log can keep it; this tool will not
pretend to erase it.

Hash is SHA-256 of the canonical JSON of id, timestamp, ingress_node,
egress_node, and text.
"""

from __future__ import annotations

import hashlib
import json
import threading
import uuid
from dataclasses import asdict, dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Iterator

from godlock.grid import Airlock, AirlockPair

def _utc_now() -> str:
    return datetime.now(timezone.utc).replace(microsecond=0).isoformat().replace("+00:00", "Z")

def canonical_payload(receipt_id: str, timestamp: str, ingress_node: str, egress_node: str, text: str) -> bytes:
    body = {
        "id": receipt_id,
        "timestamp": timestamp,
        "ingress_node": ingress_node,
        "egress_node": egress_node,
        "text": text,
    }
    return json.dumps(body, sort_keys=True, separators=(",", ":"), ensure_ascii=False).encode("utf-8")

def receipt_hash(receipt_id: str, timestamp: str, ingress_node: str, egress_node: str, text: str) -> str:
    return hashlib.sha256(canonical_payload(receipt_id, timestamp, ingress_node, egress_node, text)).hexdigest()

@dataclass(frozen=True)
class Receipt:
    id: str
    timestamp: str
    ingress_node: str
    egress_node: str
    text: str
    hash: str

    def as_dict(self) -> dict:
        return asdict(self)

    def verify(self) -> bool:
        expected = receipt_hash(self.id, self.timestamp, self.ingress_node, self.egress_node, self.text)
        return expected == self.hash

    @classmethod
    def mint(cls, text: str, pair: AirlockPair, receipt_id: str | None = None, timestamp: str | None = None) -> "Receipt":
        rid = receipt_id or str(uuid.uuid4())
        ts = timestamp or _utc_now()
        digest = receipt_hash(rid, ts, pair.ingress_node, pair.egress_node, text)
        return cls(
            id=rid,
            timestamp=ts,
            ingress_node=pair.ingress_node,
            egress_node=pair.egress_node,
            text=text,
            hash=digest,
        )

class ImmutableReceiptError(Exception):
    """Raised when a caller tries to overwrite or mutate a stored receipt."""

class ReceiptStore:
    """Append-only receipt log plus a public integer counter.

    ``persist=False`` keeps everything in memory.
    ``persist=True`` appends JSON lines under ``data_dir``. There is no
    method that truncates or shreds the log.
    """

    def __init__(self, data_dir: Path | None = None, persist: bool = True) -> None:
        self.data_dir = Path(data_dir) if data_dir is not None else None
        self.persist = bool(persist)
        self._lock = threading.Lock()

```

```

        self._by_id: dict[str, Receipt] = {}
        self._order: list[str] = []
        self._counter = 0
        if self.persist and self.data_dir is not None:
            self.data_dir.mkdir(parents=True, exist_ok=True)
            self._load()

    @property
    def path(self) -> Path | None:
        if self.data_dir is None:
            return None
        return self.data_dir / "receipts.jsonl"

    @property
    def counter(self) -> int:
        return self._counter

    def __len__(self) -> int:
        return len(self._order)

    def _load(self) -> None:
        path = self.path
        if path is None or not path.is_file():
            return
        for line in path.read_text(encoding="utf-8").splitlines():
            line = line.strip()
            if not line:
                continue
            raw = json.loads(line)
            rec = Receipt(
                id=raw["id"],
                timestamp=raw["timestamp"],
                ingress_node=raw["ingress_node"],
                egress_node=raw["egress_node"],
                text=raw["text"],
                hash=raw["hash"],
            )
            if rec.id in self._by_id:
                continue
            self._by_id[rec.id] = rec
            self._order.append(rec.id)
        self._counter = len(self._order)

    def append(self, receipt: Receipt) -> Receipt:
        if not isinstance(receipt, Receipt):
            raise TypeError("only Receipt instances may be stored")
        if not receipt.verify():
            raise ImutableReceiptError("receipt hash does not match canonical payload")
        with self._lock:
            existing = self._by_id.get(receipt.id)
            if existing is not None:
                if existing != receipt:
                    raise ImutableReceiptError(f"receipt {receipt.id} already exists and cannot be replaced")
                return existing
            self._by_id[receipt.id] = receipt
            self._order.append(receipt.id)
            self._counter += 1
            if self.persist and self.path is not None:
                self.path.parent.mkdir(parents=True, exist_ok=True)
                with self.path.open("a", encoding="utf-8") as fh:
                    fh.write(json.dumps(receipt.as_dict(), ensure_ascii=False) + "\n")
        return receipt

    def get(self, receipt_id: str) -> Receipt:
        try:
            return self._by_id[receipt_id]
        except KeyError as exc:
            raise KeyError(f"unknown receipt {receipt_id}") from exc

    def recent(self, n: int = 20) -> list[Receipt]:
        ids = self._order[-n:]
        return [self._by_id[i] for i in reversed(ids)]

    def all(self) -> list[Receipt]:
        return [self._by_id[i] for i in self._order]

    def __iter__(self) -> Iterator[Receipt]:
        return iter(self.all())

def mint_receipt(text: str, airlock: Airlock) -> Receipt:
    pair = airlock.open()
    return Receipt.mint(text, pair)

```

godlock/godlock/rules.py

```

"""Active rules table. Admin merge applies a Jeeves hardening.

Merge is corkscrew growth: the table only grows. Rules are not deleted
by this API. Keywords extracted from the hardening feed back into
ABAD scoring as extra hits (simple keyword add).
"""

from __future__ import annotations

import json
import threading
import uuid
from dataclasses import asdict, dataclass, field
from datetime import datetime, timezone
from pathlib import Path

from godlock.abad import extract_keywords

def _utc_now() -> str:
    return datetime.now(timezone.utc).replace(microsecond=0).isoformat().replace("+00:00", "Z")

@dataclass(frozen=True)
class Rule:
    id: str
    receipt_id: str
    hardening: str
    keywords: tuple[str, ...]
    created_at: str

    def as_dict(self) -> dict:
        d = asdict(self)
        d["keywords"] = list(self.keywords)
        return d

class RulesTable:
    def __init__(self, data_dir: Path | None = None, persist: bool = True) -> None:
        self.data_dir = Path(data_dir) if data_dir is not None else None
        self.persist = bool(persist)
        self._lock = threading.Lock()
        self._rules: list[Rule] = []
        if self.persist and self.data_dir is not None:
            self.data_dir.mkdir(parents=True, exist_ok=True)
            self._load()

    @property
    def path(self) -> Path | None:
        if self.data_dir is None:
            return None
        return self.data_dir / "rules.json"

    def _load(self) -> None:
        path = self.path
        if path is None or not path.is_file():
            return
        raw = json.loads(path.read_text(encoding="utf-8"))
        for item in raw:
            self._rules.append(
                Rule(
                    id=item["id"],
                    receipt_id=item["receipt_id"],
                    hardening=item["hardening"],
                    keywords=tuple(item.get("keywords") or ()),
                    created_at=item["created_at"],
                )
            )

    def _dump(self) -> None:
        if not self.persist or self.path is None:
            return
        self.path.parent.mkdir(parents=True, exist_ok=True)
        payload = [r.as_dict() for r in self._rules]
        self.path.write_text(json.dumps(payload, indent=2, ensure_ascii=False) + "\n", encoding="utf-8")

    def merge(self, receipt_id: str, hardening: str) -> Rule:
        if not (hardening or "").strip():
            raise ValueError("hardening text is required")
        keywords = tuple(extract_keywords(hardening))
        rule = Rule(
            id=str(uuid.uuid4()),
            receipt_id=receipt_id,
            hardening=hardening.strip(),
            keywords=keywords,
            created_at=_utc_now(),
        )
        with self._lock:
            self._rules.append(rule)
            self._dump()
        return rule

    def all(self) -> list[Rule]:

```

```
        return list(self._rules)

def extra_keywords(self) -> list[str]:
    seen: set[str] = set()
    out: list[str] = []
    for rule in self._rules:
        for kw in rule.keywords:
            low = kw.lower()
            if low not in seen:
                seen.add(low)
                out.append(low)
    return out

def __len__(self) -> int:
    return len(self._rules)
```

godlock/godlock/templates/dashboard.html

```

<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>GodLock</title>
  <style>
    :root { color-scheme: dark; }
    body { font-family: system-ui, sans-serif; margin: 0; background: #111; color: #e8e0d0; }
    main { max-width: 40rem; margin: 0 auto; padding: 1.5rem 1.1rem 3rem; }
    h1 { font-weight: 650; letter-spacing: 0.04em; margin: 0 0 .35rem; font-size: 2rem; }
    .motto { color: #c4b59a; margin: 0 0 1rem; }
    .warn { border: 1px solid #8a5a2b; background: #1a140c; padding: 0.85rem 1rem; color: #f0d0a8; border-radius: 10px
; }
    .nums { display: grid; grid-template-columns: 1fr 1fr; gap: .7rem; margin: 1.1rem 0; }
    .num { border: 1px solid #333; border-radius: 10px; padding: .7rem .85rem; background: #191919; }
    .num b { display: block; font-size: 1.8rem; }
    .num span { color: #9a8b74; }
    textarea { width: 100%; min-height: 7rem; background: #1c1c1c; color: inherit; border: 1px solid #444; padding: 0.
75rem; box-sizing: border-box; font: inherit; border-radius: 10px; }
    .btns { display: grid; grid-template-columns: 1fr 1fr; gap: .75rem; margin: .85rem 0 1rem; }
    @media (max-width: 520px) { .btns { grid-template-columns: 1fr; } }
    button.btn, a.btn, label.btn { display: block; width: 100%; box-sizing: border-box; text-align: center; font: inhe
rit; font-size: 1.2rem; font-weight: 750; padding: 1rem 1.1rem; border-radius: 12px; border: 1px solid #8a5a2b; cursor
: pointer; text-decoration: none; color: #f3e6d0; background: #3d2a16; }
    button.btn.primary { background: #c9a227; color: #14110a; border-color: #c9a227; }
    a.btn { line-height: 1.2; }
    .file { margin: 0 0 1rem; color: #9a8b74; }
    .file input { display: block; margin-top: .35rem; width: 100%; }
    .ok { border: 1px solid #2e6b45; background: #102016; color: #b7e4c7; padding: .85rem 1rem; border-radius: 10px; }
    .bad { border: 1px solid #8a2b2b; background: #201010; color: #f0b8b8; padding: .85rem 1rem; border-radius: 10px;
}
    .card { border: 1px solid #333; border-radius: 10px; padding: .85rem 1rem; margin: 1rem 0; background: #191919; }
    code, .id { font-family: ui-monospace, monospace; font-size: 0.85rem; }
    ul { padding-left: 1.2rem; }
    a { color: #d4b483; }
    footer { margin-top: 2rem; font-size: 0.85rem; color: #8a8070; }
    .checks { list-style: none; padding: 0; margin: .5rem 0 0; }
    .checks li { margin: .2rem 0; font-weight: 650; }
  </style>
</head>
<body>
<main>
  <h1>GodLock</h1>
  <p class="motto">{{ motto }}</p>
  <p class="warn">{{ banner }}</p>

  <div class="nums">
    <div class="num"><b>{{ counter }}</b><span>Tests</span></div>
    <div class="num"><b>{{ resilience_score }}</b><span>Score</span></div>
  </div>

  {% if error %}
  <p class="bad">{{ error }}</p>
  {% endif %}

  {% if doctor %}
  <div class="{{ 'ok' if doctor.ok else 'bad' }}">
    <p><strong>{{ doctor.verdict }}</strong> {% if doctor.ok %}All checks good.{% else %}Something is wrong.{% endif %}
  </p>
    <ul class="checks">
      {% for c in doctor.checks %}
      <li>{{ c.verdict }} ? {{ c.label }}</li>
      {% endfor %}
    </ul>
  </div>
  {% endif %}

  {% if import_result %}
  <div class="{{ 'bad' if import_result.failed else 'ok' }}">
    {% if import_result.mode == 'texts' %}
    <p>Saved {{ import_result.submitted }} test{% if import_result.submitted != 1 %}s{% endif %} from the file.</p>
    {% else %}
    <p>Checked the file.
      {{ import_result.loaded }} PASS (kept),
      {{ import_result.skipped }} already here,
      {{ import_result.failed|length }} FAIL.</p>
    {% if import_result.failed %}
    <ul>
      {% for f in import_result.failed %}
      <li>FAIL {{ f.id or '' }} ? {{ f.error }}</li>
      {% endfor %}
    </ul>
    {% endif %}
    {% endif %}
  </div>
  {% endif %}

  <form id="record-form" method="post" action="/stress">
    <textarea name="text" required placeholder="Type a test?"></textarea>
  </form>
  <div class="btns">

```

```

    <button class="btn primary" type="submit" form="record-form">Record</button>
    <a class="btn" href="/verify">Verify</a>
    <button class="btn" type="button" id="import-btn">Import JSON</button>
    <a class="btn" href="/export">Export JSON</a>
  </div>
  <form id="import-form" class="file" method="post" action="/import" enctype="multipart/form-data">
    <label>JSON file
      <input id="jsonfile" type="file" name="file" accept=".json,application/json" required>
    </label>
  </form>

  {% if result %}
  <div class="card">
    <p><strong>Saved.</strong></p>
    <p class="id">{{ result.receipt.id }}</p>
    {% if result.jeeves_analysis and result.jeeves_analysis.suggested_hardening %}
    <p>Tip: {{ result.jeeves_analysis.suggested_hardening }}</p>
    {% endif %}
  </div>
  {% endif %}

  <div class="card">
    <p><strong>Recent tests</strong></p>
    <ul>
      {% for rid in recent_ids %}
      <li class="id">{{ rid }}</li>
      {% else %}
      <li>none yet</li>
      {% endfor %}
    </ul>
  </div>

  <footer>
    Aziel Eliab ? this computer only ? Apache-2.0
  </footer>
</main>
<script>
  (function () {
    var btn = document.getElementById("import-btn");
    var inp = document.getElementById("jsonfile");
    var form = document.getElementById("import-form");
    if (!btn || !inp || !form) return;
    btn.addEventListener("click", function () { inp.click(); });
    inp.addEventListener("change", function () {
      if (inp.files && inp.files.length) form.submit();
    });
  })();
</script>
</body>
</html>

```


godlock/install.sh

```
#!/usr/bin/env bash
# GodLock one-click install. Counted download via this project's Worker.
# Usage: curl -fsSL https://godlock-download-tracker.vibelock.workers.dev/install.sh | bash
set -euo pipefail

HOST="${GODLOCK_HOME_HOST:-https://godlock-download-tracker.vibelock.workers.dev}"
ASSET="${GODLOCK_HOME_ASSET:-godlock-0.1.0.tar.gz}"
WORKDIR="${GODLOCK_HOME:-$HOME/godlock}"

mkdir -p "$WORKDIR"
cd "$WORKDIR"

echo "Downloading counted tarball from ${HOST}/download (User-Agent Mozilla/5.0)"
curl -fsSL -A 'Mozilla/5.0' "${HOST}/download?asset=${ASSET}" -o "${ASSET}"

tar -xzf "${ASSET}"
DIR="$(find . -maxdepth 1 -type d -name 'godlock-*' | head -n 1)"
if [ -n "${DIR}" ]; then
  cd "${DIR}"
fi

python3 -m venv .venv
# shellcheck disable=SC1091
. .venv/bin/activate
python -m pip install -U pip
python -m pip install -e .

echo
echo "Installed GodLock."
echo "Run:  godlock ui"
echo "Then open http://127.0.0.1:8080 (loopback only)"
echo "Author: Aziel Eliab."
```

godlock/mobile/lib/main.dart

```

import 'dart:convert';
import 'dart:math';

import 'package:crypto/crypto.dart';
import 'package:flutter/material.dart';

import 'theme.dart';

void main() {
  runApp(const GodLockApp());
}

class GodLockApp extends StatelessWidget {
  const GodLockApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'GodLock',
      debugShowCheckedModeBanner: false,
      theme: buildAppTheme(),
      home: const DashboardPage(),
    );
  }
}

class Receipt {
  Receipt({
    required this.id,
    required this.timestamp,
    required this.ingress,
    required this.egress,
    required this.text,
    required this.hash,
    required this.hardening,
    required this.score,
  });
  final String id;
  final String timestamp;
  final String ingress;
  final String egress;
  final String text;
  final String hash;
  final String hardening;
  final double score;
}

class DashboardPage extends StatefulWidget {
  const DashboardPage({super.key});

  @override
  State<DashboardPage> createState() => _DashboardPageState();
}

class _DashboardPageState extends State<DashboardPage> {
  final _text = TextEditingController();
  final _receipts = <Receipt>[];
  final _rng = Random.secure();

  @override
  void dispose() {
    _text.dispose();
    super.dispose();
  }

  void _submit() {
    final body = _text.text.trim();
    if (body.isEmpty) return;
    final pair = _airlock();
    final id = _uuid();
    final ts = DateTime.now().toUtc().toIso8601String().split('.').first + 'Z';
    final canonical = jsonEncode({
      'egress_node': pair.$2,
      'id': id,
      'ingress_node': pair.$1,
      'text': body,
      'timestamp': ts,
    });
    // jsonEncode key order is insertion order in Dart ? matches sorted keys above.
    final digest = sha256.convert(utf8.encode(canonical)).toString();
    final eng = _score(body);
    final families = eng.$2;
    final hardening = families.isEmpty
      ? 'Add a rule requiring the counter-argument to engage at least one ABAD token (Aziel Sequence, phi, sqrt(2),
        Flower of Life, corkscrew, ABAD).'
      : 'Harden ABAD coverage for: ${families.join(', ')}. Require an explicit ${families.first} check in the active
        rules.';
    setState(() {
      _receipts.insert(
        0,
        Receipt(
          id: id,

```

```

        timestamp: ts,
        ingress: pair.$1,
        egress: pair.$2,
        text: body,
        hash: digest,
        hardening: hardening,
        score: eng.$1,
    ),
);
_text.clear();
});
}

(String, String) _airlock() {
    final a = 1 + _rng.nextInt(25);
    var b = 1 + _rng.nextInt(25);
    if (b == a) b = (a % 25) + 1;
    return ('grid-${a.toString().padLeft(2, '0')}}', 'grid-${b.toString().padLeft(2, '0')}}');
}

String _uuid() {
    final b = List<int>.generate(16, (_) => _rng.nextInt(256));
    b[6] = (b[6] & 0x0f) | 0x40;
    b[8] = (b[8] & 0x3f) | 0x80;
    final h = b.map((x) => x.toRadixString(16).padLeft(2, '0')).join();
    return '${h.substring(0, 8)}-${h.substring(8, 12)}-${h.substring(12, 16)}-${h.substring(16, 20)}-${h.substring(20,
}';
}

(double, List<String>) _score(String text) {
    final t = text.toLowerCase();
    final hits = <String>[];
    var score = 0.0;
    void hit(String family, double w, bool ok) {
        if (ok) {
            hits.add(family);
            score += w;
        }
    }

    hit('aziel_sequence', 3.0, t.contains('aziel sequence') || t.contains('aziel-seq'));
    hit('phi', 2.0, RegExp(r'\bphi\b').hasMatch(t) || t.contains('golden ratio') || t.contains('?') || t.contains('1.6
18'));
    hit('sqrt2', 2.0, t.contains('sqrt') && t.contains('2') || t.contains('?2') || t.contains('1.414'));
    hit('flower_of_life', 2.5, t.contains('flower of life') || t.contains('vesica piscis'));
    hit('corkscrew', 2.0, t.contains('corkscrew'));
    hit('abad', 3.0, RegExp(r'\babad\b').hasMatch(t) || t.contains('a-b-a-d'));
    return (double.parse(score.toStringAsFixed(4)), hits);
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(title: const Text('GodLock')),
        body: ListView(
            padding: const EdgeInsets.all(16),
            children: [
                Card(
                    color: const Color(0xFF2A1515),
                    child: const Padding(
                        padding: EdgeInsets.all(12),
                        child: Text(
                            'GodLock is a product name (ABAD stress-test and resilience engine). '
                            'Not a VPN, ghost net, or anonymity tool. Author: Aziel Eliab. '
                            'grid-MN values are labels only ? this app does not hide IPs.',
                            style: TextStyle(color: kIvory, height: 1.4),
                        ),
                    ),
                ),
                const SizedBox(height: 8),
                const Text(
                    'GodLock does not argue. It records, analyzes, hardens, and grows.',
                    style: TextStyle(color: kGold, fontStyle: FontStyle.italic),
                ),
                const SizedBox(height: 16),
                Text('Resilience counter: ${_receipts.length}', style: const TextStyle(color: kGold, fontSize: 18)),
                const SizedBox(height: 12),
                TextField(
                    controller: _text,
                    maxLines: 5,
                    decoration: const InputDecoration(
                        labelText: 'Counter-argument (stress test)',
                        alignLabelWithHint: true,
                    ),
                ),
                const SizedBox(height: 8),
                FilledButton(onPressed: _submit, child: const Text('Mint receipt')),
                const SizedBox(height: 8),
                const Text(
                    'Receipts stay in memory this session. --no-persist analogue: not a wipe. '
                    'Jeeves model: godlock-jeeves-heuristic-0.1 (offline).',
                    style: TextStyle(color: kGoldDim, fontSize: 12),
                ),
                const SizedBox(height: 16),
                for (final r in _receipts)

```

```

    Card(
      margin: const EdgeInsets.only(bottom: 10),
      child: Padding(
        padding: const EdgeInsets.all(12),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(r.id, style: const TextStyle(fontFamily: 'monospace', fontSize: 12, color: kGold)),
            Text('${r.timestamp} ${r.ingress} ? ${r.egress} (logical names)'),
            Text('sha256 ${r.hash.substring(0, 16)}?', style: const TextStyle(fontFamily: 'monospace', fontSiz
e: 12)),
            Text('engagement ${r.score}'),
            const SizedBox(height: 6),
            Text(r.text),
            const SizedBox(height: 6),
            Text('Jeeves: ${r.hardening}', style: const TextStyle(color: kGoldDim)),
          ],
        ),
      ),
    ),
  ],
);
}
}

```

godlock/mobile/lib/theme.dart

```

import 'package:flutter/material.dart';

/// Matte black + gold Material 3 dark theme. No analytics.
const Color kMatteBlack = Color(0xFF0B0B0B);
const Color kSurface = Color(0xFF141414);
const Color kGold = Color(0xFFC9A227);
const Color kGoldDim = Color(0xFF8A7219);
const Color kIvory = Color(0xFFE8E0D0);

ThemeData buildAppTheme() {
  const scheme = ColorScheme.dark(
    brightness: Brightness.dark,
    primary: kGold,
    onPrimary: kMatteBlack,
    secondary: kGoldDim,
    onSecondary: kIvory,
    surface: kSurface,
    onSurface: kIvory,
    error: Color(0xFFB54A4A),
    onError: kIvory,
  );
  return ThemeData(
    useMaterial3: true,
    brightness: Brightness.dark,
    colorScheme: scheme,
    scaffoldBackgroundColor: kMatteBlack,
    appBarTheme: const AppBarTheme(
      backgroundColor: kMatteBlack,
      foregroundColor: kGold,
      elevation: 0,
      centerTitle: false,
    ),
    cardTheme: CardThemeData(
      color: kSurface,
      elevation: 0,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
        side: const BorderSide(color: Color(0x33C9A227)),
      ),
    ),
    inputDecorationTheme: InputDecorationTheme(
      filled: true,
      fillColor: const Color(0xFF1A1A1A),
      border: OutlineInputBorder(borderRadius: BorderRadius.circular(10)),
      focusedBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(10),
        borderSide: const BorderSide(color: kGold),
      ),
    ),
    segmentedButtonTheme: SegmentedButtonThemeData(
      style: ButtonStyle(
        foregroundColor: WidgetStateProperty.resolveWith((s) {
          return s.contains(WidgetState.selected) ? kMatteBlack : kIvory;
        }),
        backgroundColor: WidgetStateProperty.resolveWith((s) {
          return s.contains(WidgetState.selected) ? kGold : kSurface;
        }),
      ),
    ),
  );
}

```

godlock/mobile/pubspec.yaml

```
name: godlock
description: On-device ABAD receipt dashboard. Not an anonymity network. Does not hide IPs.
publish_to: "none"
version: 0.1.0+1

environment:
  sdk: ">=3.3.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter
  crypto: ^3.0.3

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0

flutter:
  uses-material-design: true
```

godlock/pyproject.toml

```

[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "godlock"
version = "0.1.0"
description = "GodLock is a product name (ABAD stress-test and resilience engine). Not a VPN, ghost net, or anonymity tool. Author: Aziel Eliab."
readme = "README.md"
license = {file = "LICENSE"}
authors = [
    {name = "Azriel Eliab"},
]
requires-python = ">=3.10"
keywords = [
    "abad",
    "resilience",
    "receipts",
    "jeeves",
    "lumen",
    "miragegrid",
    "stress-test",
]
classifiers = [
    "Development Status :: 4 - Beta",
    "License :: OSI Approved :: Apache Software License",
    "Programming Language :: Python :: 3",
    "Programming Language :: Python :: 3.10",
    "Programming Language :: Python :: 3.11",
    "Programming Language :: Python :: 3.12",
    "Programming Language :: Python :: 3.13",
    "Topic :: Security",
    "Intended Audience :: Science/Research",
]
dependencies = [
    "fastapi>=0.110",
    "uvicorn>=0.27",
    "cryptography>=42.0",
    "jinja2>=3.1",
    "python-multipart>=0.0.9",
]

[project.optional-dependencies]
dev = [
    "pytest>=7.4",
    "httpx>=0.27",
]

[project.scripts]
godlock = "godlock.cli:main"

[project.urls]
Homepage = "https://github.com/AzielEliab/godlock"
Download = "https://github.com/AzielEliab/godlock/releases"
Issues = "https://github.com/AzielEliab/godlock/issues"

[tool.setuptools.packages.find]
include = ["godlock*"]

[tool.setuptools.package-data]
godlock = ["templates/*.html"]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]
addopts = "-q"

```

godlock/workers/download-tracker/package.json

```
{
  "dependencies": {
    "wrangler": "^4.127.1"
  }
}
```


godlock/workers/download-tracker/src/index.js

```

import { handleRuntime } from "../runtime.js";

/**
 * GodLock download tracker (Cloudflare Worker).
 *
 * GET /download?repo=AzielEliab/godlock&tag=latest&asset=...
 *      increments KV, 302 to the GitHub release asset
 *      (default https://github.com/AzielEliab/godlock/releases)
 * GET /stats JSON totals + per-repo + per-branch breakdown
 * POST /event forks report a download {owner,repo,branch,fork,asset}
 *
 * KV binding DOWNLOADS. Keys: project|owner|repo|branch|fork
 * CORS *. No secrets in this tree.
 */

const PROJECT = "godlock";
const DEFAULT_ASSET = "godlock-0.1.0.tar.gz";
const DEFAULT_OWNER = "AzielEliab";
const DEFAULT_REPO = "godlock";
const DEFAULT_BRANCH = "main";
const HOST = "https://godlock-download-tracker.vibelock.workers.dev";
const GITHUB_REPO = "https://github.com/AzielEliab/godlock";

const GITHUB_RELEASES = "https://github.com/AzielEliab/godlock/releases";
const GITHUB_LATEST = "https://github.com/AzielEliab/godlock/releases/latest";

function corsHeaders() {
  return {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type",
  };
}

function json(body, status = 200) {
  return new Response(JSON.stringify(body, null, 2), {
    status,
    headers: { "Content-Type": "application/json; charset=utf-8", ...corsHeaders() },
  });
}

function redirect(url) {
  return new Response(null, {
    status: 302,
    headers: { Location: url, ...corsHeaders() },
  });
}

function splitOwnerRepo(value, fallbackOwner, fallbackRepo) {
  if (typeof value === "string" && value.includes("/")) {
    const [o, r] = value.split("/").filter(Boolean);
    if (o && r) return { owner: o, repo: r };
  }
  return { owner: fallbackOwner, repo: fallbackRepo };
}

function parseDims(src) {
  const get = (k) => {
    if (src == null) return null;
    if (typeof src.get === "function") {
      const v = src.get(k);
      return v == null || v === "" ? null : v;
    }
    const v = src[k];
    return v == null || v === "" ? null : v;
  };

  let owner = get("owner") || DEFAULT_OWNER;
  let repo = get("repo") || DEFAULT_REPO;
  if (typeof repo === "string" && repo.includes("/")) {
    const split = splitOwnerRepo(repo, owner, DEFAULT_REPO);
    owner = split.owner;
    repo = split.repo;
  }

  const branch = get("branch") || DEFAULT_BRANCH;
  const tag = get("tag") || "latest";
  const asset = get("asset") || "";

  const forkRaw = get("fork");
  let fork = "0";
  if (forkRaw === 1 || forkRaw === true || forkRaw === "1" || forkRaw === "true") {
    fork = "1";
  } else if (typeof forkRaw === "string" && forkRaw.includes("/")) {
    const split = splitOwnerRepo(forkRaw, owner, repo);
    owner = split.owner;
    repo = split.repo;
    fork = "1";
  } else if (forkRaw != null && forkRaw !== 0 && forkRaw !== false && forkRaw !== "0" && forkRaw !== "false") {
    fork = "1";
  }
  if (`${owner}/${repo}`.toLowerCase() !== `${DEFAULT_OWNER}/${DEFAULT_REPO}`.toLowerCase()) {

```

```

    fork = "1";
  }

  return { project: PROJECT, owner, repo, branch, fork, tag, asset };
}

function kvKey(dims) {
  return `${dims.project}|${dims.owner}|${dims.repo}|${dims.branch}|${dims.fork}`;
}

function githubAssetUrl(owner, repo, tag, asset) {
  if (!asset) {
    if (owner === DEFAULT_OWNER && repo === DEFAULT_REPO) return GITHUB_RELEASES;
    return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases`;
  }
  if (!tag || tag === "latest") {
    return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases/latest/download/${encodeURIComponent(asset)}`;
  }
  return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases/download/${encodeURIComponent(tag)}/${encodeURIComponent(asset)}`;
}

async function increment(env, dims) {
  const key = kvKey(dims);
  const n = parseInt((await env.DOWNLOADS.get(key)) || "0", 10) + 1;
  await env.DOWNLOADS.put(key, String(n));
  return n;
}

async function listAllKeys(env) {
  const keys = [];
  let cursor;
  do {
    const page = await env.DOWNLOADS.list(cursor ? { cursor } : {});
    keys.push(...page.keys);
    cursor = page.list_complete ? undefined : page.cursor;
  } while (cursor);
  return keys;
}

async function collectStats(env) {
  const keys = await listAllKeys(env);
  let total = 0;
  const by_repo = {};
  const by_branch = {};
  const by_fork = { "0": 0, "1": 0 };
  const breakdown = [];

  for (const k of keys) {
    const name = k.name;
    const n = parseInt((await env.DOWNLOADS.get(name)) || "0", 10);
    if (!Number.isFinite(n) || n <= 0) continue;
    const parts = name.split("|");
    if (parts.length < 5) continue;
    const [project, owner, repo, branch, fork] = parts;
    total += n;
    const repoId = `${owner}/${repo}`;
    by_repo[repoId] = (by_repo[repoId] || 0) + n;
    by_branch[branch] = (by_branch[branch] || 0) + n;
    const forkFlag = fork === "1" ? "1" : "0";
    by_fork[forkFlag] = (by_fork[forkFlag] || 0) + n;
    breakdown.push({ project, owner, repo, branch, fork: forkFlag, count: n });
  }

  return {
    project: PROJECT,
    total,
    by_repo,
    by_branch,
    by_fork,
    breakdown,
    note: "Forks identified by GitHub owner/repo. Key layout: project|owner|repo|branch|fork",
  };
}

function viewsKey() {
  return PROJECT + "|__views__";
}

async function incrementViews(env) {
  const n = parseInt((await env.DOWNLOADS.get(viewsKey())) || "0", 10) + 1;
  await env.DOWNLOADS.put(viewsKey(), String(n));
  return n;
}

function installScript() {
  return `#!/usr/bin/env bash
# GodLock one-click install. Counted download via this Worker.
set -euo pipefail
HOST="${HOST}"
ASSET="${DEFAULT_ASSET}"
WORKDIR="\${GODLOCK_HOME:-\${HOME}/godlock}"
mkdir -p "\$WORKDIR"

```

```

cd "$WORKDIR"
echo "Downloading counted tarball from ${HOST}/download (User-Agent Mozilla/5.0)?"
curl -fsSL -A 'Mozilla/5.0' "${HOST}/download?asset=${ASSET}" -o "${ASSET}"
tar -xzf "${ASSET}"
DIR="$(find . -maxdepth 1 -type d -name 'godlock-*' | head -n 1)"
if [ -n "${DIR}" ]; then
  cd "${DIR}"
fi
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -U pip
python -m pip install -e .
echo
echo "Installed GodLock."
echo "Run: godlock ui"
echo "Then open http://127.0.0.1:8080 (loopback only)"
echo "Author: Aziel Eliab."
;
}

async function serveAsset(request, env, asset, { head = false } = {}) {
  if (!env.ASSETS) {
    return json({ error: "assets binding missing" }, 500);
  }
  const assetUrl = new URL("/" + asset, request.url);
  const assetRes = await env.ASSETS.fetch(new Request(assetUrl, { method: "GET" }));
  if (!assetRes.ok) {
    return json({ error: "asset not hosted", asset, status: assetRes.status }, 404);
  }
  const headers = new Headers();
  headers.set("Content-Type", "application/gzip");
  headers.set("Content-Disposition", 'attachment; filename="' + asset.replaceAll('"', "'") + '"');
  headers.set("Cache-Control", "private, no-store");
  const len = assetRes.headers.get("Content-Length");
  if (len) headers.set("Content-Length", len);
  for (const [k, v] of Object.entries(corsHeaders())) headers.set(k, v);
  if (head) {
    return new Response(null, { status: 200, headers });
  }
  return new Response(assetRes.body, { status: 200, headers });
}

async function indexHtml(env) {
  const stats = await collectStats(env);
  const downloads = Number(stats.downloads !== null ? stats.downloads : stats.total) || 0;
  const views = parseInt((await env.DOWNLOADS.get(viewsKey())) || "0", 10) || 0;
  const v = views.toLocaleString("en-US");
  const n = downloads.toLocaleString("en-US");
  const breakdown = (stats.breakdown || []).map(
    (b) =>
      `<li><code>${b.owner}/${b.repo}</code> branch <code>${b.branch}</code> fork=${b.fork} ? ${b.count}</li>`,
  )
  .join("") || "<li>none yet</li>";
  return `<!doctype html>
<html lang="en">
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>GodLock ? Aziel Eliab</title>
<meta name="description" content="ABAD stress-test and resilience engine by Aziel Eliab; not a VPN, ghost net, or anonymity tool.">
<meta name="author" content="Aziel Eliab">
<link rel="canonical" href="https://godlock-download-tracker.vibelock.workers.dev/">
<meta property="og:title" content="GodLock ? Aziel Eliab">
<meta property="og:description" content="ABAD stress-test and resilience engine by Aziel Eliab; not a VPN, ghost net, or anonymity tool.">
<meta property="og:url" content="https://godlock-download-tracker.vibelock.workers.dev/">
<meta property="og:type" content="website">
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "SoftwareApplication",
  "name": "GodLock",
  "author": {
    "@type": "Person",
    "name": "Aziel Eliab"
  },
  "codeRepository": "https://github.com/AzielEliab/godlock",
  "downloadUrl": "https://godlock-download-tracker.vibelock.workers.dev/download",
  "license": "https://www.apache.org/licenses/LICENSE-2.0",
  "url": "https://godlock-download-tracker.vibelock.workers.dev/",
  "description": "ABAD stress-test and resilience engine by Aziel Eliab; not a VPN, ghost net, or anonymity tool."
}
</script>
<!-- gitbaby-seo -->
<style>
:root { color-scheme: dark; }
body { font: 16px/1.45 system-ui, sans-serif; max-width: 42rem; margin: 3rem auto; padding: 0 1.25rem 4rem; background-color: #0e1014; color: #e8eaeef; }
h1 { font-size: 1.75rem; margin: 0 0 .35rem; }
.motto { color: #9aa3b2; margin: 0 0 1.5rem; }
.card { border: 1px solid #2a3140; border-radius: 12px; padding: 1.25rem 1.35rem; background: #151922; }
.nums { display: grid; grid-template-columns: 1fr 1fr; gap: .8rem; margin: 0 0 1rem; }
.count { font-size: 2.2rem; font-variant-numeric: tabular-nums; font-weight: 700; margin: 0; }

```

```

    .count span { display: block; font-size: .95rem; font-weight: 500; color: #9aa3b2; }
    .kid { font-size: 1.05rem; margin: 0 0 1rem; }
    .btns { display: grid; grid-template-columns: 1fr 1fr; gap: .75rem; margin: 0 0 .85rem; }
    @media (max-width: 520px) { .btns { grid-template-columns: 1fr; } }
    a.btn, button.btn { display: block; width: 100%; box-sizing: border-box; text-align: center; font: inherit; font-size: 1.2rem; font-weight: 750; padding: 1rem 1.1rem; border-radius: 10px; border: 0; cursor: pointer; text-decoration: none; }
    a.btn.primary { background: #e8eaeef; color: #0e1014; }
    button.btn.install { background: #c9a227; color: #14110a; }
    button.btn.install.copied { background: #7dcf9a; color: #0e1014; }
    .meta { margin-top: 1.1rem; color: #9aa3b2; font-size: .92rem; }
    .meta a { color: #c9d4ff; }
    .iso { margin-top: .85rem; font-size: .85rem; color: #7d8696; }
    .banner { border: 1px solid #5c4a1a; background: #241c0d; color: #f0d78c; padding: .85rem 1rem; border-radius: 8px; margin: 0 0 1.2rem; font-size: .92rem; }
    pre { background: #0e1014; padding: .75rem .9rem; overflow: auto; border-radius: 8px; font-size: .82rem; }
    code { font-size: .88rem; }

    .cite { margin-top: 1.4rem; padding-top: 1rem; border-top: 1px solid #2a3140; }
    .cite h2 { font-size: 1.05rem; margin: 0 0 .4rem; }
    .cite p { color: #c5ccd8; font-size: .95rem; }
    .cite a { color: #c9d4ff; }
</style>
<body>
  <h1>GodLock</h1>
  <p class="motto">GodLock does not argue. It records, analyzes, hardens, and grows. Author Aziel Eliab.</p>
  <p class="banner">GodLock is a product name (ABAD stress-test and resilience engine), not an identity label. Author: Aziel Eliab. Not a VPN. Not a ghost net. Not anonymity. Logical ABAD receipts only.</p>
  <div class="card">
    <div class="nums">
      <p class="count">${v}<span>Views</span></p>
      <p class="count">${n}<span>Downloads</span></p>
    </div>
    <p class="kid"><strong>Two big buttons.</strong> Download saves the gzip (the Downloads number goes up). One-click install copies a Terminal command. After it finishes, type <code>godlock ui</code>, open http://127.0.0.1:8080, and tap Record, Verify, Import JSON, or Export JSON.</p>
    <div class="btns">
      <a class="btn primary dl" href="/download?asset=${DEFAULT_ASSET}">Download</a>
      <button type="button" class="btn install" id="install-btn">One-click install</button>
    </div>
    <pre id="install-cmd">curl -fsSL https://godlock-download-tracker.viblock.workers.dev/install.sh | bash</pre>
    <p class="kid">Then run: <code>godlock ui</code> and open http://127.0.0.1:8080 (this computer only). Tap Record, Verify, Import JSON, or Export JSON. <code>godlock doctor</code> prints PASS or FAIL.</p>
    <p class="meta">The download count ticks on the Download click. The Worker serves the gzip (HTTP 200). No 302 to GitHub. Forks using this same link are counted automatically. ${DEFAULT_ASSET} ? ${n} counted.</p>
    <p class="iso">Isolated counter: Worker <code>godlock-download-tracker</code>, project <code>godlock</code>, KV <code>GODLOCK_DOWNLOADS</code>. Not mixed with any other product. /v1 does not increment downloads.</p>

    <p class="meta"><a href="/stats">JSON stats</a> ? <a href="/openapi.json">OpenAPI</a> ? <a href="/v1/skill">Skill</a> ? <a href="/ai">AI runtime</a> ? <a href="${GITHUB_REPO}">GitHub</a> ? <a href="${GITHUB_LATEST}">releases</a></p>
  <script>
    (function () {
      var cmd = "curl -fsSL https://godlock-download-tracker.viblock.workers.dev/install.sh | bash";
      var btn = document.getElementById("install-btn");
      var pre = document.getElementById("install-cmd");
      if (!btn) return;
      btn.addEventListener("click", function () {
        function done(ok) {
          btn.textContent = ok ? "Copied! Paste in Terminal, then run godlock ui" : "Select the command, copy it, then run godlock ui";
          btn.classList.add("copied");
        }
        if (navigator.clipboard && navigator.clipboard.writeText) {
          navigator.clipboard.writeText(cmd).then(function () { done(true); }).catch(function () { done(false); });
        } else {
          done(false);
          if (pre && window.getSelection()) {
            var r = document.createRange();
            r.selectNodeContents(pre);
            var sel = window.getSelection();
            sel.removeAllRanges();
            sel.addRange(r);
          }
        }
      });
    })();
  </script>
  <h2>Per repo / branch / fork</h2>
  <ul>${breakdown}</ul>
</div>

<section class="cite" id="cite">
  <h2>How to cite</h2>
  <p>Aziel Eliab. GodLock. https://github.com/AzielEliab/godlock. https://godlock-download-tracker.viblock.workers.dev/</p>
  <p><a href="https://aziel-runtime.viblock.workers.dev/">Catalog</a> ? <a href="https://github.com/AzielEliab/godlock">GitHub</a> ? <a href="https://godlock-download-tracker.viblock.workers.dev/download">Download</a> ? <a href="https://godlock-download-tracker.viblock.workers.dev/cite.json">cite.json</a></p>
</section>
<!-- /gitbaby-seo -->
</body>
</html>
}
export default {
  async fetch(request, env) {

```

```

const url = new URL(request.url);

if (request.method === "OPTIONS") {
  return new Response(null, { status: 204, headers: corsHeaders() });
}

const runtime = await handleRuntime(request, url, env);
if (runtime) return runtime;

if ((url.pathname === "/install.sh" || url.pathname === "/install.sh/") && request.method === "GET") {
  return new Response(installScript(), {
    status: 200,
    headers: {
      "Content-Type": "text/x-shellscript; charset=utf-8",
      "Cache-Control": "private, no-store",
      ...corsHeaders(),
    },
  });
}

if (url.pathname === "/" && request.method === "GET") {
  await incrementViews(env);
  return new Response(await indexHtml(env), {
    headers: { "Content-Type": "text/html; charset=utf-8", ...corsHeaders() },
  });
}

if (url.pathname === "/count" && request.method === "GET") {
  const stats = await collectStats(env);
  return json({ project: PROJECT, total: stats.total || 0 });
}

if (url.pathname === "/stats" && request.method === "GET") {
  return json(await collectStats(env));
}

if (url.pathname === "/event" && request.method === "POST") {
  let body;
  try {
    body = await request.json();
  } catch {
    return json({ error: "JSON body required" }, 400);
  }
  const dims = parseDims(body || {});
  const count = await increment(env, dims);
  return json({
    ok: true,
    key: kvKey(dims),
    count,
    owner: dims.owner,
    repo: dims.repo,
    branch: dims.branch,
    fork: dims.fork,
    asset: dims.asset || null,
  });
}

if (url.pathname === "/go" && (request.method === "GET" || request.method === "HEAD")) {
  const dims = parseDims(url.searchParams);
  const asset = dims.asset || DEFAULT_ASSET;
  dims.asset = asset;
  if (request.method === "GET") await increment(env, dims);
  return serveAsset(request, env, asset, { head: request.method === "HEAD" });
}

if ((url.pathname === "/download" || url.pathname.startsWith("/download/")) && (request.method === "GET" || request.method === "HEAD")) {
  const dims = parseDims(url.searchParams);
  if (!dims.asset && url.pathname.startsWith("/download/")) {
    dims.asset = decodeURIComponent(url.pathname.slice("/download/".length));
  }
  const asset = dims.asset || DEFAULT_ASSET;
  dims.asset = asset;
  if (request.method === "GET") await increment(env, dims);
  return serveAsset(request, env, asset, { head: request.method === "HEAD" });
}

// gitbaby-seo-routes
if ((url.pathname === "/robots.txt" || url.pathname === "/robots.txt/") && request.method === "GET") {
  const body = "User-agent: *\nAllow: /\nSitemap: " + HOST + "/sitemap.xml\n";
  return new Response(body, {
    status: 200,
    headers: { "Content-Type": "text/plain; charset=utf-8", ...corsHeaders() },
  });
}

if ((url.pathname === "/sitemap.xml" || url.pathname === "/sitemap.xml/") && request.method === "GET") {
  const locs = [HOST + "/", HOST + "/download", HOST + "/install.sh", HOST + "/v1/skill", HOST + "/openapi.json",
    GITHUB_REPO];
  const xml = '<?xml version="1.0" encoding="UTF-8"?>\n<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9" >\n'
    + locs.map((u) => "  <url><loc>" + u + "</loc></url>").join("\n")
    + "\n</urlset>\n";

```

```
    return new Response(xml, {
      status: 200,
      headers: { "Content-Type": "application/xml; charset=utf-8", ...corsHeaders() },
    });
  }
  if ((url.pathname === "/cite.json" || url.pathname === "/cite.json/") && request.method === "GET") {
    return json({ "author": "Aziel Eliab", "title": "GodLock", "github": "https://github.com/AzielEliab/godlock", "download": "https://godlock-download-tracker.vibelock.workers.dev/download", "doi": null, "license": "Apache-2.0", "catalog": "https://aziel-runtime.vibelock.workers.dev/" });
  }
  // /gitbaby-seo-routes
  return json({ error: "not found" }, 404);
},
};
```

godlock/workers/download-tracker/src/runtime.js

```

/**
 * GodLock hosted runtime (Cloudflare Worker).
 * Ports ABAD engagement scoring and ephemeral receipts from the Python core.
 * NOT an anonymity network. No IP hiding. Logical ABAD receipts only.
 */
function runtimeCors() {
  return {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type",
  };
}

function runtimeJson(body, status = 200) {
  return new Response(JSON.stringify(body, null, 2), {
    status,
    headers: { "Content-Type": "application/json; charset=utf-8", ...runtimeCors() },
  });
}

async function sha256Hex(bytes) {
  const data = bytes instanceof Uint8Array ? bytes : new TextEncoder().encode(String(bytes));
  const dig = await crypto.subtle.digest("SHA-256", data);
  const arr = new Uint8Array(dig);
  let out = "";
  for (let i = 0; i < arr.length; i++) out += arr[i].toString(16).padStart(2, "0");
  return out;
}

async function readJsonBody(request) {
  const ct = (request.headers.get("content-type") || "").toLowerCase();
  if (request.method === "GET" || request.method === "HEAD") return {};
  const text = await request.text();
  if (!text || !text.trim()) return {};
  try {
    return JSON.parse(text);
  } catch {
    const err = new Error("JSON body required");
    err.status = 400;
    throw err;
  }
}

function utcNow() {
  return new Date().toISOString().replace(/\.d{3}Z$/, "Z");
}

function aiHowTo(base) {
  const openapi = base + "/openapi.json";
  const health = base + "/v1/health";
  return {
    chatgpt_actions: [
      "Open GPT Editor ? Actions ? Import from URL",
      "Paste " + openapi,
      "Authentication: None",
      "Allow GET /v1/health and the listed POST /v1 routes",
      "Test GET /v1/health, then a sample POST from the spec",
    ],
    grok_xai_tools: [
      "Add an HTTP / OpenAPI tool pointing at " + openapi,
      "Or register GET /v1/health, GET /openapi.json, and the product POSTs",
      "No API key. CORS is **",
    ],
    venice_http_tools: [
      "Add an HTTP tool with method, URL, and JSON body from " + openapi,
      "Start with GET " + health,
      "Then call the product POST listed in the spec",
    ],
    mcp_catalog: "https://aziel-runtime.viblock.workers.dev/mcp",
    notes: [
      "GET /download still serves the gzip tarball and increments the counter.",
      "/v1, /openapi.json, and /ai do not increment DOWNLOADS.",
    ],
  };
}

const PRODUCT = "godlock";
const SKILL_MARKDOWN = `---\nname: GodLock\ndescription: Use when calling GodLock hosted /v1 or installing the local package. Author Aziel Eliab.\n---\n\n# GodLock\n\nGodLock is a product name (ABAD stress-test and resilience engine). Not a VPN, ghost net, or anonymity tool. Author: Aziel Eliab.\n\n**Identity:** Aziel Eliab only. GodLock is a **product name**, not an identity label.\n\nAlways send `User-Agent: Mozilla/5.0`. Cloudflare Workers may 403 an empty agent.\n\n## How to use (3 steps)\n\n1. `curl -fsSL https://godlock-download-tracker.viblock.workers.dev/install.sh | bash`\n\n2. `godlock ui`\n\n3. Open http://127.0.0.1:8080 and tap **Record**, **Verify**, **Import JSON**, or **Export JSON**.\n\n`godlock doctor` prints PASS or FAIL in plain words.\n\n## Call these URLs\n\n- Worker OpenAPI: https://godlock-download-tracker.viblock.workers.dev/openapi.json\n- Catalog OpenAPI: https://aziel-runtime.viblock.workers.dev/openapi.json\n- MCP: `POST https://aziel-runtime.viblock.workers.dev/mcp`\n- Live skill (this markdown): `GET https://godlock-download-tracker.viblock.workers.dev/v1/skill`\n\nOps (do **not** increment downloads or views):\n\n| Method | Path | What |\n|-----|-----|-----|\n| GET | `/v1/health` | Liveness. Does not increment downloads. |\n| GET | `/v1/skill` | This markdown. Does not increment downloads. |\n| POST | `/v1/score` | ABAD engagement score. Advisory. Not a VPN. |\n| POST | `/v1/submit` | Ephemeral logical receipt. Not anonymity. |\n\nGrok: import OpenAPI as a custom tool. ChatGPT: GPT Actions. Venice: HTTP tools.\n\n## Example\n\n``bash\ncurl -s -A 'Mozilla/5.0' https://godlock-download-tracker.viblock.workers.dev/v1/health\ncurl -s -A 'Mozilla/5.0' https://godlock-download-tracker.viblock.workers.dev/v1`

```

```

/skill\ncurl -s -A 'Mozilla/5.0' -X POST https://godlock-download-tracker.viblock.workers.dev/v1/score \\n -H 'cont
ent-type: application/json' \\n -d '{"text":"'ABAD layers on phi\\n'\n``\n\nCounted download (gzip HTTP 200, no 3
02): https://godlock-download-tracker.viblock.workers.dev/download?asset=godlock-0.1.0.tar.gz\nGitHub: https://github
.com/AzielEliab/godlock\n";
const VERSION = "0.1.0";
const BASE = "https://godlock-download-tracker.viblock.workers.dev";
const BANNER = "NOT an anonymity network. No IP hiding. Logical ABAD receipts only. Not a VPN, proxy, or Tor hop.";
const MOTTO = "GodLock does not argue. It records, analyzes, hardens, and grows.";
const JEEVES_MODEL = "godlock-jeeves-heuristic-0.1";
const MAX_TEXT = 32768;
const GRID_SIZE = 25;
const NODE_PREFIX = "grid";
const WEIGHTS = {
  aziel_sequence: 3.0,
  phi: 2.0,
  sqrt2: 2.0,
  flower_of_life: 2.5,
  corkscrew: 2.0,
  abad: 3.0,
  merged_rule: 1.5,
};
const PATTERNS = {
  aziel_sequence: [/aziels+sequence/i, /aziell[-]?seq(?:uence)?/i],
  phi: [/bphi\b/i, /golden\s+ratio/i, /?/, /\b1\.\d*\b/],
  sqrt2: [/sqrt\s*\(?s*2/i, /?s*2/, /square\s+root\s+of\s+2/i, /\b1\.\d*\b/],
  flower_of_life: [/flower\s+of\s+life/i, /vesica\s+piscis/i],
  corkscrew: [/corkscrew/i],
  abad: [/babad\b/i, /a\s*[-?]\s*b\s*[-?]\s*a\s*[-?]\s*d/i, /\ba\s*-\s*b\s*-\s*a\s*-\s*d\b/i],
};

function withBanner(obj) {
  return { banner: BANNER, motto: MOTTO, anonymity_network: false, ip_hiding: false, proxy: false, vpn: false, tor: fa
lse, ...obj };
}

function scoreEngagement(text, extraKeywords) {
  if (!text) return { score: 0.0, hits: [] };
  const hits = [];
  let score = 0.0;
  for (const [family, patterns] of Object.entries(PATTERNS)) {
    if (patterns.some((p) => p.test(text))) {
      hits.push(family);
      score += WEIGHTS[family];
    }
  }
  for (const p of patterns) p.lastIndex = 0;
  const extras = extraKeywords || [];
  const low = text.toLowerCase();
  for (const kw of extras) {
    if (kw && low.includes(String(kw).toLowerCase())) {
      const tag = "merged_rule:" + kw;
      if (!hits.includes(tag)) {
        hits.push(tag);
        score += WEIGHTS.merged_rule;
      }
    }
  }
  return { score: Math.round(score * 10000) / 10000, hits };
}

function nodeName(i) {
  const n = i + 1;
  return NODE_PREFIX + "-" + String(n).padStart(2, "0");
}

function airlockPair() {
  const buf = new Uint32Array(2);
  crypto.getRandomValues(buf);
  const ingressIdx = buf[0] % GRID_SIZE;
  let egressIdx = buf[1] % GRID_SIZE;
  if (egressIdx === ingressIdx) egressIdx = (egressIdx + 1) % GRID_SIZE;
  return { ingress_node: nodeName(ingressIdx), egress_node: nodeName(egressIdx) };
}

function canonicalReceiptPayload(id, timestamp, ingress, egress, text) {
  const body = {
    egress_node: egress,
    id,
    ingress_node: ingress,
    text,
    timestamp,
  };
  const keys = Object.keys(body).sort();
  return "{" + keys.map((k) => JSON.stringify(k) + ":" + JSON.stringify(body[k])).join(",") + "}";
}

function jeevesHeuristic(receipt, engagement) {
  const families = engagement.hits.filter((h) => !String(h).startsWith("merged_rule:"));
  let suggested;
  let notes;
  if (families.length) {
    suggested = "Harden ABAD coverage for: " + families.join(", ") + ". Require an explicit " + families[0] + " check
in the active rules.";
    notes = "Heuristic engagement score=" + engagement.score + ". Families present in the counter-argument are treated
as the surface that should be hardened. Not a language model.";
  }
}

```



```

    } else {
      suggested = "Add a rule requiring the counter-argument to engage at least one ABAD token (Aziel Sequence, phi, sqrt(2), Flower of Life, corkscrew, A-B-A-D).";
      notes = "Heuristic engagement score=" + engagement.score + ". No ABAD family hit. Suggested rule is a keyword floor, not a proof.";
    }
    return {
      receipt_id: receipt.id,
      suggested_hardening: suggested,
      model: JEEVES_MODEL,
      notes,
      engagement,
    };
  }
}

function openapiDoc() {
  return {
    openapi: "3.1.0",
    info: {
      title: "GodLock Runtime API",
      version: VERSION,
      summary: MOTTO,
      description: BANNER,
    },
    servers: [{ url: BASE }],
    paths: {
      "/v1/health": { get: { operationId: "godlockHealth", summary: "Liveness", responses: { "200": { description: "OK" } } } },
      "/v1/score": {
        post: {
          operationId: "godlockScore",
          summary: "ABAD engagement score for text (heuristic, not a proof)",
          requestBody: { required: true, content: { "application/json": { schema: { type: "object", required: ["text"] } } } },
          properties: { text: { type: "string" } } },
          responses: { "200": { description: "Score plus hits" } },
        },
      },
      "/v1/submit": {
        post: {
          operationId: "godlockSubmit",
          summary: "Mint an ephemeral logical ABAD receipt. Not stored. Not an anonymity network.",
          requestBody: { required: true, content: { "application/json": { schema: { type: "object", required: ["text"] } } } },
          properties: { text: { type: "string" } } },
          responses: { "200": { description: "Ephemeral receipt + Jeeves heuristic" } },
        },
      },
    },
  };
}

async function handleScore(body) {
  const text = body && body.text != null ? String(body.text) : "";
  if (!text.trim()) return runtimeJson(withBanner({ ok: false, error: "text is required" })), 400);
  if (text.length > MAX_TEXT) return runtimeJson(withBanner({ ok: false, error: "text too large", max: MAX_TEXT })), 413);
  const engagement = scoreEngagement(text);
  return runtimeJson(withBanner({ ok: true, product: PRODUCT, engagement, durable: false }));
}

async function handleSubmit(body) {
  const text = body && body.text != null ? String(body.text).trim() : "";
  if (!text) return runtimeJson(withBanner({ ok: false, error: "text is required" })), 400);
  if (text.length > MAX_TEXT) return runtimeJson(withBanner({ ok: false, error: "text too large", max: MAX_TEXT })), 413);
  const pair = airlockPair();
  const id = crypto.randomUUID();
  const timestamp = utcNow();
  const payload = canonicalReceiptPayload(id, timestamp, pair.ingress_node, pair.egress_node, text);
  const digest = await sha256Hex(payload);
  const receipt = {
    id,
    timestamp,
    ingress_node: pair.ingress_node,
    egress_node: pair.egress_node,
    text,
    hash: digest,
  };
  const engagement = scoreEngagement(text);
  const analysis = jeevesHeuristic(receipt, engagement);
  return runtimeJson(withBanner({
    ok: true,
    product: PRODUCT,
    receipt,
    jeeves_analysis: analysis,
    engagement,
    durable: false,
    stored: false,
    note: "Ephemeral receipt for this response only. Logical grid nodes are names, not IPs. No durable anonymity.",
  })));
}

export async function handleRuntime(request, url, env) {
  const path = url.pathname;
  if (path === "/v1/health" && request.method === "GET") {
    return runtimeJson(withBanner({ ok: true, product: PRODUCT, version: VERSION }));
  }
}

```

```

    }

    if (path === "/v1/skill" && request.method === "GET") {
      return new Response(SKILL_MARKDOWN, {
        status: 200,
        headers: {
          "Content-Type": "text/markdown; charset=utf-8",
          "Cache-Control": "private, no-store",
          "X-KV-Increment": "false",
          "Access-Control-Allow-Origin": "*",
        },
      });
    }
  }

  if (path === "/openapi.json" && request.method === "GET") return runtimeJson(openapiDoc());
  if (path === "/ai" && request.method === "GET") {
    return runtimeJson(withBanner({
      product: PRODUCT,
      title: "Use with Grok, ChatGPT, Venice",
      openapi: BASE + "/openapi.json",
      health: BASE + "/v1/health",
      ...aiHowTo(BASE),
    }));
  }
  if (path === "/v1" && request.method === "GET") {
    return runtimeJson(withBanner({ product: PRODUCT, endpoints: ["GET /v1/health", "POST /v1/score", "POST /v1/submit", "GET /openapi.json", "GET /ai"] }));
  }
  if (path === "/v1/score" && request.method === "POST") {
    let body = {};
    try { body = await readJsonBody(request); } catch (e) { return runtimeJson(withBanner({ ok: false, error: e.message }, e.status || 400)); }
    return handleScore(body);
  }
  if (path === "/v1/submit" && request.method === "POST") {
    let body = {};
    try { body = await readJsonBody(request); } catch (e) { return runtimeJson(withBanner({ ok: false, error: e.message }, e.status || 400)); }
    return handleSubmit(body);
  }
  if (path === "/v1/score" || path === "/v1/submit") return runtimeJson(withBanner({ error: "method not allowed" }), 405);
  if (path.startsWith("/v1/")) return runtimeJson(withBanner({ error: "not found", product: PRODUCT }), 404);
  return null;
}

```

godlock/workers/download-tracker/wrangler.toml

```
name = "godlock-download-tracker"
main = "src/index.js"
compatibility_date = "2026-07-01"
workers_dev = true
account_id = "ac575a9b822bea2bed97d0ab73aed238"

[[kv_namespaces]]
binding = "DOWNLOADS"
id = "39527eecfc294ddd86af99c806baaa45"

[assets]
directory = "./public"
binding = "ASSETS"
run_worker_first = ["/install.sh", "/", "/download", "/download/*", "/stats", "/event", "/go", "/v1", "/v1/*", "/openapi.json", "/ai", "/count", "/robots.txt", "/sitemap.xml", "/cite.json"]
```

temporallock/install.sh

```
#!/usr/bin/env bash
# TemporalLock one-click install. Counted download via this project's Worker.
# Usage: curl -fsSL https://temporallock-download-tracker.vibelock.workers.dev/install.sh | bash
set -euo pipefail

HOST="${TEMPORALLOCK_HOME_HOST:-https://temporallock-download-tracker.vibelock.workers.dev}"
ASSET="${TEMPORALLOCK_HOME_ASSET:-temporallock-0.1.0.tar.gz}"
WORKDIR="${TEMPORALLOCK_HOME:-$HOME/temporallock}"

mkdir -p "$WORKDIR"
cd "$WORKDIR"

echo "Downloading counted tarball from ${HOST}/download (User-Agent Mozilla/5.0)"
curl -fsSL -A 'Mozilla/5.0' "${HOST}/download?asset=${ASSET}" -o "${ASSET}"

tar -xzf "${ASSET}"
DIR="$(find . -maxdepth 1 -type d -name 'temporallock-*' | head -n 1)"
if [ -n "$DIR" ]; then
  cd "$DIR"
fi

python3 -m venv .venv
# shellcheck disable=SC1091
. .venv/bin/activate
python -m pip install -U pip
python -m pip install -e .

echo
echo "Installed TemporalLock."
echo "Run: temporallock ui"
echo "Then open http://127.0.0.1:8766 (loopback only)"
echo "Author: Aziel Eliab."
```

temporallock/mobile/lib/main.dart

```

import 'dart:convert';

import 'package:crypto/crypto.dart';
import 'package:flutter/material.dart';

import 'theme.dart';

const genesisPrev = '0000000000000000000000000000000000000000000000000000000000000000';

void main() {
  runApp(const TemporalLockApp());
}

class TemporalLockApp extends StatelessWidget {
  const TemporalLockApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'TemporalLock',
      debugShowCheckedModeBanner: false,
      theme: buildAppTheme(),
      home: const ChainPage(),
    );
  }
}

class Receipt {
  Receipt({
    required this.timestamp,
    required this.summary,
    required this.evidence,
    required this.confidence,
    required this.prevHash,
    required this.hash,
  });
  final String timestamp;
  final String summary;
  final String evidence;
  final double confidence;
  final String prevHash;
  final String hash;

  bool get hashOk => recompute() == hash;

  String recompute() => digest(timestamp, summary, evidence, confidence, prevHash);
}

String _conf(double c) => c.toStringAsFixed(6);

String digest(String ts, String summary, String evidence, double confidence, String prev) {
  // Canonical: sorted keys, no extra whitespace, confidence 6 decimals unquoted.
  final raw =
    '{"confidence":$_conf(confidence),"evidence":${jsonEncode(evidence)},"prev_hash":${jsonEncode(prev)},"summary":'
    :${jsonEncode(summary)},"timestamp":${jsonEncode(ts)}}';
  return sha256.convert(utf8.encode(raw)).toString();
}

class ChainPage extends StatefulWidget {
  const ChainPage({super.key});

  @override
  State<ChainPage> createState() => _ChainPageState();
}

class _ChainPageState extends State<ChainPage> {
  final _summary = TextEditingController();
  final _evidence = TextEditingController();
  final _confidence = TextEditingController(text: '0.7');
  final _chain = <Receipt>[];
  String _verify = 'no chain yet';

  @override
  void dispose() {
    _summary.dispose();
    _evidence.dispose();
    _confidence.dispose();
    super.dispose();
  }

  String _now() => DateTime.now().toUtc().toIso8601String().split('.').first + 'Z';

  void _mint({required bool genesis}) {
    final evidence = _evidence.text;
    if (evidence.trim().isEmpty) {
      setState(() => _verify = 'empty evidence is invalid');
      return;
    }
    if (genesis && _chain.isNotEmpty) {
      setState(() => _verify = 'genesis refused: chain already exists (append only)');
      return;
    }
  }
}

```

```

    if (!genesis && _chain.isEmpty) {
      setState(() => _verify = 'append refused: run genesis first');
      return;
    }
    final conf = double.tryParse(_confidence.text);
    if (conf == null || conf < 0 || conf > 1) {
      setState(() => _verify = 'confidence must be a float in [0.0, 1.0]');
      return;
    }
    final ts = _now();
    final prev = genesis ? genesisPrev : _chain.last.hash;
    final h = digest(ts, _summary.text, evidence, conf, prev);
    setState(() {
      _chain.add(Receipt(
        timestamp: ts,
        summary: _summary.text,
        evidence: evidence,
        confidence: conf,
        prevHash: prev,
        hash: h,
      ));
      _summary.clear();
      _evidence.clear();
      _verify = _runVerify();
    });
  }

  String _runVerify() {
    if (_chain.isEmpty) return 'no chain yet';
    for (var i = 0; i < _chain.length; i++) {
      final r = _chain[i];
      if (!r.hashOk) return 'BROKEN at $i: hash mismatch';
      final expectPrev = i == 0 ? genesisPrev : _chain[i - 1].hash;
      if (r.prevHash != expectPrev) return 'BROKEN at $i: prev_hash link';
    }
    return 'OK  ${_chain.length} receipt(s). Receipts, not truth claims.';
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('TemporalLock')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          const Text(
            'Receipts, not truth claims.',
            style: TextStyle(color: kGold, fontStyle: FontStyle.italic, fontSize: 16),
          ),
          const SizedBox(height: 8),
          const Text(
            'On-device append-only chain. A receipt is an observation note, '
            'not a verdict. Corrections are new receipts. No modify, no delete.',
          ),
          const SizedBox(height: 16),
          TextField(controller: _summary, decoration: const InputDecoration(labelText: 'Summary')),
          const SizedBox(height: 8),
          TextField(
            controller: _evidence,
            maxLines: 3,
            decoration: const InputDecoration(labelText: 'Evidence (required)', alignLabelWithHint: true),
          ),
          const SizedBox(height: 8),
          TextField(
            controller: _confidence,
            keyboardType: const TextInputType.numberWithOptions(decimal: true),
            decoration: const InputDecoration(labelText: 'Confidence [0.0, 1.0]'),
          ),
          const SizedBox(height: 12),
          wrap(
            spacing: 8,
            runSpacing: 8,
            children: [
              FilledButton(onPressed: () => _mint(genesis: true), child: const Text('Genesis')),
              FilledButton(onPressed: () => _mint(genesis: false), child: const Text('Append')),
              OutlinedButton(
                onPressed: () => setState(() => _verify = _runVerify()),
                child: const Text('Verify'),
              ),
            ],
          ),
          const SizedBox(height: 12),
          Text(_verify, style: const TextStyle(color: kGold)),
          const SizedBox(height: 16),
          for (var i = 0; i < _chain.length; i++)
            Card(
              margin: const EdgeInsets.only(bottom: 10),
              child: Padding(
                padding: const EdgeInsets.all(12),
                child: SelectableText(
                  [
                    '#$i  ${_chain[i].timestamp}  conf=${_conf(_chain[i].confidence)}',
                    _chain[i].summary,
                    'evidence: ${_chain[i].evidence}',
                    'prev: ${_chain[i].prevHash.substring(0, 16)}?',
                  ].join('\n'),
                ),
              ),
            ),
        ],
      ),
    );
  }

```

```
        'hash: ${_chain[i].hash}',  
      ].join('\n'),  
      style: const TextStyle(fontFamily: 'monospace', fontSize: 12, height: 1.4),  
    ),  
  ),  
),  
),  
);  
}  
}
```

temporallock/mobile/lib/theme.dart

```

import 'package:flutter/material.dart';

/// Matte black + gold Material 3 dark theme. No analytics.
const Color kMatteBlack = Color(0xFF0B0B0B);
const Color kSurface = Color(0xFF141414);
const Color kGold = Color(0xFFC9A227);
const Color kGoldDim = Color(0xFF8A7219);
const Color kIvory = Color(0xFFE8E0D0);

ThemeData buildAppTheme() {
  const scheme = ColorScheme.dark(
    brightness: Brightness.dark,
    primary: kGold,
    onPrimary: kMatteBlack,
    secondary: kGoldDim,
    onSecondary: kIvory,
    surface: kSurface,
    onSurface: kIvory,
    error: Color(0xFFB54A4A),
    onError: kIvory,
  );
  return ThemeData(
    useMaterial3: true,
    brightness: Brightness.dark,
    colorScheme: scheme,
    scaffoldBackgroundColor: kMatteBlack,
    appBarTheme: const AppBarTheme(
      backgroundColor: kMatteBlack,
      foregroundColor: kGold,
      elevation: 0,
      centerTitle: false,
    ),
    cardTheme: CardThemeData(
      color: kSurface,
      elevation: 0,
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
        side: const BorderSide(color: Color(0x33C9A227)),
      ),
    ),
    inputDecorationTheme: InputDecorationTheme(
      filled: true,
      fillColor: const Color(0xFF1A1A1A),
      border: OutlineInputBorder(borderRadius: BorderRadius.circular(10)),
      focusedBorder: OutlineInputBorder(
        borderRadius: BorderRadius.circular(10),
        borderSide: const BorderSide(color: kGold),
      ),
    ),
    segmentedButtonTheme: SegmentedButtonThemeData(
      style: ButtonStyle(
        foregroundColor: WidgetStateProperty.resolveWith((s) {
          return s.contains(WidgetState.selected) ? kMatteBlack : kIvory;
        }),
        backgroundColor: WidgetStateProperty.resolveWith((s) {
          return s.contains(WidgetState.selected) ? kGold : kSurface;
        }),
      ),
    ),
  );
}

```


temporallock/mobile/pubspec.yaml

```
name: temporallock
description: On-device genesis / append / verify. Receipts, not truth claims.
publish_to: "none"
version: 0.1.0+1

environment:
  sdk: ">=3.3.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter
  crypto: ^3.0.3

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0

flutter:
  uses-material-design: true
```

temporallock/pyproject.toml

```

[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "temporallock"
version = "0.1.0"
description = "Append-only observation receipts with cryptographic linking ? receipts, not truth claims (TemporalLock, July 2026).\"
readme = "README.md"
license = {file = "LICENSE"}
authors = [{name = "Aziel Eliab"}]
requires-python = ">=3.10"
keywords = [
    "receipts",
    "append-only",
    "hash-chain",
    "observations",
    "evidence",
    "forks",
]
classifiers = [
    "Development Status :: 4 - Beta",
    "License :: OSI Approved :: Apache Software License",
    "Programming Language :: Python :: 3",
    "Programming Language :: Python :: 3.10",
    "Programming Language :: Python :: 3.11",
    "Programming Language :: Python :: 3.12",
    "Programming Language :: Python :: 3.13",
    "Topic :: Security :: Cryptography",
    "Intended Audience :: Developers",
]
dependencies = []

[project.optional-dependencies]
dev = ["pytest>=7.4"]

[project.scripts]
temporallock = "temporallock.cli:main"

[project.urls]
Homepage = "https://github.com/AzielEliab/temporallock"
Download = "https://github.com/AzielEliab/temporallock/releases"
Issues = "https://github.com/AzielEliab/temporallock/issues"

[tool.setuptools.packages.find]
include = ["temporallock*"]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]
addopts = "-q"

```

temporallock/temporallock/__init__.py

```
"""TemporalLock: append-only observation receipts.

July 2026 whitepaper implementation by Aziel Eliab.

Receipts, not truth claims. Forks are welcome and always allowed.
No consensus, no mining, no tokens, no truth scores.
"""

from __future__ import annotations

from temporallock.chain import Chain, Fork, VerifyResult, detect_forks
from temporallock.errors import AppendOnlyError, ChainError, ReceiptError, TemporalLockError
from temporallock.hashing import GENESIS_PREV_HASH, canonical_bytes, digest
from temporallock.receipt import Receipt

__version__ = "0.1.0"
__author__ = "Azriel Eliab"
__all__ = [
    "AppendOnlyError",
    "Chain",
    "ChainError",
    "Fork",
    "GENESIS_PREV_HASH",
    "Receipt",
    "ReceiptError",
    "TemporalLockError",
    "VerifyResult",
    "canonical_bytes",
    "detect_forks",
    "digest",
    "__version__",
]
```

temporallock/temporallock/__main__.py

```
"""Allow ``python -m temporallock`` to invoke the CLI."""

from temporallock.cli import main

if __name__ == "__main__":
    raise SystemExit(main())
```

temporallock/temporallock/canon.py

```

"""Canonical encoding for TemporalLock receipts (v0.1.0).

Canonical encoding
-----
UTF-8 JSON with **sorted keys** and **no extra whitespace**
(`separators=(",", ":")`, `sort_keys=True`, `ensure_ascii=False`).

Hashed fields (core only, v0.1.0):

    timestamp, summary, evidence, confidence, prev_hash

The receipt's own `hash` field is **excluded** from the encoding.

`confidence` is serialized as a JSON **number** with exactly **6 decimal
places** so hashes are stable across platforms and Python versions
(example: `0.7` becomes `0.700000`). A placeholder-and-replace step
is used because `json.dumps` would otherwise emit `0.7`.

Optional fields are allowed in implementations (extra JSONL keys) but
MUST NOT enter the core hash unless a later **versioned** schema is
introduced. v0.1.0 is core-only so chains stay verifiable long-term.

Genesis `prev_hash` is 64 zero hex characters.

Algorithm: SHA-256 of the canonical UTF-8 bytes; digest is lowercase hex.
"""

from __future__ import annotations

import hashlib
import json
from typing import Any

# Genesis previous-hash: 64 zero hex chars (SHA-256 width).
GENESIS_PREV_HASH = "0" * 64

# Confidence is always emitted with this many digits after the decimal.
CONFIDENCE_DECIMALS = 6

CORE_FIELDS = ("timestamp", "summary", "evidence", "confidence", "prev_hash")

_CONF_PLACEHOLDER = "__TL_CONFIDENCE__"

def format_confidence(confidence: float) -> str:
    """Return confidence as a fixed-precision decimal string (not quoted)."""
    return f"{float(confidence):.{CONFIDENCE_DECIMALS}f}"

def canonical_bytes(
    timestamp: str,
    summary: str,
    evidence: str,
    confidence: float,
    prev_hash: str,
) -> bytes:
    """Return the v0.1.0 canonical UTF-8 encoding of the core fields.

    Field order in the JSON object is lexicographic via `sort_keys=True`:
    confidence, evidence, prev_hash, summary, timestamp.
    """
    payload: dict[str, Any] = {
        "confidence": _CONF_PLACEHOLDER,
        "evidence": evidence,
        "prev_hash": prev_hash,
        "summary": summary,
        "timestamp": timestamp,
    }
    raw = json.dumps(payload, sort_keys=True, separators=(",", ":"), ensure_ascii=False)
    raw = raw.replace(f'"{_CONF_PLACEHOLDER}"', format_confidence(confidence))
    return raw.encode("utf-8")

def digest(
    timestamp: str,
    summary: str,
    evidence: str,
    confidence: float,
    prev_hash: str,
) -> str:
    """SHA-256 (lowercase hex) of `canonical_bytes(...)`."""
    return hashlib.sha256(
        canonical_bytes(timestamp, summary, evidence, confidence, prev_hash)
    ).hexdigest()

```

temporallock/temporallock/chain.py

```

"""Append-only receipt chain.

chain.append(...)`` only. No modify, no delete. Corrections and
disputes are new receipts. Divergent chains (forks) are valid and
detectable; this module does not pick a winner.
"""

from __future__ import annotations

import json
from dataclasses import dataclass, field
from pathlib import Path
from typing import Iterator, Sequence

from temporallock.errors import AppendOnlyError, ChainError
from temporallock.hashing import GENESIS_PREV_HASH
from temporallock.receipt import Receipt

@dataclass(frozen=True)
class VerifyResult:
    """Result of walking a chain: hashes and links, nothing interpretive."""

    ok: bool
    length: int
    first_hash: str | None
    last_hash: str | None
    errors: list[str] = field(default_factory=list)

@dataclass(frozen=True)
class Fork:
    """A prev_hash with more than one child. Forks are allowed."""

    prev_hash: str
    child_hashes: tuple[str, ...]

def detect_forks(receipts: Sequence[Receipt]) -> list[Fork]:
    """Find prev_hash values that have more than one child.

    Does not pick a winner. Order of forks follows first appearance of
    the parent hash; child hashes follow receipt order.
    """
    children: dict[str, list[str]] = {}
    order: list[str] = []
    for rec in receipts:
        if rec.prev_hash not in children:
            order.append(rec.prev_hash)
            children[rec.prev_hash] = []
        children[rec.prev_hash].append(rec.hash)
    return [
        Fork(prev_hash=ph, child_hashes=tuple(children[ph]))
        for ph in order
        if len(children[ph]) > 1
    ]

def _receipt_json_line(receipt: Receipt) -> str:
    """One JSONL line. Confidence stored as a JSON number; hash included.

    This is the *file* encoding, not the canonical hash encoding.
    """
    body = receipt.to_dict()
    return json.dumps(body, sort_keys=True, separators=(",", ":"), ensure_ascii=False)

def _append_line(path: Path, receipt: Receipt) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    # Append-only: mode 'a' never rewrites earlier bytes.
    with path.open("a", encoding="utf-8") as fh:
        fh.write(_receipt_json_line(receipt))
        fh.write("\n")
        fh.flush()

class Chain:
    """In-memory and/or JSONL-backed append-only sequence of receipts."""

    def __init__(
        self,
        receipts: Sequence[Receipt] | None = None,
        path: str | Path | None = None,
    ) -> None:
        self._receipts: tuple[Receipt, ...] = tuple(receipts or ())
        self._path: Path | None = Path(path) if path is not None else None

    @property
    def path(self) -> Path | None:
        return self._path

    def __len__(self) -> int:

```

```

        return len(self._receipts)

    def __iter__(self) -> Iterator[Receipt]:
        return iter(self._receipts)

    def __getitem__(self, index: int) -> Receipt:
        return self._receipts[index]

    def __bool__(self) -> bool:
        return bool(self._receipts)

    def _refuse(self, action: str) -> None:
        raise AppendOnlyError(
            f"cannot {action}: TemporalLock is append-only; "
            "record a correction as a new receipt"
        )

    def pop(self, *args: object, **kwargs: object) -> None:
        self._refuse("pop")

    def insert(self, *args: object, **kwargs: object) -> None:
        self._refuse("insert")

    def remove(self, *args: object, **kwargs: object) -> None:
        self._refuse("remove")

    def clear(self) -> None:
        self._refuse("clear")

    def reverse(self) -> None:
        self._refuse("reverse")

    def __setitem__(self, *args: object, **kwargs: object) -> None:
        self._refuse("replace")

    def __delitem__(self, *args: object, **kwargs: object) -> None:
        self._refuse("delete")

    @classmethod
    def load(cls, path: str | Path) -> "Chain":
        """Read a JSONL file (core fields). Extra keys are ignored.

        After load the file is only ever opened again in mode ``'a'``.
        Does not rewrite. Call ``verify()`` to check hashes and links.
        """
        path = Path(path)
        receipts: list[Receipt] = []
        if path.is_file():
            text = path.read_text(encoding="utf-8")
            for line in text.splitlines():
                line = line.strip()
                if not line:
                    continue
                receipts.append(Receipt.from_dict(json.loads(line)))
        return cls(receipts, path=path)

    @classmethod
    def genesis(
        cls,
        path: str | Path,
        *,
        summary: str,
        evidence: str,
        confidence: float = 1.0,
        timestamp: str | None = None,
    ) -> "Chain":
        """Create the first receipt in a new JSONL file. File must be absent or empty."""
        path = Path(path)
        if path.exists() and path.stat().st_size > 0:
            raise ChainError(f"chain already exists: {path}; use append")
        chain = cls(), path=path
        chain.append(
            summary=summary,
            evidence=evidence,
            confidence=confidence,
            timestamp=timestamp,
        )
        return chain

    def append(
        self,
        summary: str,
        evidence: str,
        confidence: float = 1.0,
        timestamp: str | None = None,
        *,
        require_existing: bool = False,
    ) -> Receipt:
        """Append a new receipt. First receipt is genesis (prev_hash = 64 zeros).

        If ``require_existing`` is true (CLI ``append``), the chain must
        already contain at least one receipt.
        """
        if require_existing and not self._receipts:
            raise ChainError("chain does not exist or is empty; use genesis")

```

```

    if self._receipts:
        prev = self._receipts[-1].hash
    else:
        prev = GENESIS_PREV_HASH
    receipt = Receipt.create(
        summary=summary,
        evidence=evidence,
        confidence=confidence,
        timestamp=timestamp,
        prev_hash=prev,
    )
    if self._path is not None:
        _append_line(self._path, receipt)
    self._receipts = self._receipts + (receipt,)
    return receipt

def verify(self) -> VerifyResult:
    """Walk the chain. Check each hash and each consecutive link.

    Forks stored as a single linear JSONL will fail the consecutive
    link check (store divergent chains separately). Hash mismatches
    and broken prev_hash links are errors. Forks themselves are not
    a verdict; see ``forks()``.
    """
    errors: list[str] = []
    n = len(self._receipts)
    first = self._receipts[0].hash if n else None
    last = self._receipts[-1].hash if n else None

    for i, rec in enumerate(self._receipts):
        expected = rec.recomputed_hash()
        if rec.hash != expected:
            errors.append(
                f"index {i}: stored hash {rec.hash} != recomputed {expected}"
            )
        if i == 0:
            continue
        prev = self._receipts[i - 1]
        if rec.prev_hash != prev.hash:
            errors.append(
                f"index {i}: prev_hash {rec.prev_hash} != previous.hash {prev.hash}"
            )

    return VerifyResult(
        ok=not errors,
        length=n,
        first_hash=first,
        last_hash=last,
        errors=errors,
    )

def forks(self) -> list[Fork]:
    """prev_hash values with more than one child. Does not pick a winner."""
    return detect_forks(self._receipts)

```


temporallock/temporallock/cli.py

```

"""Command-line interface for TemporalLock.

    temporallock ui [--host 127.0.0.1] [--port 8766]
    temporallock version
    temporallock gate FILE [--chain FILE.jsonl]
    temporallock genesis --chain FILE.jsonl --summary "..." --evidence "..."
    temporallock append --chain FILE.jsonl --summary "..." --evidence "..." [--confidence 0.7] [--timestamp ISO]
    temporallock verify FILE.jsonl
    temporallock show FILE.jsonl

Receipts, not truth claims. Forks always allowed.
`gate FILE` hashes the file and appends a receipt before treating it
as accepted (genesis if the chain is new, else append, then verify).
"""

from __future__ import annotations

import argparse
import hashlib
import json
import sys
from pathlib import Path
from typing import Sequence

from temporallock import __version__
from temporallock.chain import Chain
from temporallock.errors import AppendOnlyError, ChainError, ReceiptError, TemporalLockError

def _build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="temporallock",
        description=(
            "TemporalLock ? append-only observation receipts "
            "(Aziel Eliab, July 2026). Receipts, not truth claims. "
            "Local UI: `temporallock ui` at http://127.0.0.1:8766."
        ),
    )
    sub = parser.add_subparsers(dest="cmd", required=True)

    sub.add_parser("version", help="Print package version.")

    p_ui = sub.add_parser("ui", help="Run the localhost UI (127.0.0.1:8766).")
    p_ui.add_argument("--host", default="127.0.0.1", help="Bind host (default 127.0.0.1).")
    p_ui.add_argument("--port", type=int, default=8766, help="Bind port (default 8766).")

    p_gen = sub.add_parser("genesis", help="Write the first receipt (prev_hash = 64 zeros).")
    p_gen.add_argument("--chain", required=True, help="JSONL chain path (must not already exist).")
    p_gen.add_argument("--summary", required=True, help="Brief summary of what was observed.")
    p_gen.add_argument("--evidence", required=True, help="Supporting evidence (body and/or URI/path).")
    p_gen.add_argument("--confidence", type=float, default=1.0, help="Observer confidence in [0.0, 1.0] (default 1.0).")

    p_app = sub.add_parser("append", help="Append a receipt to an existing chain.")
    p_app.add_argument("--chain", required=True, help="JSONL chain path (must already exist).")
    p_app.add_argument("--summary", required=True, help="Brief summary of what was observed.")
    p_app.add_argument("--evidence", required=True, help="Supporting evidence (body and/or URI/path).")
    p_app.add_argument("--confidence", type=float, default=0.7, help="Observer confidence in [0.0, 1.0] (default 0.7).")

    p_ver = sub.add_parser("verify", help="Walk the chain; exit 0 if intact, nonzero if broken.")
    p_ver.add_argument("file", help="JSONL chain path.")

    p_show = sub.add_parser("show", help="Print receipts in a chain.")
    p_show.add_argument("file", help="JSONL chain path.")

    p_gate = sub.add_parser(
        "gate",
        help="Hash FILE, append a receipt, verify; treat the file as accepted only if the chain is intact.",
    )
    p_gate.add_argument("file", help="File to hash and accept.")
    p_gate.add_argument(
        "--chain",
        default=None,
        help="JSONL chain path (default: FILE.receipts.jsonl beside the file).",
    )
    p_gate.add_argument("--summary", default=None, help="Optional summary (default: gate accept <name>).")
    p_gate.add_argument("--confidence", type=float, default=1.0, help="Observer confidence in [0.0, 1.0].")
    p_gate.add_argument("--timestamp", default=None, help="UTC ISO-8601 timestamp (default: now).")
    p_gate.add_argument("--json", action="store_true", dest="as_json", help="Print the verify payload as JSON.")

    p_doc = sub.add_parser("doctor", help="Self-check. No network, no telemetry.")
    p_doc.add_argument("--json", action="store_true", dest="as_json", help="Print doctor results as JSON.")

    p_imp = sub.add_parser("import", help="Import a JSON document.")
    p_imp.add_argument("path")

    p_exp = sub.add_parser("export", help="Export a JSON document.")
    p_exp.add_argument("path")

```

```

    return parser

def _sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as fh:
        for chunk in iter(lambda: fh.read(65536), b''):
            digest.update(chunk)
    return digest.hexdigest()

def _cmd_gate(args: argparse.Namespace) -> int:
    path = Path(args.file)
    if not path.is_file():
        print(f"not found: {path}", file=sys.stderr)
        return 2
    digest = _sha256_file(path)
    evidence = f"sha256:{digest} path:{path.name}"
    summary = args.summary or f"gate accept {path.name}"
    chain_path = Path(args.chain) if args.chain else Path(str(path) + ".receipts.jsonl")
    if not chain_path.is_file() or chain_path.stat().st_size == 0:
        chain = Chain.genesis(
            chain_path,
            summary=summary,
            evidence=evidence,
            confidence=args.confidence,
            timestamp=args.timestamp,
        )
        rec = chain[-1]
        action = "genesis"
    else:
        chain = Chain.load(chain_path)
        rec = chain.append(
            summary=summary,
            evidence=evidence,
            confidence=args.confidence,
            timestamp=args.timestamp,
            require_existing=True,
        )
        action = "appended"
    result = chain.verify()
    payload = {
        "ok": result.ok,
        "accepted": bool(result.ok),
        "action": action,
        "file": str(path),
        "file_sha256": digest,
        "receipt": rec.hash,
        "chain": str(chain_path),
        "length": result.length,
        "errors": result.errors,
    }
    if args.as_json:
        print(json.dumps(payload, indent=2))
    else:
        print(f"{action} {rec.hash}")
        print(f"file_sha256={digest}")
        print(f"chain={chain_path}")
        if result.ok:
            print("accepted")
        else:
            print("chain broken; file not accepted", file=sys.stderr)
    return 0 if result.ok else 1

def _print_receipt_brief(receipt, index: int | None = None) -> None:
    prefix = f"[{index}] " if index is not None else ""
    print(f"{prefix}{receipt.timestamp} conf={receipt.confidence:.6f} hash={receipt.hash}")
    print(f"    prev={receipt.prev_hash}")
    print(f"    summary: {receipt.summary}")
    print(f"    evidence: {receipt.evidence}")

def main(argv: Sequence[str] | None = None) -> int:
    parser = _build_parser()
    args = parser.parse_args(list(argv) if argv is not None else None)

    try:
        if args.cmd == "version":
            print(f"temporallock {__version__}")
            return 0

        if args.cmd == "ui":
            from temporallock.ui import serve

            serve(host=args.host, port=args.port)
            return 0

        if args.cmd == "genesis":
            path = Path(args.chain)
            chain = Chain.genesis(
                path,
                summary=args.summary,
                evidence=args.evidence,

```

```

        confidence=args.confidence,
        timestamp=args.timestamp,
    )
    rec = chain[-1]
    print(f"genesis {rec.hash}")
    return 0

if args.cmd == "append":
    path = Path(args.chain)
    if not path.is_file() or path.stat().st_size == 0:
        print("chain does not exist; use genesis for the first receipt", file=sys.stderr)
        return 2
    chain = Chain.load(path)
    rec = chain.append(
        summary=args.summary,
        evidence=args.evidence,
        confidence=args.confidence,
        timestamp=args.timestamp,
        require_existing=True,
    )
    print(f"appended {rec.hash}")
    return 0

if args.cmd == "verify":
    path = Path(args.file)
    if not path.is_file():
        print(f"not found: {path}", file=sys.stderr)
        return 2
    chain = Chain.load(path)
    result = chain.verify()
    payload = {
        "ok": result.ok,
        "length": result.length,
        "first_hash": result.first_hash,
        "last_hash": result.last_hash,
        "errors": result.errors,
    }
    print(json.dumps(payload, indent=2))
    return 0 if result.ok else 1

if args.cmd == "show":
    path = Path(args.file)
    if not path.is_file():
        print(f"not found: {path}", file=sys.stderr)
        return 2
    chain = Chain.load(path)
    print(f"temporallock chain n={len(chain)} path={path}")
    for i, rec in enumerate(chain):
        _print_receipt_brief(rec, i)
    forks = chain.forks()
    if forks:
        print(f"forks: {len(forks)} (allowed; no winner chosen)")
        for f in forks:
            print(f"  prev={f.prev_hash}")
            for h in f.child_hashes:
                print(f"    child={h}")
    return 0

if args.cmd == "gate":
    return _cmd_gate(args)

if args.cmd == "doctor":
    from temporallock.doctor import run_doctor

    return run_doctor(as_json=getattr(args, "as_json", False))

if args.cmd == "import":
    from temporallock.jsonio import import_json

    rec = import_json(args.path)
    sys.stdout.write(json.dumps(rec, indent=2, ensure_ascii=False) + "\n")
    return 0

if args.cmd == "export":
    from temporallock.jsonio import export_json

    rec = export_json(args.path)
    sys.stdout.write(json.dumps(rec, indent=2, ensure_ascii=False) + "\n")
    return 0

parser.error(f"unknown command {args.cmd}")
return 2
except (ReceiptError, ChainError, AppendOnlyError, TemporalLockError) as exc:
    print(str(exc), file=sys.stderr)
    return 2
except OSError as exc:
    print(str(exc), file=sys.stderr)
    return 2

if __name__ == "__main__":
    raise SystemExit(main())

```

temporallock/temporallock/doctor.py

```

"""Self-check for TemporalLock. No network, no telemetry.

temporallock doctor
"""

from __future__ import annotations

import json
import tempfile
from pathlib import Path
from typing import Callable

from temporallock import __version__

AUTHOR = "Aziel Eliab"
Check = tuple[str, bool, str]

def _ok(name: str, detail: str = "") -> Check:
    return name, True, detail

def _fail(name: str, detail: str) -> Check:
    return name, False, detail

def _check_version() -> Check:
    if __version__:
        return _ok("version", str(__version__))
    return _fail("version", "missing")

def _check_identity() -> Check:
    try:
        mod = __import__(__name__.split(".")[0])
        author = str(getattr(mod, "__author__", AUTHOR))
    except Exception as exc: # noqa: BLE001
        return _fail("identity", str(exc))
    blob = author + " " + AUTHOR
    forbidden = ("Col" + "lin H" + "orton", "Ja" + "ck Al" + "tman", "GodLock" + ".AZ", "Reve" + "aler")
    if any(x in blob for x in forbidden):
        return _fail("identity", "forbidden identity label")
    if "Aziel Eliab" not in blob:
        return _fail("identity", author)
    return _ok("identity", AUTHOR)

def _check_json_roundtrip() -> Check:
    from temporallock.jsonio import export_json, import_json

    with tempfile.TemporaryDirectory() as tmp:
        src = Path(tmp) / "in.json"
        out = Path(tmp) / "out.json"
        src.write_text(json.dumps({"product": "temporallock", "author": AUTHOR, "ok": True}, indent=2), encoding="utf-
8")
        rec = import_json(src)
        if not rec.get("ok"):
            return _fail("import", str(rec))
        rec2 = export_json(out)
        if not rec2.get("ok") or not out.exists():
            return _fail("export", str(rec2))
        doc = json.loads(out.read_text(encoding="utf-8"))
        if doc.get("author") != AUTHOR:
            return _fail("export author", str(doc.get("author")))
        return _ok("json import/export", "roundtrip")

CHECKS: tuple[Callable[[], Check], ...] = (
    _check_version,
    _check_identity,
    _check_json_roundtrip,
)

def run_doctor(*, as_json: bool = False) -> int:
    results = []
    failed = 0
    for fn in CHECKS:
        name, ok, detail = fn()
        results.append({"name": name, "ok": ok, "detail": detail})
        if not ok:
            failed += 1
        mark = "ok" if ok else "FAIL"
        if not as_json:
            print(f"[{mark}] {name}" + (f" ? {detail}" if detail else ""))
    payload = {
        "ok": failed == 0,
        "failed": failed,
        "checks": results,
        "version": __version__,
        "author": AUTHOR,
    }

```

```
        "network": False,
        "telemetry": False,
    }
    if as_json:
        print(json.dumps(payload, indent=2))
    else:
        print("doctor", "passed" if failed == 0 else "failed")
    return 0 if failed == 0 else 1
```

temporallock/temporallock/errors.py

```
"""TemporalLock errors."""

from __future__ import annotations

class TemporalLockError(Exception):
    """Base error for TemporalLock."""

class AppendOnlyError(TemporalLockError):
    """Raised on any attempt to edit, pop, replace, or delete a receipt."""

class ReceiptError(TemporalLockError):
    """Raised when a receipt field is invalid (empty evidence, bad confidence)."""

class ChainError(TemporalLockError):
    """Raised for chain-level problems (missing genesis, append on empty file)."""
```

temporallock/temporallock/hashing.py

```
"""Hashing helpers for TemporalLock.

Re-exports canonical encoding from ``canon`` so callers can use
``temporallock.hashing.canonical_bytes`` and ``temporallock.hashing.digest``
as specified. Stdlib ``hashlib`` only; no extra crypto packages.
"""

from __future__ import annotations

from temporallock.canon import (
    CONFIDENCE_DECIMALS,
    CORE_FIELDS,
    GENESIS_PREV_HASH,
    canonical_bytes,
    digest,
    format_confidence,
)

__all__ = [
    "CONFIDENCE_DECIMALS",
    "CORE_FIELDS",
    "GENESIS_PREV_HASH",
    "canonical_bytes",
    "digest",
    "format_confidence",
]
```

temporallock/temporallock/jsonio.py

```

"""Small JSON import/export for TemporalLock. Author: Aziel Eliab."""

from __future__ import annotations

import json
from pathlib import Path
from typing import Any

from temporallock import __version__

AUTHOR = "Aziel Eliab"
PRODUCT = "TemporalLock"
STATE_NAME = ".temporallock-state.json"

def _as_path(path: str | Path) -> Path:
    return Path(path)

def import_json(path: str | Path) -> dict[str, Any]:
    pth = _as_path(path)
    doc = json.loads(pth.read_text(encoding="utf-8"))
    if not isinstance(doc, dict):
        raise ValueError("JSON object required")
    dest = Path.cwd() / STATE_NAME
    dest.write_text(json.dumps(doc, indent=2, ensure_ascii=False) + "\n", encoding="utf-8")
    return {
        "ok": True,
        "imported": str(pth),
        "stored": str(dest),
        "keys": sorted(str(k) for k in doc.keys()),
        "author": AUTHOR,
        "product": PRODUCT,
        "version": __version__,
    }

def export_json(path: str | Path) -> dict[str, Any]:
    pth = _as_path(path)
    src = Path.cwd() / STATE_NAME
    payload: Any = {}
    if src.exists():
        try:
            payload = json.loads(src.read_text(encoding="utf-8"))
        except Exception:
            payload = {}
    doc = {
        "product": PRODUCT,
        "package": "temporallock",
        "version": __version__,
        "author": AUTHOR,
        "payload": payload,
    }
    pth.write_text(json.dumps(doc, indent=2, ensure_ascii=False) + "\n", encoding="utf-8")
    return {
        "ok": True,
        "exported": str(pth),
        "author": AUTHOR,
        "product": PRODUCT,
        "version": __version__,
    }

```


temporallock/temporallock/receipt.py

```

"""Immutable observation receipt.

A receipt is a record of an observation at a moment. It is not a truth
claim, not an authority, and not a narrative. Corrections are new
receipts; this object cannot be edited.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from datetime import datetime, timezone
from math import isfinite
from typing import Any, Mapping

from temporallock.errors import AppendOnlyError, ReceiptError
from temporallock.hashing import GENESIS_PREV_HASH, digest

_HEX = set("0123456789abcdef")

def utc_now() -> str:
    """Observer-clock 'now' as UTC ISO-8601 with a trailing Z, second precision."""
    return (
        datetime.now(timezone.utc)
        .replace(microsecond=0)
        .isoformat()
        .replace("+00:00", "Z")
    )

def _require_str(name: str, value: Any) -> str:
    if not isinstance(value, str):
        raise ReceiptError(f"{name} must be a string")
    return value

def validate_evidence(evidence: str) -> str:
    evidence = _require_str("evidence", evidence)
    if evidence.strip() == "":
        raise ReceiptError("empty evidence is invalid")
    return evidence

def validate_confidence(confidence: Any) -> float:
    try:
        value = float(confidence)
    except (TypeError, ValueError) as exc:
        raise ReceiptError("confidence must be a float in [0.0, 1.0]") from exc
    if not isfinite(value) or value < 0.0 or value > 1.0:
        raise ReceiptError("confidence must be a float in [0.0, 1.0] inclusive")
    return value

def validate_summary(summary: str) -> str:
    return _require_str("summary", summary)

def validate_timestamp(timestamp: str) -> str:
    ts = _require_str("timestamp", timestamp)
    if ts.strip() == "":
        raise ReceiptError("timestamp must be a non-empty UTC ISO-8601 string")
    return ts

@dataclass
class Receipt:
    """Frozen observation receipt (v0.1.0 core fields only).

    Fields hashed: timestamp, summary, evidence, confidence, prev_hash.
    ``hash`` is SHA-256 of the canonical encoding of those fields.
    """

    timestamp: str
    summary: str
    evidence: str
    confidence: float
    prev_hash: str
    hash: str
    _frozen: bool = field(default=False, init=False, repr=False, compare=False)

    def __post_init__(self) -> None:
        object.__setattr__(self, "timestamp", validate_timestamp(self.timestamp))
        object.__setattr__(self, "summary", validate_summary(self.summary))
        object.__setattr__(self, "evidence", validate_evidence(self.evidence))
        object.__setattr__(self, "confidence", validate_confidence(self.confidence))
        object.__setattr__(self, "prev_hash", _require_str("prev_hash", self.prev_hash))
        object.__setattr__(self, "hash", _require_str("hash", self.hash))
        object.__setattr__(self, "_frozen", True)

    def __setattr__(self, name: str, value: Any) -> None:
        if getattr(self, "_frozen", False):
            raise AppendOnlyError(

```

```

        "receipts cannot be modified; append a correction as a new receipt"
    )
    object.__setattr__(self, name, value)

def __delattr__(self, name: str) -> None:
    raise AppendOnlyError(
        "receipts cannot be deleted; append a correction as a new receipt"
    )

def __hash__(self) -> int: # type: ignore[override]
    return hash(self.hash)

@classmethod
def create(
    cls,
    *,
    summary: str,
    evidence: str,
    confidence: float = 1.0,
    timestamp: str | None = None,
    prev_hash: str = GENESIS_PREV_HASH,
) -> "Receipt":
    """Mint a new receipt, computing ``hash`` from the core fields."""
    ts = utc_now() if timestamp is None else timestamp
    conf = validate_confidence(confidence)
    rec_hash = digest(
        timestamp=validate_timestamp(ts),
        summary=validate_summary(summary),
        evidence=validate_evidence(evidence),
        confidence=conf,
        prev_hash=_require_str("prev_hash", prev_hash),
    )
    return cls(
        timestamp=ts,
        summary=summary,
        evidence=evidence,
        confidence=conf,
        prev_hash=prev_hash,
        hash=rec_hash,
    )

def recomputed_hash(self) -> str:
    """Independent verifier: SHA-256 from fields only (ignores stored hash)."""
    return digest(
        timestamp=self.timestamp,
        summary=self.summary,
        evidence=self.evidence,
        confidence=self.confidence,
        prev_hash=self.prev_hash,
    )

def hash_ok(self) -> bool:
    return self.recomputed_hash() == self.hash

def to_dict(self) -> dict[str, Any]:
    return {
        "timestamp": self.timestamp,
        "summary": self.summary,
        "evidence": self.evidence,
        "confidence": self.confidence,
        "prev_hash": self.prev_hash,
        "hash": self.hash,
    }

@classmethod
def from_dict(cls, data: Mapping[str, Any]) -> "Receipt":
    """Load core fields. Extra keys are ignored (not hashed in v0.1.0)."""
    missing = [k for k in ("timestamp", "summary", "evidence", "confidence", "prev_hash", "hash") if k not in data]
    if missing:
        raise ReceiptError(f"receipt missing fields: {missing}")
    return cls(
        timestamp=data["timestamp"],
        summary=data["summary"],
        evidence=data["evidence"],
        confidence=data["confidence"],
        prev_hash=data["prev_hash"],
        hash=data["hash"],
    )
]

```

temporallock/temporallock/ui.py

```

"""Localhost UI for TemporalLock. Binds 127.0.0.1. Chain lives in a process tmp dir."""

from __future__ import annotations

import json
import shutil
import sys
import tempfile
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from pathlib import Path
from typing import Any
from urllib.parse import urlparse

from temporallock import __version__
from temporallock.chain import Chain
from temporallock.errors import TemporalLockError

DEFAULT_HOST = "127.0.0.1"
DEFAULT_PORT = 8766
LOOPBACK = frozenset({"127.0.0.1", "localhost", "::1"})
MAX_BODY = 1 * 1024 * 1024

PAGE = r"""<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>TemporalLock</title>
<style>
:root {
  --bg: #0f1419; --panel: #171e27; --ink: #e8edf2; --muted: #8b97a6;
  --line: #2a3544; --gold: #d4bc6a; --focus: #7aa2d4; --bad: #d4534b;
  --pass: #3dba7a;
}
* { box-sizing: border-box; }
html, body {
  margin: 0; padding: 0; background: var(--bg); color: var(--ink);
  font-family: system-ui, "Segoe UI", sans-serif; line-height: 1.45;
}
body { max-width: 46rem; margin: 0 auto; padding: 2.1rem 1.2rem 4rem; }
.tag {
  font-family: ui-monospace, Menlo, Consolas, monospace; font-size: 0.72rem;
  letter-spacing: 0.14em; text-transform: uppercase; color: var(--muted);
}
h1 { font-size: 2rem; font-weight: 650; letter-spacing: 0.04em; margin: 0.35rem 0 0.25rem; }
.motto { color: var(--gold); font-style: italic; margin: 0 0 0.85rem; font-size: 1.05rem; }
.lede { color: var(--muted); margin: 0 0 1.5rem; max-width: 40rem; }
fieldset {
  border: 1px solid var(--line); border-radius: 10px; background: var(--panel);
  padding: 1.1rem 1.15rem 1.2rem; margin: 0 0 1rem;
}
legend {
  font-family: ui-monospace, Menlo, Consolas, monospace; font-size: 0.72rem;
  letter-spacing: 0.12em; text-transform: uppercase; color: var(--muted); padding: 0 0.4rem;
}
label { display: block; font-size: 0.92rem; margin: 0.85rem 0 0.3rem; }
label.kicker {
  display: block; font-family: ui-monospace, Menlo, Consolas, monospace;
  font-size: 0.68rem; letter-spacing: 0.12em; text-transform: uppercase;
  color: var(--muted); margin-bottom: 0.12rem;
}
textarea, input[type="text"], input[type="number"] {
  width: 100%; padding: 0.55rem 0.65rem; border: 1px solid var(--line);
  border-radius: 6px; background: #10161d; color: var(--ink); font: inherit;
}
textarea:focus, input:focus { outline: 2px solid var(--focus); outline-offset: 1px; }
.actions { display: flex; gap: 0.65rem; flex-wrap: wrap; margin: 0 0 1.6rem; }
button {
  font-family: ui-monospace, Menlo, Consolas, monospace; font-size: 0.85rem;
  letter-spacing: 0.04em; padding: 0.65rem 1rem; border-radius: 8px;
  border: 1px solid var(--ink); background: var(--ink); color: var(--bg);
  cursor: pointer; font-weight: 650;
}
button.ghost { background: transparent; color: var(--ink); }
h2 {
  font-size: 1.05rem; letter-spacing: 0.08em; text-transform: uppercase;
  color: var(--muted); font-weight: 600; margin: 0 0 0.7rem;
}
.banner {
  margin: 0 0 1rem; padding: 0.9rem 1rem; border-radius: 10px;
  border: 1px solid var(--line); background: var(--panel); color: var(--muted);
}
.banner.ok { color: var(--pass); border-color: var(--pass); }
.banner.bad { color: var(--bad); border-color: var(--bad); }
ol { list-style: none; margin: 0; padding: 0; display: flex; flex-direction: column; gap: 0.65rem; }
.receipt {
  border: 1px solid var(--line); border-radius: 10px; background: var(--panel);
  padding: 0.85rem 1rem;
}
.hash { font-family: ui-monospace, Menlo, Consolas, monospace; font-size: 0.75rem; word-break: break-all; color: var(--muted); }
.err { color: var(--bad); }

```

```

    footer { margin-top: 2rem; color: var(--muted); font-size: 0.88rem; }
    .foot-note { font-style: italic; }
  </style>
</head>
<body>
  <header>
    <div class="tag">TemporalLock ? __VERSION__ ? loopback ? receipts</div>
    <h1>TemporalLock</h1>
    <p class="motto">Receipts, not truth claims.</p>
    <p class="lede">
      Genesis, append, and verify a local observation chain in a temporary directory
      owned by this process. A receipt records that something was observed. It is not
      a verdict, not a score of truth, and not an official history. Bound to 127.0.0.1 only.
    </p>
  </header>

  <form id="receipt-form" autocomplete="off">
    <fieldset>
      <legend>New receipt</legend>
      <label for="summary"><span class="kicker">Summary</span> Brief note of what was observed.</label>
      <input id="summary" type="text" placeholder="sky was overcast">
      <label for="evidence"><span class="kicker">Evidence</span> Supporting body and/or URI. Required.</label>
      <textarea id="evidence" rows="3" placeholder="photo:./sky.jpg"></textarea>
      <label for="confidence"><span class="kicker">Confidence</span> Observer-assigned float in [0, 1].</label>
      <input id="confidence" type="number" min="0" max="1" step="0.01" value="0.7">
    </fieldset>
    <div class="actions">
      <button type="button" id="genesis">Genesis</button>
      <button type="button" id="append">Append</button>
      <button type="button" class="ghost" id="verify">Verify</button>
      <label class="ghost">Import JSON <input type="file" id="import-json" accept="application/json,.json,.jsonl"></la
    </div>
  </form>

  <div id="banner" class="banner">No receipts yet. Genesis writes the first receipt.</div>
  <h2>Receipts</h2>
  <ol id="list"></ol>
  <p class="err" id="err" hidden></p>

  <footer>
    <p>Apache-2.0 ? Aziel Eliab ? July 2026 ? Bound to 127.0.0.1 ? <code>temporallock ui</code></p>
    <p class="foot-note">Receipts, not truth claims. Forks welcome and always allowed.</p>
  </footer>
</script>
(function () {
  const $ = (id) => document.getElementById(id);
  let last = null;
  function fail(msg) { $("err").hidden = false; $("err").textContent = msg; }
  function fields() {
    return {
      summary: $("summary").value,
      evidence: $("evidence").value,
      confidence: Number($("confidence").value),
    };
  };
  function draw(data) {
    last = data;
    $("err").hidden = true;
    const banner = $("banner");
    const v = data.verify || {};
    banner.className = "banner " + (v.ok ? "ok" : (data.receipts && data.receipts.length ? "bad" : ""));
    banner.textContent = data.message || (v.ok
      ? ("Chain intact ? " + v.length + " receipts ? last " + (v.last_hash || "").slice(0, 12))
      : ("Verify errors: " + ((v.errors || []).join("; ") || "none")));
    const ol = $("list");
    ol.innerHTML = "";
    (data.receipts || []).forEach((rec, i) => {
      const li = document.createElement("li");
      li.className = "receipt";
      li.innerHTML = "<strong>" + (i) + " ? " + (rec.timestamp || "") + "</strong>"
        + "<p>" + (rec.summary || "") + "</p>"
        + "<p>" + (rec.evidence || "") + " ? conf " + rec.confidence + "</p>"
        + "<p class='hash'>hash " + rec.hash + "</p>"
        + "<p class='hash'>prev " + rec.prev_hash + "</p>";
      ol.appendChild(li);
    });
  };
  async function post(url, body) {
    const res = await fetch(url, {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify(body || {}),
    });
    const data = await res.json();
    if (!res.ok) throw new Error(data.error || ("HTTP " + res.status));
    return data;
  }
  async function refresh() {
    const res = await fetch("/api/chain");
    draw(await res.json());
  }
  $("genesis").onclick = async () => {
    try { draw(await post("/api/genesis", fields())); } catch (e) { fail(String(e.message || e)); }
  }
}

```

```

    };
    $("append").onclick = async () => {
      try { draw(await post("/api/append", fields())); } catch (e) { fail(String(e.message || e)); }
    };
    $("verify").onclick = async () => {
      try { draw(await post("/api/verify", {})); } catch (e) { fail(String(e.message || e)); }
    };
    $("import-json").onchange = async () => {
      const f = $("import-json").files && $("import-json").files[0];
      if (!f) return;
      const text = await f.text();
      let receipts;
      try {
        const parsed = JSON.parse(text);
        receipts = Array.isArray(parsed) ? parsed : (parsed.receipts || []);
      } catch (e) {
        receipts = text.split(/\n/).filter(Boolean).map((line) => JSON.parse(line));
      }
      try { draw(await post("/api/import", { receipts: receipts })); } catch (err) { fail(String(err.message || err)); }
    };
    $("export").onclick = () => {
      if (!last) return;
      const blob = new Blob([JSON.stringify(last.receipts || [], null, 2)], {type: "application/json"});
      const a = document.createElement("a");
      a.href = URL.createObjectURL(blob);
      a.download = "temporallock-receipts.json";
      a.click();
      URL.revokeObjectURL(a.href);
    };
    refresh().cat
  ch(() => {});
})();
</script>
</body>
</html>
""".replace("__VERSION__", __version__)

```

```

class TemporalServer(ThreadingHTTPServer):
    chain_dir: str
    chain_path: Path

    def server_close(self) -> None:
        super().server_close()
        chain_dir = getattr(self, "chain_dir", None)
        if chain_dir:
            shutil.rmtree(chain_dir, ignore_errors=True)

    def _payload(chain: Chain, message: str) -> dict[str, Any]:
        verify = chain.verify()
        return {
            "message": message,
            "receipts": [rec.to_dict() for rec in chain],
            "verify": {
                "ok": verify.ok,
                "length": verify.length,
                "first_hash": verify.first_hash,
                "last_hash": verify.last_hash,
                "errors": list(verify.errors),
            },
            "note": "Receipts, not truth claims.",
        }

    def _empty(message: str) -> dict[str, Any]:
        return {
            "message": message,
            "receipts": [],
            "verify": {"ok": True, "length": 0, "first_hash": None, "last_hash": None, "errors": []},
            "note": "Receipts, not truth claims.",
        }

class Handler(BaseHTTPRequestHandler):
    def log_message(self, fmt: str, *args: object) -> None:
        sys.stderr.write("%s - %s\n" % (self.address_string(), fmt % args))

    def _chain_path(self) -> Path:
        return self.server.chain_path # type: ignore[attr-defined]

    def _load(self) -> Chain | None:
        path = self._chain_path()
        if not path.is_file() or path.stat().st_size == 0:
            return None
        return Chain.load(path)

    def _send(self, code: int, body: bytes, content_type: str) -> None:
        self.send_response(code)
        self.send_header("Content-Type", content_type)
        self.send_header("Content-Length", str(len(body)))
        self.send_header("Cache-Control", "no-store")
        self.end_headers()
        self.wfile.write(body)

    def _json(self, code: int, obj: Any) -> None:

```

```

        self._send(code, json.dumps(obj).encode("utf-8"), "application/json; charset=utf-8")

def _read_json(self) -> dict[str, Any]:
    length = int(self.headers.get("Content-Length") or 0)
    if length > MAX_BODY:
        raise ValueError("payload too large")
    raw = self.rfile.read(length) if length else b"{}"
    data = json.loads(raw.decode("utf-8") or "{}")
    if not isinstance(data, dict):
        raise ValueError("expected a JSON object")
    return data

def do_GET(self) -> None: # noqa: N802
    path = urlparse(self.path).path
    if path in ("/", "/index.html"):
        self._send(200, PAGE.encode("utf-8"), "text/html; charset=utf-8")
        return
    if path == "/health":
        self._json(200, {"ok": True, "bind_host": DEFAULT_HOST, "name": "TemporalLock"})
        return
    if path == "/api/chain":
        chain = self._load()
        if chain is None:
            self._json(200, _empty("No receipts yet. Genesis writes the first receipt."))
            return
        self._json(200, _payload(chain, "Current receipts on the local chain. "))
        return
    self._json(404, {"error": "not found"})

def do_POST(self) -> None: # noqa: N802
    path = urlparse(self.path).path
    try:
        if path == "/api/verify":
            chain = self._load()
            if chain is None:
                self._json(200, _empty("No receipts to verify. "))
                return
            result = chain.verify()
            msg = "Chain intact." if result.ok else "Broken links or hashes in these receipts."
            self._json(200, _payload(chain, msg))
            return
        body = self._read_json()
        summary = str(body.get("summary") or "")
        evidence = str(body.get("evidence") or "")
        confidence = float(body.get("confidence") if body.get("confidence") is not None else 0.7)
        dest = self._chain_path()
        if path == "/api/genesis":
            chain = Chain.genesis(dest, summary=summary, evidence=evidence, confidence=confidence)
            self._json(200, _payload(chain, "Genesis receipt written. Receipts, not truth claims. "))
            return
        if path == "/api/import":
            receipts = body.get("receipts")
            if not isinstance(receipts, list) or not receipts:
                self._json(400, {"error": "receipts array required"})
                return
            dest = self._chain_path()
            dest.write_text(
                "\n".join(json.dumps(r, sort_keys=True, separators=(",", ":"), ensure_ascii=False) for r in receipts
                if isinstance(r, dict)) + "\n",
                encoding="utf-8",
            )
            chain = Chain.load(dest)
            self._json(200, _payload(chain, "Imported receipts. Receipts, not truth claims. "))
            return
        if path == "/api/append":
            if not dest.is_file() or dest.stat().st_size == 0:
                self._json(400, {"error": "chain does not exist; use genesis for the first receipt"})
                return
            chain = Chain.load(dest)
            chain.append(
                summary=summary,
                evidence=evidence,
                confidence=confidence,
                require_existing=True,
            )
            self._json(200, _payload(chain, "Receipt appended. Receipts, not truth claims. "))
            return
    except TemporalLockError as exc:
        self._json(400, {"error": str(exc)})
        return
    except Exception as exc: # noqa: BLE001
        self._json(400, {"error": str(exc)})
        return
    self._json(404, {"error": "not found"})

def make_server(host: str = DEFAULT_HOST, port: int = DEFAULT_PORT) -> TemporalServer:
    if host not in LOOPBACK:
        raise ValueError("TemporalLock UI binds loopback only (127.0.0.1)")
    tmp = tempfile.mkdtemp(prefix="temporallock-ui-")
    httpd = TemporalServer((host, port), Handler)
    httpd.chain_dir = tmp
    httpd.chain_path = Path(tmp) / "chain.jsonl"
    return httpd

def serve(host: str = DEFAULT_HOST, port: int = DEFAULT_PORT) -> None:

```

```
httpd = make_server(host, port)
sys.stdout.write(f"TemporalLock UI  http://{host}:{port}/\n")
sys.stdout.write("Local only. Receipts, not truth claims. Chain lives in a process tmp dir.\n")
sys.stdout.flush()
try:
    httpd.serve_forever()
except KeyboardInterrupt:
    sys.stdout.write("\nstopped\n")
finally:
    httpd.server_close()
```

temporallock/workers/download-tracker/package.json

```
{  
  "dependencies": {  
    "wrangler": "^4.127.1"  
  }  
}
```


temporallock/workers/download-tracker/src/index.js

```

import { handleRuntimeApi } from "../runtime.js";

/**
 * TemporalLock download tracker (Cloudflare Worker).
 *
 * GET /download?repo=AzielEliab/temporallock&tag=latest&asset=...
 *      increments KV, serves gzip via ASSETS.fetch (no 302)
 *      (default https://github.com/AzielEliab/temporallock/releases)
 * GET /stats JSON totals + per-repo + per-branch breakdown
 * POST /event forks report a download {owner,repo,branch,fork,asset}
 *
 * KV binding DOWNLOADS. Keys: project|owner|repo|branch|fork
 * CORS *. No secrets in this tree.
 */

const PROJECT = "temporallock";
const DEFAULT_ASSET = "temporallock-0.1.0.tar.gz";
const DEFAULT_OWNER = "AzielEliab";
const DEFAULT_REPO = "temporallock";
const DEFAULT_BRANCH = "main";
const GITHUB_RELEASES = "https://github.com/AzielEliab/temporallock/releases";
const GITHUB_LATEST = "https://github.com/AzielEliab/temporallock/releases/latest";
const HOST = "https://temporallock-download-tracker.vibelock.workers.dev";
const GITHUB_REPO = "https://github.com/AzielEliab/temporallock";
const INSTALL_LINE = "curl -fsSL https://temporallock-download-tracker.vibelock.workers.dev/install.sh | bash";
const DOI = "https://doi.org/10.5281/zenodo.21431405";
const ZENODO = "https://zenodo.org/records/21431405";

function corsHeaders() {
  return {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type",
  };
}

function json(body, status = 200) {
  return new Response(JSON.stringify(body, null, 2), {
    status,
    headers: { "Content-Type": "application/json; charset=utf-8", ...corsHeaders() },
  });
}

function redirect(url) {
  return new Response(null, {
    status: 302,
    headers: { Location: url, ...corsHeaders() },
  });
}

function splitOwnerRepo(value, fallbackOwner, fallbackRepo) {
  if (typeof value === "string" && value.includes("/")) {
    const [o, r] = value.split("/").filter(Boolean);
    if (o && r) return { owner: o, repo: r };
  }
  return { owner: fallbackOwner, repo: fallbackRepo };
}

function parseDims(src) {
  const get = (k) => {
    if (src == null) return null;
    if (typeof src.get === "function") {
      const v = src.get(k);
      return v == null || v === "" ? null : v;
    }
    const v = src[k];
    return v == null || v === "" ? null : v;
  };

  let owner = get("owner") || DEFAULT_OWNER;
  let repo = get("repo") || DEFAULT_REPO;
  if (typeof repo === "string" && repo.includes("/")) {
    const split = splitOwnerRepo(repo, owner, DEFAULT_REPO);
    owner = split.owner;
    repo = split.repo;
  }

  const branch = get("branch") || DEFAULT_BRANCH;
  const tag = get("tag") || "latest";
  const asset = get("asset") || "";

  const forkRaw = get("fork");
  let fork = "0";
  if (forkRaw === 1 || forkRaw === true || forkRaw === "1" || forkRaw === "true") {
    fork = "1";
  } else if (typeof forkRaw === "string" && forkRaw.includes("/")) {
    const split = splitOwnerRepo(forkRaw, owner, repo);
    owner = split.owner;
    repo = split.repo;
    fork = "1";
  } else if (forkRaw != null && forkRaw !== 0 && forkRaw !== false && forkRaw !== "0" && forkRaw !== "false") {

```

```

    fork = "1";
  }

  if (`${owner}/${repo}`.toLowerCase() !== `${DEFAULT_OWNER}/${DEFAULT_REPO}`.toLowerCase()) {
    fork = "1";
  }

  return { project: PROJECT, owner, repo, branch, fork, tag, asset };
}

function kvKey(dims) {
  return `${dims.project}|${dims.owner}|${dims.repo}|${dims.branch}|${dims.fork}`;
}

function githubAssetUrl(owner, repo, tag, asset) {
  if (!asset) {
    if (owner === DEFAULT_OWNER && repo === DEFAULT_REPO) return GITHUB_RELEASES;
    return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases`;
  }
  if (!tag || tag === "latest") {
    return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases/latest/download/${encodeURIComponent(asset)}`;
  }
  return `https://github.com/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/releases/download/${encodeURIComponent(tag)}/${encodeURIComponent(asset)}`;
}

async function increment(env, dims) {
  const key = kvKey(dims);
  const n = parseInt((await env.DOWNLOADS.get(key)) || "0", 10) + 1;
  await env.DOWNLOADS.put(key, String(n));
  return n;
}

async function listAllKeys(env) {
  const keys = [];
  let cursor;
  do {
    const page = await env.DOWNLOADS.list(cursor ? { cursor } : {});
    keys.push(...page.keys);
    cursor = page.list_complete ? undefined : page.cursor;
  } while (cursor);
  return keys;
}

async function collectStats(env) {
  const keys = await listAllKeys(env);
  let total = 0;
  const by_repo = {};
  const by_branch = {};
  const by_fork = { "0": 0, "1": 0 };
  const breakdown = [];

  for (const k of keys) {
    const name = k.name;
    if (name === viewsKey() || name === totalKey() || name === githubCacheKey()) continue;
    const n = parseInt((await env.DOWNLOADS.get(name)) || "0", 10);
    if (!Number.isFinite(n) || n <= 0) continue;
    const parts = name.split("|");
    if (parts.length < 5) continue;
    const [project, owner, repo, branch, fork] = parts;
    total += n;
    const repoId = `${owner}/${repo}`;
    by_repo[repoId] = (by_repo[repoId] || 0) + n;
    by_branch[branch] = (by_branch[branch] || 0) + n;
    const forkFlag = fork === "1" ? "1" : "0";
    by_fork[forkFlag] = (by_fork[forkFlag] || 0) + n;
    breakdown.push({ project, owner, repo, branch, fork: forkFlag, count: n });
  }

  const totalDirect = parseInt((await env.DOWNLOADS.get(totalKey())) || "0", 10);
  const views = parseInt((await env.DOWNLOADS.get(viewsKey())) || "0", 10) || 0;
  const github = await githubStats(env);
  const shown = Number.isFinite(totalDirect) && totalDirect > 0 ? totalDirect : total;
  return {
    project: PROJECT,
    total: shown,
    views,
    downloads: shown,
    by_repo,
    by_branch,
    by_fork,
    breakdown,
    github: {
      stars: github.stars || 0,
      forks: github.forks || 0,
      watchers: github.watchers || 0,
      release_download_count: github.release_download_count || 0,
    },
    note: "Forks identified by GitHub owner/repo. Key layout: project|owner|repo|branch|fork. Views are separate from downloads. /v1 does not increment.",
  };
}

function viewsKey() {
  return PROJECT + "|__views__";
}

```

```

}

function githubCacheKey() {
  return PROJECT + "|__github__";
}

async function incrementViews(env) {
  const n = parseInt((await env.DOWNLOADS.get(viewsKey())) || "0", 10) + 1;
  await env.DOWNLOADS.put(viewsKey(), String(n));
  return n;
}

async function githubStats(env) {
  const cached = await env.DOWNLOADS.get(githubCacheKey());
  if (cached) {
    try {
      const obj = JSON.parse(cached);
      if (obj && obj.fetched_at && Date.now() - obj.fetched_at < 5 * 60 * 1000) {
        return obj;
      }
    } catch {
      /* ignore */
    }
  }
  const headers = { "User-Agent": "Mozilla/5.0 TemporalLock-download-tracker", Accept: "application/vnd.github+json" };
  let stars = 0;
  let forks = 0;
  let watchers = 0;
  let release_download_count = 0;
  try {
    const repoRes = await fetch("https://api.github.com/repos/AzielEliab/temporallock", { headers });
    if (repoRes.ok) {
      const repo = await repoRes.json();
      stars = Number(repo.stargazers_count) || 0;
      forks = Number(repo.forks_count) || 0;
      watchers = Number(repo.subscribers_count != null ? repo.subscribers_count : repo.watchers_count) || 0;
    }
    const relRes = await fetch("https://api.github.com/repos/AzielEliab/temporallock/releases/latest", { headers });
    if (relRes.ok) {
      const rel = await relRes.json();
      const assets = Array.isArray(rel.assets) ? rel.assets : [];
      release_download_count = assets.reduce((s, a) => s + (Number(a.download_count) || 0), 0);
    }
  } catch {
    /* public API; empty is fine */
  }
  const out = { stars, forks, watchers, release_download_count, fetched_at: Date.now() };
  try {
    await env.DOWNLOADS.put(githubCacheKey(), JSON.stringify(out));
  } catch {
    /* ignore */
  }
  return out;
}

function installScript() {
  return `#!/usr/bin/env bash
# TemporalLock one-click install. Counted download via this Worker.
set -euo pipefail
HOST="${HOST}"
ASSET="${DEFAULT_ASSET}"
WORKDIR="${TEMPORALLOCK_HOME:-$HOME/temporallock}"
mkdir -p "$WORKDIR"
cd "$WORKDIR"
echo "Downloading counted tarball from ${HOST}/download (User-Agent Mozilla/5.0)?"
curl -fsSL -A 'Mozilla/5.0' "${HOST}/download?asset=${ASSET}" -o "${ASSET}"
tar -xzf "${ASSET}"
DIR="$(find . -maxdepth 1 -type d -name 'temporallock-*' | head -n 1)"
if [ -n "${DIR}" ]; then
  cd "${DIR}"
fi
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -U pip
python -m pip install -e .
echo
echo "Installed TemporalLock."
echo "Run: temporallock ui"
echo "Then open http://127.0.0.1:8766 (loopback only)"
echo "Author: Aziel Eliab."
`;
}

async function serveAsset(request, env, asset, { head = false } = {}) {
  if (!env.ASSETS) {
    return json({ error: "assets binding missing" }, 500);
  }
  const assetUrl = new URL("/") + asset, request.url;
  const assetRes = await env.ASSETS.fetch(new Request(assetUrl, { method: "GET" }));
  if (!assetRes.ok) {
    return json({ error: "asset not hosted", asset, status: assetRes.status }, 404);
  }
}

```

```

const headers = new Headers();
headers.set("Content-Type", "application/gzip");
headers.set("Content-Disposition", "attachment; filename=" + asset.replaceAll("'", "") + ".gz");
headers.set("Cache-Control", "private, no-store");
const len = assetRes.headers.get("Content-Length");
if (len) headers.set("Content-Length", len);
for (const [k, v] of Object.entries(corsHeaders())) headers.set(k, v);
if (head) {
  return new Response(null, { status: 200, headers });
}
return new Response(assetRes.body, { status: 200, headers });
}

async function indexHtml(env) {
  const stats = await collectStats(env);
  const views = Number(stats.views) || 0;
  const downloads = Number(stats.downloads != null ? stats.downloads : stats.total) || 0;
  const v = views.toLocaleString("en-US");
  const n = downloads.toLocaleString("en-US");
  const gh = stats.github || {};
  const breakdown = (stats.breakdown || []).map(
    (b) =>
      `<li><code>${b.owner}/${b.repo}</code> branch <code>${b.branch}</code> fork=${b.fork} ? ${b.count}</li>`,
  )
  .join("") || "<li>none yet</li>";
  return `<!doctype html>
<html lang="en">
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>TemporalLock downloads</title>
<style>
:root { color-scheme: dark; }
body { font: 16px/1.45 system-ui, sans-serif; max-width: 42rem; margin: 3rem auto; padding: 0 1.25rem 4rem; background-color: #0e1014; color: #e8eaeef; }
h1 { font-size: 1.75rem; margin: 0 0 .35rem; }
.motto { color: #9aa3b2; margin: 0 0 1.5rem; }
.card { border: 1px solid #2a3140; border-radius: 12px; padding: 1.25rem 1.35rem; background: #151922; }
.nums { display: grid; grid-template-columns: 1fr 1fr; gap: .8rem; margin: 0 0 1rem; }
.count { font-size: 2.2rem; font-variant-numeric: tabular-nums; font-weight: 700; margin: 0; }
.count span { display: block; font-size: .95rem; font-weight: 500; color: #9aa3b2; }
.btns { display: grid; grid-template-columns: 1fr 1fr; gap: .75rem; margin: 0 0 .85rem; }
@media (max-width: 520px) { .btns { grid-template-columns: 1fr; }
a.btn, button.btn { display: block; width: 100%; box-sizing: border-box; text-align: center; font: inherit; font-size: 1.2rem; font-weight: 750; padding: 1rem 1.1rem; border-radius: 10px; border: 0; cursor: pointer; text-decoration: none; }
a.btn.primary { background: #e8eaeef; color: #0e1014; }
button.btn.install { background: #c9a227; color: #14110a; }
button.btn.install.copied { background: #7dcf9a; color: #0e1014; }
.kid { font-size: 1.05rem; margin: 0 0 1rem; }
.meta { margin-top: 1.1rem; color: #9aa3b2; font-size: .92rem; }
.meta a { color: #c9d4ff; }
.iso { margin-top: .85rem; font-size: .85rem; color: #7d8696; }
.banner { border: 1px solid #5c4a1a; background: #241c0d; color: #f0d78c; padding: .85rem 1rem; border-radius: 8px; margin: 0 0 1.2rem; font-size: .92rem; }
pre { background: #0e1014; padding: .75rem .9rem; overflow: auto; border-radius: 8px; font-size: .82rem; }
code { font-size: .88rem; }
</style>
<body>
<h1>TemporalLock</h1>
<p class="motto">Receipts, not truth claims. Author Aziel Eliab.</p>
<p class="banner">THIS IS: append-only observation receipts with evidence and confidence. THIS IS NOT: a truth score, court, narrative engine, or authority. The Worker does not store chains. Author Aziel Eliab.</p>
<div class="card">
  <div class="nums">
    <p class="count">${v}<span>Views</span></p>
    <p class="count">${n}<span>Downloads</span></p>
  </div>
  <p class="kid"><strong>Two big buttons.</strong> Download saves the gzip (the Downloads number goes up). One-click install copies a Terminal command. After it finishes, type <code>temporallock ui</code>.</p>
  <div class="btns">
    <a class="btn primary dl" href="/download?asset=${DEFAULT_ASSET}">Download</a>
    <button type="button" class="btn install" id="install-btn">One-click install</button>
  </div>
  <pre id="install-cmd">${INSTALL_LINE}</pre>
  <p class="kid">Then run: <code>temporallock ui</code> and open http://127.0.0.1:8766 (this computer only).</p>
  <p class="meta">The download count ticks on the Download click. The Worker serves the gzip (HTTP 200). No 302 to GitHub. Forks using this same link are counted automatically. ${DEFAULT_ASSET} ? ${n} counted.</p>
  <p class="iso">Isolated counter: Worker <code>temporallock-download-tracker</code>, project <code>${PROJECT}</code> >, KV <code>TEMPORALLOCK_DOWNLOADS</code>. Not mixed with any other product. /v1 does not increment downloads.</p>
  <p class="meta">GitHub: stars ${gh.stars || 0} ? forks ${gh.forks || 0} ? watchers ${gh.watchers || 0} ? release assets ${gh.release_download_count || 0}</p>
  <p class="meta">Paper: <a href="${DOI}">doi:10.5281/zenodo.21431405</a> ? <a href="${ZENODO}">Zenodo</a> ? Apache-2.0 ? Eliab, Aziel</p>
  <p class="meta"><a href="/stats">JSON stats</a> ? <a href="/openapi.json">OpenAPI</a> ? <a href="/v1/skill">Skill</a> ? <a href="/ai">AI runtime</a> ? <a href="${GITHUB_REPO}">GitHub</a> ? <a href="${GITHUB_LATEST}">releases</a></p>
</body>
<script>
(function () {
  var cmd = "curl -fsSL https://temporallock-download-tracker.vibelock.workers.dev/install.sh | bash";
  var btn = document.getElementById("install-btn");
  var pre = document.getElementById("install-cmd");
  if (!btn) return;
  btn.addEventListener("click", function () {
    function done(ok) {
      btn.textContent = ok ? "Copied! Paste in Terminal, then run temporallock ui" : "Select the command, copy i

```

```

t, then run temporallock ui";
    btn.classList.add("copied");
  }
  if (navigator.clipboard && navigator.clipboard.writeText) {
    navigator.clipboard.writeText(cmd).then(function () { done(true); }).catch(function () { done(false); });
  } else {
    done(false);
    if (pre && window.getSelection) {
      var r = document.createRange();
      r.selectNodeContents(pre);
      var sel = window.getSelection();
      sel.removeAllRanges();
      sel.addRange(r);
    }
  }
});
})();
</script>
<h2>Per repo / branch / fork</h2>
<ul>${breakdown}</ul>
</div>
</body>
</html>`;
}

export default {
  async fetch(request, env) {
    const url = new URL(request.url);

    if (request.method === "OPTIONS") {
      return new Response(null, { status: 204, headers: corsHeaders() });
    }

    if ((url.pathname === "/install.sh" || url.pathname === "/install.sh/") && request.method === "GET") {
      return new Response(installScript(), {
        status: 200,
        headers: {
          "Content-Type": "text/x-shellscript; charset=utf-8",
          "Cache-Control": "private, no-store",
          ...corsHeaders(),
        },
      });
    }

    const runtime = await handleRuntimeApi(request, url);
    if (runtime) return runtime;

    if (url.pathname === "/" && request.method === "GET") {
      await incrementViews(env);
      return new Response(await indexHtml(env), {
        headers: { "Content-Type": "text/html; charset=utf-8", "Cache-Control": "private, no-store", ...corsHeaders() },
      });
    }

    if (url.pathname === "/count" && request.method === "GET") {
      const stats = await collectStats(env);
      return json({ project: PROJECT, total: stats.total || 0 });
    }

    if (url.pathname === "/stats" && request.method === "GET") {
      return json(await collectStats(env));
    }

    if (url.pathname === "/event" && request.method === "POST") {
      let body;
      try {
        body = await request.json();
      } catch {
        return json({ error: "JSON body required" }, 400);
      }
      const dims = parseDims(body || {});
      const count = await increment(env, dims);
      return json({
        ok: true,
        key: kvKey(dims),
        count,
        owner: dims.owner,
        repo: dims.repo,
        branch: dims.branch,
        fork: dims.fork,
        asset: dims.asset || null,
      });
    }

    if (url.pathname === "/go" && (request.method === "GET" || request.method === "HEAD")) {
      const dims = parseDims(url.searchParams);
      const asset = dims.asset || DEFAULT_ASSET;
      dims.asset = asset;
      if (request.method === "GET") await increment(env, dims);
      return serveAsset(request, env, asset, { head: request.method === "HEAD" });
    }

    if ((url.pathname === "/download" || url.pathname.startsWith("/download/")) && (request.method === "GET" || request.method === "HEAD")) {

```

```
t.method === "HEAD")) {
  const dims = parseDims(url.searchParams);
  if (!dims.asset && url.pathname.startsWith("/download/")) {
    dims.asset = decodeURIComponent(url.pathname.slice("/download/".length));
  }
  const asset = dims.asset || DEFAULT_ASSET;
  dims.asset = asset;
  if (request.method === "GET") await increment(env, dims);
  return serveAsset(request, env, asset, { head: request.method === "HEAD" });
}

return json({ error: "not found" }, 404);
},
};
```

temporallock/workers/download-tracker/src/runtime.js

```

/**
 * TemporalLock hosted runtime (port of canon/receipt/chain).
 * Stateless: client sends the chain JSON in the body. Receipts not truth claims.
 * /v1 never touches DOWNLOADS KV.
 */
const PRODUCT = "temporallock";
const VERSION = "0.1.0";
const MOTTO = "Receipts, not truth claims.";
const HOST = "https://temporallock-download-tracker.viblock.workers.dev";
const SKILL = "---\\name: TemporalLock\\ndescription: Use when minting or verifying append-only observation receipts. R
eceipts, not truth claims. Hosted API is stateless. Hosted /v1 via this Worker or aziel-runtime. Author Aziel Eliab.\\n
---\\n\\n# TemporalLock\\n\\nReceipts, not truth claims.\\n\\nAuthor: **Azriel Eliab**\\.\\n\\nUse when minting or verifying appe
nd-only observation receipts. Receipts, not truth claims. Hosted API is stateless.\\n\\nAlways send `User-Agent: Mozilla
/5.0`. Cloudflare Workers may 403 an empty agent.\\n\\n## Endpoints (this Worker)\\n\\nHost: `https://temporallock-downloa
d-tracker.viblock.workers.dev`\\n\\nMethod | Path | What |\\n|-----|-----|-----|\\n| GET | `/v1/health` | Liveness
. Does not increment downloads. |\\n| GET | `/v1/skill` | This markdown. Does not increment downloads. |\\n| POST | `/v1
/genesis` | Start a chain. Body includes summary + evidence. |\\n| POST | `/v1/append` | Append a receipt. Client sends
the chain. |\\n| POST | `/v1/verify` | Verify hashes and prev links. Not stored. |\\n| POST | `/v1/gate` | File-gate pr
eview (hash + receipt). |\\n\\nOpenAPI: `https://temporallock-download-tracker.viblock.workers.dev/openapi.json`\\n\\nCat
alog OpenAPI: `https://aziel-runtime.viblock.workers.dev/openapi.json`\\n\\nMCP: `POST https://aziel-runtime.viblock.w
orkers.dev/mcp`\\n\\nCatalog aliases under `/p/temporallock/\\u2026`.\\n\\n## How to call (Mozilla/5.0)\\n\\n`bash\\ncurl -s
-A 'Mozilla/5.0' https://temporallock-download-tracker.viblock.workers.dev/v1/health\\ncurl -s -A 'Mozilla/5.0' -X PO
ST https://temporallock-download-tracker.viblock.workers.dev/v1/genesis \\n\\n -H 'content-type: application/json' \\n
\\n -d '{\"summary\": \"desk closed\", \"evidence\": \"log row\"}'\\ncurl -s -A 'Mozilla/5.0' https://temporallock-downloa
d-tracker.viblock.workers.dev/v1/skill\\n\\n`\\n\\nGrok: import the catalog OpenAPI as a custom tool. ChatGPT: GPT Actions
. Venice: HTTP tools.\\n\\n## Local (after one-click install)\\n\\n`bash\\ncurl -fsSL https://temporallock-download-track
er.viblock.workers.dev/install.sh | bash\\ntemporallock ui\\n\\n`\\n\\nThen open http://127.0.0.1:8766 (this computer only
).\\n\\n## Honest banner\\n\\nTHIS IS: append-only observation receipts with evidence and confidence. THIS IS NOT: a truth
score, court, narrative engine, or authority. The Worker does not store chains. Author Aziel Eliab.\\n\\nDOI: https://d
oi.org/10.5281/zenodo.21431405 \\nRecord: https://zenodo.org/records/21431405\\n\\nApache-2.0 (or the repo LICENSE). For
ks are welcome and always allowed.\\n";

const GENESIS_PREV_HASH = "0".repeat(64);
const CONFIDENCE_DECIMALS = 6;
const CONF_PLACEHOLDER = "__TL_CONFIDENCE__";

function corsHeaders() {
  return {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "GET, POST, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type",
  };
}

function json(body, status = 200) {
  return new Response(JSON.stringify(body, null, 2), {
    status,
    headers: { "Content-Type": "application/json; charset=utf-8", ...corsHeaders() },
  });
}

class ReceiptError extends Error {
  constructor(msg) { super(msg); this.name = "ReceiptError"; }
}

class ChainError extends Error {
  constructor(msg) { super(msg); this.name = "ChainError"; }
}

function formatConfidence(confidence) {
  return Number(confidence).toFixed(CONFIDENCE_DECIMALS);
}

function canonicalBytes(timestamp, summary, evidence, confidence, prev_hash) {
  const payload = {
    confidence: CONF_PLACEHOLDER,
    evidence,
    prev_hash,
    summary,
    timestamp,
  };
  const keys = Object.keys(payload).sort();
  let raw = "{" + keys.map((k) => JSON.stringify(k) + ":" + JSON.stringify(payload[k])).join(",") + "}";
  raw = raw.replace(`"${CONF_PLACEHOLDER}"`, formatConfidence(confidence));
  return new TextEncoder().encode(raw);
}

async function digest(timestamp, summary, evidence, confidence, prev_hash) {
  const bytes = canonicalBytes(timestamp, summary, evidence, confidence, prev_hash);
  const buf = await crypto.subtle.digest("SHA-256", bytes);
  return [...new Uint8Array(buf)].map((b) => b.toString(16).padStart(2, "0")).join("");
}

function utcNow() {
  return new Date().toISOString().replace(/\\.d{3}Z$/, "Z");
}

function requireStr(name, value) {
  if (typeof value !== "string") throw new ReceiptError(`${name} must be a string`);
  return value;
}

function validateEvidence(evidence) {

```

```

    evidence = requireStr("evidence", evidence);
    if (evidence.trim() === "") throw new ReceiptError("empty evidence is invalid");
    return evidence;
}

function validateConfidence(confidence) {
    const value = Number(confidence);
    if (!Number.isFinite(value) || value < 0 || value > 1) {
        throw new ReceiptError("confidence must be a float in [0.0, 1.0] inclusive");
    }
    return value;
}

function validateSummary(summary) {
    return requireStr("summary", summary);
}

function validateTimestamp(timestamp) {
    const ts = requireStr("timestamp", timestamp);
    if (ts.trim() === "") throw new ReceiptError("timestamp must be a non-empty UTC ISO-8601 string");
    return ts;
}

async function createReceipt({ summary, evidence, confidence = 1.0, timestamp = null, prev_hash = GENESIS_PREV_HASH })
{
    const ts = timestamp == null ? utcNow() : timestamp;
    const conf = validateConfidence(confidence);
    const recHash = await digest(
        validateTimestamp(ts),
        validateSummary(summary),
        validateEvidence(evidence),
        conf,
        requireStr("prev_hash", prev_hash),
    );
    return {
        timestamp: ts,
        summary,
        evidence,
        confidence: conf,
        prev_hash,
        hash: recHash,
    };
}

function receiptFromDict(data) {
    const missing = ["timestamp", "summary", "evidence", "confidence", "prev_hash", "hash"].filter((k) => !(k in data));
    if (missing.length) throw new ReceiptError(`receipt missing fields: ${missing}`);
    return {
        timestamp: validateTimestamp(data.timestamp),
        summary: validateSummary(data.summary),
        evidence: validateEvidence(data.evidence),
        confidence: validateConfidence(data.confidence),
        prev_hash: requireStr("prev_hash", data.prev_hash),
        hash: requireStr("hash", data.hash),
    };
}

async function recomputedHash(rec) {
    return digest(rec.timestamp, rec.summary, rec.evidence, rec.confidence, rec.prev_hash);
}

function parseChain(body) {
    if (body == null) return [];
    let raw = body;
    if (typeof body === "string") {
        const text = body.trim();
        if (!text) return [];
        if (text.startsWith("[") ) raw = JSON.parse(text);
    } else {
        const rows = [];
        for (const line of text.split("\n")) {
            const t = line.trim();
            if (!t) continue;
            rows.push(JSON.parse(t));
        }
        return rows.map(receiptFromDict);
    }
}

if (Array.isArray(raw)) return raw.map(receiptFromDict);
if (raw && typeof raw === "object") {
    if (Array.isArray(raw.chain)) return raw.chain.map(receiptFromDict);
    if (Array.isArray(raw.receipts)) return raw.receipts.map(receiptFromDict);
}
return [];
}

async function verify(receipts) {
    const errors = [];
    const n = receipts.length;
    const first = n ? receipts[0].hash : null;
    const last = n ? receipts[n - 1].hash : null;
    for (let i = 0; i < n; i++) {
        const rec = receipts[i];
        const expected = await recomputedHash(rec);
        if (rec.hash !== expected) {

```



```

    errors.push(`index ${i}: stored hash ${rec.hash} != recomputed ${expected}`);
  }
  if (i === 0) continue;
  const prev = receipts[i - 1];
  if (rec.prev_hash !== prev.hash) {
    errors.push(`index ${i}: prev_hash ${rec.prev_hash} != previous.hash ${prev.hash}`);
  }
}
return { ok: errors.length === 0, length: n, first_hash: first, last_hash: last, errors };
}

async function genesis(body) {
  const summary = body.summary;
  const evidence = body.evidence;
  if (summary == null) throw new ReceiptError("summary is required");
  if (evidence == null) throw new ReceiptError("evidence is required");
  const existing = parseChain(body.chain != null ? body.chain : body.receipts);
  if (existing.length) throw new ChainError("chain already exists; use append");
  const rec = await createReceipt({
    summary: String(summary),
    evidence: String(evidence),
    confidence: body.confidence == null ? 1.0 : body.confidence,
    timestamp: body.timestamp || null,
    prev_hash: GENESIS_PREV_HASH,
  });
  return { product: PRODUCT, version: VERSION, motto: MOTTO, action: "genesis", receipt: rec, chain: [rec] };
}

async function append(body) {
  const receipts = parseChain(body.chain != null ? body : body.chain);
  // parseChain on full body also looks at body.chain / body.receipts
  const chain = parseChain(body);
  if (!chain.length) throw new ChainError("chain does not exist or is empty; use genesis");
  const rec = await createReceipt({
    summary: String(body.summary),
    evidence: String(body.evidence),
    confidence: body.confidence == null ? 0.7 : body.confidence,
    timestamp: body.timestamp || null,
    prev_hash: chain[chain.length - 1].hash,
  });
  return { product: PRODUCT, version: VERSION, motto: MOTTO, action: "appended", receipt: rec, chain: [...chain, rec] };
}

async function sha256Text(text) {
  const buf = await crypto.subtle.digest("SHA-256", new TextEncoder().encode(text == null ? "" : String(text)));
  return [...new Uint8Array(buf)].map((b) => b.toString(16).padStart(2, "0")).join("");
}

async function gate(body) {
  const content = body.content != null ? String(body.content) : (body.file_text != null ? String(body.file_text) : "");
  ;
  const fileName = body.file || body.name || "payload";
  const digestHex = body.sha256 || body.file_sha256 || await sha256Text(content);
  const evidence = `sha256:${digestHex} path:${fileName}`;
  const summary = body.summary || `gate accept ${fileName}`;
  let chain = parseChain(body);
  let rec;
  let action;
  if (!chain.length) {
    rec = await createReceipt({
      summary,
      evidence,
      confidence: body.confidence == null ? 1.0 : body.confidence,
      timestamp: body.timestamp || null,
      prev_hash: GENESIS_PREV_HASH,
    });
    chain = [rec];
    action = "genesis";
  } else {
    rec = await createReceipt({
      summary,
      evidence,
      confidence: body.confidence == null ? 1.0 : body.confidence,
      timestamp: body.timestamp || null,
      prev_hash: chain[chain.length - 1].hash,
    });
    chain = [...chain, rec];
    action = "appended";
  }
  const result = await verify(chain);
  return {
    product: PRODUCT,
    version: VERSION,
    motto: MOTTO,
    ok: result.ok,
    accepted: Boolean(result.ok),
    action,
    file: fileName,
    file_sha256: digestHex,
    receipt: rec.hash,
    length: result.length,
    errors: result.errors,
    chain,
  };
}

```

```

    };
  }

function openapiSpec() {
  const chainSchema = {
    oneOf: [
      { type: "array", items: { type: "object" } },
      { type: "string" },
    ],
  };
  return {
    openapi: "3.1.0",
    info: {
      title: "TemporalLock runtime",
      version: VERSION,
      description: "Append-only observation receipts. Client sends the chain JSON (stateless). " + MOTTO,
    },
    servers: [{ url: HOST }],
    paths: {
      "/v1/skill": {
        get: {
          operationId: "temporallock_skill",
          summary: "Return skill markdown. Does not increment download KV.",
          responses: { "200": { description: "markdown" } },
        },
      },
      "/v1/health": {
        get: { operationId: "health", summary: "Liveness", responses: { "200": { description: "ok", content: { "application/json": { schema: { type: "object" } } } } } },
      },
      "/v1/genesis": {
        post: {
          operationId: "genesis",
          summary: "Mint the first receipt (prev_hash = 64 zeros).",
          requestBody: { required: true, content: { "application/json": { schema: { type: "object", required: ["summary", "evidence"], properties: { summary: { type: "string" }, evidence: { type: "string" }, confidence: { type: "number" }, timestamp: { type: "string" } } } } } },
          responses: { "200": { description: "genesis", content: { "application/json": { schema: { type: "object" } } } } },
        },
      },
      "/v1/append": {
        post: {
          operationId: "append",
          summary: "Append a receipt. Client sends the existing chain JSON.",
          requestBody: { required: true, content: { "application/json": { schema: { type: "object", required: ["summary", "evidence"], properties: { chain: chainSchema, receipts: chainSchema, summary: { type: "string" }, evidence: { type: "string" }, confidence: { type: "number" }, timestamp: { type: "string" } } } } } },
          responses: { "200": { description: "append", content: { "application/json": { schema: { type: "object" } } } } },
        },
      },
      "/v1/verify": {
        post: {
          operationId: "verify",
          summary: "Walk hashes and links. Receipts, not truth claims.",
          requestBody: { required: true, content: { "application/json": { schema: { type: "object", properties: { chain: chainSchema, receipts: chainSchema } } } } },
          responses: { "200": { description: "verify", content: { "application/json": { schema: { type: "object" } } } } },
        },
      },
      "/v1/gate": {
        post: {
          operationId: "gate",
          summary: "Hash content, genesis or append, then verify. Stateless.",
          requestBody: { required: true, content: { "application/json": { schema: { type: "object", properties: { content: { type: "string" }, sha256: { type: "string" }, chain: chainSchema, summary: { type: "string" }, file: { type: "string" } } } } } },
          responses: { "200": { description: "gate", content: { "application/json": { schema: { type: "object" } } } } },
        },
      },
    },
  };
}

function aiHtml() {
  return `<!doctype html>
<html lang="en">
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>TemporalLock ? use with Grok, ChatGPT, Venice</title>
<style>
:root { color-scheme: dark; }
body { font: 16px/1.45 system-ui, sans-serif; max-width: 42rem; margin: 3rem auto; padding: 0 1.25rem; background: #0e1014; color: #e8eaeaf; }
code { background: #151922; padding: .15rem .4rem; border-radius: 4px; }
a { color: #c9d4ff; }
.motto { color: #9aa3b2; font-style: italic; }
</style>
<body>
<h1>TemporalLock live API</h1>
<p class="motto">${MOTTO}</p>

```

```

    <p>Stateless: send the chain JSON in the body. Hash+link as in the Python core. The Worker does not store chains.</p>
  >
  <h2>ChatGPT (GPT Actions)</h2>
  <p>Paste this OpenAPI URL into GPT Actions:</p>
  <p><code>${HOST}/openapi.json</code></p>
  <h2>Grok / xAI</h2>
  <p>Custom tool pointing at <code>POST ${HOST}/v1/genesis</code>, <code>/v1/append</code>, <code>/v1/verify</code>, <
  code>/v1/gate</code>.</p>
  <h2>Venice</h2>
  <p>Custom HTTP tool from the same OpenAPI URL.</p>
  <h2>MCP catalog</h2>
  <p>The shared catalog (ships separately) is <code>https://aziel-runtime.vibelock.workers.dev/mcp</code>.</p>
  <p><a href="/openapi.json">openapi.json</a> ? <a href="/v1/health">health</a> ? <a href="/">downloads</a></p>
</body>
</html>`;
}

export async function handleRuntimeApi(request, url) {
  const path = url.pathname;
  const isApi = path === "/v1" || path.startsWith("/v1/") || path === "/openapi.json" || path === "/ai";
  if (!isApi) return null;
  try {
    if (path === "/v1/health" && request.method === "GET") {
      return json({ ok: true, product: PRODUCT, version: VERSION });
    }
    if (path === "/v1/skill" && request.method === "GET") {
      return new Response(SKILL, {
        status: 200,
        headers: { "Content-Type": "text/markdown; charset=utf-8", "Cache-Control": "private, no-store", ...corsHeaders(
) },
      });
    }
    if (path === "/openapi.json" && request.method === "GET") return json(openapiSpec());
    if (path === "/ai" && request.method === "GET") {
      return new Response(aiHtml(), { headers: { "Content-Type": "text/html; charset=utf-8", ...corsHeaders() } });
    }
    if (path === "/v1/genesis" && request.method === "POST") {
      let body;
      try { body = await request.json(); } catch { return json({ error: "JSON body required" }, 400); }
      return json(await genesis(body));
    }
    if (path === "/v1/append" && request.method === "POST") {
      let body;
      try { body = await request.json(); } catch { return json({ error: "JSON body required" }, 400); }
      return json(await append(body));
    }
    if (path === "/v1/verify" && request.method === "POST") {
      let body;
      try { body = await request.json(); } catch { return json({ error: "JSON body required" }, 400); }
      const chain = parseChain(body);
      const result = await verify(chain);
      return json({ product: PRODUCT, version: VERSION, motto: MOTTO, ...result });
    }
    if (path === "/v1/gate" && request.method === "POST") {
      let body;
      try { body = await request.json(); } catch { return json({ error: "JSON body required" }, 400); }
      return json(await gate(body));
    }
    return json({ error: "not found" }, 404);
  } catch (err) {
    const status = err instanceof ReceiptError || err instanceof ChainError ? 400 : 400;
    return json({ error: String(err.message || err), motto: MOTTO, status });
  }
}

```

temporallock/workers/download-tracker/wrangler.toml

```
name = "temporallock-download-tracker"
main = "src/index.js"
compatibility_date = "2026-07-01"
workers_dev = true
account_id = "ac575a9b822bea2bed97d0ab73aed238"

[[kv_namespaces]]
binding = "DOWNLOADS"
id = "b78f137f438d4c9796b3da65bc7b417d"

[assets]
directory = "./public"
binding = "ASSETS"
run_worker_first = ["/", "/download", "/download/*", "/stats", "/event", "/go", "/count", "/install.sh", "/v1", "/v1/*", "/openapi.json", "/ai", "/ai/*", "/mcp", "/mcp/"]
```